Nonlinear Time Series Analysis

Experimental Nonlinear Dynamics from Measured Records

David Chelidze[1]
Department of Mechanical, Industrial and Systems Engineering
University of Rhode Island, Kingston, Rhode Island
Joseph P. Cusumano[2]
Department of Engineering Science and Mechanics
Pennsylvania State University, University Park, Pennsylvania

Draft Copy, October 11, 2026

[1]email: chelidze@uri.edu ⋄ phone: 401.874.2356 ⋄ https://web.uri.edu/nld
[2]email: jpc3@psu.edu ⋄ phone: 814.865.3179 ⋄ fax: 814.863.7967

Preface

This book is about what can be learned about a nonlinear dynamical system from a measured record alone. The starting point is a scalar or vector time series. The end points are a reconstructed state space, a set of invariants that characterize the dynamics on it, and a measured record cleaned, tested and reduced to the quantities that carry information. No step assumes a known equation of motion.

Scope

The methods here grew out of dynamical systems theory and are usually grouped as nonlinear time-series analysis: delay-coordinate embedding, the selection of delay and dimension, fractal dimensions and Lyapunov exponents, nonlinear noise reduction, prediction, surrogate testing, and recurrence. The book treats each as an estimation problem. Every quantity is computed from a finite, noisy record, and the questions that matter in practice are how the estimate is biased, how it scatters, how much data it needs, and what to report alongside the number. Those questions are asked of every method, and the answers are checked on the same few systems throughout.

Two things set the treatment apart. The statistics of correlated samples—effective sample size, the Theiler window, surrogate tests—run as one thread from the first estimator to the last. And the proper and smooth orthogonal decompositions supply a second thread, from noise reduction through the tracking of slowly drifting parameters.

A companion volume, *Data-Driven Modal Identification, Characterization, and Reduction*, takes the same measured records into the setting of mechanical structures: detection of nonlinearity, modal identification from response data, backbone curves, and reduced-order models. This book assumes none of that, and the companion volume assumes only the reconstruction and decomposition chapters here.

Organization

Part I develops the estimation and spectral background: probability and estimators, correlation, Fourier analysis, sampling, and the estimation of spectra, coherence and frequency response functions. Part II opens with the qualitative theory of bifurcations and Poincaré maps and then builds the reconstruction machinery: embedding, delay and dimension selection, fractal dimensions, and Lyapunov exponents. Part III covers multivariate decompositions, noise reduction, prediction, surrogate testing, drift tracking, and recurrence.

Algorithms appear in both MATLAB and Python in Appendix A.

Contents

Preface 3

Notation 11

I Foundations 15

1 Introduction 19
- 1.1 Dynamical Systems . . . 19
- 1.2 Some Examples of Dynamical Systems . . . 20
  - 1.2.1 Linear Mass-Spring System . . . 20
  - 1.2.2 Moon-Holmes Oscillator . . . 22
  - 1.2.3 Chua-Matsumoto Circuit . . . 22
  - 1.2.4 Shooting an Arrow at a Target . . . 24
  - 1.2.5 Fly Population Dynamics . . . 24
- 1.3 Experimental Observation and Measurement Function . . . 25
- 1.4 The Book's Running Records . . . 26
  - 1.4.1 The two-well oscillator record . . . 26
- 1.5 Experimental Data Analysis and Noise . . . 28
- 1.6 From a Record to a Conclusion . . . 29

2 Probability, Estimators, and Stationarity 35
- 2.1 Probability Density and Probability Distribution . . . 36
- 2.2 Description of Probability . . . 36
  - 2.2.1 Mean or First Moment of $p(x)$ . . . 37
  - 2.2.2 Variance or Second Central Moment of $p(x)$ . . . 37
  - 2.2.3 Higher Order Expected Values . . . 38
- 2.3 Other Frequently Used Moments . . . 38
- 2.4 Estimators . . . 39
  - 2.4.1 Sample Size Consideration . . . 39
  - 2.4.2 Mean Estimator . . . 39
  - 2.4.3 Central Limit Theorem . . . 40
  - 2.4.4 Variance, Skewness, and Kurtosis Estimators . . . 40
- 2.5 Stationarity . . . 41
- 2.6 Ergodicity . . . 44

3 Correlation and Auto-Correlation 51

4 Fourier Analysis and Power Spectral Density **61**
4.1 Fourier Series and Transforms . . . 61
4.1.1 Several Important Properties of Fourier Transforms . . . 62
4.1.2 Basic Fourier Transform Pairs . . . 63
4.2 Power Spectral Density . . . 64
4.3 Sample Power Spectra . . . 68

5 Experimental Fourier Transforms **73**
5.1 Sampling and Aliasing . . . 73
5.2 Windowing . . . 75
5.2.1 Summary . . . 77
5.3 Discrete Fourier Transform . . . 79
5.3.1 Notes on Fast Fourier Transforms . . . 81

6 Spectral Estimation, Cross-Spectra, and Frequency Response **85**
6.1 The Periodogram . . . 85
6.1.1 Definition and scaling . . . 85
6.1.2 Expectation: the spectral window . . . 86
6.1.3 Variance: the periodogram is inconsistent . . . 88
6.2 Averaged Estimates: Bartlett and Welch . . . 89
6.2.1 Segment averaging . . . 89
6.2.2 Welch's method . . . 90
6.2.3 The bias–variance–resolution trade-off . . . 91
6.2.4 Window choice . . . 91
6.2.5 Worked example: the autoregressive process of Chapter 3 . . . 92
6.3 Multitaper Estimates and Line Components . . . 93
6.3.1 Slepian tapers . . . 94
6.3.2 Eigenspectra and their average . . . 94
6.3.3 The harmonic F-test . . . 95
6.3.4 Worked example: lines over a continuum . . . 96
6.4 Statistics from Correlated Samples . . . 97
6.4.1 Variance of the sample mean . . . 98
6.4.2 Variance of the sample autocorrelation . . . 99
6.5 Cross-Spectra and Coherence . . . 100
6.5.1 The cross-spectral density . . . 100
6.5.2 Coherence . . . 100
6.5.3 Estimating the coherence . . . 101
6.5.4 What low coherence means . . . 102
6.6 Frequency Response Functions . . . 102
6.6.1 Input–output relations . . . 102
6.6.2 The H_1, H_2 and H_v estimators . . . 103
6.6.3 Random errors . . . 104
6.6.4 The single-degree-of-freedom oscillator . . . 104
6.6.5 Worked example: estimating the FRF of a noisy oscillator . . . 105
6.7 Discussion . . . 105

II Reconstruction and Invariants 113

7 Bifurcations and Poincaré Maps 117
7.1 Stability of Equilibrium Points . . . 117
7.1.1 Classification of Linear Two-Dimensional Flows . . . 118
7.1.2 Connection to Potential Energy . . . 120
7.2 Bifurcations . . . 120
7.2.1 Classification of Instability . . . 121
7.3 Stability and Bifurcation of Cycles . . . 121
7.4 Dynamic Bifurcations in Continuous Systems . . . 123
7.4.1 Periodic Vector Fields . . . 123
7.4.2 A Measured Stroboscopic Section . . . 124
7.4.3 Stability and Bifurcation of Periodic Orbits . . . 125

8 Delay Coordinate Embedding 139
8.1 Delay Reconstruction . . . 141
8.1.1 What the Theorem Leaves to the Analyst . . . 145
8.2 Phase Space Reconstruction . . . 147

9 Selecting Delay Time for Phase Space Reconstruction 153
9.1 Shannon Entropy . . . 154
9.2 Mutual Information . . . 155
9.3 Delay Selection on the Measured Record . . . 159

10 Selecting Embedding Dimension for Phase Space Reconstruction 167
10.1 The Problem . . . 167
10.1.1 Test Records and Surrogates . . . 168
10.2 The False Nearest Neighbor Test . . . 169
10.2.1 The Two Criteria of Kennel, Brown and Abarbanel . . . 169
10.2.2 Why the Test Fails on Short Records . . . 170
10.3 Cao's Method . . . 172
10.4 Nearest-Neighbor Statistics . . . 173
10.4.1 Temporal Decorrelation and the Reliable Fraction . . . 174
10.4.2 False Nearest Strands . . . 174
10.4.3 Nearest-Neighbor Distances for Noise and for Attractors . . . 175
10.4.4 The Composite Fraction . . . 178
10.5 Spatial Decorrelation . . . 179
10.6 Procedure . . . 183
10.7 Discussion . . . 187

11 Dimension Theory 193
11.1 Invariant Measure of Attractor . . . 195
11.2 Dimension Spectrum D_q . . . 195
11.3 Illustrative Examples . . . 199
11.3.1 A Line Segment . . . 199
11.3.2 An Area . . . 199
11.3.3 Middle Thirds Cantor Set . . . 199

11.4 Fractal Sets from Dynamical System . . . 201
11.5 Dimensionality of Continuous Attractors . . . 202
11.5.1 Periodic Orbits . . . 203
11.5.2 Quasi-periodic Orbits . . . 203
11.5.3 Chaotic Orbits . . . 204
11.6 Experimental Estimates of Fractal Dimension . . . 205
11.6.1 Correlation Dimension Estimate . . . 205
11.6.2 Estimators with Error Bars . . . 208
11.6.3 A Measured Record . . . 211
11.6.4 Pointwise Dimension . . . 212
11.6.5 Average Pointwise Dimension Estimate . . . 215
11.6.6 Other Generalized Dimension Estimates . . . 216
11.7 Embedding Dimension from Fractal Dimension . . . 219

12 Stability and Lyapunov Exponents **225**
12.1 Stability of Periodic Orbits . . . 226
12.2 Stability of Chaotic Orbits . . . 227
12.3 Experimental Lyapunov Exponents . . . 230
12.3.1 The Largest Short-Time Lyapunov Exponent Estimate . . . 233
12.4 The Full Spectrum and the Problem of Spurious Exponents . . . 235
12.4.1 The QR Algorithm . . . 236
12.4.2 Estimating the Tangent Map from Data . . . 236
12.4.3 Spurious Exponents . . . 237
12.4.4 Separating True from Spurious Exponents . . . 237
12.4.5 Current Practice . . . 238
12.5 Systematic Errors in Lyapunov Estimation . . . 239
12.5.1 The Scaling Region Dominates Everything . . . 239
12.5.2 A Worked Example . . . 240
12.5.3 A Measured Record . . . 241
12.5.4 Temporally Correlated Neighbors . . . 243
12.5.5 Oversampling and Degenerate Embeddings . . . 244
12.5.6 Which Average? Kantz Versus Rosenstein . . . 244
12.5.7 Stochastic Data Produce Positive Exponents . . . 245
12.5.8 Limits That No Algorithm Can Evade . . . 246

III Decomposition, Modeling, and Diagnostics **251**

13 Multivariate Analysis **255**
13.1 Proper Orthogonal Decomposition . . . 255
13.2 Smooth Orthogonal Decomposition . . . 257
13.3 Practical Calculation of POD and SOD . . . 258
13.4 Geometric Interpretation of POD . . . 260
13.5 Geometric Interpretation of SOD: . . . 260
13.6 Where the Decompositions Go Next . . . 263

14 Nonlinear Noise Reduction **269**
14.1 Noise in Measured Chaotic Data . . . 269
14.1.1 Why Linear Filters Fail and Projection Does Not . . . 270
14.1.2 The Reconstruction Both Methods Share . . . 271
14.2 Local Projective Noise Reduction . . . 272
14.2.1 Centring, Local SVD, and Projection . . . 272
14.2.2 Adjusting for Bias due to Curvature . . . 273
14.2.3 Iteration and the Procedure . . . 275
14.3 Smooth Projective Noise Reduction . . . 276
14.3.1 Strands and the Rank Condition . . . 276
14.3.2 Smooth Subspace Identification and Projection . . . 276
14.3.3 Data Padding to Mitigate the Edge Effects . . . 277
14.3.4 Weighting along the Strand and Averaging . . . 277
14.4 What the Strand Bundle Buys . . . 280
14.5 Evaluation on Simulated and Measured Records . . . 282
14.5.1 Lorenz Record . . . 282
14.5.2 Duffing Record . . . 285
14.5.3 What the Comparison Shows . . . 286
14.5.4 The Two-Well Oscillator Record . . . 290
14.6 Choosing the Parameters, and When Not to Filter . . . 294
14.7 Discussion . . . 295

15 Modeling and Predicting Chaotic Time Series **301**
15.1 Model Development in Reconstructed Phase Space . . . 302
15.2 Method of Analogues and Simple Averaging . . . 303
15.3 Local Linear Models . . . 303
15.4 Global, Operator, and Learned Models . . . 308

16 Surrogate Data Analysis **313**
16.1 Examples . . . 317
16.1.1 Linear Stochastic System . . . 317
16.1.2 Lorenz Time Series Example . . . 321
16.1.3 The Two-Well Oscillator Record . . . 322

17 Invariant Features and the Tracking of Slow Drift **331**
17.1 Time Averages as Invariant Quantities . . . 332
17.1.1 Generalized Distances . . . 333
17.1.2 Characteristic Distance and Position . . . 334
17.1.3 What the Characteristic Distance Measures . . . 334
17.2 Reconstructing Slowly Drifting Variables . . . 335
17.2.1 Why the Smoothest Coordinate Tracks the Slow Variable . . . 337
17.2.2 A Toy Problem . . . 340
17.2.3 Procedure . . . 341
17.3 Numerical Examples . . . 342
17.4 Experimental Data Example . . . 346
17.5 Noise Effects . . . 352

17.6 Discussion . . . 354

18 Recurrence and Complexity Measures **361**
18.1 The recurrence matrix . . . 361
18.1.1 Reading the textures . . . 362
18.2 Choosing the threshold . . . 363
18.2.1 Temporal correlation and the Theiler window . . . 365
18.3 Recurrence quantification analysis . . . 366
18.3.1 The measures . . . 366
18.3.2 $L_{\max}$ and the largest Lyapunov exponent . . . 367
18.3.3 Correlation entropy from the line-length distribution . . . 368
18.4 Ordinal and entropy measures . . . 370
18.4.1 Permutation entropy . . . 370
18.4.2 Trajectory-similarity entropies . . . 372
18.5 Directed coupling between channels . . . 373
18.6 Significance testing . . . 374
18.7 Discussion . . . 376

Appendices

Appendix A Algorithms in MATLAB and Python **385**
A.1 Average Mutual Information . . . 385
A.1.1 Auto Average Mutual Information Algorithm . . . 385
A.1.2 Cross Average Mutual Information Algorithm . . . 387
A.2 Correlation Dimension Algorithm . . . 389
A.3 Pointwise Dimension Algorithm . . . 392
A.4 Projective Noise Reduction Algorithm . . . 394
A.5 Smooth Projective Noise Reduction Algorithm . . . 398
A.6 Largest Short-Time Lyapunov Exponent Algorithm . . . 402
A.7 Sample MATLAB Function for Local Linear Model Prediction . . . 405
A.8 False Nearest Neighbors and Cao's Statistics . . . 407
A.9 Iteratively Refined Surrogates . . . 412
A.10 Recurrence Plots and Recurrence Quantification . . . 413
A.11 Python Support Modules . . . 415
A.12 Welch Spectra, Coherence and Frequency Response . . . 418
A.13 Multitaper Spectra and the Harmonic F-Test . . . 420

Notation

Three conventions run through the book and are worth stating once.

State and measurement. The unknown state of the system is $\boldsymbol{u}$ and lives in the phase space $\mathcal{S}$. The measurement is a scalar, $x = h(\boldsymbol{u})$, sampled at Δt to give the record $\{x_n\}_{n=1}^{N}$. Delay vectors built from that record are bold, $\boldsymbol{x}_n$. Chapter 1 writes a general, possibly vector, measurement $\boldsymbol{y} = \boldsymbol{h}(\boldsymbol{u})$; from Chapter 8 on it is the scalar x.

Estimates. A hat marks a quantity computed from a finite record: $\hat{\mu}$ is an estimate of μ. A bar marks an average over neighbors or windows, a tilde a clean (noise-free) quantity.

Matrices of data. Trajectory matrices store one reconstructed point per *column*, $\boldsymbol{Y} \in \mathbb{R}^{d \times M}$, throughout Chapters 10, 14 and 17. Chapter 13 stores time down the *rows*, following the decomposition literature, and transposes where the two meet. The orientation is stated at every decomposition.

Symbols are grouped by what they describe. The chapter given is where the symbol is first defined.

Symbol	Meaning	Ch.
Sets and spaces		
$\mathcal{S}$	phase space; the true state lives here	1
$\mathcal{A}$	attractor, or other compact invariant set, $\mathcal{A} \subset \mathcal{S}$	8
$\mathcal{B}(\boldsymbol{x}, \epsilon)$	ball of radius ϵ about $\boldsymbol{x}$	11
$\mathbb{R}^d$	reconstruction space	8
Σ	Poincaré section	7
States, measurements, reconstruction		
$\boldsymbol{u}$	true (unknown) state, $\boldsymbol{u} \in \mathcal{S}$	8
x, x_n	scalar measurement and its sampled record	8
N	number of samples in the record	2
Δt, f_s	sampling interval and rate, $f_s = 1/\Delta t$	5
$\boldsymbol{f}$, $\boldsymbol{F}$	vector field; time-$\tau\Delta t$ map of its flow	1
φ_t, $\hat{\varphi}_t$	flow on $\mathcal{S}$; induced flow on the reconstruction	8
Φ	delay map, $\boldsymbol{x}_n = \Phi(\boldsymbol{u}_n)$	8
τ	delay in samples; the delay time is $\tau\Delta t$	8
d	embedding dimension	8
$\boldsymbol{x}_n$	delay vector, $[x_n, x_{n+\tau}, \ldots, x_{n+(d-1)\tau}]^{\mathrm{T}}$	8
M	number of delay vectors, $M = N - (d-1)\tau$	8
$\boldsymbol{Y}$	trajectory matrix, one delay vector per column	10
w	Theiler window, in samples	10

continued

Symbol	Meaning	Ch.
Probability and statistics		
$p(x)$	probability density	2
$\mathrm{E}[\cdot]$, $\langle\cdot\rangle$	ensemble expectation; time average	2
μ, σ^2, σ_x	mean, variance, standard deviation	2
γ_1, γ_2	skewness and kurtosis	2
$\mathrm{Var}[\cdot]$, $\mathrm{Cov}[\cdot]$, $\mathrm{Bias}[\cdot]$	variance, covariance and bias of an estimator	2
ϵ_r	normalized random error of an estimate	6
Correlation and spectra		
$R_{xx}(\tau)$, $R_{xy}(\tau)$	auto- and cross-correlation functions	3
$C_{xx}(\tau)$, $\rho_{xx}(\tau)$	autocovariance; normalized autocorrelation	3
$S_{xx}(f)$, $S_{xy}(f)$	two-sided auto- and cross-spectral density	4
$G_{xx}(f)$	one-sided (single-sided) spectral density	4
$w(t)$, $W(f)$	data window and its Fourier transform	5
L, K	Welch segment length; number of segments averaged	6
$\gamma^2(f)$	coherence	6
H_1, H_2	frequency response function estimators	6
f_N	Nyquist frequency, $f_s/2$	5
Delay selection and embedding dimension		
H, $I(\tau)$	Shannon entropy; average mutual information	9
ϵ_i, δ_i	nearest-neighbor separation in d; appended-coordinate separation	10
r, s	FNN distance-ratio and absolute thresholds	10
$E_1(d)$, $E_2(d)$	Cao's statistics	10
$R_{nn}(d)$	composite false-neighbor fraction	10
Dimensions and invariants		
μ	invariant (natural) measure on $\mathcal{A}$	11
D_q	generalized dimension of order q	11
D_0, D_1, D_2	capacity, information and correlation dimension	11
D_T, D_P	topological and pointwise dimension	11
$C(\epsilon)$	correlation integral; $\hat{C}(\epsilon)$ its finite-sample sum	11
$\Theta(\cdot)$	Heaviside step	11
λ_i, λ_1	Lyapunov exponents; the largest	12
$\boldsymbol{Q}_n$, $\boldsymbol{R}_n$	orthonormal frame and triangular factor of the QR step	12
Multivariate decomposition		
$\mathcal{R}[\cdot]$	correlation matrix of a data matrix	13
$\boldsymbol{\phi}_p$, $\boldsymbol{\Phi}$	proper orthogonal mode (POM) and the matrix of them	13
σ_p^2	proper orthogonal value (POV)	13
$\boldsymbol{q}_p$, $\boldsymbol{Q}$	proper orthogonal coordinate (POC) and its matrix	13
$\boldsymbol{\psi}_p$, $\boldsymbol{\Psi}$	smooth projective mode (SPM), $\boldsymbol{\Psi} = \boldsymbol{\Phi}^{-\mathrm{T}}$	13

continued

Symbol	Meaning	Ch.
λ_p^2	smooth orthogonal value (SOV)	13
D^ν, $\boldsymbol{D}$	differentiation operator; its discrete form	13
Noise reduction		
$\boldsymbol{s}_i$	true state (Chapter 14 only; see above)	14
$\boldsymbol{y}_i$, $\tilde{\boldsymbol{y}}_i$, $\bar{\boldsymbol{y}}_i$	noisy, clean and filtered delay vector	14
q	retained tangent-space dimension, $D_2 \le q < d$	14
r	number of neighbors in a cloud	14
$\boldsymbol{c}_k$	center of mass of a neighborhood	14
$\boldsymbol{S}_k$, l	strand of $2l+1$ points; its half-length	14
$\boldsymbol{\Xi}_{k,q}$	retained POM basis of the cloud	14
σ_γ	measurement-noise standard deviation	14
Prediction and surrogates		
$\boldsymbol{P}$, $\boldsymbol{P}^k$	one-step predictor and its k-fold iterate	15
e_k	normalized k-step prediction error	15
T_{rev}	time-reversal asymmetry statistic	16
H_0, α	null hypothesis; rejection level	16
Tracking a slow drift		
$\mathcal{T}$	time-evolution transformation on the fast state space X	17
$\boldsymbol{\phi}$	hidden slow state (the drift variable)	17
$\boldsymbol{\mu}$	system parameters, driven by $\boldsymbol{\phi}$	17
ϵ	ratio of slow to fast time scales, $0 < \epsilon \ll 1$	17
$d_q(k)$	generalized (q-Minkowski) distance	17
$D(\boldsymbol{y})$	characteristic distance from the reference point $\boldsymbol{y}$	17
$\boldsymbol{P}(\boldsymbol{\phi}, \boldsymbol{y})$	characteristic position	17
$\{\boldsymbol{y}_j\}_{j=1}^L$	fixed reference points	17
$\boldsymbol{Y} \in \mathbb{R}^{N\times L}$	tracking matrix: N windows by L features	17
$\hat{\phi}_\ell$	tracking estimate for window ℓ	17
Recurrence		
ϵ	recurrence threshold	18
R_{ij}, $\boldsymbol{R}$	recurrence matrix, $R_{ij} \in \{0, 1\}$	18
RR, DET, ENTR	recurrence rate, determinism, line-length entropy	18
ℓ_k, $\ell_{\min}$, $L_{\max}$	diagonal line lengths, cutoff and maximum	18
PE(d)	permutation entropy of order d	18

Part I

Foundations

Part I — Foundations

This Part asks one question: *is this record fit to analyze at all?* Every method later in the book inherits the answer. A correlation dimension computed from an aliased record is a number about the sampling, not about the system.

Four properties have to be established before anything nonlinear is attempted. Whether the record is stationary, whether it is long enough, what its sampling rate resolves, and how much of it is noise. Chapters 2–6 establish all four, with the uncertainty of each estimate attached.

The order is not arbitrary. Stationarity decides whether an average means anything (Chapter 2). Correlation decides how many of the N samples are independent, which sets every error bar that follows (Chapter 3). The spectrum then says what the record contains and what the digitizer has already destroyed (Chapters 4–6).

Start here if

you have a record and have not yet plotted it. Chapter 1 builds the simulated records that recur throughout the book, and introduces the measured two-well record used in the last problem of almost every chapter.

The record drifts, or its mean wanders.

Chapter 2. Test before you average. A drifting record is the subject of Chapter 17, not of a single dimension estimate.

You need an error bar on any average.

Chapter 3, then Section 6.4. Correlated samples inflate the variance of an average by a factor that depends on what is averaged. For the sample mean it is $1 + 2\sum_{k\geq 1} \rho_k$ for a long record, and that factor is what N_{eff} counts. A mean square or any other statistic has its own factor.

The record is forced at a known frequency.

Chapters 4–5. Sample at a multiple of the forcing frequency. It makes the Poincaré section of Chapter 7 exact. It has one cost: at scales below the spacing of the phase slices, a correlation sum sees the sections and not the flow (Section 11.6.3).

You have an input and an output.

Chapter 6. The coherence tells you what fraction of the output the input explains, and H_1 versus H_2 tells you which end carries the noise. Response-only data skips to Part II.

You want to know the noise level.

Section 6.2 and Problem 4.6. The flat floor above the highest active line is the number every later chapter needs.

What this Part cannot do is decide whether the record is deterministic. A broad spectrum is produced by chaos and by noise alike. That question needs the geometry of Part II and the surrogate tests of Chapter 16.

1 Introduction

This book presents and explores the *experimental analysis* of *dynamical systems.*[**TODO: DECIDE: provenance note (first draft by JPC for a Penn State course; enlarged by DC for the URI Experimental Nonlinear Dynamics course) moved out of the chapter; place it in the preface or drop it.**] The material is at the graduate level and assumes familiarity with basic calculus and linear algebra. We therefore do not repeat methods and concepts covered in undergraduate engineering and mathematics courses; for the underlying theory of nonlinear oscillations and bifurcations we refer throughout to Guckenheimer and Holmes [82], and to Strogatz [195] for a gentler introduction. We start by describing what we mean by *dynamical systems.*

1.1 Dynamical Systems

Empirically, any collection of *interacting* objects that *changes over time* is a *dynamical system* if we believe that there is some "structure" to that change. Most generally, by "structure" we mean that the following three hypotheses are valid for our system:

1. There is a *phase space* (or *state space*) $\mathcal{S}$ such that "points" or "elements" $x \in \mathcal{S}$ define the **current situation** in the system (i.e., its "state").

   - $\mathcal{S}$ may be a *discrete set* (e.g., symbols in an alphabet) but more typically, $\mathcal{S}$ is a *vector space* (e.g., $\boldsymbol{u} \in \mathbb{R}^n$) or a *manifold* (a "surface" that locally looks like $\mathbb{R}^m$)
   - $\mathcal{S}$ can be finite-dimensional or infinite-dimensional (for the latter, consider any continuous field)

2. There is some notion of *time*, t. Time can be continuous ($t \in \mathbb{R}$, e.g.) or discrete ($t \in \{t_1, t_2, t_3, \ldots\}$).[1]

3. There is an *evolution law* that acts on $\mathcal{S}$ to take the *current state* into *future states.*

   - For a *continuous time* system, the evolution law will be a system of *differential equations* (in theory)

     $$\dot{x} = f(x, t) \quad (x \in \mathcal{S}, t \in \mathbb{R}) \tag{1.1}$$

     Note: time dependence can be stochastic (random), and f may be a *partial differential operator* if Eq. (1.1) is a partial differential equation.

[1] It may be the case that $\Delta t_n = t_{n+1} - t_n \neq \Delta t_m$ for $n \neq m$!

- For *discrete time* systems, it will be a *map*

$$x_{n+1} = f(x_n, n) \quad (x_n \in \mathcal{S}, n \in \mathbb{Z}) \tag{1.2}$$

  Note: t_n can be mapped on n.

> Thus, empirically, the notion of a **dynamical system** is a *hypothesis* on the object (or collection of objects) that you are observing.

Indeed, the most *fundamental* type of experiment aims to learn about items 1 and 3 above (item 2 is usually clear from context):

- What is $\mathcal{S}$? In particular, what is its *dimension*?
- What can we say about the *evolution law*? Is it *linear* or *nonlinear*? Is it *deterministic* or *stochastic*?

> The three hypotheses listed above are the foundation of a *dynamical systems perspective* (DSP). Concepts like "stability," "chaos," "fractals," or "attractors" are *consequences* of this point of view, but *not essential* to it!

The main consequences of the dynamical systems perspective are as follows:

1. *Trajectories* of the system (i.e., "solutions" to Eq. (1.1) or Eq. (1.2)) lie on (or near with small noise) *curves* in $\mathcal{S}$
2. Curves and surfaces left *invariant* by the action of the system play an *important role* in the dynamics
3. *Asymptotic behavior* (stability) can be precisely defined using a notion of distance ($\|x - y\|$) between points $x, y \in \mathcal{S}$

In short, the DSP converts the study of *time evolution* into the study of *geometrical properties* (at least to some extent) [82]. Lorenz's study of a three-mode truncation of thermal convection [127] remains the cleanest demonstration of how much this shift in viewpoint buys: a system of three ordinary differential equations, no forcing, and an aperiodic solution confined to a set of measure zero.

1.2 Some Examples of Dynamical Systems

This section presents several sample dynamical systems and discusses the types of questions that DSP will be used to answer.

1.2.1 Linear Mass-Spring System

Imagine that we have a collection of nine masses (m_i, $i = 1, \ldots, 9$) connected by springs and excited by some motor driver as shown in Fig. 1.1. All masses are constrained to move in a plane, and we can measure the vertical and horizontal displacements of the mass m_5 using optical sensors.

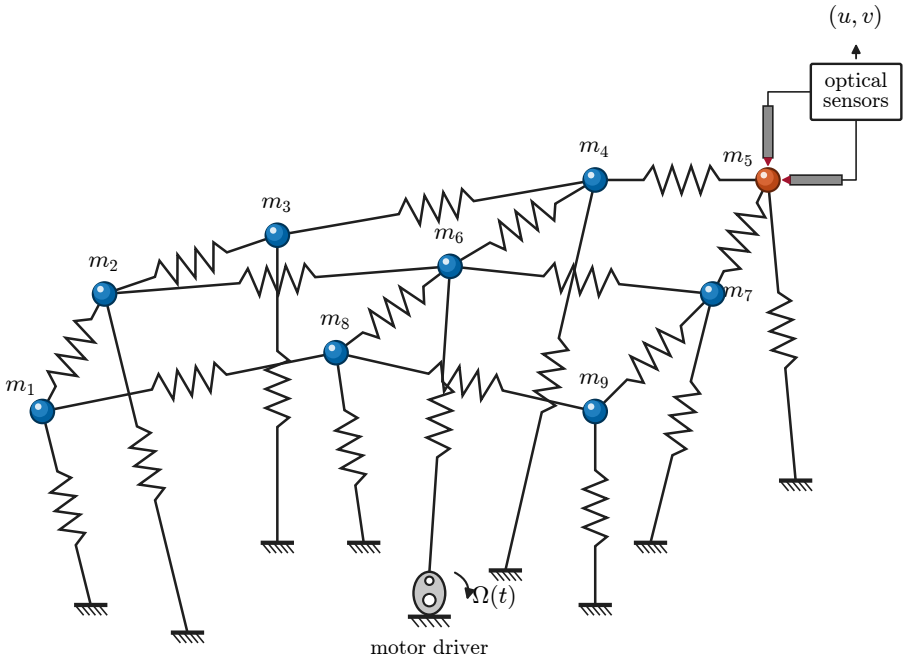


Figure 1.1: A planar network of nine masses joined by linear springs and tied to fixed supports. A motor driver forces m_6 through a cam at angular rate $\Omega(t)$; two optical sensors read the horizontal and vertical displacements of m_5, giving the observation (u, v). Script `python/mkfig_massspring.py`.

The dynamics of this system can be modeled by a system of linear ordinary differential equations if we assume that the excitation amplitude and mass motions are sufficiently small:

$$\begin{aligned} \dot{\boldsymbol{u}} &= \boldsymbol{A}(t)\boldsymbol{u} + \boldsymbol{b}(t)\,, \\ \boldsymbol{y} &= \boldsymbol{h}(\boldsymbol{u})\,, \end{aligned} \tag{1.3}$$

where $\boldsymbol{A}(t) \in \mathbb{R}^{36\times 36}$ is the system (or state) matrix[2] and its time dependence indicates parametric forcing, $\boldsymbol{b} \in \mathbb{R}^{36}$ is an external forcing function, $\boldsymbol{u} = [x_1, y_1, x_2, y_2, \ldots, \dot{x}_1, \dot{y}_1, \ldots]^{\mathrm{T}} \in \mathbb{R}^{36}$ is a state variable, $\boldsymbol{h} : \mathbb{R}^{36} \to \mathbb{R}^2$ is the observation function, generating observations $\boldsymbol{y} = (x_5, y_5)^{\mathrm{T}}$ in this particular case. Experimentally, we only have observations from the optical sensors sampled at some discrete time steps:

$$\begin{bmatrix} u_1 & u_2 & u_3 & \cdots \\ v_1 & v_2 & v_3 & \cdots \end{bmatrix} \tag{1.4}$$

Then the question is that if we can determine the mapping from one observation to another:

$$\begin{pmatrix} u_{n+1} \\ v_{n+1} \end{pmatrix} \triangleq \boldsymbol{y}_{n+1} = \boldsymbol{C} \begin{bmatrix} \boldsymbol{y}_n \\ \boldsymbol{y}_{n-1} \\ \vdots \\ \boldsymbol{y}_{n-m} \end{bmatrix} + \mathrm{OT} \tag{1.5}$$

where $\boldsymbol{C} \in \mathbb{R}^{2\times 2(m+1)}$, and $\mathrm{OT} \in \mathbb{R}^2$ indicates other possible terms such as bias. Other questions might be: what is the value for m? Can it be just 0? Is the number of columns of $\boldsymbol{C}$ $(2(m+1))$ indicative of the dimensionality of $\mathcal{S}$? What can we say about $\mathcal{S}$ and its evolution law?

[2]Not to be confused with the state-transition matrix $\boldsymbol{\Phi}(t, t_0)$, the solution operator that maps $\boldsymbol{u}(t_0)$ to $\boldsymbol{u}(t)$.

1.2.2 Moon-Holmes Oscillator

Let us consider the classic nonlinear dynamics experiments first investigated by Moon and Holmes [95, 142] shown in Fig. 1.2. The apparatus consists of a cantilever beam fixed into a horizontally moving frame, where the beam's free end is buckled by a two-well potential (see Fig. 1.2(b)) realized by two rare-earth permanent magnets. The frame is driven by an electromagnetic shaker that can provide harmonic force excitation. From physics, we expect the partial differential equation (PDE) for the field $u(\cdot, t) \in \mathcal{S}$ to model the transverse oscillations of the beam. Therefore, our system is infinite-dimensional (u is a scalar field variable).

The two-well Duffing model of this apparatus, and the strange attractor it supports, are analyzed in detail by Holmes [96]; Moon's monograph [143] places the experiment in the wider context of chaotic vibrations, and Tam and Holmes [200] revisit it with modern instrumentation.

The dynamics of this system is measured using a strain gauge attached to the beam near its clamped end. The gauge provides strain amplified voltage $s(t)$ or $\{s_1, s_2, s_3, \ldots\}$, a *scalar* signal or time series. The objectives could be as follows: What is the actual form of the double-well potential at the tip of the beam? What parameters determine the variation in dynamical behavior? Can we infer the dynamical properties of u, such as its effective dimensionality and the corresponding predictive model?

1.2.3 Chua-Matsumoto Circuit

Chua-Matsumoto circuit [135, 137] is a simple electronic circuit with chaotic behavior. A version of the circuit is shown in Fig. 1.3, where the variables used in the following model are indicated near the components (in capital letters):

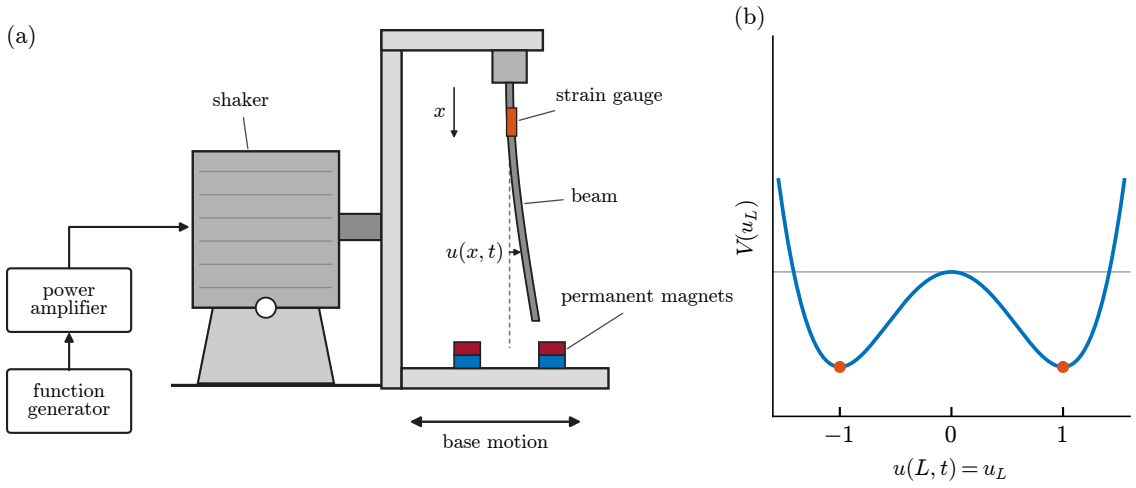


Figure 1.2: The Moon–Holmes magneto-elastic oscillator. (a) A shaker moves a rigid frame horizontally; a cantilever hangs from the frame, two permanent magnets on its base buckle the free end, and a strain gauge near the clamp gives the measured signal; $u(x, t)$ is the transverse deflection. (b) The double-well potential near the free end, $V(u_L) = -u_L^2/2 + u_L^4/4$ in scaled units. Script `python/mkfig_moonholmes.py`.

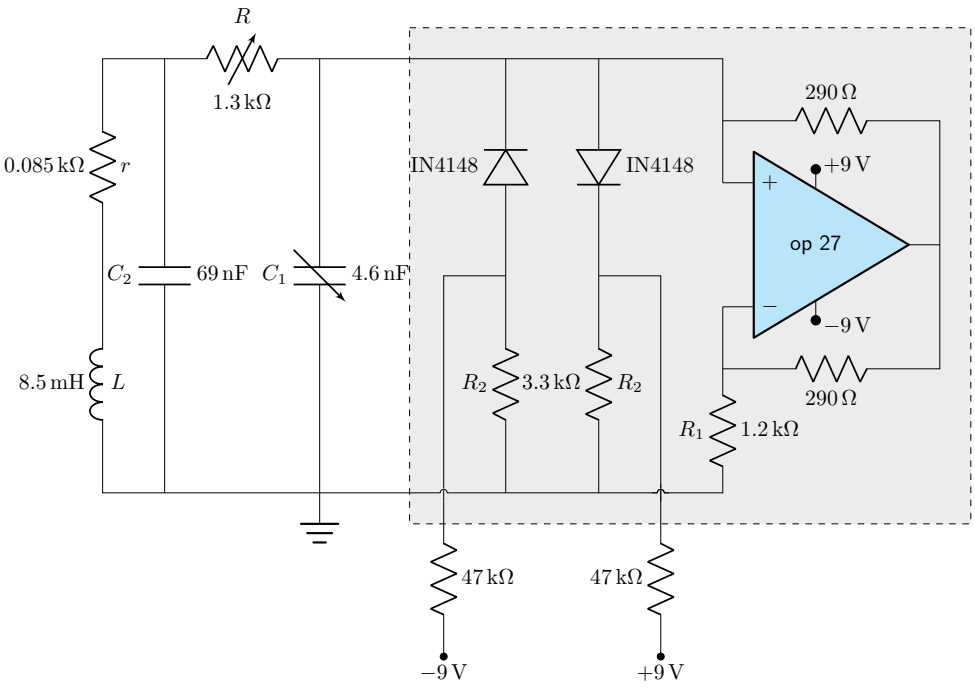


Figure 1.3: A version of the Chua-Matsumoto circuit

$$\dot{x} = \alpha(y - x - f(x))\,, \tag{1.6}$$

$$\dot{y} = x - y + z\,, \tag{1.7}$$

$$\dot{z} = -\beta y\,. \tag{1.8}$$

The function $f(x)$ describes the behavior of the nonlinear negative resistor realized in the right part of the circuit, and parameters α and β can be altered by varying particular values of the circuit components R and C_1. The bifurcation structure of the resulting double-scroll attractor, together with a rigorous proof of chaos, is established by Chua, Komuro, and Matsumoto [38].

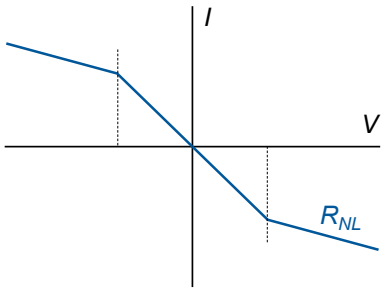


Figure 1.4: The current-voltage characteristic of the Chua-Matsumoto nonlinear resistor R_{NL}

In this circuit, the dashed rectangle encompasses elements that form a *nonlinear negative* resistor R_{NL}, [3] which can be represented by the graph on Fig. 1.4.

[3] Usually $V = RI$, or voltage equals resistance times current, where $R > 0$.

1.2.4 Shooting an Arrow at a Target

Now, let us consider the task of shooting an arrow at a target illustrated in Fig. 1.5. Here, the objective of the task is to hit a bulls-eye. After each try, we observe the error e in targeting and adjust our arrow release angle, the tension in the string, or other targeting mechanisms. Thus, we observe a sequence of $\{e_1, e_2, e_3, \ldots\}$ and ask a basic question: Is there a "first principles" model that can describe the targeting dynamics (i.e., learning dynamics)? Or, more simply, is there some general functional dependence between successive errors in targeting, $e_{n+1} = f(e_n, n)$? Further, we can investigate which parameters need more strict control for accurate targeting and which are not critical.

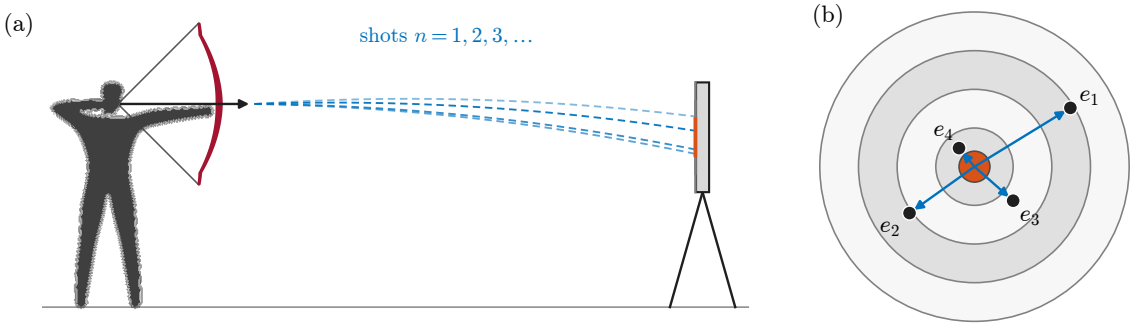


Figure 1.5: Shooting an arrow at a target. (a) Successive shots by the same archer. (b) The target face: the hit of shot n and its targeting error e_n, the vector from the bull's-eye to the hit. The hit positions are illustrative. Scripts `python/render_archer_blender.py` and `python/mkfig_target.py`.

1.2.5 Fly Population Dynamics

Another interesting example of a nonlinear map model is the fly population dynamics model [20, 217]. For example, consider a population of flies that lives in a confined space (e.g., a jar), and we control the amount or rate of the food supply. If we denote the population or number of n-th generation of flies as x_n, then a linear model of the population dynamics can be expressed as:

$$x_{n+1} = a x_n \,, \tag{1.9}$$

where the parameter $a > 0$ and depends, for example, on the available food supply. However, we see the problem of the linear model Eq. (1.9): if $a > 1$, then $x_n \to \infty$ as $n \to \infty$ (unbounded growth); and if $0 < a < 1$, then $x_n \to 0$ as $n \to \infty$ (total extinction). Thus, we have no nonzero steady-state population of flies for a linear model!

To look for a nonzero steady state solution, one may start to consider introducing nonlinearity into Eq. (1.9), e.g.:

$$x_{n+1} = a x_n + b x_n^2 \,, \tag{1.10}$$

where the parameter b can be used to control the nonlinear cost of population growth, or in general, we may look for a more appropriate model $x_{n+1} = f(x_n)$, where the form of f can be ascertained from the experimental observations or other physical/biological arguments.

Other examples of dynamical systems of interest include commodity price dynamics, global warming, and protein folding.

1.3 Experimental Observation and Measurement Function

The difference between *discrete* and *continuous* systems is not as strong as initially seems. Let us consider a continuous dynamical system:

$$\dot{\boldsymbol{u}} = \boldsymbol{f}(\boldsymbol{u}, t) \tag{1.11}$$

with a solution starting from an initial condition $\boldsymbol{u}(t_0) = \boldsymbol{u}_0$:

$$\boldsymbol{u}(t) = \boldsymbol{U}(t; t_0, \boldsymbol{u}_0) \tag{1.12}$$

then let us denote $\boldsymbol{u}_1 = \boldsymbol{U}(t_1; t_0, \boldsymbol{u}_0)$, $\boldsymbol{u}_2 = \boldsymbol{U}(t_2; t_0, \boldsymbol{u}_0)$, and

$$\boldsymbol{u}_{n+1} = \boldsymbol{U}(t_{n+1}; t_0, \boldsymbol{u}_0) \triangleq \boldsymbol{F}(\boldsymbol{u}_n, n), \tag{1.13}$$

or thinking in terms of a numerical algorithm:

$$\boldsymbol{u}_{n+1} = \boldsymbol{u}_n + \boldsymbol{f}(\boldsymbol{u}_n, t_n)\Delta t_n, \tag{1.14}$$

which is Euler step iteration. Throughout the book, all systems are treated as "discrete" at the level of observations.

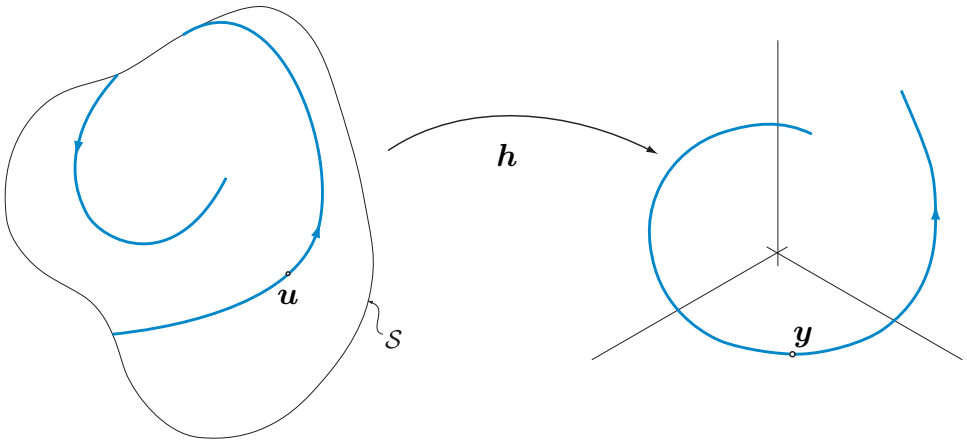


Figure 1.6: Mapping from the phase space to the measurement space

As we saw in the examples, we do not, in general, observe $\boldsymbol{u} \in \mathcal{S}$ *directly*. Instead, we usually have:

$$\begin{aligned} \boldsymbol{u}_{n+1} &= \boldsymbol{f}(\boldsymbol{u}_n, n)\,, \quad \boldsymbol{u}_n \in \mathcal{S}\,, \\ \boldsymbol{y}_{n+1} &= \boldsymbol{h}(\boldsymbol{u}_{n+1})\,, \quad \text{(measurement system)} \end{aligned} \tag{1.15}$$

where $\boldsymbol{y}_n$ does not even have the same dimension as $\boldsymbol{u}_n$. Graphically, we can picture $\boldsymbol{h}$ mapping as shown in Fig. 1.6. We usually like $\boldsymbol{h}$ to be *linear* (i.e., $\boldsymbol{y}_n = \boldsymbol{C}\boldsymbol{u}_n$), but at minimum we need it to be smooth and generic; it need not be one-to-one, since the injectivity lost in the measurement is recovered by the delay map of Chapter 8.

This loss of direct access to $\boldsymbol{u}$ appears fatal, and is not. Packard *et al.* [151] showed empirically that lagged copies of a single scalar observable recover the geometry of the underlying attractor, and

Takens [197] proved that generically they must: for a sufficiently large number of lags the delay map is an embedding. Sauer, Yorke, and Casdagli [173] later sharpened the result and clarified which dimension is the relevant one. Chapters 8–10 develop this machinery and the practical business of choosing the lag and the number of lags.

1.4 The Book's Running Records

Every method in this book is demonstrated on the same small set of records, so that the reader can follow one system from raw signal to invariant to model. Three are simulated and one is measured.

The simulated records are the Lorenz system with $(\sigma, \rho, \beta) = (10, 28, 8/3)$, the two-well Duffing oscillator $\ddot{x} + 0.25\dot{x} - x + x^3 = 0.3\cos t$, and the Hénon map with $(a, b) = (1.4, 0.3)$. Their parameters are fixed here and are not restated in later chapters unless a chapter changes them. Where a chapter needs a linear stochastic benchmark it uses the AR(2) process of Chapter 3.

1.4.1 The two-well oscillator record

The measured record is a strain time series from the magneto-elastic beam of Fig. 1.2, the physical realization of the two-well Duffing oscillator. It is the one record in the book whose governing equation is not known exactly, and it is used to show what each method returns when the model is only approximately right. The record is referred to as the *two-well oscillator record* throughout, and the problems that use it assume it is available as a single column of strain samples with the metadata below.

The record is the run of 2 June 2002 on the rig of Chelidze and Liu [35]. A notched cantilever beam sits between two rare-earth magnets on an electromagnetic shaker. A strain gauge near the clamped end, just above the notch, is the single observable. The beam was driven hard enough to produce cross-well chaos. The file is `data/twowell_2002-06-02.csv`: one column, no header, 96,000 samples in volts, loaded by `python/_twowell.py` or `matlab/twowell.m`.

Table 1.1: Metadata of the two-well oscillator record. The right column separates what was recorded at the time from what is recovered from the file itself.

quantity	value	source
sampling frequency f_s	160 Hz	reported
forcing frequency f_d	10 Hz	reported
samples per forcing period	16	measured
A/D converter	16 bit over ± 5 V	measured
record length	96,000 samples, 600 s, 6000 forcing periods	measured
mean, standard deviation	-0.187 V, 1.582 V	measured
range	-3.343 to 3.232 V	measured

The bottom five rows of Table 1.1 are worth the reader's attention, because they are read out of the file rather than out of a laboratory notebook, and a reader handed an unlabelled record can do the same. The A/D resolution appears nowhere in the file. It is recovered by sorting the distinct sample values and taking the smallest gap: every value in the record lies on a grid of spacing $10/65536$ V exactly, which is a 16-bit converter over ± 5 V. The forcing period is recovered from the spectrum, Fig. 1.7(c): the only sharp lines sit at $f_s/16$, $f_s/8$ and $3f_s/16$, a drive at 10 Hz and its first two harmonics on a broadband chaotic background. Both agree with what was recorded at

the time, which is the outcome one hopes for and should still check. A record whose recovered rate disagrees with its label is not a curiosity; it is the most common single cause of a published exponent that nobody can reproduce.

Everything below is quoted in samples or in forcing periods first. Those are the units the estimators work in, and they survive a mislabelled sampling rate. The conversion here is exact: one forcing period is 16 samples, one sample is 6.25 ms.

Fig. 1.7 shows the record. Panel (a) gives 40 forcing periods of it, and the coarseness is real: at 16 samples per period the trace is sampled about as sparsely as a delay reconstruction tolerates. Panel (b) is the amplitude distribution over the whole record, with the two wells plain and the barrier region well populated, so the motion is crossing between wells and not rattling in one of them. The mean residence time on one side of the mean is 1.6 forcing periods. Panel (d) is the delay reconstruction with $\tau = 5$ samples, and on top of it the stroboscopic section, one point per forcing period.

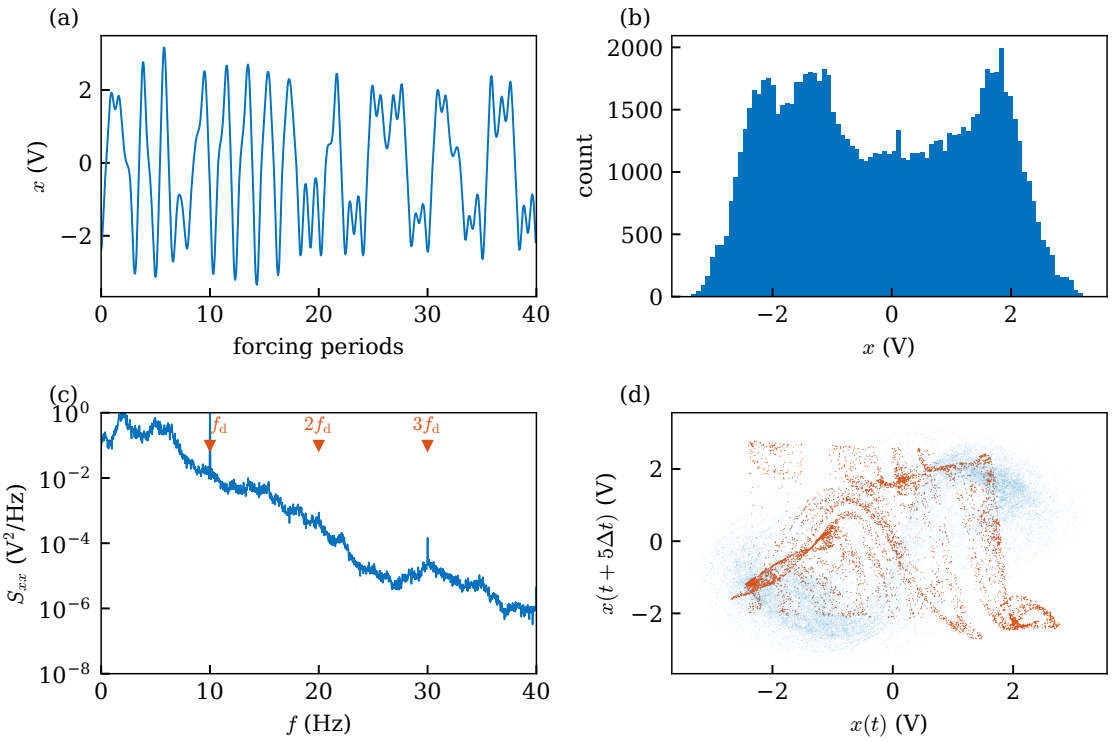


Figure 1.7: The two-well oscillator record. (a) 40 forcing periods, starting at sample 3000. (b) amplitude distribution of the whole record, bin width 512 A/D steps. (c) Welch spectral density, 8192-point segments, with the drive line $f_d = f_s/16$ and its first two harmonics marked. (d) delay reconstruction with $\tau = 5$ samples (light) and the stroboscopic section at the drive period (dark). Script `python/mkfig_twowell_data.py`.

The record is stationary to the precision that matters here. Split into ten blocks of 600 forcing periods, the block means lie between -0.14 and -0.22 V and the block standard deviations agree to within 2.5 percent. This is worth checking before anything else: the rig was accumulating fatigue damage in the notch over the course of the original experiment, which is what [35] set out to track,

and a record long enough to show that drift would not be usable as a stationary example. Ten minutes is short enough.

Before the record arrived, the model alone gave the following expectations: a Poincaré section with the two-well structure of Chapter 7; an embedding dimension of three to five (Chapter 10); a correlation dimension between two and three (Chapter 11); a largest Lyapunov exponent that is a fraction of the forcing frequency (Chapter 12); and a surrogate test that rejects the linear null (Chapter 16). Table 1.2 collects what the record returns. Each entry is derived in the chapter named, and none of them is a comparison with a known answer, because there is no known answer. They are consistency checks.

Table 1.2: What the two-well oscillator record returns. Each estimate is derived in the chapter named, where the scaling range, the fitting window and the sensitivity to them are given. The ± 0.03 on D_2 is the spread over d at fixed scales, not a confidence interval; over disjoint blocks of the record, Theiler windows and shifted scale ranges the spread is about ± 0.05 (Section 11.6.3).

quantity	estimate	stable over	chapter
AMI first minimum	$\tau = 5$ samples	—	9
autocorrelation first zero	12 samples	—	3
embedding dimension	$d_E = 4$	Kennel and Cao agree	10
correlation dimension	$D_2 = 2.68 \pm 0.03$	$d = 4$ to 8	11
largest Lyapunov exponent	0.61 nats per forcing period	$d = 4$ to 7	12
surrogate test	rejects, $p = 0.025$	39 IAAFT surrogates	16

All five expectations held, which is a pleasant outcome and not a guaranteed one. One entry deserves a caution the table cannot carry. The embedding theorem of Sauer, Yorke and Casdagli is sufficient, and it is stated for the box-counting dimension D_0. The record yields only the correlation dimension, and $D_2 \leq D_0$. With $D_2 = 2.68$ the theorem asks for at least $d = 6$, possibly more, so the data do not certify its requirement at any d. The false-nearest-neighbor count is already zero at $d = 5$. The two are not in conflict. One is a generic sufficient condition that the data cannot certify; the other shows that a working reconstruction is supported at the resolution a finite noisy record provides. Results are therefore reported across several d. Chapter 10 settles which d to use downstream. The Lyapunov exponent is $6.1\,\mathrm{s}^{-1}$ at this sampling rate; it is quoted per forcing period in Table 1.2 because that is the form in which it can be compared with the model of Chapter 12.

1.5 Experimental Data Analysis and Noise

Experimental data always contains some form of noise [105]. Even the cleanest data sets have digitization noise. Additive noise can enter as process noise $\boldsymbol{p}(n)$:

$$\boldsymbol{u}_{n+1} = \boldsymbol{f}(\boldsymbol{u}_n, n) + \boldsymbol{p}(n) \tag{1.16}$$

or as measurement noise $\boldsymbol{q}(n)$:

$$\boldsymbol{y}_{n+1} = \boldsymbol{h}(\boldsymbol{u}_{n+1}) + \boldsymbol{q}(n+1) \tag{1.17}$$

The challenge then is to learn as much about the system of equations Eq. (1.16) as we can, given only the measurement time series $\{\boldsymbol{y}_n\}_{n=0}^{N}$. This is the problem this book addresses; the treatments by Kantz and Schreiber [105] and Abarbanel [4] are the standard complementary references. We do this with two general types of data analysis:

- Characterization:
  - What does the system do?
  - How does it behave in general?
  - What is its "structure"?
  - Is it operating as expected?
- Prediction:
  - What will the system do in the future?
  - What is a good model for it?

1.6 From a Record to a Conclusion

Figure 1.8 orders the methods of this book as they are applied to one measured record. It is a sequence of decisions, not a pipeline. Each diamond is a question the record must answer, and a wrong answer near the top invalidates every number below it. A dimension estimated from a drifting record measures the drift; one estimated from an aliased record measures the sampling.

Three features of the figure are deliberate. First, the branches on the right replace a step of the main sequence rather than add one. Drift, several channels, or periodic forcing changes what is reconstructed, and Chapters 17, 13 and 7 say how. Second, reconstruction yields a range of (τ, d), not a point, and every later result is reported across that range. Third, the surrogate test is applied to the result that will be reported. The surrogates then pass through the same reconstruction, filtering and estimation as the data (Chapter 16). A rejection rules out a filtered Gaussian process. It does not, by itself, establish chaos.

The figure omits the loops that practice requires. A filter lowers the noise floor, which can change the admissible d. A test that fails to reject may send the analysis back to the choice of statistic. The openings of Parts I–III give the same decisions in more detail.

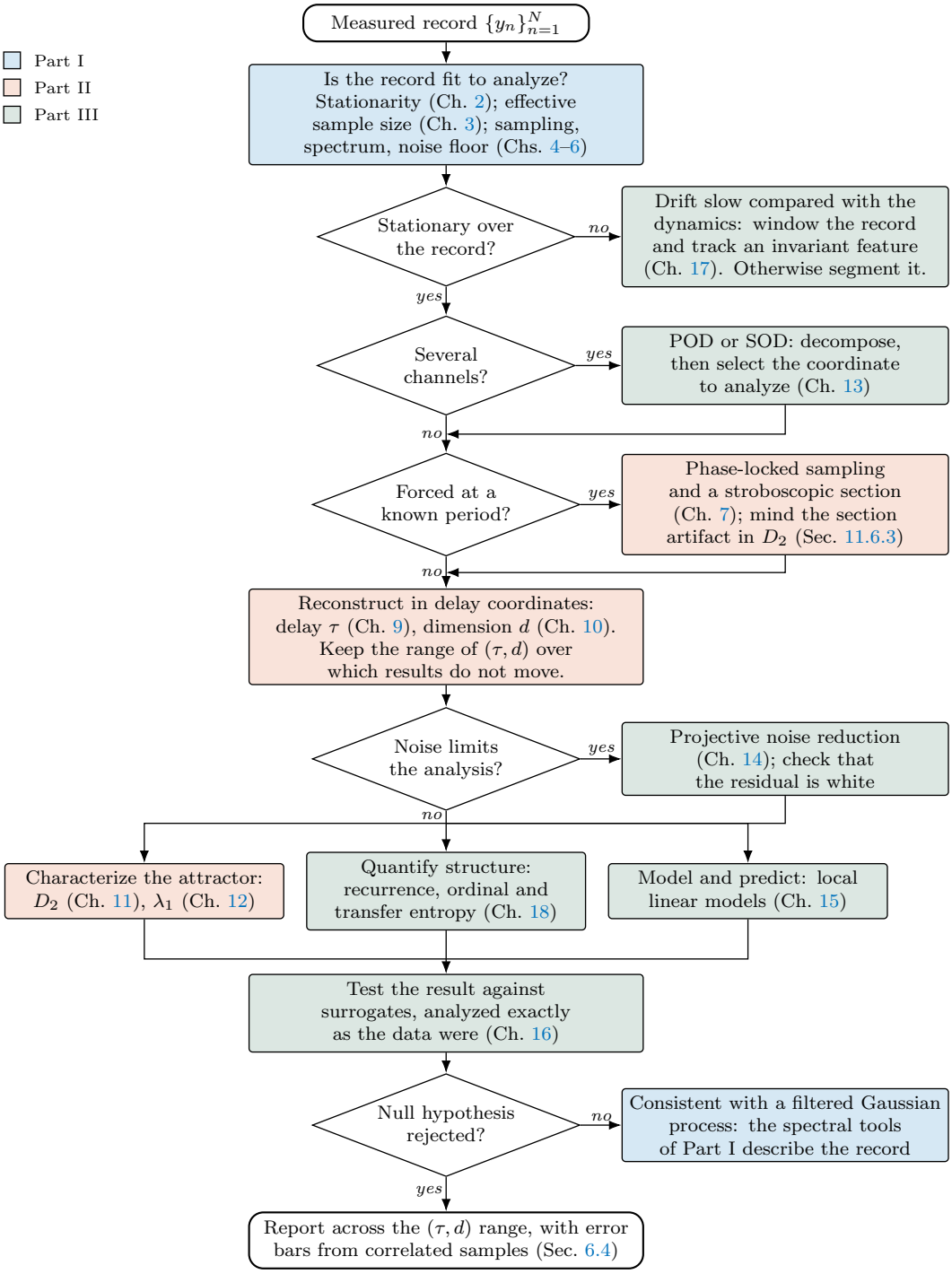


Figure 1.8: Decision workflow for a measured record. Diamonds are questions the record must answer; fill color marks the Part that treats each step. Branches on the right replace a step of the main sequence. The surrogate test is applied to the reported result, with the surrogates passed through the same steps as the data.

Problems

The problems throughout the book use *MATLAB* or *Python* (NumPy/SciPy); Appendix A supplies both versions of every algorithm. We start by familiarizing ourselves with simulations of continuous and discrete systems. Refer to the *ordinary differential equations* section of the MATLAB documentation (https://www.mathworks.com/moler/odes.pdf) or to SciPy's `solve_ivp` (https://docs.scipy.org/doc/scipy/reference/generated/scipy.integrate.solve_ivp.html). Note, in particular, how to choose an ODE solver and which tolerance options are available.

Every problem carries one label, set beside its number. Core marks a problem whose difficulty is the idea: it works the chapter's central result, usually with pencil and paper or a short script. Computational marks one whose difficulty is the code: an estimator to implement, or a sweep to run on simulated records. Experimental marks a problem that uses one of the measured records supplied with the book. Extension marks one that goes beyond the chapter, either by opening a question the chapter leaves unsettled or by asking for a comparison across chapters. A course can be built from the Core problems alone. Where a problem, or one of its parts, needs material from a later chapter, the heading says so: a grey *needs Ch.* note beside the number. Those problems are worth returning to rather than skipping, and the text names the section they wait on.

Problem 1.1 Core

Iterate a one-dimensional Logistic map [139]

$$x_{n+1} = A x_n (1 - x_n) \,.$$

Specifically, advance successively the variable x_n for $n = 0, 1, 2, \ldots$ starting with the initial condition $x_0 = 0.1$ to obtain a large sequence of values.

1. Plot the evolution of x_n versus n for a fixed value for (a) $0 < A < 3$, (b) $3 < A < 3.5699$, and (c) $A = 4$. What can you say about each of the obtained sequences?

2. What happens to the sequences if $A < 0$ and $A > 4$?

3. Make the corresponding plots of x_n versus x_{n+1} for all the above cases. What can you say about each of the plots?

Problem 1.2 Core

Iterate a two-dimensional Ikeda map [99]

$$\begin{aligned} x_{n+1} &= 1 + 0.9(x_n \cos\phi - y_n \sin\phi) \,, \\ y_{n+1} &= 0.9(x_n \sin\phi + y_n \cos\phi) \,, \end{aligned}$$

where

$$\phi = 0.4 - \frac{6}{1 + x_n^2 + y_n^2} \,,$$

using using zero initial conditions $x_0 = 0$ and $y_0 = 0$ over a large range (thousands) of n values. Plot the y_n values versus x_n (do not connect points with lines, just plot points). What can you say about the obtained plot, and how does it compare to the sequences observed in Problem 1.1?

Problem 1.3

Core

Simulate free response of damped harmonic oscillator

$$\ddot{x} + 2\zeta\dot{x} + x = 0$$

for different values of the damping ratio ζ and initial conditions. Plot the response $x(t)$ and $\dot{x}(t)$ versus time t and the corresponding trajectories in the phase space for underdamped ($0 < \zeta < 1$), critically damped ($\zeta = 1$), and overdamped systems ($\zeta > 1$).

Do the same for the harmonically forced system:

$$\ddot{x} + 2\zeta\dot{x} + x = f\cos\omega t$$

for some choice of f and $\omega \neq 1$, and plot the solutions, including transient response and some part of the steady-state.

Problem 1.4

Core

Simulate two-well Duffing oscillator [49, 118]

$$\ddot{x} + 0.25\,\dot{x} + x(x^2 - 1) = 0.3\cos t$$

using MATLAB's `ode45` or SciPy's `solve_ivp`, and using initial conditions $x(0) = -0.67079530230$ and $\dot{x}(0) = -0.36933661878$ over the time interval $t \in [0, 100]$. Sample the trajectory with the uniform sampling time interval $t_s = 0.1$. Now assume that our measurement from this system is $u_i = x_i$, and do the following:

- Plot the original systems phase space trajectory by plotting x_i versus $\dot{x}_i$.
- Now plot our measurements versus their time-delayed version: x_i versus x_{i+12}.

How do these two plots compare? Can you say that our measurement can reconstruct the original system's trajectory?

Problem 1.5

Core

Two maps that arise from the models of this chapter.

1. The nonlinear fly model Eq. (1.10) with $a > 1$ and $b < 0$. Find its nonzero fixed point x^* and show that a perturbation $\delta_n = x_n - x^*$ obeys $\delta_{n+1} \approx (2 - a)\,\delta_n$, so that the population settles to x^* only for $1 < a < 3$. Then show that the substitution $x_n = -(a/b)\,y_n$ turns Eq. (1.10) into the logistic map of Problem 1.1 with $A = a$. Which of the regimes seen in Problem 1.1 does a fly population with $a = 3.8$ therefore exhibit?

2. The Euler step $\boldsymbol{u}_{n+1} = \boldsymbol{u}_n + \boldsymbol{f}(\boldsymbol{u}_n)\Delta t$ is a map, and it need not share the properties of the flow it approximates. Write it for the undamped oscillator $\ddot{x} + x = 0$ in the state $\boldsymbol{u} = [x, \dot{x}]^{\mathrm{T}}$ and show that both eigenvalues of the resulting 2×2 matrix have modulus $\sqrt{1 + \Delta t^2}$. Conclude that the energy $x^2 + \dot{x}^2$ grows by the factor $(1 + \Delta t^2)^{2\pi/\Delta t}$ per period of the oscillator, whatever the step, and evaluate it for $\Delta t = 0.1$ and 0.01 (you should find about 1.87 and 1.06). Confirm by iterating the map for 50 periods and comparing with `ode45` or `solve_ivp` at default tolerances. The qualitative object (a center) has been turned into a different one (an unstable spiral): which of the three consequences of the dynamical systems perspective listed in this chapter is being violated by the numerical scheme?

Problem 1.6

Experimental

Load the two-well oscillator record of Section 1.4.1, a single column of strain samples $\{s_i\}$ taken at the sampling rate given with the record.

1. Plot s_i against time for a stretch of about twenty forcing periods, using the forcing frequency given with the record to convert samples to periods. Describe what the beam does: how long does it stay in one well before crossing, and is the crossing sequence periodic?

2. Plot a histogram of the whole record with at least 50 bins. How many peaks does it have, and how do they relate to the potential of Fig. 1.2? Compare with the same histogram for the x record simulated in Problem 1.4.

3. Repeat the delay plot of Problem 1.4: plot s_i against s_{i+k} for k equal to roughly one twelfth, one quarter, and one half of the forcing period, expressed in samples through the sampling rate. Which k gives a picture most like the $(x, \dot{x})$ portrait of the model, and why does the k nearest a quarter period do best? Which of the two questions asked in the Moon–Holmes example of this chapter (the form of the potential; the effective dimensionality) does this plot begin to answer?

2 Probability, Estimators, and Stationarity

We start with some basic preliminaries, following the standard engineering treatment of random data [8]; Papoulis and Pillai [152] give the underlying probability theory at greater depth. Time history records can be of two general types:

Deterministic: The record is generated by an evolution law that fixes the future state uniquely, given the present state and the inputs. Determinism does not imply long-term predictability. A chaotic system is deterministic, yet small errors in the state grow and limit prediction to a finite horizon (Chapter 12).

Random: The future is not fixed by the present state and inputs. Repeated experiments from the same state produce different time history records, which can be described only through probability distributions.

The collection of all records of time histories $x_i(t)$, $i = 1, 2, 3, \ldots$, of a random process, which might have been produced by an experiment, is called an *ensemble* which defines the random process $\{x(t)\}$ that describes the physical phenomenon (see Fig. 2.1).

2.1 Probability Density and Probability Distribution

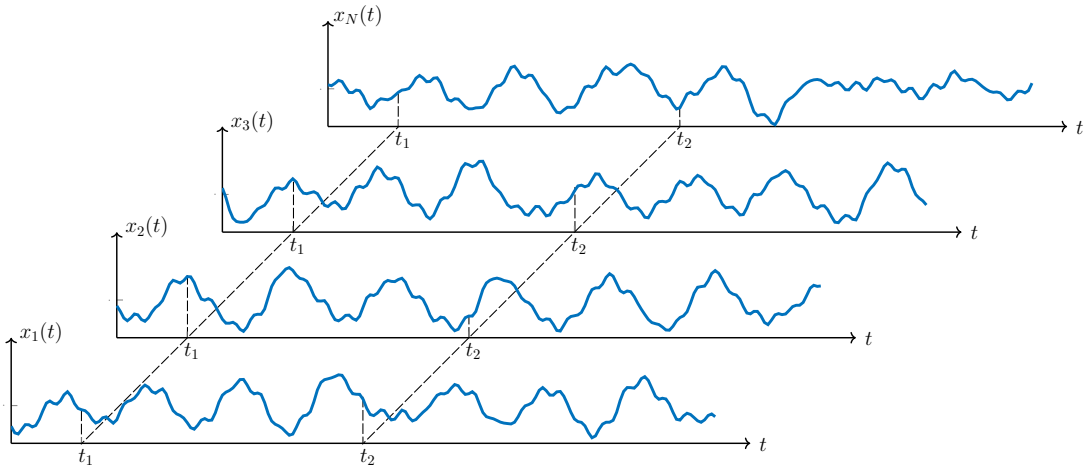


Figure 2.1: Ensemble of time history records defining a random process.

Consider a signal generated by a dynamical process, $x(t) \in \mathbb{R}$, $t \in \mathbb{R}$. Taking $x(t)$ as a function of time t, we are operating in the *time domain*. A fundamental way to *characterize* the dynamics using the data in the time domain is in terms of *probability*. Here, we assume the existence of a *probability density* $p(x)$ such that

$$\text{Prob}\{a \le x \le b\} = \int_a^b p(x)\,dx\,, \tag{2.1}$$

with a constraint that $\int_{-\infty}^{\infty} p(x)\,dx = 1$ (refer to Fig. 2.2 for illustration).

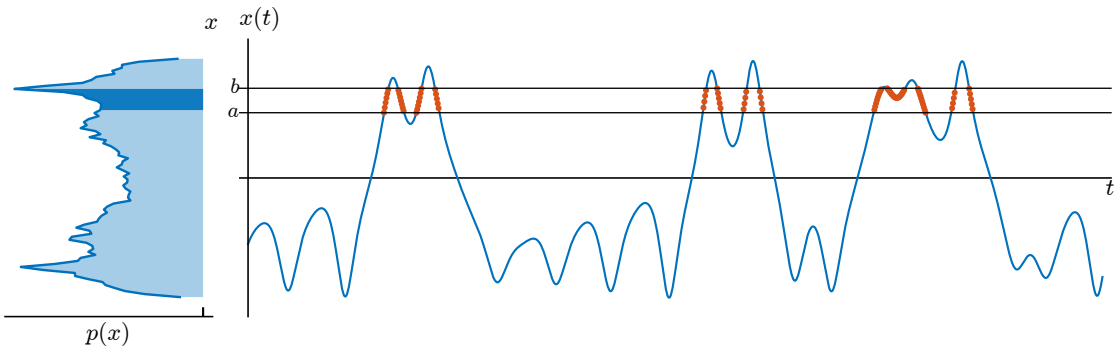


Figure 2.2: A generic signal $x(t)$ and its probability density $p(x)$, drawn sideways with the x axes aligned. The signal is the x record of the book's two-well Duffing oscillator (Section 1.4), sampled at $\Delta t = 0.1$; the first 100 time units are shown, and $p(x)$ is a histogram of $N = 40{,}000$ samples. The band $a \le x \le b$ ($a = 0.8$, $b = 1.1$) is shaded in $p(x)$ and the samples that fall in it are marked on the signal: $\text{Prob}\{a \le x \le b\}$ is the shaded area, 0.12, and equals the fraction of time the signal spends between the two lines. Script `python/mkfig_probability.py`.

In this book the *probability distribution* (Fig. 2.3) is

$$P(x) = \int_{-\infty}^{x} p(s)ds \tag{2.2}$$

so $p(x) = dP(x)/dx$. Much of the literature calls $p(x)$ a "distribution" as well, and then names $P(x)$ the *cumulative distribution*.

Thinking of $x(t)$ in terms of probability density or distribution is a *point of view* that considers it as a *random process*—but it does not imply *anything* about the actual process generating it, which may indeed be *deterministic*.

2.2 Description of Probability

Given $p(x)$, we define the *expected value* of any function of x, $f(x)$, as

$$E\left[f(x)\right] \triangleq \int_{-\infty}^{\infty} f(x)\,p(x)\,dx\,. \tag{2.3}$$

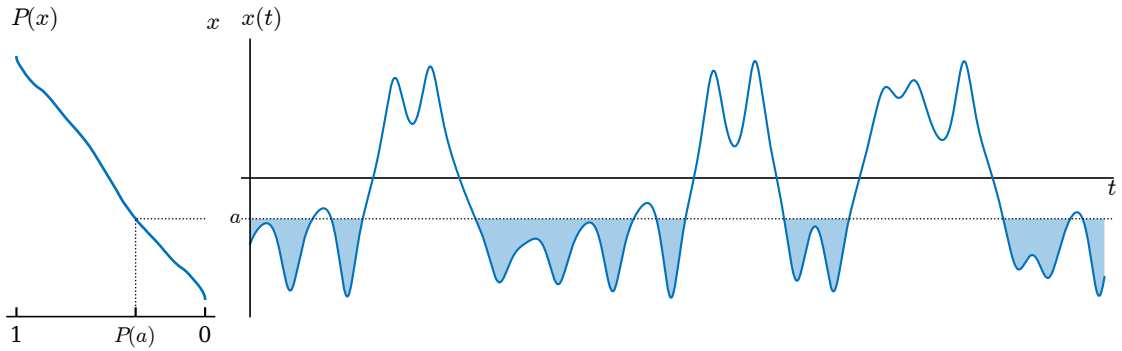


Figure 2.3: The probability distribution $P(x)$ of the signal of Fig. 2.2 (empirical distribution of the same 40,000 samples). $P(a)$ at the level $a = -0.5$ is the fraction of samples below a, 0.37, the fraction of time shaded in the signal panel. Script `python/mkfig_probability_distribution.py`.

2.2.1 Mean or First Moment of $p(x)$

In the absence of other information, $E[f(x)]$ is a good guess of the value of $f(x)$. A special case is when $f(x) \equiv x$, so for it we define the *mean* value of x or the *first moment* of $p(x)$:

$$\mu = E[x] = \int_{-\infty}^{\infty} x\, p(x)\, dx\,. \tag{2.4}$$

Median and mode, and when they equal the mean

Two other "good measures" for $f(x) = x$ are in common use:

- the *median* $x = \mu_{1/2}$, defined by $\int_{-\infty}^{\mu_{1/2}} p(x)\, dx = \int_{\mu_{1/2}}^{\infty} p(x)\, dx = \frac{1}{2}$, or
- the *mode* $x = \mu_{\max}$, defined by $p(\mu_{\max}) \geq p(x) \quad \forall x$.

For the Gaussian (Normal) density function

$$p(x) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left[-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2\right] \tag{2.5}$$

the three coincide, $\mu = \mu_{1/2} = \mu_{\max}$. In general they do not. A sufficient condition is that $p(x)$ be symmetric about $x = \mu$ and unimodal, with a finite mean. Symmetry alone makes μ a median, and the median is unique when $p(x) > 0$ near μ. Unimodality makes μ the unique mode. Symmetry without unimodality is not enough: an equal mixture of two Gaussians centred at $\pm 2\sigma$ is symmetric about zero, but its modes lie near $\pm 2\sigma$. Nor is symmetry necessary: the distribution on $\{-2, 0, 1\}$ with probabilities $(1/6, 1/2, 1/3)$ is asymmetric, yet its mean, median and mode are all zero.

2.2.2 Variance or Second Central Moment of $p(x)$

Another important expected value is for $f(x) = (x-\mu)^2$, which is called *variance* of x or the *second central moment* of $p(x)$:

$$\sigma^2 \triangleq E\left[(x-\mu)^2\right] = \int_{-\infty}^{\infty} (x-\mu)^2\, p(x)\, dx\,. \tag{2.6}$$

The square root of variance, or σ, is called the *standard deviation*. σ^2 measures the "spread" of the random process about its mean, μ.[1] Please also note that:

$$\begin{aligned}\sigma^2 &= \int_{-\infty}^{\infty} (x^2 - 2\mu x + \mu^2)\, p(x)\, dx && (2.7)\\ &= E\left[x^2 - 2\mu x + \mu^2\right] && (2.8)\\ &= E\left[x^2\right] - 2\mu E[x] + \mu^2 E[1] && (2.9)\\ &= E\left[x^2\right] - 2\mu^2 + \mu^2 && (2.10)\\ &= E\left[x^2\right] - (E[x])^2\,, && (2.11)\end{aligned}$$

where $E[1]$ and $E\left[x^2\right]$ are the zeroth and the second moments of $p(x)$, respectively.

2.2.3 Higher Order Expected Values

Other important and frequently used expected values are *skewness* and *kurtosis*. Skewness is the *standardized* third central moment, defined as

$$\alpha_3 = \gamma_1 = E\left[\frac{(x-\mu)^3}{\sigma^3}\right] = \frac{E\left[(x-\mu)^3\right]}{\left(E\left[(x-\mu)^2\right]\right)^{3/2}}\,. \tag{2.12}$$

Skewness measures *asymmetry* of the density function. Particularly, it reflects the direction and relative magnitude of a distribution's deviation from the normal distribution.

The *kurtosis* is the standardized fourth central moment, defined as

$$\alpha_4 = \beta_2 = E\left[\frac{(x-\mu)^4}{\sigma^4}\right] = \frac{E\left[(x-\mu)^4\right]}{\left(E\left[(x-\mu)^2\right]\right)^2}\,. \tag{2.13}$$

The kurtosis was understood to measures *peakedness* of the density function. However, as it is understood now, kurtosis primarily reflects the presence of outliers in the data. In addition we have *kurtosis excess* $\gamma_2 = \beta_2 - 3$. For a Gaussian distribution/density functions $\gamma_1 = \gamma_2 = 0$.

2.3 Other Frequently Used Moments

The kth *(raw) moment* of a random process $\{x(t)\}$ is defined as

$$\mu_k = E\left[x^k\right] = \int_{-\infty}^{\infty} x^k p(x) dx\,. \tag{2.14}$$

The subscripted μ_k denotes a raw moment throughout this chapter; the central moments $E[(x-\mu)^k]$ are written out explicitly, and μ without a subscript is always the mean. In addition to the mean, the most common moments in use are *mean square*:

$$\mu_2 = E\left[x^2\right] = \int_{-\infty}^{\infty} x^2 p(x) dx\,, \tag{2.15}$$

and *root mean square* (RMS):

$$\text{RMS} = \sqrt{\mu_2}\,. \tag{2.16}$$

[1] The variance is also the *covariance* of x with itself (auto-covariance in Chapter 3).

2.4 Estimators

μ, σ^2, γ_1, β_2 and other moment related quantities are *properties* of $p(x)$. When all moments exist and the moment problem is determinate, they *define* $p(x)$ through its *characteristic function* [152, Ch. 5]; the log-normal density is the standard example where two different densities share every moment, so the qualification is not empty. However, we usually do not know probability distribution $p(x)$. What we usually *do* have are samples of $x(t)$:

$$\{x(t_1), x(t_2), x(t_3), \ldots\} \equiv \{x_1, x_2, x_3, \ldots\}\,. \tag{2.17}$$

Thus the question is, given the finite sequence $\{x_i\}_{i=1}^N$, what are the best estimates of the above-described statistics?

2.4.1 Sample Size Consideration

Since only finite data records are available in practice, the limiting operations $T \to \infty$ and $M \to \infty$, where T is the individual time history's length and M is the size of an ensemble, can never be realized in practice. Therefore, the average (i.e., expected) value of data can only be estimated and never computed exactly. The resulting error in the computation of an average value due to finite sample size is of major importance in the analysis and interpretation of data.

2.4.2 Mean Estimator

Let us begin with the best estimate of the mean. Let $\widehat{\mu}$ be the estimate of μ. Then the error in the estimate is:

$$e = \sum_{i=1}^{N} (x_i - \widehat{\mu})^2\,, \tag{2.18}$$

and we seek $\widehat{\mu}$ that minimizes e:

$$e = \sum_{i=1}^{N} \left(x_i^2 - 2x_i\,\widehat{\mu} + \widehat{\mu}^2\right) = \sum_{i=1}^{N} x_i^2 - 2\widehat{\mu}\sum_{i=1}^{N} x_i + N\widehat{\mu}^2\,. \tag{2.19}$$

Thus minimization yields:

$$\frac{de}{d\widehat{\mu}} = 0 = -2\sum_{i=1}^{N} x_i + 2N\widehat{\mu}\,, \tag{2.20}$$

which gives the following

$$\widehat{\mu} = \frac{1}{N}\sum_{i=1}^{N} x_i\,, \tag{2.21}$$

where $\widehat{\mu}$ is the *sample mean* or an *estimator* of μ.

Two properties decide whether an estimator is any good, and both are used throughout the rest of the book. An estimator $\widehat{a}$ is *consistent* if it converges to a in probability,

$$\lim_{N\to\infty} \text{Prob}\{|\widehat{a} - a| > \epsilon\} = 0 \quad \forall\, \epsilon > 0\,. \tag{2.22}$$

For independent samples $\widehat{\mu} = \frac{1}{N}\sum x_i \to \mu$ in this sense as $N \to \infty$, by the Law of Large Numbers (Chebyshev's inequality). It is *unbiased* if

$$E\left[\widehat{a}\right] = a\,. \tag{2.23}$$

The sample mean is unbiased: $E[\widehat{\mu}] = \frac{1}{N}\sum E[x_i] = \frac{1}{N}(N\mu) = \mu$.

2.4.3 Central Limit Theorem

Gaussian (Normal) densities, Eq. (2.5), play an essential role in experimental measurements because of the *central limit theorem* (CLT) [152].

Central limit theorem

If $\widehat{\mu}$ is the sample mean of $\{x_i\}_{i=1}^N$, *independent and identically distributed* samples from *any* probability density with mean μ and finite variance σ^2, then the normalized random variable

$$y = \frac{\sum_{i=1}^N x_i - N\mu}{\sigma\sqrt{N}} = \frac{\widehat{\mu} - \mu}{\sigma/\sqrt{N}} \tag{2.24}$$

has probability density[a]

$$p(y) \to \mathcal{N}(0,1) \tag{2.25}$$

as $N \to \infty$, where $\mathcal{N}$ is the Normal (i.e., Gaussian) density with zero mean and unit variance.

[a] y is a random variable since $\widehat{\mu}$ is a random variable itself, which varies for different sample sets.

Written out, the two densities are

$$p(y) \sim \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{1}{2}y^2\right), \quad \text{equivalently} \quad p(\widehat{\mu}) \sim \frac{1}{\sqrt{2\pi}(\sigma/\sqrt{N})} \exp\left[-\frac{1}{2}\left(\frac{\widehat{\mu}-\mu}{\sigma/\sqrt{N}}\right)^2\right]. \tag{2.26}$$

The CLT says that the *distribution of estimates* $\widehat{\mu} \to$ Gaussian *no matter* what the original parent distribution is. The two facts $E[\widehat{\mu}] = \mu$ and $\sigma_{\widehat{\mu}} = \sigma/\sqrt{N}$ are not part of the CLT: the first holds for any identically distributed samples, the second requires that the samples be *uncorrelated*, since $\mathrm{Var}[\widehat{\mu}] = N^{-2}\sum_{i,j}\mathrm{Cov}[x_i, x_j]$. For the serially correlated records that are the subject of this book, $\mathrm{Var}[\widehat{\mu}] = \frac{\sigma^2}{N}\left[1 + 2\sum_{k=1}^{N-1}(1-\frac{k}{N})\rho_k\right]$, where ρ_k is the autocorrelation coefficient at lag k (Chapter 3); the bracket defines the effective number of independent samples N_{eff}, treated in Section 6.4. In every case $\sigma_{\widehat{\mu}} \to 0$ as $N \to \infty$ provided $\sum_k \rho_k$ converges.

2.4.4 Variance, Skewness, and Kurtosis Estimators

The obvious guess for the variance σ^2 estimator, given the logic of $\widehat{\mu}$, would be:

$$\widehat{\sigma}^2 = \frac{1}{N}\sum_{i=1}^N (x_i - \widehat{\mu})^2 = \frac{1}{N}\sum_{i=1}^N \left(x_i^2 - 2x_i\,\widehat{\mu} + \widehat{\mu}^2\right) = \frac{1}{N}\sum_{i=1}^N x_i^2 - \widehat{\mu}^2\,. \tag{2.27}$$

However,

$$\begin{aligned} E\left[\widehat{\sigma}^2\right] &= \frac{1}{N}\sum_{i=1}^N E\left[x_i^2\right] - E\left[\widehat{\mu}^2\right] \\ &= \frac{1}{N}\sum_{i=1}^N E\left[x^2\right] - E\left[\widehat{\mu}^2\right] \\ &= E\left[x^2\right] - E\left[\widehat{\mu}^2\right] \\ &= \underbrace{E\left[x^2\right] - E[x]^2}_{\sigma^2} + \underbrace{E\left[\widehat{\mu}\right]^2 - E\left[\widehat{\mu}^2\right]}_{-\sigma_{\widehat{\mu}}^2} \\ &= \sigma^2 - \sigma_{\widehat{\mu}}^2 = \sigma^2 - \frac{\sigma^2}{N} \quad \text{for uncorrelated samples.} \end{aligned} \tag{2.28}$$

where we have used the facts that $E\left[x_i^2\right] = E\left[x^2\right]$, and $E[x]^2 = E\left[\widehat{\mu}\right]^2$ since $\widehat{\mu}$ is unbiased. Therefore, $E\left[\widehat{\sigma}^2\right] = \frac{N-1}{N}\sigma^2 \neq \sigma^2$ and Eq. (2.27) is a *biased* estimator. However, it is clear that the following is *unbiased sample variance*:

$$\widehat{\sigma}^2 = \frac{1}{N-1}\sum_{i=1}^{N}(x_i - \widehat{\mu})^2 \ . \tag{2.29}$$

For correlated samples the bias is $-\mathrm{Var}[\widehat{\mu}] \approx -\sigma^2/N_{\mathrm{eff}}$ rather than $-\sigma^2/N$, and the $N-1$ correction no longer removes it (Section 6.4).

The standard *adjusted* estimators for *skewness* and *excess kurtosis* $\gamma_2 = \beta_2 - 3$ are the ratios

$$\widehat{\gamma}_1 = \frac{k_3}{k_2^{3/2}}, \quad \widehat{\gamma}_2 = \frac{k_4}{k_2^2}, \tag{2.30}$$

of the *k-statistics*

$$\begin{aligned} k_2 &= \widehat{\sigma}^2 = \frac{S_2}{N-1}, \\ k_3 &= \frac{N\,S_3}{(N-1)(N-2)}, \\ k_4 &= \frac{N(N+1)\,S_4 - 3(N-1)\,S_2^2}{(N-1)(N-2)(N-3)}, \qquad S_r = \sum_{i=1}^{N}(x_i - \widehat{\mu})^r \ . \end{aligned} \tag{2.31}$$

Only the k-statistics themselves are unbiased: $E[k_r]$ equals the r-th cumulant of $p(x)$ (the variance, the third central moment, and $\mu_4 - 3\sigma^4$ for $r = 2, 3, 4$). The ratios $\widehat{\gamma}_1$ and $\widehat{\gamma}_2$ are biased for finite N (a ratio of unbiased estimators is not unbiased), though they are consistent.

2.5 Stationarity

Intuitively, stationarity means no change over time, or properties are constant over time. The definitions below follow Bendat and Piersol [8]; Box *et al.* [12] treat the practical business of testing for it. Specifically:

Strict stationarity

A process $x(t)$ is *strictly stationary* if

$$p(x(t_1), x(t_2), \ldots, x(t_N)) = p(x(t_1 + T), x(t_2 + T), \ldots, x(t_N + T)),$$

where $\{t_1, t_2, \ldots, t_N\}$ is an arbitrary set of sample times and T is an arbitrary time shift. Here, p is a *joint* probability distribution:

$$\begin{aligned} &\mathrm{Prob}\left\{a_1 \leq x_1 \leq b_1, a_2 \leq x_2 \leq b_2, \ldots, a_N \leq x_N \leq b_N\right\} \\ &\qquad = \int_{a_1}^{b_1}\int_{a_2}^{b_2}\cdots\int_{a_N}^{b_N} p(x_1, x_2, \ldots, x_N)\, dx_N \cdots dx_2\, dx_1 \ . \end{aligned} \tag{2.32}$$

Strict stationarity is a very strong assumption. It requires, among other things, that *all* moments be constant in time. An equivalent statement of it is that $x(t)$ has the same nth-order distribution function as $x(t+T)$, for every shift T and every positive n.

Wide-sense stationarity

A process $x(t)$ is *wide sense (weakly) stationary* if:

1. $E[x(t_1)] = E[x(t_2)] = \mu \;\; \forall t_1, t_2$
2. $E[x(t_1)x(t_2)] = E[x(t_1)x(t_1+\tau)] \triangleq R(\tau)$, where $\tau = t_2 - t_1$, it is t_1 independent, and $R(\tau)$ is called the *autocorrelation*.
3. $R(0) = E[x^2] < \infty$, (i.e., $x(t)$ has a finite second moment, or bounded "power")

Wide-sense stationarity constrains the second-order statistics and nothing else. That is all *linear statistics* require, meaning the study of *linear relationships* between properties of the system at two different times.[2]

Stationarity as an invariant measure

Heuristically, a stationary signal comes from a *steady state motion*. Put more precisely, it is governed by an *invariant measure* carried on an *attractor*: the measure assigns to each region of state space the long-run fraction of time the trajectory spends there, and that fraction does not change with time. This is the point at which the statistical description of this chapter and the dynamical description of the rest of the book meet, and it is made quantitative in Chapter 11.

The difficulty in getting a handle on the concept of stationarity is that it is a *hypothesis* that is difficult to test. It is also highly context-dependent and depends on what the *sample functions* of a random process are in the probability space. In some contexts, a process can be viewed as stationary, while in others nonstationary. For practical considerations, when working with recorded time histories, we can use the following definitions:

Definition: If all averages (i.e., expected values) of interest remain constant with changes in time t_1, the data are said to be *stationary*.

Definition: If one or more of the average values of interest vary with changes in time t_1, the data are said to be *nonstationary*.

Consider the following examples

1. $x(t) = a\sin t + n(t)$, where $n(t)$ is Gaussian white noise. Examine Fig. 2.4, where 100 sample functions are plotted on top of each other. The whole function is not stationary since the mean of $x(t)$ changes with time. However, $a\sin T + n(t)$ is stationary for any fixed T.

2. $x(t) = a\sin(t+t_0)$ is stationary if the random initial time or phase t_0 is uniformly distributed on $[0, 2\pi)$ (Fig. 2.5). In other words, the moments

$$\mu_k = E\left[x(t)^k\right] = \frac{a^k}{2\pi}\int_0^{2\pi} \sin^k(t+t_0)\,dt_0 = \frac{a^k}{2\pi}\int_0^{2\pi} \sin^k\theta\,d\theta = \text{constant} \tag{2.33}$$

for all k, since the integrand is 2π-periodic in $\theta = t + t_0$. For a non-uniform phase distribution this fails: e.g., for Gaussian t_0 with variance s^2, $E[x(t)] = ae^{-s^2/2}\sin t$ depends on t.

[2] One still might use "linear statistics" even if the process generating $x(t)$ is *nonlinear* (e.g., random number generator).

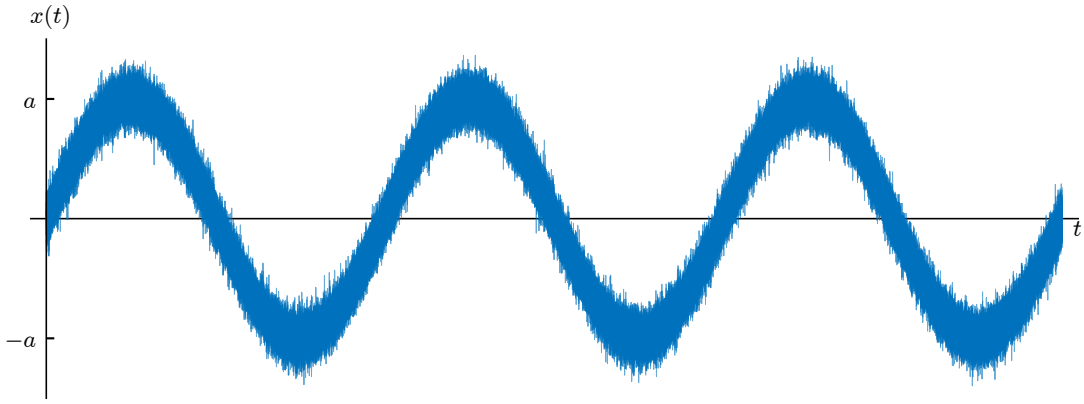


Figure 2.4: $x(t) = a\sin t + n(t)$ for 100 independent sample functions plotted on top of each other: $a = 1$, $n(t)$ Gaussian white noise of standard deviation $0.1a$, three periods at $\Delta t = 0.01$. The ensemble mean follows $a\sin t$ (at $t = \pi/2$ the mean of the 100 samples is 1.01) and changes with time. Script `python/mkfig_noisysine.py`.

3. Now lets go back to $x(t) = a\sin t + n(t)$ case. Here, averages over time scale $\Delta T_1 < 1$ period will not be stationary. However, averages over $\Delta T_2 \gg 1$ period will be (approximately) stationary (Fig. 2.6). It is hard to figure out if this is the same idea of stationarity.

4. Usually transient processes are not stationary, but if we let the transients to die out we get approximate stationarity of steady state. For example, a motion of particle in a double-well potential (refer to Fig. 2.7) with dissipation can be described by:

$$\ddot{x} + \gamma\dot{x} + x(x^2 - 1) = 0. \tag{2.34}$$

   Now let $x(t) = X(t, x_0)$, where the initial condition x_0 is randomly selected from the basin of attraction $\mathcal{B}_1$ for $x = -1$ equilibrium point. Two different sample functions (solutions for the two initial conditions $(2.2, 0)$ and $(0, -1)$, both in $\mathcal{B}_1$) are shown in Fig. 2.7(c), for $\gamma = 0.25$. Here, it is clear that the process is *not* stationary. However, we can wait until "transients die off" and then we get approximate stationarity.

Simple Stationarity Test

A simple test for the stationarity of a scalar time series $\{x_n\}_{n=1}^N$ is the examination of *moving averages* with different window lengths $\Delta t = t_w$ containing $M \ll N$ samples; refer to Fig. 2.8(a). For the data points in each sliding time window estimate the sample mean $\widehat{\mu}_i$ and its corresponding standard deviation $\sigma_{\widehat{\mu},i} = \widehat{\sigma}_i/\sqrt{M}$. If the sample means $\widehat{\mu}_i$ stay mainly within the $\pm$ one standard deviation $\sigma_{\widehat{\mu},i}$ of each other, we can say that our time series is approximately stationary. The band $\widehat{\sigma}_i/\sqrt{M}$ assumes the M samples in a window are uncorrelated; for a correlated record the correct band is $\widehat{\sigma}_i/\sqrt{M_{\text{eff}}}$ with $M_{\text{eff}} < M$ the effective number of independent samples of Section 6.4, and with the uncorrected band the test rejects stationarity for almost any oversampled record. On the other hand, if sample means show a significant variation in time in comparison to our time series, it is not stationary. For an illustration of this concept, please refer to Fig. 2.8(b) and (c). Please note that our results will depend on the time window t_w as illustrated in the third example of the previous section.

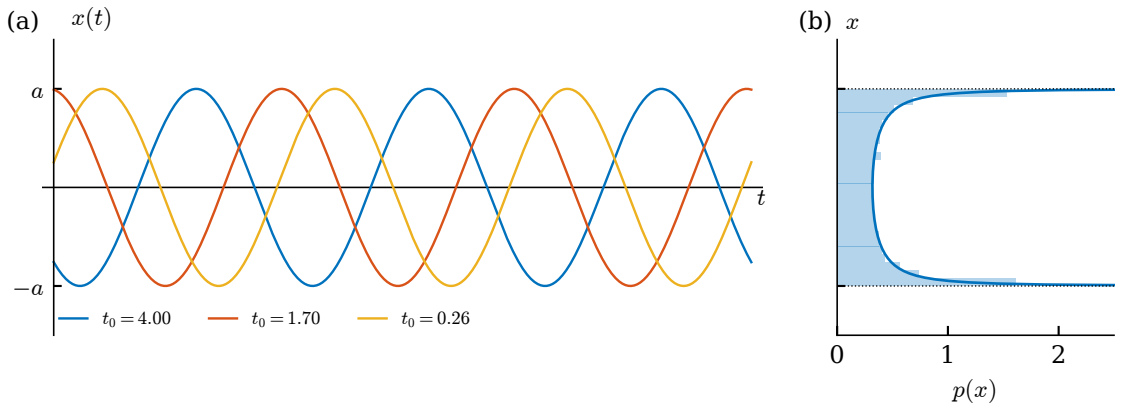


Figure 2.5: (a) $x(t) = a\sin(t+t_0)$ for three values of the random phase t_0 drawn uniformly on $[0, 2\pi)$ ($t_0 = 4.00$, 1.70, 0.26; $a = 1$). (b) The density of $x(t)$ at any fixed t when t_0 is uniform, the arcsine density $p(x) = 1/(\pi\sqrt{a^2 - x^2})$ on $|x| < a$ (curve), compared with a histogram of $x(0)$ over 20,000 random phases (bars); it does not depend on t. Script `python/mkfig_noisysine2.py`.

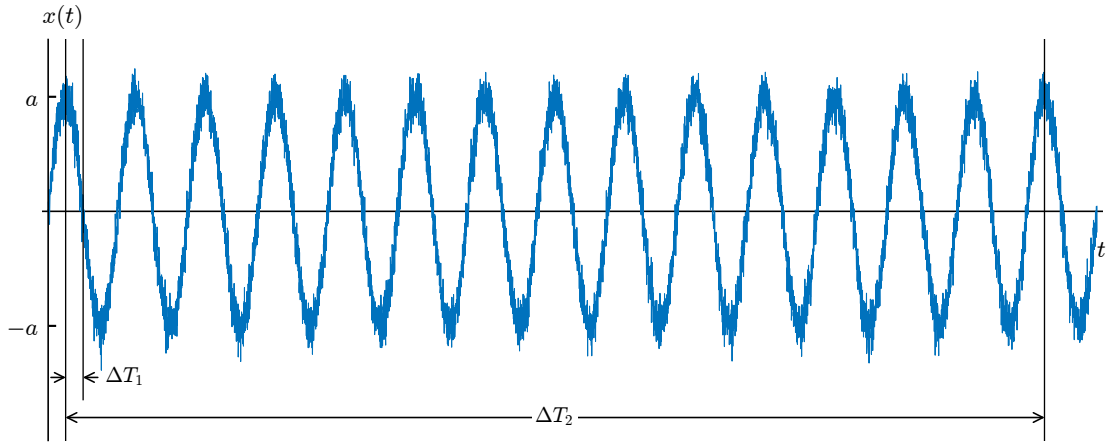


Figure 2.6: Two averaging time scales for one sample function of $x(t) = a\sin t + n(t)$ ($a = 1$, noise standard deviation $0.1a$, 15 periods). Over $\Delta T_1 = \pi/2$, a quarter period, the time average is 0.62 and depends on where the window sits; over $\Delta T_2 = 28\pi$, fourteen periods, it is 0.0004. Script `python/mkfig_noisysine3.py`.

2.6 Ergodicity

The general "signal" $x(t)$ is a *sample function* of a *random process*. Thus, every trial will generate a new function. In general, we have considered two types of averaging: *ensemble averaging* and *time averaging*. That the two coincide for almost every sample function of a suitable process is the content of Birkhoff's ergodic theorem [11], which is what licenses the practice, universal in experimental work, of estimating an ensemble property from a single record.

Ensemble average is the average of a quantity over all realizations (or trials) of a random variable. For example, if $x(t)$ is a Gaussian random variable, $p(x(t))$ for each fixed t is Gaussian. If it is a stationary random process, then $p(x(t_1)) = p(x(t_2))$. Now let us look at the mean value of our

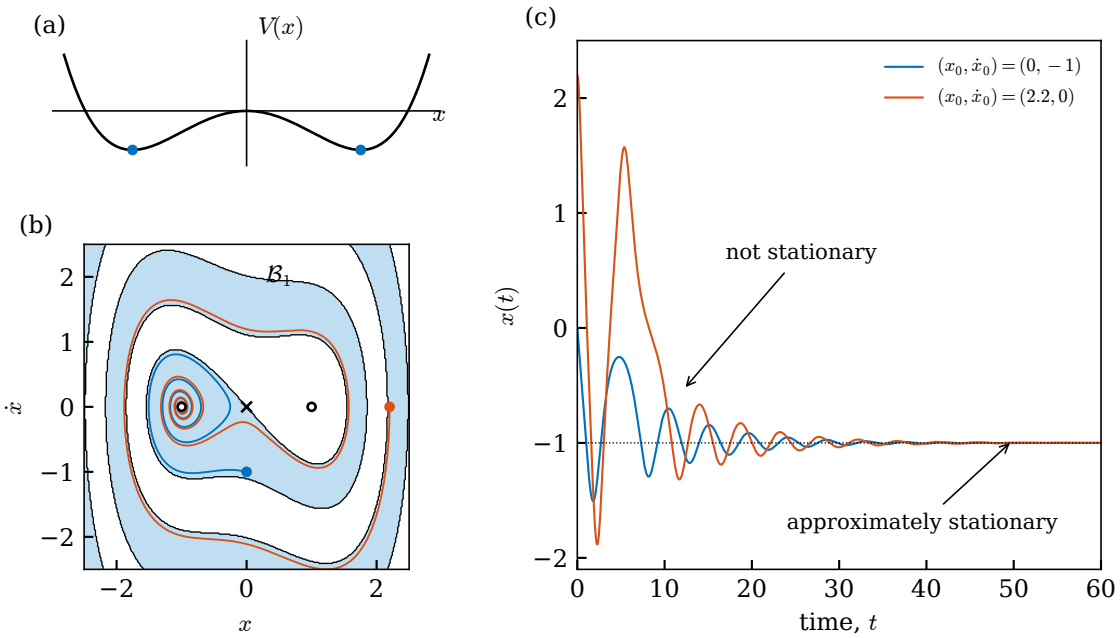


Figure 2.7: The unforced two-well oscillator $\ddot{x} + \gamma\dot{x} + x(x^2 - 1) = 0$ with $\gamma = 0.25$. (a) The potential $V(x) = -\frac{x^2}{2} + \frac{x^4}{4}$ with its two minima. (b) Basins of attraction in the $(x, \dot{x})$ plane: a 301×301 grid of initial conditions on $[-2.5, 2.5]^2$ integrated to $t = 100$ and classified by the well reached; the basin $\mathcal{B}_1$ of the equilibrium $x = -1$ is shaded (it covers half the square), with the two trajectories of (c) drawn in it. (c) Two sample functions started in $\mathcal{B}_1$, from $(x_0, \dot{x}_0) = (0, -1)$ and $(2.2, 0)$: both settle to within 0.05 of $x = -1$ after $t \approx 22$ and 28. Script `python/mkfig_twowellfig.py`.

random process for some fixed time t_j:

$$\widehat{\mu}_j \triangleq \frac{1}{N_T} \sum_{i=1}^{N_T} x_i(t_j)\,, \tag{2.35}$$

where t_j is fixed, and we have total of N_T trials of our random process. In other words the $\widehat{\mu}_j$ is an estimate for

$$\mu_j = E\left[x(t_j)\right] = \int_{-\infty}^{\infty} x(t_j)\, p(x(t_j))\, dx(t_j)\,. \tag{2.36}$$

Thus, the *ensemble average* is the expected value (or an estimate of) a random process for a particular fixed time. Now, a good question is: does $\mu_j = \mu_k$ for $j \neq k$? The answer is yes for a stationary process; the estimates $\widehat{\mu}_j$ and $\widehat{\mu}_k$ then differ only by sampling error.

Time average is the average using a *single* sample function over some time interval. For example, given any function f,

$$\bar{f} = \lim_{T\to\infty} \frac{1}{T} \int_{-T/2}^{T/2} f(x(t))\, dt \tag{2.37}$$

In particular, for the time mean value estimates:

$$\bar{\mu} = \frac{1}{T} \int_0^T x(t)dt, \quad \text{or} \quad \bar{\mu} = \frac{1}{N_s} \sum_{i=1}^{N_s} x(t_i)\,, \tag{2.38}$$

where N_s is the number of time samples of $x(t)$.

Definition

We say that a random process is *ergodic* if the ensemble averages are equivalent to the time averages. More precisely, given f the stationary process is ergodic if $\widehat{\mu}_f = \bar{\mu}_f$, or

$$\int_{-\infty}^{\infty} f(x(t))\, p(x(t))\, dx(t) = \lim_{T\to\infty} \frac{1}{T} \int_{-T/2}^{T/2} f(x(t))\, dt\,. \tag{2.39}$$

Please note that in the above equation, on the left-hand side, the t is *fixed*, and we have an ensemble average, and on the right-hand side, we have a time average of a single sample function (see Fig. 2.9).

Random processes can be ergodic for some f and not for others. For example, consider the $x(t) = A\sin(t + B)$ random process, where the phase B is uniformly distributed on $[0, 2\pi)$ and the amplitude A is an independent random variable with $E[A^2] < \infty$. The process is stationary (example 2 above). Its ensemble mean is $E[A]\,E[\sin(t+B)] = 0$ and the time average of every sample function is also 0, so the process is *ergodic in the mean* ($f(x) = x$). Its ensemble power is $E[A^2]/2$, while the time average of x^2 for one sample function is $A^2/2$, which depends on the realization of A: the process is not *ergodic in the power* ($f(x) = x^2$). Bendat and Piersol [8] give the practitioner's rule that stationary data from a physical experiment are usually ergodic, so that the average values calculated on the ensemble at time t_1 equal the corresponding average value computed over time from a single time history; the example shows that the rule fails whenever a random parameter is frozen within each realization.

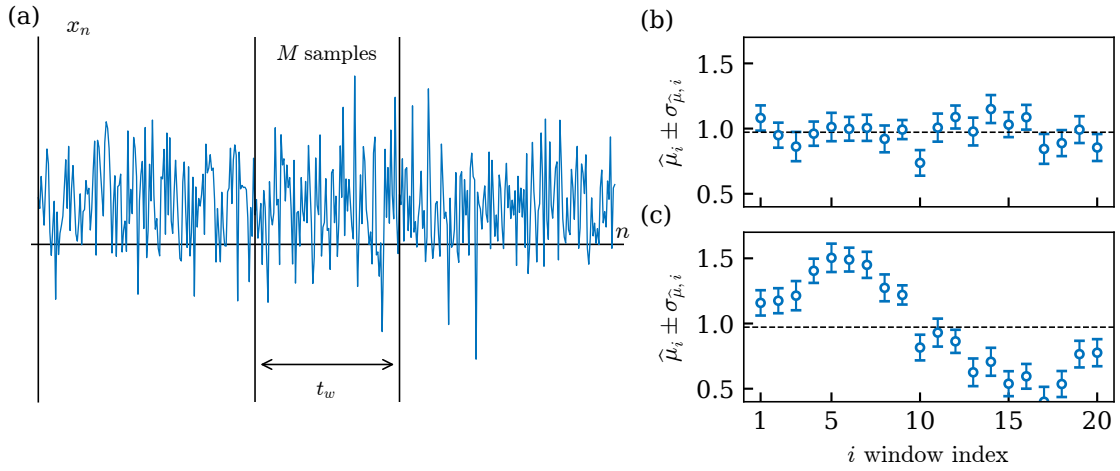


Figure 2.8: The simple stationarity test. (a) A record of $N = 2000$ independent Gaussian samples with mean 1 and unit variance (first 400 shown) and one window of $M = 100$ samples, length t_w. (b) The 20 non-overlapping window means $\widehat{\mu}_i$ with bars $\pm\widehat{\sigma}_i/\sqrt{M} \approx \pm 0.1$ and the grand mean (dashed): 35% of the means fall outside the band, close to the 32% expected for independent samples, and the record is judged stationary. (c) The same test on the record with the slow trend $0.5\sin(2\pi n/N)$ added: 95% of the means fall outside the band and their spread (0.36) is far larger than the bars. Script `python/mkfig_stationaritytest.py`.

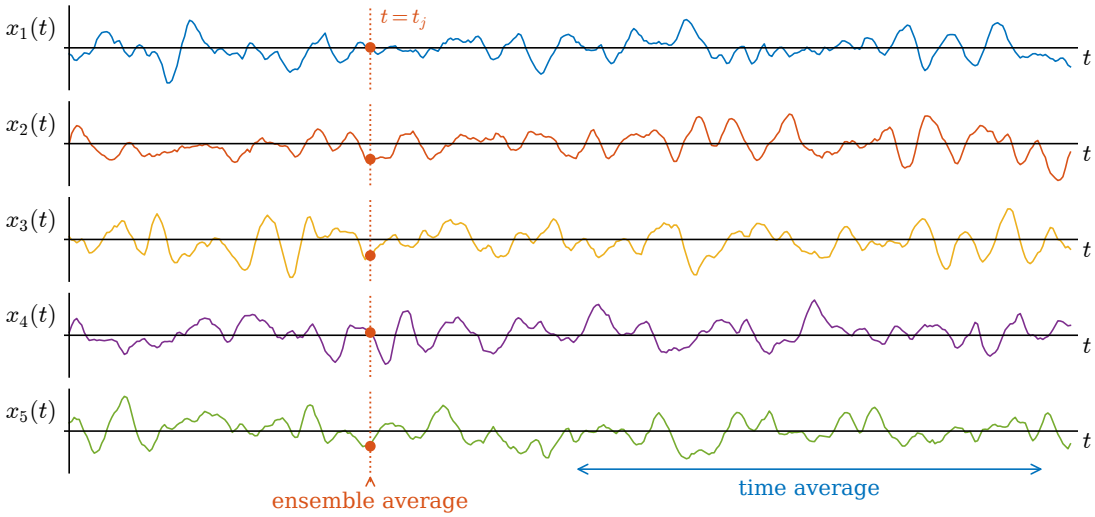


Figure 2.9: Ensemble and time averages. Five sample functions of one stationary, ergodic process, the AR(2) process of Chapter 3 with $\rho = 0.2$ (400 samples each, independent noise sequences). The ensemble average is taken across the sample functions at the fixed time t_j (dotted line); the time average is taken along one sample function over the bracketed interval. For an ergodic process the two converge to the same value, here the process mean 0; with only five records the ensemble estimate at t_j is -0.73 against a time average of -0.01 over 400 samples. Script `python/mkfig_ergo.py`.

Problems

Problem 2.1

Computational

In MATLAB or Python, generate M trials of random processes with the forms

1. $x(t) = \cos(t) + n(t)$;
2. $x(t) = \cos(t + \theta) + n(t)$; and
3. $x(t) = A\cos(t + \theta) + n(t)$,

where $0 \le t \le T$ for some sample time T, $n(t)$ is a normally-distributed (Gaussian) random process with zero mean and variance σ^2, θ is a uniformly-distributed constant random variable on $[0; 2\pi]$, and A is a normally-distributed constant with zero mean and unit variance. In each case, generate uniformly sampled time series $\{x_1; x_2; \ldots; x_N\}$ where $x_i = x(t_i)$.

Use the time series generated above to examine the *stationarity* and *ergodicity* of the different processes. Experiment with different values of M, N, T, and σ to ensure good results. Discuss your findings.

Note: Consider carefully your choice of sampling time interval and the length of the time series (relationship between N and T).

Problem 2.2

Computational

Generate a long time series of the random process (1) in Problem 2.1. Estimate the mean of the process many times from different subsets of the time series and show that the distribution of $\widehat{\mu}_x$ is approximately Gaussian with mean and variance, as predicted by the Central Limit Theorem.

Note: How you make the samples used for each estimate can have a significant impact on your results!

Problem 2.3

Core

The variance of the sample mean for correlated samples.

1. Starting from $\mathrm{Var}[\widehat{\mu}] = N^{-2}\sum_{i=1}^{N}\sum_{j=1}^{N}\mathrm{Cov}[x_i, x_j]$ for a stationary sequence with variance σ^2 and autocorrelation coefficient ρ_k at lag k, count the pairs at each lag and derive the bracket formula $\mathrm{Var}[\widehat{\mu}] = \frac{\sigma^2}{N}\left[1 + 2\sum_{k=1}^{N-1}(1 - \frac{k}{N})\rho_k\right]$ quoted after the central limit theorem.

2. For the first-order autoregression $x_{n+1} = ax_n + \eta_n$ with $|a| < 1$ and unit-variance white η_n, show that $\sigma^2 = 1/(1 - a^2)$ and $\rho_k = a^k$. Evaluate the bracket for $N \to \infty$ and show that the effective number of independent samples is $N_{\mathrm{eff}} = N(1 - a)/(1 + a)$.

3. Simulate 4000 independent records of $N = 2000$ samples with $a = 0.8$ (discard a few hundred initial samples so the process is stationary). Compare the variance of the 4000 sample means with σ^2/N and with the formula of part (1). You should find the formula correct and σ^2/N nine times too small.

4. Repeat part (3) for the unbiased variance estimator Eq. (2.29): is it still unbiased when the samples are correlated? Relate the bias you find to $-\mathrm{Var}[\widehat{\mu}]$.

Problem 2.4

Computational

Moments of the book's records (Section 1.4). Generate the Lorenz x and z records, the two-well Duffing x record, and the Hénon x record, each of at least 40,000 samples after the transient.

1. Estimate the mean, the unbiased variance, and the adjusted skewness $\widehat{\gamma}_1$ and excess kurtosis $\widehat{\gamma}_2$ from the k-statistics. Tabulate them.

2. The Lorenz equations are invariant under $(x, y, z) \to (-x, -y, z)$ and the Duffing equation under $(x, \dot{x}, t) \to (-x, -\dot{x}, t + \pi)$. What do these symmetries predict for the skewness of x in each case, and does the estimate agree to within its sampling scatter (estimate that scatter from ten independent records)? Is the skewness of the Hénon x record consistent with zero?

3. All four records have negative excess kurtosis, unlike the Gaussian. Plot their histograms and explain the sign from the shape of each density; for the Duffing record, relate the shape to the two wells.

4. Split each record into blocks of 500 samples and estimate the variance from each block with the $1/N$ estimator Eq. (2.27). Is the average over blocks biased low by the factor $(N-1)/N$ predicted for independent samples, or by more? Explain using Problem 2.3.

Problem 2.5

Computational

The simple stationarity test on correlated data.

1. Generate one record of $N = 40{,}000$ samples of the autoregression of Problem 2.3 with $a = 0.95$, which is stationary by construction. Apply the moving-average test with non-overlapping windows of $M = 200$ samples, using the band $\widehat{\sigma}_i/\sqrt{M}$ around the grand mean. What fraction of the window means falls outside the band? You should find the great majority, and the test rejects a stationary record.

2. Recompute the band with $M_{\text{eff}} = M(1-a)/(1+a)$ from Problem 2.3. What fraction now falls outside? Explain why even this is more than the 32% expected for a $\pm 1\sigma$ band (consider how many independent samples a window of 200 actually contains, and what the window's own $\widehat{\sigma}_i$ estimates).

3. Now generate a genuinely nonstationary record by adding the slow trend $0.5 \sin(2\pi i/N)$ to the same process. Can the test with the corrected band detect it? Repeat with trend amplitude 2. State what the test can and cannot decide when the correlation time is a substantial fraction of the window.

4. Lengthen the window to $M = 5000$ and repeat parts (1)–(3). Which choice of M would you use for this record, and what limits how large M can be?

Problem 2.6

needs Ch. 3 Experimental

Apply the tests of this chapter to the two-well oscillator record of Section 1.4.1.

1. Using the sampling rate and forcing frequency given with the record, choose a window length M equal to about fifty forcing periods. Compute the window means and the window standard deviations $\widehat{\sigma}_i$ and plot both against window index. Do the means stay within $\pm\widehat{\sigma}_i/\sqrt{M}$ of each

other? Estimate M_{eff} from the sample autocorrelation of Chapter 3, or, more simply, from the number of well crossings per window, and redo the comparison with the corrected band.

2. The strain histogram of Problem 1.6 is bimodal. Locate the two peaks and the trough between them, and define the well indicator $b_i = 1$ when s_i lies above the trough and 0 below. Compute the fraction of time spent in the upper well in each window. Is this fraction stationary across the record, by the same test?

3. The record is the steady-state response of a driven beam. List the physical reasons its statistics could nevertheless drift over the record (amplifier gain, temperature, the magnets' field), and state which of the window statistics of parts (1)–(2) would show each.

4. Is the record ergodic in the sense of this chapter, and can this be decided from one record at all? State what additional records would be needed, and what property of the two-well system would make two records started in different wells give the same time averages.

3
Correlation and Auto-Correlation

The correlation functions developed in this chapter are standard; Bendat and Piersol [8] and Papoulis and Pillai [152] give fuller accounts, and Jenkins and Watts [101] treat the estimation problem in detail.

Given two random processes $x(t)$ and $y(t)$

$$E\left[x(t_1)y(t_2)\right] = \int_{-\infty}^{+\infty}\int_{-\infty}^{+\infty} x(t_1)y(t_2)p(x(t_1),y(t_2))\,dx(t_1)dy(t_2) \tag{3.1}$$

is called the *cross-correlation* $R_{xy}(t_1,t_2)$ between $x(t_1)$ and $y(t_2)$, $\forall\ t_1,t_2$ fixed.[1] If the generating processes are *stationary* and $t_1 \equiv t$, $t_2 \equiv t+\tau$

$$\begin{aligned} E\left[x(t)y(t+\tau)\right] &= R_{xy}(\tau) \\ &= E\left[x(t-\tau)y(t)\right] \\ &= E\left[y(t)x(t-\tau)\right] \\ &= R_{yx}(-\tau)\,. \end{aligned} \tag{3.2}$$

For a nonstationary pair the correlation depends on both times, and the corresponding identity is $R_{xy}(t_1,t_2) = R_{yx}(t_2,t_1)$. If the two processes are statistically independent, then $R_{xy}(\tau) = \mu_x\mu_y$ for every τ, and the cross-covariance defined below vanishes identically, $C_{xy}(\tau) = 0$.

Given an ensemble of N sample functions $\{x_i(t)\}_{i=1}^N$ and $\{y_i(t+\tau)\}_{i=1}^N$, we estimate the cross correlation $R_{xy}(\tau)$ as

$$\widehat{R}_{xy}(\tau,t) = \frac{1}{N}\sum_{i=1}^{N} x_i(t)y_i(t+\tau)\,. \tag{3.3}$$

Here N counts realizations and i labels them. If the generating processes are stationary and ergodic (Chapter 2), the ensemble average can be replaced by a time average over a single pair of records; from here on N is the record length and i the time index, so that

$$\widehat{R}_{xy}(\tau) = \frac{1}{N}\sum_{i=1}^{N} x_i y_i\,, \tag{3.4}$$

where x_i and y_i are assumed to be sampled some fixed τ time apart. If we subtract off the means from these variables, then

$$\begin{aligned} C_{xy}(\tau,t) &= E\left[(x(t)-\mu_x)(y(t+\tau)-\mu_y)\right] \\ &= E\left[(x(t)y(t+\tau) - \mu_x y(t+\tau) - \mu_y x(t) + \mu_x\mu_y)\right] \\ &= R_{xy}(\tau,t) - \mu_x\mu_y \end{aligned} \tag{3.5}$$

[1] Cross correlation is a function of two variables and represents a surface.

is called the *cross-covariance*, which—for stationary processes—can be estimated from the above sample sets as:

$$\begin{aligned}\widehat{C}_{xy}(\tau) &= \frac{1}{N}\sum_{i=1}^{N}(x_i - \widehat{\mu}_x)(y_i - \widehat{\mu}_y) \\ &= \widehat{R}_{xy}(\tau) - \widehat{\mu}_x\widehat{\mu}_y\,.\end{aligned} \tag{3.6}$$

where, again, x_i and y_i are assumed to be sampled some fixed τ time apart. If we normalize cross-covariance C_{xy} by σ_x and σ_y we get the *correlation coefficient*:[2]

$$\rho_{xy}(\tau) = \frac{C_{xy}(\tau)}{\sigma_x\sigma_y}\,, \tag{3.7}$$

which means that $-1 \le \rho_{xy} \le 1$. We say that x and y are (linearly) uncorrelated if $\rho_{xy} \to 0$ (see Fig. 3.1(b)). Perfectly correlated processes, $y = ax + b$ with $a > 0$, have $\rho_{xy} = 1$ (see Fig. 3.1(a)); $a < 0$ gives $\rho_{xy} = -1$. For mean zero processes $R_{xy} = C_{xy}$.

Special case of the cross-correlation is when $x = y$ and R_{xx} is called an *autocorrelation*. For a stationary process

$$\begin{aligned}R_{xx}(\tau) &= E[x(t)x(t+\tau)] \\ &= E[x(t-\tau)x(t)] \\ &= E[x(t)x(t-\tau)] \\ &= R_{xx}(-\tau)\,.\end{aligned} \tag{3.8}$$

Thus, the autocorrelation is always symmetric. The same holds for the autocovariance $C_{xx}(\tau) = R_{xx}(\tau) - \mu_x^2$ and for the autocorrelation coefficient $\rho_{xx}(\tau) = C_{xx}(\tau)/\sigma_x^2$, with $\rho_{xx}(0) = 1$. For a purely random (white) signal, $C_{xx}(\tau) = 0$ if $\tau \neq 0$, while $R_{xx}(\tau) = \mu_x^2$ there; the zero crossings of R_{xx} carry no information unless the mean has been removed. For general random signals we see autocorrelation coefficients similar to the ones shown in Fig. 3.2.

Correlation time

The *correlation time* τ_c of a stationary process is the lag over which $\rho_{xx}(\tau)$ decays. Two definitions are in use: the $1/e$ *time*, the smallest τ with $|\rho_{xx}(\tau)| = 1/e$, and the *integral time scale*

$$\tau_c = \int_0^\infty \rho_{xx}(\tau)\,d\tau\,, \tag{3.9}$$

which exists when ρ_{xx} is integrable. For an exponential decay $\rho_{xx}(\tau) = e^{-|\tau|/\tau_0}$ both give τ_0. For the discrete-time first-order autoregression $x_{n+1} = ax_n + \eta_n$, $\rho_k = a^k$ and the integral time scale is $\sum_{k\ge1}\rho_k = a/(1-a)$ samples (four samples for $a = 0.8$), close to the $1/e$ lag $-1/\ln a = 4.5$.

For $\tau \gg \tau_c$, $x(t)$ and $x(t+\tau)$ are nearly *uncorrelated*; they need not be independent, since ρ_{xx} measures only the linear dependence, and the whole of Part II concerns dependence that ρ_{xx} cannot see. For the AR records in Fig. 3.2 ρ_{xx} decays through repeated sign changes; its first zero crossing is a quarter of the twenty-sample period (lags 6–7 for $\rho \le 0.2$), just beyond which $x(t)$ and $x(t+\tau)$ are strongly *anti*-correlated. The chaotic records behave differently: the Duffing x record, dominated by the slow well-to-well motion rather than by the forcing period of 63 samples, first crosses zero only at lag 66 and is still at -0.2 at lag 100, and the Lorenz x record decays to about 0.1 within 30 lags and has not crossed zero by lag 100. That zero crossing is nevertheless a useful

[2] The terminology use here and in the literature is a little confusing: the cross-correlation of the *centered random variables* is called the cross-covariance, and the *normalized cross-covariance* is called the *correlation coefficient*.

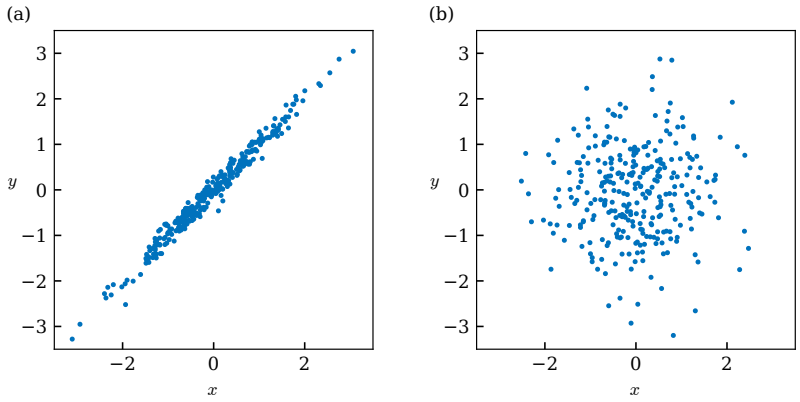


Figure 3.1: Scatter plots of $N = 300$ pairs. (a) Highly correlated data, $y = x + 0.15\,e$ with x and e independent standard Gaussian: $\widehat{\rho}_{xy} = 0.99$. (b) Uncorrelated data, x and y independent standard Gaussian: $\widehat{\rho}_{xy} = 0.03$. Script `python/mkfig_corrs.py`.

delay heuristic for the reconstructions of Chapters 8 and 9, where it is used as such and not as a correlation time. The integral time scale also fixes the effective number of independent samples in a record, $N/(1 + 2\sum_{k\geq 1} \rho_k)$, Section 6.4.

In many cases we deal with a collection of "signals" $x_1(t), x_2(t), \ldots x_n(t)$, for which they can be collected in a vector-valued signal $\boldsymbol{x}(t) \in \mathbb{R}^n$. Now if we are given a vector-valued time series $\{\boldsymbol{x}^{(k)}\}_{k=1}^N$, where $\boldsymbol{x}^{(k)} = \boldsymbol{x}(t_k)$, it can be arranged into a coordinate matrix $\boldsymbol{X} \in \mathbb{R}^{N\times n}$ as:

$$\boldsymbol{X} = \begin{bmatrix} x_1^{(1)} & x_2^{(1)} & \cdots & x_n^{(1)} \\ x_1^{(2)} & x_2^{(2)} & \cdots & x_n^{(2)} \\ \vdots & \vdots & \vdots & \vdots \\ x_1^{(N)} & x_2^{(N)} & \cdots & x_n^{(N)} \end{bmatrix} \tag{3.10}$$

Now we can define a *cross-correlation matrix* for $\boldsymbol{X}$ (assuming $\tau = 0$) as

$$\widehat{\boldsymbol{R}} = \frac{1}{N}\boldsymbol{X}^{\mathrm{T}}\boldsymbol{X}, \quad \text{or} \quad \widehat{R}_{ij} = \frac{1}{N}\sum_{k=1}^{N} x_i^{(k)} x_j^{(k)}\,. \tag{3.11}$$

Similarly, the corresponding *cross-covariance* matrix is

$$\widehat{\boldsymbol{C}} = \frac{1}{N-1}\bar{\boldsymbol{X}}^{\mathrm{T}}\bar{\boldsymbol{X}}, \quad \text{or} \quad \widehat{C}_{ij} = \frac{1}{N-1}\sum_{k=1}^{N}(x_i^{(k)} - \widehat{\mu}_i)(x_j^{(k)} - \widehat{\mu}_j)\,, \tag{3.12}$$

where $\widehat{\mu}_i = \frac{1}{N}\sum_{k=1}^{N} x_i^{(k)}$ is the full-record mean of the ith column and $\bar{\boldsymbol{X}} = \boldsymbol{X} - \boldsymbol{1}\widehat{\boldsymbol{\mu}}^{\mathrm{T}}$ has zero-mean columns. Its diagonal entries $\widehat{C}_{ii} = \widehat{\sigma}_i^2$ are the unbiased sample variances of Chapter 2.

Now, if we have a two-dimensional random variable with scalar components x and y, then our cross-covariance matrix will be two dimensional:

$$\boldsymbol{C} \cong \begin{bmatrix} \sigma_x^2 & C_{xy} \\ C_{yx} & \sigma_y^2 \end{bmatrix}, \tag{3.13}$$

and if x and y are (linearly) uncorrelated, we have

$$\boldsymbol{C} \cong \begin{bmatrix} \sigma_x^2 & 0 \\ 0 & \sigma_y^2 \end{bmatrix}. \tag{3.14}$$

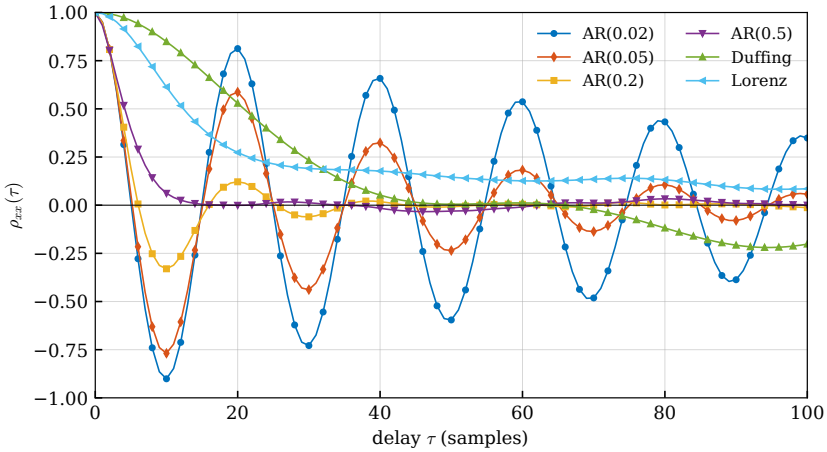


Figure 3.2: Correlation coefficient $\rho_{xx}(\tau)$ versus delay τ (in samples; $1/(N-m)$ normalization, records of $N = 40{,}000$ samples) for the colored random processes $x_{n+1} = (2-(\pi/10)^2-\rho)x_n + (\rho-1)x_{n-1}+\eta_n$, where η_n is unit-variance Gaussian white noise, $x_1 = x_2 = 0$, and $\rho = 0.02, 0.05, 0.2, 0.5$; each case is labeled AR(ρ). In addition, the autocorrelations of the x time series of the Duffing oscillator $\ddot{x} + 0.2\,\dot{x} - x + x^3 = 0.33\cos t$ from $(x, \dot{x}) = (-0.6973, 0.0729)$, sampled at 0.1, and of the Lorenz system $\dot{x} = 10\,(y-x)\,,\ \dot{y} = x\,(28-z) - y,\ \dot{z} = xy - \frac{8}{3}z$ from $(5, -5, 20)$, sampled at 0.02, are plotted (the records of Problem 3.2). Script `python/mkfig_autocorrs.py`.

If the time records of vector-valued time histories are long, we can also estimate cross-correlation between $x_i^{(k)}$ and $x_j^{(k+m)}$, where m is a delay or the number of samples separating these two components in time. To accomplish this, we first define:

$$\widehat{\boldsymbol{R}}(m) = \frac{1}{N-m}\boldsymbol{X}_0^{\mathrm{T}}\boldsymbol{X}_m, \quad \text{or} \quad \widehat{R}_{ij}(m) = \frac{1}{N-m}\sum_{k=1}^{N-m} x_i^{(k)} x_j^{(k+m)}\,, \tag{3.15}$$

where

$$\boldsymbol{X}_0 = \begin{bmatrix} x_1^{(1)} & x_2^{(1)} & \cdots & x_n^{(1)} \\ x_1^{(2)} & x_2^{(2)} & \cdots & x_n^{(2)} \\ \vdots & \vdots & \vdots & \vdots \\ x_1^{(N-m)} & x_2^{(N-m)} & \cdots & x_n^{(N-m)} \end{bmatrix}, \qquad \boldsymbol{X}_m = \begin{bmatrix} x_1^{(m+1)} & x_2^{(m+1)} & \cdots & x_n^{(m+1)} \\ x_1^{(m+2)} & x_2^{(m+2)} & \cdots & x_n^{(m+2)} \\ \vdots & \vdots & \vdots & \vdots \\ x_1^{(N)} & x_2^{(N)} & \cdots & x_n^{(N)} \end{bmatrix}. \tag{3.16}$$

Obviously, we cannot let $m \geq N$. Generally, $N - m$ should be large enough that your estimates make sense. The corresponding cross-covariance matrix as a function of delay m can be written as

$$\begin{aligned} \widehat{\boldsymbol{C}}(m) &= \frac{1}{N-m}\bar{\boldsymbol{X}}_0^{\mathrm{T}}\bar{\boldsymbol{X}}_m, \\ \widehat{C}_{ij}(m) &= \frac{1}{N-m}\sum_{k=1}^{N-m}(x_i^{(k)} - \widehat{\mu}_i)(x_j^{(k+m)} - \widehat{\mu}_j), \end{aligned} \tag{3.17}$$

where $\bar{\boldsymbol{X}}_0$ and $\bar{\boldsymbol{X}}_m$ are the blocks of $\bar{\boldsymbol{X}}$, centered with the full-record means $\widehat{\mu}_i$; block means would remove part of the lagged covariance.

Two normalizations of the lagged covariance

Dividing by $N-m$ uses the actual number of products at lag m, and for known (not estimated) means the estimate is unbiased; with estimated means it is not exactly unbiased, and the $N-1$ of Chapter 2 is not the right correction either. Its drawback appears at large m, where few products are averaged and the estimate is erratic. The alternative divides every lag by N,

$$\widehat{C}_{ij}^{(N)}(m) = \frac{1}{N}\sum_{k=1}^{N-m}(x_i^{(k)} - \widehat{\mu}_i)(x_j^{(k+m)} - \widehat{\mu}_j) = \frac{N-m}{N}\,\widehat{C}_{ij}(m)\,, \tag{3.18}$$

which is biased toward zero by the triangular factor $1-m/N$ but has a property the first lacks: the sequence $\{\widehat{C}_{xx}^{(N)}(m)\}$ is *positive semi-definite*, because it is the autocorrelation of the finite, zero-padded record, so its Fourier transform, the periodogram of Chapter 6, is non-negative. The $1/(N-m)$ sequence is not, and its transform can be negative at some frequencies (in a check with $N=30$ Gaussian samples the Toeplitz matrix built from it was indefinite in every one of 200 trials). This is why the spectral estimators of Chapters 4–6 are built on the $1/N$ sequence [101], while the $1/(N-m)$ form is preferred when individual lags are the object of interest.

Problems

Problem 3.1

Core

[Linear Prediction] Suppose we wish to predict the values of a variable Y by observing (recording) the values of another variable X. In particular, the available data (see Fig. 3.3) suggests that a good prediction model for Y is the linear function

$$Y_p \triangleq \alpha X + \beta.$$

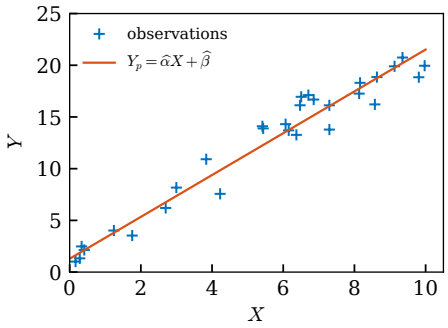


Figure 3.3: Pairwise observations on (X, Y) constitute a scatter diagram, and a straight line approximates the relationship between X and Y. Synthetic data: 30 points with X uniform on $[0, 10]$ and $Y = 2X + 1 + 1.5\,e$, e standard Gaussian; the least-squares line has $\widehat{\alpha} = 2.02$, $\widehat{\beta} = 1.30$, and $\widehat{\rho}_{XY} = 0.98$. Script `python/mkfig_Problem31.py`.

Now although Y can be related to X, the values it takes on may be influenced by other sources that do not affect X. Thus, in general, $|\rho_{XY}| \neq 1$ and we expect that there will be an error between the predicted value of Y, that is, Y_p and the value that Y actually assumes. Our task is to adjust the coefficients α and β in order to minimize the mean-square error

$$\epsilon \triangleq E[(Y - Y_p)^2].$$

In other words, to come up with *optimum linear prediction* or *linear regression*. Do the following:

- Find expressions that need to be satisfied to minimize ϵ with respect to α and β.
- Solve this expressions for estimated $\hat{\alpha}$ and $\hat{\beta}$.
- Write the expression for the best predictor and the corresponding $\epsilon_{\min}$.
- What happens if $\rho_{XY} = 0$? In this case, what is the best predictor for Y, and how does it relate to X?
- How can we use the correlation coefficient to establish the relationship between two variables?

Problem 3.2

Computational

Generate the data for the systems shown in Fig. 3.2. That is, generate 40,000 point records for each of these:

1. Colored random processes described by $x_{n+1} = (2 - \omega_0^2 - \rho)x_n + (\rho - 1)x_{n-1} + \eta_n$, where $\omega_0 = 2\pi/20$ (rad/sample), η_n is Gaussian noise with unit variance (use `randn` function), and for $\rho = [0.02, 0.05, 0.2, 0.5]$ values. For the initialization use $x_1 = 0$ and $x_2 = 0$.

2. x time series from the Duffing's oscillator

$$\ddot{x} + 0.2\,\dot{x} - x + x^3 = 0.33\cos t\,,$$

   starting from $x(0) = -0.6973$ and $\dot{x}(0) = 0.0729$ with 0.1 sampling time.

3. x time series from the Lorenz oscillator

$$\dot{x} = 10\,(y - x)\,, \quad \dot{y} = x\,(28 - z) - y, \quad \text{and} \quad \dot{z} = xy - \frac{8}{3}z\,,$$

   starting from $x(0) = 5$, $y(0) = -5$, and $z(0) = 20$ initial condition and with 0.02 sampling time.

Now calculate and plot:

1. Autocorrelations for all the time series just as shown in Fig. 3.2

2. Cross-covariances between the Duffing and Lorenz time series, and between AR(0.2) and AR(0.5) time series.

3. Cross-covariances between the Duffing and AR(0.5), and Lorenz and AR(0.5) time series.

What can you infer from all these plots?

Problem 3.3

Core

The autocorrelation of the AR(2) process of Fig. 3.2, $x_{n+1} = a_1 x_n + a_2 x_{n-1} + \eta_n$ with $a_1 = 2 - (\pi/10)^2 - \rho$, $a_2 = \rho - 1$, is known exactly.

1. Multiply the recursion by x_{n+1-k}, take expectations, and use $\mathrm{E}[\eta_n x_{n+1-k}] = 0$ for $k \geq 1$ to show that $\rho_k = a_1\rho_{k-1} + a_2\rho_{k-2}$ for $k \geq 2$, with $\rho_0 = 1$ and $\rho_1 = a_1/(1 - a_2)$ (the Yule–Walker equations).

2. For $\rho = 0.05$ compute ρ_k for $k = 0, \ldots, 60$. You should find $\rho_1 = 0.949$ and the first zero crossing between lags 5 and 6, a quarter of the twenty-sample period of the process.

3. Generate one record of 40,000 samples and estimate ρ_k with the $1/(N - m)$ normalization. Plot the estimate over the exact curve. Where do they differ most, and does the difference grow with the lag as the remark on the two normalizations predicts?

4. Show that the roots z of $z^2 - a_1 z - a_2 = 0$ have modulus $\sqrt{1 - \rho}$, and that ρ_k therefore decays as $(1 - \rho)^{k/2}$ times a cosine. Compute the $1/e$ correlation time and the integral time scale of Eq. (3.9) for each ρ in $\{0.02, 0.05, 0.2, 0.5\}$. Which of the two definitions is the more useful description of an oscillatory correlation, and why does the integral time scale nearly vanish?

Problem 3.4

needs Ch. 4 Computational

The two normalizations of the lagged covariance differ in a property that matters for everything built on them.

1. Generate 200 independent records of $N = 30$ Gaussian white samples. For each, compute $\widehat{C}_{xx}(m)$ for $m = 0, \dots, N-1$ with the $1/(N-m)$ normalization and with the $1/N$ normalization Eq. (3.18), and build the $N \times N$ Toeplitz matrix whose (i, j) entry is $\widehat{C}_{xx}(|i-j|)$. Count how many of the 200 matrices have a negative eigenvalue under each normalization. You should find that the $1/(N-m)$ matrix is indefinite in essentially every trial and the $1/N$ matrix in none.

2. Show analytically why the $1/N$ sequence cannot fail: write it as the autocorrelation of the finite zero-padded record and show that its discrete Fourier transform is $|X_k|^2/N \geq 0$.

3. Compute the discrete Fourier transform of both sequences for one record and plot them. How negative does the transform of the $1/(N-m)$ sequence become, relative to its mean level? Why does this rule the $1/(N-m)$ sequence out as a spectral estimator (Chapter 4) even though each of its lags is the less biased estimate?

4. Repeat part (1) with $N = 300$. Does the fraction of indefinite matrices fall? Explain in terms of which lags are responsible.

Problem 3.5

needs Ch. 4 Core

Uncorrelated is not independent.

1. Generate 40,000 iterates of the Hénon map with $(a, b) = (1.4, 0.3)$ and estimate the autocorrelation coefficient of x_n for lags 0 to 30. You should find $\rho_1 \approx -0.31$, $\rho_3 \approx -0.39$, an irregular decay with $|\rho_k| \approx 0.1$ still at lags 12–13, and values within the band $\pm 2/\sqrt{N}$ only beyond about lag 20.

2. Plot x_{n+1} against x_n and x_{n+20} against x_n. The first is a curve, the second a cloud. State exactly what ρ_1 and ρ_{20} each measure about these two pictures, and what neither can see.

3. Now iterate the Ulam map of Problem 4.2, $y_{n+1} = 1 - 2y_n^2$, for 40,000 steps and estimate its autocorrelation. You should find every lag $k \geq 1$ inside the white-noise band, although y_{n+1} is an exact function of y_n. Prove that $\mathrm{E}[y_{n+1} y_n] = 0$ under the invariant density $p(y) = 1/(\pi\sqrt{1-y^2})$.

4. What would the linear predictor of Problem 3.1, with $X = y_n$ and $Y = y_{n+1}$, predict for the Ulam map, and what is its mean-square error $\epsilon_{\min}$ relative to the variance of y? Contrast with the error of the nonlinear predictor $y_{n+1} = 1 - 2y_n^2$. This gap is what Part II of the book is about.

Problem 3.6

Experimental

Autocorrelation of the two-well oscillator record of Section 1.4.1.

1. Estimate $\rho_{xx}(m)$ of the strain record out to a lag of about ten forcing periods, using the sampling rate and forcing frequency given with the record to convert lags to periods. Plot it beside the Duffing curve of Fig. 3.2.

2. Locate the first zero crossing and express it as a fraction of the forcing period. Is it near a quarter period, as it is for a harmonic signal? Locate also the $1/e$ lag. Which of the two lies closer to the delay you chose by eye in Problem 1.6?

3. Does $\rho_{xx}(m)$ show a persistent oscillation at the forcing period, superposed on a decay? Estimate the amplitude of that oscillation at lags of five to ten periods. What does its persistence say about the fraction of the strain variance that is phase-locked to the forcing, and what does the decay say about the well-to-well motion?

4. Compute the cross-covariance between the strain and the well indicator b_i of Problem 2.6, and between the strain and a unit-amplitude cosine at the forcing frequency. Which of the two accounts for more of the strain variance, and how do you know from the correlation coefficient alone?

4 Fourier Analysis and Power Spectral Density

4.1 Fourier Series and Transforms

Recall Fourier series for periodic functions [192]

$$x(t) = \frac{1}{2}a_0 + \sum_{n=1}^{\infty}\left[a_n \cos\frac{2\pi nt}{T} + b_n \sin\frac{2\pi nt}{T}\right] \tag{4.1}$$

for $x(t+T) = x(t)$, where

$$\begin{aligned} a_0 &= \frac{2}{T}\int_0^T x(t)\,dt \quad \left(\frac{a_0}{2} = \bar{x}\right) \\ a_n &= \frac{2}{T}\int_0^T x(t)\cos n\omega t\,dt \quad \left(\omega = \frac{2\pi}{T}\right) \\ b_n &= \frac{2}{T}\int_0^T x(t)\sin n\omega t\,dt\,. \end{aligned} \tag{4.2}$$

Dirichlet Theorem

For $x(t)$ periodic on $0 \le t < T$, if $x(t)$ is bounded, has a finite number of maxima, minima, and discontinuities, then the Fourier Series Eq. (4.1) converges $\forall t$ to $\frac{1}{2}[x(t^+) + x(t^-)]$.

Complex form of Eq. (4.1) is better for experimental applications. Using Euler's (or de Moivre's) formulas we get:

$$\begin{aligned} \cos\omega t &= \frac{1}{2}\left(e^{i\omega t} + e^{-i\omega t}\right) \\ \sin\omega t &= \frac{1}{2i}\left(e^{i\omega t} - e^{-i\omega t}\right)\,. \end{aligned} \tag{4.3}$$

Using above Eq. (4.1) can be rewritten as:

$$x(t) = \sum_{-\infty}^{\infty} X_n\, e^{in\omega t}\,, \tag{4.4}$$

where

$$\begin{aligned} X_0 &= \frac{a_0}{2} \\ X_{\pm n} &= \frac{1}{2}\left(a_n \mp i b_n\right)\,. \end{aligned} \tag{4.5}$$

For real $x(t)$ the coefficients satisfy $X_n = X^*_{-n}$. Orthogonality of the exponentials over one period gives

$$X_n = \frac{1}{T}\int_0^T x(t)\, e^{-in\omega t}\, \mathrm{d}t = \frac{1}{T}\int_{-T/2}^{T/2} x(t)\, e^{-in\omega t}\, \mathrm{d}t\,. \tag{4.6}$$

If "signal" $x(t)$ is not periodic, we let $f_n = \frac{n}{T}$ (i.e., in Eq. (4.6) $n\omega = 2\pi f_n$). Now, we define a function $X(f)$ by $X(f_n) = TX_n$ (i.e., $X_n = X(f_n)/T$) to get the following:

$$x(t) = \sum_{-\infty}^{\infty} X_n\, e^{in\omega t} = \sum_{n=-\infty}^{\infty} \frac{1}{T} X(f_n)\, e^{i2\pi f_n t} = \sum_{n=-\infty}^{\infty} X(f_n)\, e^{i2\pi f_n t} \Delta f_n\,, \tag{4.7}$$

where we used the fact that $\Delta f_n = f_{n+1} - f_n = \frac{n+1}{T} - \frac{n}{T} = \frac{1}{T}$. Therefore, in the limit as $T \to \infty$ and $\Delta f_n \to 0$ in Eq. (4.7), we get our signal in the *time domain* as

$$x(t) = \int_{-\infty}^{\infty} X(f)\, e^{i2\pi f t}\, \mathrm{d}f\,. \tag{4.8}$$

Now, assuming that everything converges and using Eq. (4.6) we get the corresponding *frequency domain* expression

$$X(f) = \int_{-\infty}^{\infty} x(t)\, e^{-i2\pi f t}\, \mathrm{d}t\,. \tag{4.9}$$

Therefore, $x(t) \leftrightarrow X(f)$ are Fourier Transform pair, where $x(t)$ is in *time domain* and $X(f)$ is in *frequency domain*.

4.1.1 Several Important Properties of Fourier Transforms

We denote a Fourier transform (FT) as $X(f) = \mathcal{F}(x(t))$ and $x(t) = \mathcal{F}^{-1}(X(f))$. Now, we can write several of the properties of FT:

1. **Linearity:** $\mathcal{F}[\alpha x(t) + \beta y(t)] = \alpha X(f) + \beta Y(f)$
2. **Duality:** $x(t) \leftrightarrow X(f) \quad \Rightarrow \quad X(t) \leftrightarrow x(-f)$
3. **Conjugation:** $x(t) \leftrightarrow X(f) \quad \Rightarrow \quad x^*(t) \leftrightarrow X^*(-f)$.

   Therefore, for real signal $x(t)$, $X(f) = X^*(-f)$. This instead gives:

   $$|X(f)|^2 = X(f)X^*(f) = X^*(-f)X(-f) = |X(-f)|^2\,, \tag{4.10}$$

   i.e., for real $x(t)$, $|X(f)|$ is symmetric.
4. **Convolution:**

   $$\mathcal{F}\left[\int_{-\infty}^{\infty} x(\tau)y(t-\tau)\, d\tau\right] = X(f)Y(f) \triangleq \mathcal{F}[x * y]\,,$$

   where $x * y$ indicates time convolution between $x(t)$ and $y(t)$. In addition,

   $$\mathcal{F}[xy] = \int_{-\infty}^{\infty} X(\phi)Y(f-\phi)\, d\phi \triangleq X * Y\,,$$

   where $X * Y$ indicates frequency convolution between $X(f)$ and $Y(f)$ (also, $X * Y = Y * X$).
5. **Differentiation:**

   $$\mathcal{F}\left[\frac{\mathrm{d}^k x}{\mathrm{d}t^k}\right] = (i2\pi f)^k X(f)\,,$$

6. **Time Scaling and Shifting:**

$$x(at+b) \leftrightarrow \frac{e^{2\pi i f \frac{b}{a}}}{|a|} X\left(\frac{f}{a}\right) .$$

Theorem

Provided $x(t) \in \mathcal{L}^1$ (i.e., $\int_{-\infty}^{\infty} |x(t)| \mathrm{d}t < \infty$) and $x(t)$ satisfies the Dirichlet conditions (a finite number of maxima, minima, and discontinuities in every finite interval), $X(f)$ exists, and

$$x(t) = \begin{cases} \mathcal{F}^{-1}\left[X(f)\right] & \text{for } x \text{ continuous at } t\,, \\ \frac{1}{2}\left[x(t^+) + x(t^-)\right] & \text{for } x \text{ discontinuous at } t\,. \end{cases}$$

There is a problem with the above theorem if we consider the following:

$$\int_{-\infty}^{\infty} |\sin t| \, \mathrm{d}t = \infty \,,$$

which can be fixed using the theory of *generalized functions (or distributions)*, *duality* and other basic properties.

4.1.2 Basic Fourier Transform Pairs

1. **Delta (δ) "function":** This actually is a generalized function or distribution defined as:

$$\int_{-\infty}^{\infty} \delta(t) \mathrm{d}t = 1 \,, \tag{4.11}$$

where $\delta(t) = 0$ for all $t \neq 0$ and $\delta(0) = \infty$. In addition, by definition of $\delta(t)$,

$$\Delta(f) = \int_{-\infty}^{\infty} e^{-2\pi i f t} \delta(t - t_0) \mathrm{d}t = e^{-2\pi i f t_0},$$

which is also called "sifting property." Note that Δ is a complex constant with $|\Delta| = 1$. Therefore:

$$\delta(t - t_0) \leftrightarrow e^{-2\pi i f t_0} \,, \tag{4.12}$$

and in particular, $\delta(t) \leftrightarrow 1$. Therefore, by duality property

$$e^{2\pi i f_0 t} \leftrightarrow \delta(f - f_0) \,, \tag{4.13}$$

and in particular, $1 \leftrightarrow \delta(f)$.

2. **Trigonometric functions:**

$$\cos(2\pi f_0 t) = \frac{1}{2}\left(e^{2\pi i f_0 t} + e^{-2\pi i f_0 t}\right) \,, \tag{4.14}$$

so

$$\mathcal{F}\left[\cos(2\pi f_0 t)\right] = \frac{\delta(f - f_0) + \delta(f + f_0)}{2} \,, \tag{4.15}$$

as shown in Fig. 4.1. Similarly,

$$\mathcal{F}\left[\sin(2\pi f_0 t)\right] = \frac{\delta(f - f_0) - \delta(f + f_0)}{2i} \,. \tag{4.16}$$

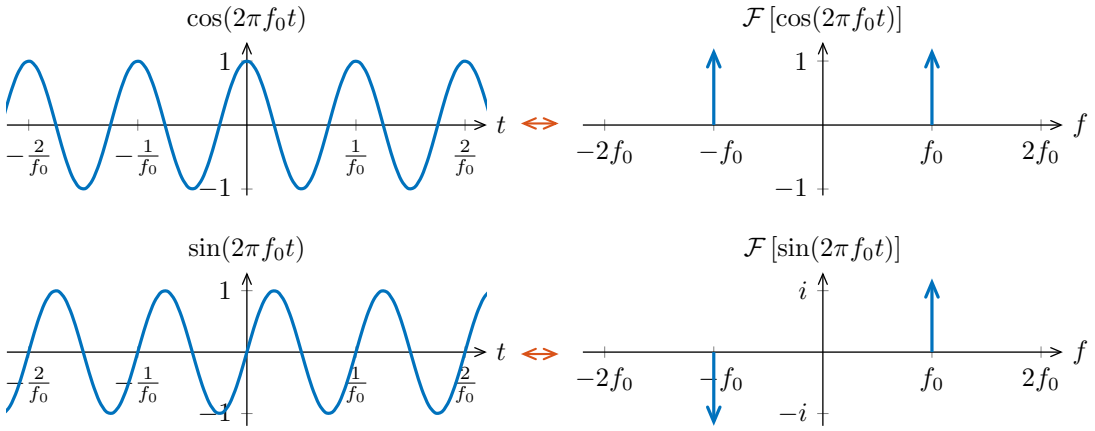


Figure 4.1: Fourier transform pairs for the trigonometric Functions

3. **Modulated trigonometric functions:** As an example consider $x(t)\cos(2\pi f_c t) \leftrightarrow X(f) * C(f)$, where f_c is called *carrier frequency* and $C(f)$ is given by Eq. (4.15). Then,

$$x(t)\cos(2\pi f_c t) \leftrightarrow \int_{-\infty}^{\infty} X(s)\frac{\delta(f-f_c-s)+\delta(f+f_c-s)}{2}\mathrm{d}s\,, \tag{4.17}$$

where on the right hand side we have a convolution integral, which gives:

$$x(t)\cos(2\pi f_c t) \leftrightarrow \frac{X(f-f_c)+X(f+f_c)}{2}\,. \tag{4.18}$$

Therefore, if we already know $X(f)$, modulation scales and shifts it to $\pm f_c$ as shown in Fig. 4.2.

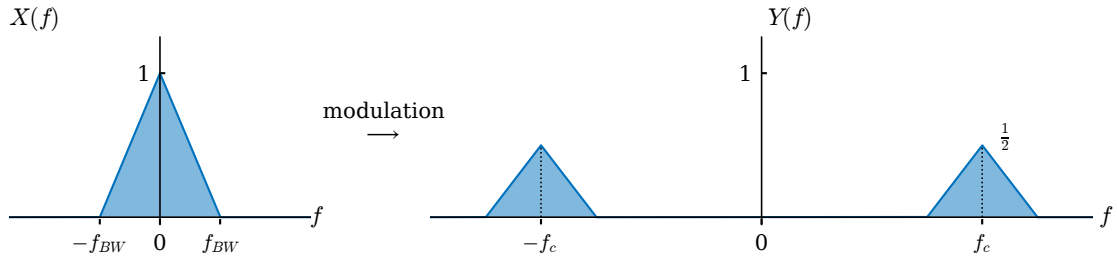


Figure 4.2: Signal modulation in the frequency domain. A band-limited $X(f)$, here the triangle $1-|f|/f_{BW}$, becomes $Y(f)=[X(f-f_c)+X(f+f_c)]/2$ for $x(t)\cos 2\pi f_c t$: two copies of half height centered at $\pm f_c$ ($f_c = 4f_{BW}$ in the drawing). Script `python/mkfig_modulation.py`.

4.2 Power Spectral Density

The autocorrelation of a real, stationary signal $x(t)$ is defined by Eq. (3.8) as $R_{xx}(\tau) = E[x(t)x(t+\tau)]$. *Power Spectral Density* (PSD), $S_{xx}(f)$, can be defined as a Fourier transform of $R_{xx}(\tau)$. That this definition is equivalent to the spectral decomposition of the process itself is the *Wiener–Khinchin theorem*, established independently by Wiener [222] and Khintchine [115]:

$$S_{xx}(f) = \int_{-\infty}^{\infty} R_{xx}(\tau)\, e^{-i2\pi f\tau}\,\mathrm{d}\tau\,. \tag{4.19}$$

The natural questions are: what is the PSD? What does it mean? What is a "spectral density," and why is S_{xx} called a *power* spectral density?

To answer these questions, recall that

$$X(f) = \int_{-\infty}^{\infty} x(t)\, e^{-i2\pi ft}\, \mathrm{d}t\,. \tag{4.20}$$

To avoid convergence problems, we consider only a version of the signal observed over a finite-time T,[1] $x_T = x(t)w_T(t)$, where

$$w_T = \begin{cases} 1 & \text{for} \quad |t| \le T/2\,, \\ 0 & \text{for} \quad |t| > T/2\,. \end{cases} \tag{4.21}$$

Then x_T has the Fourier transform

$$\begin{aligned} X_T(f) &= \int_{-\infty}^{\infty} x_T(t)\, e^{-i2\pi ft}\, \mathrm{d}t\,, \\ &= \int_{-T/2}^{T/2} x(t)\, e^{-i2\pi ft}\, \mathrm{d}t\,, \end{aligned} \tag{4.22}$$

and so

$$\begin{aligned} X_T X_T^* &= \left[\int_{-T/2}^{T/2} x(t)\, e^{-i2\pi ft}\, \mathrm{d}t\right] \left[\int_{-T/2}^{T/2} x^*(s)\, e^{i2\pi fs}\, \mathrm{d}s\right]\,, \\ &= \int_{-T/2}^{T/2}\int_{-T/2}^{T/2} x(t)x(s)\, e^{-i2\pi f(t-s)}\, \mathrm{d}t\mathrm{d}s\,, \end{aligned} \tag{4.23}$$

where the star denotes complex conjugation and for compactness the frequency argument of X_T has been suppressed. Taking the expectation of both sides of Eq. (4.23)[2], we get

$$E\left[X_T X_T^*\right] = \int_{-T/2}^{T/2}\int_{-T/2}^{T/2} E\left[x(t)x(s)\right]\, e^{-i2\pi f(t-s)}\, \mathrm{d}t\mathrm{d}s\,. \tag{4.24}$$

Letting $s = t - \tau$, one sees that $E[x(t)x(s)] \triangleq E[x(t-\tau)x(t)] = E[x(t)x(t+\tau)] = R_{xx}(\tau)$, and thus

$$E\left[X_T X_T^*\right] = \int_{-T/2}^{T/2}\int_{-T/2}^{T/2} R_{xx}(\tau)\, e^{-i2\pi f(t-s)}\, \mathrm{d}t\mathrm{d}s\,. \tag{4.25}$$

To actually evaluate the above integral, *both* variables of integration must be changed. Let

$$\begin{aligned} &\tau = f(t,s) = t - s \quad \text{as already defined for Eq. (4.24)}, \\ &\eta = g(t,s) = t + s\,. \end{aligned} \tag{4.26}$$

Then, the integral of Eq. (4.25) is transformed (except for the limits of integration) using the *change of variables formula*:[3]

$$\int_{-T/2}^{T/2}\int_{-T/2}^{T/2} R_{xx}(\tau)\, e^{-i2\pi f(t-s)}\, \mathrm{d}t\mathrm{d}s = \int_{?}^{?}\int_{?}^{?} R_{xx}(\tau)\, e^{-i2\pi f\tau}\, |J|^{-1}\, \mathrm{d}\tau\mathrm{d}\eta\,, \tag{4.27}$$

where $|J|$ is the absolute value of the Jacobian for the change of variables Eq. (4.26) given by

$$|J| = \begin{vmatrix} \frac{\partial f}{\partial t} & \frac{\partial f}{\partial s} \\ \frac{\partial g}{\partial t} & \frac{\partial g}{\partial s} \end{vmatrix} = \begin{vmatrix} 1 & -1 \\ 1 & 1 \end{vmatrix} = 2\,. \tag{4.28}$$

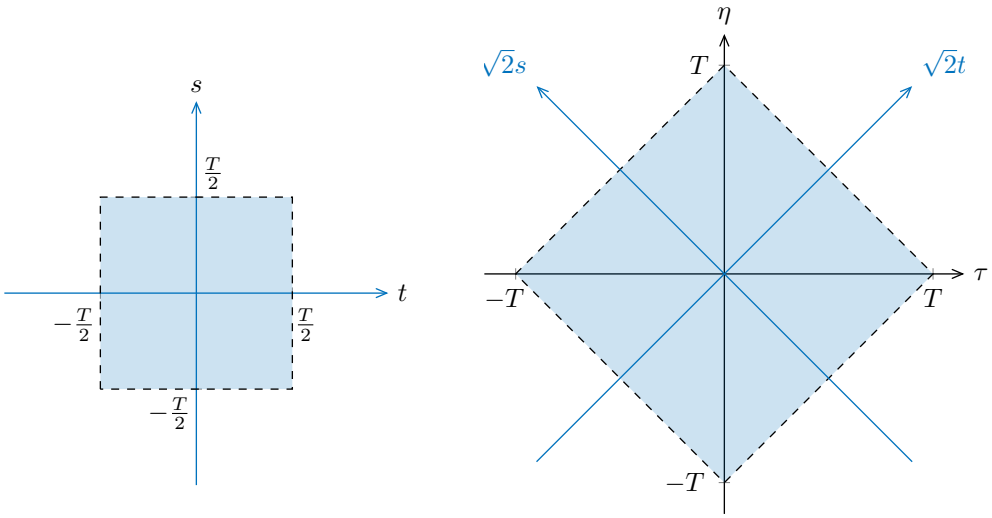


Figure 4.3: The domain of integration (light blue regions) for the Fourier transform of the autocorrelation Eq. (4.25): (left) for the original variables, t and s; (right) for the transformed variables, η and τ, obtained by the change of variables Eq. (4.26). Notice that the square region on the left is not only rotated, but its area is increased by a factor of $|J| = 2$. The lines into which the t and s axes are mapped are also shown (they are both rotated by 45 degrees and scaled by $\sqrt{|J|} = \sqrt{2}$).

To determine the limits of integration needed for the right hand side of Eq. (4.27), we need to refer to Fig. 4.3, in which the domain of integration is plotted in both the original (t, s) variables and the transformed (τ, η) variables. Since we wish to integrate on η first, we hold τ fixed. For $\tau > 0$, a vertical cut through the diamond-shaped region in Fig. 4.3 (right) shows that $-T + \tau \leq \eta \leq T - \tau$, whereas for $\tau < 0$ one finds that $-T - \tau \leq \eta \leq T + \tau$. Putting this all together yields:

$$E\left[X_T X_T^*\right] = \int_{-T}^{T} \int_{-(T-|\tau|)}^{T-|\tau|} \frac{1}{2} R_{xx}(\tau)\, e^{-i2\pi f\tau}\, \mathrm{d}\eta \mathrm{d}\tau = T \int_{-T}^{T} \left[1 - \frac{|\tau|}{T}\right] R_{xx}(\tau)\, e^{-i2\pi f\tau}\, \mathrm{d}\tau\,. \tag{4.29}$$

Finally, dividing both sides of Eq. (4.29) by T and taking the limit as $T \to \infty$ gives

$$\begin{aligned} \lim_{T\to\infty} \frac{1}{T} E\left[X_T X_T^*\right] &= \lim_{T\to\infty} \int_{-T}^{T} \left[1 - \frac{|\tau|}{T}\right] R_{xx}(\tau)\, e^{-i2\pi f\tau}\, \mathrm{d}\tau \\ &= \int_{-\infty}^{\infty} \left(\lim_{T\to\infty} \left[1 - \frac{|\tau|}{T}\right] w_{2T}(\tau)\right) R_{xx}(\tau)\, e^{-i2\pi f\tau}\, \mathrm{d}\tau \\ &= \int_{-\infty}^{\infty} R_{xx}(\tau)\, e^{-i2\pi f\tau}\, \mathrm{d}\tau \\ &= S_{xx}(f)\,. \end{aligned} \tag{4.30}$$

[1] This restriction is necessary because not all of our signals will be square integrable. However, they will be mean square integrable, which is what we will take advantage of here.

[2] To understand what this means, remember that Eq. (4.23) holds for any $x(t)$. So imagine computing Eq. (4.24) for different $x(t)$ obtained from different experiments on the same system (each one of these is called a sample function). The expectation is over all possible sample functions. Since the exponential kernel inside the integral of Eq. (4.24) is the same for each sample function, it can be pulled outside of the expectation.

[3] This is a basic result from multivariable calculus; see, for example, Sokolnikoff and Redheffer [192].

Here $w_{2T}(\tau)$ is the rectangular window of width $2T$, so that the integration limits no longer depend on T and the limit can be taken inside the integral. That step requires a hypothesis: the integrand is bounded by $|R_{xx}(\tau)|$, so if $R_{xx} \in \mathcal{L}^1$ the exchange is justified by dominated convergence, and $S_{xx}(f)$ is an ordinary, continuous function. This is the case for the broadband random processes of Section 4.3, whose correlations decay. It is *not* the case for the periodic and quasi-periodic responses that dominate this book: for $x(t) = a\cos(2\pi f_0 t + \phi)$ with random phase, $R_{xx}(\tau) = \frac{a^2}{2}\cos 2\pi f_0 \tau$ does not decay, $\frac{1}{T}E|X_T(f)|^2$ grows without bound at $f = \pm f_0$, and S_{xx} exists only as a distribution, $S_{xx}(f) = \frac{a^2}{4}[\delta(f - f_0) + \delta(f + f_0)]$, a *line spectrum*. The area under each line is the power in that component, so Eq. (4.33) below continues to hold. Every finite-record estimate of a line spectrum shows a peak of finite height and width, set by the window, which is the leakage phenomenon of Chapters 5 and 6.

Thus, in summary, the above demonstrates that

$$S_{xx}(f) = \lim_{T\to\infty} \frac{1}{T} E\left[|X_T(f)|^2\right] . \tag{4.31}$$

Recalling that $X_T(f)$ has units SU/Hz (where SU stands for "signal units," i.e., whatever units the signal $x_T(t)$ has), it is clear that $E\left[|X_T(f)|^2\right]$ has units $(\text{SU/Hz})^2$. However, $1/T$ has units of Hz, so that Eq. (4.31) shows that the PSD has units of $(\text{SU}^2)/\text{Hz}$.[4]

Although it is not always literally true, in many cases, the mean square of the signal is proportional to the amount of *power* in the signal.[5] The fact that S_{xx} is therefore interpreted as having units of "power" per unit frequency explains the name Power Spectral Density.

Notice that power at a frequency f_0 that does not repeatedly reappear in $x_T(t)$ as $T \to \infty$ will result in $S_{xx}(f_0) \to 0$, because of the division by T in Eq. (4.31). In fact, based on this idealized mathematical definition, any signal of finite duration (or, more generally, any mean square integrable signal), will have power spectrum identical to zero! In practice, however, we do not let T extend much past the support $[T_{\min}, T_{\max}]$ of the signal, the smallest interval outside of which $x(t) = 0$. Since all signals that we measure in the laboratory have the form $y(t) = x(t) + n(t)$, where $n(t)$ is broadband noise, extending T to infinity for any signal with finite support will end up giving $S_{xx} \approx S_{nn}$.

We conclude by mentioning some important properties of S_{xx}. First, since S_{xx} is an average of the magnitude squared of the Fourier transform, $S_{xx}(f) \in \mathbb{R}$ and $S_{xx}(f) \geq 0$ for all f. A simple change of variables in the definition Eq. (4.19) shows that $S_{xx}(-f) = S_{xx}(f)$.

Given the definition Eq. (4.19), we also have the dual relationship

$$R_{xx}(\tau) = \int_{-\infty}^{\infty} S_{xx}(f)\, e^{i2\pi f\tau}\, \mathrm{d}f\, . \tag{4.32}$$

Setting $\tau = 0$ in the above gives

$$R_{xx}(0) = E\left[x(t)^2\right] = \int_{-\infty}^{\infty} S_{xx}(f)\, \mathrm{d}f\, , \tag{4.33}$$

which, for a mean zero signal gives

$$\sigma_x^2 = \int_{-\infty}^{\infty} S_{xx}(f)\, \mathrm{d}f\, . \tag{4.34}$$

[4] Of course, the units can also be determined by examining the definition of Eq. (4.19).

[5] This comes primarily from the fact that, in electrical circuits, the power can be written in terms of the voltage as V^2/Z, or in terms of the current as $I^2 Z$, where Z is the circuit impedance. Thus, for electrical signals, it is precisely true that the mean square of the signal will be proportional to the power. Be forewarned, however, that the mean square of the scaled signal, expressed in terms of the actual measured variable (such as displacement or acceleration), will *not* in general be equal to the average mechanical power in the structure being measured.

Finally, if we assume that $x(t)$ is ergodic in the autocorrelation, that is, that

$$R_{xx}(\tau) = E[x(t)x(t+\tau)] = \lim_{T\to\infty} \frac{1}{T} \int_{-T/2}^{T/2} x(t)x(t+\tau)\mathrm{d}t\,,$$

where the last equality holds for any sample function $x(t)$, then Eq. (4.33) can be rewritten as

$$\lim_{T\to\infty} \frac{1}{T} \int_{-T/2}^{T/2} x(t)^2 \mathrm{d}t = \int_{-\infty}^{\infty} S_{xx}(f)\,\mathrm{d}f\,. \tag{4.35}$$

The above relationship is known as *Parseval's Identity*.

This last identity makes it clear that, given any two frequencies f_1 and f_2, the quantity

$$\int_{f_1}^{f_2} S_{xx}(f)\,\mathrm{d}f$$

represents the portion of the average signal power contained in signal frequencies between f_1 and f_2, and hence S_{xx} is indeed a "spectral density."

4.3 Sample Power Spectra

Estimating a spectrum from a finite record means averaging modified periodograms over overlapping segments, the procedure introduced by Welch [221] and now standard; Chapter 6 develops it, together with cross-spectra, coherence and frequency response functions. Jenkins and Watts [101] and Bendat and Piersol [8] treat the bias and variance of the resulting estimators; the choice of taper is analyzed exhaustively by Harris [88], and the whole enterprise rests on the fast algorithm of Cooley and Tukey [40].

1. **White noise:** $S_{xx}(f) = 1$, where we have power at all frequencies. The corresponding autocorrelation is $R_{xx}(\tau) = \delta(\tau)$, see Fig. 4.4.

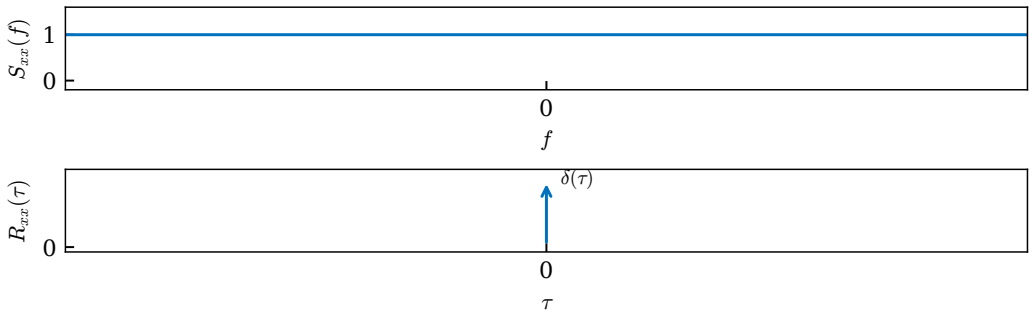


Figure 4.4: White noise has power at all frequencies, $S_{xx}(f) = 1$, and is uncorrelated for $\tau \neq 0$: $R_{xx}(\tau) = \delta(\tau)$, drawn as a unit arrow. Script `python/mkfig_whitenoise.py`.

2. **Band limited noise:** $S_{xx}(f) = W(f) = \begin{cases} 1, & 0 \le |f| \le f_{BW} \\ 0, & \text{otherwise} \end{cases}$

$$\begin{aligned} R_{xx}(\tau) &= \int_{-\infty}^{\infty} W(f) e^{i2\pi f\tau} \mathrm{d}f \\ &= \int_{-f_{BW}}^{f_{BW}} e^{i2\pi f\tau} \mathrm{d}f \\ &= \frac{1}{2i\pi\tau} \left[e^{i2\pi f_{BW}\tau} - e^{-i2\pi f_{BW}\tau} \right] \\ &= \frac{\sin(2\pi f_{BW}\tau)}{\pi\tau} = 2f_{BW} \operatorname{sinc}(2\pi f_{BW}\tau) \ . \end{aligned} \tag{4.36}$$

For the corresponding graphs refer to Fig. 4.5.

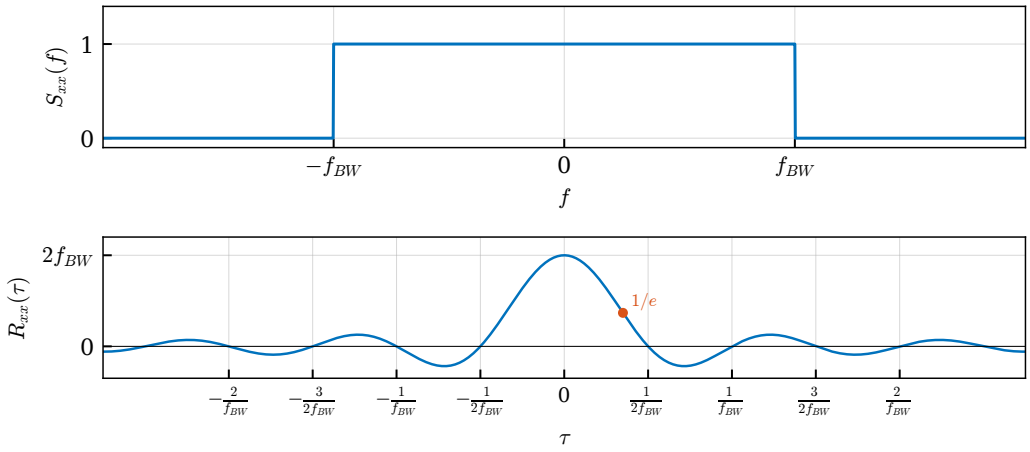


Figure 4.5: Band-limited noise, $S_{xx}(f) = 1$ on $|f| \le f_{BW}$, and its autocorrelation $R_{xx}(\tau) = 2f_{BW}\operatorname{sinc}(2\pi f_{BW}\tau)$. The first zero of R_{xx} is at $\tau = 1/(2f_{BW})$; in the terms of Chapter 3 the $1/e$ lag of ρ_{xx} is $0.35/f_{BW}$ (dot) and the integral time scale is $1/(4f_{BW})$, so the correlation time is of order $1/(2f_{BW})$ whichever definition is used. Script `python/mkfig_bandlimited.py`.

Problems

Problem 4.1

needs Ch. 6 Core

Create a sample $\{x_n\}_{n=1}^{1024+14}$[6] of uncorrelated Gaussian random variables (command `randn` in MATLAB, `numpy.random.standard_normal` in Python). Now apply the moving average filter $s_n = 1/15 \sum_{i=-7}^{7} x_{n+i}$ to obtain 1024 correlated Gaussian variates. Estimate the power spectrum with Welch's method (Section 6.2; `pwelch` in MATLAB, `scipy.signal.welch` in Python) for both data sequences and observe the differences.

Problem 4.2

Core

Create two time series: (1) $\{x_n\}_{n=1}^{4096}$ of uncorrelated uniform random variables (command `rand` in MATLAB, `numpy.random.random` in Python), and (2) deterministic evolution of the *Ulam map* [216] $\{y_n\}_{n=1}^{4096}$, which follows rules $y_0 = 0.1$ and $y_{n+1} = 1 - 2y_n^2$. The values of y_n are measured through a nonlinear observation function $s_n = \arccos(-y_n)/\pi$. Compare the mean, variance, and power spectra of these three time series ($\{x_n\}_{n=1}^{4096}$, $\{y_n\}_{n=1}^{4096}$, $\{s_n\}_{n=1}^{4096}$).

Problem 4.3

Core

Two transform pairs that recur throughout the book.

1. A stationary process has the exponential autocorrelation $R_{xx}(\tau) = \sigma^2 e^{-|\tau|/\tau_0}$ (the continuous-time analogue of the first-order autoregression of Problem 2.3). Evaluate Eq. (4.19) to show that $S_{xx}(f) = 2\sigma^2\tau_0/[1 + (2\pi f\tau_0)^2]$. Verify Eq. (4.34) by integrating S_{xx} over all f, and show that the spectrum falls to half its peak value at $f = 1/(2\pi\tau_0)$. State the units of S_{xx} and check them.

2. Let $x(t) = a\cos(2\pi f_0 t + \phi)$ with ϕ uniform on $[0, 2\pi)$. Show that $R_{xx}(\tau) = \frac{a^2}{2}\cos 2\pi f_0\tau$ and hence that $S_{xx}(f) = \frac{a^2}{4}[\delta(f - f_0) + \delta(f + f_0)]$. Verify that the total area under the two lines equals the power $a^2/2$. Then take the finite-record route: compute $\frac{1}{T}\mathrm{E}\left[|X_T(f_0)|^2\right]$ for this signal and show that it grows in proportion to T, which is why the limit Eq. (4.31) exists only as a distribution.

3. Sum the two: for $x(t) = a\cos(2\pi f_0 t + \phi) + n(t)$ with n the process of part (1), independent of ϕ, write $S_{xx}(f)$. A finite record of length T replaces each line by a peak of width about $1/T$. Find the record length at which the peak height $a^2T/4$ equals the noise density at f_0, for $a = 1$, $\sigma = 1$, $\tau_0 = 1$ s and $f_0 = 1$ Hz.

Problem 4.4

needs Ch. 6 Computational

Spectra of the running records of Section 1.4.

1. Generate the Lorenz x record of Problem 3.2 ($\Delta t = 0.02$) and the two-well Duffing x record with thirty samples per forcing period, each of 40,000 samples after the transient. Estimate both spectra with Welch's method (Hann window, segments of 1024 samples, half overlap) and plot them on a decibel scale against frequency in cycles per unit time.

[6] we only need 1024 points for the spectral analysis, additional 14 points are to generate 1024 point filtered time series

2. Check Parseval's identity Eq. (4.35) numerically: sum the one-sided estimate over frequency, multiply by the bin width, and compare with the sample variance. You should find agreement to within one percent for both records and for segment lengths of 256, 1024 and 4096.

3. The Duffing spectrum has a line at the forcing frequency $f_0 = 1/(2\pi)$ on a broadband floor. Integrate the estimate over the few bins around f_0 and express the result as a fraction of the variance. Repeat for the third harmonic $3f_0$. With the segment length of 4096 you should find the fundamental line carrying only a few percent of the variance: the chaotic two-well motion is broadband, and the forcing line is a small part of it.

4. The Lorenz spectrum has no line. Read the frequency of its broad maximum and compare it with the mean rate at which x circles one lobe, estimated by counting local maxima of x per unit time. Why is the peak broad even though the circulation is regular?

Problem 4.5

needs Ch. 6 Computational

The PSD of a signal with finite support is zero, and the text warns that letting T grow past the support gives $S_{xx} \approx S_{nn}$. Make this concrete.

1. Simulate the free decay $x(t) = e^{-\zeta\omega_n t}\cos(\omega_n\sqrt{1-\zeta^2}\,t)$ with $f_n = 1$ Hz and $\zeta = 0.02$, sampled at 100 Hz, and add white noise of standard deviation 0.05. Compute the periodogram (rectangular window, no averaging) of records of length $T = 20$, 80 and 320 s, all starting at $t = 0$.

2. Tabulate the peak value near f_n and the mean level above 10 Hz. You should find the peak falling by about a factor of four each time T is quadrupled, while the floor stays at $2\sigma_n^2\Delta t$ (one-sided). Explain both from Eq. (4.31).

3. At what T does the peak of the decay fall below the noise floor? Compare with the time $1/(\zeta\omega_n)$ over which the signal itself decays. State the general rule for choosing T for a transient of known decay rate.

4. Now let the noise be absent and increase T further. The peak keeps falling as $1/T$ with no floor. The correct object for a transient is the energy spectrum $|X_T(f)|^2$ without the $1/T$: show that it converges as $T \to \infty$ and that its peak height is $\approx 1/(2\zeta\omega_n)^2$ for small ζ.

Problem 4.6

needs Ch. 6 Experimental

Spectrum of the two-well oscillator record of Section 1.4.1.

1. Estimate the PSD of the strain record by Welch's method with a Hann window, choosing the segment length so that at least ten bins fall between the forcing line and its second harmonic (use the sampling rate and forcing frequency given with the record). Plot it in decibels against frequency in hertz, with the forcing frequency and its first five harmonics marked.

2. Check Parseval's identity as in Problem 4.4 and report the fraction of the variance under the forcing line, under all harmonics together, and in the broadband remainder. Compare with the fractions found for the Duffing model in Problem 4.4. Is the measured record more or less phase-locked to the forcing than the model?

3. Identify the flat floor at the highest frequencies and read its level in V^2/Hz. (The record is stored in volts; no gauge factor is supplied with it, so every level in this problem stays in volts. A calibration would rescale all of them by the same constant and change none of the conclusions.) Multiply by the bandwidth it occupies to estimate the variance of the measurement noise, and express the result as a signal-to-noise ratio in decibels. Chapters 14 and 13 return to this number.

4. Between the harmonics the spectrum is continuous, as for the Duffing model. Estimate the frequency at which the continuous part is largest and compare it with the mean rate of well crossings estimated from the well indicator of Problem 2.6. What motion produces the low-frequency part of the spectrum?

5 Experimental Fourier Transforms

5.1 Sampling and Aliasing

Given $x(t)$, we observe only *sampled data* $x_s(t) = x(t)s(t;T_s)$ (Fig. 5.1), where s is called sampling or "comb" function and can be described as

$$s(t;T_s) = \sum_{n=-\infty}^{\infty} \delta(t - nT_s)\,, \tag{5.1}$$

where T_s is *sampling period* and $f_s = 1/T_s$ is *sampling frequency* (Fig. 5.2).

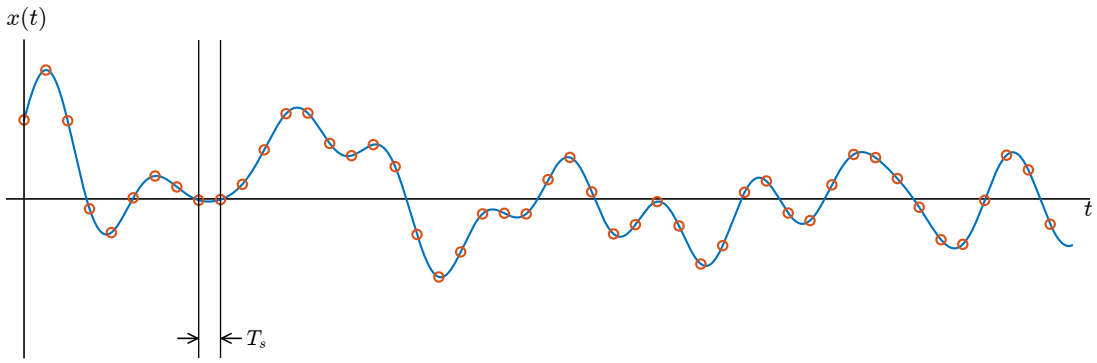


Figure 5.1: We observe only sampled data $x(nT_s)$ (circles) of a continuous signal $x(t)$, one sample every sampling period T_s. The signal is band-limited Gaussian noise, a sum of 60 random-phase cosines with frequencies below $f_{BW} = 1$ Hz, sampled at $T_s = 0.25$ s ($f_s = 4f_{BW}$). Script `python/mkfig_sampled.py`.

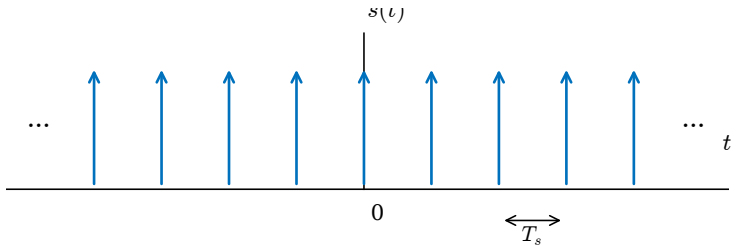


Figure 5.2: Sampling or comb function $s(t;T_s) = \sum_n \delta(t - nT_s)$, each delta drawn as a unit arrow. Script `python/mkfig_comb.py`.

Therefore, using the *sifting property* of the Dirac delta function, we get

$$x_s(t) = x(t) \sum_{n=-\infty}^{\infty} \delta(t - nT_s) = \sum_{n=-\infty}^{\infty} x(nT_s)\delta(t - nT_s)\,. \tag{5.2}$$

To take the Fourier Transform of x_s, it is useful to note that $s(t; T_s)$ is *periodic* with period T_s. Thus, the corresponding Fourier Series will be:

$$s(t; T_s) = \sum_{k=-\infty}^{\infty} X_k e^{ik\frac{2\pi}{T_s}t}\,, \tag{5.3}$$

where

$$X_k = \frac{1}{T_s}\int_{-T_s/2}^{T_s/2}\left[\sum_{n=-\infty}^{\infty}\delta(t - nT_s)\right]e^{-ik\frac{2\pi}{T_s}t}\mathrm{d}t = \frac{1}{T_s}e^0 = \frac{1}{T_s}\,, \tag{5.4}$$

since only the $n = 0$ delta lies inside the interval of integration. So

$$s(t; T_s) = \frac{1}{T_s}\sum_{k=-\infty}^{\infty} e^{i2\pi\frac{k}{T_s}t}\,. \tag{5.5}$$

Now, from previously calculated δ function Fourier Transform pairs, we get

$$S(f) = \frac{1}{T_s}\sum_{n=-\infty}^{\infty}\delta\left(f - \frac{n}{T_s}\right)\,. \tag{5.6}$$

Therefore, since $x_s(t) = x(t)s(t; T_s)$, by convolution we get

$$\mathcal{F}\left[x_s\right] = X_s(f) = X(f) * S(f)\,, \tag{5.7}$$

that is

$$X_s(f) = \frac{1}{T_s}\sum_{n=-\infty}^{\infty}\int_{-\infty}^{\infty} X(s)\delta\left(f - \frac{n}{T_s} - s\right)\mathrm{d}s\,, \tag{5.8}$$

or

$$X_s(f) = \frac{1}{T_s}\sum_{n=-\infty}^{\infty} X\left(f - \frac{n}{T_s}\right)\,. \tag{5.9}$$

That is, sampling results in an ∞ number of periodically spaced copies of the true $X(f)$. Now, we see that if $f_s < 2f_{BW}$, we will get spectral overlap or *aliasing*: high frequencies are "folded" onto lower ones. This leads to the *Nyquist sampling criterion*: $f_s > 2f_{BW}$ (strictly, since a component at exactly $f_s/2$ is not recoverable) for accurate frequency domain estimates with sampled data (see Fig. 5.3). Shannon [148, 184] was the one who credited Nyquist when he stated his theorem:

Shannon's Sampling Theorem

If a function $x(t)$ contains no frequencies higher than f_{BW} hertz, it is completely determined by giving its ordinates at a series of points spaced $T_s = \frac{1}{2f_{BW}}$ seconds apart.

This means that the continuous signal $x(t)$ can be completely reconstructed from its samples, by sinc interpolation, if the Nyquist criterion is satisfied.

Pitfall: sampling at the Nyquist rate itself

The theorem is a statement about a signal that is exactly band-limited, and no measured signal is. Any energy a record carries above f_{BW} folds into the band of interest, so in practice $f_s = 3f_{BW}$ or even $4f_{BW}$ is needed before what is folded in is small.

The energy above f_{BW} has to be removed before the sampler sees it, by an analog low-pass *anti-aliasing* filter; Fig. 5.4 shows one, as an example only. Such a filter does not cut off sharply, so its own roll-off corrupts the spectrum near f_c. If $f_c \approx f_{BW}$, sample at $f_s = 4f_c$ and discard the upper half of the one-sided transform, $f_c < f < f_s/2$, where the roll-off has taken hold.

5.2 Windowing

Another artifact of data collection is that we take only a *finite time record* of our data (see Fig. 5.5). Mathematically we can express this as we are sampling the following signal $x_w(t) = x(t)w(t)$, where $w(t)$ is a data collection window function. If our data collection window $w(t)$ is a rectangular window function, we get what is shown in Fig. 5.6.

Now, we know how to find the Fourier transform of a rectangular window:

$$\begin{aligned} W(f) &= \int_{-\infty}^{\infty} w(t) e^{-2\pi i f t} \mathrm{d}t = \int_{-a}^{a} e^{-2\pi i f t} \mathrm{d}t \\ &= \frac{1}{2\pi i f} \left(e^{2\pi i f a} - e^{-2\pi i f a} \right) = \frac{\sin 2\pi f a}{\pi f} = 2a \,\mathrm{sinc}(2\pi f a)\,, \end{aligned} \tag{5.10}$$

where the sinc function has the form shown in Fig. 5.7.

As an example of the effect of the window, let $x(t) = \cos 2\pi f_0 t$, then using convolution property we have the following:

$$\begin{aligned} X_w(f) &= X(f) * W(f) \\ &= \int_{-\infty}^{\infty} \frac{1}{2} \left[\delta\left(f + f_0 - s\right) + \delta\left(f - f_0 - s\right) \right] \frac{\sin 2\pi s a}{\pi s} \mathrm{d}s \\ &= \frac{1}{2} \left[\frac{\sin\left(2\pi (f + f_0) a\right)}{\pi (f + f_0)} + \frac{\sin\left(2\pi (f - f_0) a\right)}{\pi (f - f_0)} \right] \end{aligned} \tag{5.11}$$

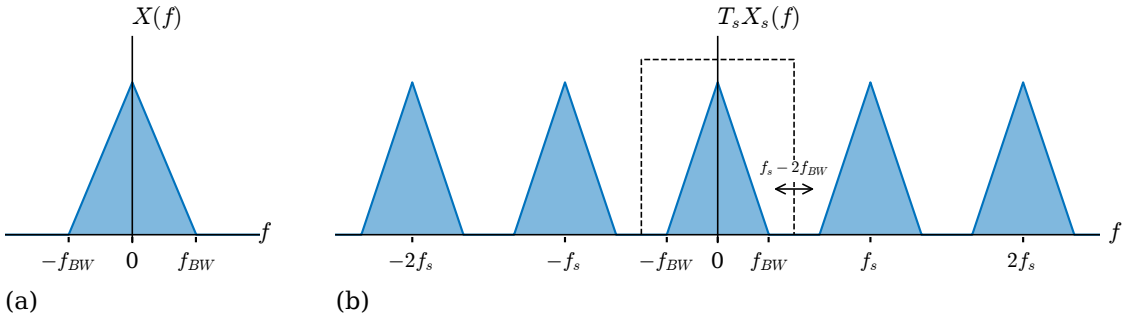


Figure 5.3: (a) The true Fourier transform $X(f)$ of a signal band-limited to f_{BW} (drawn as the triangle $1 - |f|/f_{BW}$), and (b) the Fourier transform of the sampled data, $X_s(f) = \frac{1}{T_s} \sum_n X(f - n/T_s)$, scaled by T_s, for $f_s = 1/T_s = 3f_{BW}$. The copies are separated by the gap $f_s - 2f_{BW}$ and do not overlap; the dashed box is the Nyquist band $|f| < f_s/2$. For $f_s < 2f_{BW}$ the copies would overlap and alias. Script `python/mkfig_sampleFT.py`.

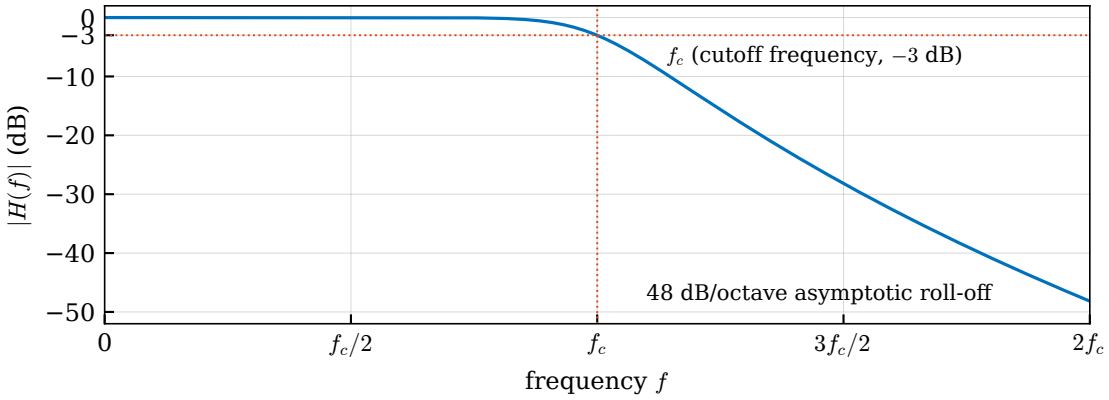


Figure 5.4: Example of an analog low-pass (anti-aliasing) filter frequency response function magnitude versus frequency: an eighth-order Butterworth filter, $|H(f)| = [1 + (f/f_c)^{16}]^{-1/2}$, which is -3 dB at the cutoff f_c, -48 dB at $2f_c$, and rolls off asymptotically at 48 dB per octave. Script `python/mkfig_antial.py`.

The results of this transform are shown in Fig. 5.8, which shows that windowing causes *spectral leakage* into neighboring frequencies.

Leakage cannot be removed, only redistributed. A tapered window function [88] reduces the leakage far from a spectral line, by lowering the sidelobes of $W(f)$, at the price of a wider main lobe (poorer resolution, more bias near the line) and a shorter effective record (larger variance of the estimate). Some of the widely used window functions are shown in Fig. 5.9; Section 6.2 quantifies the trade-off.

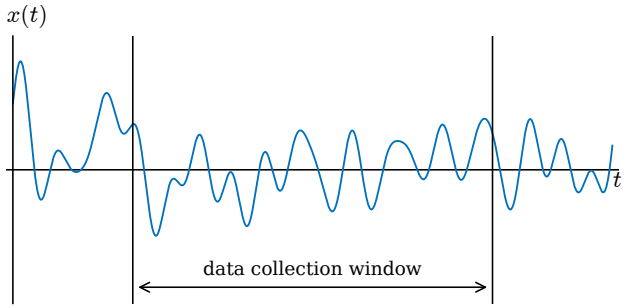


Figure 5.5: We only take a finite time record of data during the data collection window (the band-limited signal of Fig. 5.1, of which the 12 s between the vertical lines are kept). Script `python/mkfig_datacollw.py`.

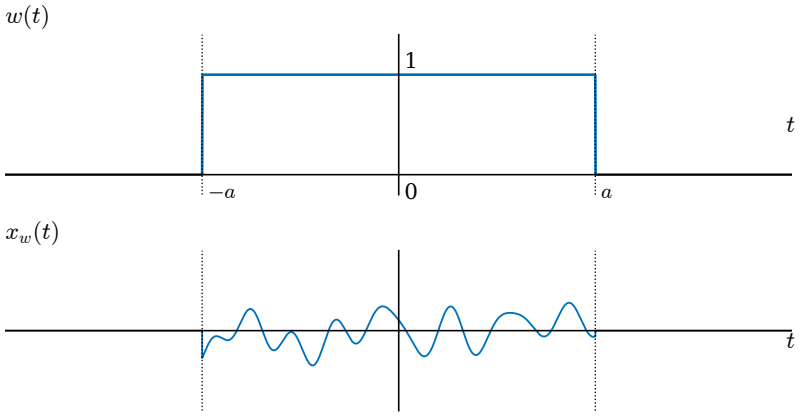


Figure 5.6: Rectangular window function $w(t) = 1$ for $|t| \le a$, 0 otherwise ($a = 5$ s), and the resultant observed signal $x_w(t) = w(t)x(t)$ for the band-limited signal of Fig. 5.1. Script `python/mkfig_windowed.py`.

5.2.1 Summary

The window functions shown here are just a few of the possible windows. Two numbers decide the choice: the width of the main lobe of $W(f)$, which sets the frequency resolution and the bias near a line, and the level and decay rate of the sidelobes, which set the leakage far from the line. Table 6.1 of Chapter 6 lists both for the common windows, computed from the window sequences themselves, and the discussion following it derives the recommendations collected in Table 5.1. These recommendations follow the window-selection guidance of the National Instruments application note on FFTs and windowing [144], from which the earlier version of this table was drawn; that note's claim that the Hann window "is satisfactory in 95 percent of cases" is folklore and is not repeated here. They are a starting point, not a rule. Always compare the performance of different window functions on the data at hand.

- Spectral leakage is caused by the finite record: a sine wave with a non-integer number of periods in the record is discontinuous when the record is wrapped, and its power spreads through the

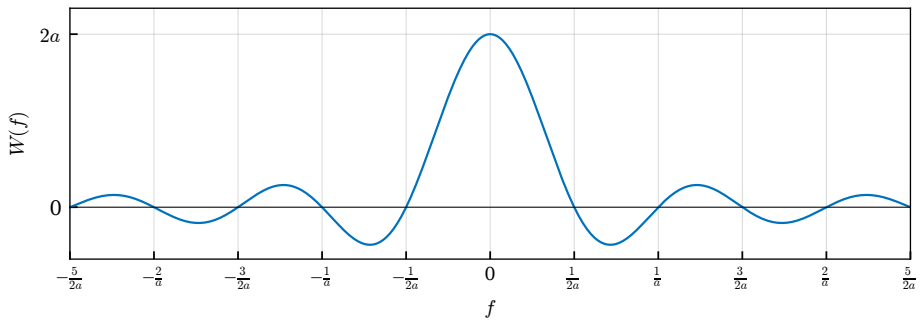


Figure 5.7: Fourier transform $W(f) = 2a\,\mathrm{sinc}(2\pi f a)$ of the rectangular window $w(t)$ of Fig. 5.6: peak $2a$, zeros at multiples of $1/(2a)$, and a first sidelobe of $-0.217 \times 2a$, i.e. 13.3 dB below the peak. Script `python/mkfig_sinc.py`.

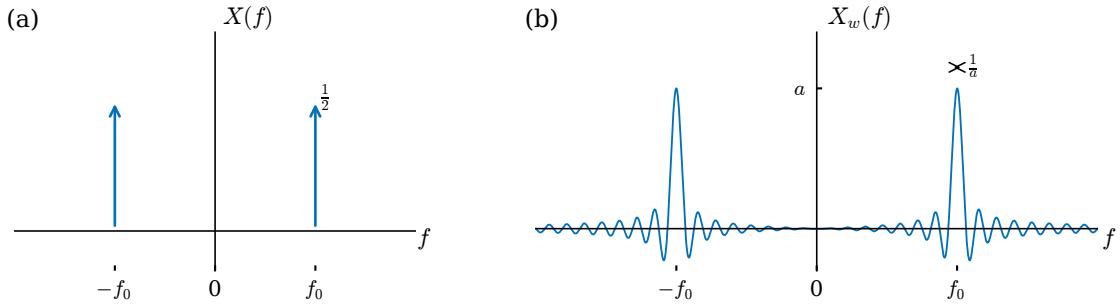


Figure 5.8: True Fourier transform $X(f) = \mathcal{F}(\cos 2\pi f_0 t)$, two deltas of weight $\frac{1}{2}$ (a), and windowed Fourier transform $X_w(f) = \mathcal{F}(w(t)\cos 2\pi f_0 t)$ for the rectangular window $|t| \le a$ with $f_0 = 8/a$ (b): each line becomes a main lobe of height a and width $1/a$ between zeros, with sidelobes (the first 12.9 dB down) that leak into every frequency. Script `python/mkfig_windowedcos.py`.

sidelobes of $W(f)$. Windowing reduces the far leakage; it does not remove it.

- Windowing reduces the amplitude of the discontinuities at the boundaries of each finite sequence acquired by the digitizer.
- "No window" is called the uniform or rectangular window because there is still a windowing effect.
- The Hann window is a reasonable default when the signal content is unknown: it has moderate resolution and sidelobes that decay as $1/f^3$.

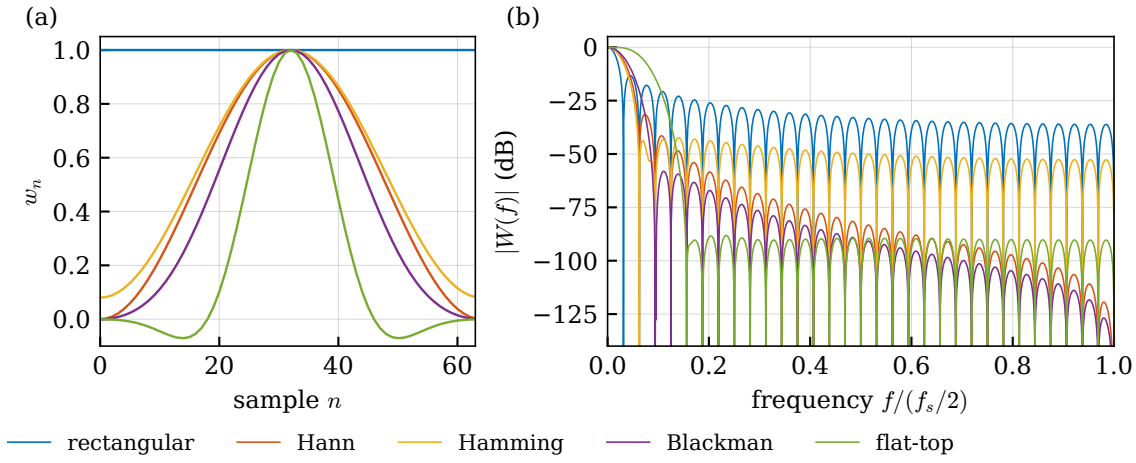


Figure 5.9: The five windows tabulated in Chapter 6 (Section 6.2, Table 6.1), $L = 64$ samples: (a) the window sequences w_n; (b) the magnitude of their transforms relative to the peak, over $0 \le f \le f_s/2$. The rectangular window has the narrowest main lobe (first zero one bin from the center) and the highest sidelobes (−13 dB); Hann (−31 dB) and Hamming (−42 dB) widen the lobe to two bins, Blackman (−58 dB) to three, and the flat-top window (−88 dB at this length) to five. Script `python/mkfig_windows.py`.

Signal Content	Window	Reason
Broadband random signal (smooth spectrum)	Rectangular	nothing to leak; narrowest lobe
Sine wave, or several with different amplitudes	Hann	fast sidelobe decay ($1/f^3$)
Sine wave, amplitude accuracy required	Flat top	flat main lobe across a bin
Two tones close in frequency, similar amplitude	Rectangular	narrowest main lobe
Two tones close in frequency, very different amplitude	Blackman, Kaiser	lowest sidelobes
Narrowband random signal (vibration data)	Hann	compromise resolution/leakage
Transient fully contained in the record	Rectangular	no discontinuity to suppress
Impact excitation / free decay (modal testing)	Force / Exponential	suppress noise after the event
Unknown content	Hann	compromise

Table 5.1: Initial window choices for common signal contents, as justified by the main-lobe and sidelobe figures of Table 6.1.

- Compare the performance of different window functions to find the best one for the application.

5.3 Discrete Fourier Transform

The continuous Fourier transform was defined in Eq. (4.9):

$$X(f) = \int_{-\infty}^{\infty} x(t)\, e^{-i2\pi ft}\, \mathrm{d}t\,. \tag{5.12}$$

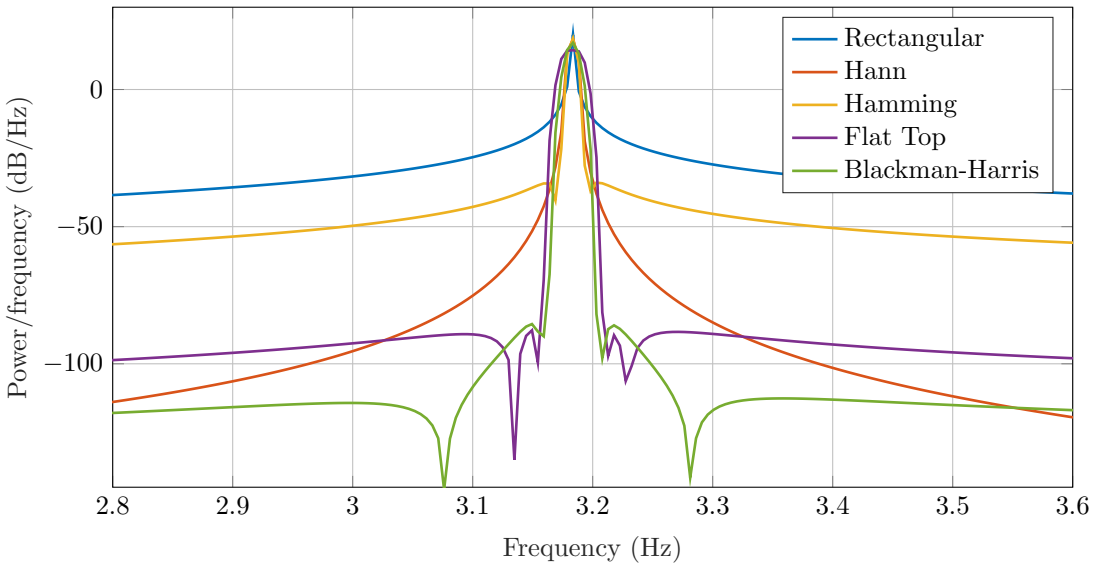


Figure 5.10: Some window function applied to sampled pure sine wave

However, we only measure over a *finite* time T, with equally-spaced samples with $\Delta t \equiv \Delta = \frac{T}{N}$, where N is the number of samples (see Fig. 5.11). Therefore,

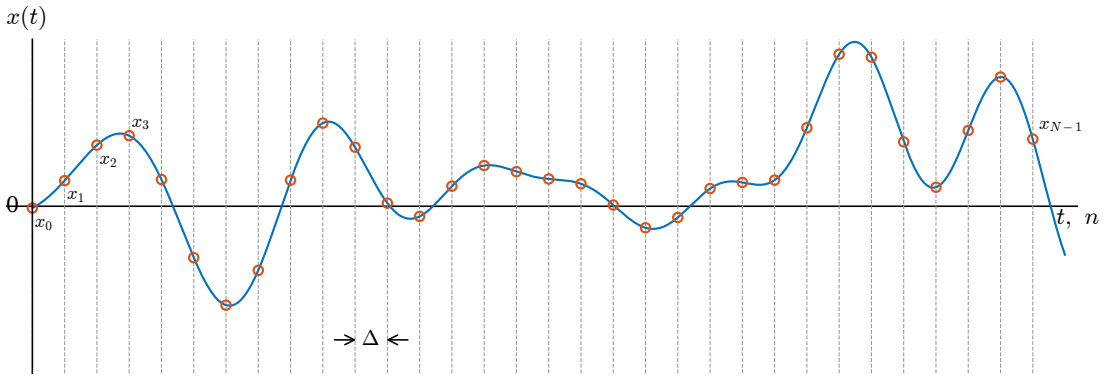


Figure 5.11: A finite record of $N = 32$ samples $x_n = x(n\Delta)$, $n = 0, \ldots, N-1$, of a continuous signal (the band-limited signal of Fig. 5.1, $\Delta = 0.25$ s, $T = N\Delta = 8$ s). Script `python/mkfig_sampleddata.py`.

$$X(f) \approx \int_0^T x(t)\, e^{-i2\pi f t}\, \mathrm{d}t \approx \sum_{n=0}^{N-1} x_n e^{-i2\pi f(n\Delta)} \Delta\,, \tag{5.13}$$

where we ignore the fact that $X(f)$ is no longer "true" $\mathcal{F}(x(t))$, and $x_n \equiv x(t_n) = x(n\Delta)$. Now, let $f = \frac{m}{T} \equiv f_m$ and $X(f_m) = \Delta X_m$ (Δ scaling is needed to have correct units), then we get

$$X_m = \frac{1}{\Delta} \sum_{n=0}^{N-1} x_n\, e^{-i2\pi \frac{m}{T}(n\Delta)} \Delta = \sum_{n=0}^{N-1} x_n e^{-i2\pi \frac{\Delta}{T}(mn)}\,. \tag{5.14}$$

However, $\Delta = \frac{T}{N}$, so $\frac{\Delta}{T} = \frac{1}{N}$. Then,

$$X_m = \sum_{n=0}^{N-1} x_n\, e^{-i2\pi \frac{mn}{N}}\,, \tag{5.15}$$

which is *discrete Fourier transform* or DFT.

The DFT returns X only at the bin frequencies $f_m = m\Delta f$, $\Delta f = \frac{m+1}{T} - \frac{m}{T} = \frac{1}{T}$. Values between bins are not lost, however: the finite record has a continuous transform $X_T(f) = \Delta \sum_n x_n e^{-i2\pi f n\Delta}$, and appending zeros to the record (*zero padding* to length $N' > N$) evaluates it on the finer grid $m/(N'\Delta)$. Zero padding interpolates the spectrum; it does not improve the resolution, which is fixed by T through the window of Section 6.1. Likewise, if the Nyquist condition holds, $x(t)$ between the samples $t_n = n\Delta$ is determined by sinc interpolation. Given $\{x_n\}_{n=0}^{N-1}$ we get $\{X_m\}_{m=0}^{N-1}$, and

$$x_n = \frac{1}{N} \sum_{m=0}^{N-1} X_m\, e^{\frac{i2\pi nm}{N}}\,, \tag{5.16}$$

which is called *inverse discrete Fourier transform* (IDFT) and *discrete* values are transformed *exactly*.

We can prove the above as follows. Given

$$X_m = \sum_{p=0}^{N-1} x_p\, e^{-i2\pi \frac{mp}{N}}\,. \tag{5.17}$$

we can write:

$$
\begin{aligned}
x_n &= \frac{1}{N}\sum_{m=0}^{N-1}\left(\sum_{p=0}^{N-1} x_p\, e^{-i2\pi\frac{mp}{N}}\right) e^{i2\pi\frac{nm}{N}}\,, \\
&= \frac{1}{N}\sum_{p=0}^{N-1} x_p\left(\sum_{m=0}^{N-1} e^{i2\pi\frac{m(n-p)}{N}}\right)\,, \qquad \left(\sum_{m=0}^{N-1} e^{i2\pi\frac{m(n-p)}{N}} = \begin{cases} 0 & \text{for } n\neq p\,, \\ N & \text{for } n=p\,. \end{cases}\right) \\
&= \frac{1}{N}x_n N = x_n\,. \qquad \square
\end{aligned}
\tag{5.18}
$$

In conclusion, the *discrete and inverse discrete Fourier transforms are exact transforms.*

5.3.1 Notes on Fast Fourier Transforms

Fast Fourier Transform (FFT) is a fast algorithm for DFT (see, e.g., [158, Ch. 12]). It has the following requirements and parameters:

- Time records of N points, Δt between points must satisfy $\frac{1}{\Delta t} \geq 2f_{BW}$ of signal.
- Simplest and fastest implementation is when $N = 2^m$, so data records (buffers) are usually 2^m points long (1024, 4096, ...); modern FFT libraries handle any N efficiently, but powers of two remain the convention.
- We get $\frac{N}{2}$ lines in the frequency domain. Heuristically, the N real data points give $\frac{N}{2}$ complex (amplitude and phase) data points in the frequency domain.
- Δf in the FFT is $\frac{1}{T}$ of data record. Thus, $f_{\max} = \frac{N}{2}\Delta f = \frac{N}{2T}$. However, $T = N\Delta t$ and, thus, $f_{\max} = \frac{1}{2\Delta t} = \frac{f_s}{2}$.

Problems

Problem 5.1

Computational

Consider the driven, two-well Duffing's oscillator

$$\ddot{x} + \epsilon\gamma\dot{x} - x + x^3 = \epsilon F\cos(\omega t)$$

which can be written in state variable form as

$$\begin{aligned}\dot{x} &= v\\ \dot{v} &= x - x^3 + \epsilon(-\gamma v + F\cos(\omega t)).\end{aligned}$$

In the above expressions, γ is the linear (viscous) damping coefficient, F is the forcing amplitude, and ω is the forcing frequency.[1]

Develop a MATLAB or Python code to simulate this system. You need to be able to generate uniformly sampled time series of any length, with any sampling time t_s (recall that the sampling frequency is $f_s = 1/t_s$, and be very careful when choosing your sampling time).

Experiment with the simulations, and by varying the system parameters and examples of:

1. a periodic, approximately simple harmonic response;
2. a periodic response that is not simple harmonic; and
3. a chaotic (i.e., aperiodic) response.

Plot the *steady state* results in the time domain and in the x–v phase plane. To help you get started, reasonable values for the parameters to use in the simulations are $\epsilon = 0.1$, $\gamma \in [0.7; 2.0]$, $\omega \in [0.9; 1.1]$, $F \in [1.5; 2.8]$.

Problem 5.2

needs Ch. 6 Computational

Using the time series $\{x_i\}$ generated in Problem 5.1, make an observable via $\{y_i\} = \{x_i + n_i\}$, where the n_i are independent identically distributed (IID) mean zero Gaussian random variable with variance σ^2 (be careful when choosing your parameters—we want to approximate real-life situations). Find the digital Power Spectral Density (PSD) for the three types of response found in Problem 5.1, using the estimators of Sections 6.1 and 6.2 (MATLAB `pwelch` or `periodogram`; SciPy `signal.welch` or `signal.periodogram`). By experimentation, examine the effect of σ, t_s, the number of points used in the FFT, and windowing (including no window, which is called `rectwin` in MATLAB). A good window to use is the Hann; however, you can experiment to see the differences. Type `help window` at the command line for more information. Plot your results using a semilog or Decibel (dB) scale. Discuss your results: what do you see?

For this problem see Guckenheimer and Holmes [82, Ch. 4.5, Eq. (4.5.18)].

[1] The parameter ϵ is included here only for convenience when comparing our results to certain theoretical treatments. In particular, the damping and forcing are scaled by ϵ to indicate that the oscillator can be thought of as a perturbation of a conservative system, with the conservative system corresponding to the case $\epsilon = 0$. See, for example, Guckenheimer and Holmes (1983).

Problem 5.3

needs Ch. 6 Computational

Aliasing of a periodically forced response.

1. A harmonic at frequency kf_0 is sampled at $f_s = nf_0$ samples per forcing period. Show from $X_s(f) = \frac{1}{T_s}\sum_m X(f - m/T_s)$ that it appears in the band $[0, f_s/2]$ at the frequency $|kf_0 - mf_s|$ for the integer m that makes this smallest. Tabulate where the harmonics $k = 1, \ldots, 5$ land for $n = 3$, 4 and 6. You should find that with $n = 3$ the third harmonic lands on zero frequency and the second, fourth and fifth all land on the fundamental.

2. Generate the two-well Duffing x record of Section 1.4 with 30 samples per forcing period, then keep every tenth sample to obtain a record with $n = 3$. Estimate both spectra by Welch's method. Fold the finely sampled two-sided spectrum onto the coarse Nyquist band by summing $S_{xx}(\pm f + mf_s)$ over m, and compare with the spectrum of the decimated record. You should find agreement to within a few percent at every bin except the one at the coarse Nyquist frequency, and a coarse spectrum that departs from the simply restricted fine spectrum mainly near that Nyquist frequency, where the folded broadband floor is added.

3. The chaotic record is broadband, so the folding raises the whole coarse spectrum rather than producing spurious lines. Repeat part (2) for a *periodic* response of the oscillator of Problem 5.1 and show that now spurious lines do appear, at the frequencies predicted in part (1).

4. Which of the two effects, folded floor or spurious lines, can be removed by an anti-aliasing filter applied before sampling, and which cannot be removed after the fact? Given a record already sampled at $n = 3$, is there any way to tell that aliasing has occurred?

Problem 5.4

needs Ch. 6 Core

Amplitude accuracy of a windowed line.

1. A sine of frequency f_0 is sampled for N points. Show that when f_0 lies exactly on a DFT bin, $f_0 = m/T$, the rectangular-window periodogram has a single nonzero bin at m (apart from the mirror at $-m$), and that when f_0 falls halfway between bins the largest bin is reduced by the factor $|\mathrm{sinc}(\pi/2)| = 2/\pi$, a loss of 3.9 dB (the *scalloping loss*).

2. Confirm numerically with $N = 1024$: compute the peak bin magnitude for $f_0 = 100/T$ and $f_0 = 100.5/T$ for the rectangular, Hann and flat-top windows. You should find losses of about 3.9, 1.4 and 0.01 dB respectively. Relate the ordering to the main-lobe widths in Table 6.1.

3. The half-bin case also spreads the line's power over its neighbors. Show numerically that the sum of $|X_m|^2$ over *all* bins is the same wherever f_0 falls (Parseval), and that the sum over the peak bin and its four neighbors on each side recovers all but 0.1% of it for the Hann window but loses about 5% for the rectangular window at the half-bin offset. Where has the missing rectangular-window power gone, and why is the peak height not conserved while the total is?

4. A vibration test reads the amplitude of a resonance line from the peak bin of a Hann spectrum without knowing where the line falls relative to the bins. State the worst-case amplitude error, and two ways of removing it (one by choice of window, one by processing).

Problem 5.5

Core

Zero padding interpolates; it does not resolve.

1. Verify numerically that the DFT and the IDFT are exact inverses: for a random record of $N = 1000$ samples compute $\mathrm{IDFT}(\mathrm{DFT}(x))$ and report the largest deviation from x (you should find it at the level of machine precision, about 10^{-15}).

2. Sample $x(t) = \cos(2\pi\, 50\, t) + \cos(2\pi\, 50.5\, t)$ at 1 kHz for $T = 1$ s. Compute the magnitude of the DFT with no padding and with the record padded by zeros to sixteen times its length. Locate the local maxima between 48 and 53 Hz in each. You should find a single maximum in both cases: the padded transform has more points but the two tones are not separated.

3. Repeat with $T = 4$ s. Now two maxima appear at 50 and 50.5 Hz. State the resolution rule that the two cases illustrate, in terms of T and the frequency separation.

4. Zero padding does have a use. With $T = 4$ s and a single tone at 50.3 Hz, compare the frequency read from the peak bin without padding and with padding by a factor of sixteen, and with the true value. Which error is reduced by padding, the bin-quantization error or the resolution?

Problem 5.6

Experimental

Sampling and windowing of the two-well oscillator record of Section 1.4.1.

1. From the sampling rate and forcing frequency given with the record, compute the number of samples per forcing period and the Nyquist frequency in units of the forcing frequency. How many harmonics of the forcing lie below the Nyquist frequency? Is the sampling rate consistent with the practical rule $f_s \geq 4 f_{BW}$ of the anti-aliasing discussion in Section 5.1, taking f_{BW} as the highest frequency at which the spectrum of Problem 4.6 stands clearly above its floor?

2. Compute the DFT of a segment of exactly 2^{12} samples with the rectangular window and with the Hann window, and plot both magnitudes in decibels around the forcing line. Read the leakage floor a few bins away from the line in each case. Does the forcing frequency fall on a bin? Compute the offset in fractions of a bin from $f_0 T$, where T is the segment duration, and estimate the scalloping loss of Problem 5.4 for each window.

3. Choose the segment length N so that the forcing frequency falls as close to a bin as the record allows, i.e., so that $f_0 N / f_s$ is nearly an integer. Repeat part (2) with that N. By how many decibels does the rectangular-window leakage floor drop, and does the Hann floor change?

4. Decimate the record by keeping every k-th sample, for the k that gives about three samples per forcing period, and compare the spectrum with the original as in Problem 5.3. Which harmonics of the forcing alias onto which frequencies, and does the record's anti-aliasing filter (if the metadata report one) protect against this?

6
Spectral Estimation, Cross-Spectra, and Frequency Response

Chapters 3–5 defined the autocorrelation, the power spectral density, and the discrete Fourier transform, and showed what sampling and finite record length do to a transform. They did not say how a spectrum is *estimated* from one finite, noisy record, nor how good the estimate is. The distinction matters. The quantity $S_{xx}(f)$ of Eq. (4.31) is an expectation over an ensemble and a limit in T; a laboratory supplies a single record of N samples, and the obvious estimator built from it—the periodogram—does not converge to $S_{xx}(f)$ as N grows. Every spectral plot in the rest of this book, and every quantity derived from one, rests on knowing why, and on the averaging that repairs it.

This chapter supplies that estimation theory and extends it in two directions that later chapters need. The first is the statistics of correlated samples: a stationary record of N correlated values carries fewer than N independent pieces of information, which changes the variance of every sample statistic computed from it, from the mean of Chapter 2 to the autocorrelation of Chapter 3. The Theiler window of Chapters 11 and 12 and the surrogate tests of Chapter 16 are the nonlinear descendants of this one fact. The second direction is the pair of records. The cross-spectrum, the coherence, and the frequency response function of a linear input–output system are defined and their estimators analyzed, with the single-degree-of-freedom oscillator as the running example. Coherence also serves as a screening test for nonlinearity, and the frequency response function is the starting point of experimental modal analysis [58]; both rest on the estimators developed here.

The treatment follows Bendat and Piersol [8] and Jenkins and Watts [101] in substance, in the notation of Chapters 4 and 5: the two-sided density $S_{xx}(f)$, the one-sided $G_{xx}(f) = 2S_{xx}(f)$ for $f > 0$, sampling interval Δt, N samples, record length $T = N\Delta t$, and DFT coefficients $X_k = \sum_{n=0}^{N-1} x_n e^{-2\pi i k n/N}$ at the frequencies $f_k = k/T$.

6.1 The Periodogram

The periodogram is the sample version of Eq. (4.31) with the expectation and the limit removed. It is unbiased in the limit, but its variance does not shrink with record length.

6.1.1 Definition and scaling

Chapter 5 showed that for a record sampled at Δt the finite-time transform of Eq. (4.22) is approximated at the bin frequencies by $X_T(f_k) \approx \Delta t\, X_k$. Substituting into Eq. (4.31) without the expectation or the limit gives the *periodogram*, first used by Schuster [183] to search for hidden periodicities:

$$\hat{S}_{xx}(f_k) = \frac{1}{T}\,|\Delta t\, X_k|^2 = \frac{\Delta t}{N}\,|X_k|^2\,, \qquad f_k = \frac{k}{N\Delta t}\,, \quad k = 0, 1, \ldots, N-1\,. \tag{6.1}$$

The units are right: X_k has the units of x, and $\Delta t/N$ has units of seconds, so $\hat{S}_{xx}$ has units of x^2/Hz, the same as $S_{xx}(f)$. The estimate is a two-sided density on the N bins $0 \le f_k < f_s$; the bins above the Nyquist frequency $f_N = f_s/2$ are the negative frequencies $f_k - f_s$, and for real x_n they duplicate the positive ones. Summing the estimate over all N bins with the bin width $\Delta f = 1/T$ recovers the mean square of the record exactly,

$$\sum_{k=0}^{N-1} \hat{S}_{xx}(f_k)\,\Delta f = \frac{\Delta t}{N} \cdot \frac{1}{N\Delta t} \sum_{k=0}^{N-1} |X_k|^2 = \frac{1}{N} \sum_{n=0}^{N-1} x_n^2\,, \tag{6.2}$$

which is the discrete form of Parseval's identity, Eq. (4.35); the second equality is the DFT identity $\sum_k |X_k|^2 = N \sum_n x_n^2$. In practice the sample mean is subtracted first, so that the $k = 0$ bin does not carry $N\Delta t\,\bar{x}^2$ and leak into its neighbors.

The one-sided estimate, which is what one plots, doubles the interior bins:

$$\hat{G}_{xx}(f_k) = \begin{cases} \hat{S}_{xx}(f_k)\,, & k = 0 \text{ or } k = N/2\,, \\ 2\hat{S}_{xx}(f_k)\,, & 0 < k < N/2\,, \end{cases} \tag{6.3}$$

so that $\sum_{k=0}^{N/2} \hat{G}_{xx}(f_k)\Delta f$ again equals the mean square. Software differs on whether it returns $\hat{S}$ or $\hat{G}$, whether it divides by f_s (a density, x^2/Hz) or not (a "power spectrum," x^2 per bin), and on the treatment of the window normalization introduced below. The two checks that settle the convention are the units and Eq. (6.2): a density integrates to the variance, a power spectrum sums to it.

6.1.2 Expectation: the spectral window

The expectation of the periodogram was computed in Chapter 4 on the way to the Wiener–Khinchin theorem. Equation (4.29), divided by T, states that for a rectangular window of length T

$$\mathrm{E}\left[\hat{S}_{xx}(f)\right] = \int_{-T}^{T} \left[1 - \frac{|\tau|}{T}\right] R_{xx}(\tau)\; e^{-2\pi i f \tau}\,\mathrm{d}\tau = \int_{-\infty}^{\infty} S_{xx}(\nu)\; W_B(f-\nu)\,\mathrm{d}\nu\,, \tag{6.4}$$

where the second form follows from the convolution property of Chapter 4: the transform of a product is the convolution of the transforms, and the transform of the triangle $1 - |\tau|/T$ on $[-T, T]$ is the *Fejér kernel*

$$W_B(f) = T\,\frac{\sin^2(\pi f T)}{(\pi f T)^2} = T\,\mathrm{sinc}^2(\pi f T)\,, \qquad \int_{-\infty}^{\infty} W_B(f)\,\mathrm{d}f = 1\,. \tag{6.5}$$

Chapter 4 let $T \to \infty$ here, under which the kernel tends to a delta function and the expectation to $S_{xx}(f)$. At finite T the expectation is the true spectrum *smoothed* by a kernel of unit area whose main lobe has width $2/T$ between its first zeros and whose sidelobes decay only as $1/f^2$. The Fejér kernel is the square of the sinc transform of the rectangular window derived in Chapter 5, normalized by T; the estimate is therefore biased wherever $S_{xx}(f)$ varies appreciably over a few bins, and the bias has two components. *Resolution bias* is the smoothing by the main lobe: peaks are lowered and broadened, valleys are filled. *Leakage* is the transfer of power through the sidelobes from frequencies far from f; it is invisible where the spectrum is flat and dominant where a strong component sits next to a weak one.

Tapering the record before transforming changes the kernel. For a window w_n applied to the samples, the *modified periodogram* is

$$\hat{S}_{xx}(f_k) = \frac{\Delta t}{\sum_{n=0}^{N-1} w_n^2} \left| \sum_{n=0}^{N-1} w_n x_n \, e^{-2\pi i k n/N} \right|^2 , \tag{6.6}$$

and its expectation is Eq. (6.4) with the Fejér kernel replaced by the normalized *spectral window*

$$W_w(f) = \frac{|W(f)|^2}{\Delta t \sum_n w_n^2} , \qquad W(f) = \Delta t \sum_{n=0}^{N-1} w_n e^{-2\pi i f n \Delta t} , \tag{6.7}$$

which again has unit area over one period $[-f_s/2, f_s/2]$; this is what the denominator $\sum_n w_n^2$ in Eq. (6.6) guarantees, and it is the normalization under which a white spectrum is estimated without bias by any window. Rectangular $w_n = 1$ recovers Eq. (6.1) and, for $\Delta t \sum_n w_n^2 = T$, the Fejér kernel. The windows of Chapter 5 trade a wider main lobe for lower sidelobes; Fig. 6.1 shows the kernels and Section 6.2 tabulates them.

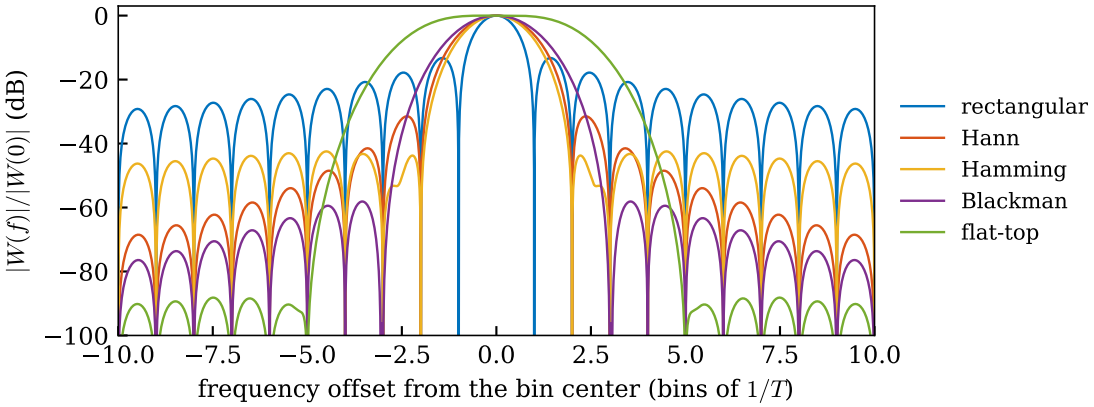


Figure 6.1: Spectral windows $|W(f)|$, normalized to their peak, for five tapers of length $L = 64$ samples, against frequency offset from the bin center in units of the bin width $1/T$. The rectangular window has the narrowest main lobe and the highest sidelobes (-13 dB, decaying as $1/f$); the Hann, Hamming and Blackman windows buy progressively lower sidelobes with a wider main lobe; the flat-top window is designed for a main lobe flat to within 0.01 dB across a full bin, at the cost of a lobe nearly four bins wide.

A signal with a line spectrum makes the kernel visible. For $x(t) = A\cos(2\pi f_0 t)$, $R_{xx}(\tau) = \frac{1}{2}A^2\cos(2\pi f_0\tau)$ and $S_{xx}(f) = \frac{1}{4}A^2[\delta(f-f_0)+\delta(f+f_0)]$, so Eq. (6.4) gives $\mathrm{E}\big[\hat{S}_{xx}(f)\big] = \frac{1}{4}A^2[W_w(f-f_0)+W_w(f+f_0)]$: the periodogram of a sinusoid is the spectral window itself, centered on $\pm f_0$, with peak height $\frac{1}{4}A^2T$ for the rectangular window. The peak grows without bound as $T \to \infty$, which is the density of a line, and the area under it is the power $\frac{1}{4}A^2$ in each of the two lines, independent of T. This is the point noted after Eq. (4.30): for a periodic or quasi-periodic record $R_{xx}(\tau)$ does not decay, $S_{xx}(f)$ exists only as a sum of lines, and it is the power under a peak, not the height, that means anything. If f_0 falls on a bin, $f_0 = k_0/T$, the rectangular kernel is sampled at its zeros and the periodogram is exactly two nonzero bins; if f_0 falls between bins, every bin receives power through the $1/f$ sidelobes. Fig. 6.2 shows both cases.

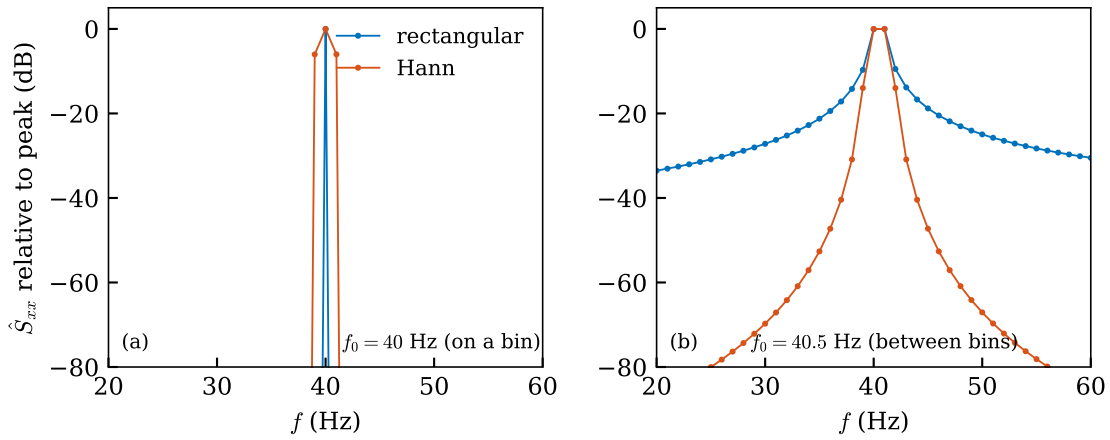


Figure 6.2: Periodogram of a unit-amplitude sinusoid, $N = 256$ samples at $f_s = 256$ Hz (bins 1 Hz wide), normalized to its peak, with the rectangular and the Hann window. (a) $f_0 = 40$ Hz lies on a bin: the rectangular window samples its own kernel at the zeros and leaks nothing; the Hann window spreads the line over three bins. (b) $f_0 = 40.5$ Hz lies midway between bins: the rectangular window leaks into every bin at a level decaying as $1/f$ (the visible "skirt" is -13 dB at the nearest bins and still -40 dB twenty bins away), while the Hann leakage falls below -60 dB within five bins.

6.1.3 Variance: the periodogram is inconsistent

For a Gaussian record the DFT coefficients X_k are complex Gaussian variables, since they are linear combinations of the x_n. For a spectrum smooth over the main lobe, the real and imaginary parts of X_k at an interior bin are uncorrelated with equal variance, and the periodogram, proportional to $|X_k|^2$, is proportional to the sum of two squared independent Gaussians. Hence

$$\hat{S}_{xx}(f_k) \sim S_{xx}(f_k)\,\frac{\chi^2_2}{2}\,, \qquad \mathrm{E}\Big[\hat{S}_{xx}(f_k)\Big] = S_{xx}(f_k)\,, \qquad \mathrm{Var}\Big[\hat{S}_{xx}(f_k)\Big] = S_{xx}(f_k)^2\,, \tag{6.8}$$

a chi-square variable with two degrees of freedom, scaled to have the right mean [101]. At $k = 0$ and $k = N/2$ the coefficient is real, the variable is χ^2_1, and the variance is $2S_{xx}(f_k)^2$. Nothing in Eq. (6.8) depends on N. Lengthening the record adds bins and narrows the kernel; it does not reduce the scatter of any one bin, which remains of the order of the quantity being estimated. The periodogram is an *inconsistent* estimator: its bias vanishes as $N \to \infty$ but its variance does not. The ragged trace in Fig. 6.3(a) is the visible consequence, and it would be equally ragged for a record ten times longer.

Periodogram statistics by simulation

For 4000 independent Gaussian white-noise records of $N = 256$ samples at $\Delta t = 0.01$ s with variance chosen so that $S_{xx}(f) = S_0 = 3\ x^2/\mathrm{Hz}$, the periodogram Eq. (6.1) over the interior bins had sample mean $0.997\,S_0$ and sample variance $0.995\,S_0^2$; at $k = 0$ and $k = N/2$ the variance was $2.02\,S_0^2$ and $1.97\,S_0^2$. The 5%, 50% and 95% quantiles of $\hat{S}/S_0$ were 0.051, 0.689 and 2.994, against 0.051, 0.693 and 2.996 for $\chi^2_2/2$. Repeating with $N = 64$ and $N = 1024$ gave variances of $1.009\,S_0^2$ and $1.000\,S_0^2$: the variance is independent of the record length. A single periodogram ordinate lies within a factor of two of the truth with probability $P\left(\tfrac{1}{2} < \chi^2_2/2 < 2\right) = e^{-1/2} - e^{-2} \approx 0.47$.

The variance can be reduced only by averaging, and averaging can be paid for only in resolution or in record length. The next section develops the accounting.

6.2 Averaged Estimates: Bartlett and Welch

Averaging K periodograms of independent records divides the variance by K. A single record of N samples is split into K segments of $L = N/K$ samples each, and the price is a spectral window K times wider.

6.2.1 Segment averaging

Bartlett [7] proposed splitting the record into K contiguous non-overlapping segments $x_n^{(j)} = x_{n+(j-1)L}$, $n = 0, \dots, L-1$, and averaging their periodograms,

$$\hat{S}_{xx}(f_k) = \frac{1}{K} \sum_{j=1}^{K} \frac{\Delta t}{L} \left| X_k^{(j)} \right|^2 , \qquad f_k = \frac{k}{L\Delta t} . \tag{6.9}$$

Each term has the expectation Eq. (6.4) with the Fejér kernel of a record of length $L\Delta t = T/K$, so the resolution is coarsened from $1/T$ to K/T. If the segments are long compared with the correlation time of x, their periodograms are nearly independent, each is $S_{xx}(f_k)\,\chi^2_2/2$, and their average is a scaled chi-square with $\nu = 2K$ degrees of freedom:

$$\hat{S}_{xx}(f_k) \sim S_{xx}(f_k)\ \frac{\chi^2_\nu}{\nu} , \qquad \nu = 2K , \qquad \mathrm{Var}\left[\hat{S}_{xx}(f_k)\right] = \frac{S_{xx}(f_k)^2}{K} , \qquad \epsilon_r = \frac{\sqrt{\mathrm{Var}\left[\hat{S}\right]}}{S_{xx}(f)} = \frac{1}{\sqrt{K}} . \tag{6.10}$$

The *normalized random error* ϵ_r is the standard deviation of the estimate divided by the quantity estimated; Bendat and Piersol [8] write it $1/\sqrt{n_d}$ with n_d the number of averages, and it is the number to quote with any spectral plot. A $(1-\alpha)$ confidence interval follows from the chi-square distribution,

$$\frac{\nu\,\hat{S}_{xx}(f)}{\chi^2_{\nu;\,1-\alpha/2}} \le S_{xx}(f) \le \frac{\nu\,\hat{S}_{xx}(f)}{\chi^2_{\nu;\,\alpha/2}} , \tag{6.11}$$

where $\chi^2_{\nu;p}$ is the p-quantile. The interval is a constant *ratio*, not a constant offset, which is why spectra are drawn on a logarithmic axis: the confidence band then has the same height everywhere. For $\nu = 28$ (the estimate of Fig. 6.3(b)) the 95% interval is $[0.63, 1.83]$ times the estimate; for $\nu = 200$ it is $[0.82, 1.24]$.

6.2.2 Welch's method

Two changes to Eq. (6.9) make it the estimator in universal use [221]. Each segment is tapered by a window w_n and its modified periodogram Eq. (6.6) is used, which lowers the leakage of each term; and the segments are overlapped by D samples, which recovers the information the taper discards near the segment ends. With overlap the number of segments that fit in N samples is

$$K = \left\lfloor \frac{N-L}{L-D} \right\rfloor + 1 \,, \tag{6.12}$$

which for 50% overlap ($D = L/2$) is nearly twice the number without overlap. Overlapped segments share data, so their periodograms are correlated and the variance does not fall as $1/K$. Welch showed that, for Gaussian data,

$$\operatorname{Var}\left[\hat{S}_{xx}(f)\right] \approx \frac{S_{xx}(f)^2}{K}\left[1 + 2\sum_{j=1}^{K-1}\left(1-\frac{j}{K}\right)c_j^2\right], \qquad c_j = \frac{\sum_n w_n w_{n+j(L-D)}}{\sum_n w_n^2}\,, \tag{6.13}$$

where c_j is the correlation coefficient between the window and its copy shifted by j segment steps, and the sum runs over the shifts for which the copies still overlap. For the Hann window at 50% overlap $c_1 = 1/6$ and $c_j = 0$ for $j \geq 2$, so the bracket is $1 + 2/36 \approx 1.06$: the overlapped estimate behaves almost as if its $K \approx 2N/L$ segments were independent, and the degrees of freedom are $\nu \approx 2K/1.06 \approx 1.9\,K$, nearly double what the same record gives without overlap at the same resolution. For the rectangular window at 50% overlap $c_1 = 1/2$ and the bracket is 1.5; overlapping untapered segments gains little. Overlap beyond about 50% for Hann (or 75% for the smoother windows) gains nothing further, since the additional segments are nearly copies of those already counted.

Welch variance by simulation

For 1000 Gaussian white-noise records of $N = 8192$ samples, segment length $L = 256$: Bartlett's estimate (rectangular, no overlap, $K = 32$) had $K \operatorname{Var}\left[\hat{S}\right]/S_0^2 = 0.99$ and equivalent degrees of freedom $2\mathrm{E}\left[\hat{S}\right]^2/\operatorname{Var}\left[\hat{S}\right] = 64.5$ against $2K = 64$. Welch's estimate with a Hann window and 50% overlap ($K = 63$ by Eq. (6.12)) had $K \operatorname{Var}\left[\hat{S}\right]/S_0^2 = 1.054$, against 1.056 from Eq. (6.13), and 119 degrees of freedom against 126 for independent segments; the same window without overlap gave 64. The rectangular window at 50% overlap gave $K \operatorname{Var}\left[\hat{S}\right]/S_0^2 = 1.49$ against 1.5. In every case the mean was S_0 to within 0.1%.

Welch's averaged modified periodogram

Given x_n, $n = 0, \dots, N-1$, sampled at Δt; a segment length L, overlap D, and window w_n of length L:

1. Form the K segments of Eq. (6.12), $x_n^{(j)} = x_{n+(j-1)(L-D)}$, and remove the mean of each.
2. Compute $X_k^{(j)} = \sum_{n=0}^{L-1} w_n x_n^{(j)} e^{-2\pi i k n/L}$ by FFT.
3. Average: $\hat{S}_{xx}(f_k) = \dfrac{\Delta t}{K \sum_n w_n^2} \displaystyle\sum_{j=1}^{K} \left| X_k^{(j)} \right|^2$, at $f_k = k/(L\Delta t)$.
4. Report $\hat{G}_{xx}$ by Eq. (6.3) for $0 \le k \le L/2$, together with the resolution $\Delta f = 1/(L\Delta t)$ and the random error $\epsilon_r \approx 1/\sqrt{K}$ (times 1.03 for Hann at 50% overlap).

The routine `psdwelch` of Appendix A.12 implements these steps, and the cross-spectral extensions of Section 6.5.

6.2.3 The bias–variance–resolution trade-off

For a record of fixed length, the segment length L is the only free choice, and it sets everything: the resolution $\Delta f = 1/(L\Delta t)$, the number of averages $K \approx 2N/L$ at 50% overlap, and hence the random error $\epsilon_r \approx 1/\sqrt{K} \approx \sqrt{L/2N}$. Halving L halves the variance and doubles the width of the spectral window. The bias is the smoothing of Eq. (6.4) by that window, and it is significant wherever the spectrum has features narrower than a few bins. Bendat and Piersol's rule is that the narrowest feature of interest—for a resonance, the half-power bandwidth B_r of Section 6.6—should span at least four bins, $\Delta f \le B_r/4$, before the peak height is trusted; this fixes the minimum L, and the record length N then fixes the random error. The product $B_r T$ is the number of independent samples the record contains at that resolution, and no choice of window or overlap changes it.

Pitfall: under-resolved resonances

An estimate whose bins are wider than the resonance it is meant to measure returns a peak that is too low and too wide, and the damping ratio inferred from the half-power bandwidth is too high. In the checks for Section 6.6, an oscillator with $B_r = 0.20$ Hz estimated with $\Delta f = 0.098$ Hz and a Hann window returned a peak 17% low, with a coherence of 0.82 at resonance where the adequately resolved estimate gave 0.98. The AR(2) example below shows the same thing: with $L = 256$, two bins across the half-power width, the mean Welch estimate of the peak was 19% low; with $L = 512$, four bins, 4% low; with $L = 1024$, 2%. Both symptoms are cured by lengthening L; neither is cured by more averaging. Compare the same oscillator, adequately resolved, in Fig. 6.7.

6.2.4 Window choice

Table 6.1 gives, for the five windows of Fig. 6.1, the two numbers that decide between them: the main-lobe width, which sets the resolution bias, and the peak sidelobe level, which sets the leakage from distant components. The third column is the *equivalent noise bandwidth*, $B_e = L \sum_n w_n^2 / (\sum_n w_n)^2$ bins: the width of an ideal rectangular filter passing the same white-noise power as the window, and the factor by which a line's power is spread across bins. Harris [88] tabulates these and a dozen

other figures of merit for some forty windows; the values below were computed directly from the window sequences.

Table 6.1: Figures of merit of common windows, computed from sequences of length $L = 1024$. Widths are in bins of $1/T$; the 3 dB width is the full width of the main lobe at half power. The flat-top window is the five-term cosine series used by MATLAB and SciPy.

Window	3 dB width (bins)	Equiv. noise bandwidth (bins)	Peak sidelobe (dB)	Sidelobe decay
Rectangular	0.89	1.00	-13	$1/f$
Hann	1.44	1.50	-31	$1/f^3$
Hamming	1.30	1.36	-43	$1/f$
Blackman	1.64	1.73	-58	$1/f^3$
Flat-top	3.72	3.77	-93	—

The table replaces the folklore of Chapter 5 with arithmetic. For a broadband random signal with a smooth spectrum there is nothing to leak, the rectangular window has the narrowest lobe, and it is the correct choice; the Hann window costs 50% in resolution for nothing. For a spectrum with a strong narrow component beside a weak one, the rectangular sidelobes at -13 dB decaying as $1/f$ bury everything within 40 dB of the peak for tens of bins, and the Hann or Blackman window is required. The Hamming window has a lower first sidelobe than the Hann but its sidelobes decay as $1/f$, so far from the peak it leaks more. The flat-top window exists for one purpose, reading the amplitude of a line whose frequency is not on a bin: its main lobe is flat across a full bin so the peak value does not depend on where the line falls, at the cost of a lobe nearly four bins wide. Windowing does not remove leakage; it moves power from the far sidelobes into a wider main lobe, and it reduces the effective record length, so that the estimate has roughly $1/B_e$ of the degrees of freedom the same segment would have untapered. This is the accounting behind the recommendations of Table 5.1.

Segment averaging is not the only way to buy degrees of freedom. Section 6.3 develops the alternative, which tapers the whole record several times instead of cutting it into segments.

6.2.5 Worked example: the autoregressive process of Chapter 3

The colored process of Fig. 3.2 is the second-order autoregression

$$x_{n+1} = a_1 x_n + a_2 x_{n-1} + \eta_n\,, \qquad a_1 = 2 - (\pi/10)^2 - \rho\,, \quad a_2 = \rho - 1\,, \tag{6.14}$$

driven by Gaussian white noise η_n of unit variance. Its spectrum is known exactly, which makes it the right test of an estimator. The recursion is a linear time-invariant filter acting on η_n, and Section 6.6 shows that a linear filter with frequency response $H(f)$ multiplies the input spectrum by $|H(f)|^2$. Taking the transform of Eq. (6.14) with $z = e^{2\pi i f \Delta t}$ gives $X(1 - a_1 z^{-1} - a_2 z^{-2}) = z^{-1}\,\mathrm{H}$, where the factor z^{-1} is a one-sample delay of unit modulus. The input is white noise of variance σ_η^2 sampled at Δt, whose two-sided density is $S_{\eta\eta} = \sigma_\eta^2 \Delta t$ on $|f| \le f_s/2$ (the variance σ_η^2 spread evenly over the band of width $f_s = 1/\Delta t$). Hence

$$S_{xx}(f) = \frac{\sigma_\eta^2\,\Delta t}{\left|1 - a_1 e^{-2\pi i f \Delta t} - a_2 e^{-4\pi i f \Delta t}\right|^2}\,, \qquad |f| \le \frac{1}{2\Delta t}\,. \tag{6.15}$$

For $\rho = 0.05$ the characteristic roots $z^2 - a_1 z - a_2 = 0$ have modulus $\sqrt{1-\rho} = 0.975$ and argument close to $\pi/10$, so the spectrum has a single peak near $f = 0.05/\Delta t$, twenty samples per cycle, whose

half-power width is set by the distance of the poles from the unit circle, $B_r \approx \rho/(2\pi\Delta t)$ for small ρ; for $\rho = 0.05$ the exact width is $0.0082/\Delta t$. Integrating Eq. (6.15) numerically over $|f| \le 1/2$ (with $\Delta t = 1$ s) gives a variance of 104.0, against a sample variance of 103.2 from 2000 records of 1000 samples: Parseval's identity, Eq. (4.34), confirms the scaling.

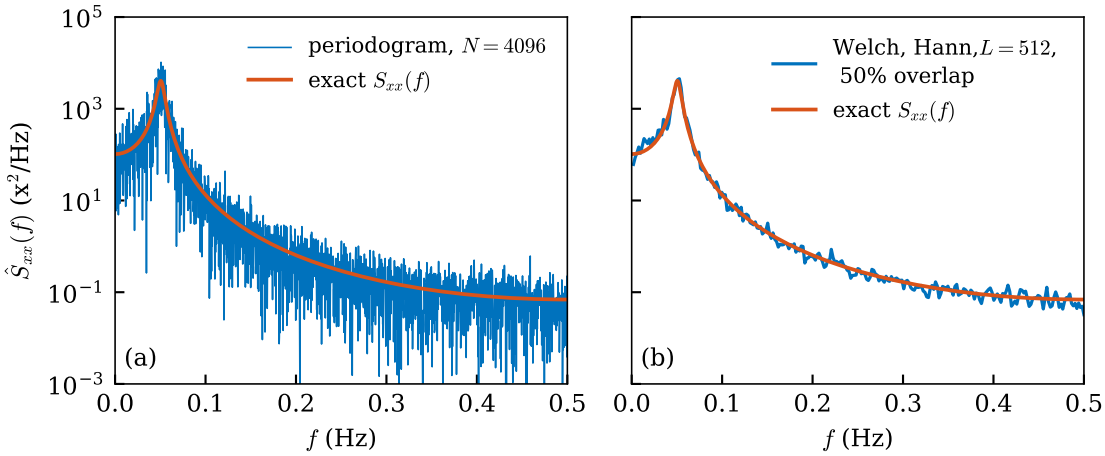


Figure 6.3: Spectral estimates for one record of $N = 4096$ samples of the AR(2) process Eq. (6.14) with $\rho = 0.05$, $\Delta t = 1$ s, against the exact spectrum Eq. (6.15). (a) The periodogram Eq. (6.1): unbiased on average but scattered over a factor of $\chi^2_2/2$, roughly 0.05 to 3, about the truth at every bin. (b) Welch's estimate, Hann window, $L = 512$, 50% overlap, $K = 15$ segments, $\nu \approx 28$: the scatter is reduced by about $\sqrt{15}$ and the resolution coarsened from 1/4096 to 1/512 Hz, which puts four bins across the half-power width of the peak, 0.0082 Hz, the minimum the resolution rule of Section 6.2 allows.

Fig. 6.3 compares the periodogram and Welch's estimate of one record against Eq. (6.15). The periodogram scatters about the truth over the two decades predicted by Eq. (6.8); the Welch estimate follows it to within the factor of 0.63 to 1.83 that $\nu \approx 28$ degrees of freedom allow at 95% confidence, and is, on average over many records, 4% low at the peak from resolution bias. Both are drawn on a logarithmic axis, on which the random error is a constant vertical thickness.

6.3 Multitaper Estimates and Line Components

Welch's method buys degrees of freedom by cutting the record into segments, so every term in its average sees only L of the N samples. Thomson's multitaper method [212] keeps the whole record and averages over a family of orthogonal tapers instead. Two things follow. The spectral window is confined to a band whose width the analyst chooses, rather than to a main lobe fixed by the segment length. And the separate tapered transforms permit a statistical test for a sinusoid buried in a continuous background. The second property is the reason the method belongs here. The response of a forced nonlinear oscillator is a *mixed spectrum*: lines at the forcing frequency and its harmonics, superposed on a broadband continuum that is the signature of chaos or noise. A density estimate of either kind draws both on one curve and cannot say which is which. The development follows Thomson [212] and Percival and Walden [156]; the numerical results are this book's own and are labelled as illustrations.

6.3.1 Slepian tapers

Fix a half-bandwidth W in hertz. Among all sequences v_n, $n = 0, \dots, N-1$, the one whose transform $V(f) = \sum_n v_n e^{-2\pi i f n \Delta t}$ puts the largest fraction of its energy inside $|f| \le W$ maximizes the *concentration*

$$\lambda = \frac{\int_{-W}^{W} |V(f)|^2 \, \mathrm{d}f}{\int_{-f_s/2}^{f_s/2} |V(f)|^2 \, \mathrm{d}f} \,. \tag{6.16}$$

Writing out both integrals turns Eq. (6.16) into a Rayleigh quotient, and its stationary points solve the eigenproblem

$$\sum_{m=0}^{N-1} \frac{\sin[2\pi W \Delta t \, (n-m)]}{\pi (n-m)} \, v_m = \lambda \, v_n \,, \qquad n = 0, \dots, N-1 \,, \tag{6.17}$$

with the diagonal term read as its limit $2W\Delta t$. The eigenvectors $v_n^{(k)}$, ordered by decreasing eigenvalue $1 > \lambda_0 > \lambda_1 > \cdots$, are the *discrete prolate spheroidal sequences* of Slepian [186], or *Slepian tapers*. They are orthonormal, $\sum_n v_n^{(j)} v_n^{(k)} = \delta_{jk}$; even orders are symmetric about the record centre and odd orders antisymmetric. The *time–bandwidth product* $TW = NW\Delta t$ sets how many of them are useful: about $2TW - 1$ have λ_k close to one, and the rest leak most of their energy outside the band. Software works in cycles per sample and calls this product `NW`. For $N = 1024$ and $TW = 4$ the out-of-band fractions $1 - \lambda_k$ of the first seven tapers are 3×10^{-10}, 3×10^{-8}, 1×10^{-6}, 3×10^{-5}, 6×10^{-4}, 7.5×10^{-3} and 6.3×10^{-2}. The first taper is better concentrated than any classical window; the seventh is worse than the Hann window, whose sidelobes hold about 5×10^{-4} of its energy. That gradient governs everything below.

6.3.2 Eigenspectra and their average

Each taper defines an *eigencoefficient* and an *eigenspectrum*,

$$J_k(f) = \sum_{n=0}^{N-1} v_n^{(k)} x_n \, e^{-2\pi i f n \Delta t} \,, \qquad \hat{S}_k(f) = \Delta t \, |J_k(f)|^2 \,, \qquad k = 0, \dots, K-1 \,, \tag{6.18}$$

which is the modified periodogram Eq. (6.6) with window $w_n = v_n^{(k)}$ and $\sum_n w_n^2 = 1$. Its expectation is Eq. (6.4) with a spectral window that holds the fraction λ_k of its area inside $\pm W$. For a spectrum nearly constant over a band $2W$, orthogonality of the tapers makes the $J_k(f)$ nearly uncorrelated, and the plain average

$$\hat{S}_{xx}^{(\mathrm{mt})}(f) = \frac{1}{K} \sum_{k=0}^{K-1} \hat{S}_k(f) \sim S_{xx}(f) \, \frac{\chi^2_{2K}}{2K} \tag{6.19}$$

has $\nu = 2K$ degrees of freedom at resolution $2W$. For white Gaussian noise with $N = 1024$, $TW = 4$, $K = 7$, 3000 records gave a mean of $0.999 \, S_{xx}(f)$ and $\nu = 2\mathrm{E}\big[\hat{S}\big]^2 / \mathrm{Var}\big[\hat{S}\big] = 14.0$. The accounting is the same as Welch's: about $2TW$ independent pieces of information per band of width $2W$, the product $B_r T$ of Section 6.2 in another guise. What differs is the shape of the window. A Welch estimate has the main lobe and sidelobes of one taper of length L; a multitaper estimate has a window close to a rectangle of width $2W$, with the leakage of its worst taper outside.

That leakage matters where the spectrum spans several decades. Power leaking from a strong region into the high-order eigenspectra at a weak frequency biases the plain average upward. Thomson's *adaptive weights* [212] suppress it. Given a current estimate $\hat{S}(f)$ and the white level $\sigma^2 \Delta t$ of

the record,

$$d_k(f) = \frac{\sqrt{\lambda_k}\,\hat{S}(f)}{\lambda_k \hat{S}(f) + (1-\lambda_k)\,\sigma^2 \Delta t}\,, \qquad \hat{S}(f) \leftarrow \frac{\sum_k d_k^2(f)\,\hat{S}_k(f)}{\sum_k d_k^2(f)}\,, \tag{6.20}$$

iterated from the average of the first two eigenspectra until it settles. A taper is down-weighted wherever its expected broadband leakage, $(1-\lambda_k)\sigma^2\Delta t$, is comparable with the local spectrum. The price is variance: the equivalent degrees of freedom become

$$\nu(f) = \frac{2\big(\sum_k d_k^2(f)\big)^2}{\sum_k d_k^4(f)}\,, \tag{6.21}$$

equal to $2K$ where the spectrum is high and smaller where it is low. Both estimates are reported one-sided by Eq. (6.3).

6.3.3 The harmonic F-test

Let the record contain a line, $x_n = A\cos(2\pi f_0 n\Delta t + \phi) + (\text{continuum})$, with f_0 farther than W from 0 and from $f_s/2$. At $f = f_0$ the positive-frequency half of the line contributes μV_k to each eigencoefficient, where $\mu = \frac{1}{2}Ae^{i\phi}$ is the complex amplitude and $V_k = \sum_n v_n^{(k)}$ is the taper sum, zero for odd k. The negative-frequency half contributes only through the far tails of the windows and is negligible. Across the K tapers, then, $J_k(f_0)$ is a linear regression on the known V_k with complex slope μ and a residual that is the continuum. Least squares gives the amplitude and the test statistic

$$\hat{\mu}(f) = \frac{\sum_k V_k J_k(f)}{\sum_k V_k^2}\,, \qquad F(f) = \frac{(K-1)\,|\hat{\mu}(f)|^2 \sum_k V_k^2}{\sum_k |J_k(f) - \hat{\mu}(f)V_k|^2}\,, \tag{6.22}$$

the ratio of the power explained by a line at f to the power left over, each per degree of freedom. If there is no line and the continuum is locally flat, $F(f)$ is distributed as F with 2 and $2K-2$ degrees of freedom [212]. On the 3000 white-noise records above, 0.98% of the interior frequencies exceeded the 99% point of $F_{2,12}$, which is 6.93. A significant F identifies a line; $\hat{A} = 2\,|\hat{\mu}(f_0)|$ and $\hat{\phi} = \arg\hat{\mu}(f_0)$ estimate it. Subtracting $2\,\mathrm{Re}[\hat{\mu}\,e^{2\pi i f_0 n\Delta t}]$ from the record and estimating the spectrum of the residual gives the continuous part alone. The output is the mixed spectrum written out: a list of lines with frequency, amplitude and phase, and a density $G_c(f)$ for the rest.

Two choices control the test. The first is the frequency grid. A line between Fourier frequencies is seen at $f \neq f_0$, where the regression on V_k is mismatched; both $\hat{\mu}$ and F fall. The remedy is to zero-pad the transform Eq. (6.18) and take the local maximum of F. The second is the threshold. When the line frequencies are known in advance, as for the harmonics of a forcing, each is a single test at the nominal level. When the record is searched for lines at unknown frequencies, every frequency is a test and the threshold must allow for the number of looks; here the $1 - 1/N$ point of $F_{2,2K-2}$ is used, and its false-line rate is measured rather than assumed.

Multitaper spectrum with line removal

Given x_n, $n = 0, \dots, N-1$, sampled at Δt; a half-bandwidth W; and $K \le 2TW - 1$ tapers:

1. Remove the mean. Compute the Slepian tapers $v_n^{(k)}$ and concentrations λ_k of Eq. (6.17).
2. Compute $J_k(f)$ of Eq. (6.18) by FFT, zero-padded to eight times N.
3. Compute $F(f)$ and $\hat{\mu}(f)$ by Eq. (6.22). Take as lines the local maxima of F above the chosen threshold.
4. Subtract each line, $2\,\mathrm{Re}[\hat{\mu}(f_0)\, e^{2\pi i f_0 n \Delta t}]$, from the record.
5. Estimate the continuum of the residual by Eq. (6.20), unpadded, and report it one-sided with $\nu(f)$ of Eq. (6.21), the resolution $2W$, and the table of lines.

The routines `psdmt` and `mtlines` of Appendix A.13 implement these steps.

6.3.4 Worked example: lines over a continuum

The test record is the AR(2) process Eq. (6.14) with $\rho = 0.05$ and $\Delta t = 1$ s, plus three harmonics of $f_1 = 0.13$ Hz with amplitudes 10, 0.5 and 0.1 and random phases; $N = 1024$. Its continuum is known exactly, Eq. (6.15), so every bias can be measured. The lines fall between Fourier frequencies ($f_1 T = 133.12$). In a Welch bin of width $1/256$ Hz the three lines stand 32, 18 and 9 dB above the continuum. Four estimators are compared over 1000 records: Welch with a Hann window at $L = 256$ ($K = 7$ segments) and at $L = 512$ ($K = 3$), both with 50% overlap; and multitaper with $TW = 4$, $K = 7$, plainly averaged and adaptively weighted. The Hann main lobe extends $\pm 2/L$, which is $\pm 2W$ at $L = 256$ and $\pm W$ at $L = 512$.

Away from the lines the adaptive multitaper and the $L = 256$ Welch estimates are equally good. Their median ratios to the truth were 1.004 and 1.007, and both were 18% low at the AR(2) peak (0.820 and 0.825). That peak has a half-power width of 0.0082 Hz, about $2W$, so the resolution rule of Section 6.2 is not met by either. Welch at $L = 512$ resolves it better (0.952) with $\nu = 5.8$ instead of 13.4. The plainly averaged multitaper estimate was 10% high everywhere away from the peak (1.101). The AR(2) spectrum spans 48 dB, and the sixth and seventh tapers carry power from the peak into the floor. Adaptive weighting removed the bias and lowered the degrees of freedom from 13.1 to 10.1 by Monte Carlo; Eq. (6.21) gave 9.0 above 0.2 Hz and 14.0 at the peak.

Beside the strong line the estimators differ, Fig. 6.4(b). The Welch estimate at $L = 256$ spreads the line over its main lobe: it is 8.7 times the continuum at $1.7W$ from the line and 33% high at $2.7W$. At $L = 512$ the line is confined like the multitaper's, at $\nu = 5.8$. The multitaper estimate confines the bulk of the line within about $1.5W$ at $\nu \approx 10$: it is 36% high at $1.7W$. It leaves a low tail from its high-order tapers, still 12% high at $2.7W$. Neither estimator separates the line from the continuum. The F-test does. With the $1 - 1/N$ threshold, 13.05, the first two lines were found in every record and the third in 94% of them; the strong line's amplitude came out 10.00 ± 0.09. Without zero padding the third line was found in 12.5% of records and the first was read 2% low, 9.78. Outside the line bands the test found 1.46 false lines per record, and 1.82 on white noise, above the $N/2 \times 1/N = 0.5$ that counting Fourier frequencies suggests: the padded grid searches the maxima of F between them. After the detected lines were subtracted, the adaptive estimate of the residual was within 2% of the continuum at $1.2W$, $2.5W$ and $5.2W$ from the strong line. Inside $\pm W$ of the two lines found in every record it was 8% and 11% low, because the fitted sinusoid also

absorbs the continuum's share of that band.

The example supports a short rule. For a long stationary record with a smooth spectrum the two methods give the same answer, and Welch's segment structure is what the coherence and FRF estimators of Sections 6.5–6.6 are built on. The multitaper estimate earns its place when the record is short relative to the resolution needed, when strong narrow components sit beside weak broadband ones, and above all when lines must be separated from a continuum. Its cross-spectral form averages $J_k^{(x)*} J_k^{(y)}$ over tapers instead of segments; it is not developed here.

Forcing harmonics in the two-well record

The two-well record of Section 1.4.1 is forced at 10 Hz and sampled at 160 Hz, 16 samples per forcing period. Its response is cross-well chaos. Over the full record (N = 96,000, $TW = 4$, $K = 7$), F at the first four harmonics was 4491, 28.9, 163 and 74.0 (Fig. 6.4(d)). These frequencies are known in advance, so each is one test, and all four exceed the 99% point, 6.93. A blind search at the $1 - 1/N$ point, 34.6, finds 10, 30 and 40 Hz, misses 20 Hz, and adds one line at 11.77 Hz that is not a harmonic, as the false-line rate above leads one to expect.

The lines are the part of the response phase-locked to the forcing. That part can be computed independently by *synchronous averaging*: average the record over its 6000 forcing periods at each of the 16 phases, and take the Fourier coefficients c_h of the averaged period. The amplitude estimates agree:

Harmonic (Hz)	10	20	30	40
$\lvert\hat{\mu}\rvert$, F-test (V)	0.2338	2.21×10^{-3}	1.361×10^{-3}	2.17×10^{-4}
$\lvert c_h\rvert$, synchronous average (V)	0.2336	2.26×10^{-3}	1.361×10^{-3}	2.10×10^{-4}

The harmonic lines carry 0.109 V^2 of the 2.50 V^2 variance, 4.4%; the chaotic continuum carries the rest. Synchronous averaging needs the forcing period and an integer number of samples per period. The F-test needs neither, and it reports a significance for each line.

Pitfall: the F-test inherits the leakage of the worst taper

The null distribution $F_{2,2K-2}$ assumes that the continuum is flat across $\pm W$ as seen by *every* taper. Where the spectrum lies many decades below a strong component, the high-order tapers see mostly leaked power, the residual in Eq. (6.22) is inflated, and F loses its upper tail. Lines there go undetected. In the two-well record the spectrum between 65 and 78 Hz is about 3×10^{-10} V^2/Hz, more than 100 dB below the 10 Hz line. With $K = 7$ no frequency in that band exceeded the 99% point, against about 1% elsewhere, and Eq. (6.21) gave $\nu \approx 2.3$ there. With $K = 5$, dropping the two leakiest tapers, the exceedance returned to 0.8%. Low adaptive degrees of freedom are the symptom; fewer tapers at the same TW is the cure.

6.4 Statistics from Correlated Samples

Chapter 2 gave the variance of the sample mean as σ^2/N. That result assumes uncorrelated samples, and the records of this book are not uncorrelated: they are sampled many times per correlation time, precisely so that the dynamics can be resolved. The spectral tools of this chapter give the correction in one line, and the correction propagates to every statistic computed from a record.

6.4.1 Variance of the sample mean

Let x_n be a stationary sequence with mean μ, variance σ^2 and normalized autocorrelation $\rho_k = C_{xx}(k\Delta t)\,/\sigma^2$, the correlation coefficient of Chapter 3 at lag k samples. The sample mean $\bar{x} = N^{-1}\sum_{n=1}^{N} x_n$ is unbiased, and its variance is the sum over all pairs of the covariance of the pair:

$$\mathrm{Var}[\bar{x}] = \frac{1}{N^2}\sum_{n=1}^{N}\sum_{m=1}^{N}\mathrm{Cov}[x_n, x_m] = \frac{\sigma^2}{N}\left[1 + 2\sum_{k=1}^{N-1}\Big(1 - \frac{k}{N}\Big)\rho_k\right], \tag{6.23}$$

where the second form counts the $N-k$ pairs at each lag k. For $\rho_k = 0$ this is σ^2/N. For a sequence whose correlation decays within a lag $q \ll N$ the triangular weights are immaterial and

$$\mathrm{Var}[\bar{x}] \approx \frac{\sigma^2}{N_{\mathrm{eff}}}, \qquad N_{\mathrm{eff}} = \frac{N}{1 + 2\sum_{k=1}^{\infty}\rho_k}, \tag{6.24}$$

which defines the *effective number of independent samples*. The sum in the denominator is the integral correlation time in units of Δt, up to the factor of two, and the comparison with Eq. (4.19) shows what it is: since $S_{xx}(0) = \Delta t\sum_{k=-\infty}^{\infty} C_{xx}(k\Delta t) = \Delta t\,\sigma^2(1 + 2\sum_{k\geq 1}\rho_k)$ for a discrete-time process, Eq. (6.24) reads

$$\mathrm{Var}[\bar{x}] \approx \frac{S_{xx}(0)}{T}. \tag{6.25}$$

The variance of the mean of a record of length T is the spectral density at zero frequency divided by the record length. This is exactly the statement that the periodogram at $f = 0$ has variance of the order of its mean, made in Section 6.1, seen from the time domain: the sample mean is the $k = 0$ Fourier coefficient, and it is one number no matter how long the record.

For the first-order autoregression $x_{n+1} = ax_n + \eta_n$, $\rho_k = a^k$ and $N_{\mathrm{eff}} = N(1-a)/(1+a)$; with $a = 0.8$, $N_{\mathrm{eff}} = N/9$, and the $\sigma/\sqrt{N}$ error bar on the mean is three times too small. A check with 4000 records of $N = 2000$ samples gave $\mathrm{Var}[\bar{x}] = 0.0130$ against 0.0125 from Eq. (6.24). The correction can run the other way. For the oscillatory AR(2) process Eq. (6.14) with $\rho = 0.05$, ρ_k alternates in sign with a period of twenty lags, the sum $\sum_k \rho_k = -0.006$ nearly cancels, and $N_{\mathrm{eff}} = 1.013\,N$: the record is as informative about its mean as an independent one of the same length, because the spectrum Eq. (6.15) is small at $f = 0$. Monte Carlo over 2000 records of $N = 1000$ samples gave $\mathrm{Var}[\bar{x}] = 0.1042$ against 0.1039 from Eq. (6.23) and 0.1032 for independent samples.

The same identity corrects the sample variance. Since $\mathrm{E}\big[\bar{x}^2\big] = \mu^2 + \mathrm{Var}[\bar{x}]$,

$$\mathrm{E}\left[\frac{1}{N}\sum_{n=1}^{N}(x_n - \bar{x})^2\right] = \sigma^2 - \mathrm{Var}[\bar{x}] \approx \sigma^2\Big(1 - \frac{1}{N_{\mathrm{eff}}}\Big), \tag{6.26}$$

which reduces to the factor $(N-1)/N$ of Chapter 2 for uncorrelated samples and is otherwise smaller: a correlated record underestimates its own variance, because it has explored less of the distribution than its length suggests. For the AR(1) example the factor is 0.9955, and simulation gave 0.9953.

Pitfall: the $\sqrt{N}$ rule on correlated data

Every test that compares a sample statistic with $\sigma/\sqrt{N}$—the stationarity check of Chapter 2 that compares the means of two halves of a record, a confidence interval on a mean, a t-test between two runs—is wrong by the factor $\sqrt{N/N_{\text{eff}}}$ when the samples are correlated, and it is wrong in the direction of false confidence: a nonstationary trend is "detected" in a record that is merely correlated, because the natural wandering of a correlated mean exceeds $\sigma/\sqrt{N}$. Replace N by N_{eff} from Eq. (6.24), estimating $\sum_k \rho_k$ from the sample autocorrelation out to the lag where it dies, or equivalently estimate $S_{xx}(0)$ and use Eq. (6.25).

6.4.2 Variance of the sample autocorrelation

The sample autocorrelation of Chapter 3, normalized to $\hat{\rho}_k = \hat{C}_{xx}(k)/\hat{C}_{xx}(0)$, is itself a statistic with a sampling distribution. Bartlett's formula [12] gives its large-sample variance for a Gaussian process; for lags beyond those at which the true correlation is appreciable it reduces to

$$\operatorname{Var}[\hat{\rho}_k] \approx \frac{1}{N} \sum_{j=-\infty}^{\infty} \rho_j^2 \qquad (\rho_k \approx 0)\,, \tag{6.27}$$

and in particular, for a sequence with $\rho_j = 0$ for $j > q$, $\operatorname{Var}[\hat{\rho}_k] \approx N^{-1}(1 + 2\sum_{j=1}^{q} \rho_j^2)$ for $k > q$. For white noise every ρ_j with $j \neq 0$ vanishes, $\operatorname{Var}[\hat{\rho}_k] \approx 1/N$, and the estimates are approximately Gaussian, so

$$|\hat{\rho}_k| \leq \frac{2}{\sqrt{N}} \qquad \text{for about 95\% of the lags} \tag{6.28}$$

of a white record. This is the band drawn on autocorrelation plots, and it is the test of whether a record, or the residual of a model fitted to it, is white. Simulation of 2000 white records of $N = 1000$ samples gave $\sqrt{N}\,\mathrm{SD}[\hat{\rho}_k] = 0.996$ and 4.5% of the lags outside the band.

The band is misread as often as it is used. It is the sampling error of the autocorrelation of a *white* record; for a correlated record the error at large lags is larger by $\sqrt{\sum_j \rho_j^2}$, which for the AR(2) process of Fig. 6.5(b) is $\sqrt{20} \approx 4.5$. The slow wandering of the sample autocorrelation of a correlated process about zero at large lag, plainly visible in the figure, is sampling error, not structure, and a lag at which it crosses zero is not a correlation time. A check with 1000 records of $N = 20{,}000$ samples gave $\sqrt{N}\,\mathrm{SD}[\hat{\rho}_{500}] = 4.52$ against $\sqrt{\sum_j \rho_j^2} = 4.47$ from Eq. (6.27).

Where this reappears

The reduction from N to N_{eff} is the first form of a problem that runs through Parts II and III. When Chapter 11 counts pairs of delay vectors closer than ϵ, the pairs (i, j) with $|i - j|$ less than a correlation time are close because they are consecutive points on one trajectory segment, not because the attractor brings them together, and they are excluded by the Theiler window (Section 12.5.4 of Chapter 12 treats the same exclusion for Lyapunov exponents). When Chapter 16 asks whether a statistic computed from a record is significant, the null distribution is built from surrogate records with the same $S_{xx}(f)$, and hence the same N_{eff}, precisely because the Gaussian formulas of this section do not extend to the statistics of interest there. In each case the question is the one answered by Eq. (6.24): how many independent samples does the record actually contain?

6.5 Cross-Spectra and Coherence

Two records measured together carry more than two spectra. The cross-spectrum is the frequency-domain form of the cross-correlation of Chapter 3, and the coherence built from it measures, frequency by frequency, how much of one record is a linear function of the other.

6.5.1 The cross-spectral density

For jointly stationary $x(t)$ and $y(t)$ with cross-correlation $R_{xy}(\tau) = \mathrm{E}[x(t)\,y(t+\tau)]$ as defined in Chapter 3, the *cross-spectral density* is its Fourier transform,

$$S_{xy}(f) = \int_{-\infty}^{\infty} R_{xy}(\tau)\; e^{-2\pi i f \tau}\,\mathrm{d}\tau\,, \tag{6.29}$$

and the argument of Chapter 4 that led from Eq. (4.23) to Eq. (4.31) goes through unchanged with $x(s)$ replaced by $y(s)$, giving

$$S_{xy}(f) = \lim_{T\to\infty} \frac{1}{T}\mathrm{E}[X_T^*(f)\,Y_T(f)]\;. \tag{6.30}$$

The conjugate falls on the first index; with the opposite convention for $R_{xy}(\tau)$ it falls on the second, and software must be checked for which it uses (the sign of the estimated phase, and hence of every time delay inferred from it, depends on it). Unlike $S_{xx}(f)$, the cross-spectrum is complex: since $R_{xy}(\tau)$ is not symmetric in τ its transform is not real. Its properties follow from those of $R_{xy}(\tau)$. From $R_{xy}(\tau) = R_{yx}(-\tau)$, $S_{yx}(f) = S_{xy}(f)^*$; from the reality of $R_{xy}(\tau)$, $S_{xy}(-f) = S_{xy}(f)^*$. It is written in either of two forms,

$$S_{xy}(f) = C_{xy}(f) - i\,Q_{xy}(f) = |S_{xy}(f)|\; e^{-i\theta_{xy}(f)}\,, \tag{6.31}$$

where the *co-spectrum* C_{xy} is the transform of the even part of $R_{xy}(\tau)$, the *quadrature spectrum* Q_{xy} that of the odd part, and $\theta_{xy}(f)$ is the *phase spectrum*. The phase carries the timing information. For a pure delay, $y(t) = x(t-\tau_0)$, one has $R_{xy}(\tau) = R_{xx}(\tau-\tau_0)$ and $S_{xy}(f) = S_{xx}(f)\; e^{-2\pi i f\tau_0}$: the phase is a straight line through the origin, $\theta_{xy}(f) = 2\pi f\tau_0$, and its slope is the delay. A phase that is linear over a band identifies a propagation time; a phase that jumps by π across a frequency identifies a resonance passed through, as in Section 6.6.

The estimator is the cross-periodogram, averaged over segments exactly as in Section 6.2:

$$\hat{S}_{xy}(f_k) = \frac{\Delta t}{K\sum_n w_n^2} \sum_{j=1}^{K} X_k^{(j)*}\,Y_k^{(j)}\,, \tag{6.32}$$

with the same windows, overlap, resolution and random error as the auto-spectral estimates; its one-sided form $\hat{G}_{xy}$ doubles the interior bins as in Eq. (6.3).

6.5.2 Coherence

The Cauchy–Schwarz inequality applied to Eq. (6.30) gives $|S_{xy}(f)|^2 \le S_{xx}(f)\;S_{yy}(f)$, and the ratio

$$\gamma_{xy}^2(f) = \frac{|S_{xy}(f)|^2}{S_{xx}(f)\;S_{yy}(f)}\,, \qquad 0 \le \gamma_{xy}^2(f) \le 1\,, \tag{6.33}$$

is the *ordinary coherence function*. It is the squared correlation coefficient of Chapter 3 between the components of x and y in a narrow band about f, and its interpretation is the same: $\gamma^2_{xy}(f) = 1$ when y is, at that frequency, a fixed linear function of x, and $\gamma^2_{xy}(f) = 0$ when the two are linearly unrelated there. Section 6.6 makes the first statement precise: for the output of a noise-free linear system driven by x, the coherence is identically one. The product $\gamma^2_{xy}(f)\, S_{yy}(f)$ is the *coherent output spectrum*, the part of the power of y at f that is linearly accounted for by x; the remainder $(1 - \gamma^2_{xy})S_{yy}$ is due to everything else.

6.5.3 Estimating the coherence

The estimate replaces each density in Eq. (6.33) by its averaged estimate,

$$\hat{\gamma}^2_{xy}(f_k) = \frac{\left|\hat{S}_{xy}(f_k)\right|^2}{\hat{S}_{xx}(f_k)\,\hat{S}_{yy}(f_k)}\,, \tag{6.34}$$

and here averaging is not merely helpful but constitutive. With a single segment, $K = 1$, the estimate is

$$\frac{|X_k^* Y_k|^2}{|X_k|^2\,|Y_k|^2} \equiv 1 \tag{6.35}$$

at every frequency, whatever the records: the unaveraged coherence is identically unity and contains no information. The coherence measures the consistency of the amplitude ratio and phase difference between x and y *across* segments, and a single segment has nothing to be consistent with. The check on any coherence estimate is therefore the number of averages n_d (here K), and the bias and random error both scale with it [8]:

$$\mathrm{Bias}\big[\hat{\gamma}^2_{xy}\big] \approx \frac{(1-\gamma^2_{xy})^2}{n_d}\,, \qquad \epsilon_r[\hat{\gamma}^2_{xy}] = \frac{\sqrt{\mathrm{Var}\big[\hat{\gamma}^2_{xy}\big]}}{\gamma^2_{xy}} \approx \frac{\sqrt{2}\,(1-\gamma^2_{xy})}{|\gamma_{xy}|\,\sqrt{n_d}}\,. \tag{6.36}$$

The bias is upward and largest where the true coherence is smallest: for $\gamma^2_{xy} = 0$ the estimate has expectation $1/n_d$ exactly for Gaussian records and rectangular non-overlapping segments (it is then a beta variable with parameters 1 and $n_d - 1$), so that eight averages of unrelated records return a coherence of 0.125. The random error is largest where the coherence is small and diverges as $\gamma^2_{xy} \to 0$; where the coherence is near one both errors vanish. A rule of thumb from Eq. (6.36): to distinguish $\gamma^2 = 0.5$ from zero at three standard deviations requires $n_d \gtrsim 20$; to estimate $\gamma^2 = 0.9$ to ± 0.01 requires $n_d \approx 50$.

Coherence bias by simulation

Records $y_n = a x_n + n_n$ with x_n and n_n independent Gaussian white sequences have $\gamma^2_{xy} = a^2/(1 + a^2)$ at every frequency. For 4000 such pairs, rectangular non-overlapping segments of $L = 128$: with $\gamma^2 = 0$ and $n_d = 8$ the mean estimate was 0.1251 (bias formula: 0.1250); with $n_d = 32$, 0.0313 (0.0312). With $\gamma^2 = 0.5$ and $n_d = 32$ the bias was 0.0078 (0.0078) and the normalized random error 0.175 (0.177); with $\gamma^2 = 0.9$ and $n_d = 32$, bias 0.0003 (0.0003) and random error 0.027 (0.026). With $n_d = 8$ and $\gamma^2 = 0.5$ the observed bias, 0.036, exceeded the first-order formula, 0.031; Eq. (6.36) is a large-n_d approximation.

6.5.4 What low coherence means

A coherence below one between an input and an output has four possible causes, and the estimate alone does not distinguish them.

1. *Extraneous noise* in either record. Section 6.6 shows that uncorrelated noise of density S_{nn} on the output of a linear system with input density S_{uu} reduces the coherence to $\gamma^2 = [1 + S_{nn}/(|H(f)|^2\, S_{uu})]^{-1}$: the coherence is the output signal-to-noise ratio in disguise, and drops wherever the response is weak—between resonances, at antiresonances, and at high frequency.

2. *Resolution bias.* Where either spectrum changes appreciably within one bin, the segment estimates of $\hat{S}_{xy}$ do not add in phase and the coherence drops; the effect is the one in the pitfall box of Section 6.2, and it appears at sharp resonances and with rectangular windows. It is diagnosed by changing L and cured by increasing it.

3. *Unmeasured inputs.* If y responds to a second input uncorrelated with x, that part of y is noise as far as γ^2_{xy} is concerned. The multiple and partial coherence functions of Bendat and Piersol [8] extend the analysis to several measured inputs; an unmeasured one cannot be recovered.

4. *Nonlinearity.* If y depends on x but not linearly, the power that appears at harmonics and combination frequencies of the input is uncorrelated with the input at those frequencies, and the coherence drops there.

All four produce the same symptom, but they respond differently to the excitation level. Raising the input amplitude recovers coherence lost to extraneous noise and to unmeasured inputs: by the expression in item 1, both enter through the ratio of a fixed disturbance to a growing response. It leaves resolution bias unchanged. Nonlinearity works the other way. For a weak nonlinearity the power moved to harmonics and combination frequencies grows faster than the linear response, so the coherence falls as the amplitude rises [227]. Repeating the measurement at two input levels is therefore the simplest way to separate the fourth cause from the other three, and it is what makes the coherence usable as a test for nonlinearity despite its ambiguity.

6.6 Frequency Response Functions

A linear time-invariant system maps its input spectrum to its output spectrum by a single complex function of frequency. That function can be estimated from the cross- and auto-spectra of measured input and output, in more than one way, and the ways differ in what noise they tolerate.

6.6.1 Input–output relations

Let the system be linear and time-invariant with impulse response $h(t)$, so that its output is the convolution $y(t) = \int_{-\infty}^{\infty} h(\theta)\, x(t-\theta)\, \mathrm{d}\theta$; for a physical system $h(\theta) = 0$ for $\theta < 0$. The convolution property of Chapter 4 gives $Y(f) = H(f)\, X(f)$, where $H(f) = \mathcal{F}\{h\}$ is the *frequency response function* (FRF). It is complex, $H(f) = |H(f)|\, e^{i\phi(f)}$: a sinusoidal input at f produces a sinusoidal output of amplitude $|H(f)|$ times the input and phase shifted by $\phi(f)$. For random inputs the same function relates the correlations. From the convolution and the definition of $R_{xy}(\tau)$,

$$R_{xy}(\tau) = \mathrm{E}\left[x(t) \int h(\theta)\, x(t+\tau-\theta)\, \mathrm{d}\theta\right] = \int_{-\infty}^{\infty} h(\theta)\, R_{xx}(\tau-\theta)\ \mathrm{d}\theta\,, \tag{6.37}$$

a convolution of h with $R_{xx}()$, and similarly $R_{yy}(\tau) = \iint h(\theta)h(\theta')R_{xx}(\tau+\theta-\theta')\,\mathrm{d}\theta\,\mathrm{d}\theta'$. Transforming,

$$S_{xy}(f) = H(f)\,S_{xx}(f)\,, \qquad S_{yy}(f) = |H(f)|^2\,S_{xx}(f)\,. \tag{6.38}$$

The second relation was used to derive the AR(2) spectrum Eq. (6.15). Together they give $|S_{xy}(f)|^2 = |H(f)|^2\,S_{xx}(f)^2 = S_{xx}(f)\,S_{yy}(f)$, so $\gamma^2_{xy}(f) = 1$ at every frequency: for a linear system with noise-free measurements the coherence is identically one, which is the benchmark against which every measured coherence is read. The first relation is the important one for estimation. The cross-spectrum carries the phase of $H(f)$, which the output auto-spectrum has lost, and dividing it by the input auto-spectrum removes the input entirely: $H(f) = S_{xy}(f)\,/S_{xx}(f)$ whatever the spectrum of x, provided only that $S_{xx}(f) \neq 0$ at the frequencies of interest. A broadband random input therefore measures the whole FRF at once.

6.6.2 The H_1, H_2 and H_v estimators

In a measurement neither record is the true signal. Write the measured input as $x = u + m$ and the measured output as $y = v + n$, where v is the response to the true input u, $V(f) = H(f)\,U(f)$, and m and n are measurement noises uncorrelated with u and with each other. Then $S_{xx}(=)\,S_{uu} + S_{mm}$, $S_{yy} = |H|^2\,S_{uu} + S_{nn}$, and $S_{xy}(=)\,HS_{uu}$, since the noise terms have zero cross-correlation with everything else. Two estimators follow from Eq. (6.38) by replacing densities by their averaged estimates:

$$\hat{H}_1(f) = \frac{\hat{S}_{xy}(f)}{\hat{S}_{xx}(f)} \;\longrightarrow\; \frac{HS_{uu}}{S_{uu}+S_{mm}} = \frac{H(f)}{1+S_{mm}/S_{uu}}\,, \tag{6.39}$$

$$\hat{H}_2(f) = \frac{\hat{S}_{yy}(f)}{\hat{S}_{yx}(f)} \;\longrightarrow\; \frac{|H|^2\,S_{uu}+S_{nn}}{H^*S_{uu}} = H(f)\left[1+\frac{S_{nn}}{|H(f)|^2\,S_{uu}}\right]\,, \tag{6.40}$$

where the arrows give the limits for many averages. The H_1 estimator is unbiased by output noise, however large, and biased low by input noise in the ratio of input noise to input signal power. The H_2 estimator is unbiased by input noise and biased high by output noise in the ratio of output noise to output signal power, which is worst where the response is weakest. The two are ordered, $\left|\hat{H}_1\right| \leq \left|\hat{H}_2\right|$, and their ratio is the coherence estimate:

$$\frac{\hat{H}_1(f)}{\hat{H}_2(f)} = \frac{\left|\hat{S}_{xy}\right|^2}{\hat{S}_{xx}\hat{S}_{yy}} = \hat{\gamma}^2_{xy}(f)\,, \tag{6.41}$$

exactly, for the same segments. Where the coherence is one the two estimators agree and there is no noise problem; where they differ, the coherence says by how much, and the noise model says which to believe. In a vibration test with a shaker, the force is measured well and the response is buried in noise at the antiresonances, so H_1 is the default; at a sharp resonance the force signal drops (the shaker cannot push a structure that moves with it) while the response is large, so the input becomes the noisy channel and H_2 is preferred there.

The H_v estimator treats both channels as noisy. It is the total-least-squares solution, minimizing over the segments the sum of $|Y_k - HX_k|^2\,/(1+|H|^2)$ rather than of $|Y_k - HX_k|^2$ (which gives H_1) or of $|X_k - Y_k/H|^2$ (which gives H_2), and it is computed at each frequency from the eigenvector

belonging to the smaller eigenvalue of the 2×2 Hermitian matrix of estimated spectra, $\begin{bmatrix}\hat{S}_{xx} & \hat{S}_{xy}\\ \hat{S}_{yx} & \hat{S}_{yy}\end{bmatrix}$: if that eigenvector is $[v_1, v_2]^{\mathrm{T}}$ then $\hat{H}_v = -v_1/v_2$. It is unbiased when the two noise densities are equal in the units in which the matrix is formed, and lies between H_1 and H_2 otherwise; a numerical check with equal noise on both channels gave $\left|\hat{H}_v\right| / |H| = 1.01$ at resonance where $\left|\hat{H}_1\right| / |H| = 0.81$ and $\left|\hat{H}_2\right| / |H| = 1.01$. Unlike H_1 and H_2 it depends on the relative scaling of the two channels, and it should be used with that scaling chosen deliberately.

6.6.3 Random errors

The random errors of the H_1 estimate follow from those of the spectra it is built from, and Bendat and Piersol [8] give, for n_d averages,

$$\epsilon_r\left[\,\left|\hat{H}_1\right|\,\right] \approx \frac{\sqrt{1-\gamma_{xy}^2}}{|\gamma_{xy}|\sqrt{2n_d}}, \qquad \mathrm{SD}\left[\hat{\phi}\right] \approx \frac{\sqrt{1-\gamma_{xy}^2}}{|\gamma_{xy}|\sqrt{2n_d}}\ \mathrm{rad}, \tag{6.42}$$

the same expression for the normalized error of the magnitude and the standard deviation of the phase in radians. Both vanish where the coherence is one—an FRF measured with unit coherence needs no averaging beyond the two segments that make the coherence meaningful—and both diverge as the coherence falls, so that a measured FRF is trustworthy exactly where the accompanying coherence is high. A simulation of the oscillator of the next subsection with $n_d = 32$ averages gave, at a frequency where $\gamma^2 = 0.60$, magnitude error 0.102 and phase standard deviation 0.104 rad against 0.101 from Eq. (6.42); at $\gamma^2 = 0.20$, 0.240 and 0.253 against 0.248. At the resonance peak, with $\gamma^2 = 0.985$, the observed errors (0.025) exceeded the formula (0.016): the residual is leakage of the rectangular window used in that check, and the formula holds only where the resolution is adequate.

6.6.4 The single-degree-of-freedom oscillator

The running example is the oscillator

$$m\ddot{x} + c\dot{x} + kx = f(t), \tag{6.43}$$

with mass m, viscous damping c and stiffness k. Transforming with the differentiation property of Chapter 4, $(-m(2\pi f)^2 + ic\,2\pi f + k)X(f) = F(f)$, so the FRF from force to displacement, called the *receptance*, is

$$H(f) = \frac{X(f)}{F(f)} = \frac{1}{k - m(2\pi f)^2 + i\,c\,2\pi f} = \frac{1/k}{1 - r^2 + 2i\zeta r}, \qquad r = \frac{f}{f_n}, \tag{6.44}$$

where $f_n = \frac{1}{2\pi}\sqrt{k/m}$ is the undamped *natural frequency* and $\zeta = c/(2\sqrt{km})$ the *damping ratio*. Its magnitude and phase are

$$|H(f)| = \frac{1/k}{\sqrt{(1-r^2)^2 + (2\zeta r)^2}}, \qquad \phi(f) = -\arctan\frac{2\zeta r}{1-r^2}, \tag{6.45}$$

with the phase taken in $(-\pi, 0]$: the displacement lags the force, by nothing at low frequency, by $\pi/2$ exactly at $r = 1$, and by π at high frequency where the response is mass-controlled. The magnitude peaks at $r = \sqrt{1-2\zeta^2}$ with value $(1/k)/(2\zeta\sqrt{1-\zeta^2})$, which for light damping is $1/(2\zeta k)$ at f_n:

the peak is $1/(2\zeta)$ times the static deflection $1/k$. The FRFs from force to velocity and acceleration, the *mobility* and *accelerance*, are $2\pi i f\, H(f)$ and $-(2\pi f)^2 H(f)$, which is why accelerometer measurements rise as f^2 relative to the receptance and are level, not falling, above resonance.

The damping is read from the width of the peak. The *half-power points* are the frequencies at which $|H(f)|^2$ has fallen to half its peak value, i.e. $|H|$ to $1/\sqrt{2}$ of the peak, 3 dB down. Setting $(1-r^2)^2 + (2\zeta r)^2 = 8\zeta^2(1-\zeta^2)$ and solving for r^2 gives $r^2 = 1 - 2\zeta^2 \pm 2\zeta\sqrt{1-\zeta^2}$, so for $\zeta \ll 1$ the two roots are $r_{1,2} \approx 1 \mp \zeta$ and the *half-power bandwidth* is

$$\Delta f_{1/2} = f_2 - f_1 \approx 2\zeta f_n\,, \qquad \zeta \approx \frac{\Delta f_{1/2}}{2 f_n}\,, \tag{6.46}$$

correct to first order in ζ. For $f_n = 5$ Hz and $\zeta = 0.02$ the exact width is 0.2001 Hz against 0.2000 Hz from Eq. (6.46); for $\zeta = 0.05$, 0.5013 against 0.5000. This is the bandwidth B_r of Section 6.2: an estimate with bins wider than $B_r/4$ smooths the peak, and the damping read from its width is then the damping of the spectral window, not of the structure. The quantity $Q = 1/(2\zeta)$ is the *quality factor*. To first order in ζ it equals both the height of the peak in units of the static response, $1/(2\zeta\sqrt{1-\zeta^2})$, and the reciprocal of its fractional width, $f_n/\Delta f_{1/2}$. Neither equality is exact: at $\zeta = 0.05$ the two are 10.01 and 9.97.

A structure with many degrees of freedom has an FRF between every pair of points, each a sum of terms of the form Eq. (6.44), one per mode; the identification of the natural frequencies, damping ratios and mode shapes from measured FRFs is modal analysis [58]. Everything in the present section applies to each such FRF unchanged.

6.6.5 Worked example: estimating the FRF of a noisy oscillator

Fig. 6.7 shows the estimators at work on the oscillator with $f_n = 5$ Hz, $\zeta = 0.05$, $m = 1$ kg, driven by Gaussian white noise and sampled at $f_s = 100$ Hz for $N = 409{,}600$ samples (4096 s), simulated exactly by zero-order-hold discretization; the reference curve is the response of the discretized system, which differs from Eq. (6.44) by a phase lag of $\pi f \Delta t$ (about 9° at f_n) and a negligible magnitude change. The estimates use Hann windows of $L = 2048$ samples ($\Delta f = 0.049$ Hz, ten bins across $B_r = 0.5$ Hz) at 50% overlap, $K = 399$ averages. In the left column, noise with standard deviation half that of the response is added to the measured output; in the right column, noise with standard deviation half that of the input is added to the measured input.

The figure verifies Eqs. (6.39)–(6.40) numerically. With output noise, $\left|\hat{H}_1\right| / |H|$ was 0.996 at resonance, 1.007 at 2 Hz and 0.80 at 12 Hz (where the coherence of 0.06 makes the random error, by Eq. (6.42), about 0.14), while $\left|\hat{H}_2\right| / |H|$ was 1.012, 1.265 and 12.7: the output-noise bias of H_2 is negligible at the peak, where the response dominates the noise, and ruinous in the tails. With input noise of one quarter the input power, $S_{mm}/S_{uu} = 0.25$, $\left|\hat{H}_1\right| / |H|$ was 0.803, 0.804 and 0.792 at the same frequencies against the predicted 0.80, and $\left|\hat{H}_2\right| / |H|$ was 1.005, 0.983 and 0.997. The coherence panels are the diagnostic: a coherence that follows the response magnitude indicates output noise and calls for H_1; a coherence that is flat and below one indicates input noise and calls for H_2.

6.7 Discussion

The chapter has one theme and three variations. The theme is that a finite record yields a spectral estimate whose expectation is the true spectrum smoothed by a window of width about $1/T$ and

whose variance, unaveraged, equals the square of the quantity estimated. Averaging K segments divides the variance by K and multiplies the window width by K; tapering lowers the far leakage and widens the window further; overlapping recovers about half of what tapering discards. Tapering the whole record with K orthogonal Slepian tapers spends the same budget differently, and adds what no density estimate has: a test that separates lines from a continuum. The record length T times the bandwidth of the narrowest feature of interest is the number of independent samples available, and the analyst's only decision is how to spend them between resolution and variance. Any spectral plot without a stated resolution and number of averages is unreadable.

The first variation is the effective sample size. Correlation changes the variance of every statistic of a correlated record, but the size of the change depends on the statistic. For the sample mean the variance is $S_{xx}(0)/T$ rather than σ^2/N. For the sample autocorrelation at large lag the scatter is set by $\sum_j \rho_j^2$ rather than by unity. For a Gaussian AR(1) record with $a = 0.8$ the variance of the mean is inflated by 9, and that of the mean square by 4.56. The $\pm 2/\sqrt{N}$ band is a test for whiteness and nothing else. The Theiler window and the surrogate ensemble of later chapters are the same correction applied to statistics for which no Gaussian formula exists.

The second variation is that two records yield a cross-spectrum whose magnitude and phase are the linear relation between them, and a coherence that measures how completely that relation accounts for the output. The coherence is the one estimate in the chapter that averaging does not merely improve but creates: it is identically one from a single segment. Its bias $(1-\gamma^2)^2/n_d$ and its random error, both growing as the coherence falls, mean that a low measured coherence is only as low as the number of averages allows one to say, and its four possible causes—noise, resolution, unmeasured inputs and nonlinearity—are indistinguishable in a single measurement. Measurements at two excitation levels separate the fourth: for a weak nonlinearity it is the only cause that lowers the coherence as the input grows.

The third variation is the frequency response function. For a linear system it is the ratio of cross- to input auto-spectrum, and the two noise-robust estimators H_1 and H_2 bracket it, with their ratio equal to the coherence. Which to use is decided by which channel is noisy, and the coherence, read together with the response magnitude, usually says which. The single-degree-of-freedom receptance, with its peak of height $1/(2\zeta k)$ and width $2\zeta f_n$, is the element from which modal analysis [58] assembles the response of structures with many modes, and the resolution rule of Section 6.2—at least four bins across each half-power bandwidth—is the first constraint on any modal test.

Two things the chapter has not done. It has treated only stationary records; a spectrum estimated from a record whose statistics drift is a time average over conditions that never coexisted, and the short-time and time–frequency methods that address this are beyond Part I. And it has treated only second-order statistics. Two processes with the same $S_{xx}(f)$ are indistinguishable by anything in this chapter, which is exactly why the surrogate tests of Chapter 16 preserve the spectrum and vary everything else: what the spectrum cannot see is what the rest of the book is about.

Problems

Problem 6.1

Computational

Verify the sampling distributions of Sections 6.1 and 6.2.

1. Generate 2000 records of $N = 512$ Gaussian white-noise samples with $\Delta t = 0.001$ s and unit variance. What is $S_{xx}(f)$, in x^2/Hz? Compute the periodogram Eq. (6.1) of each record and

confirm that the interior bins have mean $S_{xx}(f)$ and variance $S_{xx}(f)^2$, and that a histogram of $\hat{S}/S$ matches the density of $\chi^2_2/2$, i.e. e^{-s}. Show that the $k = 0$ bin has twice the variance.

2. Repeat with $N = 128$ and $N = 4096$ and confirm that the variance does not depend on N.

3. Now estimate each record's spectrum by Welch's method with a Hann window, $L = 64$, at 0%, 50% and 75% overlap. For each, report K from Eq. (6.12), the equivalent degrees of freedom $\nu = 2\mathrm{E}\left[\hat{S}\right]^2 / \mathrm{Var}\left[\hat{S}\right]$, and compare with Eq. (6.13). What does 75% overlap gain over 50%?

4. Using Eq. (6.11), construct the 95% confidence interval for the 50%-overlap estimate and check that it contains the true value in about 95% of the bins.

Problem 6.2

Computational

Generate the Lorenz $x(t)$ record of Problem 3.2 ($\sigma = 10$, $\rho = 28$, $\beta = 8/3$; 40,000 points at $\Delta t = 0.02$), discarding the first 1000 points of transient.

1. Compute the periodogram of the whole record and the Welch estimate with a Hann window at $L = 256$, 1024 and 4096 samples, 50% overlap. Plot all four on a logarithmic axis with the resolution and ϵ_r of each stated in the legend. Which features survive all four estimates, and which appear only in the periodogram?

2. The autocorrelation of this record in Fig. 3.2 decays within a few time units. Estimate N_{eff} from Eq. (6.24) and from $\hat{S}_{xx}(0)$ via Eq. (6.25); do they agree? How many independent samples does the record contain?

3. Repeat part 1 with the rectangular window. Is there a visible leakage difference? Explain why not, in terms of Table 6.1 and the shape of the spectrum.

4. Now generate the Duffing record of Problem 3.2 ($\ddot{x} + 0.2\dot{x} - x + x^3 = 0.33\cos t$, $\Delta t = 0.1$), which is chaotic but forced at $\omega = 1$. Its spectrum has a line at $f = 1/(2\pi)$ Hz and its odd harmonics on a broadband floor. Compare rectangular and Hann Welch estimates at $L = 1024$: where does the choice of window now matter, and by how many decibels? Confirm that the power under the fundamental line, $\sum \hat{G}_{xx}\Delta f$ over its main lobe, is the same for both windows although the peak heights differ.

Problem 6.3

Computational

The AR(2) family of Eq. (6.14) with $\rho \in \{0.02, 0.05, 0.2, 0.5\}$.

1. Derive the exact autocorrelation ρ_k from the Yule–Walker recursion $\rho_k = a_1\rho_{k-1} + a_2\rho_{k-2}$ with $\rho_0 = 1$, $\rho_1 = a_1/(1 - a_2)$, and plot it against the sample autocorrelation of one record of $N = 2000$ samples for each ρ, with the white-noise band Eq. (6.28) and the large-lag band Eq. (6.27).

2. Compute N_{eff}/N from Eq. (6.24) for each ρ. For which values does it exceed one, and why? Confirm by Monte Carlo over 1000 records that $\mathrm{Var}[\bar{x}]$ matches Eq. (6.23).

3. Plot the exact spectrum Eq. (6.15) for each ρ and read off the half-power bandwidth B_r of the peak. Verify that $B_r \approx \rho/(2\pi\Delta t)$ for $\rho \le 0.05$ by locating the poles of the transfer function,

explain why the approximation fails for $\rho = 0.5$, and state the minimum Welch segment length L at which the peak is resolved by the four-bin rule for each ρ.

4. For $\rho = 0.5$, estimate the spectrum from one record of $N = 8192$ samples with $L = 64$, 256 and 1024 (Hann, 50% overlap). Which choice best reproduces the exact spectrum, and by what measure?

Problem 6.4

Computational

Coherence between two channels of the Lorenz system. This problem prepares Problem 16.6.

1. From the Lorenz simulation of Problem 6.2, take $x(t)$ and $y(t)$ together. Estimate $\hat{S}_{xy}$, the phase spectrum $\theta_{xy}(f)$ and the coherence $\hat{\gamma}^2_{xy}(f)$ with a Hann window, $L = 512$, 50% overlap. Where is the coherence high, and what is the phase there? Interpret the phase in terms of the first Lorenz equation, $\dot{x} = \sigma(y - x)$, which makes x a low-pass filtered version of y: what FRF does it predict, and does $\hat{H}_1 = \hat{S}_{yx}/\hat{S}_{yy}$ from y to x agree with it?

2. Explain why the coherence between x and y should be close to one at all frequencies where either has appreciable power, even though the system is nonlinear, and where the nonlinearity would be expected to show.

3. Now compute the coherence between $x(t)$ and $z(t)$. Is it high or low, and why? Relate the answer to the symmetry $(x, y, z) \to (-x, -y, z)$ of the Lorenz equations.

4. Repeat part 1 with $L = 64$ and with $L = 8192$. Report K in each case and explain the changes in $\hat{\gamma}^2_{xy}$ using Eq. (6.36) and the resolution argument of Section 6.5.

Problem 6.5

Computational

Estimate the FRF of the oscillator Eq. (6.43) with $f_n = 5$ Hz, $\zeta = 0.02$, $m = 1$ kg, from simulated data.

1. Simulate the response to Gaussian white-noise force at $f_s = 100$ Hz for 2000 s (a zero-order-hold discretization, `c2d` in MATLAB or `scipy.signal.cont2discrete`, is exact for piecewise-constant input). What is the half-power bandwidth B_r, and what is the minimum L by the four-bin rule?

2. Estimate $\hat{H}_1$ with Hann windows of $L = 512$, 2048 and 8192, 50% overlap. For each, read ζ from the half-power bandwidth of $\left|\hat{H}_1\right|$ using Eq. (6.46) and from the height of the peak using $|H|_{\max} = 1/(2\zeta k)$. Which estimate of ζ is biased by inadequate resolution, and in which direction?

3. Add noise to the output with standard deviation equal to that of the response, and estimate $\hat{H}_1$, $\hat{H}_2$ and $\hat{\gamma}^2_{xy}$ with $L = 8192$. Verify Eq. (6.41) bin by bin. Where does $\hat{H}_2$ fail, and does the coherence there agree with $\gamma^2 = [1 + S_{nn}/(|H|^2 S_{uu})]^{-1}$?

4. Add noise instead to the input, with variance equal to one third of the input variance. Predict the coherence and the bias of $\hat{H}_1$ from Eq. (6.39), then confirm both from the estimates.

5. Estimate the standard deviation of the phase of $\hat{H}_1$ at 3 Hz from 200 independent simulations with $n_d = 16$, and compare with Eq. (6.42).

Problem 6.6

Extension

Design and diagnosis.

1. A structure has modes near 12 Hz and 12.6 Hz with $\zeta \approx 0.01$, and its FRF is to be measured with $\epsilon_r \le 0.05$ where the coherence is 0.9. Using the four-bin rule, Eq. (6.42) and Eq. (6.12) with a Hann window at 50% overlap, find the required $L\Delta t$, the number of averages, and the total record length, for $f_s = 200$ Hz. Is the Hann window's 3 dB width from Table 6.1 sufficient to separate the two modes at that L?

2. Two sinusoids of 1 V and 1 mV at 50.0 and 53.3 Hz are sampled at 1 kHz for $N = 1024$ samples. Using Table 6.1 and Fig. 6.1, decide which windows allow the weaker line to be seen above the leakage of the stronger, and confirm by computing the modified periodogram with each. Which window gives the most accurate amplitude for the 1 mV line, and why?

3. Take the AR(2) record of Problem 6.3 with $\rho = 0.2$, $N = 8192$, and form $y_n = x_{n-7}$. Estimate the phase spectrum $\theta_{xy}(f)$ and show that its slope gives the delay of seven samples. Now add independent white noise to y with the same variance as x and repeat: over which band of frequencies can the delay still be read, and how does that band relate to the coherence?

4. A colleague reports a coherence of 0.3 between a force and an acceleration at a frequency between two resonances, from $n_d = 4$ averages, and concludes that the structure is nonlinear there. List every alternative explanation in the order you would test them, and state what measurement would rule each one out.

Problem 6.7

Computational Experimental

Multitaper estimates and the harmonic F-test. Use `psdmt` and `mtlines` of Appendix A.13.

1. For $N = 512$ and $TW = 2, 3, 4, 6$, compute the Slepian tapers and their concentrations λ_k. For each TW, how many tapers have $1 - \lambda_k < 10^{-3}$? Compare with $2TW - 1$.

2. Generate 2000 records of Gaussian white noise, $N = 512$. With $TW = 4$ and $K = 7$, confirm that the plain multitaper average Eq. (6.19) has $\nu = 2K$, and that the fraction of interior frequencies at which F of Eq. (6.22) exceeds the 95% and 99% points of $F_{2,12}$ is 5% and 1%.

3. Add $A\cos(2\pi f_0 n\Delta t)$ with $f_0 = 0.2031$ Hz, $\Delta t = 1$ s, to each record. For $A = 0.1, 0.2, 0.3, 0.5$, find the fraction of records in which a line is detected within $\pm W$ of f_0 at the $1 - 1/N$ point, with and without eightfold zero padding. Explain the difference in terms of where f_0 falls on the Fourier grid.

4. Take the Duffing record of Problem 6.2. Its forcing frequency, $1/(2\pi)$ Hz, is known. Test its first five odd and first five even harmonics at the 99% point and report the amplitudes $2\,|\hat{\mu}|$ that are significant. The equation is unchanged by $(x, t) \mapsto (-x, t + \pi)$; which harmonics does that symmetry remove from the phase-locked response, and does the test agree? Why would synchronous averaging be awkward for this record?

5. For the two-well record, compute $F(f)$ with $TW = 4$ and $K = 3, 5, 7$. In each of the bands 12–18 Hz and 65–78 Hz, compare the fraction of frequencies above the 99% point of $F_{2,2K-2}$ with 1%, and plot the adaptive degrees of freedom Eq. (6.21). Relate what you find to the concentrations λ_k and to the dynamic range of the spectrum.

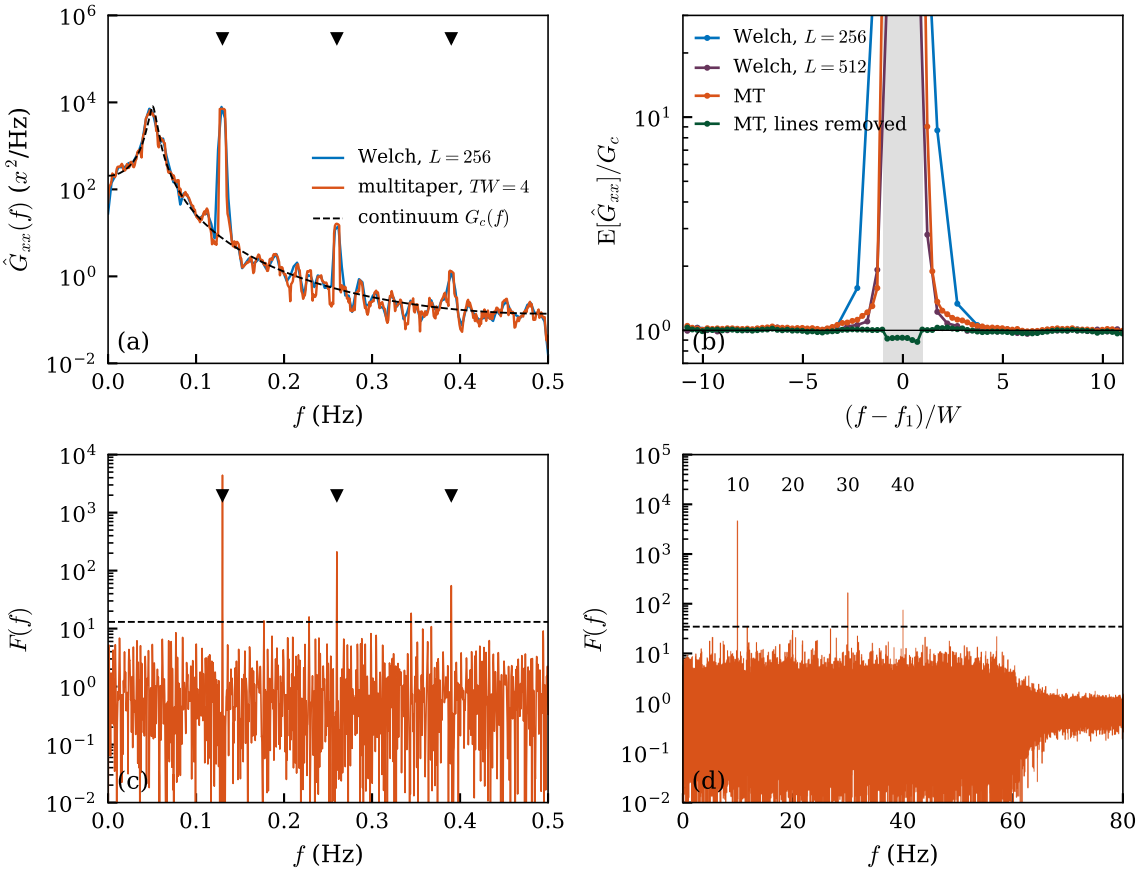


Figure 6.4: Multitaper estimates and the harmonic F-test. (a) One record of the AR(2) process Eq. (6.14) ($\rho = 0.05$, $\Delta t = 1$ s, $N = 1024$) plus three lines at $f_1 = 0.13$ Hz and its harmonics (triangles), amplitudes 10, 0.5, 0.1: Welch (Hann, $L = 256$, 50% overlap) and adaptive multitaper ($TW = 4$, $K = 7$) against the exact continuum. (b) Mean of estimate over continuum beside the strong line, 600 records, against offset in units of $W = 0.0039$ Hz; MT is the adaptive multitaper estimate; the shaded band is $\pm W$. After the lines are removed by the F-test, the continuum is unbiased outside $\pm W$. (c) $F(f)$ of the record in (a), zero-padded eightfold; dashed, the $1 - 1/N$ point of $F_{2,12}$. (d) $F(f)$ of the full two-well record (96,000 samples, $TW = 4$, $K = 7$); labels mark the forcing harmonics; dashed, the $1 - 1/N$ point. Above about 60 Hz the spectrum is more than 100 dB below the 10 Hz line, the seventh taper's leakage dominates the residual, and F loses its upper tail (see the pitfall box).

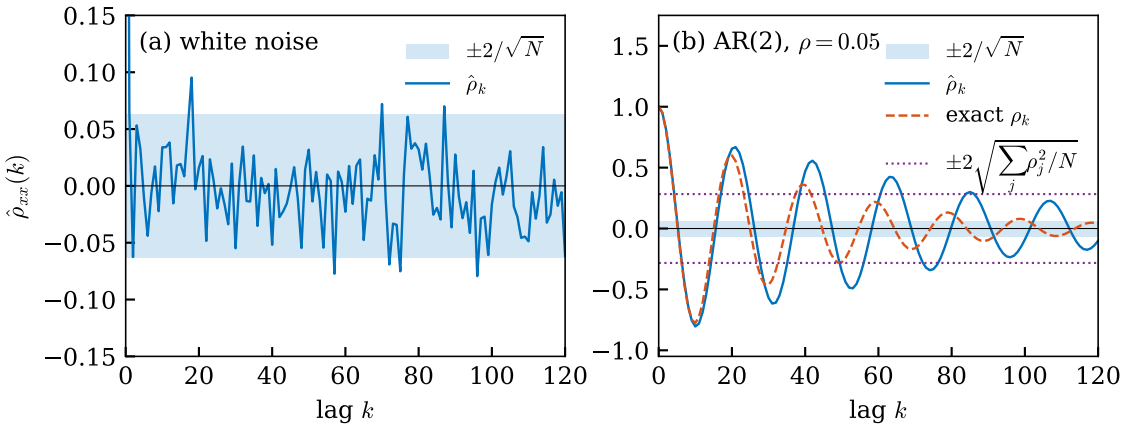


Figure 6.5: Sample autocorrelation of one record of $N = 1000$ samples with the white-noise band $\pm 2/\sqrt{N} = \pm 0.063$ of Eq. (6.28). (a) Gaussian white noise: the estimates scatter within the band, with a few excursions as expected. (b) The AR(2) process Eq. (6.14), $\rho = 0.05$: the estimate follows the exact ρ_k closely at small lags but by lag 100 has drifted from it by amounts far exceeding the white band. The dotted lines are the large-lag band of Eq. (6.27), $\pm 2\sqrt{\sum_j \rho_j^2/N} = \pm 0.28$, with $\sum_j \rho_j^2 = 20.0$ for this process: the correct scale of the error, and 4.5 times the white-noise band.

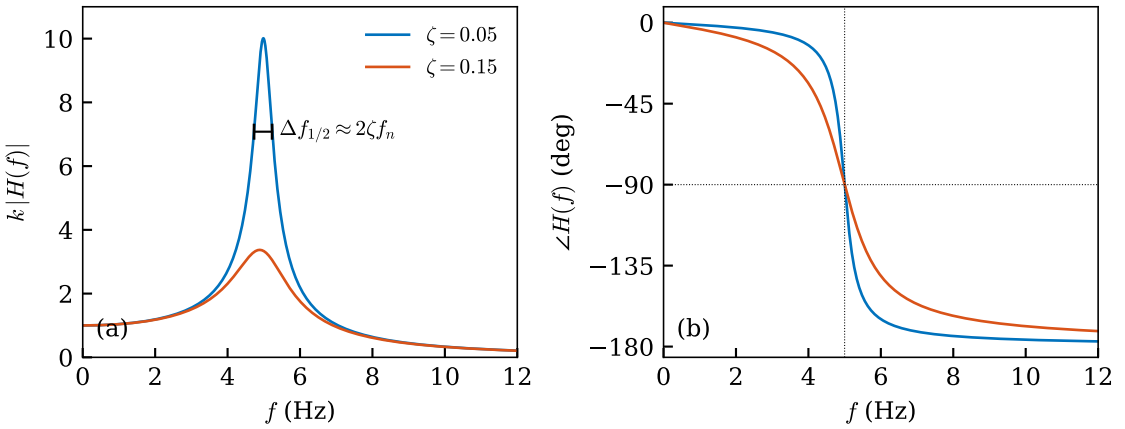


Figure 6.6: Receptance Eq. (6.44) of the oscillator Eq. (6.43) with $f_n = 5$ Hz, for $\zeta = 0.05$ and $\zeta = 0.15$, normalized by the static deflection $1/k$. (a) Magnitude, with the half-power points of the lightly damped curve marked: the exact width between them is $\Delta f_{1/2} = 0.501$ Hz, against $2\zeta f_n = 0.5$ Hz from Eq. (6.46). (b) Phase, which passes through $-90°$ at f_n for any damping; the steepness of the passage, like the height of the peak, measures ζ.

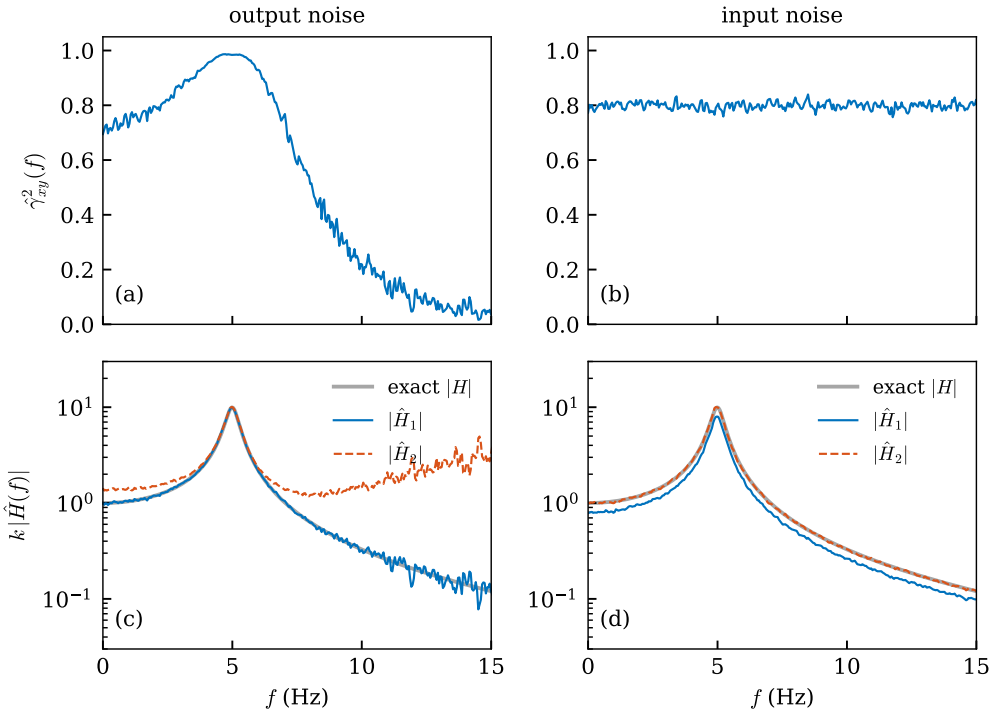


Figure 6.7: Coherence and FRF estimates for the oscillator of Fig. 6.6 ($\zeta = 0.05$) driven by white noise, $K = 399$ Hann segments of $L = 2048$ samples at 50% overlap. Left: uncorrelated noise added to the measured output, with variance one quarter of the response variance. Right: uncorrelated noise added to the measured input, with variance one quarter of the input variance. (a) Coherence follows the output signal-to-noise ratio: near one at resonance, falling to 0.06 at 12 Hz where the response is weak. (b) Coherence is flat at $0.80 = 1/(1 + S_{mm}/S_{uu})$, independent of frequency. (c) $\hat{H}_1$ follows the exact magnitude everywhere; $\hat{H}_2$ is biased high by the factor $1 + S_{nn}/(|H|^2 S_{uu})$, reaching 12.7 at 12 Hz. (d) $\hat{H}_2$ follows the exact magnitude; $\hat{H}_1$ is biased low by the constant factor 0.80 at every frequency. In both columns $\hat{H}_1/\hat{H}_2$ equals the coherence in the panel above, by Eq. (6.41).

Part II

Reconstruction and Invariants

Part II — Reconstruction and Invariants

This Part asks: *what is the geometry behind the record, and how many degrees of freedom does it carry?* The answer is a reconstructed phase space and a small set of numbers that do not depend on how the measurement was made.

Two choices control everything downstream: the delay τ and the embedding dimension d. Neither has a correct value. Both have a range over which the answers do not move, and the working practice of this Part is to find that range and report across it.

The invariants come last for a reason. A correlation dimension or a Lyapunov exponent is a slow estimator. It needs a scaling region, a Theiler window, and thousands of orbits. Each of Chapters 11 and 12 spends as much space on when the estimate fails as on how to compute it.

The system is forced at a known period.
Chapter 7. Strobe first. A stroboscopic section turns a flow into a map, removes one dimension, and makes a period-doubling cascade visible without any reconstruction at all.

The record is autonomous, or the forcing is unknown.
Chapter 8, then 9 and 10. Reconstruct in delay coordinates and choose τ and d from the record itself.

Choosing the delay.
Chapter 9. Average mutual information for a first minimum; the autocorrelation for a sanity check. A short delay collapses the reconstruction onto the hyperdiagonal, a long one folds it.

Choosing the dimension.
Chapter 10. False nearest neighbors with the composite fraction, Cao's E_1 and E_2 alongside, and the record-length bound $N \gtrsim r^d$ checked before either is believed.

You want a number that identifies the attractor.
Chapter 11 for D_2, Chapter 12 for λ_1. Report both over a range of d, with the scaling region shown, not asserted.

The record is short or noisy.
Read the pitfalls first: Sections 10.6, 12.5. A dimension estimate from a few hundred points is an estimate of the record length.

Everything in this Part assumes a single scalar channel from a stationary system on a fixed attractor. Multiple channels are the subject of Chapter 13; a system whose attractor is moving is the subject of Chapter 17.

7 Bifurcations and Poincaré Maps

In this chapter, the purely nonlinear phenomenon of *bifurcations* will be considered. The standard references are Guckenheimer and Holmes [82] for the theory and Strogatz [195] for a first course; we follow their terminology throughout. Mathematically, bifurcation theory studies the changes in the qualitative or topological structure of the solutions of a family of differential equations. We can say that bifurcation occurs in a dynamical system when a small smooth change of a (bifurcation) parameter causes a sudden 'qualitative' or topological change in the dynamical system's behavior.

7.1 Stability of Equilibrium Points

In nonlinear systems, the transition from stable to unstable behavior is of practical concern since it can happen due to *perturbations* (global change) or due to the smooth drift in parameters (local bifurcations). One or both of these are possible during bifurcation: (1) one set of solutions can lose stability, while others can become stable, and (2) some solutions can appear or disappear suddenly. To understand bifurcations, we need to understand the stability of fixed points and steady states. We first start by considering stability of the static equilibrium points. Consider a two-well Duffing equation:

$$\ddot{x} + \dot{x} - x + x^3 = 0\,. \tag{7.1}$$

In static equilibrium both velocity and the acceleration are set to zero ($\ddot{x} = \dot{x} = 0$), thus

$$-x + x^3 = 0\,, \tag{7.2}$$

which has three solutions $x = 0$, and $x = \pm 1$.

To examine the stability of these equilibrium points, we study the behavior of small perturbations ($|\delta| \ll 1$) about the equilibrium point:

$$x = x_e + \delta\,, \quad \Rightarrow \quad \dot{x} = \dot{\delta} \quad \text{and} \quad \ddot{x} = \ddot{\delta}\,. \tag{7.3}$$

Then, our equation can be rewritten in terms of δ as:

$$\ddot{\delta} + \dot{\delta} - \delta + \delta^3 + 3x_e^2\delta + 3x_e\delta^2 = 0\,. \tag{7.4}$$

Now ignoring the terms of $\mathcal{O}(\delta^2)$ or higher, we get:

$$\ddot{\delta} + \dot{\delta} + (3x_e^2 - 1)\delta = 0\,. \tag{7.5}$$

Please note, this is equivalent to the Taylor series expansion of Eq. (7.1) about the equilibrium point.

Now we can see that for $x_e = 0$, we have a negative stiffness, and thus the solutions to Eq. (7.5) would experience exponentially growing (or unstable) motion. In contrast, the solutions for $x_e = \pm 1$

would experience decaying (or stable) motion. Therefore, we have observed how this nonlinear system behaves locally in the vicinity of equilibrium points. In general, we want to use this local behavior to gain insight into the entire nonlinear behavior.

7.1.1 Classification of Linear Two-Dimensional Flows

Generically, single-degree-of-freedom oscillator can be linearized about the equilibrium point by the following two-dimensional flow:

$$\begin{aligned} \dot{x} &= ax + bv\,, \\ \dot{v} &= cx + dv\,. \end{aligned} \tag{7.6}$$

For an invariant solution to Eq. (7.6) we are looking for straight line trajectories on a plane:

$$\boldsymbol{x}(t) = e^{\lambda t}\boldsymbol{\xi}\,, \tag{7.7}$$

where $\boldsymbol{x}(t) = [x(t),\, v(t)]^{\mathrm{T}}$ is our state vector, $\boldsymbol{\xi}$ is some fixed vector (determining the direction of the invariant trajectory or eigenvector), and λ is the corresponding growth rate or eigenvalue. To find these variables, we substitute Eq. (7.7) into Eq. (7.6) (i.e., $\dot{\boldsymbol{x}} = \boldsymbol{A}\boldsymbol{x}$), where

$$\boldsymbol{A} = \begin{bmatrix} a & b \\ c & d \end{bmatrix}\,, \tag{7.8}$$

which results into $\lambda e^{\lambda t}\boldsymbol{\xi} = e^{\lambda t}\boldsymbol{A}\boldsymbol{\xi}$. Now, cancelling $e^{\lambda t}$ on both sides results in the following *eigenvalue problem*:

$$\boldsymbol{A}\boldsymbol{\xi} = \lambda\boldsymbol{\xi}\,, \tag{7.9}$$

or

$$(\boldsymbol{A} - \lambda\boldsymbol{I})\boldsymbol{\xi} = 0\,. \tag{7.10}$$

To satisfy the above equation for a nontrivial $\boldsymbol{\xi}$, we need $\det(\boldsymbol{A} - \lambda\boldsymbol{I}) = 0$, or

$$\det\begin{pmatrix} a-\lambda & b \\ c & d-\lambda \end{pmatrix} = 0\,, \tag{7.11}$$

which gives us the *characteristic equation*:

$$\lambda^2 - \tau\lambda + \Delta = 0\,, \tag{7.12}$$

where $\tau = \mathrm{trace}(\boldsymbol{A}) = a + d$ and $\Delta = \det(\boldsymbol{A}) = ad - bc$ (in this section only; later in the chapter τ is a flow time and, elsewhere in the book, a delay, and Δ is a sampling interval in Chapter 5). Therefore,

$$\lambda_{1,2} = \frac{\tau \pm \sqrt{\tau^2 - 4\Delta}}{2}\,. \tag{7.13}$$

Thus, the eigenvalues also called *characteristic exponents* or CEs, depend only on the trace and determinant of $\boldsymbol{A}$.

For $\lambda_1 \neq \lambda_2$, we can find linearly independent $\boldsymbol{\xi}_1$ and $\boldsymbol{\xi}_2$ that span the whole phase (state) plane. Thus, any initial condition can be written as:

$$\boldsymbol{x}_0 = c_1\boldsymbol{\xi}_1 + c_2\boldsymbol{\xi}_2\,, \tag{7.14}$$

which can be used to write the solution corresponding to that initial condition as a linear combination of solutions to $\dot{\boldsymbol{x}} = \boldsymbol{A}\boldsymbol{x}$:

$$\boldsymbol{x}(t) = c_1 e^{\lambda_1 t}\boldsymbol{\xi}_1 + c_2 e^{\lambda_2 t}\boldsymbol{\xi}_2\,. \tag{7.15}$$

Therefore, by the existence and uniqueness property, this is the *only* solution.

The type of the solution depends on the values of τ and $\tau^2 - 4\Delta$, and we have the following possibilities:

1. $\Delta < 0$, results in two real eigenvalues, with one positive and one negative. This characterizes a *saddle point* equilibrium that has one unstable eigendirection for $\lambda > 0$ and one stable eigendirection for $\lambda < 0$.

2. $\Delta > 0$ and $\tau^2 < 4\Delta$, results in two complex conjugate eigenvalues, for which we could have three cases:

   (a) If $\tau < 0$ we have a *stable spiral* solution;

   (b) If $\tau > 0$ we have a *unstable spiral* solution; and

   (c) If $\tau = 0$ we have a *center* solution.

3. $\Delta > 0$ and $\tau^2 > 4\Delta$, results in two real eigenvalues:

   (a) For $\tau < 0$, both are negative and we have a *stable node*; and

   (b) For $\tau > 0$, both are positive and we have a *unstable node*.

4. $\Delta = 0$, we have $\lambda_1 = 0$ and $\lambda_2 = \tau$, which gives a *non-isolated fixed point* or an equilibrium line span by $\boldsymbol{\xi}_1$ is either attracting ($\tau < 0$) or repelling ($\tau > 0$).

5. $\tau^2 = 4\Delta$, we have repeated eigenvalues that give *degenerate nodes*. In special cases, when the matrix $\boldsymbol{A}$ is diagonal (i.e., $a = d$ and $b = c = 0$), we can also get *stars*.

For the illustration of all possible phase portraits, please see the Fig. 7.1.

For the damped oscillator $m\ddot{x} + c\dot{x} + kx = 0$ linearized about an equilibrium, $a = 0$, $b = 1$, $c \to -k/m$, $d \to -c/m$, so that

$$\tau = -2\zeta\omega_n\,, \qquad \Delta = \omega_n^2\,, \qquad \omega_n = \sqrt{k/m}\,, \quad \zeta = \frac{c}{2\sqrt{km}}\,, \tag{7.16}$$

and the eigenvalues are $\lambda_{1,2} = -\zeta\omega_n \pm \omega_n\sqrt{\zeta^2 - 1}$. The classification above is then the familiar one: $0 < \zeta < 1$ (i.e., $\tau^2 < 4\Delta$) gives the underdamped stable spiral, $\zeta > 1$ the overdamped stable node, $\zeta = 1$ the critically damped degenerate node, $\zeta = 0$ the center, and $\zeta < 0$ (negative damping) the unstable cases; a negative stiffness, $k < 0$, gives $\Delta < 0$ and the saddle. The same (ω_n, ζ) pair fixes the frequency response function of Section 6.6, where the peak height $1/(2\zeta)$ and half-power bandwidth $2\zeta f_n$ measure the two parameters from data.

When linearization decides

The classification is exact for the linear system Eq. (7.6). For the nonlinear system it decides the stability of the equilibrium only when the equilibrium is *hyperbolic*, that is, when no eigenvalue has zero real part ($\Delta \neq 0$ and $\tau \neq 0$ in the complex case). In that case the Hartman–Grobman theorem [82] guarantees that the nonlinear flow near the equilibrium is topologically equivalent to its linearization: a linear saddle, node, or spiral remains one. When an eigenvalue has zero real part (a center, $\tau = 0$; or $\Delta = 0$) the linearization is inconclusive and the higher-order terms decide. The conservative two-well oscillator of Problem 5.1 with $\epsilon = 0$ is exactly this case, and it is why the potential-energy argument of the next section is needed.

7.1.2 Connection to Potential Energy

For one-degree-of-freedom system with a potential function $V = V(x)$, equilibrium points are located at the extrema of the potential determined by solving the following

$$\frac{\mathrm{d}V}{\mathrm{d}x} = 0\,. \tag{7.17}$$

In general, we can expand the potential into Taylor series near the equilibrium point $x = x_e + \delta$:

$$\begin{aligned} V(\delta) &= V(x_e) + \cancelto{0}{\left.\frac{\mathrm{d}V}{\mathrm{d}x}\right|_{x=x_e}}\delta + \frac{1}{2!}\left.\frac{\mathrm{d}^2V}{\mathrm{d}x^2}\right|_{x=x_e}\delta^2 + \mathcal{O}(\delta^3) \\ &= V(x_e) + \frac{1}{2}\left.\frac{\mathrm{d}^2V}{\mathrm{d}x^2}\right|_{x=x_e}\delta^2 + \mathcal{O}(\delta^3)\,, \end{aligned} \tag{7.18}$$

where $V(x_e)$ is an arbitrary construct and can be set to zero. Then, we can write:

$$V(\delta) = \frac{1}{2}\left.\frac{\mathrm{d}^2V}{\mathrm{d}x^2}\right|_{x=x_e}\delta^2\,. \tag{7.19}$$

Thus, if $\left.\frac{\mathrm{d}^2V}{\mathrm{d}x^2}\right|_{x=x_e} > 0$ we have a local minimum, if $\left.\frac{\mathrm{d}^2V}{\mathrm{d}x^2}\right|_{x=x_e} < 0$ a local maximum, and if $\left.\frac{\mathrm{d}^2V}{\mathrm{d}x^2}\right|_{x=x_e} = 0$ we need to go to higher-order approximation (e.g., a locally cubic potential has an inflection point at the equilibrium, which is attracting from one side and repelling from the other: a semi-stable equilibrium).

Now, we can use the following theorem to specify when the equilibrium is stable.

Lagrange–Dirichlet theorem

> For a conservative mechanical system, an equilibrium state at which the potential energy has a strict (isolated) local minimum is stable in the sense of Lyapunov.

The theorem gives the minimum ⇒ stable direction only, and for conservative systems; the total energy serves as the Lyapunov function. The converse, that a maximum implies instability, is a separate and harder result (the Lyapunov and Chetaev instability theorems), and it needs hypotheses on how the maximum is detected [82]. For the one-degree-of-freedom systems considered here it is not needed: when $V''(x_e) < 0$ the linearized stiffness is negative, $\Delta < 0$, the equilibrium is a hyperbolic saddle, and the classification of the previous section already decides that it is unstable, with or without damping. That is what was used for the origin of Eq. (7.1), where $3x_e^2 - 1 = -1 < 0$. With damping added, a minimum of V becomes asymptotically stable (a stable spiral or node), again by linearization when $V''(x_e) > 0$.

In summary, referring to Fig. 7.2 we can say: if the potential energy has a local minimum at an equilibrium point, it is stable; if the potential has a local maximum at equilibrium, it is unstable; and if the potential has an inflection point, we have a semi-stable equilibrium.

7.2 Bifurcations

To illustrate what bifurcations are, let us consider the Duffing oscillator [49, 118]:

$$\ddot{x} + \gamma\dot{x} - \beta x + x^3 = F\cos\omega t\,. \tag{7.20}$$

Generally, we start the analysis by determining the *static equilibria* of the unforced system ($F = 0$). In static equilibrium both velocity and the acceleration are set to zero ($\ddot{x} = \dot{x} = 0$), thus

$$-\beta x + x^3 = 0\,, \tag{7.21}$$

which has three possible solutions $x = 0$, and $x = \pm\sqrt{\beta}$ if $\beta \geq 0$. As we vary β from $-\infty$ to ∞, our system goes from one real solution ($x = 0$ for $\beta < 0$), to three equilibrium solutions as β passes through 0. This type of change in the possible solutions is called a "pitchfork bifurcation." Please note that the $x = 0$ solution also changes character as β passes through 0. Linearizing about the origin gives $\lambda^2 + \gamma\lambda - \beta = 0$, so $\lambda = \left(-\gamma \pm \sqrt{\gamma^2 + 4\beta}\right)/2$. For $\beta > 0$ the square root exceeds γ and the two roots have opposite signs: the origin is a *saddle*. For $\beta < 0$ it is stable, a node for $-\gamma^2/4 < \beta < 0$ and a spiral only once $\beta < -\gamma^2/4$. Therefore, bifurcations are also associated with changes in the stability. The illustration of this bifurcation is shown in Fig. 7.3.

7.2.1 Classification of Instability

The nature of the loss of stability relates to how the eigenvalues (i.e., CEs) exit the negative real half-plane as a single bifurcation (or control) parameter is changed, which also determines the bifurcation type. For example, if a real eigenvalue passes through zero (i.e., Δ changes sign), the fixed point changes between node and saddle type, see Fig. 7.4(a). This is the common signature of all *steady* (static) bifurcations—*saddle-node* (also called *fold* or *divergence*), *transcritical*, and *pitchfork*—and the linearization alone cannot tell them apart: which one occurs is decided by the symmetry and the nonlinear terms of the system. In a saddle-node bifurcation a saddle and a node collide and annihilate, so that the fixed point ceases to exist past the bifurcation; in the pitchfork of the Duffing example above, the origin persists but changes from node to saddle while two new equilibria are born. In mechanical or engineered systems a real eigenvalue crossing zero corresponds to buckling (stiffness goes to zero and then becomes negative): a pitchfork for a perfectly symmetric structure and a saddle-node (fold) once imperfections break the symmetry. In contrast, if the real parts of the complex conjugate eigenvalues change sign (i.e., $\tau < 0 \to \tau > 0$), we effectively change the sign of damping. This type of bifurcation is called *Hopf* [97], after the 1942 paper whose English translation appears in Marsden and McCracken [132]; see Fig. 7.4(b), and common mechanical examples are flutter and vortex-induced vibrations. In both cases an eigenvalue on the imaginary axis is necessary but not sufficient. The eigenvalues must also cross the axis with nonzero speed as the parameter varies (transversality), and a coefficient of the nonlinear terms must not vanish (nondegeneracy); for the Hopf bifurcation this coefficient is the first Lyapunov coefficient, whose sign decides whether the bifurcating limit cycle is stable [82].

7.3 Stability and Bifurcation of Cycles

We start by considering the logistic map [139]:

$$x_{n+1} = \mu x_n(1 - x_n)\,. \tag{7.22}$$

Equilibria or "fixed points" for the map are actually *periodic orbits* since $x_{n+1} = x_n$ defines the fixed point. To find fixed points of the map, set $x_n \equiv x$ for "period 1" or P-1 fixed point to get:

$$x = \mu x(1 - x)\,, \tag{7.23}$$

or

$$x(1-\mu) = -\mu x^2 \,. \tag{7.24}$$

So $x = 0$, and $x = \frac{\mu-1}{\mu}$ if $\mu \neq 0$, are the P-1 orbits or fixed points.

To evaluate the stability near $x = \frac{\mu-1}{\mu}$, let $x_n = \frac{\mu-1}{\mu} + \delta_n$, where $|\delta_n| \ll 1$ is a small perturbation from the fixed point. Thus, the Logistic map can be written as:

$$\frac{\mu-1}{\mu} + \delta_{n+1} = \mu \left(\frac{\mu-1}{\mu} + \delta_n \right) \left(1 - \frac{\mu-1}{\mu} - \delta_n \right) \,, \tag{7.25}$$

or

$$\delta_{n+1} = (2-\mu)\delta_n - \mu\delta_n^2 \,. \tag{7.26}$$

Remembering that we have only *small* perturbations, we get

$$\delta_{n+1} \cong (2-\mu)\delta_n \,. \tag{7.27}$$

For *asymptotic* stability we want $|\delta_n| \to 0$ as $n \to \infty$, which requires $|2-\mu| < 1$, or $-1 < 2-\mu < 1$. Thus, for asymptotic stability we need:

$$1 < \mu < 3 \,. \tag{7.28}$$

At the end points $|2-\mu| = 1$ the fixed point is non-hyperbolic and the linearization is inconclusive; the neglected term $-\mu\delta_n^2$ decides, and these are exactly the bifurcation values. The same calculation at the other fixed point $x = 0$ gives $\delta_{n+1} \cong \mu\delta_n$, so the origin is asymptotically stable for $0 < \mu < 1$ and loses stability at $\mu = 1$, where $x = \frac{\mu-1}{\mu}$ "collides" with the $x = 0$ solution and takes over its stability (a transcritical bifurcation). Also, $2-\mu$ switches sign at $\mu = 2$ as shown in Fig. 7.5(a). Then the approach to the steady state P-1 orbit will have either monotonic as in Fig. 7.5(b) or oscillatory as in Fig. 7.5(c) character.

Now the question is what happens when $\mu = 3$? To answer this, consider a "period 2" (P-2) fixed point $x_{n+2} = x_n$:

$$x_{n+2} = \mu x_{n+1}(1-x_{n+1}) = \mu^2 x_n(1-x_n)\left(1-\mu x_n(1-x_n)\right) \,. \tag{7.29}$$

We seek the solution for $x_{n+2} = x_n \equiv x$:

$$x = \mu^2 x(1-x)\left(1-\mu x(1-x)\right) \,. \tag{7.30}$$

The roots to the above equation are the roots for the P-1 fixed points (as P-1 orbits are also a P-2 fixed points) plus

$$x_{a,b} = \frac{1}{2} + \frac{1}{2\mu} \pm \frac{1}{2\mu}\sqrt{\mu^2 - 2\mu - 3} \,. \tag{7.31}$$

For roots $x_{a,b}$ to exist, we need to satisfy the following condition

$$\mu^2 - 2\mu - 3 \geq 0 \,,$$

which is only satisfied for $\mu \leq -1$ and $\mu \geq 3$. Therefore, at $\mu = 3$, the P-1 fixed point $x = \frac{\mu-1}{\mu}$ becomes unstable and $x_{a,b}$ become stable (which we have not shown here). Bifurcation points like this (see Fig. 7.6, when stable P-1 fixed point looses stability and P-2 orbits become stable, are called *period-doubling bifurcation points*.

Doing all this, we are trying to make a point that with maps, we can study bifurcations of periodic motions (and even nonperiodic or chaotic motions) since they (i.e., periodic orbits) "look like" static equilibria.

7.4 Dynamic Bifurcations in Continuous Systems

The study of the stability of continuous dynamical systems can be converted into the study of the stability of maps using the idea of *Poincaré Map*. We start exploring this idea by considering an example of dynamical system:

$$\dot{\boldsymbol{x}} = \boldsymbol{f}(\boldsymbol{x})\,, \tag{7.32}$$

where $\boldsymbol{x} \in \mathbb{R}^3$. A solution starting from some initial condition $\boldsymbol{x}(0) = \boldsymbol{x}_0$ can be written in the following form:

$$\boldsymbol{x}(t) = \boldsymbol{X}(\boldsymbol{x}_0, t) = \boldsymbol{X}_t(\boldsymbol{x}_0)\,. \tag{7.33}$$

For a fixed time $t = \tau$, $\boldsymbol{X}_\tau(\boldsymbol{x}) : \mathbb{R}^3 \to \mathbb{R}^3$. This can be used to develop a corresponding map equation:

$$\boldsymbol{x}_{n+1} = \boldsymbol{X}_\tau(\boldsymbol{x}_n)\,, \tag{7.34}$$

where $\boldsymbol{x}_n = \boldsymbol{x}(n\tau)$, so that $\boldsymbol{x}_{n+1} = \boldsymbol{x}((n+1)\tau)$. Then, if the original continuous system has a P-1 solution with a period of $\tau = T$ (i.e., $\boldsymbol{X}(\boldsymbol{x}_0, t) = \boldsymbol{X}(\boldsymbol{x}_0, t+T)$), we can write:

$$\boldsymbol{x}_{n+1} = \boldsymbol{X}_T(\boldsymbol{x}_n) = \boldsymbol{x}_n\,. \tag{7.35}$$

Alternatively we can define a surface or manifold in $\mathbb{R}^3$ using:

$$\Sigma = \{\boldsymbol{x} \mid g(\boldsymbol{x}) = 0, h(\boldsymbol{x}) > 0\}\,, \tag{7.36}$$

where g and h are scalar functions and we require *transversality*, $\boldsymbol{f}(\boldsymbol{x}) \cdot \nabla g(\boldsymbol{x}) \neq 0$, $\forall\, \boldsymbol{x} \in \Sigma$, and, more specifically, $\boldsymbol{f} \cdot \nabla g > 0$, $\forall\, \boldsymbol{x} \in \Sigma$, refer to Fig. 7.7. The first condition requires that trajectories actually pierce the Σ manifold rather than touch it tangentially; they need not cross it perpendicularly. The second condition requires counting only points generated by trajectories going through the Σ surface from only one side. Simplest case for g and h functions is:

$$g(\boldsymbol{x}) = x_1 = 0 \quad \text{and} \quad h(\boldsymbol{x}) = x_2 > 0\,. \tag{7.37}$$

Therefore, the Poincaré section Σ is a half-plane $x_1 = 0$, $x_2 > 0$. The point is that Σ is transverse to the flow $\boldsymbol{X}_T$.

Given Σ, *uniqueness* of solution to ODE defines a *map* form:

$$\boldsymbol{x}_{n+1} = \boldsymbol{X}_{\tau_n}(\boldsymbol{x}_n) \triangleq \boldsymbol{P}(\boldsymbol{x}_n)\,, \tag{7.38}$$

where $\boldsymbol{x}_n \in \Sigma$ and τ_n is defined by first t such that $\boldsymbol{x}_{n+1} \in \Sigma$. Therefore, $\boldsymbol{P} : \Sigma \to \Sigma$ is the Poincaré map as shown in Fig. 7.8(a). Now, we can ask a question: what does a fixed point of $\boldsymbol{P}$, $\boldsymbol{x}^* = \boldsymbol{P}(\boldsymbol{x}^*)$, mean? Let us look at Fig. 7.8(b), where we can see that if $\boldsymbol{x}^* = \boldsymbol{P}(\boldsymbol{x}^*)$, $\boldsymbol{x}^*$ is the intersection of a P-1 orbit with Σ. Thus, each fixed point of $\boldsymbol{P}$ corresponds to a *unique periodic orbit*.

The return times τ_n between successive visits to Σ need not be equal, and in general are not. The discrete-time map defined by $\boldsymbol{P}$ is therefore *asynchronous*: its index n counts crossings, not equal intervals of time.

7.4.1 Periodic Vector Fields

Consider a driven oscillator:

$$\dot{\boldsymbol{x}} = \boldsymbol{f}(\boldsymbol{x}, t)\,, \tag{7.39}$$

where $\boldsymbol{f}(\boldsymbol{x}, t+T) = \boldsymbol{f}(\boldsymbol{x}, t)$ and $T = 2\pi/\omega$ (i.e., ω is the forcing frequency). If $\boldsymbol{x} \in \mathbb{R}^2$, then phase space $\mathcal{S} = \mathbb{R}^2 \times \mathbb{S}^1$ (i.e., direct product of a plane and a circle). For this particular case, a commonly used definition of Poincaré section is:

$$\Sigma = \left\{ (\boldsymbol{x}, t) \;\middle|\; t \mod \frac{2\pi}{\omega} = \varphi \right\} . \tag{7.40}$$

This is called a *stroboscopic* Poincaré section. Figs. 7.9 and 7.10 show how data is sampled every $T = 2\pi/\omega$ time period at a constant forcing phase φ; Fig. 7.9(c) shows the resulting section for the chaotic response of the book's Duffing oscillator, the set that is studied further in Problem 7.5 and Chapter 11.

Making the strobe land on a sample

In a simulation there is no need to interpolate for the section. Choose the integration time step to divide the forcing period,

$$\Delta t = \frac{2\pi}{\omega n} , \tag{7.41}$$

for an integer $n \geq 1$. Every nth step then lands exactly on Σ, and no root-finding is required. The same run also supplies n different sections, one for each phase $\varphi = 2\pi j/n$, $j = 0, \ldots, n-1$; played in order they animate how the section deforms through the forcing cycle. The measured record of Section 7.4.2 works the same way, because its sampling rate is an integer multiple of the drive frequency.

7.4.2 A Measured Stroboscopic Section

Of everything in this chapter, the stroboscopic section is what a measured record gives up most easily. The two-well oscillator record of Section 1.4.1 carries 16 samples per forcing period, and that number is an integer. Every sixteenth sample therefore already lies in Σ. No event detection, no interpolation and no numerical derivative is needed. The 96,000 samples supply 6000 section points, one per forcing period, returned by `section()` in `python/_twowell.py`.

Two things must be settled before the section can be drawn. The first is the sampling phase φ. The drive phase was not recorded with the file, so φ is unknown, and the four choices $\varphi = 0$, $T/4$, $T/2$ and $3T/4$ give four different point sets. They are one object seen at four instants, exactly as for the model in Fig. 7.9. The second is the missing coordinate. The section of a periodically forced planar oscillator is two-dimensional and the strain gauge supplies one number. We use the delay pair (s_k, s'_k), where s'_k is the strain four samples, a quarter of a forcing period, after s_k. This is the pseudo-velocity of Chapter 8. The model is strobed the same way in Fig. 7.11(b), so that the two panels differ in the data and not in the coordinates.

Fig. 7.11(a) is the result, and it is neither a curve nor a filled region. Two lobes, one per well, are joined by a folded band, and each lobe has a spiral core. That is the object of Fig. 7.9(c): the section of a dissipative map that stretches along one direction and folds back. The beam does what the model does.

Before reading anything else off the section, check the stride. Panel (e) scans it. For each assumed number of samples per period n from 12 to 20 the record is strobed with stride n, the section is truncated to the same 5300 points, and the mean nearest-neighbour distance in the plane is computed. A section taken at the true period is a thin set and its points have close neighbours. One taken at the wrong period is a smear of successive phases, and its nearest-neighbour distances grow. The scan has a sharp minimum at $n = 16$, 0.0192 V, against 0.026 to 0.034 V at every other

stride. The four sampling phases at $n = 16$ agree to three percent, 0.0181 to 0.0186 V. The record fixes its own strobe period, without reference to the sampling and forcing frequencies reported with it.

What the measurement adds to the model is in panels (c) and (d), the marked boxes magnified. In the model the layers of the folded band stay apart: three filaments, each a few marker widths thick, with clear gaps between them. In the record they do not. The same magnification gives a diagonal grain and no resolved filament. Three limits are at work, and none of them is a plotting choice.

1. Point count. The section holds 6000 points because the record is 6000 forcing periods long, and the section discards fifteen samples in every sixteen. The box in (c) holds 362 of them, against 701 in the model box. A fractal set needs points at every scale, and a section this size resolves three or four layers and no more.

2. Noise. The strands in (c) have a transverse width that the model's do not. Measurement noise is the natural cause; quantization is not, since the A/D step is $10/65536$ V and the section spans about 36,000 of those steps.

3. Coordinates. s'_k is a strain four samples later, not a velocity. At 16 samples per period the difference between the two is not small, and it deforms the section by a smooth but unknown change of coordinates.

The two panels are not expected to agree in detail, and should not be read as though they were. The magnet spacing, damping and drive amplitude of the rig are not those of the model, the observable is strain and not displacement, and (a) is a delay pair while Fig. 7.9(c) is $(x, \dot{x})$. What the comparison supports is the identification of the class: two lobes, one fold, a spiral core in each lobe, and a set of measure zero in the plane. Chapter 11 turns that last statement into a number.

7.4.3 Stability and Bifurcation of Periodic Orbits

For linear systems, we have Floquet theory [82, Ch. 1.5] to study local stability based on *Monodromy Matrix Eigenvalues* of the global solution for linear ordinary differential equations with periodically time-varying coefficients. With the Poincaré section, we can develop a similar procedure for evaluating the stability of close orbits for nonlinear dynamical systems. For a P-k periodic orbit we have $\boldsymbol{P}^k(\boldsymbol{x}^*) = \boldsymbol{x}^*$, and its stability is that of $\boldsymbol{x}^*$ as a fixed point of the k-fold iterate $\boldsymbol{P}^k$; to keep the notation light we write $\boldsymbol{P}$ for $\boldsymbol{P}^k$ in what follows, so that $k = 1$ is the case written out. The corresponding solution to our original dynamical system Eq. (7.39) can be expressed as:

$$\boldsymbol{x}(t, \boldsymbol{x}^*) = \boldsymbol{X}_t(\boldsymbol{x}^*)\,. \tag{7.42}$$

Now we introduce a small perturbation to the initial condition $\boldsymbol{x}_0 = \boldsymbol{x}^* + \boldsymbol{\delta}_0$ ($\|\boldsymbol{\delta}_0\| \ll 1$), then we can use $\boldsymbol{x}_n = \boldsymbol{x}^* + \boldsymbol{\delta}_n$ in $\boldsymbol{x}_{n+1} = \boldsymbol{P}(\boldsymbol{x}_n)$ to get:

$$\boldsymbol{x}^* + \boldsymbol{\delta}_{n+1} = \boldsymbol{P}(\boldsymbol{x}^* + \boldsymbol{\delta}_n) = \boldsymbol{P}(\boldsymbol{x}^*) + D\boldsymbol{P}(\boldsymbol{x}^*)\boldsymbol{\delta}_n + \mathcal{O}(\|\boldsymbol{\delta}_n\|^2)\,, \tag{7.43}$$

where $D\boldsymbol{P}(\boldsymbol{x}^*)$ is the Jacobian of the Poincaré map evaluated at $\boldsymbol{x}^*$. Then,

$$\boldsymbol{\delta}_{n+1} \approx D\boldsymbol{P}(\boldsymbol{x}^*)\boldsymbol{\delta}_n\,. \tag{7.44}$$

Now the fixed point $\boldsymbol{x}^*$ is stable if $\boldsymbol{\delta}_n \to \mathbf{0}$ as $n \to \infty$, which is equivalent to requiring that the magnitude of all the eigenvalues—called *characteristic multipliers* (CMs)—of the Jacobian are less than unity.

For a two-dimensional Poincaré section, the Poincaré map near $\boldsymbol{x}^*$, Eq. (7.44), can be written as:

$$\begin{aligned} \delta_{1,n+1} &= a\,\delta_{1,n} + b\,\delta_{2,n} \\ \delta_{2,n+1} &= c\,\delta_{1,n} + d\,\delta_{2,n} \end{aligned} \tag{7.45}$$

or

$$\boldsymbol{\delta}_{n+1} = \boldsymbol{A}\boldsymbol{\delta}_n\,. \tag{7.46}$$

where $\boldsymbol{A} = D\boldsymbol{P}(\boldsymbol{x}^*)$, and its eigenvalues are also CMs. If none of the CMs lies on the unit circle, the fixed point is termed *hyperbolic*; if one of them does, it is *non-hyperbolic*. Non-hyperbolicity is necessary for a local bifurcation of the cycle but not sufficient. A specific bifurcation (saddle-node, period-doubling, Neimark) also requires that the multipliers cross the unit circle with nonzero speed as the parameter varies (*transversality*), and that a bifurcation-specific coefficient of the nonlinear terms does not vanish (*nondegeneracy*) [82]. CMs are related to the characteristic exponents (CEs) of the ODE solution as $\mathrm{CM} = e^{\mathrm{CE}\,T}$, where T is the period of the orbit. The period is not optional: the multiplier is dimensionless while the exponent carries units of inverse time. One bookkeeping point: the monodromy matrix of the flow, $D\boldsymbol{X}_T(\boldsymbol{x}^*)$, acts on the full n-dimensional state space and carries one extra multiplier, equal to 1, whose eigenvector is the flow direction $\boldsymbol{f}(\boldsymbol{x}^*)$ along the orbit; the Jacobian of the Poincaré map acts on the $(n-1)$-dimensional section and has only the remaining $n-1$ multipliers. For an autonomous system the two sets agree apart from this trivial multiplier. Bifurcations of cycles happen when the CMs cross the unit circle. Some of the examples of bifurcations are shown in Fig. 7.12: (a) saddle-node or jump bifurcation where cycle loses stability and settles to another periodic solution; (b) period-doubling bifurcation when P-k becomes P-$2k$ and eventually (as $k \to \infty$) leads to chaos, the cascade whose universal scaling was established by Feigenbaum [64]; and (c) the Neimark bifurcation [145, 171], in which a pair of complex-conjugate multipliers crosses the unit circle and an invariant torus is born. It is less common in mechanical systems than the other two.

Problems

Problem 7.1

Computational

[Bifurcation Diagram for the Logistic Map] Consider the logistic map:

$$x_{n+1} = Ax_n(1 - x_n)\,. \tag{7.47}$$

Run the simulations with the initial condition $x_0 = 0.1$, for different values of A starting from $A = 2.9$ and going up to $A = 4$. Make increments in A small enough, e.g., $\Delta A < 0.01$ (e.g., $\Delta A = 0.001$). For each magnitude of A, record long time series and retain only the last 1,000 (or more, e.g., 4,000) points representing its *steady state* response. Now plot the bifurcation diagram by plotting all the response points versus the corresponding value of A. What can you see in the diagram?

Problem 7.2

Computational

Numerically integrate the Lorenz system described in Problem 3.2 and perform a Poincaré section. Namely, record (x, z) every time the y-coordinate equals zero and its derivative is negative. In MATLAB, pass `options = odeset('Events',@events);` to `ode45`, where `events` is the function that specifies when to sample the flow (i.e., the $y = 0$ and $\dot{y} < 0$ conditions); see ODE Event Location in the MATLAB documentation. In Python, `solve_ivp` takes the same event functions through its `events` argument (set `direction=-1`). Both locate the crossing by interpolating the solver's dense output between steps, so the section points inherit the interpolation error; tighten the tolerances until the section no longer changes. Once you have the points, plot the Poincaré section by plotting z vs. x as points.

Problem 7.3

Computational

Numerically integrate the Lorenz system described in Problem 3.2 and collect all the maxima Z_n of the z variable, the return map Lorenz [127] himself used. Again, you can use `events` ($\dot{z} = 0$, $\ddot{z} < 0$) or the `findpeaks` command in MATLAB (`scipy.signal.find_peaks` in Python) on a sufficiently densely sampled z time series. Plot the successive maxima Z_{n+1} versus Z_n. The points fall nearly on a single-humped curve: a one-dimensional map. Explain how this map is related to the two-dimensional Poincaré section of Problem 7.2 (a section is a set of points; the return map is a function on it), and why the curve is not exactly one-dimensional.

Problem 7.4

Core

Stability of the period-two orbit of the logistic map. The text found the P-2 points $x_{a,b}$ for $\mu \geq 3$ but did not show that they are stable.

1. Show that a P-2 orbit is a fixed point of the second iterate $f^2 = f \circ f$, and that by the chain rule its characteristic multiplier is the product $f'(x_a)\, f'(x_b)$ of the slopes at the two points of the orbit.

2. Using $x_a + x_b = 1 + 1/\mu$ and $x_a x_b = (1+\mu)/\mu^2$, which follow from the quadratic for $x_{a,b}$, show that $f'(x_a)f'(x_b) = -\mu^2 + 2\mu + 4$. Check that it equals $+1$ at $\mu = 3$, where the P-2 orbit is born from the P-1 point.

3. Find the range of μ over which the P-2 orbit is asymptotically stable, and show that it loses stability when the multiplier reaches -1, at $\mu = 1+\sqrt{6} \approx 3.449$. Which bifurcation of Fig. 7.12 is this, and what orbit is born there?

4. Confirm both bifurcation values from the diagram of Problem 7.1 by locating where the number of steady-state points doubles from one to two and from two to four. Then use the P-2 result to predict the sign of the multiplier at $\mu = 3.2$ and verify it: iterate from a point displaced by 10^{-3} from x_a and check whether the deviation after two steps has the same sign as the initial one, and by what factor it has shrunk (you should find 0.16).

Problem 7.5

needs Ch. 11 Computational

The stroboscopic Poincaré map of the two-well Duffing oscillator $\ddot{x} + \gamma\dot{x} - x + x^3 = F\cos t$ with $\gamma = 0.25$ and $F = 0.3$ (Section 1.4).

1. Integrate the equation and record $(x, \dot{x})$ once per forcing period $T = 2\pi$, at forcing phase $\varphi = 0$, for at least 5000 periods after the transient. Plot the section; it is the fractal set shown later in Fig. 11.14. Repeat at $\varphi = \pi/2$ and π; the section is a different set at each phase but the same object.

2. Show, from the divergence of the vector field, that the Jacobian $D\boldsymbol{P}$ of the stroboscopic map has determinant $e^{-\gamma T}$ at *every* point of the plane, not only at fixed points. Evaluate it for $\gamma = 0.25$: you should find 0.2079.

3. Estimate $D\boldsymbol{P}$ numerically at three points of the section by central differences of the map (integrate from $\boldsymbol{x} \pm h\boldsymbol{e}_j$ over one period with $h = 10^{-6}$ and tight tolerances). Report the determinant and the two eigenvalues at each point. The determinant should agree with part (2) to four digits at every point, while the eigenvalues vary from point to point; explain why one is constant and the others are not.

4. Lower F until the steady-state response is periodic (a P-1 orbit in one well), locate its fixed point $\boldsymbol{x}^*$ on the section by iterating the map, and compute the characteristic multipliers at $\boldsymbol{x}^*$. Verify that their product is again $e^{-\gamma T}$ and that both lie inside the unit circle. Convert them to characteristic exponents by $\mathrm{CE} = \ln(\mathrm{CM})/T$ and compare with the eigenvalues of the linearization about the well bottom, Eq. (7.5) with the damping coefficient γ in place of 1.

Problem 7.6

Experimental

A stroboscopic section from the two-well oscillator record of Section 1.4.1.

1. The record carries $n = 16$ samples per forcing period, so the stroboscopic sequence is $s_k = s_{1+kn}$ and no interpolation is needed. Form it and state how many section points the record supplies. Then form it again at the three other sampling phases, s_{1+j+kn} for $j = 4$, 8 and 12. Are the four sets the same? Should they be? (Had n not been an integer, the section would have to be built by interpolating the record at the times $t_k = kT$, with cubic interpolation adequate at this sampling density.)

2. A section needs two coordinates and the record supplies one. Use the delayed pair (s_k, s'_k) where s'_k is the strain a quarter of a forcing period later, which plays the role of the velocity

as in Problem 1.6. Plot s'_k against s_k. Compare with the model section of Problem 7.5: are the two wells and the folded structure visible?

3. Also plot the return map s_{k+1} against s_k. Is it a function, as the Lorenz maxima of Problem 7.3 nearly are, or a scatter? Explain the difference in terms of the dimension of the sections.

4. Suppose the drive period assumed in part (1) is in error by a fraction δ. Show that the sampling phase then drifts by $2\pi\delta$ per period, and estimate how many periods it takes for the drift to reach $\pi/2$ for $\delta = 10^{-3}$ and 10^{-4}. Now reproduce the scan of Fig. 7.11(e): strobe the record with integer strides $n = 12$ to 20, truncate every section to the same number of points, and plot the mean nearest-neighbour distance against n. Why must the point count be held fixed? Extend the scan to non-integer strides between 15.5 and 16.5 by interpolation, and report how sharply the minimum locates the drive period.

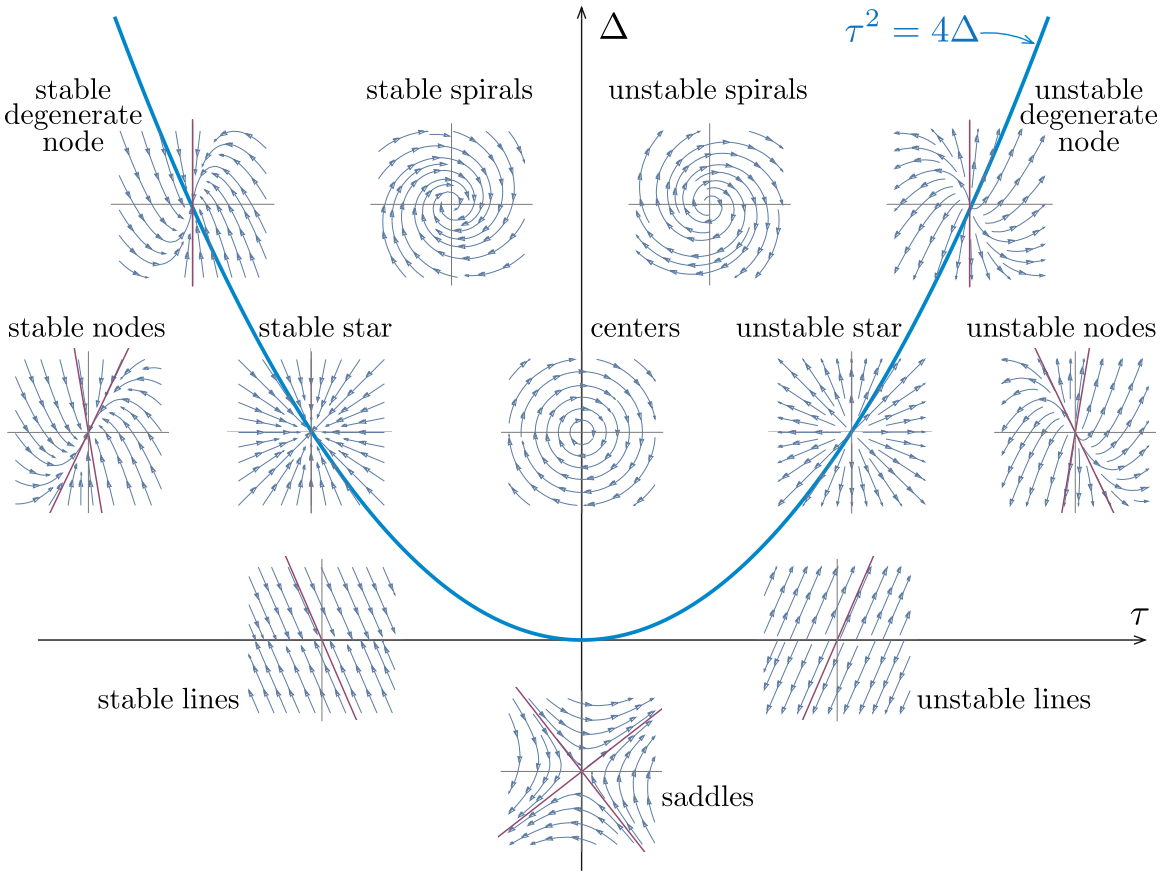


Figure 7.1: Types of two-dimensional static equilibria of the linear flow $\dot{\boldsymbol{x}} = \boldsymbol{A}\boldsymbol{x}$, arranged in the (τ, Δ) plane with the parabola $\tau^2 = 4\Delta$. Every phase portrait is computed: trajectories $\boldsymbol{x}(t) = e^{\boldsymbol{A}t}\boldsymbol{x}_0$ from initial conditions on a circle (or, for the lines, on the edges of the box), integrated forward and backward in time and clipped to a square about the equilibrium; arrows point in the direction of increasing time and the straight lines in red are the real eigendirections. The matrices have eigenvalues $-0.25 \pm i$ and $0.25 \pm i$ (spirals), $\pm i$ (centers), $\{-1, -3\}$ and $\{1, 3\}$ (nodes), ∓ 1 double with $\boldsymbol{A} = \mp \boldsymbol{I}$ (stars) and with a single eigenvector (degenerate nodes), $\{-1, 1\}$ (saddles), and $\{0, \mp 1\}$ (lines of equilibria). Script `python/mkfig_stability_diagram.py`.

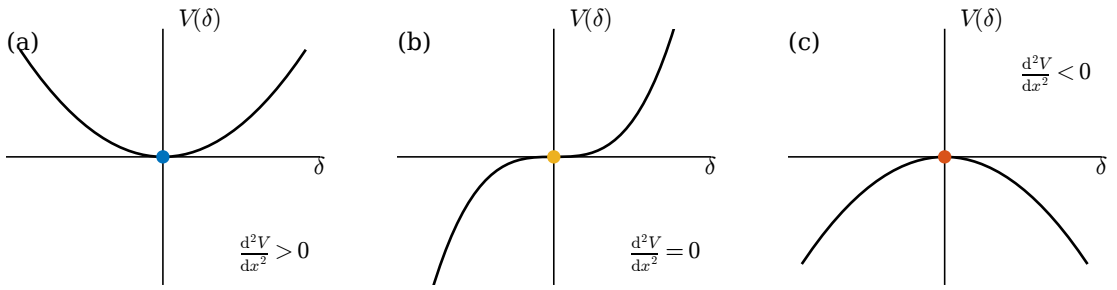


Figure 7.2: Potential energy extrema at equilibrium points: the quadratic approximation $V(\delta) = \frac{1}{2}V''(x_e)\,\delta^2$ with $V'' > 0$ (a, minimum, stable) and $V'' < 0$ (c, maximum, unstable), and the cubic $V = \delta^3$ that decides the case $V'' = 0$ (b, inflection, semi-stable). Script `python/mkfig_potentialst.py`.

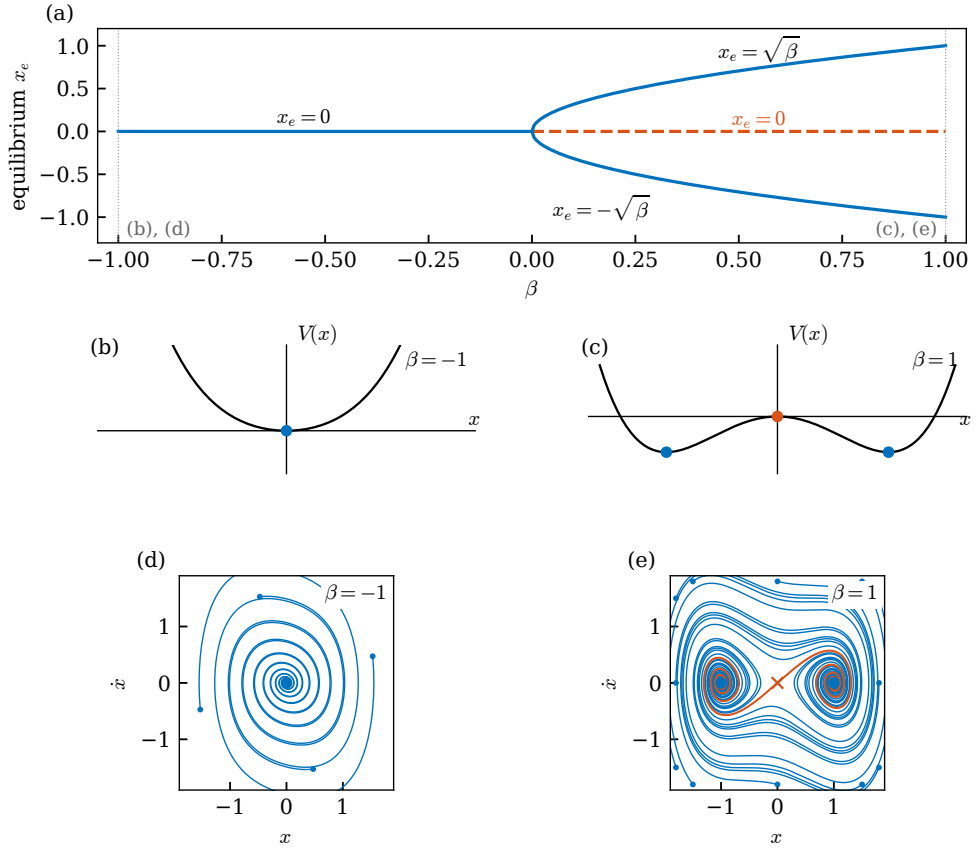


Figure 7.3: Pitchfork bifurcation of the unforced Duffing oscillator $\ddot{x} + \gamma\dot{x} - \beta x + x^3 = 0$. (a) Equilibria x_e against β: blue curves are stable equilibria, the red dashed line the unstable one. (b), (c) The potential $V(x) = -\beta x^2/2 + x^4/4$ at $\beta = -1$ and $\beta = 1$, equilibria marked (blue stable, red unstable). (d), (e) Computed phase portraits at the same two β with $\gamma = 0.25$: trajectories from four initial conditions on the circle of radius 1.6 (d) and from twelve initial conditions on the edges of the box $|x|, |\dot{x}| \le 1.8$ (e), $t \in [0, 30]$; the origin is a stable spiral in (d) (since $\beta < -\gamma^2/4$) and a saddle in (e), whose unstable manifold (red) leads to the two stable spirals at $x = \pm 1$. Script `python/mkfig_pitchfork.py`.

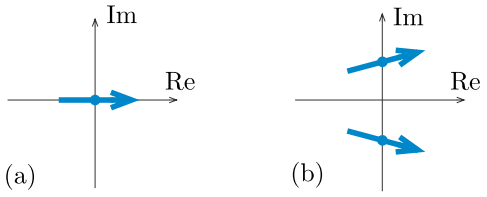


Figure 7.4: CE behavior during the loss of stability due to a steady bifurcation with a real eigenvalue crossing zero (a) and a Hopf bifurcation (b)

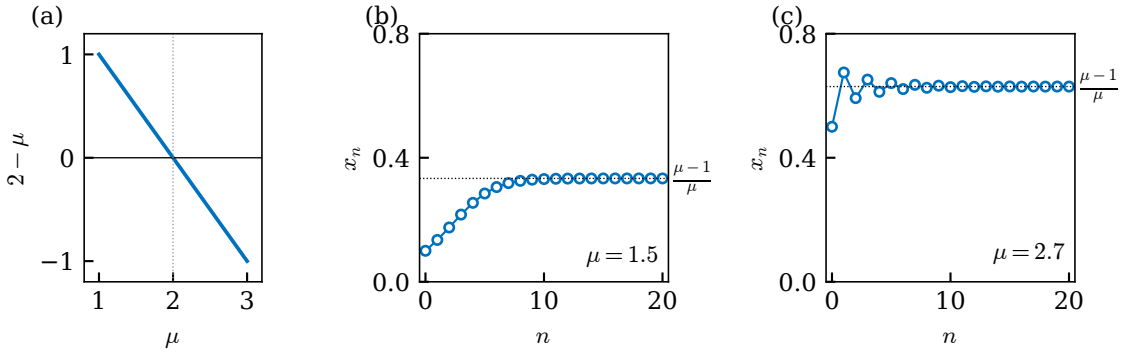


Figure 7.5: In the stability region $1 < \mu < 3$ of the equilibrium $x = \frac{\mu-1}{\mu}$ of the logistic map, the multiplier $2 - \mu$ switches sign at $\mu = 2$ (a); for $1 < \mu < 2$ the approach to the equilibrium is monotonic, shown for $\mu = 1.5$ from $x_0 = 0.1$ (b), and for $2 < \mu < 3$ it is oscillatory, shown for $\mu = 2.7$ from $x_0 = 0.5$ (c). The dotted line is the fixed point. Script `python/mkfig_mum2.py`.

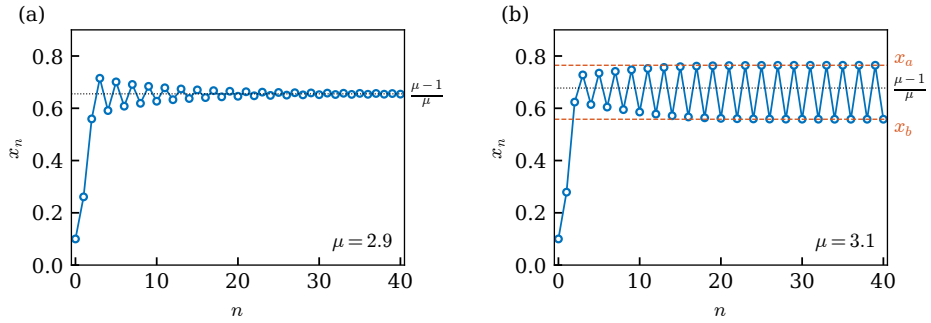


Figure 7.6: Iterates of the logistic map from $x_0 = 0.1$: (a) for $\mu = 2.9$ the P-1 point $\frac{\mu-1}{\mu} = 0.655$ (dotted) is still stable, with multiplier $2 - \mu = -0.9$, and the approach is oscillatory and slow; (b) for $\mu = 3.1$ it is unstable (multiplier -1.1) and the iterates settle on the P-2 orbit $x_{a,b} = 0.765, 0.558$ (red dashed). Script `python/mkfig_p1p2b.py`.

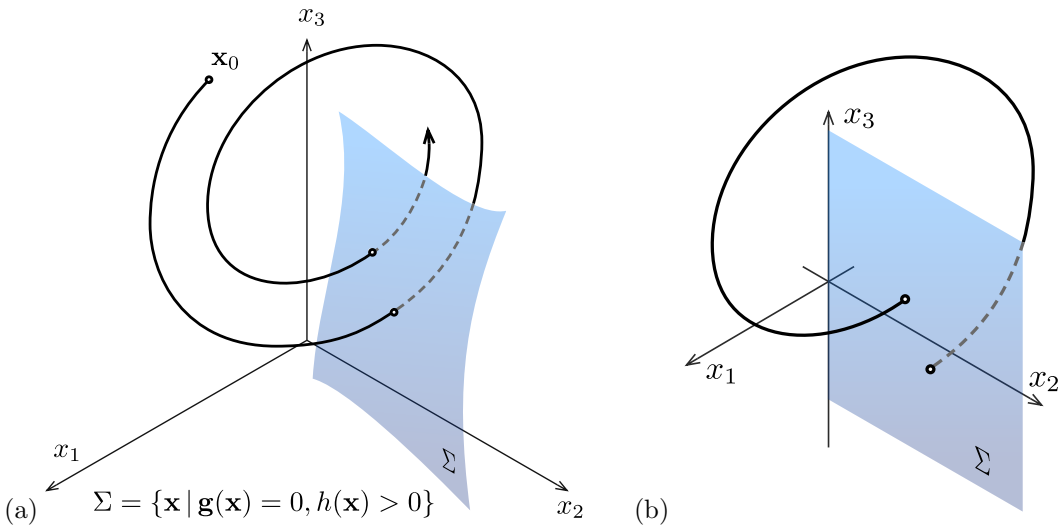


Figure 7.7: Poincaré section or surface of section (a), and its simple example with $g(\boldsymbol{x}) = x_1 = 0$ and $h(\boldsymbol{x}) = x_2 > 0$ (b).

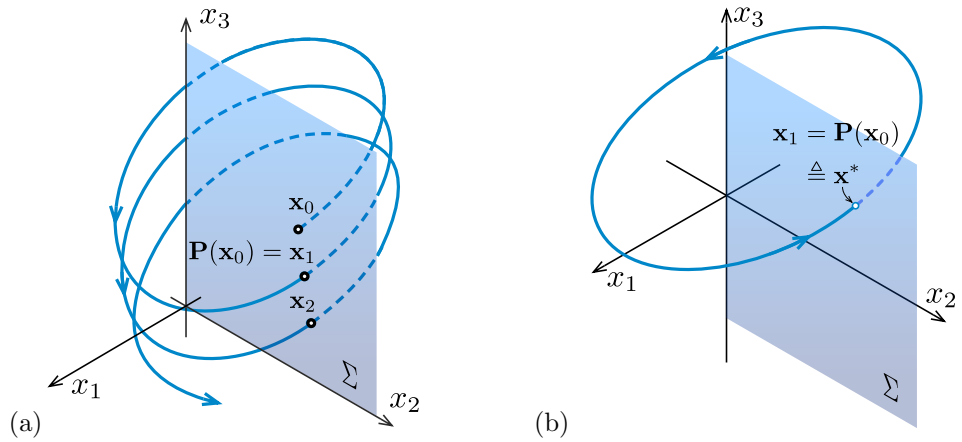


Figure 7.8: Poincaré map $\boldsymbol{P} : \Sigma \to \Sigma$ (a), and its P-1 fixed point $\boldsymbol{x}^* = \boldsymbol{P}(\boldsymbol{x}^*)$ or P-1 orbit (b).

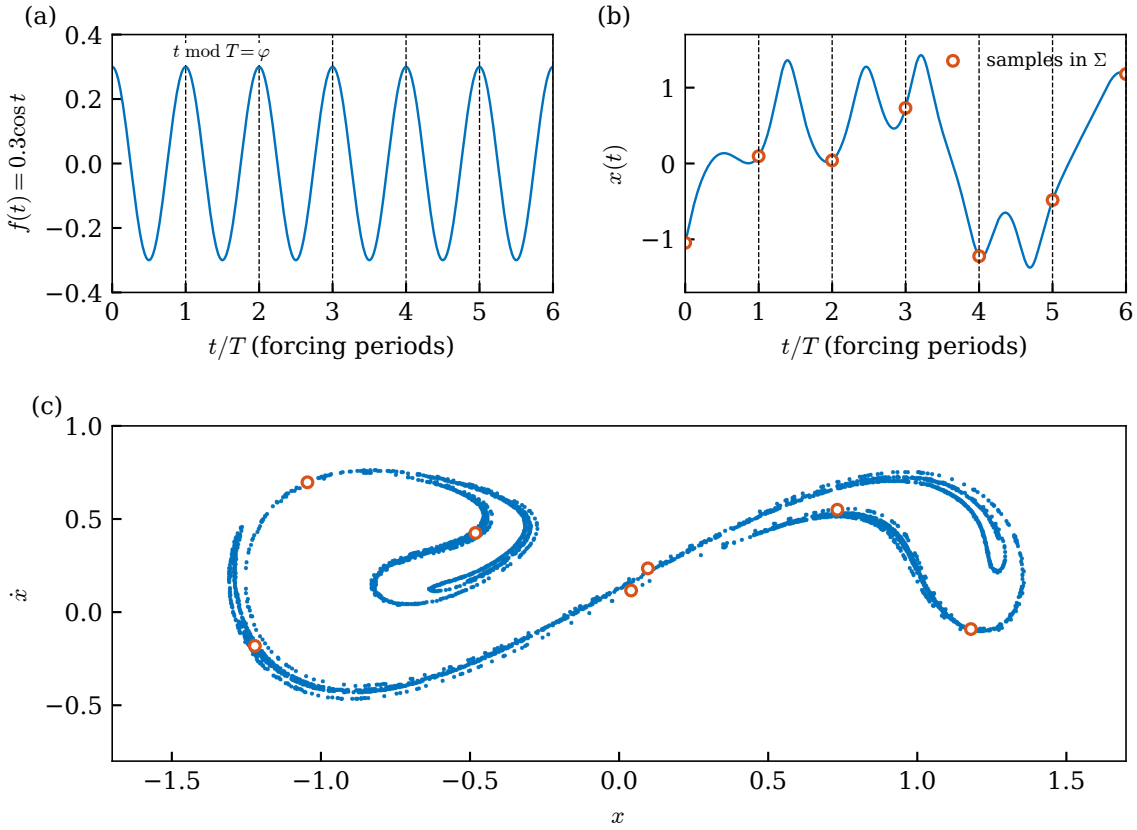


Figure 7.9: Sampling for a stroboscopic Poincaré section of the two-well Duffing oscillator $\ddot{x}+0.25\dot{x}-x+x^3 = 0.3\cos t$ of Section 1.4, $T = 2\pi$: (a) the forcing $f(t) = 0.3\cos t$ and (b) the response $x(t)$ over six forcing periods (after a transient of 200 time units), with the sampling instants $t \bmod T = \varphi$, $\varphi = 0$, marked by dashed lines; the circles are the samples that lie in Σ. (c) The section itself: $(x, \dot{x})$ at $t \bmod T = 0$ for 5000 consecutive periods (fixed-step RK4, 200 steps per period, 50 transient periods discarded); the circles are the seven samples of (b). This is the fractal set of Problem 7.5 and Fig. 11.14. Script `python/mkfig_stroboscopic.py`.

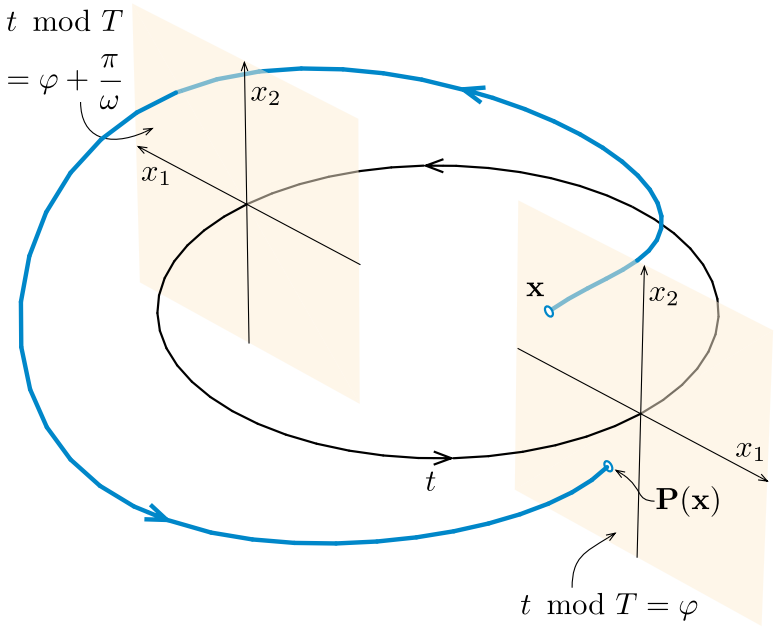


Figure 7.10: Stroboscopic sampling in the phase space $\mathbb{R}^2 \times \mathbb{S}^1$, where the black circle represents time as a forcing phase and blue curve is the trajectory.

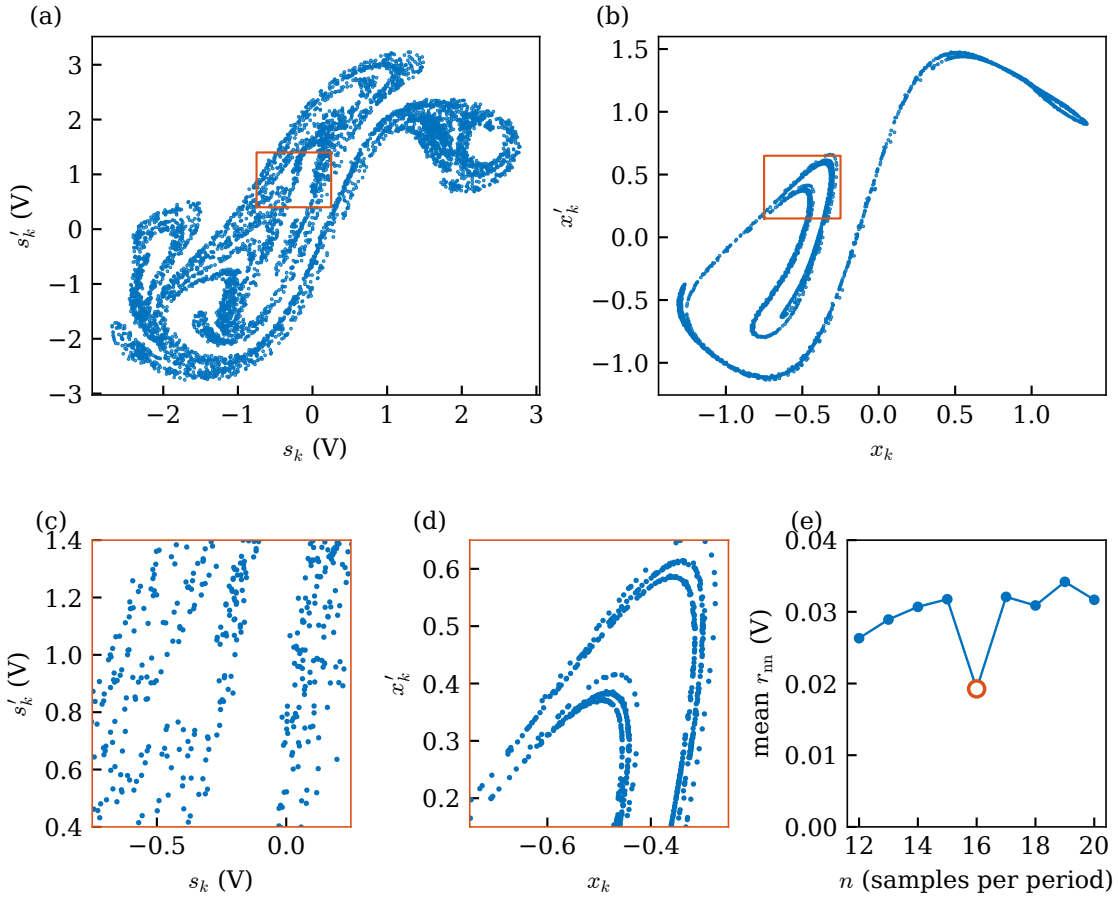


Figure 7.11: Stroboscopic Poincaré section of the two-well oscillator record of Section 1.4.1, against the section of the simulated two-well Duffing oscillator $\ddot{x} + 0.25\dot{x} - x + x^3 = 0.3\cos t$ of Section 1.4. (a) the record strobed once per forcing period, every sixteenth sample, in the delay pair (s_k, s'_k) with s'_k the sample four samples (a quarter period) later; 6000 section points, the whole record, sampling phase $\varphi = 0$. (b) the model in the same delay coordinates, strobed at $t \bmod T = 0$ and $t \bmod T = T/4$ of the same periods (fixed-step RK4, 200 steps per period, 50 transient periods, 5000 section points). (c), (d) the boxes marked in (a) and (b), each about one fifth of the plotted range on a side. (e) mean distance r_{nn} to the nearest neighbour in the section against the assumed number n of samples per forcing period, every stride truncated to the same 5300 points; the marked minimum is $n = 16$. Script `python/mkfig_twowell_section.py`.

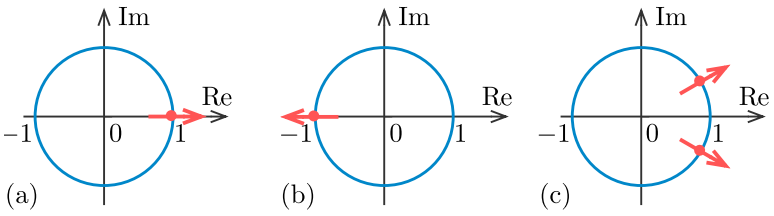


Figure 7.12: Instabilities arise as CMs leave the unit circle during saddle-node (a), period-doubling (b), and Neimark (c) bifurcations.

8 Delay Coordinate Embedding

Up to this point, we have known our phase space explicitly. However, what if we do not know it? How can we then study the dynamics in *phase space*? A typical case is when our data is a *scalar time series* (e.g., the temperature measured at uniform time intervals in a specific geographic point). Let us consider a continuous time systems with *true* state $\boldsymbol{u} \in \mathcal{S}$, where $\mathcal{S}$ is our phase space.[1] Then our measured signal is

$$x_n \triangleq x\left(\boldsymbol{u}(t_n)\right) + \eta_n \,, \tag{8.1}$$

where η_n is measurement noise. If we have uniform time sampling Δt (which is a generic case), $t_n = n\,\Delta t$. Thus, we usually have a *time series*

$$\{x_n\}_{n=1}^{N} = \{x_1, x_2, x_3, \ldots, x_N\} \,. \tag{8.2}$$

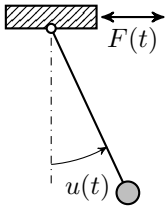


Figure 8.1: Driven pendulum

Example 1: Driven Pendulum

We consider a driven pendulum in Fig. 8.1, described by the following equation:

$$\ddot{u} + \gamma\dot{u} + \sin u = F\cos\omega t \,, \tag{8.3}$$

or

$$\begin{aligned} \dot{u} &= v \,, \\ \dot{v} &= -\gamma v - \sin u + F\cos\theta \,, \\ \dot{\theta} &= \omega \,, \end{aligned} \tag{8.4}$$

or

$$\dot{\boldsymbol{u}} = \boldsymbol{f}(\boldsymbol{u}) \,, \tag{8.5}$$

where $\boldsymbol{u} = [u, v, \theta]^{\mathrm{T}} \in \mathbb{S}^1 \times \mathbb{R}^1 \times \mathbb{S}^1$. Now let us assume that we measure only the first coordinate u as

$$x = h\left(\Pi_1(\boldsymbol{u})\right) = cu \,, \tag{8.6}$$

[1] In general, $\mathcal{S}$ is not even known precisely.

where $h : \mathbb{S}^1 \to \mathbb{R}$ is the angular position sensor function, $\Pi_1 : \mathbb{S}^1 \times \mathbb{R} \times \mathbb{S}^1 \to \mathbb{S}^1$ is the projection onto the first coordinate $u \in \mathbb{S}^1$, and c is the linear sensitivity ($c = h'(u^*)$ at the sensor's operating point u^*).

In this example, we know $\mathcal{S}$, and the question is if we can reconstruct the dynamics in the phase space and estimate all the relevant system parameters by just having time series of x.

Example 2: Kicked Elastica

As an another example we consider an experimental system [43] shown in Fig. 8.2. Here, we have

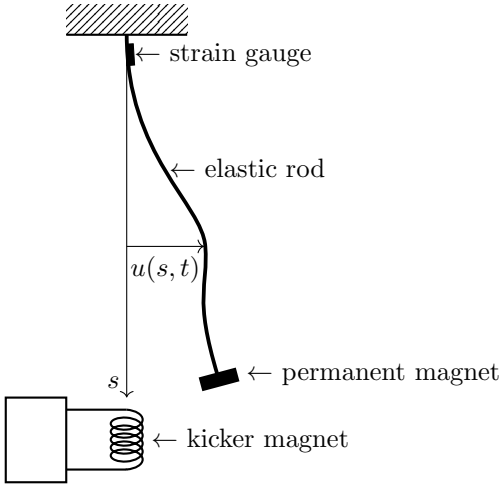


Figure 8.2: Kicked elastica

a cantilevered elastic rod with a permanent magnet mounted at its free end. The kicker magnet located near the free end of the rod applies transverse load to the rod tip as it passes over it. The rod displacement can be expressed in terms of its normal modes as

$$u(s,t) = \sum_{n=1}^{\infty} q_n(t)\varphi_n(s)\,, \tag{8.7}$$

where $\varphi_n : [0, L] \to \mathbb{R}$ are normal modes of the system satisfying the boundary conditions, and the $q_n(t)$ are the corresponding time coordinates. The corresponding velocity is $v = \dot{u}$. Therefore, $\boldsymbol{u} = [u, v]^{\mathrm{T}} \in \mathcal{S}$ is the ∞-dimensional function space. We measure strain $x(t)$ at $s = s^*$. The strain $x \propto \kappa(s^*) \approx cu''(s^*)$, where κ is the curvature of the rod and the approximation is valid for small u and u'. The question here would be: can we use the measured strain time series $\{x_n\}_{n=1}^N$ to reconstruct the observed dynamics of the elastic rod? Namely, can we determine the number of active modes and their contribution to the response at the various supply voltage values to the kicker magnet? Can we develop a bifurcation diagram while varying supply voltage to the kicker magnet? etc.

Example 3: Walking Human

We have seen many examples of the studies on humans kinematics [175, 193] shown in Fig. 8.3, where their gait dynamics are recorded using goniometers (for joint angles) or other instruments (such as video-based motion capture). Here, the state of human locomotion $\boldsymbol{u} \in \mathcal{S}$ is unknown.

We do not even know what is our phase space $\mathcal{S}$, which might be a collection of joint angles, their velocities, plus many unknown variables. We would usually measure some joint angle $x = c\,\theta$, where θ is the actual joint angle or track motion using a video-based system.

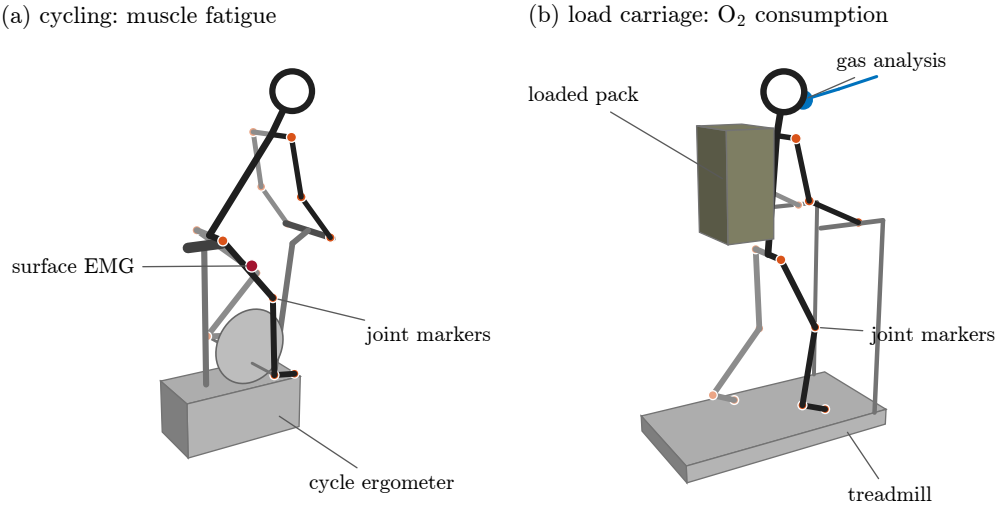


Figure 8.3: Two studies that track a physiological process through measured kinematics, drawn schematically in perspective. (a) Cycling on an ergometer with reflective joint markers and a surface EMG electrode, relating muscle fatigue to movement kinematics [193]. (b) Load-carriage walking on a treadmill with joint markers and respiratory gas analysis, relating oxygen consumption to gait kinematics [175]. Script `python/mkfig_humans.py`.

A pertinent question, when we do not know $\mathcal{S}$: is it reasonable to believe that one even exists? Additionally, can we use the measured kinematic variable to track and predict other physiological processes like muscle fatigue or oxygen consumption?

8.1 Delay Reconstruction

Given the scalar measurement $\{x_n\}_{n=1}^{N}$, we can attempt to form a multi-dimensional observable using *delay reconstruction*. That is, given $\{x_1, x_2, \ldots\}$, we define:

$$\boldsymbol{x}_n = [x_n, x_{n+\tau}, \ldots, x_{n+(d-2)\tau}, x_{n+(d-1)\tau}]^{\mathrm{T}} \in \mathbb{R}^{d\times 1}, \qquad n = 1, \ldots, N-(d-1)\tau\,, \tag{8.8}$$

where the integer τ is the *lag* and $\tau\,\Delta t$ the corresponding *delay time*, and d is the *embedding dimension*. The last $(d-1)\tau$ samples cannot start a delay vector, so a record of N points yields $N-(d-1)\tau$ reconstructed points — a loss worth watching when τ or d is large.[2] A simple illustration

[2]The delay vector is built here from *later* samples, $x_{n+j\tau}$. This forward convention is used throughout the book and in every routine of Appendix A. The backward convention $x_{n-j\tau}$ embeds the attractor equally well. The two differ only by bookkeeping: the backward vector at index n is the forward vector at index $n-(d-1)\tau$ with its coordinates listed in reverse order, and both sequences of vectors advance forward in n. A coordinate permutation preserves every Euclidean distance, so neighbor searches, divergence curves and Jacobian estimates agree up to the index offset $(d-1)\tau$, and no invariant (dimension, Lyapunov exponents) depends on the choice. What does depend on it is which sample plays the role of the $(d+1)$-th coordinate in a false-nearest-neighbor test, so the convention must be stated once and kept.

of three-dimensional reconstruction is shown in Fig. 8.4.

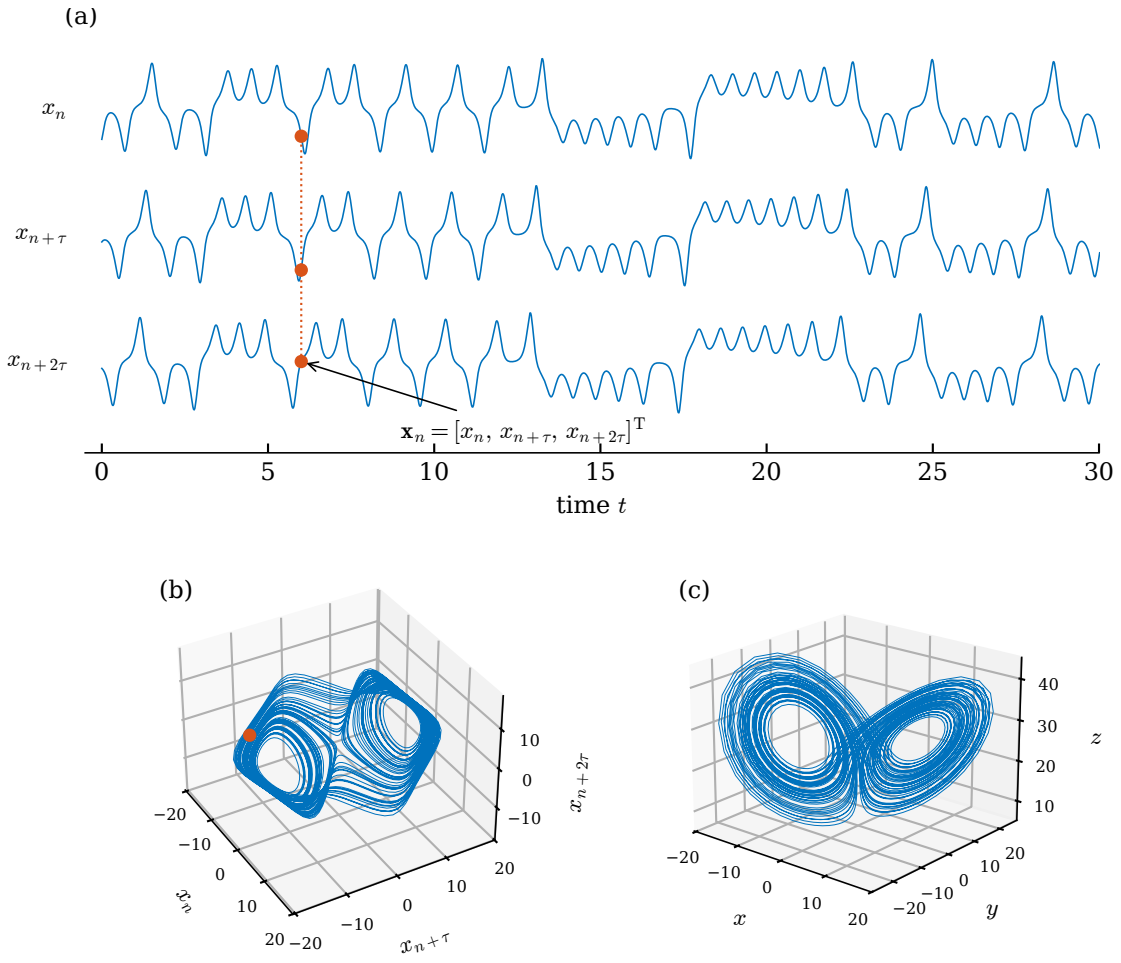


Figure 8.4: Illustration of delay reconstruction of a scalar time series into a three-dimensional phase space trajectory, for the x record of the Lorenz system of Section 1.4 ($\Delta t = 0.02$, $N = 4000$ samples after a transient of 20 time units). (a) The record x_n and its two forward shifts $x_{n+\tau}$ and $x_{n+2\tau}$, $\tau = 9$ samples, drawn one under the other over the first 30 time units; the dots are the three coordinates of one delay vector $\boldsymbol{x}_n$ ($n = 300$). (b) The reconstruction $(x_n, x_{n+\tau}, x_{n+2\tau})$, with $\boldsymbol{x}_{300}$ marked. (c) The true state (x, y, z) over the same interval, for comparison. Script `python/mkfig_delayrec.py`.

Why do we think that the delay reconstruction might work? Consider

$$\begin{aligned}\boldsymbol{x}_n &= [x_n, x_{n+\tau}, \ldots, x_{n+(d-2)\tau}, x_{n+(d-1)\tau}]^{\mathrm{T}} \in \mathbb{R}^{d\times 1},\\ &= [x(\boldsymbol{u}(t_n)), x(\boldsymbol{u}(t_n+\tau\Delta t)), \ldots, x(\boldsymbol{u}(t_n+(d-2)\tau\Delta t)), x(\boldsymbol{u}(t_n+(d-1)\tau\Delta t))]^{\mathrm{T}} \in \mathbb{R}^{d\times 1}.\end{aligned} \tag{8.9}$$

If our time series are from a deterministic system then $\boldsymbol{u}(t_n + \tau\Delta t) = \boldsymbol{F}(\boldsymbol{u}(t_n))$, where $\boldsymbol{F} = \varphi_{\tau\Delta t}$ is

the time-$\tau\Delta t$ map of the flow.[3] Thus,

$$\begin{aligned} \boldsymbol{u}(t_n + 2\tau\Delta t) &= \boldsymbol{F}(\boldsymbol{u}(t_n + \tau\Delta t)) = \boldsymbol{F}(\boldsymbol{F}(\boldsymbol{u}(t_n))) = \boldsymbol{F}^2(\boldsymbol{u}(t_n))\,, \\ &\vdots \\ \boldsymbol{u}(t_n + (d-1)\tau\Delta t) &= \boldsymbol{F}^{d-1}(\boldsymbol{u}(t_n))\,, \end{aligned} \tag{8.10}$$

and

$$\boldsymbol{x}_n = \Phi(\boldsymbol{u}(t_n)) \equiv \Phi(\boldsymbol{u}_n)\,, \qquad \Phi(\boldsymbol{u}) = \left[x(\boldsymbol{u}),\, x(\boldsymbol{F}(\boldsymbol{u})),\, \ldots,\, x(\boldsymbol{F}^{d-1}(\boldsymbol{u}))\right]^{\mathrm{T}}. \tag{8.11}$$

The *delay map* $\Phi : \mathcal{S} \to \mathbb{R}^d$ is a function of the state alone: every coordinate is the measurement evaluated along the orbit through $\boldsymbol{u}$. It cannot be factored through the single number $x(\boldsymbol{u})$, since one scalar does not determine a d-vector; what it is built from is the measurement x and the flow φ together, which is why the embedding theorem below is a statement about the triple (x, φ, τ). To have any hope of this working, we need to get the dimension d and delay τ right.

In summary, as shown in Fig. 8.5, our deterministic trajectory $\varphi_t(\boldsymbol{u}_0) : \mathcal{S} \to \mathcal{S}$ evolves on a set $\mathcal{A} \subset \mathcal{S}$, with dim $\mathcal{A} = k$. We are looking for the delay reconstruction $\Phi : \mathcal{S} \to \mathbb{R}^d$ of Eq. (8.11), assembled from the measurement $x : \mathcal{S} \to \mathbb{R}$ evaluated at d successive times along the orbit. The delay embedding carries the initial state $\boldsymbol{u}_0$ to the reconstructed point $\boldsymbol{x}_0 = \Phi(\boldsymbol{u}_0)$, whose orbit in $\mathbb{R}^d$ is $\hat{\varphi}_t(\boldsymbol{x}_0)$.

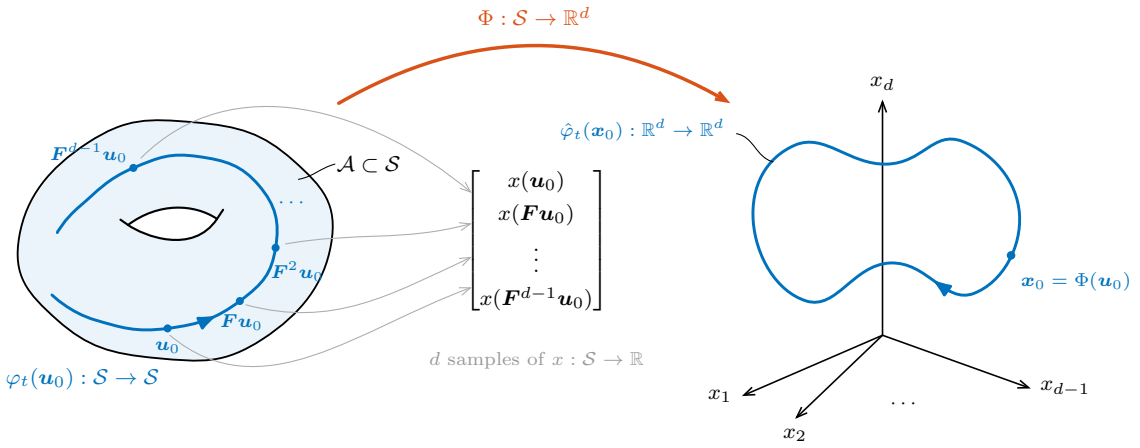


Figure 8.5: Delay coordinate reconstruction. The orbit $\varphi_t(\boldsymbol{u}_0)$ evolves on the invariant set $\mathcal{A} \subset \mathcal{S}$ (left). The delay map Φ acts on the state directly (orange), but its d coordinates are assembled from the scalar measurement x evaluated at the d successive states $\boldsymbol{u}_0, \boldsymbol{F}\boldsymbol{u}_0, \ldots, \boldsymbol{F}^{d-1}\boldsymbol{u}_0$ along that orbit (center). The image orbit $\hat{\varphi}_t(\boldsymbol{x}_0)$ lives in $\mathbb{R}^d$ (right). A single value $x(\boldsymbol{u}_0)$ does not determine $\boldsymbol{x}_0$; the flow supplies the rest.

The big question is: when will this process work properly? We would like all the *topological properties* to be preserved (i.e., dimension and type of orbit), as well as key *dynamical* properties like *stability*, and more generally, we would like $\hat{\varphi}_t$ to be "like" φ_t.

The big answer developed by Whitney, Takens, Sauer *et al.* is:

[3] Think of $\boldsymbol{u}(t_n)$ as an initial condition and integrate forward by $\tau\,\Delta t$.

Fractal Delay Embedding Prevalence Theorem

Let $\mathcal{A} \subset \mathcal{S}$ be a *compact* invariant set of the flow φ_t with capacity (i.e., box counting) dimension D_F, and let $d > 2D_F$ be an integer. Fix a delay time $\tau\,\Delta t$ such that:

1. $\mathcal{A}$ contains *no* periodic orbits of period $\tau\,\Delta t$ or $2\tau\,\Delta t$;
2. $\mathcal{A}$ contains only a *finite* number of periodic orbits of period $p\,\tau\,\Delta t$ for $3 \le p \le d$, and the linearization of the flow about each of them has distinct eigenvalues;
3. $\mathcal{A}$ contains only a finite number of equilibria.

Then for *almost every*[a] smooth[b] measurement $x : \mathcal{S} \to \mathbb{R}$ the delay map $\Phi(x, \varphi, \tau) : \mathcal{S} \to \mathbb{R}^d$ is one-to-one on $\mathcal{A}$, and is an immersion on each compact subset of a smooth manifold contained in $\mathcal{A}$.

[a] In the sense of *prevalence*, the measure-theoretic notion of "almost every" in an infinite-dimensional function space introduced for this purpose by Sauer, Yorke, and Casdagli.
[b] That is, $x \in C^1$.

This is the *fractal delay embedding prevalence theorem* of Sauer, Yorke, and Casdagli (1991) [173]. Note that the delay is *fixed* in the statement: the genericity is in the measurement function, not in τ. The older formulation in which the pair (flow, measurement) is generic and the result holds for a smooth compact manifold $\mathcal{A}$ of integer dimension k with $d \ge 2k + 1$ is Takens' theorem [197]; the loose statement that the embedding works "for almost every τ" descends from that setting, where the conditions on periodic orbits are what a generic τ avoids. Because a fractal $\mathcal{A}$ is not a manifold, the conclusion splits into two parts: injectivity on all of $\mathcal{A}$, and the full-rank (immersion) property only on the smooth pieces $\mathcal{A}$ may contain. To understand this theorem, we need to clarify some terms.

Definition

An *embedding* of the manifold $\mathcal{A}$ is a map $\mathbf{\Psi}$ that is a C^1 *diffeomorphism* onto its image $\mathbf{\Psi}(\mathcal{A})$ — one-to-one, and so invertible there — and for which the Jacobian $D\mathbf{\Psi}$ has full rank everywhere in $\mathcal{A}$. Note that $\mathbf{\Psi}$ is *not* onto $\mathbb{R}^d$: the image is a $\dim \mathcal{A}$-dimensional subset of it, and the whole point of taking d larger than $\dim \mathcal{A}$ is to leave room for that image to be free of self-intersections.

Schematic of an embedding is shown in Fig. 8.6. Please note that the manifold $\mathcal{A}$ can be distorted by the embedding $\mathbf{\Psi}$.

We still need to understand what the *capacity dimension* D_F is, which we will address in the *Dimension Theory* chapter. However, right now we can consider a case when $D_F = D$ a "normal" integer dimension. Then the requirement that $d > 2D$ (or $d \ge 2D + 1$) can be understood with the cartoons shown in Fig. 8.7. In Fig. 8.7(a), when $d < D$, we do not have large enough room for the embedding. In Fig. 8.7(b), when $d = D$, we see that the mapping cannot be one-to-one globally (though locally we are ok). In Fig. 8.7(c), when $d = 2D$, we might have an embedding $\mathbf{\Psi}$ if we are lucky (top case), but generically we cannot guarantee one-to-one mapping due to expected intersections caused by projection (bottom case). In addition, there is an infinite number of "nearby" $\mathbf{\Psi}$ that fail too. Finally, in Fig. 8.7(d), when $d = 2D+1$, we have an embedding that is geometrically ok and preserves the original trajectories' dynamical characteristics. Cartoons in Fig. 8.7 are actually describing the *Whitney Embedding Theorem* [55], which the previously discussed *Embedding Theorem*

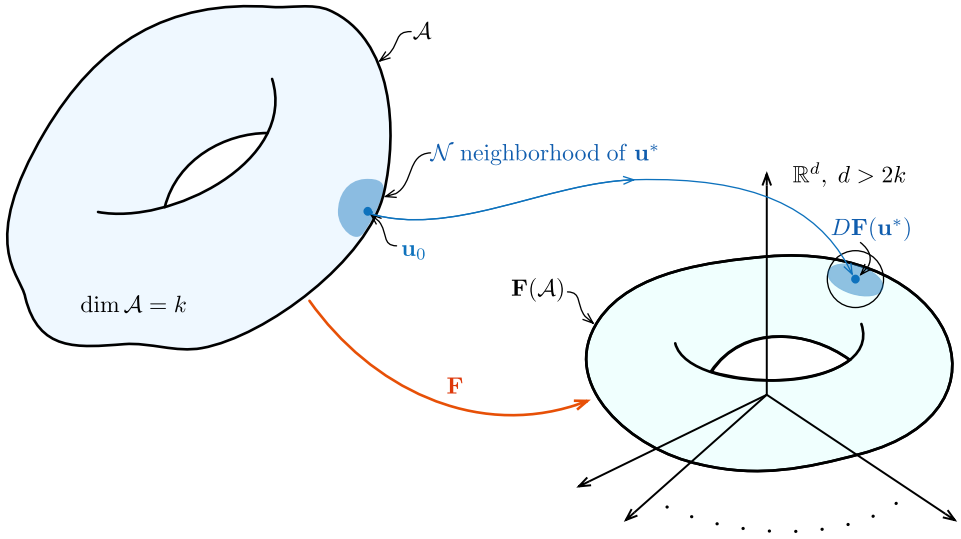


Figure 8.6: Schematic of an embedding

generalizes to fractal dimensions.

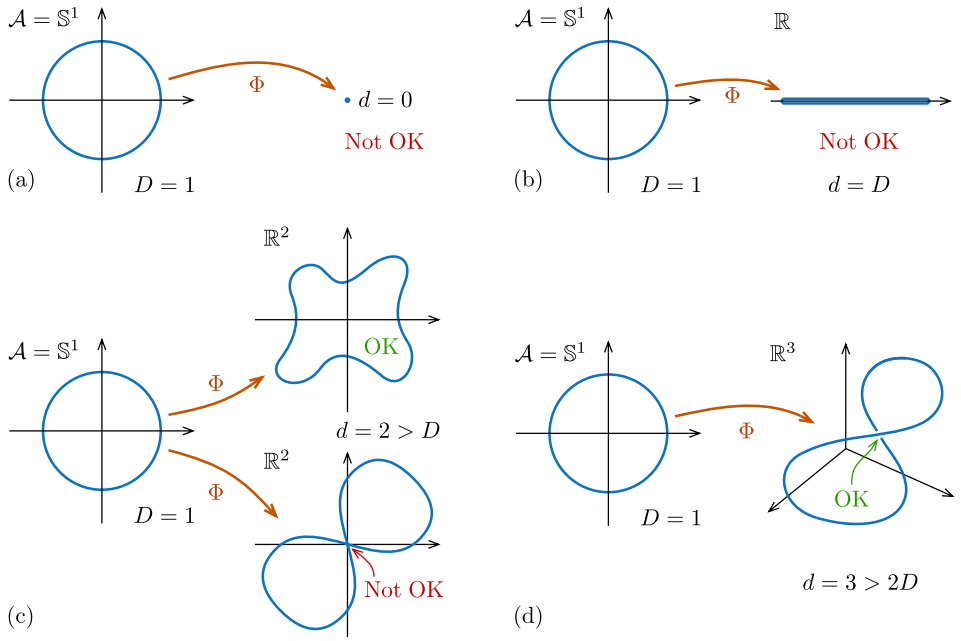


Figure 8.7: Cartoon illustration of the *Whitney Embedding Theorem*

8.1.1 What the Theorem Leaves to the Analyst

The theorem settles existence and leaves every practical choice open. Three gaps follow from it, and Chapters 9 and 10 exist to close them.

The first is D_F itself. The condition $d > 2D_F$ is stated in terms of a quantity that is usually unknown, and is often the very thing the analysis is meant to estimate. The dimension of Chapter 11

and the embedding dimension of Chapter 10 are therefore estimated from the record, each without the other.

The second is the delay. The theorem holds for almost every τ, so on paper the choice does not matter beyond avoiding a period of the motion. On a finite noisy record it matters a great deal. Too small a τ collapses $\mathcal{A}$ onto the hyper-diagonal of the embedding space, where the coordinates are nearly equal and the attractor is lost in the noise. Too large a τ leaves successive coordinates of a chaotic record effectively uncorrelated, and the deterministic structure is folded past recognition. The theorem is a statement about noise-free data of unlimited length. Casdagli, Eubank, Farmer and Gibson [23] analyzed the reconstruction with measurement noise and finite N, and showed that its quality is governed by a trade-off between *redundancy* — coordinates too close in time carry nearly the same information, so noise dominates the transverse directions — and *irrelevance* — coordinates too far apart in time are, for a chaotic record, nearly independent, so the map from the reconstruction to the future is complicated. Both limits amplify noise, and the "almost every τ" of the theorem becomes a window of acceptable delays whose width shrinks as the noise grows. That analysis is the theoretical basis for the delay-selection rules of Chapter 9, which takes this up.

The third is that $d > 2D_F$ is sufficient and not necessary. It buys a generic embedding for every set of that dimension; a particular attractor often embeds in fewer coordinates, and the false-nearest-neighbor counts of Chapter 10 measure how few. The gap between the two answers is not a contradiction, and Section 1.4.1's record shows it. Its correlation dimension $D_2 = 2.68$ is a lower bound on D_F, so it puts the bound at no less than $d = 6$ without certifying any d, while the neighbor count is already zero at $d = 5$. The neighbor count supports a working reconstruction at the resolution of the record; it does not prove an embedding, and results are reported across several d.

In summary, if the conditions of the Embedding Theorem are satisfied, we have the commutative diagram shown in Fig. 8.8: Φ is the delay map Eq. (8.11), and $\hat{\varphi}_t = \Phi \circ \varphi_t \circ \Phi^{-1}$ is the reconstructed flow on $\Phi(\mathcal{A}) \subset \mathbb{R}^d$. Following either path around the square from the top-left corner to the bottom-right gives the same point. Thus, the true dynamics can be expressed as:

$$\begin{aligned} \boldsymbol{u}(t) &= \varphi_t(\boldsymbol{u}_0) \\ &= \Phi^{-1} \circ \hat{\varphi}_t \circ \Phi(\boldsymbol{u}_0)\,, \end{aligned} \tag{8.12}$$

that is

$$\varphi_t = \Phi^{-1} \circ \hat{\varphi}_t \circ \Phi\,. \tag{8.13}$$

Therefore, in this sense, the dynamics in $\mathbb{R}^d$ and $\mathcal{A}$ are *equivalent*. Further, if we study, estimate, model $\hat{\varphi}_t$ on $\mathbb{R}^d$, we know *important properties* of φ_t on $\mathcal{A}$.

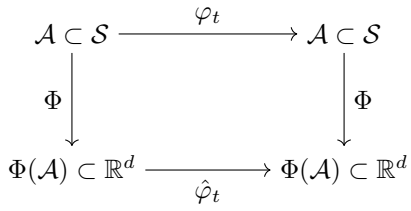


Figure 8.8: Commutative diagram for the delay embedding: $\Phi \circ \varphi_t = \hat{\varphi}_t \circ \Phi$ on $\mathcal{A}$.

8.2 Phase Space Reconstruction

Two primary parameters need to be determined for delay coordinate embedding: delay time τ and the embedding dimension d. The delay time should be large enough that each coordinate of the reconstructed phase point is distinct and not redundant. Taking a *too small delay* results in delay vectors close to the hyper-diagonal of the reconstructed phase space, and any variations transverse to it are not well defined. At the same time, for a chaotic response, the delay should be *small enough* that coordinates are not statistically independent of each other and the attractor geometry is not complicated or obscured. For example, the ideal delay for a harmonic signal would be a quarter of its period. This example suggests that the first zero crossing of the autocorrelation is a good choice for the delay (e.g., for a harmonic signal, this happens at a quarter of the period as shown in Fig. 8.9). However, the autocorrelation measures only linear dependence, and for a chaotic time series its zero crossing need not agree with delays chosen from nonlinear criteria such as the mutual-information minimum introduced next. The disagreement has no general direction or size: for the Rössler x record ($\Delta t = 0.05$) the two give 29 and 27 samples; for the Duffing x record ($\Delta t = 0.1$) 75 and 46; and for the Lorenz x record ($\Delta t = 0.02$) 182 and 9, because the sign of x persists over many lobe circuits and the autocorrelation decays slowly while the local geometry unfolds within a fraction of a circuit (`python/chk_delay_zc.py`). The zero crossing is a heuristic borrowed from the harmonic case, not a correlation time in the sense of Chapter 3.

Average mutual information (AMI), being a measure of nonlinear correlations in the data provides another alternative. Its first minimum, when identifiable, is usually a reasonable estimate of optimal delay as it identifies a delay at which a new coordinate provides maximal new information compared to its delayed version while still being nonlinearly correlated to it. However, AMI estimates are susceptible to noise, which can make it impossible to identify the first minimum in AMI data even if it exists; Chapter 9 discusses the estimator and its bias. The simplest of all methods is to look at the reconstructed phase portrait and, when possible, identify the delay that provides a good cover of space without causing complex geometry.

The selection of the embedding dimension can be guided by the embedding theorem, which

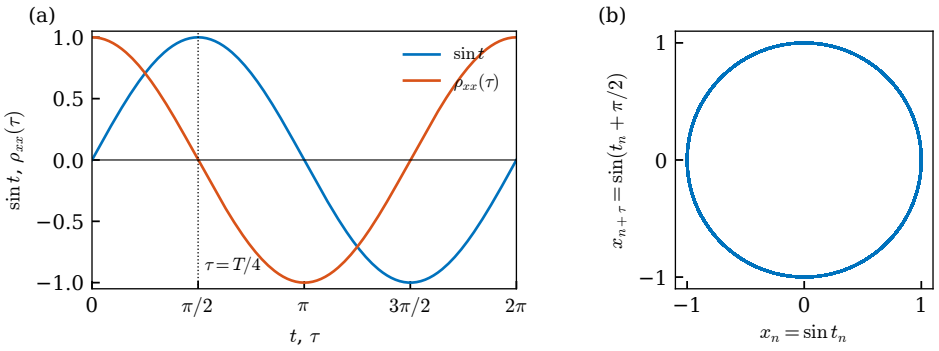


Figure 8.9: Sinusoid and its autocorrelation show that the optimal delay should be $T/4$ since it provides a $\pi/2$ shift in phase. (a) $x(t) = \sin t$ and the autocorrelation coefficient $\rho_{xx}(\tau)$ estimated with the $1/(N-m)$ normalization from 100 periods sampled 200 times per period; it equals $\cos\tau$ to 2×10^{-3} and first crosses zero at $\tau = T/4 = \pi/2$. (b) The two-dimensional forward delay reconstruction $(x_n, x_{n+\tau})$ with $\tau = T/4$ is a circle. Script `python/mkfig_delaycorr.py`.

stipulates that any dimension higher than twice the capacity dimension of the attractor provides a good embedding. However, we do not usually know the system's capacity dimension *a priori* and need to estimate it using the measured data, and the corresponding reconstructed trajectory. In addition, this is only a *sufficient* condition, and there may as well be perfectly good lower-dimensional embeddings. An empirical estimate of the smallest embedding dimension that actually suffices for the data at hand is usually provided by the *false nearest neighbor* (FNN) method of Chapter 10. It is a criterion applied to the measured trajectory rather than a theorem about the attractor, so it neither implies nor is implied by the Embedding Theorem.

The idea behind the FNN method is straightforward (see Fig. 8.10): if two points in d dimensions are *true* nearest neighbors to each other, then adding $(d+1)$-th coordinate should not separate them from each other. If, however, they were nearest neighbors only due to the projection from a higher dimensional structure, they will separate from each other in $(d+1)$-dimensional space and thus would constitute *false neighbors* in d dimensions. The minimum embedding dimension is identified as the smallest dimension having a zero fraction of FNNs.

Pitfall: what delay reconstruction can and cannot establish

A delay reconstruction that meets the conditions of the theorem is a one-to-one image of the attractor. Quantities invariant under such a change of coordinates, the dimensions of Chapter 11 and the Lyapunov exponents of Chapter 12, can therefore be estimated without knowing $\mathcal{S}$. That is all the theorem supplies. It does not recover the state variables or their units: Φ is unknown, and it distorts $\mathcal{A}$ (Fig. 8.6). It does not certify a value of d from data, since D_F is unknown and an estimate of D_2 bounds it only from below (Section 8.1.1). A false-nearest-neighbor count of zero shows that a reconstruction works at the resolution of the record; it does not prove an embedding. Nor does a reconstruction establish determinism. A stochastic record embeds as readily as a chaotic one, and narrow-band noise draws a structured-looking portrait; that question belongs to Chapter 16. Finally, the theorem assumes a stationary, noise-free record of unlimited length. A drifting record (Chapter 17) or a noise-dominated one (Chapter 14) violates the premise, not only the precision.

Problems

Problem 8.1

Core

Consider the Hénon map [93],

$$x_{n+1} = a - x_n^2 + by_n\,, \quad y_{n+1} = x_n\,,$$

which yields chaotic solutions for $a = 1.4$ and $b = 0.3$.

1. Using a typical sequence of x_n, create different two dimensional phase portraits using delay times $\tau = 1, 2, \ldots$.
2. Which picture gives the clearest information about the original system? Why?
3. The map as given is already in delay form, since $y_{n+1} = x_n$ makes $y_n = x_{n-1}$. Write it as the single scalar recurrence in x alone, and interpret what the delay reconstruction of part (1) recovers in light of it.

Problem 8.2

Core

The geometry of a two-dimensional delay reconstruction of a harmonic signal, which is the reasoning behind the quarter-period rule of Fig. 8.9.

1. Let $x(t) = A\cos\omega t$ and $\theta = \omega\tau\Delta t$ be the phase advance over one lag. Show that the points $(x_n, x_{n+\tau})$ lie on an ellipse whose principal axes are the diagonals $x_{n+\tau} = \pm x_n$, with semi-axes $A\sqrt{1+\cos\theta}$ and $A\sqrt{1-\cos\theta}$, so that the ratio of minor to major axis is $\tan(\theta/2)$. For which θ is the ellipse a circle?
2. Now let the record be $x_n + \eta_n$ with white noise of standard deviation σ. Show that the noise adds σ^2 to the mean-square width of the cloud along *every* direction, so that the observed root-mean-square width along the minor axis is $\sqrt{A^2(1-\cos\theta)/2 + \sigma^2}$. Find the smallest θ for which the dynamical width along the minor axis exceeds the noise, for $\sigma/A = 0.05$ (you should find about 0.1 rad, a sixtieth of a period). This is the redundancy limit of Casdagli *et al.* in its simplest form.
3. Verify part (2) on the Lorenz x record sampled at $\Delta t = 0.01$ with 5% additive noise: for $\tau = 1, 2, 5, 10, 20$ samples, form the centered $2 \times N$ matrix of delay pairs and compute its two singular values divided by $\sqrt{N}$, for the clean and the noisy record. You should find the minor axis of the noisy record equal to the root-sum-square of the clean minor axis and σ to within about one percent, exceeding the clean value by some 65% at $\tau = 1$ and 20% at $\tau = 2$, but by only one percent at $\tau = 10$.
4. The Lorenz signal is not harmonic. Which of the conclusions of parts (1)–(2) carried over unchanged, and which needed the numerical check?

Problem 8.3

Computational

Test the embedding theorem directly on the Lorenz system, where the true state is available.

1. Integrate the Lorenz equations (Section 1.4) at $\Delta t = 0.02$ for 20,000 samples after the transient, keeping the full state $\boldsymbol{u}_n = (x_n, y_n, z_n)$. Embed the x record with $\tau = 9$ and $d = 1, \ldots, 5$.

2. For each d, find for every delay vector $\boldsymbol{x}_n$ its nearest neighbor $\boldsymbol{x}_{j(n)}$ in $\mathbb{R}^d$ (use a kd-tree; MATLAB `knnsearch`, SciPy `cKDTree`). Compute the distance $\|\boldsymbol{u}_n - \boldsymbol{u}_{j(n)}\|$ between the corresponding *true* states and record its 99th percentile and its maximum.

3. Plot both against d. You should find the percentile falling from more than 25 (a large fraction of the attractor's size) at $d = 1$ to about 1.5 at $d = 3$, and not falling further at $d = 4$ and 5. Explain the two regimes in terms of the one-to-one property of Φ, and why the distance does not go to zero even when Φ is an embedding.

4. Repeat with $\tau = 1$ and $\tau = 60$. Which regime does each choice damage, and how does the damage appear in your statistic? Relate the answer to the redundancy and irrelevance limits described in this chapter.

Problem 8.4

Core

The theorem holds for *almost every* measurement function. The Lorenz z coordinate is not one of them.

1. The Lorenz equations are invariant under $(x, y, z) \to (-x, -y, z)$. Show that $z(t)$ is therefore the same for a trajectory and for its mirror image, so that the delay map built from z satisfies $\Phi(x, y, z) = \Phi(-x, -y, z)$ and cannot be one-to-one on the attractor, for any d and τ.

2. Repeat the nearest-neighbor test of Problem 8.3 with the z record for $d = 3, 5, 8$. Record the fraction of nearest-neighbor pairs whose true states lie on opposite wings (opposite signs of x). You should find about one half at every d, against a fraction below 1% for the x record at $d = 3$.

3. Plot the three-dimensional delay reconstruction of z and of x side by side. Describe what the z reconstruction looks like. Which later estimates built on it would be wrong (a one-step predictor at a point where the two wings are superposed), and which might survive (the correlation dimension of a set folded onto itself)? Give the reason in each case.

4. Propose a measurement $h(x, y, z)$ that breaks the symmetry, and confirm numerically that its reconstruction passes the test of part (2). Then state what a practitioner with an unknown system should conclude from a reconstruction that fails it.

Problem 8.5

needs Ch. 10 Core

Bookkeeping of the forward delay convention, and its consequences for the routines of Appendix A.

1. A record has N samples. Show that the forward vectors Eq. (8.8) number $N-(d-1)\tau$, and tabulate this count, as a fraction of N, for $N = 5000$ and $(d, \tau) \in \{(3, 10), (6, 10), (6, 50), (10, 50)\}$. At what point does the loss become a concern?

2. Show that the backward vector $[x_n, x_{n-\tau}, \ldots, x_{n-(d-1)\tau}]^{\mathrm{T}}$ at index n equals the forward vector at index $n - (d - 1)\tau$ with its coordinates reversed, and that reversing coordinates is an orthogonal transformation. Conclude that every Euclidean distance between reconstructed points, and hence every nearest-neighbor relation, is the same under the two conventions up to a shift of index.

3. Implement both conventions for the Hénon x record with $d = 3$, $\tau = 1$, and confirm numerically that the nearest-neighbor index lists agree after the shift. Then show where they differ: for the false-nearest-neighbor test of Chapter 10 the appended $(d + 1)$-th coordinate is $x_{n+d\tau}$ in one convention and $x_{n-d\tau}$ in the other. Which one asks whether neighbors stay neighbors in the *future*?

4. The routine `loclin` predicts the last coordinate of the next forward vector. Write down which sample that is in terms of n, d and τ, and what the same prediction would be called in the backward convention.

Problem 8.6

Experimental

Reconstruct the two-well oscillator record of Section 1.4.1.

1. Using the sampling rate and forcing frequency given with the record, express one forcing period in samples. Build two-dimensional delay plots $(s_n, s_{n+\tau})$ for τ equal to about 1/20, 1/8, 1/4, 1/2 and 1 forcing period. Describe how the picture changes, and identify the too-small (points bunched along the diagonal), about-right, and too-large (structure obscured) regimes described in this chapter.

2. Build the three-dimensional reconstruction with the τ you chose and plot it from two viewpoints. Compare with the $(x, \dot{x})$ portrait of the Duffing model (Problem 1.4) and with the stroboscopic section of Problem 7.6. Which features of the model does the measured record reproduce, and which does it not?

3. State the number of reconstructed points for $d = 3$ and $d = 6$ with your τ, and the embedding window $(d - 1)\tau$ in forcing periods. For the chaotic two-well motion the record spends many periods in one well between crossings: how does that constrain τ from above if the reconstruction is to resolve the motion within a well?

4. The embedding theorem requires that the attractor contain no periodic orbit of period $\tau\Delta t$ or $2\tau\Delta t$. The forcing period is a natural period of this system. Which τ values does that rule out, and did your visual choice in part (1) avoid them?

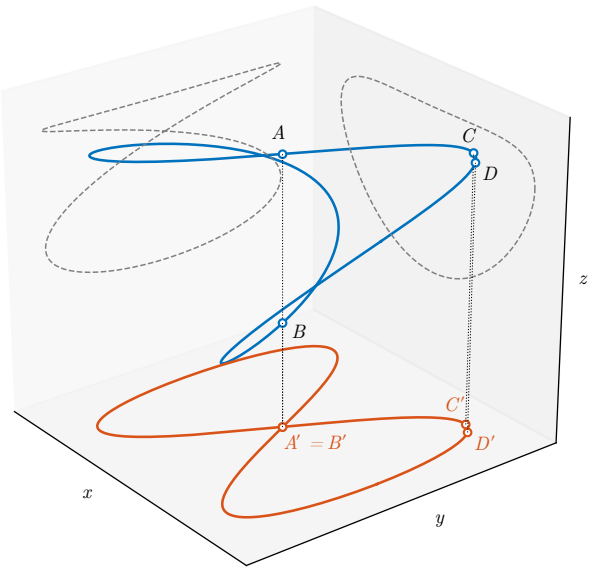


Figure 8.10: Basic idea of false nearest neighbors is illustrated by a three-dimensional closed curve (blue line), $\boldsymbol{r}(s) = (\cos s,\ \sin 2s,\ 0.7\sin s + 0.2\cos 2s + 0.4)$, and its two-dimensional projection onto the (x, y) plane (red line), a figure-eight that crosses itself. The grey dashed curves are the projections onto the two other planes. The points $A = \boldsymbol{r}(\pi/2)$ and $B = \boldsymbol{r}(3\pi/2)$, a distance 1.4 apart in three dimensions, project onto the same point $A' = B'$: they are false neighbors in the projection. The true neighbors C and D ($\Delta s = 0.15$, 0.12 apart) remain close neighbors C' and D' in the projection. Script `python/mkfig_fnn.py`.

9
Selecting Delay Time for Phase Space Reconstruction

The estimation of optimal delay time can be approached from a purely geometrical perspective [110] or by considering linear/nonlinear (auto)correlations in the time series [65, 66]. As discussed in the previous chapter, there are primarily three methods for estimating the delay time using: (1) visual inspection of the reconstructed phase portrait (see Fig. 9.1 for the illustrations); (2) the first zero crossing of the autocorrelation coefficient (refer to Fig. 8.9), a heuristic distinct from the correlation time of Chapter 3; and (3) the first minimum of the average mutual information [66]. In some cases, selection of the optimal delay time and embedding dimension cannot be decoupled (e.g., when mutual information changes with the embedding dimension [81]). Some studies have advocated focusing on the so-called *embedding window* ($\tau_e = (d-1)\tau$) that describes the total time span covered by all coordinates in each of the reconstructed phase-space points [14, 72, 123, 168]. Others use model-based parameter estimation [19, 103, 116, 123, 189] to estimate both optimal delay and embedding dimension simultaneously. More recently, the optimal short-term prediction metric

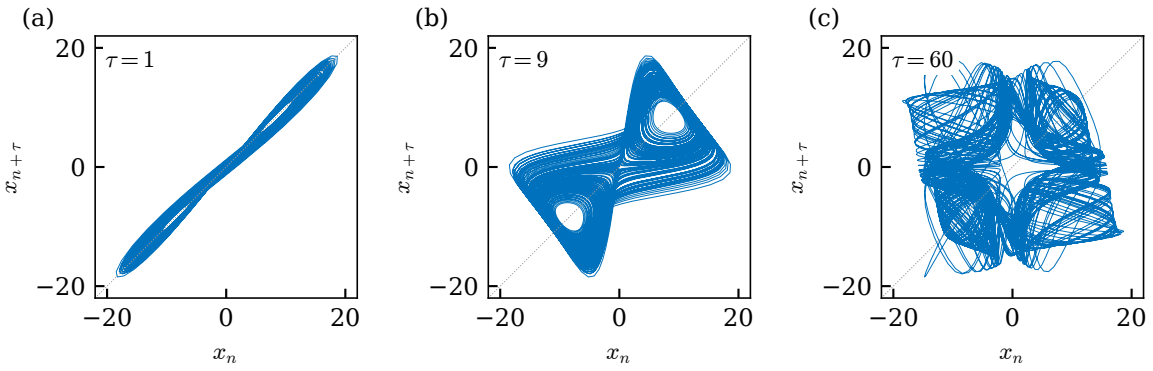


Figure 9.1: Determining delay "by eye" on the two-dimensional forward delay plots $(x_n, x_{n+\tau})$ of the Lorenz x record of Section 1.4 ($\Delta t = 0.02$, $N = 10{,}000$ samples after a transient of 20 time units): if points are bunched along the diagonal, then τ is too small ($\tau = 1$ sample, a); if we see the dynamical structure and the phase portrait looks "nice," then τ is about right ($\tau = 9$, the first minimum of the average mutual information, b); and if the dynamical structure is obscured or "too complex," then τ is too long ($\tau = 60$, more than a lobe circuit, c). Script `python/mkfig_visdel.py`.

(based on artificial neural network model) was used to estimate appropriate delays and embedding dimension [138]. A similar idea, in conjunction with average mutual information for the delay estimation, was used in estimating embedding dimension in Ref. [29]. Some of these methods are applicable in specific scenarios; others have some deficiencies, as discussed in Refs. [4, 81, 105]. Others advocate choosing the lag conservatively and then applying a spatial decorrelation transform [111] to allow for the even unfolding.

Before we discuss the nonlinear methodologies used in selecting the delay time, we need to briefly review the concept of *information entropy* addressed in *information theory* [184]. Entropy, as viewed from the *information theory* perspective, is an average measure of information coming from a probabilistic or stochastic data source. This view is connected to but is not the same as thermodynamic entropy (a measure of disorder in a system) in statistical mechanics.

9.1 Shannon Entropy

A measure of "disorder" or "uncertainty" in a system can also be called *entropy* from information perspective. Given a set $\mathcal{A}$ with partition of "boxes" $\{\mathcal{B}_i\}_{i=1}^{n_b}$, as shown in Fig. 9.2 by partitioning the x axis into equal-sized bins for one-dimensional data or as in Fig. 9.3 for a two-dimensional set, *Shannon Entropy* is defined as:

$$H(\mathcal{A}) = -\langle \log p_i \rangle = -\sum_{i=1}^{n_b} p_i \log p_i \,, \tag{9.1}$$

where $p_i = \text{Prob}\{x \in \mathcal{B}_i\}$.[1] The base of the logarithm fixes the unit: base two gives *bits*, which is what the routine of Appendix A.1 returns, and the natural logarithm gives *nats*. Only the unit changes, not the location of any minimum.

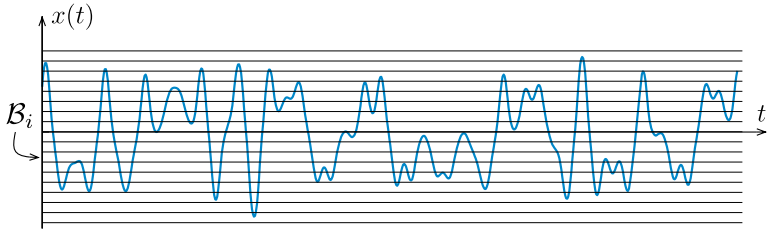


Figure 9.2: A time series can be partitioned by segmenting x axis into equal-sized bins $\mathcal{B}_i$

Consider the special case of a *certain* state on a partition, shown in Fig. 9.4, where $p_k = 1$ and, since $\sum_i p_i = 1$, $p_i = 0$ for all $i \neq k$. In addition we also have (see Fig. 9.5)[2]

$$\lim_{p\to 0} p \ln p = \lim_{p\to 0} \frac{\ln p}{1/p} = \lim_{p\to 0} \frac{1/p}{-1/p^2} = \lim_{p\to 0} (-p) = 0 \,. \tag{9.2}$$

Therefore, when the state is known to lie in one box, $H(\mathcal{A}) = 0$. In other words, if the uncertainty in *state* is zero (only one box has $p = 1$), entropy is also zero. This is a statement about the distribution of states, not about the dynamics. A deterministic chaotic trajectory spreads over many boxes. The logistic map at $r = 4$ has $H = 3.97$ nats on 64 equal bins, and a positive entropy rate of $\ln 2$ per step. Determinism fixes the next state exactly, given the present one; it does not concentrate the

[1] In terms of Shannon Information the partition is an "alphabet" and each box is a "letter" or "symbol."

[2] This result is obtained using l'Hospital's rule

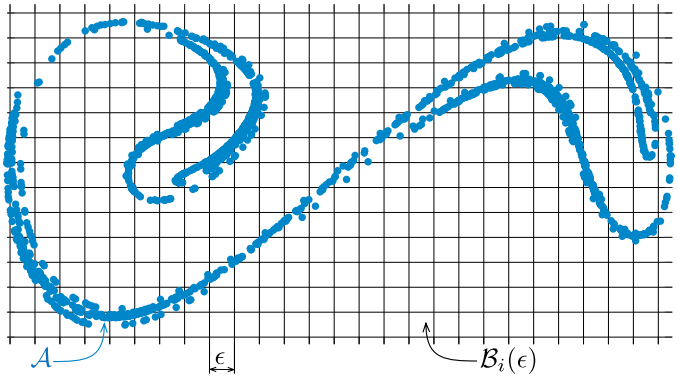


Figure 9.3: A set of points $\mathcal{A}$ in a Poincaré section covered by the boords of size ϵ.

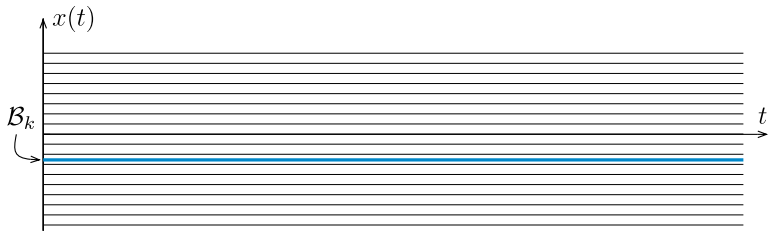


Figure 9.4: A certain state on a partition $\mathcal{B}_i$, where $p_k = 1$

distribution of states. On the other hand, H achieves its maximum $\log n_b$ when $p_i = p_j = 1/n_b$ (please refer to Fig. 9.6 for a binary set example, when we only have two boxes). Thus, maximum entropy is associated with maximum uncertainty in state (any box is equally likely to be populated).

9.2 Mutual Information

Information is measured by the reduction of entropy: learning the outcome of one process reduces the uncertainty about another, and the amount of that reduction, $I = \Delta H$, is the information the first carries about the second. The definition below makes this precise. Given a partition $\{\mathcal{B}_i\}_{i=1}^m$ on $\mathcal{A}$ of process x we denote the probability of each partition as:

$$p_i^{(x)} = \text{Prob}\,\{x \in \mathcal{B}_i\} \tag{9.3}$$

and if we have another process y with partition $\{\mathcal{C}_i\}_{i=1}^n$,

$$p_i^{(y)} = \text{Prob}\,\{y \in \mathcal{C}_i\} \tag{9.4}$$

Now we also define the *joint probability* (a probability mass function on the product partition):

$$p_{ij}^{(xy)} \triangleq \text{Prob}\,\{x \in \mathcal{B}_i, y \in \mathcal{C}_j\} \tag{9.5}$$

and the corresponding *joint entropy* is

$$H_{xy} = -\sum_{i=1}^{m}\sum_{j=1}^{n} p_{ij}^{(xy)} \log p_{ij}^{(xy)} \tag{9.6}$$

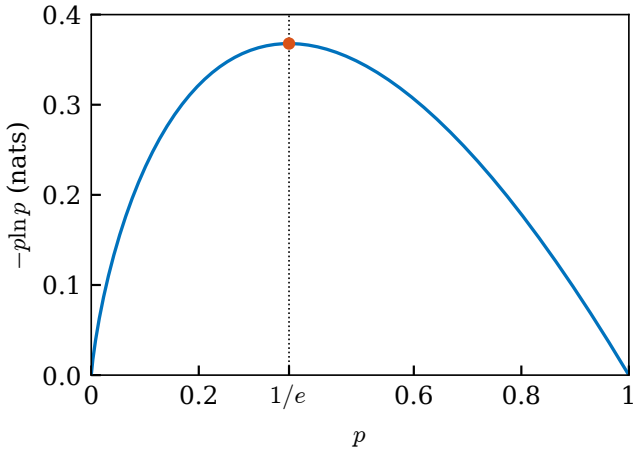


Figure 9.5: Plot of $-p\ln p$ versus p (nats). The limit at $p\to 0$ is zero and the maximum, $1/e$, is at $p=1/e$. Script `python/mkfig_entropy.py`.

If the two processes are independent, i.e.

$$p_{ij}^{(xy)} = p_i^{(x)} p_j^{(y)} \tag{9.7}$$

then

$$\begin{aligned} H_{xy} &= -\sum_{i=1}^{m}\sum_{j=1}^{n}\left(p_{ij}^{(xy)}\log p_i^{(x)} + p_{ij}^{(xy)}\log p_j^{(y)}\right) \\ &= -\sum_{i=1}^{m} p_i^{(x)}\log p_i^{(x)} - \sum_{j=1}^{n} p_j^{(y)}\log p_j^{(y)} \\ &= H_x + H_y \end{aligned} \tag{9.8}$$

where we have used the fact that $\sum_j p_{ij}^{(xy)} = p_i^{(x)}$. Therefore, Shannon entropy is additive for x and y independent. Now, we define the *mutual information* between x and y processes as

$$\begin{aligned} I_{xy} &= H_x + H_y - H_{xy} \\ &= \sum_{i=1}^{m}\sum_{j=1}^{n}\left(p_{ij}^{(xy)}\log p_{ij}^{(xy)} - p_{ij}^{(xy)}\log p_i^{(x)} - p_{ij}^{(xy)}\log p_j^{(y)}\right) \\ &= \sum_{i=1}^{m}\sum_{j=1}^{n} p_{ij}^{(xy)}\left(\log p_{ij}^{(xy)} - \log p_i^{(x)} - \log p_j^{(y)}\right) , \end{aligned} \tag{9.9}$$

and

$$I_{xy} = \sum_{i=1}^{m}\sum_{j=1}^{n} p_{ij}^{(xy)}\log\frac{p_{ij}^{(xy)}}{p_i^{(x)}p_j^{(y)}} . \tag{9.10}$$

Therefore, if $p_{ij}^{(xy)} = p_i^{(x)}p_j^{(y)}$, then $I_{xy}=0$. That is, mutual information $I_{xy}=0$ if x process is completely *independent* of y, which is a much stronger condition than x and y being merely *uncorrelated*.

Now consider a case when $x=x(t)$ and $y=x(t+\tau)$. Then, for a stationary signal, $p_i^{(x)} = p_i^{(y)} \equiv p_i$ and $p_{ij}^{(xy)} = p_{ij}^{(x(t)x(t+\tau))} \equiv p_{ij}(\tau)$, so

$$I(\tau) = \sum_{i=1}^{m}\sum_{j=1}^{m} p_{ij}(\tau)\log\frac{p_{ij}(\tau)}{p_i p_j} . \tag{9.11}$$

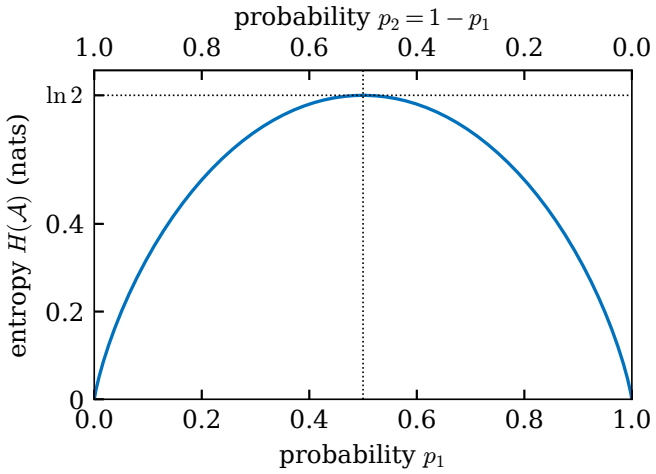


Figure 9.6: Plot of $H(\mathcal{A}) = -p_1 \ln p_1 - p_2 \ln p_2$ versus the probabilities of a two-box partition ($p_1 + p_2 = 1$). The maximum $\ln 2$ nats (1 bit) is at $p_1 = p_2 = 1/2$. Script `python/mkfig_entropy2.py`.

When we use $\log_2$ in I equation, then it is given in *bits*. In addition, we can simplify further using the log property:

$$\begin{aligned} I(\tau) &= \sum_{i=1}^{m}\sum_{j=1}^{m} p_{ij}(\tau)\,(\log p_{ij}(\tau) - \log p_i - \log p_j)\ , \\ &= \sum_{i=1}^{m}\sum_{j=1}^{m} p_{ij}(\tau)\log p_{ij}(\tau) - \sum_{i=1}^{m} p_i \log p_i - \sum_{j=1}^{m} p_j \log p_j\ , \end{aligned} \tag{9.12}$$

or

$$I(\tau) = \sum_{i=1}^{m}\sum_{j=1}^{m} p_{ij}(\tau)\log p_{ij}(\tau) - 2\sum_{i=1}^{m} p_i \log p_i\ . \tag{9.13}$$

This is the form used in Ref. [105] for *average mutual information*.

Choosing the Delay from $I(\tau)$

Fraser and Swinney [66] proposed the first *minimum* of $I(\tau)$ — the first τ at which $\frac{\mathrm{d}I}{\mathrm{d}\tau} = 0$ *and* the curve turns upward — as a "good" delay; Kantz and Schreiber [105] report the criterion and are sceptical of any single rule. The reasoning runs as follows. $I(\tau) \geq 0$ measures the *redundancy* between $x(t)$ and $x(t+\tau)$, and vanishes only when the two are perfectly independent, in which case the new coordinate carries entirely new information. Since the information a coordinate *adds* moves oppositely to I, a local minimum of I is a local maximum of the added information. That is what makes it the natural candidate for the delay. Note that a stationary point alone is not enough: an inflection or a maximum of I satisfies $\mathrm{d}I/\mathrm{d}\tau = 0$ too.

Fig. 9.7 shows average mutual information estimates for the AR(2) records of Fig. 3.2 and for the Duffing and Lorenz records of Section 1.4. For the AR(2) processes, which are Gaussian, the mutual information is fixed by the correlation coefficient alone, $I(\tau) = -\frac{1}{2}\log_2(1-\rho_\tau^2)$ (Problem 9.4), so the minima of I are the zero crossings of ρ_{xx} and the estimate follows the exact curve to within a few hundredths of a bit at lags of five or more (Fig. 9.7(b)); at small lags the estimate falls short by about 0.06 bit, and at large lags it sits on the positive bias floor of the plug-in estimator, $(n_b - 1)^2/2N$

nats, described below. For the Duffing record the curve has only a shoulder near $\tau = 14$ and its first clear minimum at $\tau = 46$ samples; for the Lorenz record the first minimum is at $\tau = 8$ samples ($n_b = 32$; 9 with $n_b = 16$).

The minimum may not exist at all, and additive noise or too short a record can make it hard to locate. The delay the criterion returns is a candidate, not an answer. The *optimal* delay depends on what the reconstruction is for. One may instead ask which τ minimizes the variability of a Lyapunov exponent, or of a fractal dimension, estimated from the reconstruction.

The estimator matters as much as the criterion. The plug-in estimate replaces p_i and p_{ij} by bin counts divided by N, and that substitution carries a bias whose sign is known.

Pitfall: the estimated $I(\tau)$ never falls to zero

Because $-\sum p \log p$ is concave, the plug-in entropy is biased low, by approximately $(n_{\text{bins}} - 1)/(2N)$ nats for a histogram with n_{bins} occupied cells (the Miller–Madow correction). The joint histogram has n_b^2 cells against n_b for each marginal, so the bias of $\hat{I} = \hat{H}_x + \hat{H}_y - \hat{H}_{xy}$ is *positive*, about $(n_b - 1)^2/(2N)$ nats: 0.11 nats for $n_b = 16$ and $N = 1000$, against 0.12 measured on independent data. This is why an estimated $I(\tau)$ does not fall to zero at large τ but to a floor that rises with n_b^2/N, why the first minimum becomes shallower as n_b grows, and why Problem 9.2 asks for the mean occupancy N/n_b^2 to exceed about ten. An estimated $I(\tau)$ therefore does not fall to zero at large τ but to a floor that rises with n_b^2/N. The first minimum becomes shallower as n_b grows, and Problem 9.2 asks for the mean occupancy N/n_b^2 to exceed about ten for that reason. The routine of Appendix A.1 uses n_b equal-width bins over the range of the data (default $n_b = 32$) and the plug-in formula, so its output carries the positive floor described above; report n_b and N with any delay chosen from it.

Fraser and Swinney [66] avoided the fixed grid with an *adaptive partition* that subdivides a cell only while its occupancy is inconsistent with a uniform distribution, so that dense regions are resolved finely and empty ones coarsely. The k-nearest-neighbor estimator of Kraskov, Stögbauer, and Grassberger [119] uses no partition at all: it estimates I from the distances to the k-th neighbor in the joint space and the counts within that distance in each marginal, is nearly unbiased for independent data, and has a single parameter k (typically 2–6) with a variance–bias trade-off in place of n_b. For delay selection the location of the first minimum is what matters, and it is usually insensitive to the estimator once $N/n_b^2 \gtrsim 10$; the depth of the minimum and the value of I at large τ are not, and should not be interpreted without the bias in mind.

Mutual information also has a use beyond delay selection. I_{xy} can provide evidence of nonlinear coupling between two processes x and y that appear linearly uncorrelated, and a structured $I(\tau)$ is weak evidence of dynamical structure in a single record.

Some Properties of I_{xy}

1. $I_{xy} = H_x + H_y - H_{xy} = I_{yx}$, or x and y give equal information about each other.

2. Using the joint probability property $p(x,y) = p(x|y)p(y)$, we get

$$\begin{aligned} I_{xy} &= \sum_{i=1}^{m}\sum_{j=1}^{n} p_{ij}^{(xy)} \log \frac{p_{ij}^{(xy)}}{p_i^{(x)} p_j^{(y)}} \,, \\ &= \sum_{i=1}^{m}\sum_{j=1}^{n} p_{ij}^{(xy)} \log \frac{p_{ij}^{(x|y)}}{p_i^{(x)}} = \sum_{i=1}^{m}\sum_{j=1}^{n} p_{ij}^{(xy)} \log \frac{p_{ij}^{(y|x)}}{p_j^{(y)}} \,, \\ &= \sum_{i=1}^{m}\sum_{j=1}^{n} p_{ij}^{(xy)} \log p_{ij}^{(x|y)} - \sum_{i=1}^{m} p_i^{(x)} \log p_i^{(x)} \,. \end{aligned} \tag{9.14}$$

Now, considering that

$$H_{x|y} = -\sum_{i=1}^{m}\sum_{j=1}^{n} p_{ij}^{(xy)} \log p_{ij}^{(x|y)} \,, \tag{9.15}$$

we get that

$$I_{xy} = H_x - H_{x|y} = H_y - H_{y|x} \,, \tag{9.16}$$

where the first equality gives the reduction in the uncertainty about x once y is known, and the second the reduction in the uncertainty about y once x is known.

9.3 Delay Selection on the Measured Record

The two-well oscillator record of Section 1.4.1 is the case the preceding sections were written for. One measured channel, no model, and no delay to check the answer against. Fig. 9.8 applies the criteria of this chapter to it.

The settings first. The whole record is used, $N = 96{,}000$ samples, and the lags run from 0 to 40 samples, two and a half forcing periods. The average mutual information is the plug-in estimate of Appendix A.1 at $n_b = 16$, 32 and 64 bins. The mean occupancies N/n_b^2 are 375, 94 and 23, so all three satisfy the rule of Problem 9.2. The autocorrelation coefficient is the estimator of Chapter 3 on the mean-removed record.

In panel (a) the value at zero lag is the entropy of the record, $I(0) = H = 4.790$ bits at $n_b = 32$ (3.803 and 5.786 bits at $n_b = 16$ and 64), which is a check on the binning and nothing more. I then falls steeply, turns up at $\tau = 5$ samples, and reaches a local maximum near $\tau = 8$. The location of that first minimum is $\tau = 5$ for all three bin counts. Its depth is not: 0.614, 0.656 and 0.700 bits at $n_b = 16$, 32 and 64. Location stable, depth not, which is the split Problem 9.2 asks about and the reason only the location is reported. In panel (b) the autocorrelation falls to $1/e$ at 8 samples and crosses zero at 12, with $\rho(5) = 0.636$. Three candidates, then: 5, 8 and 12 samples, or 0.31, 0.50 and 0.75 of a forcing period.

The value to report is $\tau = 5$ samples, together with $n_b = 32$ and $N = 96{,}000$, since the estimate is meaningless without them. It is the first minimum of the average mutual information and it is what every later chapter uses on this record.

Why 5 and not the zero crossing at 12 is settled by the sampling density. The record carries only 16 samples per forcing period, so a lag of 12 samples is three quarters of a period. Panels (c) to (e) show what each choice produces. At $\tau = 2$, an eighth of a period, the cloud lies along the diagonal, $\rho = 0.931$, and the two coordinates are nearly the same number: the redundant case of Fig. 9.1(a). At $\tau = 5$ the set opens out, the two wells separate, and the fold between them is visible. At $\tau = 12$ the set is creased and folded across itself, ρ has already gone negative, -0.0195, and

points on opposite sides of a well are mapped on top of one another. The embedding window says the same thing. With $d_E = 4$ (Chapter 10) the window $(d_E - 1)\tau$ is 15 samples at $\tau = 5$, just under one forcing period, and 36 samples at $\tau = 12$, or 2.25 forcing periods. In the second case the four coordinates of a single delay vector straddle more than two cycles of the drive.

The general point is that the first zero crossing of the autocorrelation is a decorrelation time. On a periodically driven record it is set by the drive period, not by the folding time of the dynamics, and there is no reason for the two to coincide. The criterion is a useful default and it is not one here.

Three checks support $\tau = 5$. First, the first minimum does not move with the bin count, as panel (a) shows. Second, it does not move along the record: the first and second halves, 48,000 samples each, both give $\tau = 5$, at 0.662 and 0.664 bits, and so does each of ten blocks of 600 forcing periods. Third, the delay feeds something measurable. At $\tau = 5$ the false-nearest-neighbor fraction of Chapter 10 falls as 0.9997, 0.7591, 0.1731, 0.0013 for $d = 1$ to 4 and is zero at $d = 5$. At $\tau = 12$ it stalls, at 0.0432, 0.0247 and 0.0210 for $d = 4$, 5 and 6, and never reaches zero: the long delay does not unfold the set at any dimension the record supports. At $\tau = 2$ it does reach zero, 0.0001 at $d = 4$, so the neighbor count rejects the long delay but not the short one. That is what panel (c) is for.

What would have made the number untrustworthy is worth stating, because none of it happened. A first minimum that moved with n_b, or between halves of the record, would mean the estimator and not the dynamics was being read. A curve whose only minimum lay on the large-lag floor would mean the same. That floor is near 0.2 bits at $n_b = 32$, and it is not estimator bias: the positive bias $(n_b - 1)^2/(2N)$ is 0.005 nats, or 0.007 bits. It is the redundancy the periodic drive leaves at every lag. The later minima at 21, 28 and 36 samples all sit on it, within three hundredths of a bit of one another, and none of them is a delay. Finally, a record drifting in mean or variance would make any of this local in time; Section 1.4.1 shows that this one does not drift.

Problems

Problem 9.1

Core

Consider the x time series from the Lorenz oscillator:

$$\dot{x} = 10\,(y - x)\,, \quad \dot{y} = x\,(28 - z) - y, \quad \text{and} \quad \dot{z} = xy - \frac{8}{3}z\,,$$

starting from $x(0) = 5$, $y(0) = -5$, and $z(0) = 20$ initial condition and with 0.02 sampling time. Estimate and plot the corresponding average mutual information and autocorrelation coefficient versus the delay time. Estimate what should be the optimal delay time using both metrics. Now plot the phase portraits using various delays and identify the one that most resembles the original phase portrait z versus x. Is there any difference between this reconstruction and the optimal delay reconstructions? What happens if we increase the delay further? Is the first zero crossing of the autocorrelation a reasonable estimate for the delay?

Problem 9.2

Computational

Problem 9.1 compared the first minimum of the average mutual information against the autocorrelation. This problem asks what the AMI estimate itself depends on.

1. Implement the average mutual information $I_{xx}(k)$ using the binned estimator of the appendix. Reproduce the AMI curve for the Lorenz record of Problem 9.1.

2. Recompute it with $n_b = 8, 16, 32, 64$ and 128 bins. Plot the curves together. Does the *location* of the first minimum move? Does its *depth*?

3. Explain the two competing effects: too few bins washes out structure, too many leaves most bins empty. Estimate, for your record length, the largest n_b for which the mean occupancy exceeds ten points per bin.

4. Repeat parts (1)–(2) for records of length $N = 10^3$, 10^4 and 10^5. At which N does the first minimum become stable against the bin count?

5. State what you would report as the delay, and what you would have to say alongside it for another reader to reproduce your number.

Problem 9.3

needs Ch. 10 Extension

The AMI is one of several delay criteria and they do not always agree.

1. For the Lorenz record, locate: the first zero crossing of $R_{xx}(k)$; the delay at which R_{xx} first falls to $1/e$; the first minimum of $I_{xx}(k)$; and the delay at which I_{xx} first reaches a fixed small fraction of $I_{xx}(0)$. Tabulate all four.

2. Embed with each and compute the false-nearest-neighbor fraction of Chapter 10 for each embedding. Which delay gives the cleanest FNN curve?

3. Repeat the whole exercise for a quasiperiodic signal $x_n = \sin(2\pi n\sqrt{2}/100) + \sin(2\pi n\sqrt{3}/100)$, whose two frequencies (0.0141 and 0.0173 cycles per sample) are incommensurate and well below the Nyquist frequency. Does the AMI have a clear first minimum at all? What does that tell you about the criterion's scope?

4. Now repeat for the two-well Duffing record. The AMI of a signal that switches between wells often has a shallow, ambiguous minimum. Report what you find and how you would proceed.

Problem 9.4

Core

For a Gaussian process the mutual information is a function of the correlation coefficient alone, which makes the AR(2) process of Chapter 3 an exact test of the estimator.

1. Let (x, y) be jointly Gaussian with zero means, unit variances and correlation coefficient ρ. Starting from the continuous analogue of $I_{xy} = H_x + H_y - H_{xy}$ with $H = -\int p \ln p$, and the entropy $\tfrac{1}{2}\ln\left((2\pi e)^n \det \boldsymbol{C}\right)$ of an n-variate Gaussian with covariance $\boldsymbol{C}$, show that $I_{xy} = -\tfrac{1}{2}\ln(1-\rho^2)$ nats. Convert to bits and evaluate for $\rho = 0.95$ and $\rho = 0.05$.

2. Conclude that for a stationary Gaussian process $I(k) = -\tfrac{1}{2}\ln\left(1-\rho_k^2\right)$, so that the minima of the AMI are exactly the zero crossings of the autocorrelation coefficient, and the AMI carries no information the autocorrelation does not.

3. Generate 40,000 samples of the AR(2) process with $\rho = 0.05$ and compute ρ_k exactly from the Yule–Walker recursion of Problem 3.3. Estimate $I(k)$ for $k = 0, \ldots, 40$ with the routine of Appendix A.1 at $n_b = 16$ and 32 and plot both over the exact curve of part (2). You should find agreement to within a few hundredths of a bit at lags of five or more, a first minimum at lag 5 for both bin counts, where ρ_k crosses zero, and an estimate that falls short of the exact value at lag 1 (by about 0.07 bit at $n_b = 32$ and 0.23 bit at $n_b = 16$), where the joint density is concentrated near the diagonal and the coarse bins cannot resolve it.

4. The estimated minimum does not reach zero. Compare its value with the positive bias $(n_b - 1)^2/(2N)$ nats quoted in this chapter, for both n_b. Then explain why, for this process, a reader who computed only the autocorrelation would have chosen the same delay, and what kind of record would make the AMI and the autocorrelation disagree.

Problem 9.5

Core

Uncorrelated is not independent: the case the AMI exists for.

1. Iterate the Ulam map $y_{n+1} = 1 - 2y_n^2$ of Problem 4.2 for 40,000 steps from $y_0 = 0.1$. Problem 3.5 showed that every lag of its autocorrelation coefficient lies inside the white-noise band. Estimate $I(k)$ for $k = 0, \ldots, 5$ with $n_b = 32$. You should find $I(1)$ above 3 bits, decreasing by roughly 0.8 bit per lag, against about 0.015 bit at every lag $k \geq 1$ for Gaussian white noise of the same length.

2. Explain the value of $I(1)$ from the definition: given the bin of y_n, how many bins can y_{n+1} fall in? Why then is $I(1)$ not equal to H_y, the entropy of one sample?

3. Explain the decrease of $I(k)$ with k from the fact that the map is two-to-one, so that y_n determines y_{n+k} but y_{n+k} determines y_n only up to 2^k preimages. What is the expected decrease per lag, in bits, for a fine partition? Your estimate will fall somewhat short of it; explain why coarse bins cannot keep the 2^k preimages apart.

4. The zero-crossing rule for the delay does not exist for this record, and the AMI has no minimum at all. What delay would you use to embed it, and what property of the record tells you that a delay of one is right? State the general lesson for map-generated data.

Problem 9.6

needs Ch. 11 Experimental

Section 9.3 chose $\tau = 5$ samples for the two-well oscillator record of Section 1.4.1. This problem asks how much record that choice needed, and how far it can be pushed.

1. Reproduce Fig. 9.8(a) and (b): the AMI at $n_b = 16$, 32 and 64 and the autocorrelation coefficient (Problem 3.6), for lags 0 to 40 samples, which is two and a half forcing periods. Confirm the occupancy N/n_b^2 for each bin count and the values $I(0)$, which should be the entropy of the record.

2. Tabulate the four delay candidates of Problem 9.3: the first zero crossing of ρ_{xx}, the $1/e$ lag, the first AMI minimum, and the first lag at which the AMI falls to a tenth of $I(0)$. Express each as a fraction of the forcing period, using the 16 samples per period measured from the record. Which of the four would a reader with only the autocorrelation have chosen, and by how much would it have been wrong?

3. Truncate the record to $N = 6000$, 24,000 and 48,000 samples, that is 375, 1500 and 3000 forcing periods, and repeat part (1) for each. Does the first minimum move, either with N or with n_b? Compare the large-lag floor of each curve with the bias $(n_b - 1)^2/(2N)$ quoted in this chapter, and say how much record a delay estimate needs against how much the correlation dimension of Chapter 11 needs.

4. The AMI of this record has further minima at 21, 28 and 36 samples, all within three hundredths of a bit of one another. Plot the two-dimensional reconstruction at $\tau = 21$ alongside the one at $\tau = 5$. Explain, from the fact that the record is driven at a fixed period, why none of these later minima is a usable delay, and what a delay near a multiple of the forcing period does to a delay vector.

5. Split the record into ten blocks of 600 forcing periods and estimate the first AMI minimum in each at $n_b = 32$. Report the spread; you should find none. Then state what you would have concluded had one block disagreed, and which of stationarity, record length and bin count you would have tested first.

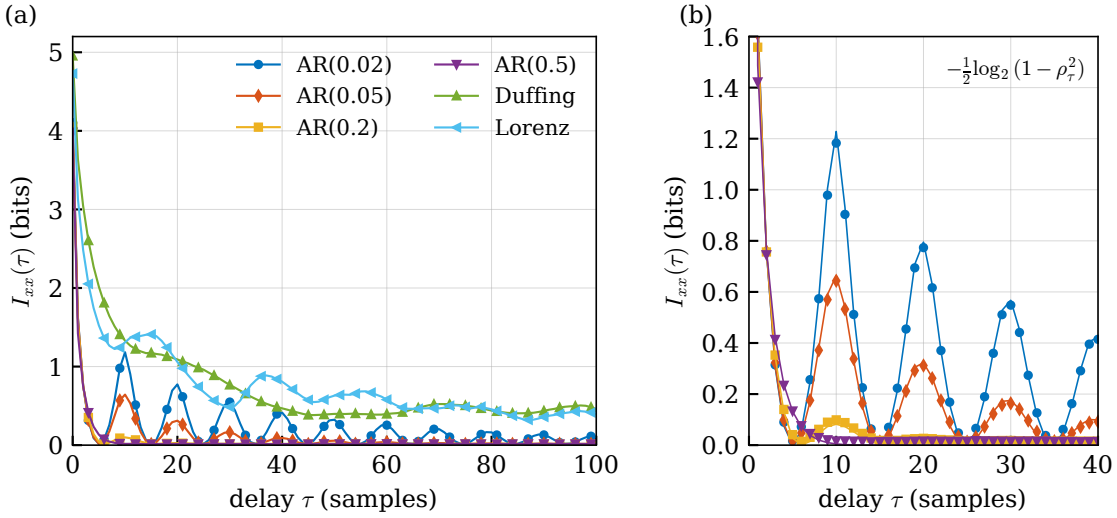


Figure 9.7: (a) Average mutual information $I_{xx}(\tau)$ versus delay τ (in samples; plug-in estimator with $n_b = 32$ equal-width bins, records of $N = 40{,}000$ samples) for the colored random processes $x_{n+1} = (2-(\pi/10)^2-\rho)x_n + (\rho-1)x_{n-1} + \eta_n$, where η_n is unit-variance Gaussian white noise, $x_1 = x_2 = 0$, and $\rho = 0.02, 0.05, 0.2, 0.5$ (the records of Fig. 3.2); each case is labeled AR(ρ). In addition, the average mutual information of the x time series of the Duffing oscillator $\ddot{x} + 0.25\,\dot{x} - x + x^3 = 0.3\cos t$ from $(x, \dot{x}) = (-0.67, -0.37)$, sampled at 0.1 after a transient of 200 time units, and of the Lorenz system ($\dot{x} = 10\,(y-x)$, $\dot{y} = x\,(28-z) - y$, $\dot{z} = xy - \frac{8}{3}z$) from $(5, -5, 20)$, sampled at 0.02 after a transient of 20 time units, are plotted. (b) The AR(ρ) estimates (markers) against the exact Gaussian value $-\frac{1}{2}\log_2(1-\rho_\tau^2)$ (thin lines) with ρ_τ from the Yule–Walker recursion. Script `python/mkfig_amiplots.py`.

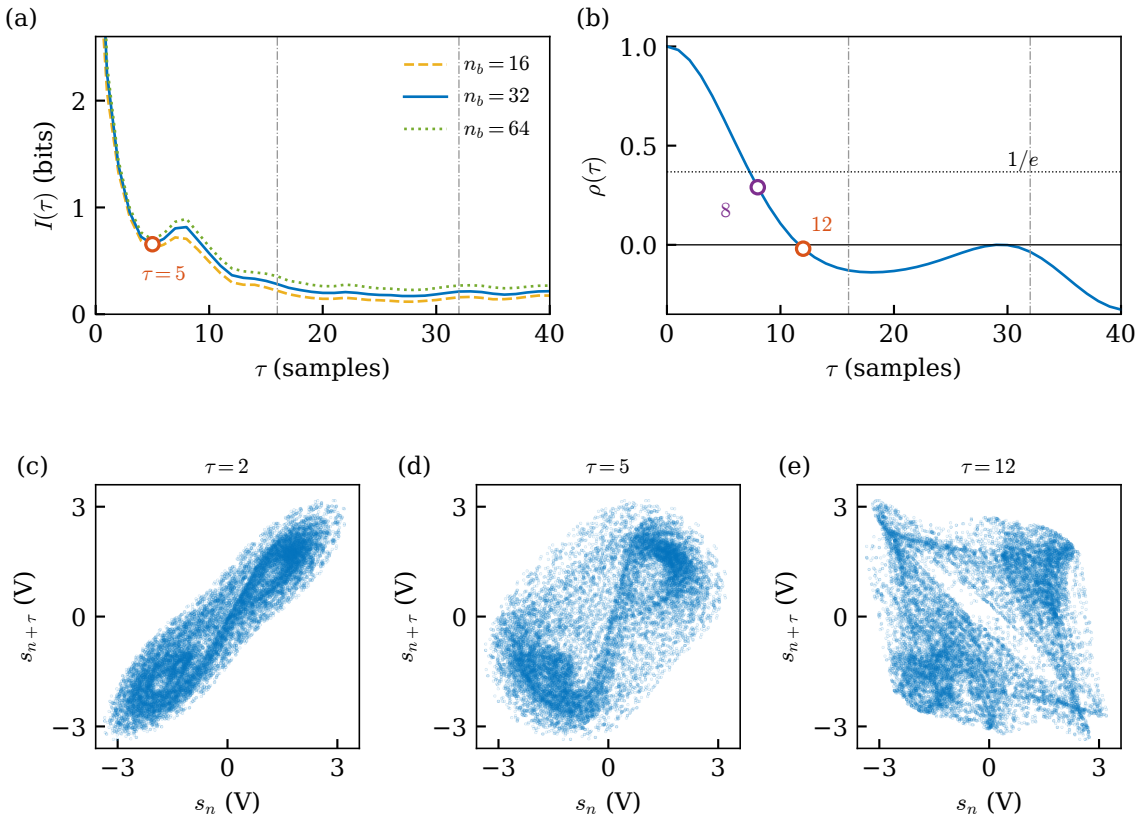


Figure 9.8: Delay selection on the two-well oscillator record of Section 1.4.1, $N = 96{,}000$ samples. (a) average mutual information $I(\tau)$ in bits, plug-in estimator of Appendix A.1 at $n_b = 16$, 32 and 64 bins, lags 0 to 40 samples; the vertical scale is clipped at 2.6 bits, so $I(0) = H$ is off the panel. The first minimum is marked. (b) the autocorrelation coefficient $\rho(\tau)$ over the same lags, with the $1/e$ level and the lags 8 and 12 marked. The dash-dotted lines in (a) and (b) sit at one and two forcing periods, 16 and 32 samples. (c), (d), (e) delay reconstructions $(s_n, s_{n+\tau})$ of the first 16,000 samples, 1000 forcing periods, at $\tau = 2$, 5 and 12 samples. Script `python/mkfig_twowell_delay.py`.

10 Selecting Embedding Dimension for Phase Space Reconstruction

Chapter 8 supplied the delay vector $\boldsymbol{x}_n = [x_n, x_{n+\tau}, \ldots, x_{n+(d-1)\tau}]^{\mathrm{T}}$ and the theorem that guarantees an embedding for $d > 2D_F$. Chapter 9 fixed the lag τ. This chapter fixes the remaining parameter, the embedding dimension d, from the measured record itself. The theorem is of little direct use here: D_F is unknown, the bound is only sufficient, and a working embedding is often found well below it.

The tool is the *false nearest neighbor* (FNN) test of Kennel, Brown and Abarbanel [112]. Two reconstructed points that are close in d dimensions but far apart in $d+1$ were close only by projection. The smallest d at which no such pairs remain is the dimension sought. The test is a geometric criterion applied to a finite, noisy trajectory, so it is an estimation problem with its own bias and record-length requirement. The chapter treats it as one.

Section 10.1 states the problem and the geometry of a false neighbor. Section 10.2 gives the Kennel–Brown–Abarbanel test with the rationale of both of its criteria, and the analysis of Hegger and Kantz [90] that shows how it fails on short records. Section 10.3 gives Cao's threshold-free alternative [18]. Section 10.4 develops the statistical view: what nearest-neighbor distances look like for noise, why temporal correlation must be removed, and how a composite fraction separates deterministic from stochastic records. Section 10.5 discusses a spatial decorrelation step. Section 10.6 collects the results into a procedure with its pitfalls. Every later chapter that reconstructs a phase space uses the dimension found here. Among them are the correlation sum of Chapter 11, the Lyapunov estimators of Chapter 12, the noise reduction of Chapter 14, and the recurrence analysis of Chapter 18.

10.1 The Problem

The reconstructed trajectory must be an image of the true one: one-to-one, with no self-intersections. Chapter 8 argued this with the cartoons of Fig. 8.7. When d is too small the projection folds the attractor onto itself. Distinct states $\boldsymbol{u}$ and $\boldsymbol{u}'$ then map to the same or nearly the same $\boldsymbol{x}$. The reconstructed point has two futures, so no deterministic map $\hat{\varphi}_t$ exists on the reconstruction. Every estimate built on neighborhoods is corrupted: local linear models fit two flows at once, divergence curves mix separating and non-separating pairs, and the correlation sum sees a denser set than exists. This is *under-embedding*.

When d is too large the reconstruction is still an embedding, but a poor one for a finite record. Each additional coordinate consumes τ samples, so only $N - (d-1)\tau$ vectors remain. Noise, which

has no preferred dimension, fills every added coordinate with variance σ_η^2. The distances between genuine neighbors on the attractor grow with d while the attractor itself does not, so neighborhoods that were well populated become sparse [23]. The redundancy of the extra coordinates also lengthens the embedding window $(d-1)\tau$ and folds the attractor in the way a too-large lag does (Chapter 9). This is *over-embedding*. The minimal sufficient d is wanted, not the largest safe one.

False nearest neighbor

Let $\boldsymbol{x}_i$ be a reconstructed point in d dimensions and $\boldsymbol{x}_{j(i)}$ its nearest neighbor. Append the $(d+1)$-th coordinate to both. The pair is a *true* neighbor pair if the two points remain close in $d+1$ dimensions, and a *false* pair if they separate. A false pair was close only because the projection from $d+1$ to d dimensions brought two distant pieces of the attractor together.

Figure 8.10 of Chapter 8 shows the geometry. A closed curve in three dimensions is projected onto a plane. Two pieces of the curve that pass over one another in three dimensions cross in the plane, and the points A', B' at the crossing are neighbors of the projection alone. The pair C', D' lies on one piece of the curve and stays close when the third coordinate is restored. Under-embedding produces false pairs. At the minimal embedding dimension the fraction of false pairs among all nearest-neighbor pairs falls to zero. It stays at zero for every larger d, because appending coordinates cannot create new intersections. The FNN test is a search for the first zero of this fraction.

Two facts complicate the search. First, on a finite record a nearest neighbor need not be a neighbor at all. In high dimension the nearest point may sit at a distance comparable to the attractor size, and its $(d+1)$-th coordinate is then unrelated to that of the reference point. Second, noise creates neighbors of its own: two points may be close because noise pushed them together, and they separate when the next noisy coordinate is appended. Both effects make the fraction of false pairs depend on N and on the noise level as well as on d. Every criterion below is a way of controlling them.

10.1.1 Test Records and Surrogates

The chapter uses six records of $N = 20{,}000$ samples each, the systems of Figs. 3.2 and 9.7 with the running parameters of Section 1.4: the Lorenz x coordinate, the Duffing x coordinate, Gaussian and uniform white noise, and the AR(2) process of Chapter 3 at $\rho = 0.05$ and 0.5. The AR records are colored noise with a decaying oscillation in their autocorrelation, and their stochastic component grows with ρ. They are the hard case: a method that reports a low dimension for them has confused correlation with determinism. Noisy versions of the Lorenz and Duffing records carry additive Gaussian noise whose standard deviation is quoted as a percentage of σ_x. Every record is normalized to zero mean and unit variance before use, so all distances below are in units of σ_x. Table 10.1 lists the parameters used in every figure; the delay is the first minimum of the average mutual information of each record (Chapter 9), and the Theiler window of Section 10.4.1 is three times that delay.

Every result is shown against a *surrogate*. A surrogate is a record that shares some properties of the data but has no deterministic structure; any feature the data and the surrogate share is not evidence of determinism. Chapter 16 treats surrogates in general [180]. Here the simpler device of Kennel and Abarbanel [111] is used throughout. For each d-dimensional neighbor pair the $(d+1)$-th coordinate of the neighbor is replaced by a value drawn at random from the whole record. This

destroys *all* dependence of the appended coordinate on the neighbor relation, the linear dependence included, so a linearly correlated process separates from this surrogate as well as a deterministic one does; Section 10.4.4 shows this on the AR records, and Section 10.6 adds a surrogate of Chapter 16 that preserves the linear correlation. A dimension estimate is trusted only where the data curve separates from the surrogate curve.

Table 10.1: Records and parameters used in the figures of this chapter. Every record has $N = 20{,}000$ samples and is normalized to $\sigma_x = 1$; the delay τ is the first minimum of the average mutual information (one sample for white noise, the quarter period for the AR records); the Theiler window is $w = 3\tau$ in every figure. Thresholds are swept where a family of curves is shown. k and l are the neighbor count and strand half-length of Section 10.4.4. Scripts `python/mkfig_<label>.py`, records and cache in `python/_ch10data.py`.

Record	Parameters	Δt	τ, w
Lorenz x	$(\sigma, \rho, \beta) = (10, 28, 8/3)$ from $(5, -5, 20)$, 50 time units discarded	0.02	8, 24
Duffing x	$\ddot{x} + 0.25\dot{x} - x + x^3 = 0.3\cos t$, 200 time units discarded	0.1	14, 42
Hénon x	$(a, b) = (1.4, 0.3)$, 1000 iterates discarded	—	1, 3
Gaussian, uniform	white noise, seed 0	—	1, 3
AR(ρ)	the AR(2) process of Chapter 3, $\rho = 0.05$ and 0.5, 200 samples discarded	—	5, 15

Figure	Statistic	r	s	k, l
10.1	Kennel fraction Eq. (10.5), $a = 2$	$2, 4, \ldots, 20$	—	1, 0
10.3	$\langle \epsilon_i \rangle$, $\langle \delta_i \rangle$, $\langle \delta_i / \epsilon_i \rangle$	—	—	1, 0
10.2	Cao's E_1, E_2	—	—	1, 0
10.4	composite FNN Eq. (10.20)	$2, \ldots, 20$; 4	0.5; $0.1, \ldots, 1$	1, 0
10.5	Kennel and composite, noisy	$2, \ldots, 20$; 10	—; 1	1, 0 and 10, τ
10.6	composite, Cao, noisy Lorenz	$2, \ldots, 20$; 10	0.5	1, 0 and 10, τ

10.2 The False Nearest Neighbor Test

Work in the d-dimensional reconstruction. For each point $\boldsymbol{x}_i$ find its nearest neighbor $\boldsymbol{x}_{j(i)}$ in the Euclidean norm. Define the d-dimensional separation and the separation of the appended coordinates,

$$\epsilon_i \triangleq \left\| \boldsymbol{x}_i - \boldsymbol{x}_{j(i)} \right\| , \qquad \delta_i \triangleq \left| x_{i+d\tau} - x_{j(i)+d\tau} \right| . \tag{10.1}$$

The dependence of j on i is understood from here on. The $(d+1)$-dimensional separation is $\sqrt{\epsilon_i^2 + \delta_i^2}$, since the first d coordinates are shared.

10.2.1 The Two Criteria of Kennel, Brown and Abarbanel

Kennel *et al.* [112] declare the pair false if either of two conditions holds.

The first is the *ratio criterion*. The appended coordinate separates the pair by more than r times its d-dimensional separation,

$$\frac{\delta_i}{\epsilon_i} > r \, , \tag{10.2}$$

with r (Kennel's R_{tol}) a fixed threshold, typically $r = 10$. The rationale is dynamical. For a true neighbor pair the $(d+1)$-th coordinates are values of one smooth measurement at two nearby states. Then δ_i is of the order of ϵ_i times a derivative of the delay map. A ratio of ten or more cannot come

from a smooth map over a small separation. The fraction of pairs failing this criterion,

$$f_{nn}(d;r) \triangleq \text{Prob}\left\{\delta_i > r\,\epsilon_i\right\}\,, \tag{10.3}$$

is the *original fraction*. The smallest d at which it reaches zero is taken as the embedding dimension.

The second is the *size criterion*. The $(d+1)$-dimensional separation is a substantial fraction of the attractor size,

$$\frac{\sqrt{\delta_i^2+\epsilon_i^2}}{\sigma_x} > a\,, \tag{10.4}$$

with σ_x the standard deviation of the record and a (Kennel's A_{tol}) typically 2. The rationale is statistical, and Kennel *et al.* state it: on a finite record in high dimension the nearest neighbor is not close. When ϵ_i is itself comparable to σ_x, the ratio δ_i/ϵ_i is of order one whatever the origin of the data, and the ratio criterion passes every pair. The size criterion catches these pairs. A neighbor that is a whole attractor diameter away is not a neighbor, and a pair that is stretched to the attractor size by the appended coordinate is false. Together the two criteria give the *Kennel fraction*

$$\hat{f}_{nn}(d;r) \triangleq \text{Prob}\left\{(\delta_i > r\,\epsilon_i)\ \vee\ \left(\delta_i^2+\epsilon_i^2 > a^2\sigma_x^2\right)\right\}\,. \tag{10.5}$$

For a clean record from a low-dimensional attractor both fractions fall to zero at the same d and stay there. Figure 10.1(a,b) shows the Kennel fraction for the Lorenz and Duffing records: at $r=10$ it is 0.07 at $d=2$ and zero from $d=3$ for Lorenz, and 0.50, 0.04 and zero at $d=2$, 3, 4 for Duffing; the reading is $d=3$ and $d=4$ for every $r \geq 4$, well separated from the surrogates. At $r=2$ a few per cent of true pairs are counted as false at every d, which is the threshold dependence of the remark box below.

10.2.2 Why the Test Fails on Short Records

The ratio criterion should report a large fraction of false neighbors for noise at every d, since noise is infinite-dimensional. Hegger and Kantz [90] showed that it does not, and why. Take white noise uniform on $[0,1]$. The appended coordinates of a pair are independent draws, so $\delta_i = |U_1 - U_2|$ has $\text{Prob}\{\delta_i > c\} = (1-c)^2$ for $0 \leq c \leq 1$ and zero for $c > 1$. Conditioning on the d-dimensional separation ϵ_i and averaging over pairs,

$$f_{nn}(d;r) = \left\langle (1 - r\,\epsilon_i)^2\ \Theta(1 - r\,\epsilon_i) \right\rangle\,, \tag{10.6}$$

where Θ is the Heaviside function. The truncation matters. Without it the expression grows again for $r\epsilon_i > 1$, where the true probability is zero; with it every pair whose neighbor lies farther than $1/r$ contributes nothing. The fraction is therefore small whenever most nearest-neighbor separations exceed $1/r$, and not merely when their average does.

For N points of white noise in d dimensions the typical nearest-neighbor separation scales as $\langle \epsilon_i \rangle \simeq N^{-1/d}$. The fraction collapses when this reaches $1/r$, that is, at $N \simeq r^d$. To see a non-zero original fraction for noise, and so to distinguish noise from determinism, one needs $N \geq r^d$. Read as a bound on the dimension being swept, the same condition says the test cannot be trusted once

$$d > d_{\max} \triangleq \frac{\ln N}{\ln r}\,. \tag{10.7}$$

Beyond $d_{\max}$ the record is too short to populate a neighborhood at threshold r. The original fraction then reports a low embedding dimension whatever the data. The limit is severe. At $r=10$ a record

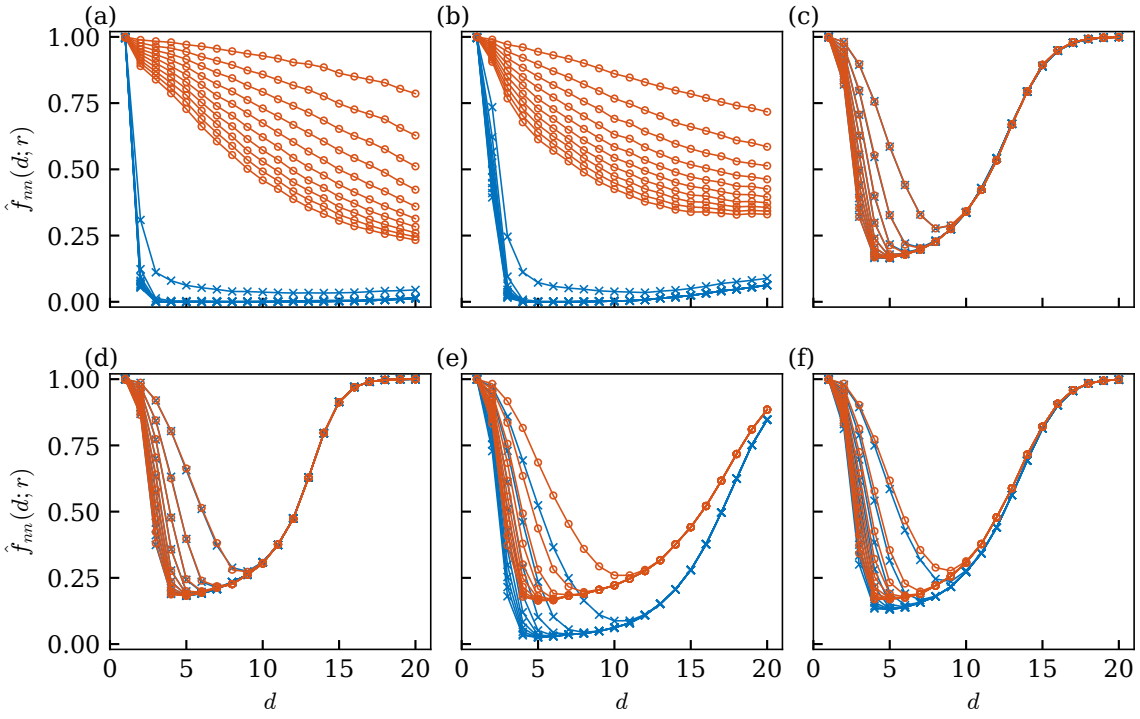


Figure 10.1: Kennel fraction of false nearest neighbors, Eq. (10.5) with $a = 2$, for the Lorenz (a), Duffing (b), Gaussian (c), uniform (d), AR(0.05) (e) and AR(0.5) (f) records of Table 10.1, $N = 20{,}000$, Theiler window $w = 3\tau$. The family of curves is $r = 2, 4, \ldots, 20$; the fraction decreases with r. Lines with circles are the surrogate results. Script `python/mkfig_CFNNO.py`.

of $N = 10^5$ points is exhausted by $d = 5$, one of $N = 2 \times 10^4$ points by $d = 4.3$, and one of $N = 10^4$ points by $d = 4$. Lowering r raises $d_{\max}$ but admits more true pairs as false. On the Gaussian record of Fig. 10.1 the original fraction at $r = 10$ is 0.55, 0.19, 0.04 and 0.00 at $d = 3, 4, 5, 6$: it collapses where Eq. (10.7) says it should. The size criterion Eq. (10.4) was Kennel's answer to the same problem; Fig. 10.1(c,d) shows what it achieves. For white noise the Kennel fraction rises back to one at large d, as it should, because the nearest neighbor is then an attractor size away. But between the two regimes it dips: at $r = 10$ to 0.17 at $d = 5$, and it does not return to 0.5 until $d = 12$; the surrogates dip with it. On the AR records, Fig. 10.1(e,f), the dip is deeper, to 0.03 for AR(0.05), and the recovery toward one comes later the smaller ρ. A dip that noise and colored noise both produce cannot be read as evidence of an attractor.

Hegger and Kantz instead discard pairs whose d-dimensional separation exceeds σ_x/r, on the grounds that such a pair is not a neighbor pair and should not be counted either way. The fraction implemented in the TISEAN package [91, 105] is

$$\tilde{f}_{nn}(d;r) \triangleq \frac{\sum_{i=1}^{N-d\tau} \Theta\left(\delta_i - r\epsilon_i\right) \Theta\left(\sigma_x - r\epsilon_i\right)}{\sum_{i=1}^{N-d\tau} \Theta\left(\sigma_x - r\epsilon_i\right)}\,, \tag{10.8}$$

the *TISEAN fraction*. The upper limit $N - d\tau$ is the number of points for which $x_{i+d\tau}$ exists. Hegger and Kantz report that this fraction stays near 0.5–0.6 for white noise, independent of d and r, when N is large. On the test records of this chapter, however, the denominator of Eq. (10.8) collapses: at $r = 10$ the share of admissible pairs for the Gaussian record is 0.69 at $d = 3$, 0.06 at

$d = 4$ and zero from $d = 6$, and the AR records behave the same way. With no admissible pairs the fraction is undefined, and in practice it is reported as zero. The fraction then indicates a low dimension for noise and does not separate from its surrogate. Problem 10.2 asks the reader to plot this denominator.

Known weaknesses of the FNN test

1. *Threshold dependence.* The answer depends on r (and a). The dependence is weak for clean deterministic data and strong for noisy or stochastic data. A single curve at $r = 10$ is not a result; a family over r is.

2. *Record length.* The test is uninformative for $d > \ln N / \ln r$, Eq. (10.7). This is a bias toward *low* d: a short record always looks low-dimensional.

3. *Temporal correlation.* For a densely sampled flow the nearest neighbor of $\boldsymbol{x}_i$ is usually $\boldsymbol{x}_{i\pm1}$, which is a true neighbor at every d and carries no information about the geometry. Without a Theiler window the fraction is biased low (Section 10.4.1).

4. *Noise.* Additive noise creates false pairs at every d, so the fraction floors at a positive value and the zero is never reached. The floor rises with the noise level. This is a bias toward *high* d when the zero is sought, and toward ambiguity when a knee is sought instead.

10.3 Cao's Method

Cao [18] removed the threshold by replacing the count of pairs that fail a criterion with the mean of the quantity the criterion tests. Let $n(i, d)$ be the index of the nearest neighbor of $\boldsymbol{x}_i$ in the d-dimensional reconstruction, in the maximum norm, with $\left\|\boldsymbol{x}_i - \boldsymbol{x}_{n(i,d)}\right\|_\infty > 0$. Form the ratio of the $(d+1)$-dimensional to the d-dimensional separation of the same pair and average it,

$$a(i,d) = \frac{\left\|\boldsymbol{x}_i^{(d+1)} - \boldsymbol{x}_{n(i,d)}^{(d+1)}\right\|_\infty}{\left\|\boldsymbol{x}_i^{(d)} - \boldsymbol{x}_{n(i,d)}^{(d)}\right\|_\infty}, \qquad E(d) = \frac{1}{N - d\tau} \sum_{i=1}^{N-d\tau} a(i,d), \tag{10.9}$$

where the superscript gives the dimension of the delay vector. In the maximum norm $a(i, d) \geq 1$ always, and $a(i, d) = 1$ exactly when the appended coordinates differ by no more than the existing separation. The mean $E(d)$ is the average stretching that one more coordinate produces. Cao's first statistic is its relative change,

$$E_1(d) = \frac{E(d+1)}{E(d)}. \tag{10.10}$$

While d is below the embedding dimension, adding a coordinate unfolds false pairs and $E(d)$ changes substantially. Once the attractor is embedded, the stretching is set by the smoothness of the delay map alone and no longer depends on d, so $E_1(d)$ stops changing and settles at one. If E_1 stops changing at $d = d_0$, the minimum embedding dimension is $d_0 + 1$.

The statistic is free of thresholds, but "stops changing" is a judgment, and for noise $E_1(d)$ also rises toward one, slowly, as d grows. Cao therefore added a second statistic that separates deterministic from stochastic records. It uses the appended coordinates alone,

$$E^*(d) = \frac{1}{N - d\tau} \sum_{i=1}^{N-d\tau} \left| x_{i+d\tau} - x_{n(i,d)+d\tau} \right|, \qquad E_2(d) = \frac{E^*(d+1)}{E^*(d)}. \tag{10.11}$$

For white noise the appended coordinates are independent of the neighbor relation, so $E^*(d)$ is the same at every d and $E_2(d) = 1$ for all d. For a deterministic record the appended coordinate of a true neighbor is close to that of the reference point, $E^*(d)$ falls as the attractor unfolds, and $E_2(d) \neq 1$ for at least some d. A record for which E_2 is one at every d is treated as stochastic and the value of E_1 is not read.

Figure 10.2 shows both statistics for four records, computed as in Eqs. (10.9)–(10.11) with the nearest neighbor sought outside the Theiler window of Table 10.1. On the Lorenz record E_1 jumps from 0.18 at $d = 2$ to 0.91 at $d = 3$ and creeps upward thereafter; E_2 departs from one at every d, 0.58 at $d = 2$ and 1.08–1.19 beyond. On the Duffing record E_1 is 0.05, 0.50 and 0.93 at $d = 2, 3, 4$, and E_2 settles near 1.15. On the Hénon record E_1 reaches 0.94 at $d = 2$ and E_2 sits near 1.4. On Gaussian noise E_1 climbs from 0.14 to 0.93 over $d = 2, \ldots, 10$ with no plateau, while E_2 stays within one per cent of unity. The reading is $d = 3$ for Lorenz, $d = 4$ for Duffing, $d = 2$ for Hénon, and "stochastic" for the noise.

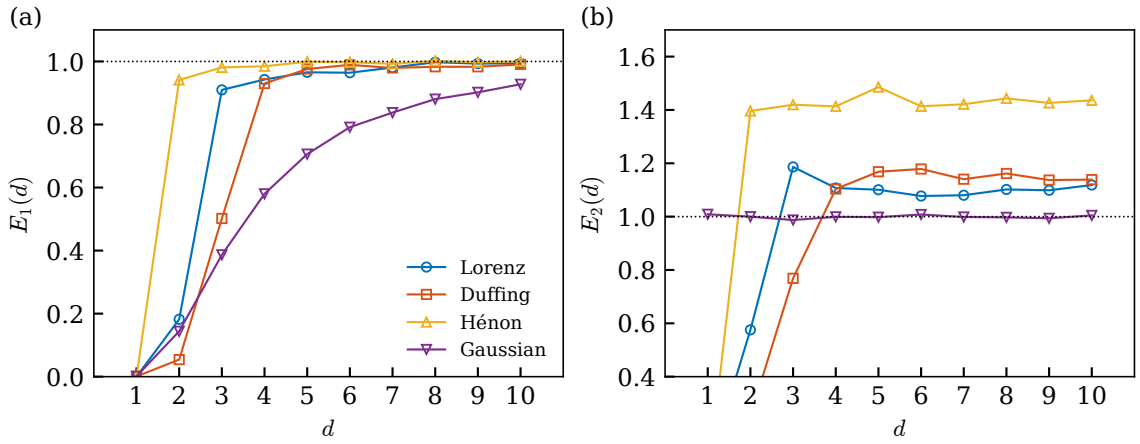


Figure 10.2: Cao's statistics $E_1(d)$ (a) and $E_2(d)$ (b), Eqs. (10.10), and (10.11), for the Lorenz, Duffing, Hénon and Gaussian-noise records of Table 10.1 ($N = 20{,}000$, maximum norm, nearest neighbor outside the Theiler window $w = 3\tau$). The dotted line is $E = 1$. Script `python/mkfig_caoE.py`.

Cao's method is the standard threshold-free alternative to the FNN fraction and should be run alongside it. Its weaknesses are the ones the FNN test has, in different form. The plateau of E_1 is approached from below, so a slowly rising E_1 on a short or noisy record can be misread as a plateau at too low a d. Noise lifts E_1 toward one at all d and pushes E_2 toward one, so a noisy deterministic record drifts toward the stochastic verdict. The record-length bound Eq. (10.7) applies unchanged, since E_1 is built from the same nearest-neighbor separations. And the method as published does not use a Theiler window; one was added for Fig. 10.2 and should be for any flow data, exactly as below.

10.4 Nearest-Neighbor Statistics

The FNN fraction counts pairs that cross a threshold. Its behavior is easier to understand, and to repair, from the distributions of ϵ_i and δ_i themselves. This section develops those distributions for noise and for deterministic records, following Kennel and Abarbanel [111] and extending their analysis. The result is a composite fraction that reads correctly on all eight test records and on

noisy deterministic data.

10.4.1 Temporal Decorrelation and the Reliable Fraction

For a flow sampled at Δt much shorter than the orbital period, consecutive points $\boldsymbol{x}_i$ and $\boldsymbol{x}_{i+1}$ are close at every d. They are true neighbors, trivially, and they are also the nearest neighbors of one another for most i. A test that admits them measures the sampling rate and not the geometry. The remedy is the *Theiler window* [202, 203]. The nearest neighbor is sought only among points separated in time by more than w samples,

$$j = \operatorname*{argmin}_{j'} \|\boldsymbol{x}_i - \boldsymbol{x}_{j'}\| \;, \quad \text{subject to} \quad |i - j'| > w \,. \tag{10.12}$$

Kennel and Abarbanel [111] recommend w of two to three times the first minimum of the average mutual information. An alternative is the lag at which the average mutual information reaches its asymptotic floor. Section 12.5.4 returns to the window for Lyapunov estimation and Chapter 11 for the correlation sum. The window is a number of samples and must be rescaled when the sampling rate changes (Problem 10.3).

With temporally uncorrelated neighbors, Kennel and Abarbanel test the appended coordinate against the attractor size rather than against ϵ_i,

$$\bar{f}_{nn}(d; s) \triangleq \text{Prob}\,\{\delta_i > s\,\sigma_x\} \;, \tag{10.13}$$

the *reliable fraction*, with s a threshold in units of σ_x. Its behavior is read off the lower half of Fig. 10.4 below, where the composite fraction at $r = 4$ coincides with Eq. (10.13) for all $d \geq 4$ (the ratio clause fires only at small d). For the Lorenz and Duffing records the fraction reaches zero at $d = 3$ and 4 and then rises slowly with d at small s, to 0.03 and 0.10 at $d = 20$ for $s = 0.5$. For white noise it is constant in d at the value $\text{Prob}\{\delta_i > s\sigma_x\}$ of two independent samples, 0.72 at $s = 0.5$ and 0.48 at $s = 1$ for Gaussian noise, and the data and surrogate curves coincide. For the AR records the fraction is also constant in d, below the surrogate for the weakly stochastic record and at the surrogate for the strongly stochastic one. The reliable fraction never reports a low dimension for noise. Its cost is the threshold s, and a slow rise with d for deterministic data that Section 10.4.3 explains.

10.4.2 False Nearest Strands

Additive noise creates neighbor pairs that are close by chance. Such a pair separates when the next coordinate is appended, and is counted as false, even when the underlying states were not neighbors at all. Kennel and Abarbanel [111] suppress this by following pairs in time. If $\boldsymbol{x}_j$ is the neighbor of $\boldsymbol{x}_i$ because the two trajectory segments run side by side, then $\boldsymbol{x}_{j+1}$ is the neighbor of $\boldsymbol{x}_{i+1}$ as well, and the pairs link into a *nearest-neighbor strand*. A chance pair created by noise has no such continuation.

Nearest-neighbor strands

1. For each $\boldsymbol{x}_i$ find the temporally uncorrelated nearest neighbor $\boldsymbol{x}_{j(i)}$ by Eq. (10.12).
2. Look back over the preceding pairs $\{\boldsymbol{x}_{i-k}, \boldsymbol{x}_{j(i-k)}\}_{k=1}^{w}$. If for some k, and take the smallest such $k = k^\star$, the neighbor indices advance in step, $j(i) = k^\star + j(i - k^\star)$, assign the pair $[\boldsymbol{x}_i, \boldsymbol{x}_{j(i)}]$ to the strand that contains $[\boldsymbol{x}_{i-k^\star}, \boldsymbol{x}_{j(i-k^\star)}]$. Otherwise start a new strand.
3. After all points are assigned there are N_s strands with index sets S_k, $k = 1, \ldots, N_s$. Average the appended-coordinate separation along each strand,

$$\langle \delta_i \rangle_k \triangleq \frac{1}{|S_k|} \sum_{i \in S_k} \delta_i \,, \tag{10.14}$$

where $|S_k|$ is the number of pairs in the strand. Declare the strand false if $\langle \delta_i \rangle_k > s\,\sigma_x$.

The *reliable FNS fraction* is the fraction of false strands,

$$\bar{f}_{ns}(d; s) \triangleq \text{Prob}\left\{ \langle \delta_i \rangle_k > s\,\sigma_x \right\} . \tag{10.15}$$

Kennel and Abarbanel report that it falls to zero more slowly than the reliable FNN fraction, so that a threshold on the fraction is needed to read a dimension from it, and that it is less sensitive to additive noise. The chapter does not pursue the strand fraction itself. It uses the strand idea in a different place: as a rule for choosing, among several candidate neighbors, the one whose trajectory segment tracks that of the reference point (Section 10.4.4).

10.4.3 Nearest-Neighbor Distances for Noise and for Attractors

The behavior of every fraction above follows from how $\langle \epsilon_i \rangle$ and $\langle \delta_i \rangle$ depend on d. Consider first the d-dimensional nearest-neighbor separation $\epsilon_i(d)$ for Gaussian white noise. The separation of two arbitrary points follows a χ_d distribution scaled by $\sqrt{2}\sigma_x$ [83]. The separation to the *nearest* of N points is smaller by a factor that scales with the point density as $N^{-1/d}$. Combining the mean of χ_d with this density factor gives the heuristic

$$\langle \epsilon_i(d) \rangle \simeq \sqrt{2}\, \frac{\Gamma\left((d+1)/2\right)}{\Gamma\left(d/2\right)} \left(\frac{\sqrt{d}}{N} \right)^{1/d} \sigma_x \,, \tag{10.16}$$

where Γ is the gamma function. It is a heuristic and not a derivation, and its accuracy should be stated. For Gaussian noise with $N = 2 \times 10^4$ it underestimates the simulated mean nearest-neighbor separation. The shortfall is 40 per cent at $d = 2$, 24 per cent at $d = 3$, 12 per cent at $d = 5$, 6 per cent at $d = 8$ and 4 per cent at $d = 12$. The underestimate at small d is invisible on the scale of Fig. 10.3(a), where the separations at $d \leq 3$ are below $0.1\sigma_x$, and the formula follows the Gaussian and uniform records closely from $d \approx 5$ upward. What matters for the FNN test is the growth: $\langle \epsilon_i \rangle$ for noise grows roughly as $\sqrt{d}\, N^{-1/d}$, reaching the attractor size σ_x at $d = 8$ for $N = 2 \times 10^4$ and $3\sigma_x$ at $d = 20$.

For a deterministic record the separation behaves differently, Fig. 10.3(a). Below the embedding dimension it is small and nearly constant: the attractor is folded and neighbors are abundant. Above it the separation grows, but along the attractor and not into the extra dimensions. Two points close on the attractor at time i are the initial conditions of two trajectory segments, and the appended

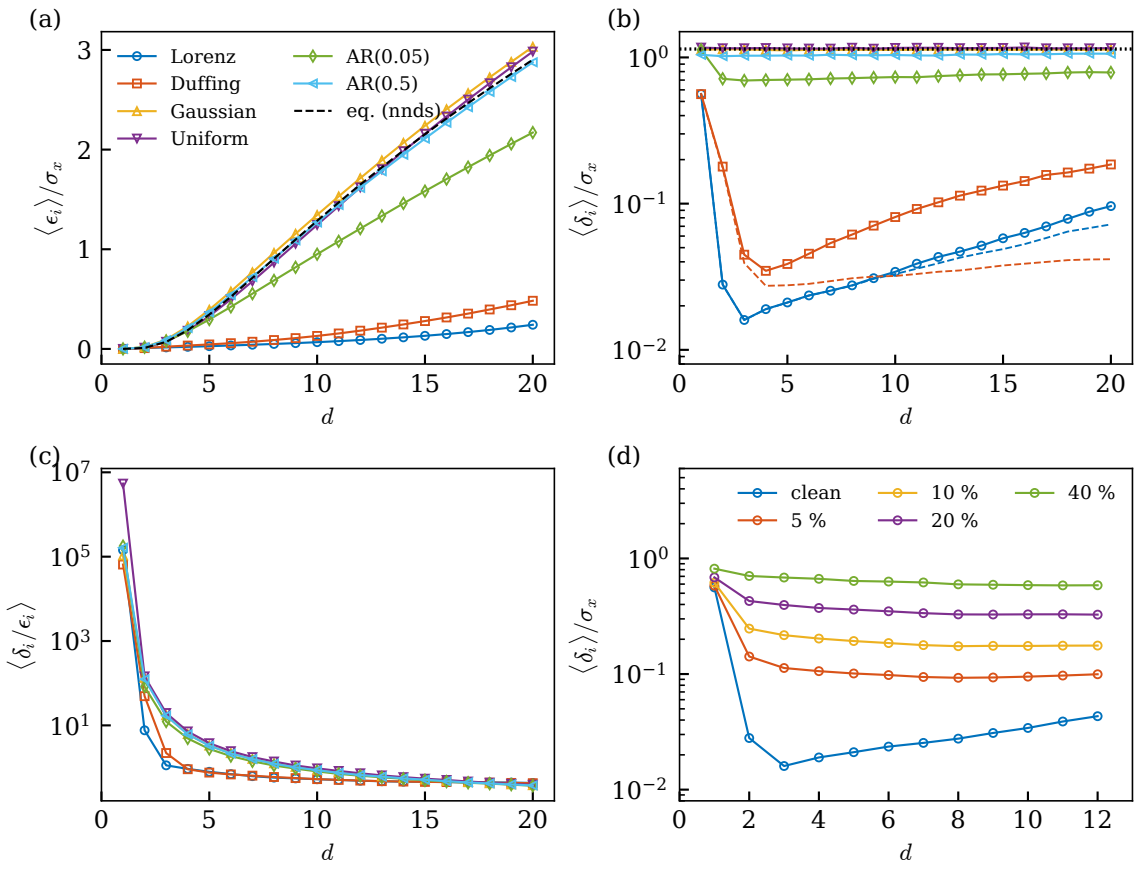


Figure 10.3: Nearest-neighbor statistics against embedding dimension for the six records of Table 10.1, Theiler window $w = 3\tau$. (a) Mean separation $\langle\epsilon_i(d)\rangle$ of the temporally uncorrelated nearest neighbor; dashed, the heuristic Eq. (10.16) for $N = 20{,}000$. (b) Mean appended separation $\langle\delta_i\rangle$; the dashed Lorenz and Duffing curves are computed without the Theiler window ($w = 0$); dotted, the white-noise levels $2/\sqrt{\pi}$ and $2/\sqrt{3}$ of Eqs. (10.18), and (10.19). (c) Mean ratio $\langle\delta_i/\epsilon_i\rangle$. (d) $\langle\delta_i\rangle$ for the Lorenz record with additive Gaussian noise of 0, 5, 10, 20 and 40 per cent of σ_x. Script `python/mkfig_nnd.py`.

coordinates $x_{i+k\tau}$, $k = d, \ldots$, sample those segments $k\tau\Delta t$ later. Their separation grows with the local divergence rate. The empirical form

$$\langle \epsilon_i \rangle \approx \epsilon_0 \left(e^{\lambda d \tau \Delta t} - 1 \right) , \qquad d \geq d_{\min} , \tag{10.17}$$

with ϵ_0 a scale set by the point density, fits the curves of Fig. 10.3(a) for $d \geq d_{\min}$. With the Theiler window the fitted parameters are $\epsilon_0 = 0.042\sigma_x$, $\lambda\tau\Delta t = 0.095$ for Lorenz ($d \geq 3$) and $\epsilon_0 = 0.081\sigma_x$, $\lambda\tau\Delta t = 0.098$ for Duffing ($d \geq 4$). Without the window the Lorenz rate falls to 0.068 and the Duffing curve becomes linear in d, with no exponential growth at all: temporally correlated neighbors stay close whatever d is and depress the apparent growth, severely so for Duffing. With $\tau\Delta t = 0.16$ and 1.4 the rates correspond to $\lambda \approx 0.6$ and 0.07 per time unit, below the largest Lyapunov exponents of Chapter 12; Eq. (10.17) is a description of the curves, not an exponent estimator. Its use is the comparison with noise. The exponential growth is much slower than the $\sqrt{d}\, N^{-1/d}$ growth of noise and does not overtake it until $d \sim 60$, far beyond any dimension of interest. The nearest-neighbor separation of a low-dimensional attractor stays well below that of noise at the same d: $0.24\sigma_x$ for Lorenz and $0.6\sigma_x$ for Duffing against $3\sigma_x$ for noise at $d = 20$. The AR records lie between: AR(0.5) is indistinguishable from noise, and AR(0.05) drifts below it as d grows, to $2.2\sigma_x$ at $d = 20$.

Now consider the appended-coordinate separation δ_i. For white noise the appended coordinates are independent of the neighbor relation, so δ_i has the distribution of $|x_i - x_j|$ for two arbitrary samples. For Gaussian data this is $\sqrt{2}\sigma_x\, \chi_1$,[1] with mean

$$\langle \delta_i \rangle = \mathbb{E}\left[\delta_i\right] = \frac{2\,\Gamma(1)}{\Gamma(1/2)}\,\sigma_x = \frac{2}{\sqrt{\pi}}\,\sigma_x \approx 1.128\,\sigma_x\,, \tag{10.18}$$

and for uniform data

$$\langle \delta_i \rangle = \frac{2}{\sqrt{3}}\,\sigma_x \approx 1.155\,\sigma_x\,. \tag{10.19}$$

For any uncorrelated record the mean appended separation is therefore slightly above σ_x, about 1.12–$1.16\,\sigma_x$, and independent of d. The same holds for colored noise once the lag exceeds the correlation time. For a deterministic record, by contrast, δ_i is small for a true neighbor pair.

Figure 10.3(b) confirms this. The Gaussian and uniform records sit at the predicted levels for all d. The Lorenz and Duffing records start at $0.56\,\sigma_x$ at $d = 1$, drop to minima of $0.016\,\sigma_x$ at $d = 3$ and $0.035\,\sigma_x$ at $d = 4$, and then rise slowly at the divergence rate of Eq. (10.17). Without the Theiler window (dashed) the minima are as deep but the rise is suppressed, to a flat $0.04\,\sigma_x$ for Duffing, because the temporally adjacent neighbor stays close at every d. The AR records are constant in d from $d = 2$ on: AR(0.5) at the noise level, AR(0.05) at $0.75\,\sigma_x$, below the noise level because the appended coordinate of a nearly deterministic linear oscillator is predictable from its neighbor. Of all the statistics of this section, the temporally decorrelated $\langle \delta_i \rangle$ gives the cleanest indication of the minimal embedding dimension for clean data and the widest separation between deterministic and stochastic records.

Figure 10.3(d) shows what noise does to it. With 5 per cent noise the minimum of $\langle \delta_i \rangle$ for Lorenz rises from $0.016\,\sigma_x$ to $0.09\,\sigma_x$ and the knee at $d = 3$ is still visible; with 10 per cent the curve is $0.17\,\sigma_x$ and flat from $d = 4$; with 20 and 40 per cent it is flat at 0.33 and $0.59\,\sigma_x$ from $d = 3$. The location of the knee survives, its depth does not. Figure 10.3(c) shows the ratio $\langle \delta_i / \epsilon_i \rangle$ that the original criterion tests. It falls from 10^5–10^7 at $d = 1$ to order one by $d \approx 10$ for every record, deterministic or not, because the denominator ϵ_i grows with d. A threshold of $r = 5$–10 therefore separates the records only for d below about ten, which is one way of seeing the bound Eq. (10.7).

[1] If $x_i \sim \mathcal{N}[0, \sigma_x]$ then $x_i - x_j \sim \mathcal{N}[0, \sqrt{2}\sigma_x]$ and $|x_i - x_j| / (\sqrt{2}\sigma_x) \sim \chi_1$.

Three conclusions carry forward. Temporal correlation must be removed; it weakens every statistic and misrepresents the AR records. The appended separation δ_i measured against σ_x separates deterministic from stochastic records at every d, but loses its minimum under noise. The ratio δ_i/ϵ_i locates the minimal dimension sharply at low d and is uninformative at high d. The two criteria are complementary, and the composite fraction below uses both.

10.4.4 The Composite Fraction

Combine the ratio criterion of Eq. (10.3) with the size criterion of the reliable fraction Eq. (10.13),

$$R_{nn}(d; r, s) \triangleq \text{Prob}\left\{(\delta_i > r\,\epsilon_i) \;\vee\; (\delta_i > s\,\sigma_x)\right\}\,, \tag{10.20}$$

the *composite fraction*. The first clause locates the minimal dimension at small d. The second holds the fraction at the noise level whenever the appended coordinates are unrelated to the neighbor relation: a fixed share of unrelated pairs has $\delta_i > s\sigma_x$ at every d. It thus prevents the collapse of Eq. (10.7). Neighbors are found with the Theiler window Eq. (10.12).

The composite fraction still counts pairs that are close by noise alone. Instead of following strands, as Section 10.4.2 did, look up k temporally uncorrelated neighbors of each point and choose among them the one whose trajectory segment stays closest to that of the reference point.

Composite-fraction FNN

Given the reconstruction at dimension d, thresholds r and s, Theiler window w, neighbor count k and strand half-length $l \le w/2$:

1. For each $\boldsymbol{x}_i$ find its k nearest neighbors $\{\boldsymbol{x}_{j(m)}\}_{m=1}^{k}$ subject to $|i - j(m)| > w$.
2. Select the neighbor whose strand of $2l+1$ points tracks the reference strand most closely relative to its central separation,

$$m^\star = \arg\min_m \frac{\sum_{q=-l}^{l} \left\|\boldsymbol{x}_{j(m)+q} - \boldsymbol{x}_{i+q}\right\|}{(2l+1)\left\|\boldsymbol{x}_{j(m)} - \boldsymbol{x}_i\right\|}\,, \qquad j = j(m^\star)\,. \tag{10.21}$$

   For $l = 0$ the ratio is identically one and $m^\star = 1$, the nearest neighbor.
3. Compute ϵ_i and δ_i from Eq. (10.1) with this j, and evaluate Eq. (10.20).

The fraction now depends on five parameters,

$$R_{nn} = R_{nn}(d; r, s, k, l)\,. \tag{10.22}$$

For clean data $k = 1$, $l = 0$ suffices. For noisy data $k = 10$ and $l = \tau$ are used below; the dependence on k and l has not been studied systematically and is left as an exercise. Appendix A.8 supplies the routine `fnn`, which returns ϵ_i and δ_i for every d together with the surrogate δ_i, and `fnnfrac`, which turns them into every fraction of this chapter.

Figure 10.4 shows the composite FNN fraction, with $s = 0.5$ and r swept in the upper half and $r = 4$ and s swept in the lower half. For the Lorenz and Duffing records it falls below 0.02 at $d = 3$ and 4 for every $r \ge 4$ and every $s \ge 0.3$, and to zero there for $r \ge 10$; it then rises slowly: to 0.03 for Lorenz and 0.10 for Duffing at $d = 20$, $s = 0.5$, which is the slow rise of $\langle\delta_i\rangle$ in Fig. 10.3(b) crossing the threshold. For white noise it never falls below the surrogate level, 0.72 at $s = 0.5$, and coincides with its surrogate at every d. For the AR records it is constant from $d \approx 4$

on: AR(0.5) at 0.71, indistinguishable from its surrogate, and AR(0.05) at 0.58–0.61 for $s = 0.5$, below its surrogate at 0.73, and below it for every other r and s as well. The gap is not evidence of determinism. It measures the linear predictability of the appended coordinate from the neighbor, which the simple surrogate destroys: a surrogate of the AR(0.05) record that preserves its power spectrum and amplitude distribution (Chapter 16, routine `irsurr`) gives $R_{nn}(d; 10, 0.5) = 0.58$ at every $d \geq 4$, exactly the data value, while the same surrogate of the Lorenz record stays at 0.6 where the data reach zero. The gap is also independent of the Theiler window (w from 5 to 100 samples gives the same 0.58), so it is not a temporal-correlation artifact. A record that separates from the simple surrogate but not from the spectrum-preserving one is linearly correlated noise.

Figure 10.5 shows what noise does to the fractions, on the Lorenz and Duffing records with 10 and 40 per cent additive Gaussian noise. The Kennel fraction (left column) loses its zero: at 10 per cent it reaches 0.04 at $d = 4$ and 0.005 at $d = 5$ for Lorenz, and at 40 per cent it floors at 0.03–0.04 from $d = 5$, after a transition that is spread over $d = 2$–5 instead of ending at $d = 3$. Its floor is low because a noisy pair that separates is usually also stretched beyond $a\sigma_x$ and is counted once. The composite fraction with $k = 1$ (middle column) at $r = 10$, $s = 1$ floors at the noise level, 0.01 at 10 per cent and 0.16 at 40 per cent, with the knee blurred over $d = 3$–5. With $k = 10$ and $l = \tau$ (right column) the fall is earlier and the floor lower, 0.09–0.10 at 40 per cent, and the knee is at $d = 3$ for both records at both noise levels. The surrogate sits at 0.5 throughout, since with $s = 1$ half of all unrelated appended coordinates exceed σ_x.

The strand selection is not a free improvement. Choosing among k candidates the one whose segment tracks the reference segment prefers a neighbor on the same sheet of the attractor whenever one exists, and in a folded projection such neighbors exist: on the *clean* records the $k = 10$ fraction at $d = 2$ is 0.002 for Lorenz and 0.02 for Duffing, where the $k = 1$ fraction is 0.07 and 0.50. The selection therefore biases the reading toward low d, and its knee should be confirmed against the $k = 1$ curve and against Cao's E_1. The measured record of Section 1.4.1, analyzed with the algorithm box implemented as stated (Section 10.6), shows the same one-dimension shift on data with no added noise, so the bias belongs to the selection rule and not to the noise. Additive measurement noise and dynamical noise are not distinguished by these statistics.

10.5 Spatial Decorrelation

Kennel and Abarbanel [111] propose a further step, intended to undo the effect of a too-small lag. With a small τ the reconstructed points bunch in a thin tube along the hyperdiagonal of the embedding space, and the FNN fraction is small whatever the dynamics. The step is a change of basis of the $(d + 1)$-dimensional reconstruction, and it has two parts: a whitening that rescales the tube, and a rotation that fixes which coordinates the last one is allowed to touch.

Let $\boldsymbol{Y} \in \mathbb{R}^{(d+1)\times M}$, $M = N - d\tau$, hold the reconstructed trajectory with the delay vector $\boldsymbol{x}_i \in \mathbb{R}^{d+1}$ as its i-th column, and let

$$\boldsymbol{Y} = \boldsymbol{U}\,\boldsymbol{\Sigma}\,\boldsymbol{V}^{\mathrm{T}} \tag{10.23}$$

be its singular value decomposition [74]. Define

$$\boldsymbol{T} = \sqrt{M}\,\boldsymbol{U}\,\boldsymbol{\Sigma}^{-1} \in \mathbb{R}^{(d+1)\times(d+1)}\,, \tag{10.24}$$

so that $\boldsymbol{T}^{\mathrm{T}}\boldsymbol{Y}$ has sample covariance $\boldsymbol{I}$: the coordinates are uncorrelated and each has unit variance. Whitening alone leaves the choice of axes inside that unit ball free, and the second part of the step

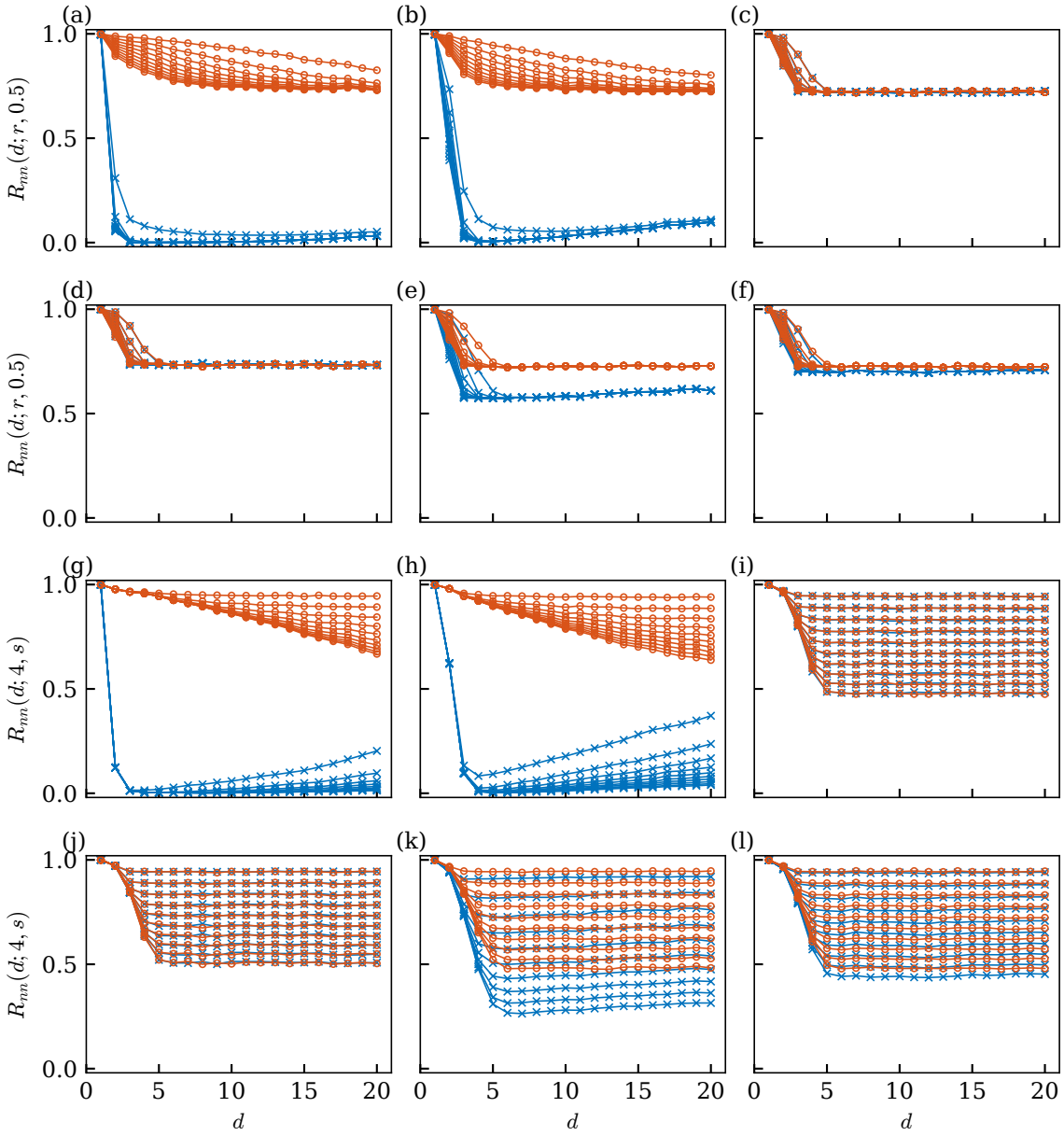


Figure 10.4: Composite FNN fraction Eq. (10.20), $k = 1$, $l = 0$, Theiler window $w = 3\tau$, for the six records of Table 10.1. (a)–(f): $s = 0.5$ and $r = 2, 4, \ldots, 20$ for Lorenz (a), Duffing (b), Gaussian (c), uniform (d), AR(0.05) (e) and AR(0.5) (f); (g)–(l): $r = 4$ and $s = 0.1, 0.2, \ldots, 1$ for the same records in the same order. The fraction decreases with r and with s. Lines with circles are the surrogate results. Script `python/mkfig_CFNN.py`.

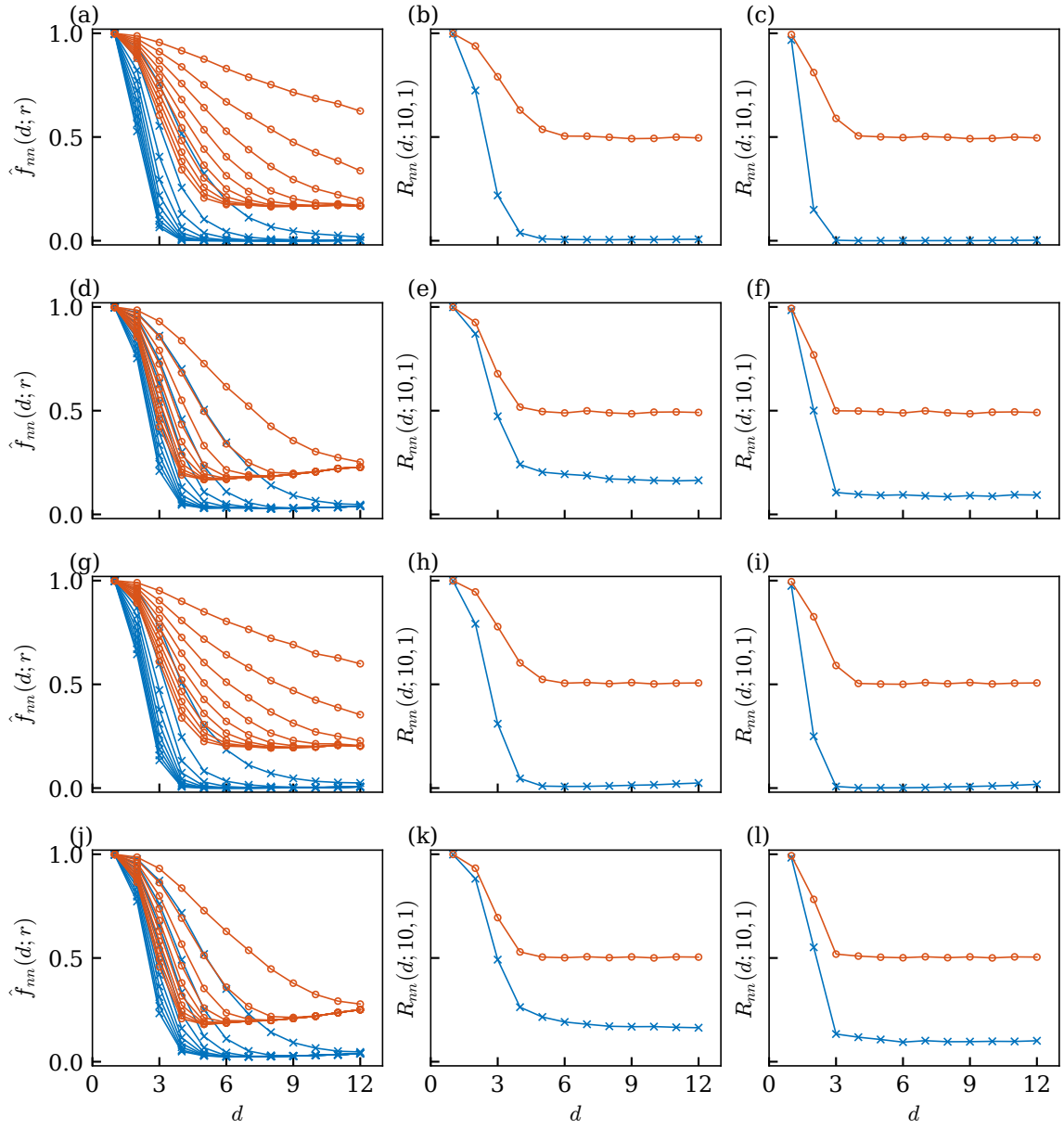


Figure 10.5: FNN fractions on noisy records: rows, the Lorenz record with 10 % (a–c) and 40 % (d–f) additive Gaussian noise and the Duffing record with 10 % (g–i) and 40 % (j–l); columns, the Kennel fraction Eq. (10.5) with $a = 2$, $r = 2, 4, \ldots, 20$ (left), the composite fraction Eq. (10.22) at $r = 10$, $s = 1$ with $k = 1$, $l = 0$ (middle) and with $k = 10$, $l = \tau$ (right). $N = 20{,}000$, Theiler window $w = 3\tau$ (Table 10.1). Lines with circles are the surrogate results. Script `python/mkfig_FNN_Noise.py`.

spends that freedom. Take $\boldsymbol{t} \in \mathbb{R}^{d+1}$, the last *row* of $\boldsymbol{T}$, and form the Householder matrix [74]

$$\boldsymbol{Q} = \boldsymbol{H}\left(\boldsymbol{t} - \|\boldsymbol{t}\|\, \boldsymbol{e}_{d+1}\right), \qquad \boldsymbol{H}(\boldsymbol{v}) = \boldsymbol{I} - 2\,\frac{\boldsymbol{v}\boldsymbol{v}^{\mathrm{T}}}{\boldsymbol{v}^{\mathrm{T}}\boldsymbol{v}}, \tag{10.25}$$

where $\boldsymbol{e}_{d+1}$ is the unit vector in the $(d+1)$ direction. By construction $\boldsymbol{t}\boldsymbol{Q} = \|\boldsymbol{t}\|\, \boldsymbol{e}_{d+1}^{\mathrm{T}}$, so the last row of $\boldsymbol{T}\boldsymbol{Q}$ is zero except in its last entry. The transformed phase space is

$$\bar{\boldsymbol{Y}} = (\boldsymbol{T}\boldsymbol{Q})^{\mathrm{T}}\, \boldsymbol{Y} = \boldsymbol{Q}^{\mathrm{T}}\, \sqrt{M}\, \boldsymbol{\Sigma}^{-1}\boldsymbol{U}^{\mathrm{T}}\boldsymbol{Y}\,. \tag{10.26}$$

Two conventions

The source paper stacks the delay vectors as the *rows* of an $M \times (d+1)$ matrix $\boldsymbol{A} = \boldsymbol{Y}^{\mathrm{T}}$ and writes the step as $\boldsymbol{B} = \boldsymbol{A}\boldsymbol{T}\boldsymbol{Q}$, its equations (A1)–(A5). Equation (10.26) is the same map written for column-stored trajectories, which is the convention of this book.

Two properties follow. First, $\boldsymbol{Q}$ is orthogonal, so the whitening survives it: $\bar{\boldsymbol{Y}}$ still has sample covariance $\boldsymbol{I}$, and the linear cross correlation the step exists to remove is gone at every d. Second, because the last row of $\boldsymbol{T}\boldsymbol{Q}$ vanishes except in its last entry, the last column of $(\boldsymbol{T}\boldsymbol{Q})^{\mathrm{T}}$ does too, and the first d rows of $\bar{\boldsymbol{Y}}$ are functions of the first d rows of $\boldsymbol{Y}$ alone. The appended coordinate $x_{n+d\tau}$ enters only the $(d+1)$-th transformed coordinate, which is what the FNN criteria assume when they call δ_i the error made by the shorter embedding.

The whitening is what does the work, and the rotation is what keeps it legitimate. This is worth saying because the second part is easy to mistake for the whole step. A reflector built from the last *column* of $\boldsymbol{U}$, applied to $\boldsymbol{U}^{\mathrm{T}}\boldsymbol{Y}$ with no rescaling, is a natural-looking construction and a useless one: the composite map is then orthogonal, it preserves every Euclidean distance between delay vectors, and a thin tube is a statement about absolute distances. Worse, such a map is not block-diagonal, so it mixes the appended coordinate into the first d and destroys the second property above while leaving $\epsilon_i^2 + \delta_i^2$ fixed.

The records separate the two readily. The Lorenz record at $\Delta t = 0.02$ has its average-mutual-information minimum at $\tau = 8$; setting $\tau = 1$ is the thin-tube regime the step addresses. Untransformed, the composite fraction reads $R_{nn} = 0.012$ at $d = 2$ and zero at $d = 3$, so a reader applying the rule of Section 10.6 reports $d_E = 2$ and is wrong by one. The failure is real. Equation (10.26) repairs it: 0.087 at $d = 2$ and 0.002 at $d = 3$, hence $d_E = 3$, and the reading stays at 3 for $\tau = 2, 3, 5$ and 8 as well. The orthogonal-rotation-only variant reads 0.011 at $d = 2$ and zero at $d = 3$, the same wrong answer as no transform at all, and at $\tau = 5$ and 8 it drifts upward to $d_E = 4$ on a record where the untransformed fraction is already right (`python/chk_spatdecorr.py`).

Two things to keep in view. The step is not orthogonal, so it changes every neighbor statistic in the chapter: the fractions it produces are on a different scale from the untransformed ones, and the thresholds r and s have to be set again against a surrogate. And it divides by the singular values, so on a record where the smallest of them is set by measurement noise rather than by the geometry, it amplifies that noise into the neighbor search. Kennel and Abarbanel say as much themselves, noting that successive whitened coordinates behave like successive derivatives and that the last one is the noisiest. Report the singular value spectrum of $\boldsymbol{Y}$ alongside any dimension estimated this way.

For the test records of this chapter the lag is its AMI value, and the step was not applied in any figure of this chapter; the routine `fnn` of Appendix A.8 does not include it. A reader with a lag fixed by hardware, and therefore possibly too small, should treat it as an option to test rather than a default.

10.6 Procedure

Procedure: choosing d for a measured record

Given a scalar record $\{x_n\}_{n=1}^{N}$ with lag τ from Chapter 9:

1. *Fix the Theiler window.* Set w to two to four times the first minimum of the average mutual information, in samples. Record w; it must scale with the sampling rate.

2. *Fix the reachable range.* Compute $d_{\max} = \ln N / \ln r$ for the r you will use, Eq. (10.7). Sweep d from 1 to at most $d_{\max} + 2$; results beyond $d_{\max}$ are shown only to demonstrate the collapse.

3. *Compute the composite fraction* $R_{nn}(d; r, s)$ of Eq. (10.20) with $k = 1$, $l = 0$, for $r \in \{2, 4, \ldots, 20\}$ at $s = 0.5$, and for $s \in \{0.1, \ldots, 2\}$ at $r = 4$ or 10. Compute the surrogate fraction alongside (Section 10.1.1).

4. *Compute Cao's* $E_1(d)$ *and* $E_2(d)$, Eqs. (10.10), and (10.11), with the same τ and w.

5. *Test for determinism first.* If the data and surrogate fractions coincide, or $E_2(d) \approx 1$ for all d, the record is stochastic at this N and no embedding dimension is defined. Report that.

6. *Read d.* Take the smallest d at which R_{nn} reaches its floor for all $r \geq 4$ and $s \geq 0.3$, and confirm against the plateau of E_1. If the floor is not zero the record is noisy: read the knee, and repeat with $k = 10$, $l = \tau$ (Section 10.4.4), remembering that the selection biases toward low d. If the two methods disagree by one, take the larger.

7. *Check the delay.* Repeat step 3 at $\tau/2$ and 2τ. A dimension that moves by more than one is not yet an estimate of the attractor; revisit Chapter 9.

Report: N, Δt, τ, w, the ranges of r and s, k and l, the value of d, the floor of the fraction at that d, and $d_{\max}$. A dimension without its floor and its $d_{\max}$ cannot be judged by another reader.

Worked example: the Lorenz record with 20 per cent noise

Figure 10.6 applies the procedure to the Lorenz x record of Table 10.1 with additive Gaussian noise of 20 per cent of σ_x: $N = 20{,}000$, $\Delta t = 0.02$, $\tau = 8$ (first AMI minimum of the noisy record as well), $w = 24$. Step 2: $d_{\max} = \ln N / \ln 10 = 4.3$, so the sweep to $d = 12$ is shown but only $d \le 5$ is read. Step 3, panel (a): with $k = 1$ the fraction at $r = 10$, $s = 0.5$ is 1.00, 0.81, 0.39, 0.27, 0.24 at $d = 1, \ldots, 5$ and floors at 0.19; the simple surrogate floors at 0.73 and the spectrum-preserving surrogate (dashed) at 0.62. The data separate from both, so the record is not linear noise (step 5). Cao's E_2 in panel (c) is 0.63, 0.92, 0.94, 0.97 at $d = 1, \ldots, 4$ and one from $d = 8$, a weak but consistent departure; E_1 rises from 0.16 to 0.96 without a plateau, which is the noise behavior of Section 10.3 and gives no reading. Step 6: the floor of R_{nn} is not zero, so the knee is read: the largest drop is between $d = 2$ and 3 and the curve is within 0.03 of its floor from $d = 5$; with $k = 10$, $l = \tau$ (panel (b)) the fraction is 0.98, 0.33, 0.14, 0.12 and flat from $d = 4$. Step 7, panel (d): at $\tau = 4$ and 16 the $k = 10$ curves have the same knee at $d = 3$ with floors 0.09 and 0.17–0.20, so the reading does not move with the delay. Report: $N = 20{,}000$, $\Delta t = 0.02$, $\tau = 8$, $w = 24$, $r \in \{2, \ldots, 20\}$, $s = 0.5$, $k = 1$ and 10, $l = 0$ and 8; $d = 3$ from the knee, with $d = 4$ where the floor is reached; floor 0.27 ($k = 1$) and 0.12 ($k = 10$) at $d = 4$ against 0.73 for the surrogate; $d_{\max} = 4.3$; E_1 without plateau, $E_2 < 1$ for $d \le 6$. The clean record gives $d = 3$ with floor zero (Fig. 10.4(a)). Noise reduction (Chapter 14) before re-embedding is indicated at this noise level.

The estimate is biased. A short record biases every FNN statistic toward low d, Eq. (10.7). Noise biases the zero-crossing reading toward high d and blurs the knee. Temporal correlation biases toward low d, and so does the strand selection among $k > 1$ neighbors. These have known signs, and the report above lets a reader see which applies. Where a measured record is analyzed the same report should accompany it. The rest of this section does that for the measured record of Section 1.4.1.

Worked example: the two-well oscillator record

The record of Section 1.4.1 is $N = 96{,}000$ strain samples at 16 samples per forcing period, with $\tau = 5$ samples from Chapter 9. *Step 1*: $w = 32$ samples, two forcing periods and 6.4τ. This is above the rule of the procedure box, and is used so that a neighbor comes from a different drive cycle. *Step 2*: $d_{\max} = \ln N / \ln 10 = 4.98$ at $r = 10$ and 8.27 at $r = 4$. The sweep runs to $d = 8$, and any reading above $d = 5$ must come from $r \le 4$. *Steps 3 and 4*, Fig. 10.7(a,b,c): the Kennel and composite fractions agree to three digits, because $\langle \epsilon_i \rangle$ never exceeds $0.06\,\sigma_x$ and the size criterion never fires. At $r = 10$, $s = 0.5$ both are 1.000, 0.759, 0.173, 0.0016, 0.0006, 0.0016 for $d = 1, \ldots, 6$. Cao gives $E_1 = 0.02$, 0.07, 0.14, 0.93, 0.98, 0.98 and $E_2 = 0.43$, 0.38, 0.20, 1.05, 1.15, 1.10. *Step 5*: the simple surrogate floors at 0.85 and the spectrum-preserving surrogate (`irsurr`, 20 iterations) at 0.53, both far above the data. The surrogate E_2 stays within two per cent of one where the data E_2 does not. The record is not linear noise. *Step 6*: R_{nn} reaches its floor at $d = 4$, where it is below 0.013 for every $r \ge 4$ and every $s \ge 0.3$, and E_1 plateaus from $d = 4$. Both readings are $d_E = 4$. The Kennel fraction is exactly zero from $d = 5$. *Step 7*: at $\tau = 3$ and $\tau = 10$ the reading does not move, with $R_{nn}(4; 10, 0.5) = 5 \times 10^{-5}$ and 0.037. The two halves of the record give 0.0010 and 0.0006 and the same E_1 plateau. *Report*: $N = 96{,}000$; $\tau = 5$ and $w = 32$ samples; $r \in \{2, \ldots, 20\}$, $s \in \{0.1, \ldots, 1\}$; $k = 1$ and 10, $l = 0$ and 5; $d_E = 4$; floor $R_{nn}(4; 10, 0.5) = 0.0016$ against 0.85 and 0.53 for the two surrogates; $d_{\max} = 4.98$ at $r = 10$.

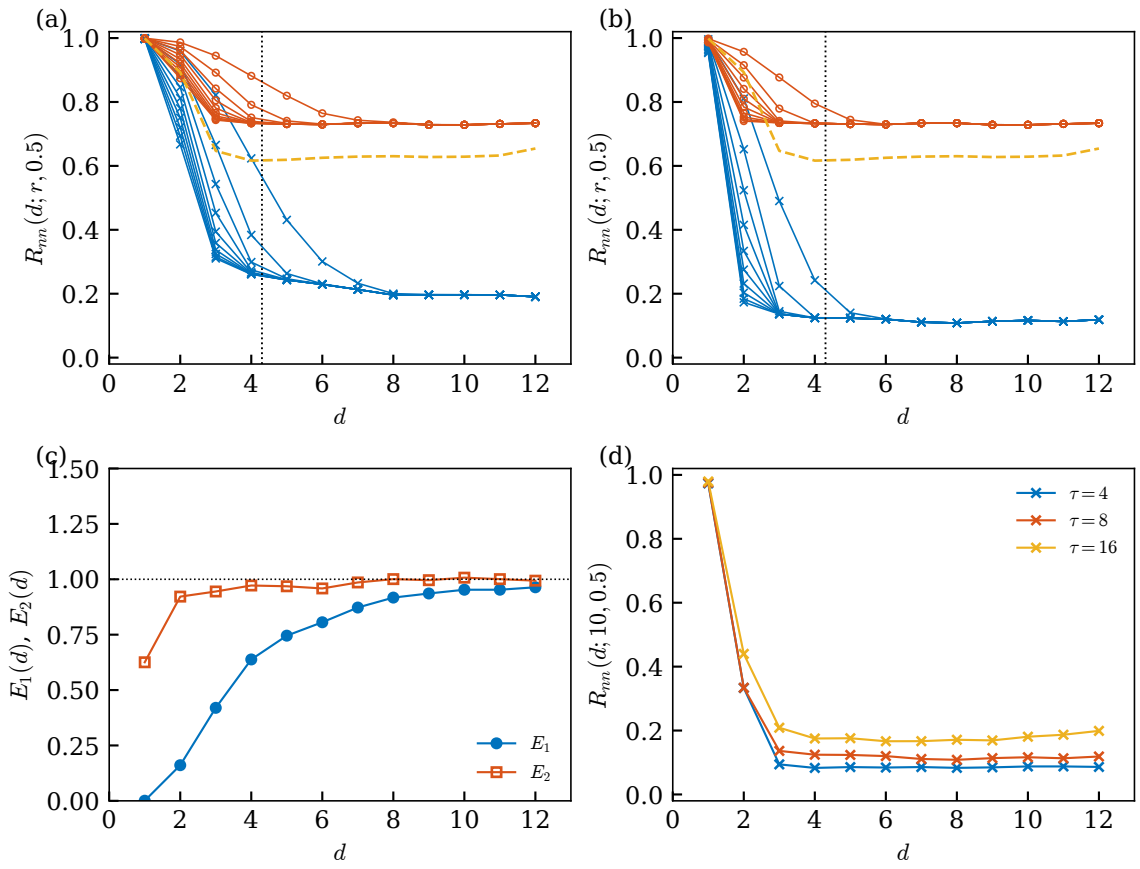


Figure 10.6: The procedure applied to the Lorenz x record with 20 % additive Gaussian noise (N = 20,000, Δt = 0.02, τ = 8, w = 24). (a) Composite fraction $R_{nn}(d; r, 0.5)$, k = 1, l = 0, $r = 2, 4, \ldots, 20$ (crosses), the simple surrogate of Section 10.1.1 (circles) and, at r = 10, the surrogate of Chapter 16 that preserves the power spectrum and amplitude distribution of the whole record (`irsurr`, 20 iterations; dashed); the dotted vertical line is $d_{\max} = \ln N / \ln 10 = 4.3$. (b) The same with $k = 10$, $l = \tau$. (c) Cao's E_1 and E_2 with the same τ and w. (d) Delay check: $R_{nn}(d; 10, 0.5)$ with $k = 10$, $l = \tau$ at τ = 4, 8 and 16, w = 24 fixed. Script `python/mkfig_edworked.py`.

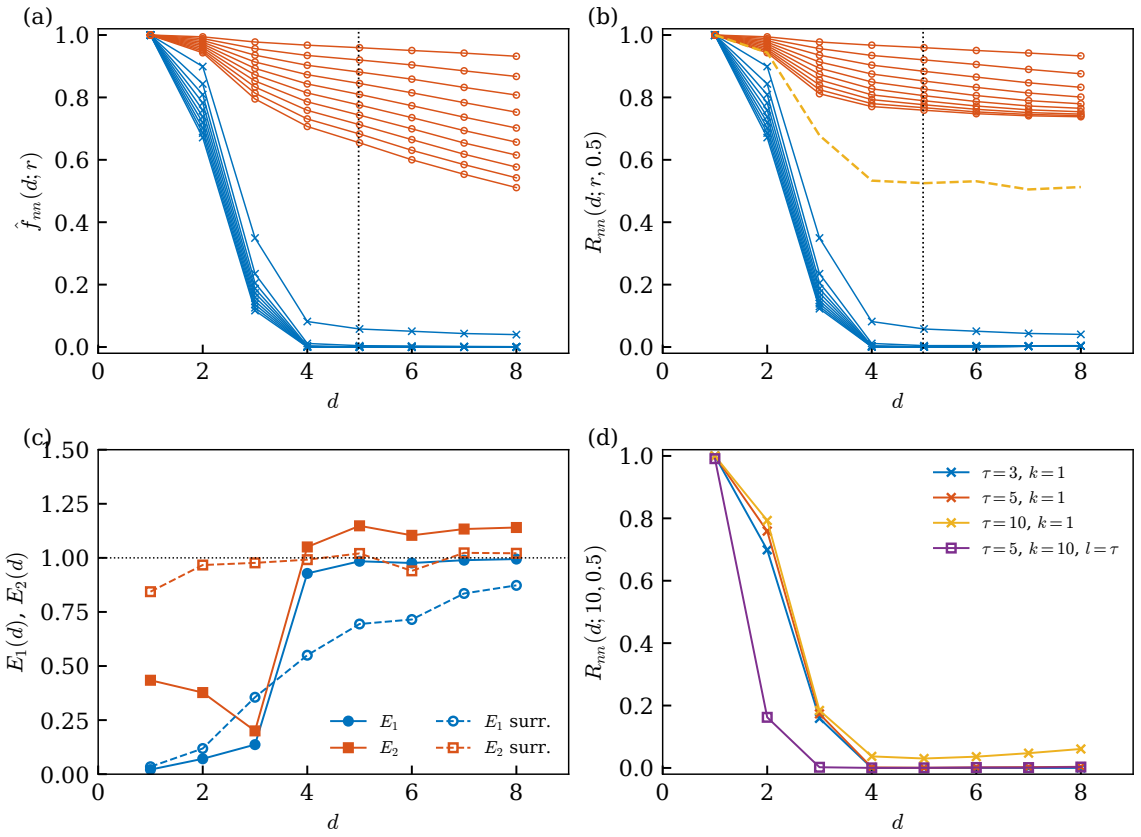


Figure 10.7: The procedure applied to the two-well oscillator record of Section 1.4.1 ($N = 96{,}000$ samples, 16 samples per forcing period, $\tau = 5$ samples, Theiler window $w = 32$ samples). (a) Kennel fraction $\hat{f}_{nn}(d;r)$, Eq. (10.5) with $a = 2$, for $r = 2, 4, \ldots, 20$ (crosses) with the simple surrogate of Section 10.1.1 (circles). (b) Composite fraction $R_{nn}(d;r,0.5)$, Eq. (10.20), $k = 1$, $l = 0$, same r family and surrogate, with the surrogate of Chapter 16 that preserves the amplitude distribution and power spectrum of the whole record (`irsurr`, 20 iterations) at $r = 10$ (dashed). The dotted vertical line in (a) and (b) is $d_{\max} = \ln N / \ln 10 = 4.98$. (c) Cao's E_1 and E_2 for the record (filled) and for the same `irsurr` surrogate (open), maximum norm, same τ and w. (d) Delay and strand checks: $R_{nn}(d;10,0.5)$ with $k = 1$ at $\tau = 3$, 5 and 10, $w = 32$ fixed, and with $k = 10$, $l = \tau$ at $\tau = 5$. Script `python/mkfig_edtwowell.py`.

Which of the three biases acts on this record follows from the same run. Oversampling does not. Repeating step 3 with $w = 0$ changes $R_{nn}(4;10,0.5)$ from 0.0016 to 0.0014, and the mean separations by under one per cent. At 16 samples per forcing period the temporal neighbor is rarely the nearest one. Noise is small but not absent: the quantization step 10/65536 V is $10^{-4}\sigma_x$, and the floor of R_{nn} is 6×10^{-4} rather than zero. Record length binds only above $d = 5$, where at $r = 10$ the record cannot test at all. The strand selection biases toward low d here as it does on the clean simulated records. With $k = 10$ and $l = \tau$, Fig. 10.7(d), the fraction is 0.162 at $d = 2$ and 0.0023 at $d = 3$, so its knee sits at $d = 3$. That is one dimension below the $k = 1$ reading, on a record with no added noise, and it should not be used here.

The record also settles a tension that Section 1.4.1 raised and left to this chapter. The Sauer–Yorke–Casdagli condition is $d > 2D_0$, with D_0 the upper box-counting dimension of the attractor.

The record supplies no estimate of D_0. It supplies $\hat{D}_2 = 2.68$ at finite scales (Chapter 11), and $D_2 \leq D_0$. The theorem therefore asks for at least $d = 6$, possibly more, and the data certify its requirement at no d. The false-neighbor count is already zero at $d = 5$ and at its floor at $d = 4$. These answer different questions. The condition is sufficient and generic. It guarantees that almost every delay map of any set of that dimension is an embedding, the worst such set included. The false-neighbor count is a measurement on one attractor, with one observable, at one record length. It says only that no self-intersection is resolvable at the resolution the data supply. It cannot say more. At $r = 10$ this record supports $d \leq 4.98$ by Eq. (10.7), so every dimension the bound could ask for lies outside the range the test can examine. What the count does support is a working reconstruction at the resolution of the record. Absence of resolvable false neighbors is not a proof of injectivity, and the bound is not a claim that $d = 4$ fails.

What to use downstream is a third question, and its cost side is measurable. Going from $d = 4$ to $d = 6$ raises the mean nearest-neighbor separation from $0.028\,\sigma_x$ to $0.042\,\sigma_x$, half as far again, and the mean appended separation from $0.024\,\sigma_x$ to $0.029\,\sigma_x$. Neighborhoods that carry the local geometry become that much less local, for no gain this record can display. The working choice is therefore $d = 4$, with every quantity derived from it reported over a range of d that reaches the smallest value the theorem could allow. Chapter 11 does this and finds D_2 between 2.69 and 2.62 for $d = 4$ to 8. Chapter 12 finds λ_1 between 0.62 and 0.60 nats per forcing period for $d = 4$ to 7. Neither estimate turns on whether the bound is met, which is the evidence that the tension is not consequential here. Had either moved between $d = 4$ and $d = 8$, the choice of d would have been doing work, and the larger d would be the safer report.

Pitfall: three ways to get the wrong dimension

Noise. Additive noise floors every fraction above zero, so the "first zero" does not exist. Reading it anyway gives d too large, or the end of the sweep. Read the knee, use the strand selection with $k > 1$, and compare with the surrogate floor. If the noise level exceeds about 20 per cent of σ_x, filter first (Chapter 14) and re-embed the filtered record with the same τ.

Short records. Beyond $d_{\max} = \ln N / \ln r$ the nearest neighbor of every point is far away and the ratio criterion passes everything. The fraction falls to zero for noise as readily as for an attractor. A record of 10^4 points at $r = 10$ supports $d \leq 4$ and nothing more; a reported $d = 7$ from such a record is a statement about N.

Oversampling. For a densely sampled flow the nearest neighbor of $\boldsymbol{x}_i$ is $\boldsymbol{x}_{i\pm1}$, a true neighbor at every d. Without a Theiler window the fraction is near zero from $d = 1$ or 2 and the dimension is underestimated. Set w in samples from the AMI minimum, and rescale it whenever the sampling rate changes; a window carried unchanged from one sampling rate to another is wrong at the new rate (Problem 10.3).

10.7 Discussion

The embedding dimension is the first quantity in this book that is estimated from a reconstructed trajectory, and it shows the pattern every later estimate follows. A geometric criterion that is exact on an infinite clean trajectory becomes a statistic on a finite noisy one. The statistic has a bias whose sign depends on which failure dominates: too few points, too much noise, too dense a sampling. The remedy in each case is the same. Understand the statistic for the null case, here white and colored noise, and require the data to separate from it.

The Kennel–Brown–Abarbanel test remains the reference method because its two criteria have clear rationales: a smooth map cannot stretch a small separation tenfold, and a neighbor a whole attractor away is no neighbor. Cao's statistics replace the thresholds by ratios of means and add an explicit determinism test, and should be reported alongside. The analysis of Hegger and Kantz supplies the record-length bound $N \geq r^d$ that governs both. The statistical view of Section 10.4 explains the shape of every fraction from two distributions: the nearest-neighbor separation, which grows as $\sqrt{d}\,N^{-1/d}$ for noise and exponentially in d for an attractor, and the appended separation, which sits at $1.13\,\sigma_x$ for noise and near zero for true neighbors. The composite fraction uses both. It holds noise at the surrogate level at every d, reads $d = 3$ and 4 on the clean Lorenz and Duffing records, and keeps a knee at $d = 3$ on the Lorenz record with 40 per cent additive noise, at the price of a floor that must be reported with the dimension. The strand selection among k neighbors lowers that floor but also lowers the fraction below the embedding dimension, so it is a check and not a replacement; its parameters k and l are fixed by trial in this chapter and their systematic choice is open. A gap between the data and the simple surrogate of Section 10.1.1 is produced by linear correlation alone, as the AR(0.05) record shows; only a surrogate that preserves the linear correlation (Chapter 16) tests for more. The spatial decorrelation of Section 10.5 repairs a dimension read too small at a too-short lag, but it whitens, so its thresholds must be reset against a surrogate and its noise amplification reported with the estimate.

Two further points connect to later chapters. The embedding dimension found here is a property of the record at its length, not of the attractor: Chapter 11 estimates the dimension of the same records at just above 2 for Lorenz and 2.16 for Duffing, so $d = 3$ and $d = 4$ satisfy $d > D_2$ but lie below $2D_2$, and therefore below the sufficient $d > 2D_F$, since $D_2 \leq D_F$. And a separation from the simple surrogate is evidence of predictability, not of determinism: the weakly stochastic AR record produces one too. The surrogate comparison here is the first instance of the hypothesis-testing view that Chapter 16 makes systematic.

Problems

Problem 10.1

Computational

Create an artificial time series by integrating the Chua's circuit [136, 137] by Matsumoto *et al.* (1985):

$$\dot{x} = \alpha \left(y - x + bx + \frac{1}{2}(a - b)\left(|x + 1| - |x - 1|\right) \right) \tag{10.27}$$

$$\dot{y} = x - y + z \tag{10.28}$$

$$\dot{z} = -\beta y \tag{10.29}$$

using these parameters: $\alpha = 9$, $\beta = 100/7$, $a = 8/7$, and $b = 5/7$. Start the simulation using the following initial conditions: $x(0) = 0$, $y(0) = 0$, and $z(0) = 0.6$. In particular, using the x and y time series:

1. Make sure you use an appropriate sampling frequency or sampling period to generate the trajectory (check using power spectrum and auto-correlation), and record at least 60,000 evenly sampled points.

2. Determine the appropriate delay for the delay coordinate reconstruction and verify it visually.

3. Use false nearest neighbors method to determine the minimally necessary embedding dimension using composite FNN fraction algorithms.

4. Now add some Gaussian noise to the data and do the same calculations for different signal-to-noise ratios.

5. Interpret your results, and report the quantities listed in the procedure of Section 10.6.

The routines `fnn`, `fnnfrac` and `cao` of Appendix A.8 compute every statistic of this chapter; the scripts `python/mkfig_CFNN.py` and `python/mkfig_edworked.py` show how the figures of this chapter are built from them.

Problem 10.2

Computational

Problem 10.1 applied the composite FNN algorithms to Chua data. This problem separates the three fractions defined in this chapter and asks what distinguishes them.

1. Implement all three: the original fraction $f_{nn}(d;r)$ of Eq. (10.3), the Kennel fraction $\hat{f}_{nn}(d;r)$ of Eq. (10.5), and the TISEAN fraction $\tilde{f}_{nn}(d;r)$ of Eq. (10.8).

2. Apply all three to a Lorenz record and to a Gaussian white noise record of the same length, sweeping r and d. Present the results as the chapter does.

3. For the noise record, verify the scaling argument of Section 10.2 directly: show that the original fraction falls toward zero as N decreases at fixed r, and locate the $N \simeq r^d$ crossover for $d = 3$ and $r = 10$.

4. Now hold N fixed at a value *below* r^d — equivalently, sweep d past the bound of Eq. (10.7) — and observe that the original fraction reports a low embedding dimension for pure noise. This is the failure the Kennel and TISEAN fractions are designed to repair — confirm that each does. Mark $\ln N / \ln r$ on your plots against d and check that the collapse sets in where the bound says it should.

5. For the TISEAN fraction, plot the *denominator* of Eq. (10.8) against d. At what d does the number of admissible neighbors collapse, and what does that imply about the largest usable d for your record?

Problem 10.3

Computational

The reliable fraction of Eq. (10.13) depends on a Theiler window. This problem asks how much.

1. Implement Eqs. (10.13), and (10.12). Apply it to a finely sampled Lorenz record with $w = 0$, and then with w equal to one, two, three and four times the first minimum of the AMI.

2. Plot the resulting fraction against d for each w. How large must w be before the answer stabilizes? Compare with the rule of thumb quoted in this chapter.

3. Now oversample: halve the sampling interval and repeat with the *same* w in samples. Explain what happens, and state the correct way to hold w fixed when the sampling rate changes.

4. Generate the simple surrogates described in Section 10.1.1 — for each d-dimensional NN pair, draw the $(d+1)$-th coordinate at random from the data — and overlay the surrogate fractions. For which of your records do the data and surrogates separate?

Problem 10.4

Computational

Cao's method of Section 10.3 is threshold-free. This problem asks what it depends on instead.

1. Implement $E_1(d)$ and $E_2(d)$, Eqs. (10.10), and (10.11), with the maximum norm and a Theiler window. Reproduce Fig. 10.2 for the Lorenz record and for Gaussian noise.

2. Repeat for the Hénon map with $\tau = 1$ and for the two-well Duffing record. State the d at which E_1 stops changing, and how you decided that it had.

3. Add Gaussian noise of 5, 10 and 20 per cent of σ_x to the Lorenz record. Plot E_1 and E_2 against d for each level. At which level does E_2 become indistinguishable from one, and what does the method then report?

4. Shorten the clean Lorenz record to $N = 2000$, 1000 and 500 points and repeat part (1). Relate what you see to the bound Eq. (10.7), and state whether the bias is toward high or low d.

5. Compare the readings of parts (1)–(4) with the composite fraction Eq. (10.20) on the same records. Where do the two methods disagree, and which would you report?

Problem 10.5

Core

The noise statistics of Sections 10.2 and 10.4.3, derived and checked.

1. Let U_1, U_2 be independent and uniform on $[0, 1]$. Show that $\mathrm{Prob}\{|U_1 - U_2| > c\} = (1 - c)^2$ for $0 \le c \le 1$, the starting point of Eq. (10.6). Show also that $\mathrm{E}[|U_1 - U_2|] = 1/3$, and hence that the mean appended separation of uniform noise is $\mathrm{E}[\delta_i] = (2/\sqrt{3})\,\sigma_x$, Eq. (10.19). Derive the Gaussian value $2\sigma_x/\sqrt{\pi}$ of Eq. (10.18) from the density of $|x_i - x_j|$.

2. Check all three numerically with 2×10^5 samples. You should reproduce $(1 - c)^2$ at $c = 0.3$ and the two means to three digits.

3. Draw N points uniformly in the unit cube of dimension d, for $N = 1000, 2000, \ldots, 16000$ and $d = 2, 3, 5$, and compute the mean nearest-neighbor separation $\langle \epsilon_i \rangle$ with a kd-tree. Fit the slope of $\ln\langle \epsilon_i \rangle$ against $\ln N$. You should find -0.50, -0.34 and -0.21, confirming the scaling $N^{-1/d}$ that underlies the bound Eq. (10.7).

4. Combine parts (1) and (3): for uniform noise with $N = 10^4$ points, at what d does the typical nearest-neighbor separation reach $1/r$ for $r = 10$, so that the original fraction Eq. (10.3) collapses? Compare with $d_{\max} = \ln N / \ln r$ and with what Problem 10.2 found empirically.

Problem 10.6

needs Ch. 11, 15 Experimental

The two-well oscillator record of Section 1.4.1, whose embedding dimension is worked out in Section 10.6. This problem asks what that reading depends on.

1. Reproduce the report of the worked example: $R_{nn}(d; r, 0.5)$ with $k = 1$, $l = 0$ and its two surrogates, and Cao's E_1 and E_2, at $\tau = 5$ and $w = 32$ samples. Confirm $d_E = 4$ and state the floor of R_{nn} at that d.

2. Repeat part (1) on the first $N = 48{,}000$, 12,000 and 3,000 samples, with τ and w unchanged. Compute $d_{\max} = \ln N / \ln 10$ for each. At which length does the reading first move, and in which direction does Eq. (10.7) say it must move?

3. Subsample the record by two, so that it carries eight samples per forcing period. Recompute the first minimum of the average mutual information, rescale τ and w, and repeat part (1). Then repeat it once more with τ and w carried over unchanged in samples, and explain the difference in terms of the oversampling pitfall.

4. Chapter 11 gives $\hat{D}_2 = 2.68$ for this record. Explain why this places the sufficient condition $d > 2D_0$ at $d \geq 6$ but does not certify $d = 6$. Embed at $d = 4$ and at $d = 6$ and compare the mean nearest-neighbor separation, the mean appended separation and the number of delay vectors available. State what is bought and what is paid for by the two extra coordinates, and which d you would use for a local model of Chapter 15.

11
Dimension Theory

We are used to the notion of *dimension* from *vector spaces*: dimension of a vector space V is the minimum number of independent bases vectors needed to *span* V. Therefore, a *point* has dim $= 0$; a *line* has dim $= 1$; a plane has dim $= 2$, and etc. In other words, *dimension* is the number of independent coordinates needed to specify a point in a "space." However, there is a problem: in 1877–1878 papers, Georg Cantor (1845–1918) showed that there is a one-to-one correspondence between $\mathbb{R}^1$ and $\mathbb{R}^n$ (i.e., they have the same *cardinality*), and Giuseppe Peano actually constructed a continuous *surjection* of the interval $[0, 1]$ onto the square $[0, 1]^2$, a space-filling curve (see Fig. 11.1 for the first three iterations in the construction of Peano curve).

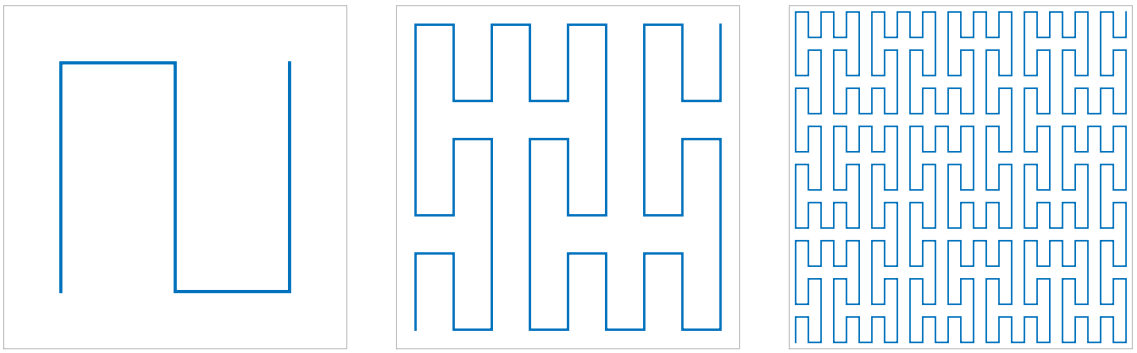


Figure 11.1: The first three iterations of Peano's construction, whose limit is a space-filling curve. Iteration k visits the $3^k \times 3^k$ cell centers of the square; it is the serpentine of nine reflected copies of iteration $k-1$. Script `python/mkfig_peano.py`.

Then, the question is, do $\mathbb{R}^1$ and $\mathbb{R}^2$ have the same dimension? Mandelbrot [131] is the standard general account of the fractal viewpoint that resolves the tension, and Theiler [205] reviews what can and cannot be estimated from data.

The resolution is to define dimension by covering rather than by coordinates. This is the *topological*, or *Lebesgue covering*, definition:

Topological Dimension (i.e., Lebesgue Covering Dimension)

A set $\mathcal{X}$ has Lebesgue covering dimension or topological dimension D if every open cover $\mathcal{C}$ of $\mathcal{X}$ has a refinement $\mathcal{C}^1$ such that every point $x \in \mathcal{X}$ is contained in *at most* $D+1$ sets in $\mathcal{C}^1$, and D is the *minimum* such integer.

It is convenient to read the covering sets $\mathcal{C}$ as open balls of radius r centered at x,

$$\mathcal{B}(x,r) = \{y \in \mathcal{X} \,|\, d(x,y) < r\} \ , \tag{11.1}$$

although topology itself defines open sets without any distance function. Read this way, the covering dimension reproduces the usual notion: for $\mathbb{R}^m$ it gives $D = m$ (see Figs. 11.2 and 11.3). What it

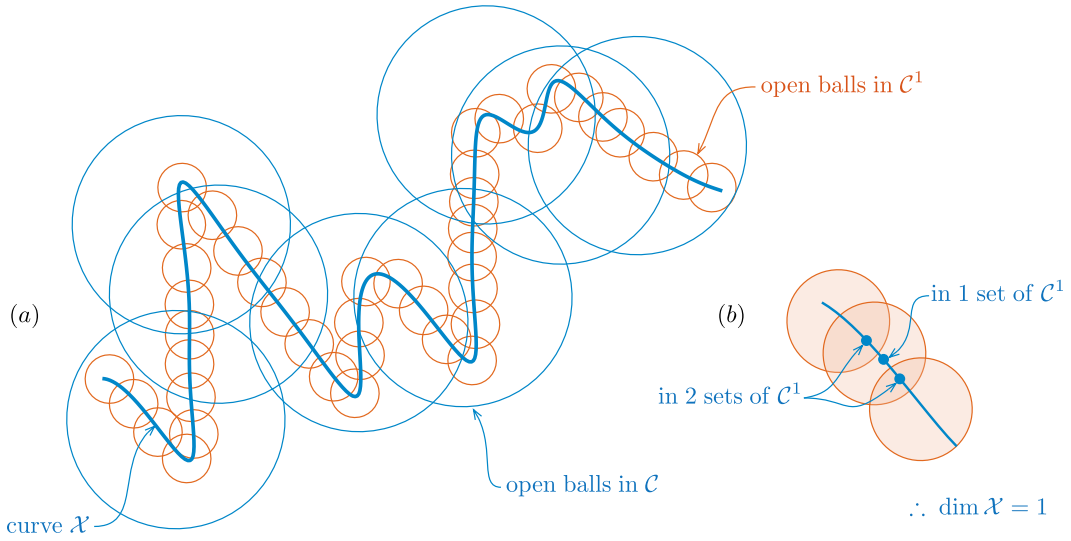


Figure 11.2: For a curve $\mathcal{X}$, refinement cover $\mathcal{C}^1$ is such that every point $x \in \mathcal{X}$ is contained in *at most* 2 sets in $\mathcal{C}^1$, therefore $\dim \mathcal{X} = 1$.

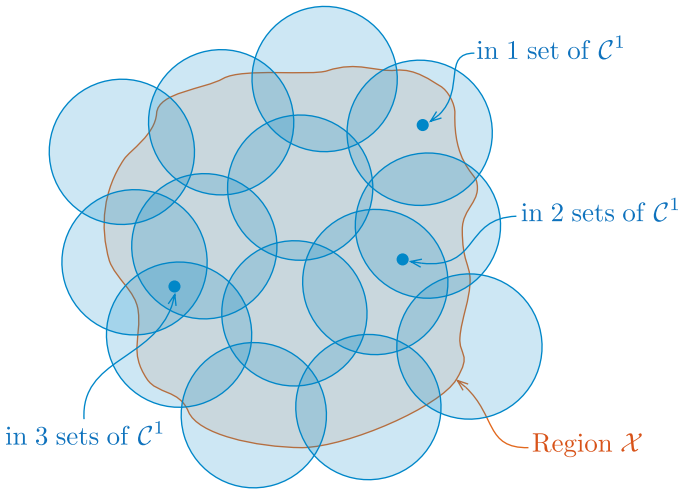


Figure 11.3: For a region $\mathcal{X}$, refinement cover $\mathcal{C}^1$ is such that every point $x \in \mathcal{X}$ is contained in *at most* 3 sets in $\mathcal{C}^1$, therefore $\dim \mathcal{X} = 2$.

does not reproduce is anything about dynamics. The covering dimension, also called the *Brouwer dimension*, is a property of the set alone. Our set $\mathcal{A}$ is generated by an orbit, and the orbit visits different portions of $\mathcal{A}$ with different frequencies. That information is invisible to a covering argument, and recovering it is the business of the rest of this chapter.

11.1 Invariant Measure of Attractor

Recall that we are discussing *invariant sets* $\mathcal{A} \subset \mathcal{S}$. This means that for every $\boldsymbol{u}_0 \in \mathcal{A}$, we have $\varphi_t(\boldsymbol{u}_0) \in \mathcal{A}\ \forall t$ (see Fig. 11.4). As the dynamics travels over $\mathcal{A}$, the probability of visiting different parts of $\mathcal{A}$ will be given by a *probability density* p defined by

$$\lim_{T\to\infty} \frac{1}{T} \int_0^T f\left(\varphi_t(\boldsymbol{u}_0)\right) \mathrm{d}t = \int_{\mathcal{A}} f(\boldsymbol{u}) p(\boldsymbol{u}) \mathrm{d}\boldsymbol{u}\,, \tag{11.2}$$

where by definition $\int_{\mathcal{A}} p(\boldsymbol{u})\mathrm{d}\boldsymbol{u} = 1$.

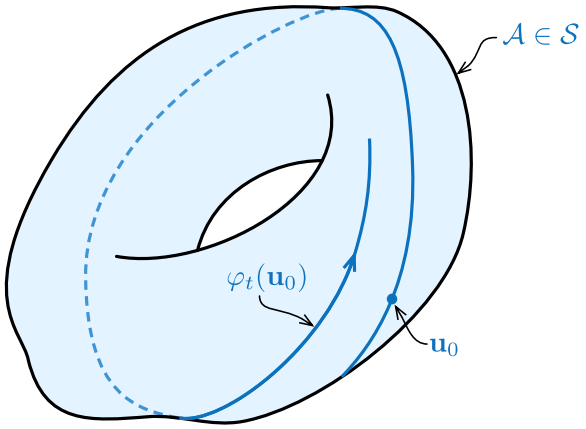


Figure 11.4: For every $\boldsymbol{u}_0 \in \mathcal{A}$, we have $\varphi_t(\boldsymbol{u}_0) \in \mathcal{A}\ \forall t$.

If $p(\boldsymbol{u})$ is the same for almost all[1] $\boldsymbol{u}_0$ in the basin of attraction of a given attractor, p is a *natural density*. The density is *invariant* if probability is conserved,

$$\int_{\mathcal{B}} p(\boldsymbol{u})\mathrm{d}\boldsymbol{u} = \int_{\varphi_t(\mathcal{B})} p(\boldsymbol{u})\mathrm{d}\boldsymbol{u}\,, \tag{11.3}$$

for every $\mathcal{B} \subset \mathcal{A}$ (see Fig. 11.5): the orbit visits $\mathcal{B}$ with the same probability as it visits $\varphi_t(\mathcal{B})$, for all t. Invariance alone does not give *ergodicity*: an invariant measure may decompose into several ergodic components, one for each attractor. We assume throughout that the natural density on a single attractor is ergodic with respect to the dynamics. That assumption is what makes the time averages below independent of $\boldsymbol{u}_0$. In practice, then, the *natural invariant density* is the one generated by all orbits observed to lie on the *same attractor*.

The density itself is a fragile object. On a fractal attractor $p(\boldsymbol{u})$ is nonsmooth, spiky, and may not exist at all. The *invariant probability measure* μ, the cumulative distribution, always does, so we write $\int_{\mathcal{B}} \mathrm{d}\mu$ rather than $\int_{\mathcal{B}} p\,\mathrm{d}\boldsymbol{u}$ throughout. When p does exist, $\mathrm{d}\mu(\boldsymbol{u}) = p(\boldsymbol{u})\mathrm{d}\boldsymbol{u}$, and μ is said to be absolutely continuous with respect to *Lebesgue measure*, with $p = \mathrm{d}\mu/\mathrm{d}\boldsymbol{u}$.

11.2 Dimension Spectrum D_q

Viewing from the perspective of topological dimension, consider an attractor $\mathcal{A}$ covered by uniform boxes $\mathcal{B}_i(\epsilon)$ of size ϵ (see Fig. 11.6). Then

[1] Here, "almost all" means except on a set $\mathcal{C}$ with $\int_{\mathcal{C}} p(\boldsymbol{u})\mathrm{d}\boldsymbol{u} = 0$ (set of measure zero).

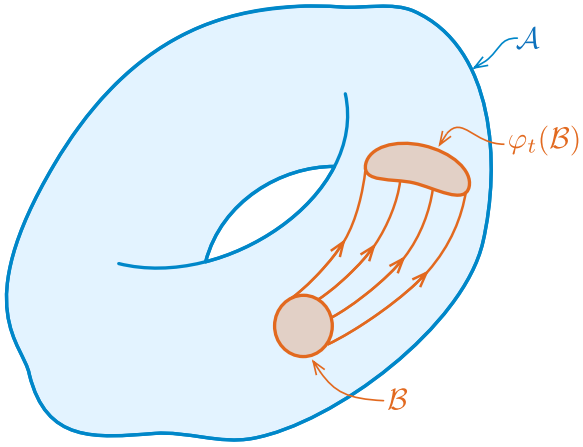


Figure 11.5: Conservation of probability: the orbit visits $\mathcal{B}$ with the same probability as its image $\varphi_t(\mathcal{B})$, for all t.

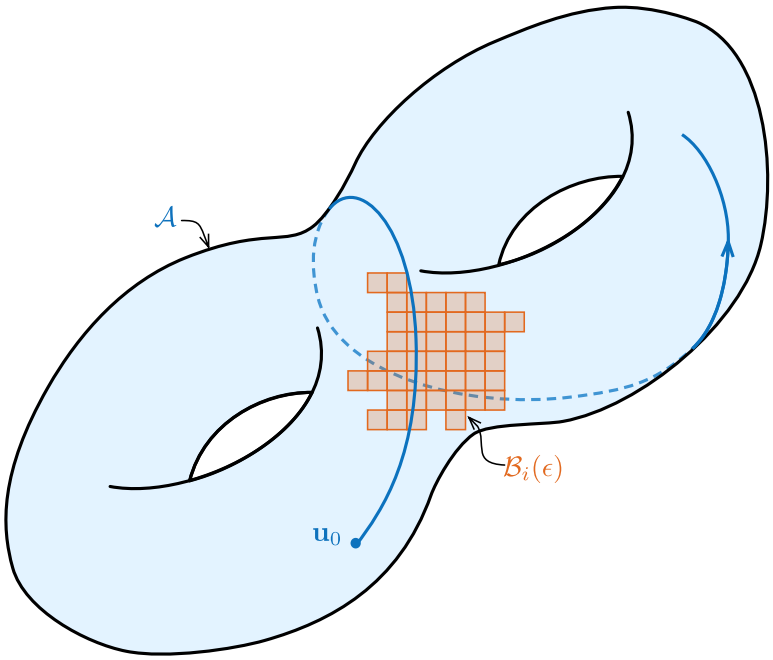


Figure 11.6: $\mathcal{A}$ is covered by uniform boxes $\mathcal{B}_i(\epsilon)$ of size ϵ.

$$p_i = \int_{\mathcal{B}_i} \mathrm{d}\mu = \lim_{T\to\infty} \eta\left(\mathcal{B}_i, \boldsymbol{u}_0, T\right) , \tag{11.4}$$

where $\eta\left(\mathcal{B}_i, \boldsymbol{u}_0, T\right)$ is a function of time (i.e., relative number of data points) the orbit spends in box $\mathcal{B}_i$. If μ is the invariant measure, this limit will be the same for almost all $\boldsymbol{u}_0$. Then define

$$I_q(\epsilon) = \sum_{i=1}^{N(\epsilon)} p_i^q \tag{11.5}$$

for real q. The idea is that for $q > 0$ the larger p_i dominate I_q, while for $q < 0$ the sparsely visited boxes do; we take $q \geq 0$ throughout.

Observe that

$$I_q(\epsilon) = \sum_{i=1}^{N(\epsilon)} p_i^{q-1} p_i = \langle p_i^{q-1} \rangle \,, \tag{11.6}$$

where $\langle p_i^{q-1} \rangle$ is average of p_i^{q-1} on $\mathcal{A}$. Therefore,

$$\sum_{i=1}^{N(\epsilon)} p_i^{q-1} p_i \approx \int_{\mathcal{A}} p_\epsilon(\boldsymbol{u})^{q-1} p \mathrm{d}\boldsymbol{u} = \int_{\mathcal{A}} p_\epsilon(\boldsymbol{u})^{q-1} \mathrm{d}\mu \,, \tag{11.7}$$

where $p_\epsilon(\boldsymbol{u})$ is the probability in a ball of radius ϵ centered at $\boldsymbol{u}$. But from definition of p (or μ), this is

$$C_q(\epsilon) \triangleq \int_{\mathcal{A}} p_\epsilon(\boldsymbol{u})^{q-1} \mathrm{d}\mu = \lim_{T\to\infty} \frac{1}{T} \int_0^T p_\epsilon\left(\varphi_t(\boldsymbol{u}_0)\right)^{q-1} \mathrm{d}t \,, \tag{11.8}$$

where $C_q(\epsilon)$ is the *Generalized Correlation Integral*, and we have

$$I_q(\epsilon) \sim C_q(\epsilon) \,, \tag{11.9}$$

or $I_q(\epsilon)$ is asymptotic to $C_q(\epsilon)$ as $\epsilon \to 0$.

If $\mathcal{A}$ attractor is *self similar*

$$C_q(\epsilon) \sim \left(\epsilon^{q-1}\right)^{D_q} = \epsilon^{(q-1)D_q} \,. \tag{11.10}$$

For some exponent D_q.

The right-hand side of Eq. (11.10) was historically read as a "generalized volume." Nothing is being derived here, though. Eq. (11.10) is a definition of *self similarity*, and D_q exists only for sets that satisfy it.

Thus

$$D_q \sim \frac{1}{q-1} \frac{\ln C_q(\epsilon)}{\ln \epsilon} \,, \tag{11.11}$$

or

$$D_q = \lim_{\epsilon\to 0} \frac{1}{q-1} \frac{\ln C_q(\epsilon)}{\ln \epsilon} \,, \tag{11.12}$$

which is called the *generalized dimension* of order q, when it exists. The spectrum was introduced by Hentschel and Procaccia [94] and, independently, by Grassberger [77]; it rests on the order-q entropy of Rényi [165]. Since, $I_q \sim C_q$, we can also write

$$D_q = \lim_{\epsilon\to 0} \frac{1}{q-1} \frac{\ln I_q(\epsilon)}{\ln \epsilon} \,, \tag{11.13}$$

Definition Eq. (11.13), written in boxes, is the easier one to picture. Definition Eq. (11.12), written in the generalized correlation integral, is the one that survives contact with data, because it requires no partition of the state space. Three members of the spectrum carry names.

For $q = 0$, $I_0(\epsilon) = \sum_{i=1}^{N(\epsilon)} p_i^0 = N(\epsilon)$, so

$$D_0 \sim \frac{\ln N(\epsilon)}{\ln(1/\epsilon)}\,, \tag{11.14}$$

which records how the number of boxes needed to cover $\mathcal{A}$ scales with ϵ. This is the *box counting* or *capacity* dimension, and $D_0 = D_F$ is the dimension appearing in the embedding theorem.

For $q = 1$ the prefactor $1/(q-1)$ diverges and l'Hospital's rule is needed:

$$\begin{aligned} D_1 &\sim \lim_{q\to 1} \frac{\ln \sum p_i^q}{(q-1)\ln\epsilon} \\ &= \lim_{q\to 1} \frac{\frac{1}{\sum p_i^q}\sum \frac{\mathrm{d}}{\mathrm{d}q} p_i^q}{\ln\epsilon} \\ &= \lim_{q\to 1} \frac{\frac{1}{\sum p_i^q}\sum \frac{\mathrm{d}}{\mathrm{d}q} e^{q\ln p_i}}{\ln\epsilon} \\ &= \lim_{q\to 1} \frac{\frac{1}{\sum p_i^q}\sum p_i^q \ln p_i}{\ln\epsilon}\,. \end{aligned} \tag{11.15}$$

Since $\lim_{q\to 1}\sum p_i^q = \sum p_i = 1$,

$$D_1 \sim \frac{\sum p_i \ln p_i}{\ln\epsilon}\,. \tag{11.16}$$

D_1 is the *information dimension* [165]. The name is earned because $-\sum p_i \ln p_i = -\langle \ln p_i\rangle$ is the Shannon information: the average number of digits, or bits with $\log_2$, needed to specify the state to an accuracy of ϵ.

For $q = 2$ the exponent $q-1$ is unity and C_q reduces to

$$C_2(\epsilon) = \int_{\mathcal{A}} p_\epsilon(\boldsymbol{u})\mathrm{d}\mu\,, \tag{11.17}$$

the *correlation integral*, and D_2 is the *correlation dimension* [76, 79]. It is the easiest member of the spectrum to measure, and the estimation sections below are written for it.

The spectrum collapses when the measure is uniform. If $p_i = p_j \equiv p$, then $p = N(\epsilon)^{-1}$ and

$$\begin{aligned} D_q &\sim \frac{1}{q-1}\frac{\ln I_q(\epsilon)}{\ln\epsilon} = \frac{1}{q-1}\frac{\ln\sum N^{-q}}{\ln\epsilon} \\ &= \frac{1}{q-1}\frac{\ln\left(N(\epsilon)\,N^{-q}\right)}{\ln\epsilon} = \frac{1-q}{q-1}\frac{\ln N(\epsilon)}{\ln\epsilon} \\ &= \frac{\ln N(\epsilon)}{\ln\frac{1}{\epsilon}} = D_0 \equiv D_F \end{aligned} \tag{11.18}$$

independently of q. In general $D_{q_1} \le D_{q_2}$ for $q_1 > q_2$: the spectrum is non-increasing. For ordinary sets — points, lines, planes, surfaces — $D_q = D$ for every q. If $D_F \neq D$ the object is a *fractal*. If D_q depends on q it is a *multifractal*: a single exponent no longer describes it, and the scaling is carried instead by the singularity spectrum $f(\alpha)$ of Halsey *et al.* [85, 86].

The ordering of the spectrum has a practical consequence for embedding. The embedding theorem asks for $d > 2D_F = 2D_0$ [173], and D_0 is the member of the spectrum hardest to estimate, so D_2 is substituted in practice. Since $D_2 \le D_0$, the substitution *under*estimates the requirement. A d obtained from $2D_2$ is a lower bound, not a safe value, and Fig. 11.18's $D_2 = 2.16$ for the two-well Duffing record is the kind of number this matters for. Read in the other direction, the theorem says nothing about a minimal adequate d: it is sufficient, and many attractors embed in $d < 2D_0$. The only general necessary condition is weaker. A delay map that is bi-Lipschitz on $\mathcal{A}$ cannot raise the box-counting dimension, so it requires $d \ge D_0 \ge D_1 \ge D_2$.

11.3 Illustrative Examples

11.3.1 A Line Segment

Let us consider a line segment shown in Fig. 11.7(a) covered by balls $\mathcal{B}_i$. Each $x \in \mathcal{A}$ lies in at most two of the $\mathcal{B}_i$. Thus, topological dimension $D = 1$. For a line segment of length L in Fig. 11.7(b) covered by uniform boxes of size ϵ, the number of boxes $N(\epsilon) \sim L/\epsilon$. Therefore,

$$\begin{aligned} D_0 &= \lim_{\epsilon \to 0} \frac{\ln\left(\frac{L}{\epsilon}\right)}{\ln\left(\frac{1}{\epsilon}\right)} = \lim_{\epsilon \to 0} \frac{\ln L - \ln \epsilon}{\ln 1 - \ln \epsilon} \\ &= \lim_{\epsilon \to 0} \frac{\ln \epsilon}{\ln \epsilon} = 1 \,. \end{aligned} \tag{11.19}$$

Therefore, $D_F = D = 1$ or topological dimension is equal to the box counting dimension.

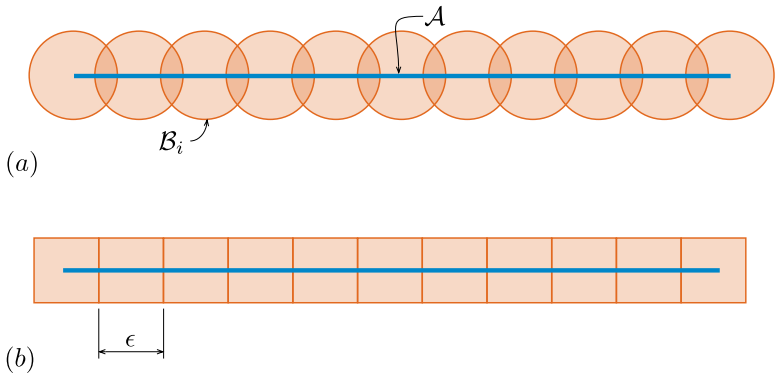


Figure 11.7: A line segment $\mathcal{A}$ covered by balls $\mathcal{B}_i$ (a) and covered by uniform boxes of size ϵ (b)

11.3.2 An Area

We know that the topological dimension for the area $D(\mathcal{A}) = 2$. Now, if we imagine covering this area by N uniform boxes of size ϵ, then

$$N(\epsilon) \approx \frac{A(\mathcal{A})}{A(\mathcal{B}_\epsilon)} = \frac{A}{\epsilon^2} \,, \tag{11.20}$$

where A is the total area. Now, we consider a box counting dimension

$$\begin{aligned} D_0 &= \lim_{\epsilon \to 0} \frac{\ln\left(\frac{A}{\epsilon^2}\right)}{\ln\left(\frac{1}{\epsilon}\right)} = \lim_{\epsilon \to 0} \frac{\ln A - 2\ln \epsilon}{\ln 1 - \ln \epsilon} \\ &= \lim_{\epsilon \to 0} \frac{2\ln \epsilon}{\ln \epsilon} = 2 \,. \end{aligned} \tag{11.21}$$

Therefore, $D_F = D = 2$ or, again, we show that topological dimension is equal to the box counting dimension.

11.3.3 Middle Thirds Cantor Set

In this example, $\mathcal{A}$ is generated as shown in Fig. 11.9:

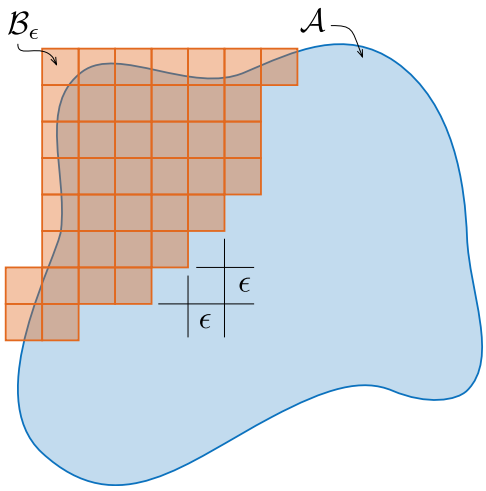


Figure 11.8: An area $\mathcal{A}$ of size A covered by uniform boxes $\mathcal{B}_\epsilon$ of size ϵ

1. Take interval $[0,1]$ and remove the interval $(\frac{1}{3},\frac{2}{3})$ to get

$$\left[0,\frac{1}{3}\right]\cup\left[\frac{2}{3},1\right]$$

2. Remove the middle $1/3$ of each of those to get

$$\left[0,\frac{1}{9}\right]\cup\left[\frac{2}{9},\frac{1}{3}\right]\cup\left[\frac{2}{3},\frac{7}{9}\right]\cup\left[\frac{8}{9},1\right]$$

3. Repeat this at each step as $n\to\infty$.

The total length after n^{th} step can be expressed as $L_n=\left(\frac{2}{3}\right)^n\to 0$ as $n\to\infty$. Therefore, $\mathcal{A}$ has *zero Lebesgue measure*. Then the question is: is $\mathcal{A}$ just a countable set of *isolated points*? However, $\mathcal{A}$ is also *uncountable*, since we can make a one-to-one correspondence between $\mathcal{A}$ and $[0,1]$ using binary tree:

$$\begin{array}{cccccccccccccccc}
[& & & & 0 & & & &] & & & & [& & & & 1 & & & &] & & n=1 \\
[& & 0 & &] & & & [& & 1 &] & & [& & 0 & &] & & [& & 1 &] & & n=2 \\
[0] & & & [1] & & & [0] & & & [1] & & & [0] & & & [1] & & [0] & & & [1] & & n=3
\end{array} \tag{11.22}$$

Therefore, all $x\in\mathcal{A}$ can be written as $\{b_1\,b_2\,\ldots\,b_n\}$, where $b_i=0,1$. For example, the first member of $\mathcal{A}$ can be written as an infinite sequence of zeros, and the last member as an infinite sequence of ones. Nor does $\mathcal{A}$ contain any isolated point: every point of $\mathcal{A}$ is a limit of other points of $\mathcal{A}$ (flip b_n for arbitrarily large n), so $\mathcal{A}$ is *perfect*. Its topological dimension is nevertheless $D=0$, because $\mathcal{A}$ is *totally disconnected*: every point has arbitrarily small neighborhoods whose boundaries miss $\mathcal{A}$ (the gaps removed at stage n), so a cover by such neighborhoods can be chosen with no two overlapping. However, for D_0 we can pick $\epsilon=\left(\frac{1}{3}\right)^n\equiv\epsilon_n$ for each iteration n. Then at $n=1$, $N(\epsilon_1)=2$; at $n=2$, $N(\epsilon_2)=4$. So, $N(\epsilon_n)=2^n$ and thus,

$$D_0=\lim_{n\to\infty}\frac{\ln 2^n}{\ln 3^n}=\frac{\ln 2}{\ln 3}\approx 0.6309\,. \tag{11.23}$$

Therefore, the middle thirds Cantor set is a *fractal*.

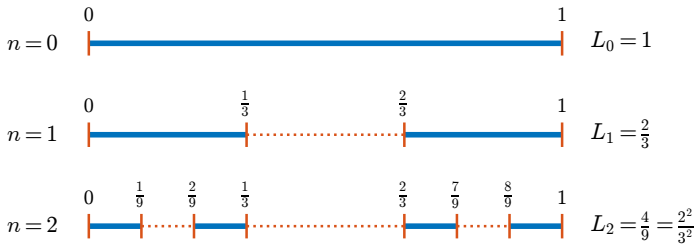


Figure 11.9: Generation of the middle thirds Cantor set: the surviving intervals at stages $n = 0, 1, 2$ (the removed open middle thirds are dotted), with the end points and the total length $L_n = (2/3)^n$. Drawn from exact interval arithmetic by `python/mkfig_cantor.py`.

The definition

$$D_0 = \lim_{n\to\infty} \frac{\ln N(\epsilon)}{\ln(1/\epsilon)} \tag{11.24}$$

is equivalent to the box count scaling as

$$N(\epsilon) \sim \epsilon^{-D_0}\,, \tag{11.25}$$

which is what *self similarity* means here. Measured sets are never perfectly self similar. What one can hope to see in an experiment is the scaling

$$N(\epsilon) \approx C\epsilon^{-D_0} \tag{11.26}$$

over a range of ϵ. The range is bounded below by the noise and by the finite number of points, and above by the size of the attractor. Finding it, and defending it, is the whole of the estimation problem treated in Section 11.6.2.

11.4 Fractal Sets from Dynamical System

The logical question is: can something like a Cantor set arise from dynamics? To answer this, we consider the following one-dimensional map:

$$x_{n+1} = F(x_n)\,, \tag{11.27}$$

where

$$F(x) = \begin{cases} 2\mu x & \text{for} \quad x < \frac{1}{2}\,, \\ 2\mu(x-1)+1 & \text{for} \quad x > \frac{1}{2}\,. \end{cases} \tag{11.28}$$

Please note that for $\mu = 1$, this is a $2x$ mod 1 map. In addition, for $x_0 > 1$, $x_n \to \infty$; and for $x_0 < 0$, $x_n \to -\infty$. Therefore, we focus on $x_n \in [0,1]$ region. Considering the diagram shown in Fig. 11.10, at each iteration the middle strip of width $\left(1 - \frac{1}{2\mu}\right) - \frac{1}{2\mu} = \frac{\mu-1}{\mu} \triangleq \Delta$ leaves $[0,1]$ never to return. We could ask if any points remain as $n \to \infty$. To answer this, we consider uniform probability density of initial conditions x_0, where $p(x_0) = 1$ on $[0,1]$. Then $x_1 = F(x_0)$ for all $x_0 \in [0,1]$. Now consider the integral:

$$\int_{-\infty}^{\infty} p(x_0)\mathrm{d}x_0 = \int_{-\infty}^{\infty} \left[\sum_{x_0 \in F^{-1}(x_1)} p(x_0) \left| \frac{\mathrm{d}F}{\mathrm{d}x_0} \right|^{-1} \right] \mathrm{d}x_1 \equiv \int_{-\infty}^{\infty} p(x_1)\mathrm{d}x_1 = 1\,. \tag{11.29}$$

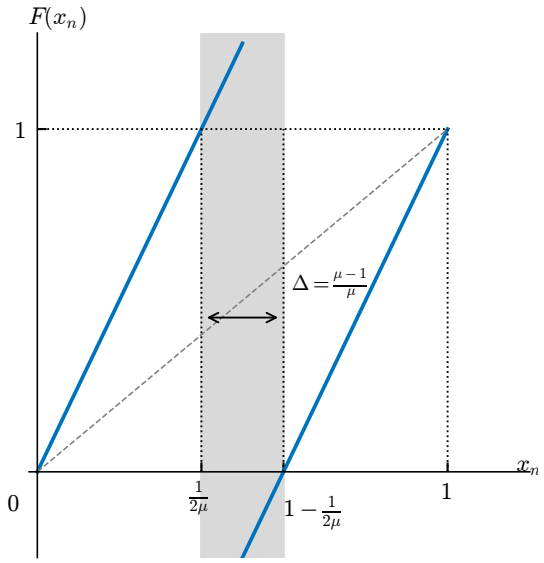


Figure 11.10: The map Eq. (11.28), drawn for $\mu = 1.25$. The shaded strip $[1/(2\mu),\, 1-1/(2\mu)]$, of width $\Delta = (\mu-1)/\mu = 0.2$, is mapped out of $[0,1]$ in one iteration. Script: `python/mkfig_1Dmap.py`.

So $p(x_1) = \sum_{x_0 \in F^{-1}(x_1)} p(x_0) \left|\frac{\mathrm{d}F}{\mathrm{d}x_0}\right|^{-1}$, the sum running over all preimages of x_1 (this is the *Frobenius–Perron* equation; the map is not monotone, so the sum cannot be dropped). In our case, every $x_1 \in [0,1]$ has exactly two preimages, one on each branch of Eq. (11.28), and $\frac{\mathrm{d}F}{\mathrm{d}x_0} = 2\mu$ on both. Thus $p(x_1) = 2 \cdot \frac{1}{2\mu} = 1/\mu$ *uniformly* on $[0,1]$. In other words, $p(x_1) = 1 - \frac{\mu-1}{\mu} = \frac{1}{\mu}$, the middle Δ is the "lost" probability. So the probability of remaining in $[0,1]$ goes to 0 as $\left(\frac{1}{\mu}\right)^n$. Therefore, if points remain, they form a set of zero Lebesgue measure.

A bit of thinking about points that remain after n iterations shows a picture shown in Fig. 11.11. That is, we get a generalized Cantor set. However, this is *not* an "attractor"—in fact, it is unstable, though invariant.

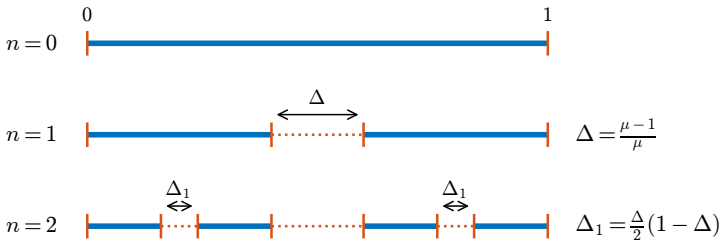


Figure 11.11: The generalized Cantor set left by the map Eq. (11.28): the points still in $[0,1]$ after $n = 0, 1, 2$ iterations, for $\mu = 5/4$, so that $\Delta = (\mu-1)/\mu = 1/5$ and $\Delta_1 = \Delta(1-\Delta)/2 = 2/25$. Drawn from exact interval arithmetic by `python/mkfig_cantor.py`.

11.5 Dimensionality of Continuous Attractors

11.5.1 Periodic Orbits

For any periodic response, we get periodic orbits in the phase space for which $D = 1 = D_0$. **Paradigm:** Any periodic response (see Fig. 11.12).

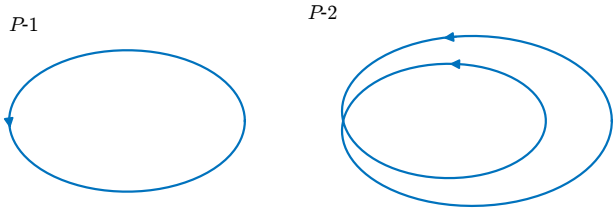


Figure 11.12: Closed orbits of any period: $D = 1 = D_0$. A period-one orbit and a period-two orbit, which closes only after two turns. Script: `python/mkfig_orbits.py`.

11.5.2 Quasi-periodic Orbits

For a quasi-periodic orbit, we need to consider a linear combination of periodic signals with incommensurate periods. This orbit generates a flow on n-torus, where n is the number of incommensurate frequencies. An example for $n = 2$ can be $x(t) = A_1 \sin t + A_2 \sin \sqrt{7}\, t$, which can be visualized as evolving on a surface of a donut (2-torus) as shown in Fig. 11.13. In this case, $D = n = D_0$.

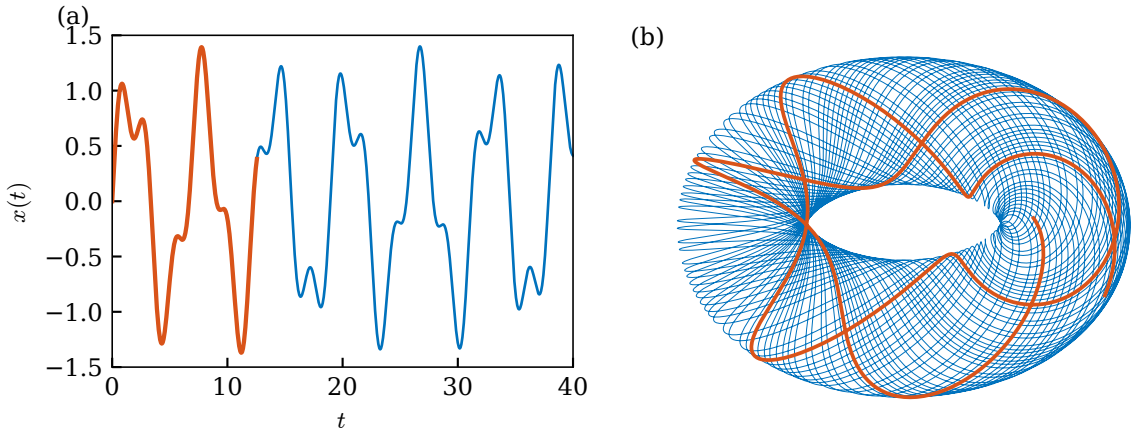


Figure 11.13: A quasi-periodic response $x(t) = A_1 \sin t + A_2 \sin \sqrt{7}\, t$ with $A_1 = 1$, $A_2 = 0.4$: (a) the signal over 40 time units; (b) the same motion for $0 \le t \le 250$ as the orbit $(\theta_1, \theta_2) = (t, \sqrt{7}\, t)$ on the 2-torus of major radius A_1 and tube radius A_2. The first two turns are drawn heavy in both panels. The orbit never closes and fills the torus densely, so $D = 2 = D_0$. Script: `python/mkfig_orbits.py`.

Paradigm: Two weakly coupled van der Pol oscillators with incommensurate natural frequencies,

$$\begin{aligned} \ddot{x} - \mu(1 - x^2)\dot{x} + x &= \kappa y\,, \\ \ddot{y} - \mu(1 - y^2)\dot{y} + \omega^2 y &= \kappa x\,, \end{aligned} \tag{11.30}$$

with, e.g., $\mu = 0.2$, $\omega = \sqrt{2}$ and $\kappa = 0.05$. Each oscillator alone settles onto a limit cycle of amplitude near 2; the weak coupling shifts the two frequencies slightly (to 0.996 and 1.414 for these parameters) without locking them, and the joint motion is a flow on a 2-torus. A Poincaré section at $x = 0$, $\dot{x} > 0$ shows the $(y, \dot{y})$ crossings filling a closed curve densely, the largest angular gap between crossings shrinking in proportion to the number of crossings, which is the signature of an irrational rotation number. Increase κ, or move ω close to a rational, and the pair locks onto a periodic orbit.

11.5.3 Chaotic Orbits

Here we will have a continuous directing along the time axis (or forcing phase), and we would have *fractal directions* usually seen in Poincaré sections as in Fig. 11.14. Here, $D < D_0$, where D_0 is a non-integer.

Paradigms:

1. Driven two-well Duffing chaos ($2 < D_0 < 3$);
2. Lorenz equation ($D_0 \approx 2 + \epsilon$); and
3. Chua circuit ($2 < D_0 < 3$).

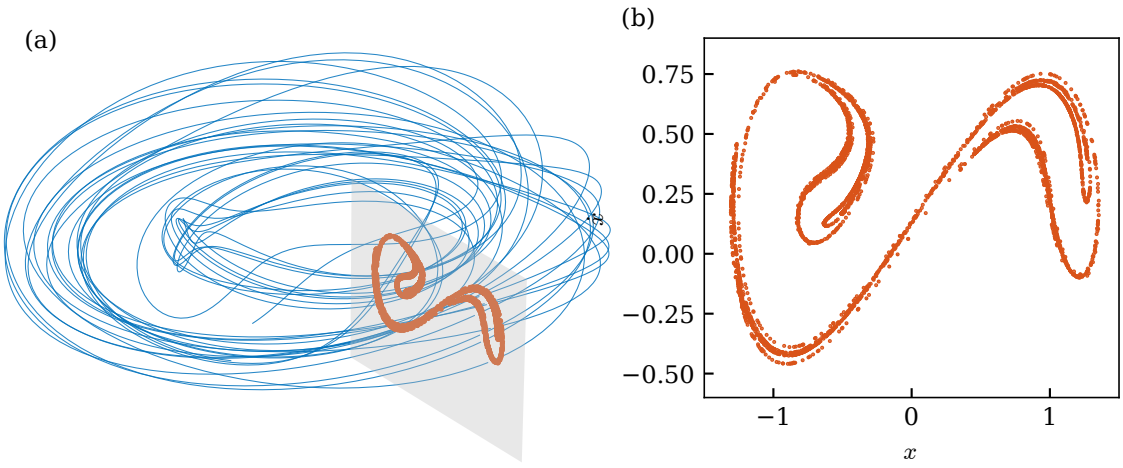


Figure 11.14: A chaotic orbit of the driven two-well Duffing oscillator $\ddot{x} + 0.25\dot{x} - x + x^3 = 0.3\cos t$ (Section 1.4). (a) Thirty forcing periods of the trajectory in the cylindrical phase space $(x, \dot{x}, \phi = t \bmod 2\pi)$, with the section plane $\phi = 0$ shaded; (b) the stroboscopic Poincaré section on that plane, 4000 crossings, showing the fractal transverse structure. Script: `python/mkfig_orbits.py`.

Please note that a continuous-time flow must live in a phase space of dimension at least three for chaotic behavior to be possible (the attractor dimension then exceeds two), while maps can be chaotic in one dimension. The reason is, in $2 - D$, the uniqueness of the solutions "traps" bounded orbits into either fixed points or limit cycles. In other words, there is not enough room in $2 - D$ for chaos as shown in Fig. 11.15.

11.6 Experimental Estimates of Fractal Dimension

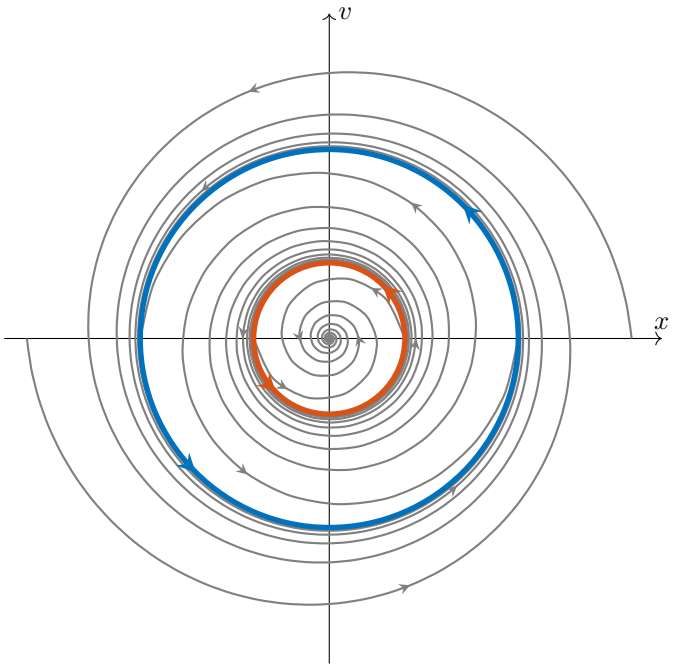


Figure 11.15: Uniqueness of solution in two-dimensions traps bounded flows either to fixed points (e.g., $(0,0)$ stable spiral) or limit cycles (red is unstable and blue is stable).

In the previous sections we discussed dimension theory and defined the *dimension spectrum*. It was also mentioned that the easiest and most reliable estimation is for D_2, *correlation dimension*, based on:

$$C_2(\epsilon) \equiv C(\epsilon) = \int_{\mathcal{A}} p_\epsilon(\boldsymbol{u}) \mathrm{d}\mu \,, \tag{11.31}$$

where ϵ is the size of partition on the attractor $\mathcal{A}$, $\boldsymbol{u} \in \mathcal{A}$, and μ is the corresponding invariant probability measure.

11.6.1 Correlation Dimension Estimate

Given a time scalar series $\{x_n\}_{n=1}^N$, we start by selecting a "good" delay τ and embedding dimension d to get a vector time series $\{\boldsymbol{x}\}_{n=1}^{N-(d-1)\tau}$. In what follows, for simplicity, we will assume that we have $\{\boldsymbol{x}\}_{n=1}^N$ (considering that $N \approx N-(d-1)\tau$ for N large). Then, the estimator of $C(\epsilon)$ can be thought of as a *cumulative distribution* of distances $\|\boldsymbol{x}_i - \boldsymbol{x}_j\| < \epsilon$ in our data set. Therefore, we consider the following *correlation sum* [76]:

$$\hat{C}(\epsilon) = \frac{2}{N(N-1)} \sum_{i=1}^{N-1} \sum_{j=i+1}^{N} \Theta\left(\epsilon - \|\boldsymbol{x}_i - \boldsymbol{x}_j\|\right) \,, \tag{11.32}$$

where Θ is the Heaviside function.

Eq. (11.32) is a good estimator for the $C(\epsilon)$ if our data consists of points randomly drawn from our attractor $\mathcal{A}$. However, we usually have a continuously sampled time series, which cannot be considered to be randomly drawn from $\mathcal{A}$. We know from the false nearest neighbors chapter that temporal correlations can lead to skewed or biased estimates. In the case of estimating the correlation dimension, the temporal correlations can lead to false low-dimensional results. If we have a large

amount of data, we can randomly draw points from it to generate the needed randomly sampled data to avoid the effect of temporal correlations. However, the availability of abundant data is only viable in simulations and not in experiments. Then, the solution is to only measure distances $\|\boldsymbol{x}_i - \boldsymbol{x}_j\|$ between the points which are separated by time greater than the Theiler window w [202, 203] (i.e., $|i-j| > w$). The same exclusion is required of the Lyapunov estimators of Section 12.5.4, where, unlike here, the direction of the bias it removes is not established in the literature. Then the corresponding *correlation sum* can be written as:

$$\hat{C}(\epsilon) = \frac{2}{(N-w)(N-w-1)} \sum_{i=1}^{N-w-1} \sum_{j=i+w+1}^{N} \Theta\left(\epsilon - \|\boldsymbol{x}_i - \boldsymbol{x}_j\|\right) , \tag{11.33}$$

For small ϵ, we expect $\hat{C}(\epsilon) \sim \epsilon^{D_2}$[2], or $\ln \hat{C}(\epsilon) \sim D_2 \ln \epsilon$. Therefore:

$$D_2 \sim \frac{\mathrm{d}\left(\ln \hat{C}(\epsilon)\right)}{\mathrm{d}\left(\ln \epsilon\right)} , \tag{11.34}$$

if we are lucky and our data shows this trend.

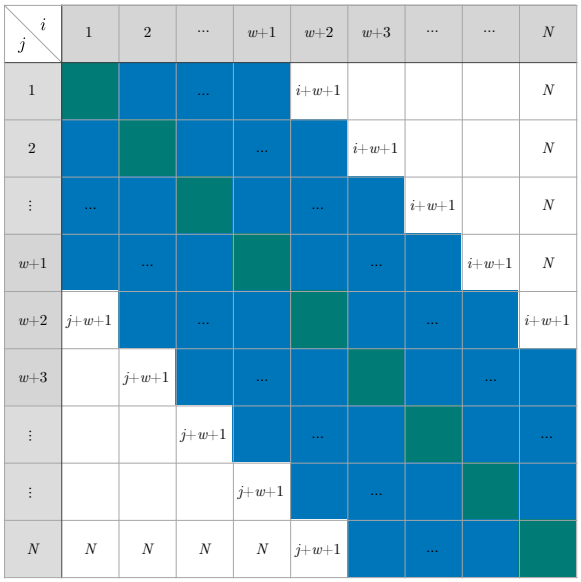


Figure 11.16: Visualization of all the temporally uncorrelated pairs of points $\boldsymbol{x}_i$ and $\boldsymbol{x}_j$ for which the interpoint distance needs to be calculated. The green squares represent $i = j$ and the blue squares are the temporally correlated points within $|i-j| \leq w$.

The record holds $N(N-1)/2$ distinct pairs, which is the divisor in Eq. (11.32). Excluding the pairs closer in time than the Theiler window leaves $(N-w)(N-w-1)/2$ of them, the divisor in Eq. (11.33). Fig. 11.16 shows which distances survive that exclusion.

Plotted as $\ln \hat{C}(\epsilon)$ against $\ln \epsilon$, a chaotic record shows three regions (Fig. 11.17). Above $\epsilon_{\max} = \|\boldsymbol{x}_i - \boldsymbol{x}_j\|_{\max}$ every pair is counted, $\hat{C}(\epsilon) = 1$, and $\ln \hat{C} \to 0$. Below $\epsilon_{\min} = \|\boldsymbol{x}_i - \boldsymbol{x}_j\|_{\min}$ no pair is counted, $\hat{C}(\epsilon) = 0$, and $\ln \hat{C} \to -\infty$; well before that the curve stops looking smooth, because too few pairs remain at those scales. Between the two lies the scaling region, in which $\ln \hat{C}(\epsilon)$ is linear in $\ln \epsilon$ with slope D_2.

[2]Here, $\sim$ means as $\epsilon \to 0$, $N \to \infty$.

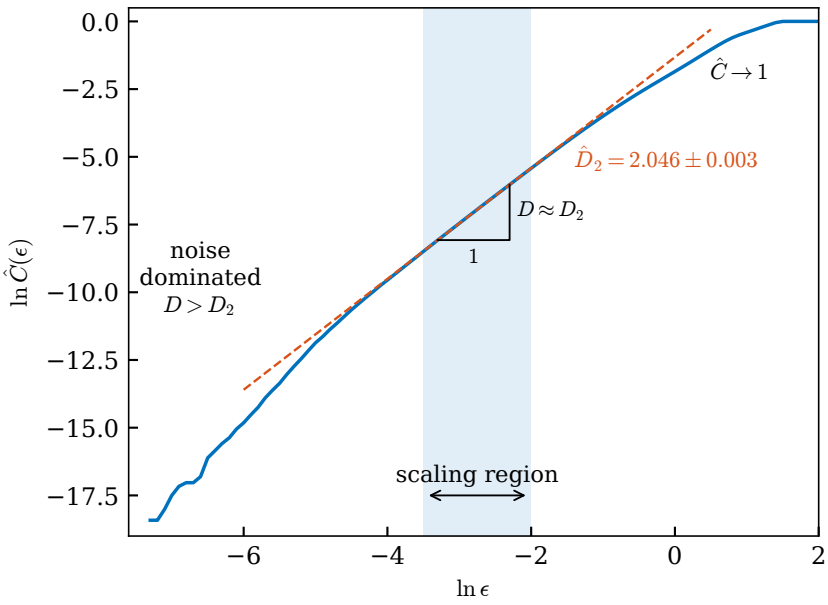


Figure 11.17: Correlation sum estimate $\hat{C}(\epsilon)$ against ϵ for the Lorenz x signal ($N = 20{,}000$, $\Delta t = 0.02$, normalized to unit variance, $d = 3$, $\tau = 8$, Theiler window $w = 3\tau = 24$) with 0.3% Gaussian measurement noise added, which is what puts the noise-dominated region inside the plot. Least squares over the shaded scaling range $\ln \epsilon \in [-3.5, -2.0]$ gives $\hat{D}_2 = 2.046 \pm 0.003$ (standard error of the fitted slope); over $\ln \epsilon \in [-6, -4.5]$ the local slope is 2.4 to 3.2, rising toward d, and above $\ln \epsilon \approx 1.5$ it falls to zero as $\hat{C} \to 1$. Script: `python/mkfig_C2Lorenz.py`.

Pitfall: noise fills whatever dimension you embed in

The measured point is $\boldsymbol{x}_i = \boldsymbol{y}_i + \boldsymbol{n}_i$, with $\boldsymbol{y}_i$ the true reconstructed point and $\boldsymbol{n}_i$ the noise. If the $\boldsymbol{n}_i$ are independent, the noise has no finite dimension: embedded in dimension d it fills the reconstruction space, so $D_2(\text{noise}) = d$ and grows without bound with d. At the scales of ϵ where noise dominates the pair distances, the local slope of $\ln \hat{C}(\epsilon)$ therefore climbs toward d rather than settling (Fig. 11.17). This is also the check: a $\hat{D}_2$ that keeps rising as d is increased is measuring the noise. If the noise scale lies above the top of the scaling region, D_2 cannot be estimated from the record at all.

Estimating D_2 means identifying the tangent line to $\ln \hat{C}(\epsilon)$ against $\ln \epsilon$ in the scaling region, as in Fig. 11.17. It is usually not easy to identify the suitable scaling region, even if it is there, and it may not exist at all, because the record length required grows exponentially with the dimension being estimated.

Pitfall: an estimate above $2 \log_{10} N$ is an artifact of the record length

Two bounds on what a record of N points can support are standard. Eckmann and Ruelle [53] argue that a scaling region must extend below $\epsilon = \rho\, \epsilon_{\max}$ with $\rho \lesssim 0.1$, where $C(\epsilon) \approx \rho^{D_2}$; since the correlation sum is built from $N^2/2$ pairs, it can be resolved there only if $\rho^{D_2} \gtrsim N^{-2}$, which for $\rho = 0.1$ gives

$$D_2 \lesssim 2 \log_{10} N \ : \tag{11.35}$$

any estimate above $2 \log_{10} N$ (6 for $N = 1000$, 10 for $N = 10^5$) is an artifact of the record length, whatever the data. Smith [191] requires the estimate to be within about 5% of the true value over a scaling region and obtains the much stricter $N \gtrsim 42^D$, which is 7×10^4 points for $D = 3$ and 3×10^6 for $D = 4$; the constant is debated but the exponential form is not.

Together the two bounds place attractors of dimension much above five out of reach of any realistic experiment, and they mean that for the Lorenz attractor ($D_2 \approx 2.05$) a few thousand points suffice while for the two-well Duffing oscillator (the Kaplan–Yorke dimension of the flow is 2.33 at the parameters of Section 1.4, one more than that of its stroboscopic section, and Fig. 11.18 measures $D_2 = 2.16$) tens of thousands are needed.

Within those limits, the scaling region is found from the local slope $\mathrm{d} \ln \hat{C}(\epsilon) / \mathrm{d} \ln \epsilon$ evaluated at each $\ln \epsilon$, as in Fig. 11.18. Read it from the small-ϵ end. The slope scatters wildly at first, where noise and pair depletion dominate. It then settles into a smooth, nearly horizontal stretch. That stretch is the scaling region. Beyond it the slope drops away as the distances approach the size of the attractor. Only the flat stretch is evidence.

11.6.2 Estimators with Error Bars

Reading a slope off Fig. 11.18 by eye gives a number without an uncertainty, and a least-squares line through the points $(\ln \epsilon_k, \ln \hat{C}(\epsilon_k))$ gives a standard error that is meaningless, because the points are cumulative counts of the same pairs and are strongly correlated. Two estimators with a statistical basis exist.

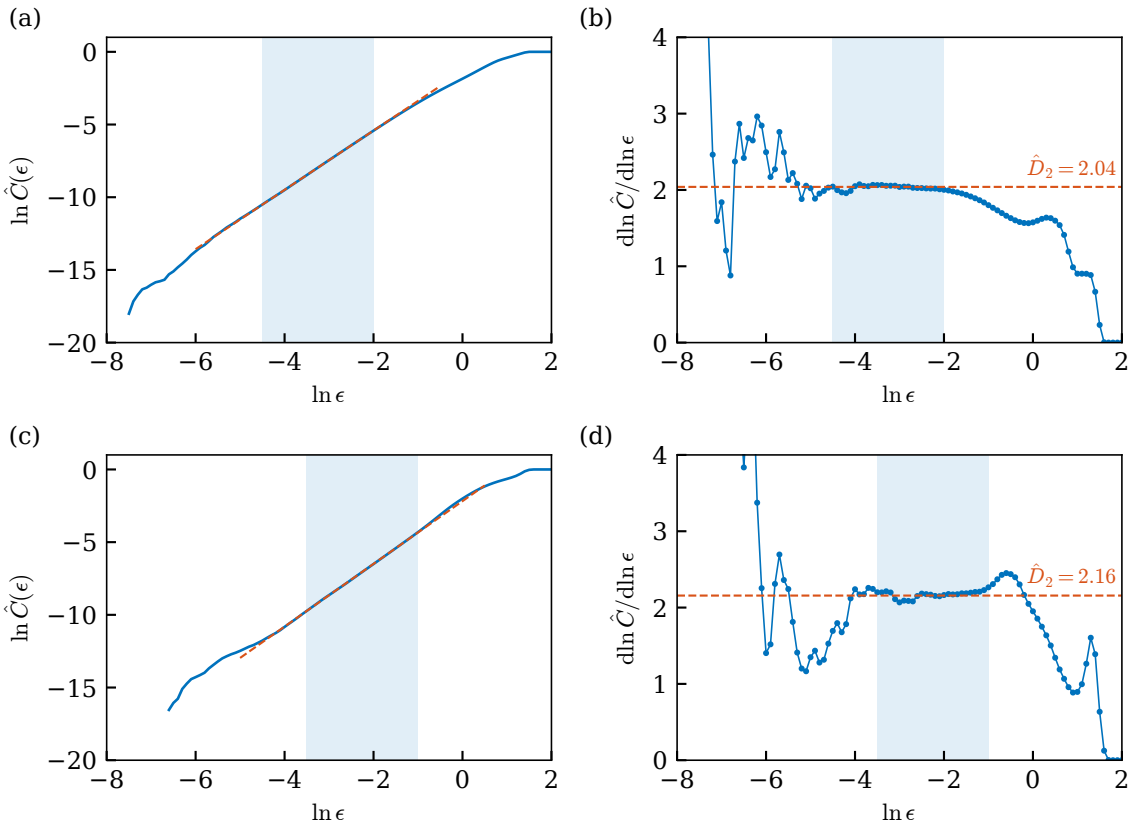


Figure 11.18: Correlation sum and the corresponding estimate of D_2 for the Lorenz (a–b) and two-well Duffing (c–d) signals. Both records have $N = 20{,}000$ points normalized to unit variance: Lorenz x at $\Delta t = 0.02$ with $\tau = 8$, $d = 3$; Duffing x at $\Delta t = 0.1$ with $\tau = 14$, $d = 4$; Theiler window $w = 3\tau$ in both. The shaded scaling regions are $\ln \epsilon \in [-4.5, -2.0]$ and $[-3.5, -1.0]$. Least squares over them gives $\hat{D}_2 = 2.040 \pm 0.002$ and 2.157 ± 0.003, the errors being the standard error of the fitted slope; the local slopes scatter about those values by ± 0.03 and ± 0.05, and Takens' estimator Eq. (11.36) at three ϵ_0 spanning the upper half of each range gives 2.02–2.06 and 2.14–2.20, which is the honest uncertainty (Section 11.6.2). Script: `python/mkfig_D2Plots.py`.

Takens' maximum-likelihood estimator

Suppose $C(\epsilon) = A\epsilon^D$ exactly for $\epsilon \le \epsilon_0$. Then the distances r between pairs of points with $r < \epsilon_0$ have the density Dr^{D-1}/ϵ_0^D, the variables $\ln(\epsilon_0/r)$ are exponentially distributed with mean $1/D$, and the maximum-likelihood estimate from the M pairs with $r_k < \epsilon_0$ is

$$\hat{D}_{\mathrm{T}} = \frac{M}{\sum_{k=1}^{M} \ln(\epsilon_0/r_k)} = -\left\langle \ln \frac{r_k}{\epsilon_0} \right\rangle^{-1}, \tag{11.36}$$

with standard error $\hat{D}_{\mathrm{T}}/\sqrt{M}$ if the pairs are independent [198, 206].

The estimator uses every distance below ϵ_0 rather than a handful of bin counts, and it requires no choice of lower cutoff. For 2000 points uniform in the unit square and $\epsilon_0 = 0.05$ it returns $\hat{D}_{\mathrm{T}} = 1.971$ with empirical scatter 0.016 over 200 replicates, against 0.016 from the formula (`python/chk_takens.py`). The 1.5% shortfall is an *edge effect*: a point within ϵ_0 of the boundary of the set has fewer neighbors at the larger distances than the power law predicts, so the small distances are over-represented and the slope is biased low. On an attractor the edges are the outer folds of the set, and the effect is why the local slope of $\ln \hat{C}$ always droops as ϵ approaches the attractor size (Fig. 11.18). The remedy is to keep ϵ_0 well below that size, or to count only pairs whose reference point is farther than ϵ_0 from the boundary.

The formal error bar $\hat{D}_{\mathrm{T}}/\sqrt{M}$ is a statement about precision, not accuracy, and it is too small for two reasons. First, the $M \sim N^2/2$ pairs are not independent: each point enters $N-1$ of them, and Theiler [206] showed that the effective number of independent pairs is closer to N than to N^2, so that the variance of any dimension estimate scales at best as $1/N$, not $1/N^2$. Second, the correlation sum of a fractal set is not an exact power law but $C(\epsilon) = \epsilon^D \Phi(\ln \epsilon)$ with Φ log-periodic; this *lacunarity* makes the local slope oscillate about D by an amount that depends on the set and that no amount of data removes [204]. The Cantor set of Fig. 11.9 is the extreme case, with a discrete scaling ratio 1/3; for the Hénon attractor with $N = 20\,000$ the Takens estimate moves from 1.166 to 1.214 to 1.208 as ϵ_0 is changed from 0.02 to 0.05 to 0.10, while the formal error at each is ± 0.002 (`python/chk_takens.py`). Judd [102] fits the oscillating correction as a polynomial in ϵ and obtains an estimator and confidence intervals that account for it. The practical rule is the following.

Reporting a correlation dimension

1. Compute $\ln \hat{C}(\epsilon)$ with the Theiler window w for $d = 1, 2, \ldots$ and plot the local slopes; identify the scaling range $[\epsilon_{\mathrm{lo}}, \epsilon_{\mathrm{hi}}]$ where the slopes for successive d agree and are flat.

2. Compute Eq. (11.36) with ϵ_0 at three or four values spanning the upper half of the scaling range, and with the record split into two or four non-overlapping segments.

3. Report the mean over ϵ_0 and segments as $\hat{D}_2$, and the spread (not $\hat{D}_{\mathrm{T}}/\sqrt{M}$) as the error E. Report N, d, τ, w, the scaling range in units of the attractor size, and $2\log_{10} N$ from Eq. (11.35).

4. Check the estimate against d: it should be flat once d exceeds D_2 (Section 11.7). If it grows with d, the record is noise-dominated at the scales used.

11.6.3 A Measured Record

The procedure above is worth carrying out once on data whose dimension nobody knows. Take the two-well oscillator record of Section 1.4.1: 96,000 strain samples, 16 samples per forcing period. The delay is $\tau = 5$ samples, the first minimum of the average mutual information (Chapter 9). Embed $n = 30{,}000$ points in $d = 2, \ldots, 8$ and use the Theiler window $w = 5d$ samples that `c2r` takes by default. Evaluate the correlation sum at 76 ball sizes, log-spaced over $\ln(\epsilon/\sigma) \in [-6.5, 1]$, with σ the standard deviation of the record:

```
[lnr, lnc] = c2r( x, d, 5, 30000, r, 5*d );
```

Fig. 11.19 shows the correlation sums and, in panel (b), the local slopes. The local slopes are the evidence; the sums by themselves look straight over almost any range one cares to pick.

One band of scales, $\ln(\epsilon/\sigma) \in [-1.45, -0.35]$, has a flat local slope that does not move as d increases. Least squares over it gives 2.697, 2.693, 2.677, 2.648 and 2.625 for $d = 4, \ldots, 8$. It gives 1.71 and 2.33 for $d = 2$ and 3, which are projections. The five plateau values have median 2.68 and standard deviation 0.03, with a slow downward drift as d grows that is the expected thinning of the pair statistics. Reported as the algorithm box requires,

$$\hat{D}_2 = 2.68 \pm 0.03 \quad \text{over } \epsilon/\sigma \in [0.23,\, 0.70],\ d = 4 \text{ to } 8\,, \tag{11.37}$$

with $N = 30{,}000$, $\tau = 5$ samples and $w = 5d$ samples. The error is the spread over d. It is neither a fitted standard error nor a confidence interval. The interval of scales spans 0.48 decades, less than half a decade, so Eq. (11.37) is a finite-scale description of this record and not an asymptotic scaling law.

Its sensitivity to the other analysis choices is larger than the spread over d. Splitting the record in half and repeating at $d = 5$ with $n = 15{,}000$ points gives 2.69 and 2.73. Three disjoint blocks of 30,000 points, Theiler windows of $5d$ and 160 samples, and $d = 4$ to 6 give 2.67 to 2.75 over the same scales, with mean 2.71; the block-to-block spread, about 0.05, is the largest single contribution. Moving the interval by 0.3 in $\ln\epsilon$ either way widens the range to 2.60 to 2.78. The Theiler window has no measurable effect, and redrawing the random reference points moves the slope by about 0.015 (script `python/chk_d2_sensitivity.py`). A fairer finite-scale statement is therefore $\hat{D}_2 = 2.70 \pm 0.05$ over the stated scales, with the $\pm$ again a spread over blocks, windows and d. The Eckmann–Ruelle bound Eq. (11.35) is $2\log_{10} N = 9.0$, so pair counting is not what limits the estimate.

Two features of Fig. 11.19(b) lie outside the scaling range and must be reported with it. Below the range the local slope does not rise toward d, as it does for the noisy Lorenz record of Fig. 11.17. It *falls*, into a broad trough near 1.68 that runs from $\ln(\epsilon/\sigma) = -3.3$ to -2.4. The trough is at the same depth for every d from 4 to 8. It is not an artifact of the Theiler window: raising w from 20 to 320 samples leaves it unchanged. It is not the delay either: $\tau = 3$, 8 and 12 all produce it.

The trough is produced by sampling locked to the forcing, not by sparse sampling. The record carries exactly 16 samples per forcing period. The phase of the drive line at $f_s/16$ drifts by 0.016 samples over the whole 600 s run, so sample i and sample $i + 16k$ are taken at the same forcing phase. The sampled set is therefore not the suspended attractor. It is sixteen stroboscopic sections, one per phase. A ball smaller than the distance between adjacent sections meets only its own section. The flow direction then contributes nothing to the count, and the local slope falls toward the section dimension $D_2 - 1$. The trough level 1.68 is that value. The record shows the mechanism directly. Among pairs closer than ϵ at $d = 5$, the fraction whose indices agree modulo 16 is 0.32 at

$\ln(\epsilon/\sigma) = -0.9$, 0.60 at -1.5, 0.83 at -1.8 and 1.00 from -2.4 down; chance is 1/16. In the trough every close pair lies on one phase slice. At the lower end of the scaling range more than half of them already do, so the plateau sits in the crossover between the geometry of the flow and that of the slices.

Fig. 11.20 separates the two candidate causes on the noise-free two-well Duffing model of Section 1.4, over 6000 forcing periods in every case. Sampled locked at 16 points per period, matching the record, the local slope has a plateau near 2.22 that ends near $\ln(\epsilon/\sigma) = -1.8$ and a level of 1.16 below it, one less than the plateau. Sampled locked at 64 points per period, the crossover moves down to about -3.0, and the level below it is the same. Sampled at 16.618 points per period, nearly the density of the record but with no two samples at the same forcing phase, there is no trough: the slope stays between 2.1 and 2.5 down to $\ln(\epsilon/\sigma) = -5$, with mean 2.19 over $[-5, -2.5]$. A faster locked rate only moves the trough; unlocked sampling removes it. Under unlocked sampling a small ball still meets each pass of the trajectory in at most one point, but the phase of that point varies from pass to pass, so the flow direction still contributes to the count.

The scale of the crossover still follows from the sampling interval, because adjacent phase slices are one sample apart. In the $d = 5$ reconstruction the one-sample displacement has a median of 0.39σ, with tenth and ninetieth percentiles at 0.24σ and 0.58σ. Those percentiles bracket the scaling range of Eq. (11.37) almost exactly. The estimate is therefore made just above the scale at which the phase slices separate. A record sampled asynchronously with the drive would remove the trough and extend the plateau; a record with more samples per forcing period, still locked, would only move it.

The second feature is at the small end. Below $\ln(\epsilon/\sigma) \approx -4.3$ the slope climbs back to about 2.3, then breaks up, and the break-up starts at larger ϵ the larger d is. Both are what an additive noise floor does: independent noise fills the reconstruction space, so its correlation dimension is d and grows with d. The noise-free model sampled at the same locked rate does not climb back at all; its slope stays near 1.16 down to the smallest scales computed. Pair starvation contributes as well. Of the 4.5×10^8 pairs, about 1700 survive at $\ln(\epsilon/\sigma) = -4.3$, about 190 at -5.3 and about 20 at -6.3. The A/D quantization is not yet acting there: the converter step is $9.6 \times 10^{-5}\sigma$, or $\ln(\epsilon/\sigma) = -9.25$, three decades below the break-up.

Neither feature invalidates Eq. (11.37), and neither could have been noticed from the correlation sums of Fig. 11.19(a). This is the reason for the rule that closes the algorithm box. A correlation dimension is reported with two things: the range of ϵ over which the local slope is flat, and the range of d over which that plateau does not move. Quoted without them, "$D_2 = 2.68$" is not a measurement. The same record returns 1.7 or 2.3 at other scales, and 2.33 at $d = 3$.

11.6.4 Pointwise Dimension

From a mathematical perspective, the *pointwise dimension* is used to differentiate between the probability measures μ_1 on a phase space $\mathcal{S}_1$ and μ_2 on another space $\mathcal{S}_2$ which are not equivalent by a locally bi-Lipschitz injection. For a point $\boldsymbol{u} \in \mathcal{S}$ and and a number $\epsilon > 0$, we can define an ϵ-ball as

$$\mathcal{B}(\boldsymbol{u}, \epsilon) \triangleq \{\boldsymbol{v} \in \mathcal{S} \,|\, d(\boldsymbol{u}, \boldsymbol{v}) < \epsilon\} \,. \tag{11.38}$$

Then we can call a map $F : \mathcal{S}_1 \to \mathcal{S}_2$ bi-Lipschitz if for all $\boldsymbol{u} \in \mathcal{S}_1$ there exists $r > 0$ and a constant $K \geq 1$ such that for all $\boldsymbol{x}, \boldsymbol{y} \in \mathcal{B}(\boldsymbol{u}, r)$ we have:

$$\frac{1}{K} d_1(\boldsymbol{x}, \boldsymbol{y}) \leq d_2\left(F(\boldsymbol{x}), F(\boldsymbol{y})\right) \leq K d_1(\boldsymbol{x}, \boldsymbol{y}) \,, \tag{11.39}$$

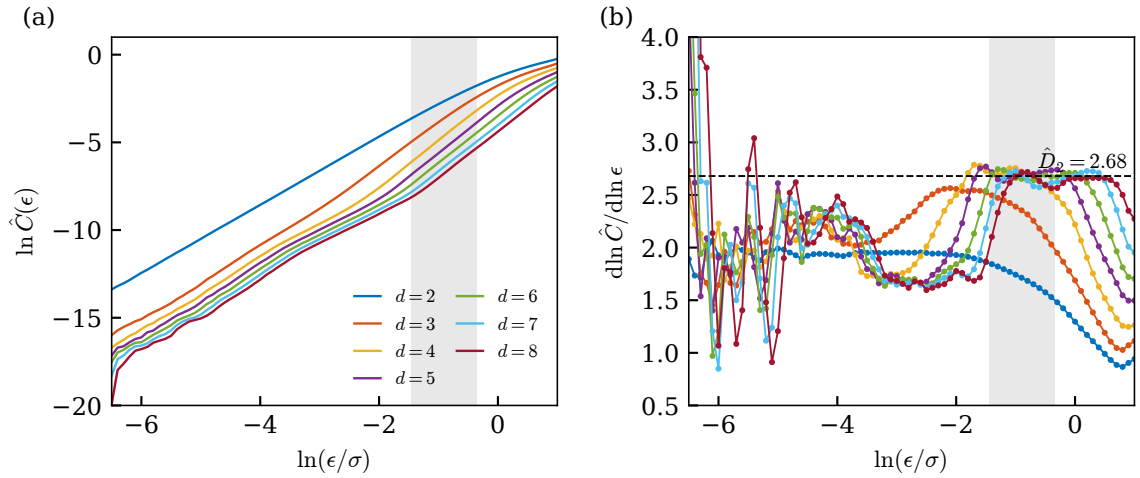


Figure 11.19: Correlation dimension of the two-well oscillator record of Section 1.4.1. Delay $\tau = 5$ samples, $n = 30{,}000$ reconstructed points, Theiler window $w = 5d$ samples. The record is normalized to unit variance, so ϵ is in units of its standard deviation σ. (a) $\ln \hat{C}(\epsilon)$ for $d = 2, \ldots, 8$; (b) the local slopes of the same curves, which are what the estimate is read from. The shaded band is the scaling range $\ln(\epsilon/\sigma) \in [-1.45, -0.35]$. Over it the slope is flat and independent of d for $d \geq 4$, giving $\hat{D}_2 = 2.68 \pm 0.03$. The trough near $\ln(\epsilon/\sigma) = -3.0$ follows from sampling locked at 16 points per forcing period, and the noise-free model sampled the same way reproduces it (Fig. 11.20); the break-up below -4.3 is the noise floor and the exhaustion of pairs. Script: `python/mkfig_twowell_d2.py`.

where d_i is the distance metric in $\mathcal{S}_i$. From this perspective, the measures μ_1 and μ_2 are *equivalent* if there exists a locally bi-Lipschitz injection $F : \mathcal{S}_1 \to \mathcal{S}_2$, such that for all $\boldsymbol{u} \in \mathcal{S}_2$ we have

$$\mu_1(F^{-1}(\boldsymbol{u})) = \mu_2(\boldsymbol{u})\,. \tag{11.40}$$

Consider geometric objects like a line segment $L \in \mathbb{R}$, area $A \in \mathbb{R}^2$, or, in general, hyper-volume $V \in \mathbb{R}^m$, then for any $\boldsymbol{u} \in V$ we can observe that (see Fig. 11.21)

$$\mu\left(\mathcal{B}(\boldsymbol{u}, \epsilon)\right) \sim \epsilon^m\,. \tag{11.41}$$

This motivates the definition of *pointwise dimension* at a point $\boldsymbol{u} \in \mathcal{A}$ as:

$$D_\mu(\boldsymbol{u}) = \lim_{\epsilon \to 0} \frac{\ln \mu(\mathcal{B}(\boldsymbol{u}, \epsilon))}{\ln \epsilon}\,. \tag{11.42}$$

This, in turn, motivates the generalization of *pointwise dimension* over the whole attractor $\mathcal{A}$ as

$$D_{\mathrm{P}} = \int_{\mathcal{A}} D_\mu(\boldsymbol{u}) \mathrm{d}\mu(\boldsymbol{u}) = \lim_{\epsilon \to 0} \frac{\int_{\mathcal{A}} \ln \mu_{\boldsymbol{u}}(\epsilon)\, \mathrm{d}\mu(\boldsymbol{u})}{\ln \epsilon} = \lim_{\epsilon \to 0} \frac{\langle \ln \mu_{\boldsymbol{u}}(\epsilon) \rangle_{\mathcal{A}}}{\ln \epsilon}\,, \tag{11.43}$$

where $\mu_{\boldsymbol{u}}(\epsilon) = \mu(\mathcal{B}(\boldsymbol{u}, \epsilon))$, and the exchange of the limit and the integral in the second equality is assumed. Which generalized dimension the pointwise dimension is related to is not a matter of opinion. Comparing with the asymptotics

$$D_1 \sim \frac{\langle \ln p_i \rangle}{\ln \epsilon}\,, \qquad \text{and} \qquad D_2 \sim \frac{\ln \langle p_i \rangle}{\ln \epsilon}\,, \tag{11.44}$$

shows that the averaged pointwise dimension is the *information dimension* D_1 written in terms of balls instead of boxes, since the measure inside the ϵ-ball is $P_i(\epsilon) = \int_{\mathcal{B}(\boldsymbol{x}_i, \epsilon)} p_i \, \mathrm{d}\boldsymbol{x} = \int_{\mathcal{B}(\boldsymbol{x}_i, \epsilon)} \mathrm{d}\mu =$

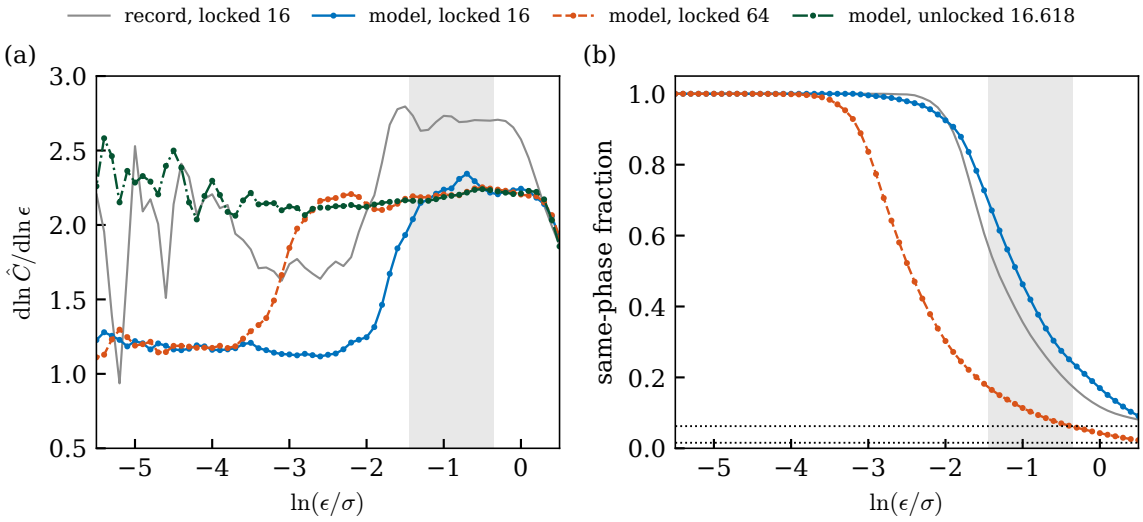


Figure 11.20: Locked against unlocked sampling. Noise-free two-well Duffing model $\ddot{x} + 0.25\dot{x} - x + x^3 = 0.3\cos t$, 200 forcing periods discarded and 6000 kept, sampled from one trajectory locked at 16 points per period (solid), locked at 64 points per period (dashed) and unlocked at 16.618 points per period (dash-dot). Reconstruction in $d = 5$ with delay $\tau = 5$ samples and Theiler window $w = 25$ samples at 16 points per period, both scaled with the rate to fixed physical time; correlation sum from 2000 random reference points against all points, ϵ in units of the standard deviation σ. Gray: the measured record of Fig. 11.19, $n = 30{,}000$, same d, τ and w. Shaded: the scaling range of Eq. (11.37). (a) Local slope of $\ln \hat{C}$. Locked sampling falls from the plateau to about one less than it; the faster locked rate moves the crossover but not the level; unlocked sampling at nearly the same density has no trough. (b) Fraction of pairs closer than ϵ taken at the same forcing phase, $(i - j)$ divisible by the number of samples per period; dotted, the chance levels 1/16 and 1/64. Under unlocked sampling no two samples share a phase, so that curve is identically zero and not drawn. Script: `python/mkfig_phaselock.py`.

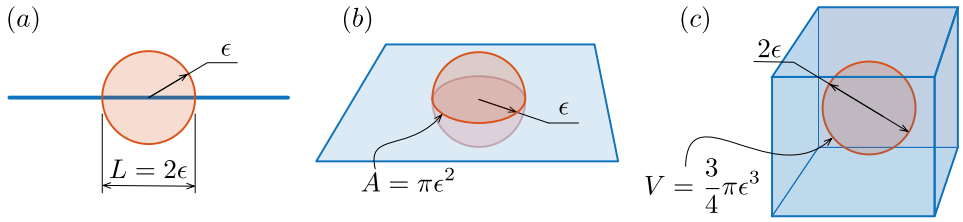


Figure 11.21: The measure inside the ϵ-ball scales like: ϵ for a line segment (a), ϵ^2 for a surface area (b), and ϵ^3 for a three-dimensional volume (c).

$\mu_{\boldsymbol{x}_i}(\epsilon)$: the average of the logarithm gives D_1, the logarithm of the average gives D_2. More is true: for an ergodic measure with a well-defined pointwise dimension, $D_\mu(\boldsymbol{u}) = D_1$ for μ-almost every $\boldsymbol{u}$, so the pointwise dimension is constant on the attractor and equal to the information dimension (Young [229]; Farmer, Ott, and Yorke [59]). Statements in the applied literature that identify it with the correlation dimension [143] are loose: $D_2 \leq D_1$, with equality only when the measure is uniform on its support.

11.6.5 Average Pointwise Dimension Estimate

Given a reconstructed phase space vectors $\{\boldsymbol{x}\}_{n=1}^N$, we can estimate the measure μ for ϵ-ball $\mathcal{B}_i(\epsilon) = \mathcal{B}(\boldsymbol{x}_i, \epsilon)$ centered at $\boldsymbol{x}_i$ as:

$$\hat{\mu}_i(\epsilon) = \frac{1}{N}\sum_{j=1}^{N} \Theta\left(\epsilon - \|\boldsymbol{x}_i - \boldsymbol{x}_j\|\right) = \frac{N\left(\mathcal{B}_i(\epsilon)\right)}{N} = \frac{N_i(\epsilon)}{N}\,, \tag{11.45}$$

where Θ is the Heaviside function, and $N_i(\epsilon) = N(\mathcal{B}_i(\epsilon))$ is the number of point inside the ball $\mathcal{B}_i(\epsilon)$. Then, the average log of pointwise measures inside the ϵ-ball can be estimated as:

$$\begin{aligned} \langle \ln\, \hat{\mu}_i(\epsilon)\rangle &= \frac{1}{N}\sum_{i=1}^{N} \ln\left(\frac{1}{N}\sum_{j=1}^{N}\Theta\left(\epsilon - \|\boldsymbol{x}_i - \boldsymbol{x}_j\|\right)\right) \\ &= \frac{1}{N}\sum_{i=1}^{N} \ln N_i(\epsilon) - \ln N\,. \end{aligned} \tag{11.46}$$

Thus, we can estimate the average pointwise dimension as:

$$\hat{D}_{\mathrm{P}} = \lim_{\epsilon\to 0}\frac{\langle \ln\, \hat{\mu}_i(\epsilon)\rangle}{\ln\epsilon} = \lim_{\epsilon\to 0}\frac{\frac{1}{N}\sum_{i=1}^{N}\ln N_i(\epsilon) - \ln N}{\ln\epsilon}\,, \tag{11.47}$$

or

$$\hat{D}_{\mathrm{P}} \sim \frac{\mathrm{d}\,\langle \ln\, \hat{\mu}_i(\epsilon)\rangle}{\mathrm{d}\ln\epsilon} = \frac{\sum_{i=1}^{N}\mathrm{d}\ln N_i(\epsilon)}{N\mathrm{d}\ln\epsilon}\,. \tag{11.48}$$

Algorithms using fixed distance-based metrics (e.g., $\hat{D}_{\mathrm{P}}$ estimator is using fixed ϵ to estimate the number of points inside the corresponding ϵ-balls) is called *fixed distance* method or algorithm. There are also algorithms that are called *fixed mass* algorithms. In fixed mass algorithms, instead of counting points inside the fixed size ϵ-balls, we first fix the number of points n we want to be contained inside the balls, and then find the size of these balls $\epsilon_i(n)$ centered at $\boldsymbol{x}_i$ and containing n points. Using a fixed mass algorithm, we can estimate the average log of $\epsilon_i(n)$-ball as:

$$\langle \ln\epsilon_i(n)\rangle = \frac{1}{N}\sum_{i=1}^{N}\ln\epsilon_i(n)\,, \tag{11.49}$$

where $\epsilon_i(n)$ is defined as the distance between $\boldsymbol{x}_i$ and its n-th nearest neighbor $\boldsymbol{x}_{j(n)}$:

$$\epsilon_i(n) = \left\| \boldsymbol{x}_i - \boldsymbol{x}_{j(n)} \right\| . \tag{11.50}$$

Thus, we can try to estimate fixed mass-based average pointwise dimension as:

$$\bar{D}_{\mathrm{P}} = \lim_{n\to 1} \frac{\ln(n/N)}{\langle \ln \epsilon_i(n) \rangle} = \lim_{n\to 1} \frac{N \ln(n/N)}{\sum_{i=1}^{N} \ln \epsilon_i(n)} . \tag{11.51}$$

However, as $n \to 1$, $\epsilon_i(n) \to \epsilon_i(1)$ and we cannot make it smaller. Therefore,

$$\bar{D}_{\mathrm{P}} \approx \frac{N \ln(1/N)}{\sum_{i=1}^{N} \ln \epsilon_i(1)} = \frac{-N \ln N}{\sum_{i=1}^{N} \ln \epsilon_i(1)} . \tag{11.52}$$

Or more precisely, to let $\langle \epsilon_i(1) \rangle \to 0$, we need to let $N \to \infty$, then

$$\bar{D}_{\mathrm{P}} = \lim_{N\to\infty} \frac{N \ln N}{\sum_{i=1}^{N} \ln(1/\epsilon_i(1))} . \tag{11.53}$$

Please note, if we want to use the above expression for the estimation, then we really need to worry about eliminating temporarily correlated nearest neighbors from the calculation. However, if the sampling time period is reasonable and we let $N \to \infty$ or some fairly large number, we may not need to worry about temporal correlations since the nearest neighbors will most likely be there due to geometry of $\mathcal{A}$.

In practice neither $\epsilon_i(1) \to 0$ nor $N \to \infty$ is available, so $\bar{D}_{\mathrm{P}}$ must be read off an identified scaling region, exactly as D_2 was. The constraints are the same ones, and beyond the artifacts caused by noise they arise from the finite size of the data set.

Both algorithms return zero at both ends of the range of ϵ. For the fixed distance algorithm, $N_i(\epsilon) = 1$ for $0 < \epsilon < d_{\min}$, where $d_{\min}$ is the smallest nearest neighbor distance in the record; hence $\hat{D}_{\mathrm{P}} = 0$ there, and $\hat{D}_{\mathrm{P}} \to 0$ as $\epsilon_i(1) \to d_{\min}$. At the other end $N_i(\epsilon) = N$ once $\epsilon > d_{\max}$, so again $\hat{D}_{\mathrm{P}} \to 0$ as $\epsilon \to d_{\max}$. A flat region between the two collapses is the only part of the curve that carries a dimension. For the fixed mass algorithm the corresponding limit is $n \to 1$, where $\bar{D}_{\mathrm{P}} \to N \ln N / \sum_i \ln(1/\epsilon_i(1))$, which is accurate only as $N \to \infty$.

Grassberger [78] removes part of that finite-sample bias in the fixed mass estimator by replacing $\ln(n/N)$ with $\psi(n) - \ln N$, where ψ is the *digamma function*. The estimate actually computed is therefore

$$\bar{D}_{\mathrm{P}}(n) = \frac{N\,(\psi(n) - \ln N)}{\sum_{i=1}^{N} \ln \epsilon_i(n)} . \tag{11.54}$$

Time-correlated neighbors should be excluded from both algorithms, for the reason given for the correlation sum. The exclusion applies to the measure $\hat{\mu}(\epsilon)$ and to the average ball size $\epsilon(n)$ alike. Fig. 11.22 shows both algorithms applied to the Lorenz and two-well Duffing records.

11.6.6 Other Generalized Dimension Estimates

The generalized dimension

$$D_q = \lim_{\epsilon\to 0} \frac{1}{q-1} \frac{\ln C_q(\epsilon)}{\ln \epsilon} \tag{11.55}$$

can be estimated for different values of q to explore the multi-fractal nature of the attractor. The estimation of box-counting $D_0 = D_F$ is problematic if we do not have access to the abundant clean

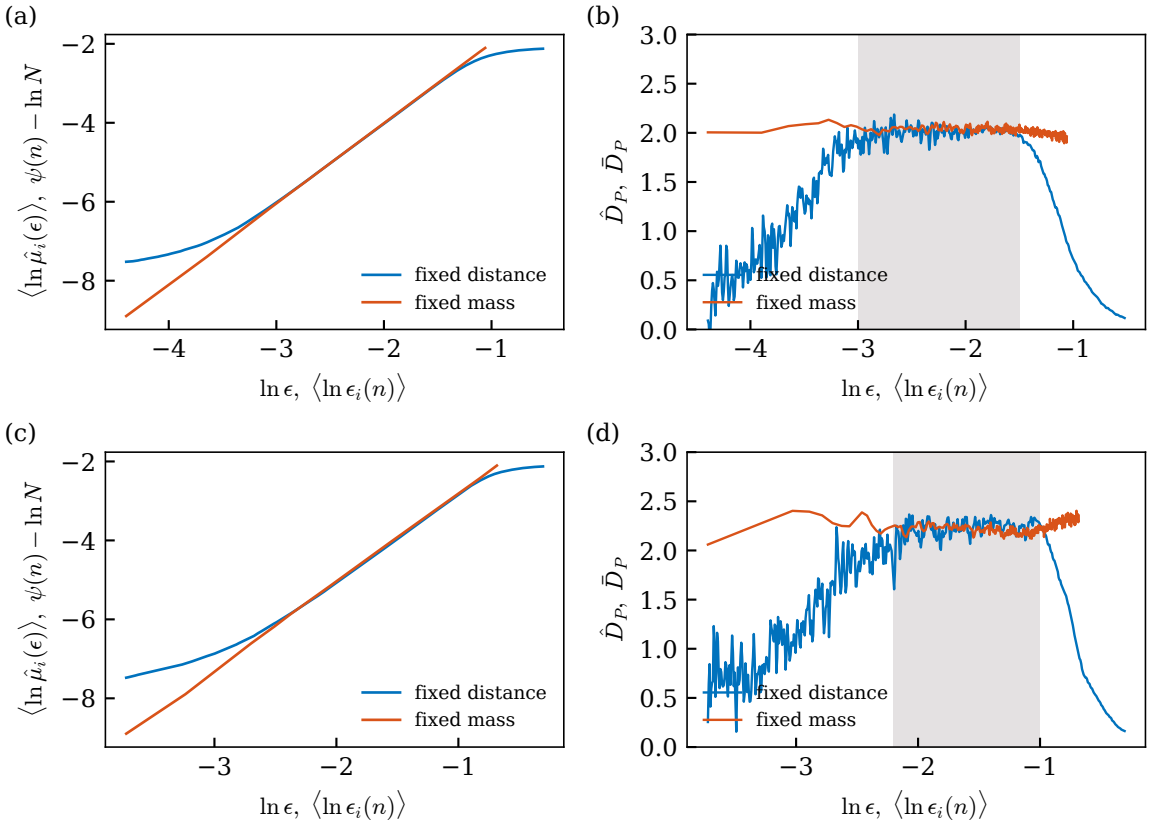


Figure 11.22: Pointwise dimension estimation for the Lorenz (a–b) and the two-well Duffing (c–d) signals, by the fixed distance and the fixed mass algorithms (`pwdim`, Appendix A.3; $k = 500$ bins or neighbors, 4096 reference points, the records of Fig. 11.18 normalized to unit variance, $d = 3$ and 4, $\tau = 8$ and 14, $w = 3\tau$). (a),(c) the measures; (b),(d) their local slopes, with the scaling ranges shaded. Over $\ln \epsilon \in [-3, -1.5]$ the Lorenz slopes are $\hat{D}_\mathrm{P} = 2.00 \pm 0.07$ (fixed distance) and $\bar{D}_\mathrm{P} = 2.04 \pm 0.03$ (fixed mass); over $[-2.2, -1.0]$ the Duffing slopes are 2.22 ± 0.10 and 2.20 ± 0.04, the errors being the standard deviation of the local slopes across the range. Both estimate D_1, so they lie above the D_2 of Fig. 11.18, and both are below the Kaplan–Yorke dimensions of the models, 2.06 and 2.33. Script: `python/mkfig_DmuLDs.py`.

data. For the D_1 we can use the pointwise dimension estimates, and for the D_2 we can use the correlation dimension. However, in general, to estimate the generalized dimension D_q for $q \geq 1$ we can use two different approaches: (1) is similar to correlation sum estimation, and (2) uses pointwise dimension procedure. The correlation sum estimation used the distribution of the inter-point lengths in the data. We can follow the same example and define the estimate of C_q as

$$\hat{C}_q(\epsilon) = \frac{1}{N}\sum_{i=1}^{N}\left[\frac{1}{N_i}\sum_{j\notin[i-w,i+w]}\Theta\left(\epsilon - \|\boldsymbol{x}_i - \boldsymbol{x}_j\|\right)\right]^{q-1}, \tag{11.56}$$

where $N_i = N - w - \min(i-1, w, N-i) - 1$, which is equal to $N_i = N - 2w - 1$ when $w < i < N - w$. For, $q > 2$ this leads to the biased estimator due to nontrivial power in the second sum. In general, this algorithm tends to give lower dimension estimates than the C_2 algorithm, and the estimates decrease with q ($\hat{D}_q < \hat{D}_l$ for $q > l$) even for sets that are not multifractal. The theoretical spectrum is non-increasing in q, so a *strictly* decreasing estimate on a set expected to be uniform is an estimator artifact, not evidence of multifractality. Its origin is the power $q-1$ applied to the local count in Eq. (11.56): at small ϵ the counts are small integers with large relative scatter, and raising them to a power above one weights the few dense neighborhoods, which have a lower local scaling exponent at that ϵ, far more than the sparse ones. The bias therefore grows with q and with decreasing ϵ, and it should be checked by comparing $\hat{D}_q$ against $\hat{D}_2$ on a surrogate of known dimension (a uniform distribution embedded in the same d) with the same N.

For $q \leq 1$, fractal dimension estimation is problematic as some of the sums may be zero. Here, a fixed mass algorithm is more appropriate, working for $q \leq 1$ and providing more consistent results. First, we assume that we have a total of N phase space points, and we want to use M query points for pointwise estimation. Then, we estimate the measure inside the range of ϵ-balls for each query point $\{\boldsymbol{x}_i\}_{i=1}^{M}$ as

$$\hat{\mu}_i(\epsilon) = \frac{1}{N}\sum_{k=1}^{K}\Theta\left(\epsilon - \epsilon_i(k)\right) = \frac{1}{N}\sum_{k=1}^{K}\Theta\left(\epsilon - \left\|\boldsymbol{x}_i - \boldsymbol{x}_{n(k)}\right\|\right), \tag{11.57}$$

where $\epsilon_i(k) = \left\|\boldsymbol{x}_i - \boldsymbol{x}_{n(k)}\right\|$, $\boldsymbol{x}_{n(k)}$ is the k-th nearest neighbor of $\boldsymbol{x}_i$, and K is large enough so that $\langle\epsilon_i(K)\rangle$ is close to the attractor scale. Then, the generalized correlation sum can be estimated as:

$$\bar{C}_q(\epsilon) = \frac{1}{M}\sum_{i=1}^{M}\left[\frac{1}{N}\sum_{k=1}^{K}\Theta\left(\epsilon - \epsilon_i(k)\right)\right]^{q-1}. \tag{11.58}$$

Finally, to find D_q, we need to identify a scaling range at intermediate length scales, or look at the plateau of

$$\hat{D}_q(\epsilon) = \frac{\mathrm{d}\ln\hat{C}_q(\epsilon)}{\mathrm{d}\ln\epsilon}, \tag{11.59}$$

or

$$\bar{D}_q(\epsilon) = \frac{\mathrm{d}\ln\bar{C}_q(\epsilon)}{\mathrm{d}\ln\epsilon}. \tag{11.60}$$

These calculations are still problematic if done for $\epsilon < \max_i \epsilon_i(1)$, the largest nearest-neighbor distance in the set, since some query points then have empty balls. Therefore, we need to make sure that we only consider (or count) points that have nonzero measures for the minimum ϵ used in the calculation. Alternatively, we need to increase the minimum ϵ, so that $\epsilon_i(1) > \epsilon_{\min}$ for all i or points considered in the calculation.

11.7 Embedding Dimension from Fractal Dimension

The previous chapter discussed that the estimation of the embedding dimension could also be done by considering how it affects the estimation of some nonlinear metrics. Since the slopes of scaling regions determine the corresponding correlation or pointwise dimensions, insufficiently large embedding dimension will cause the changes in these slopes due to projections. Only after reaching the sufficient embedding dimension will the slopes stabilize and remain the same even if we increase the embedding dimension. Ding *et al.* [46] examine when that plateau sets in and find that it can begin at an embedding dimension well below the $d > 2D_F$ bound, which makes the plateau a useful practical diagnostic rather than merely a consistency check. Thus, we can use the estimated correlation sum or pointwise measure $\hat{\mu}(\epsilon)$ to find the minimally necessary embedding dimension by observing at which embedding dimension the slopes of the metric becomes invariant to the increase in the embedding dimension. Examples of this calculation for the Lorenz's and Duffing's signals are shown in Fig. 11.23 using correlations sum. From the trends in the plots, using this criterion the minimally necessary embedding dimension is three for both records: the plateau of the local slope is already at its final value at $d = 3$ for the Duffing record as well, although the false-nearest-neighbor criterion of Chapter 10 asks for $d = 4$ there. This is the behavior Ding *et al.* describe: the correlation dimension saturates at an embedding dimension below the one the embedding theorems require, so the plateau is a lower bound on a safe d, not a substitute for one. Examples of applying fixed-distance- and fixed-mass-based pointwise measure estimation to Lorenz and two-well Duffing's signals are shown in Fig. 11.24.

Problems

Problem 11.1

Core

Given a periodic solution, show that $D_2 \approx 1$. Now generate a quasi-periodic time series and show $D_2 \approx n$, where n is the number of incommensurate frequencies. Discuss how closely you can get the result. Remember that the estimation of D_2 requires that you choose an embedding dimension d, fix a delay τ, and generate C_2. All estimates should be presented in the form, $D_2 = \widehat{D}_2 \pm E$, where E is the standard error computed as in Section 11.6.2, together with the scaling range over which it applies. You should also always look at PSD, $S(f)$, and autocorrelation, $R(\tau)$.

Problem 11.2

Computational

Take at least two chaotic, or otherwise "complex" time series from the Hénon Map ($a = 1.4$ and $b = 0.3$), Lorenz, double-well Duffing, or Chua's Circuit models and estimate D_2 and $\bar{D}_p$. Discuss the quality of your result and how does it change with the addition of noise. Does the data appear to be "fractal?" Over what range of length scales does your data appear to have a scaling region? Estimate other Generalized dimensions D_q for both $q > 1$ and $q < 1$; what can you say about the results?

Problem 11.3

needs Ch. 12 Extension

Now we want to estimate *Kolmogorov-Sinai entropy*, which according to Ruelle's inequality [170] is bounded above by the sum of the *positive* Lyapunov exponents discussed in the next chapter, $h_{\mathrm{KS}} \leq \sum_{\lambda_i > 0} \lambda_i$ (the sum of *all* exponents is negative for a dissipative system). It is usually difficult

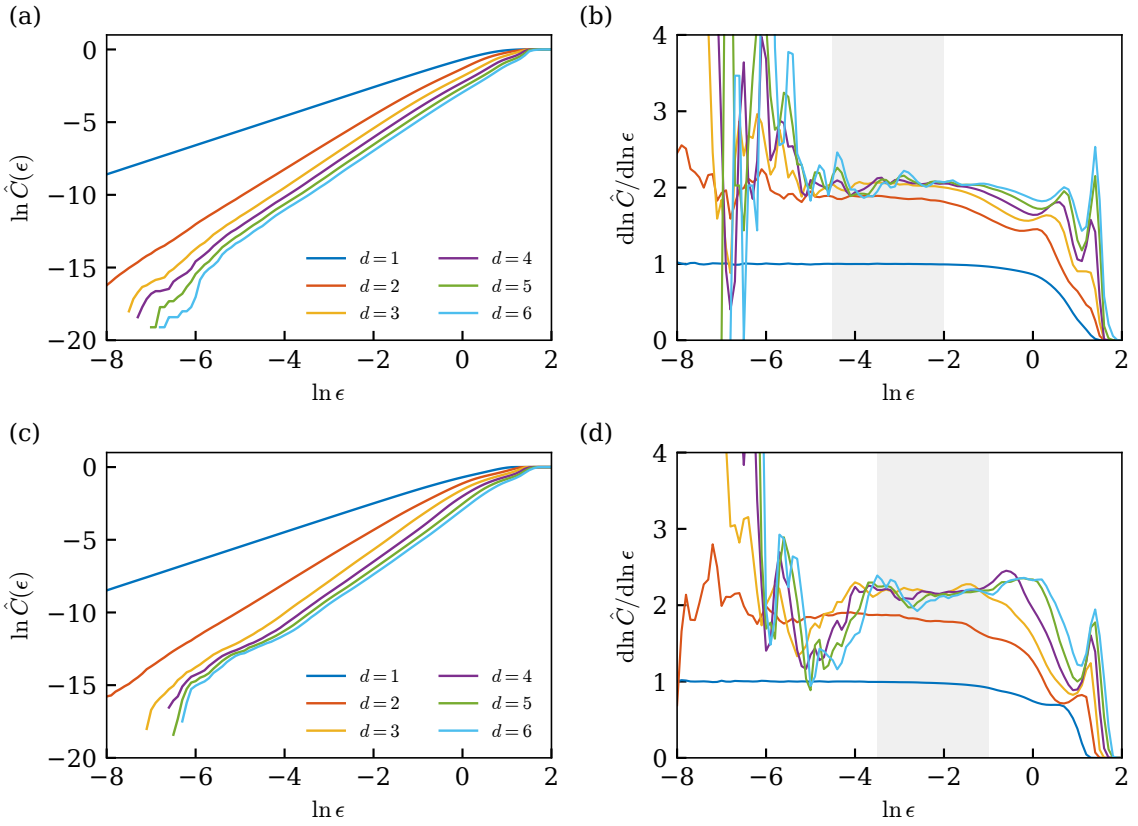


Figure 11.23: Correlation sums (a),(c) and their local slopes (b),(d) for the Lorenz and two-well Duffing records of Fig. 11.18 embedded in $d = 1, \ldots, 6$. Averaged over the scaling ranges of Fig. 11.18 (shaded), the slope is 1.00, 1.86, 2.03, 2.05, 2.06, 2.07 for the Lorenz record and 0.97, 1.78, 2.19, 2.17, 2.14, 2.15 for the Duffing record: both stop changing at $d = 3$, while the scatter of the local slopes across the range grows with d (± 0.03 at $d = 3$, ± 0.14 at $d = 6$ for the Lorenz record). Script: `python/mkfig_D2Plots2.py`.

to calculate, but one can estimate the *correlation entropy* K_2, a lower bound on the KS entropy that is close to it in practice. Grassberger and Procaccia [80] showed that it is given by the rate at which the correlation sum decreases when one delay coordinate is added:

$$K_2 = \lim_{d\to\infty} \lim_{\epsilon\to 0} \lim_{N\to\infty} \frac{1}{\tau\Delta t} \ln \frac{C_2(d,\epsilon)}{C_2(d+1,\epsilon)}\,, \tag{11.61}$$

where $C_2(d,\epsilon)$ is the correlation sum for an embedding dimension d and $\tau\Delta t$ is the delay time, so that K_2 has units of inverse time (nats per unit time; divide by $\ln 2$ for bits). Without the factor $1/(\tau\Delta t)$ the quantity is an entropy per delay step and cannot be compared with Lyapunov exponents in physical units. In the scaling region the curves $\ln C_2(d,\epsilon)$ for successive d are parallel lines of slope D_2, vertically separated by $K_2\tau\Delta t$; Chapter 18 obtains the same K_2 from the distribution of diagonal line lengths in a recurrence plot.

Choose one of the time series from continuous ODEs from Problem 11.2. Plot family of curves $\ln C_2(\epsilon)$ vs. $\ln\epsilon$ for different values of embedding dimension from 1 to 8. How does the slope of these curves vary with d? Can you estimate the KS entropy?

Problem 11.4

Core

A set with a known, nontrivial dimension spectrum, on which every estimator of this chapter can be tested. Build the middle-thirds Cantor set of Fig. 11.9 but visit its two halves with unequal probability: at each stage the measure of an interval is split in the ratio $p:(1-p)$ between its left and right thirds.

1. At stage n the set consists of 2^n intervals of length 3^{-n}, of which $\binom{n}{j}$ carry measure $p^j(1-p)^{n-j}$. Show from Eq. (11.13) that $D_q = \dfrac{\ln\left(p^q+(1-p)^q\right)}{(q-1)\ln(1/3)}$, and hence that $D_0 = \ln 2/\ln 3$ independently of p, $D_1 = -[p\ln p+(1-p)\ln(1-p)]/\ln 3$, and $D_2 = -\ln\left(p^2+(1-p)^2\right)/\ln 3$. Evaluate all three for $p=0.3$ (you should find 0.631, 0.556 and 0.496) and verify that D_q is non-increasing in q. For which p do they coincide?

2. Generate $N = 20{,}000$ points of this measure by the random iteration $v \leftarrow v/3$ with probability p and $v \leftarrow v/3+2/3$ with probability $1-p$, keeping every iterate after a short transient. Successive iterates are not independent, since each keeps the ternary digits of its predecessor, but they are rarely close, so a Theiler window changes nothing here. Compute the correlation sum Eq. (11.32) directly in one dimension for ϵ from 10^{-3} to 0.3 and fit the slope over $[10^{-3}, 0.1]$. You should find $D_2 \approx 0.49$, against 0.633 for the equal-weight set with $p=0.5$.

3. Compute Takens' estimator Eq. (11.36) with $\epsilon_0 = 0.05$ and its formal error $\hat{D}_{\mathrm{T}}/\sqrt{M}$. You should find an estimate a few percent *below* the exact D_2 with a formal error far smaller than that shortfall. Name the two effects, both described in Section 11.6.2, that account for the discrepancy, and demonstrate one of them by plotting the local slope of $\ln\hat{C}$ against $\ln\epsilon$ and reading its period in $\ln\epsilon$.

4. Estimate D_1 by the fixed-mass pointwise algorithm and D_0 by box counting on the same points. Which of the three is the hardest to estimate at this N, and why does the ordering $D_0 \geq D_1 \geq D_2$ survive the estimation errors here while the D_q estimates for $q>2$ in Eq. (11.56) may not?

Problem 11.5

Computational

The Eckmann–Ruelle bound Eq. (11.35) is a statement about the estimator, not the data. Show it on data with no attractor at all.

1. Generate $N = 4000$ samples of Gaussian white noise and embed them with $\tau = 1$ in $d = 2, 4, 6, 8, 10$. Compute the correlation sum with `c2r` (Appendix A.2, no Theiler window) and the local slope $\mathrm{d}\ln\hat{C}/\mathrm{d}\ln\epsilon$. Record the slope over the range where $\hat{C}$ lies between 10^{-4} and 10^{-2}.

2. Plot that slope against d, with the line $D_2 = d$ and the horizontal line $2\log_{10} N = 7.2$. You should find the slope tracking d closely up to $d = 6$ and falling increasingly short of it beyond, so that noise embedded in ten dimensions returns an estimate near 8.5 rather than 10. Explain from the number of pairs available below a given $\hat{C}$ why the shortfall must set in near the bound.

3. Repeat with $N = 400$ and $N = 40{,}000$ and show that the d at which the slope departs from d moves with $2\log_{10} N$. State what a reported "$D_2 = 6.8$ from $N = 4000$ points" means for a record whose dynamics are unknown.

4. Now take the Lorenz x record of 4000 points and embed in $d = 3, \ldots, 10$. The slope should stay near 2 at every d. This is the plateau of Section 11.7; explain why it, and not the value at any single d, is the evidence for a low-dimensional attractor, and why noise can never produce it.

Problem 11.6

Experimental

Correlation dimension of the two-well oscillator record of Section 1.4.1, following the reporting procedure of Section 11.6.2.

1. With the delay of Problem 9.6 and the Theiler window of Problem 10.6, compute $\ln\hat{C}(\epsilon)$ with `c2r` for $d = 2, \ldots, 7$ and plot the local slopes. Identify the scaling range on which the slopes for $d \geq 4$ agree and are flat, in units of the attractor size (the standard deviation of the record).

2. Compute $2\log_{10} N$ for the record length. Is the plateau value below it, and by how much? If the record is long, repeat with its first quarter and state how the scaling range shrinks.

3. Apply Takens' estimator with ϵ_0 at three values spanning the upper half of the scaling range and on two non-overlapping halves of the record. Report $\hat{D}_2 \pm E$ with E the spread, together with N, d, τ, w and the scaling range. Compare with the model value for the Duffing oscillator (Kaplan–Yorke dimension 2.33 at the parameters of Section 1.4, correlation dimension 2.16 measured in Fig. 11.18) and with the expectation of Section 1.4.1.

4. Below the scaling range the local slope first *falls* and only then rises, which is not the textbook noise picture. Locate the minimum and express its ϵ as a fraction of the standard deviation of the record. Then compute the distribution of the one-sample displacement $\|\boldsymbol{x}_{i+1} - \boldsymbol{x}_i\|$ in the same reconstruction, and compare its median with the scale at which the slope leaves the plateau. Section 11.6.3 attributes the dip to sampling locked to the forcing. Test that by counting, at each ϵ, the fraction of close pairs whose indices agree modulo 16, and by repeating the calculation on the simulated two-well Duffing record sampled at 16, at 64 and at a non-integer number of points per forcing period, over equal durations.

5. Below ϵ_{noise} the slope rises again and then breaks up, and the break-up moves to larger ϵ as d grows. Read ϵ_{noise} as a fraction of the standard deviation and compare it with the measurement-noise level estimated from the spectral floor in Problem 4.6. Do the two estimates of the noise agree? How many pairs remain below ϵ_{noise}, and is that enough to read a slope?

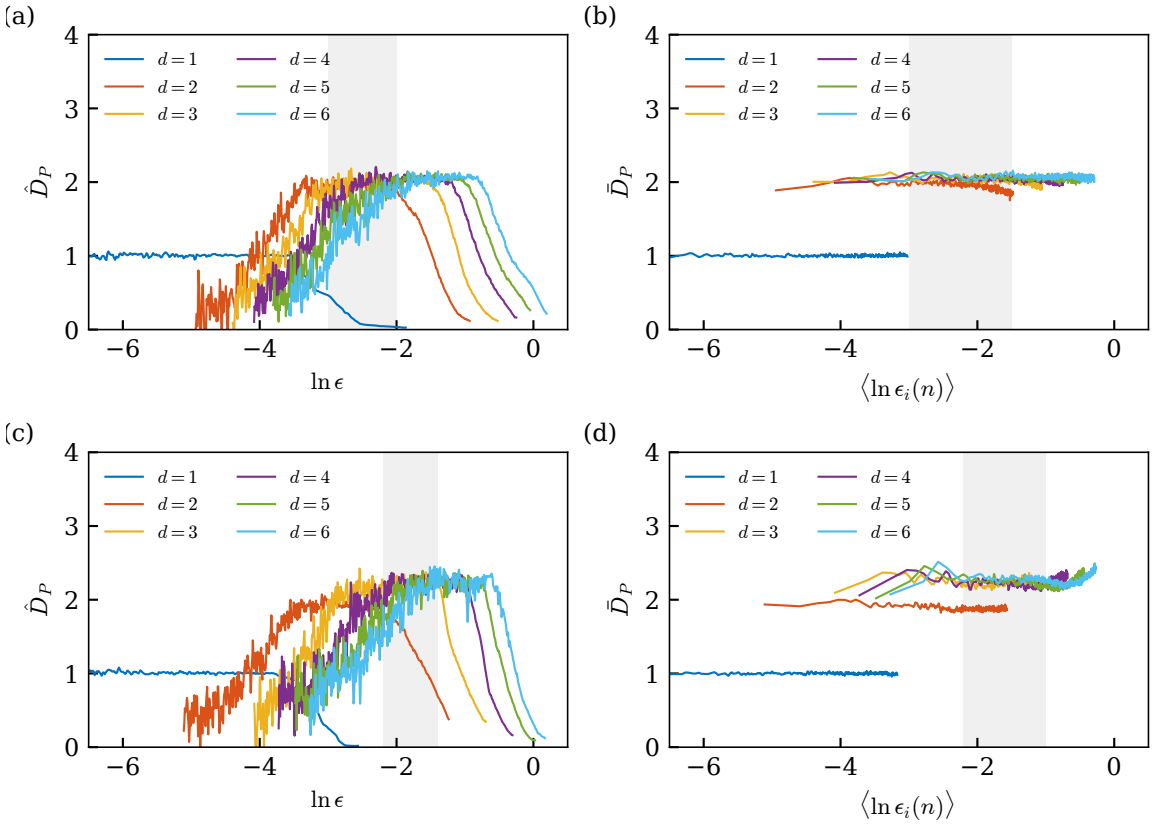


Figure 11.24: Local slopes of the pointwise measure for the Lorenz (a–b) and two-well Duffing (c–d) records of Fig. 11.22 embedded in $d = 1, \ldots, 6$: fixed distance on the left, fixed mass on the right. Averaged over the shaded ranges the fixed mass estimate is 1.93, 2.04, 2.05, 2.06, 2.08 for $d = 2, \ldots, 6$ (Lorenz) and 1.88, 2.24, 2.20, 2.23, 2.28 (Duffing), flat from $d = 3$ on. The fixed distance estimate is read on the narrower range $\ln \epsilon \in [-3, -2]$ and $[-2.2, -1.4]$, because `pwdim` counts at most k neighbors, so $\langle \ln \hat{\mu}_i \rangle$ saturates at $\epsilon_i(k)$ and its slope then falls to zero; it drifts downward for $d \geq 5$ (1.94, 1.99, 1.91, 1.74, 1.53 for the Lorenz record) as the same k neighbors are spread over more dimensions. Script: `python/mkfig_DmuLDDs.py`.

12 Stability and Lyapunov Exponents

Recall the analysis of *stability* for the *fixed point* of a map:

$$\boldsymbol{x}_{n+1} = \boldsymbol{F}\left(\boldsymbol{x}_n\right) , \tag{12.1}$$

where $\boldsymbol{x}_n \in \mathbb{R}^m$. The period-one fixed point needs to satisfy $\boldsymbol{x}^* = \boldsymbol{F}(\boldsymbol{x}^*)$. For example, consider the Hénon map [92]:

$$\begin{aligned} x_{n+1} &= a - x_n^2 + b y_n \\ y_{n+1} &= x_n \, , \end{aligned} \tag{12.2}$$

where $\boldsymbol{x}_n = [x, y]^{\mathrm{T}}$. Fixed point for the period-one orbit is determined by solving the following equations:

$$\begin{aligned} x &= a - x^2 + b y \\ y &= x \, , \end{aligned} \tag{12.3}$$

which results into: $x = a - x^2 + by$ or $x^2 + (1-b)x - a = 0$. This quadratic equation has two solutions

$$x_{1,2}^* = \frac{(b-1) \pm \sqrt{(b-1)^2 + 4a}}{2} \, , \tag{12.4}$$

when $(b-1)^2 + 4a \geq 0$ or $a \geq \frac{-(b-1)^2}{4}$ as shown in Fig. 12.1. That is, as a crosses over the value $\frac{-(b-1)^2}{4}$ we would experience the *saddle-node bifurcation* during which we transition from having no fixed points to having one stable and one unstable fixed points.

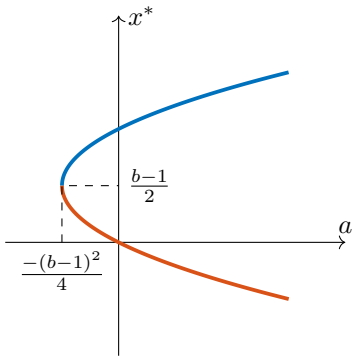


Figure 12.1: Saddle-node bifurcation generating one stable, one unstable fixed points

12.1 Stability of Periodic Orbits

To study the stability of fixed points, we perturb the $\boldsymbol{x}^*$ solution by a small vector $\boldsymbol{\delta}$, $\|\boldsymbol{\delta}\| \ll 1$,

$$\boldsymbol{x}_n = \boldsymbol{x}^* + \boldsymbol{\delta}_n \tag{12.5}$$

and study how this perturbation behaves as we let $n \to \infty$. Thus, we plug Eq. (12.5) into Eq. (12.1) to get:

$$\begin{aligned} \boldsymbol{x}^* + \boldsymbol{\delta}_{n+1} &= \boldsymbol{F}(\boldsymbol{x}^* + \boldsymbol{\delta}_n)\,, \\ &= \boldsymbol{F}(\boldsymbol{x}^*) + D\boldsymbol{F}(\boldsymbol{x}^*)\boldsymbol{\delta}_n + \mathcal{O}(\|\boldsymbol{\delta}_n\|^2)\,. \end{aligned} \tag{12.6}$$

Now, remembering that $\boldsymbol{x}^* = \boldsymbol{F}(\boldsymbol{x}^*)$ and ignoring the higher order terms, we get

$$\boldsymbol{\delta}_{n+1} = D\boldsymbol{F}(\boldsymbol{x}^*)\boldsymbol{\delta}_n = \boldsymbol{A}\boldsymbol{\delta}_n \tag{12.7}$$

where

$$\boldsymbol{A} = D\boldsymbol{F}(\boldsymbol{x}^*) = \left[\frac{\partial F_i}{\partial x_j}\right]\Bigg|_{\boldsymbol{x}=\boldsymbol{x}^*} \tag{12.8}$$

is the Jacobian matrix evaluated at $\boldsymbol{x}^*$. Since the perturbations can be expressed as:

$$\boldsymbol{\delta}_n = q_1\hat{\boldsymbol{e}}_1 + q_2\hat{\boldsymbol{e}}_2 + \cdots + q_m\hat{\boldsymbol{e}}_m = \boldsymbol{V}\boldsymbol{q}\,, \tag{12.9}$$

where columns of $\boldsymbol{V} \in \mathbb{R}^{m\times m}$ are $\hat{\boldsymbol{e}}_i$, and $\hat{\boldsymbol{e}}_i$ satisfy the following *eigenvalue problem*:

$$\boldsymbol{A}\hat{\boldsymbol{e}}_i = s_i\hat{\boldsymbol{e}}_i\,. \tag{12.10}$$

Then, $\boldsymbol{x}^*$ is *asymptotically stable* if $|s_i| < 1$ for all i (the s_i themselves may be complex), in which case $\|\boldsymbol{\delta}_n\| \to 0$ as $n \to \infty$, and *unstable* if $|s_i| > 1$ for some i. If some $|s_i| = 1$ and none exceeds one, the fixed point is *non-hyperbolic* and the linearization is inconclusive: the neglected higher-order terms of $\boldsymbol{F}$ decide, and even for the linear map a unit-modulus eigenvalue with a non-trivial Jordan block gives $\|\boldsymbol{\delta}_n\|$ growing linearly in n. In particular, if $\boldsymbol{\delta}_0 = \alpha_0\hat{\boldsymbol{e}}_i$,

$$\boldsymbol{\delta}_1 = \alpha_1\hat{\boldsymbol{e}}_i = \boldsymbol{A}\,(\alpha_0\hat{\boldsymbol{e}}_i) = \alpha_0\boldsymbol{A}\hat{\boldsymbol{e}}_i = s_i\alpha_0\hat{\boldsymbol{e}}_i\,, \tag{12.11}$$

so

$$\alpha_{n+1} = s_i\alpha_n\,. \tag{12.12}$$

Thus,

$$\alpha_n = s_i^n\alpha_0\,. \tag{12.13}$$

The eigenvalues s_i are *stability numbers*. Often, people like to write them in terms of an *exponential* as

$$s_i = e^{\lambda_i} \qquad \text{or} \qquad \lambda_i = \ln s_i\,, \tag{12.14}$$

and λ_i are called *stability exponents*. In this case, asymptotic stability is determined by the familiar requirement, for continuous time systems: $\Re\{\lambda_i\} < 0$ for all i (with $\Re\{\lambda_i\} = 0$ again inconclusive).

We can also look for Period-k fixed points (periodic orbits):

$$\{\cdots, \boldsymbol{x}_m, \boldsymbol{x}_{m+1}, \boldsymbol{x}_{m+2}, \cdots, \boldsymbol{x}_{m+k}, \cdots\}\,, \tag{12.15}$$

where

$$\boldsymbol{x}_{m+k} = \boldsymbol{x}_m = \boldsymbol{x}^*\,. \tag{12.16}$$

In other words,

$$\boldsymbol{x}^* = \underbrace{\boldsymbol{F}(\boldsymbol{F}(\cdots(\boldsymbol{F}}_{k \text{ times}}(\boldsymbol{x}^*))\cdots)\,, \tag{12.17}$$

or

$$\boldsymbol{x}^* = \boldsymbol{F}^k(\boldsymbol{x}^*)\,. \tag{12.18}$$

Then stability of Period-k orbit is determined by mapping

$$\boldsymbol{\delta}_{n+1} = \boldsymbol{A}\boldsymbol{\delta}_n\,, \tag{12.19}$$

where

$$\boldsymbol{A} = D\boldsymbol{F}^k(\boldsymbol{x}^*) = \left[\frac{\partial F_i^k}{\partial x_j}\right]\Bigg|_{\boldsymbol{x}=\boldsymbol{x}^*}\,. \tag{12.20}$$

Therefore, we look for *stability numbers*, s_i, or *exponents*, $\lambda_i = \ln s_i$, where

$$\boldsymbol{A}\hat{\boldsymbol{e}}_i = s_i\hat{\boldsymbol{e}}_i\,. \tag{12.21}$$

12.2 Stability of Chaotic Orbits

We have shown how the *stability* of periodic orbits is evaluated. However, What if we have an orbit that never repeats? Can we still talk about its stability? The answer is yes, and we can do it as outlined below.

Consider a 'fiducial' trajectory, that is a sample steady state trajectory starting from an initial point on the attractor $\boldsymbol{x}_0^*$,

$$\{\boldsymbol{x}_0^*, \boldsymbol{x}_1^*, \cdots, \boldsymbol{x}_n^*, \cdots\}\,. \tag{12.22}$$

By definition $\boldsymbol{x}_{n+1}^* = \boldsymbol{F}(\boldsymbol{x}_n^*)$. As before, we can always consider a perturbation about this fiducial trajectory, $\boldsymbol{x}_n = \boldsymbol{x}_n^* + \boldsymbol{\delta}_n$, so

$$\begin{aligned}\boldsymbol{x}_{n+1}^* + \boldsymbol{\delta}_{n+1} &= \boldsymbol{F}(\boldsymbol{x}_n^* + \boldsymbol{\delta}_n)\\ &= \boldsymbol{F}(\boldsymbol{x}_n^*) + D\boldsymbol{F}(\boldsymbol{x}_n^*)\boldsymbol{\delta}_n + \mathcal{O}(\|\boldsymbol{\delta}_n\|^2)\,,\end{aligned} \tag{12.23}$$

so, as before, neglecting higher order terms, we get

$$\boldsymbol{\delta}_{n+1} = D\boldsymbol{F}(\boldsymbol{x}_n^*)\boldsymbol{\delta}_n = \boldsymbol{A}_n\boldsymbol{\delta}_n\,, \tag{12.24}$$

where, now, the matrix $\boldsymbol{A}_n$ depends on n. Unfortunately, we cannot just look at the eigenvalues of $\boldsymbol{A}_n$, since it changes at each step. However, since

$$\begin{aligned}\boldsymbol{\delta}_n &= \boldsymbol{A}_{n-1}\boldsymbol{A}_{n-2}\boldsymbol{A}_{n-3}\cdots\boldsymbol{A}_0\boldsymbol{\delta}_0 = \left(\prod_{i=n-1}^{0}\boldsymbol{A}_i\right)\boldsymbol{\delta}_0\\ &\triangleq \boldsymbol{A}^n\boldsymbol{\delta}_0 = \boldsymbol{A}^n(\boldsymbol{x}_0^*)\boldsymbol{\delta}_0\,,\end{aligned} \tag{12.25}$$

$\boldsymbol{A}^n$ depends on $\boldsymbol{x}_0^*$ *only*. (The superscript n on $\boldsymbol{A}^n$ denotes this ordered product of n different matrices, not a matrix power; the notation is kept because it reduces to the power when the orbit is a fixed point.) Then we could look at eigenvalues of $\boldsymbol{A}^n$. Unfortunately, in practice, that is not a simple task. Instead, it is better to look at the behavior of $\|\boldsymbol{\delta}_n\|$ *directly*:

$$\begin{aligned}\|\boldsymbol{\delta}_n\|^2 &= \boldsymbol{\delta}_n^{\mathrm{T}}\boldsymbol{\delta}_n = (\boldsymbol{A}^n\boldsymbol{\delta}_0)^{\mathrm{T}}(\boldsymbol{A}^n\boldsymbol{\delta}_0)\\ &= \boldsymbol{\delta}_0^{\mathrm{T}}(\boldsymbol{A}^n)^{\mathrm{T}}(\boldsymbol{A}^n)\boldsymbol{\delta}_0 \triangleq \boldsymbol{\delta}_0^{\mathrm{T}}\boldsymbol{B}_n\boldsymbol{\delta}_0\,,\end{aligned} \tag{12.26}$$

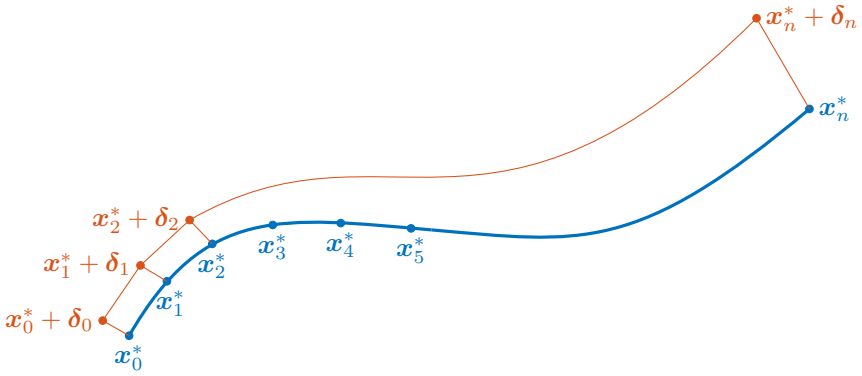


Figure 12.2: Fiducial trajectory starting from an initial point $\boldsymbol{x}_0^*$ on the attractor and the corresponding perturbation starting from $\boldsymbol{\delta}_0$

where $\boldsymbol{B}_n = (\boldsymbol{A}^n)^{\mathrm{T}} \boldsymbol{A}^n$ is a real, symmetric, positive semi-definite matrix. Therefore, all eigenvalues and eigenvectors are real, and the eigenvalues are all nonnegative; they are strictly positive whenever $\boldsymbol{A}^n$ is nonsingular. Now, we can denote these eigenvalues as s_{ni}^2 to get

$$\boldsymbol{B}_n \hat{\boldsymbol{e}}_{ni} = s_{ni}^2 \hat{\boldsymbol{e}}_{ni} \,. \tag{12.27}$$

Now, we assume a particular perturbation $\boldsymbol{\delta}_0 = \alpha_0 \hat{\boldsymbol{e}}_{ni}$, then

$$\frac{\|\boldsymbol{\delta}_n\|^2}{\|\boldsymbol{\delta}_0\|^2} = \frac{\boldsymbol{\delta}_0^{\mathrm{T}} \boldsymbol{B}_n \boldsymbol{\delta}_0}{\|\boldsymbol{\delta}_0\|^2} = \frac{\alpha_0 \hat{\boldsymbol{e}}_{ni}^{\mathrm{T}} \left(s_{ni}^2 \alpha_0 \hat{\boldsymbol{e}}_{ni}\right)}{\alpha_0^2} = s_{ni}^2 \triangleq \left(\bar{s}_i^n\right)^2 \,. \tag{12.28}$$

In other words, we define an *average constant* $\bar{s}_i$, such that if iterated n times would give the same value: $\bar{s}_i^n = s_{ni}$. Therefore, we can write

$$\bar{s}_i^n \triangleq \left(e^{\bar{\lambda}_i}\right)^n = e^{n\bar{\lambda}_i} = \frac{\|\boldsymbol{\delta}_n\|}{\|\boldsymbol{\delta}_0\|} \tag{12.29}$$

or

$$\bar{\lambda}_i = \frac{1}{n} \ln\left(\frac{\|\boldsymbol{\delta}_n\|}{\|\boldsymbol{\delta}_0\|}\right) \,, \tag{12.30}$$

where $\boldsymbol{\delta}_0$ is chosen such that $\boldsymbol{\delta}_0 \parallel \hat{\boldsymbol{e}}_{ni}$, and $\bar{\lambda}_i$ is called n-step *stability exponent*. These finite-n quantities are of interest in their own right as *local* Lyapunov exponents, which quantify predictability over finite horizons rather than asymptotically [2]. In addition, we can write

$$\bar{\lambda}_i = \frac{1}{n} \ln\left(\frac{\left(\boldsymbol{\delta}_0^{\mathrm{T}} \boldsymbol{B}_n \boldsymbol{\delta}_0\right)^{\frac{1}{2}}}{\|\boldsymbol{\delta}_0\|}\right) = \frac{1}{2n} \ln\left(\boldsymbol{\delta}_0^{\mathrm{T}} \boldsymbol{B}_n \boldsymbol{\delta}_0\right) - \frac{1}{n} \ln\|\boldsymbol{\delta}_0\| \,, \tag{12.31}$$

where the last term $\frac{1}{n} \ln\|\boldsymbol{\delta}_0\| \to 0$ for large n. Then, for each $\boldsymbol{\delta}_0 \parallel \hat{\boldsymbol{e}}_i$, we *define*

$$\lambda_i = \lim_{n\to\infty} \frac{1}{2n} \ln\left(\boldsymbol{\delta}_0^{\mathrm{T}} \boldsymbol{B}_n \boldsymbol{\delta}_0\right) \tag{12.32}$$

to be the *Lyapunov Exponent*, which we can always order as

$$\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_d \,, \tag{12.33}$$

which we call *Lyapunov Spectrum*.

Eq. (12.32) can also be written directly in terms of the Jacobian product,

$$\lambda_i = \lim_{n\to\infty} \frac{1}{2n} \ln \left\| D\boldsymbol{F}^n(\boldsymbol{x}_0^*)\boldsymbol{\delta}_0 \right\|^2 , \tag{12.34}$$

or

$$\lambda_i = \lim_{n\to\infty} \frac{1}{n} \ln \left\| D\boldsymbol{F}^n(\boldsymbol{x}_0^*)\boldsymbol{\delta}_0 \right\| , \tag{12.35}$$

where $\boldsymbol{\delta}_0 \parallel \hat{\boldsymbol{e}}_i$, and $D\boldsymbol{F}^n(\boldsymbol{x}_0^*) = \prod_{i=0}^{n-1} D\boldsymbol{F}(\boldsymbol{x}_i^*)$, the product taken in order of decreasing i.

That these limits exist at all is guaranteed by the following theorem.

Multiplicative Ergodic Theorem (Oseledec [150])

Given an ergodic invariant measure μ on $\mathcal{A}$ with $\ln^+ \|D\boldsymbol{F}\| \in L^1(\mu)$, the following *Oseledec matrix*

$$\boldsymbol{H} = \lim_{n\to\infty} (\boldsymbol{B}_n)^{\frac{1}{2n}} \tag{12.36}$$

exists for μ-almost every $\boldsymbol{x}_0^*$. Its eigenvalues (the Lyapunov numbers) are the same for μ-almost every $\boldsymbol{x}_0^*$; its eigenvectors (the Oseledec subspaces) depend on $\boldsymbol{x}_0^*$.

Note that

$$\boldsymbol{H}_n^{2n} \triangleq \boldsymbol{B}_n , \tag{12.37}$$

where $\boldsymbol{B}_n$ is a positive matrix, so it has a square root. Since

$$\boldsymbol{B}_n \hat{\boldsymbol{e}}_{ni} = s_{ni}^2 \hat{\boldsymbol{e}}_{ni} = \left(\bar{s}_i^n\right)^2 \hat{\boldsymbol{e}}_{ni} , \tag{12.38}$$

we know

$$\boldsymbol{H}_n^{2n} \hat{\boldsymbol{e}}_{ni} = \bar{s}_i^{2n} \hat{\boldsymbol{e}}_{ni} , \tag{12.39}$$

and, therefore,

$$\boldsymbol{H}_n \hat{\boldsymbol{e}}_{ni} = \bar{s}_i \hat{\boldsymbol{e}}_{ni} . \tag{12.40}$$

Thus, the Lyapunov numbers (and exponents) are those of $\boldsymbol{H} = \lim_{n\to\infty} \boldsymbol{H}_n$.

Three properties of the spectrum follow, and each is used repeatedly below. First, the λ_i are independent of the initial condition for almost every $\boldsymbol{x}_0$, so they are *properties* of the attractor rather than of the orbit one happens to have measured. Eckmann and Ruelle [51, 52] give the standard account of this and of the ergodic-theoretic setting generally. Second, if $\lambda_1 > 0$ and $\sum \lambda_i < 0$, the attractor is chaotic: it combines stability, since it is an attractor after all, with instability, since $\|\boldsymbol{\delta}_n\|$ grows in at least the direction $\hat{\boldsymbol{e}}_1$. Third, the λ_i are *invariant* under diffeomorphisms and therefore under delay coordinate embedding [173, 197]. That invariance is what makes them estimable from a scalar measurement at all.

The second of these has a companion statement about volumes. Since $\bar{\lambda}_i = \ln \bar{s}_i$,

$$\sum_{i=1}^{m} \bar{\lambda}_i = \ln \left(\prod_{i=1}^{m} \bar{s}_i \right) = \ln \left(\det \boldsymbol{H}_n \right) = \frac{1}{n} \ln \left| \det D\boldsymbol{F}^n(\boldsymbol{x}_0) \right| , \tag{12.41}$$

so, if the system is *dissipative* (i.e., the phase space volumes collapse in time), we have

$$\sum_{i=1}^{m} \bar{\lambda}_i = \frac{1}{n} \ln \left| \det D\boldsymbol{F}^n(\boldsymbol{x}_0) \right| < 0 . \tag{12.42}$$

For example, recall the Hénon map of Eq. (12.2):

$$\begin{aligned} x_{n+1} &= a - x_n^2 + b y_n \\ y_{n+1} &= x_n \,, \end{aligned} \tag{12.43}$$

where $\boldsymbol{x}_n = [x, y]^{\mathrm{T}}$. Then

$$D\boldsymbol{F} = \begin{pmatrix} -2x_n & b \\ 1 & 0 \end{pmatrix} , \tag{12.44}$$

so $\det(D\boldsymbol{F}) = -b$ and is not a function of $\boldsymbol{x}_n$. Thus, we can evaluate

$$\frac{1}{n} \ln \left| \det D\boldsymbol{F}^n(\boldsymbol{x}_0) \right| = \frac{1}{n} \ln |b|^n = \ln |b| \,. \tag{12.45}$$

Therefore, $\lambda_1 + \lambda_2 = \ln |b|$ and it is negative if $|b| < 1$.

Flows versus maps. For flows, one $\lambda_i = 0$, corresponding to the direction along the orbit ($e^0 = 1$). The exponent sum is then $\sum \lambda_i = \langle \nabla \cdot \boldsymbol{f}(\boldsymbol{x}) \rangle$, the time average of the divergence along the trajectory, where $\dot{\boldsymbol{x}} = \boldsymbol{f}(\boldsymbol{x})$. When the divergence is constant, as for the Lorenz system ($\nabla \cdot \boldsymbol{f} = -(\sigma + 1 + \beta)$), the average is that constant. For a map the corresponding statement is $\sum \lambda_i = \langle \ln |\det D\boldsymbol{F}| \rangle$, which for the Hénon map with its constant Jacobian determinant $-b$ gives $\ln |b|$. For a Poincaré map, $\lambda_{\mathrm{map}} = \lambda_{\mathrm{flow}}\, \bar{t}_\Sigma$, where $\bar{t}_\Sigma$ is the average time between crossings of the section Σ; the flow's zero exponent does not appear in the map spectrum, which has one fewer exponent.

Relation to dimension. The spectrum also bounds the dimension of the attractor through

$$D_L = k + \frac{1}{|\lambda_{k+1}|} \sum_{i=1}^{k} \lambda_i \,, \tag{12.46}$$

where k is the largest integer such that $\sum_{i=1}^{k} \lambda_i \geq 0$. $D_L \equiv D_{KY}$ is the Lyapunov or *Kaplan-Yorke* dimension [67, 108]. There is also the Kaplan-Yorke conjecture, $D_L \equiv D_1$, relating it to the information dimension of Chapter 11. Ledrappier [126] proved the general inequality $D_1 \leq D_L$; Young [229] proved, for C^2 diffeomorphisms of surfaces with an ergodic measure having exponents $\lambda_1 > 0 > \lambda_2$, the equality $D_1 = h_\mu \left(1/\lambda_1 - 1/|\lambda_2|\right)$, which reduces to D_L exactly when $h_\mu = \lambda_1$ (Pesin's identity, satisfied by SRB measures). Equality in higher dimensions remains open in general.

12.3 Experimental Lyapunov Exponents

Now we will discuss how can we experimentally estimate Lyapunov exponent as defined by Eq. (12.35),

$$\lambda_i = \lim_{n \to \infty} \frac{1}{n} \ln \left\| D\boldsymbol{F}^n(\boldsymbol{x}_0) \boldsymbol{\delta}_0 \right\| \,, \tag{12.47}$$

from experimentally measured scalar time series or measured trajectory. Two broad families of methods exist: those that estimate the local Jacobians and accumulate them, following Sano and Sawada [172] and Eckmann *et al.* [50], and those that track the divergence of nearby trajectories directly [107, 167, 225]. We develop the first below and the second in Section 12.3.1.

The historically first algorithm for measured data, and the one most often met in the older applied literature, is that of Wolf *et al.* [225]. It follows a single pair: a reference point on the reconstructed trajectory and its nearest neighbor, tracked forward in time while their separation

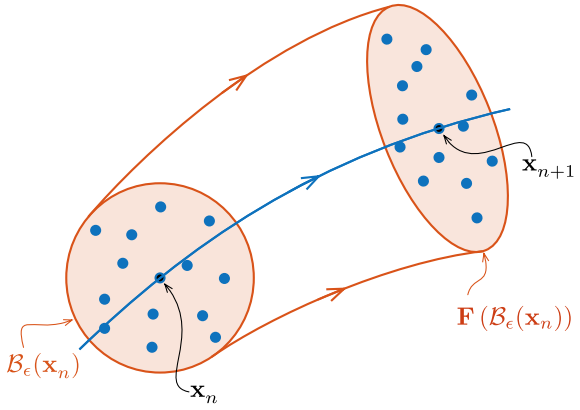


Figure 12.3: Estimating Jacobian through regression

grows from δ_0 to δ_k. When the separation exceeds a threshold, or when the pair has evolved for a preset time, the follower is *replaced* by a new neighbor of the current reference point, chosen as close as possible to the reference and with an orientation as close as possible to that of the old separation vector, and λ_1 is accumulated as the sum of $\ln(\delta_k/\delta_0)$ over all replacement intervals divided by the total time. The replacement step is what makes the method work on a single record, and it is also its weakness: the results depend on the replacement thresholds and on the orientation heuristic, there is no graphical check that a scaling region exists, and the single-pair statistics are noisy. The estimators of Rosenstein *et al.* [167] and Kantz [107], developed below, replace the following of one pair by an average over all reference points of the logarithm of the distance after a fixed time; they have no replacement heuristics, and the plot of that average against time shows the scaling region, or its absence, directly. They are preferred for those reasons, and the Wolf algorithm is described here so that its numbers in the literature can be read for what they are.

As was the case with the fractal dimensions, we start from the phase space trajectory $\{\boldsymbol{x}_n\}_{n=1}^N$. Generally one might try to estimate $D\boldsymbol{F}(\boldsymbol{x}_n)$ at each phase space point using regression on its nearest neighbor points contained inside an ϵ-ball $\mathcal{B}_\epsilon(\boldsymbol{x}_n)$ and the corresponding points at next time step contained in $\boldsymbol{F}\left(\mathcal{B}_\epsilon(\boldsymbol{x}_n)\right)$ as shown in Fig. 12.3. In other words, we minimize

$$e^2 = \frac{1}{N_\epsilon} \sum_{\boldsymbol{x}_j \in \mathcal{B}_\epsilon(\boldsymbol{x}_n)} \|\boldsymbol{x}_{j+1} - \boldsymbol{F}(\boldsymbol{x}_j)\|^2 \,. \tag{12.48}$$

However, in general, we do not know the form of $\boldsymbol{F}$. Therefore, if ϵ is small enough ($\|\boldsymbol{x}_j - \boldsymbol{x}_n\| < \epsilon$)

$$\boldsymbol{x}_{j+1} = \boldsymbol{F}(\boldsymbol{x}_j) \approx \boldsymbol{F}(\boldsymbol{x}_n) + D\boldsymbol{F}(\boldsymbol{x}_n)(\boldsymbol{x}_j - \boldsymbol{x}_n) = \boldsymbol{x}_{n+1} + D\boldsymbol{F}(\boldsymbol{x}_n)(\boldsymbol{x}_j - \boldsymbol{x}_n)\,, \tag{12.49}$$

or

$$\boldsymbol{x}_{j+1} = \boldsymbol{A}_n \boldsymbol{x}_j + \boldsymbol{b}_n\,, \tag{12.50}$$

where $\boldsymbol{A}_n = D\boldsymbol{F}(\boldsymbol{x}_n)$, and $\boldsymbol{b}_n = \boldsymbol{x}_{n+1} - D\boldsymbol{F}(\boldsymbol{x}_n)\boldsymbol{x}_n$. Therefore, we can estimate these parameters (i.e., $\boldsymbol{A}_n$ and $\boldsymbol{b}_n$) by minimizing:

$$e^2 = \frac{1}{N_\epsilon} \sum_{\boldsymbol{x}_j \in \mathcal{B}_\epsilon(\boldsymbol{x}_n)} \left\|\boldsymbol{x}_{j+1} - \hat{\boldsymbol{A}}\boldsymbol{x}_j - \hat{\boldsymbol{b}}\right\|^2 \,. \tag{12.51}$$

Solution to this minimization problem can be determined using matrix algebra. First, let us define two matrices containing point inside the corresponding balls:

$$\boldsymbol{X}_n = [\boldsymbol{x}_{j(1)}, \boldsymbol{x}_{j(2)}, \cdots, \boldsymbol{x}_{j(N_\epsilon)}]\,, \qquad \text{and} \qquad \boldsymbol{X}_{n+1} = [\boldsymbol{x}_{j(1)+1}, \boldsymbol{x}_{j(2)+1}, \cdots, \boldsymbol{x}_{j(N_\epsilon)+1}]\,. \tag{12.52}$$

Then we are looking for such $\hat{\boldsymbol{A}}_n$ and $\hat{\boldsymbol{b}}_n$ that the following is satisfied in the least squares sense:

$$\boldsymbol{X}_{n+1} = \hat{\boldsymbol{A}}_n \boldsymbol{X}_n + \hat{\boldsymbol{b}}_n = \hat{\boldsymbol{G}}_n \tilde{\boldsymbol{X}}_n \,, \tag{12.53}$$

where $\hat{\boldsymbol{G}}_n = [\hat{\boldsymbol{A}}_n, \hat{\boldsymbol{b}}_n]$ and $\tilde{\boldsymbol{X}}_n$ is the same matrix as $\boldsymbol{X}_n$ with additional row of ones at the bottom. Now if we multiply this equation on the right by $\tilde{\boldsymbol{X}}_n^{\mathrm{T}}$, we get:

$$\boldsymbol{X}_{n+1} \tilde{\boldsymbol{X}}_n^{\mathrm{T}} = \hat{\boldsymbol{G}}_n \tilde{\boldsymbol{X}}_n \tilde{\boldsymbol{X}}_n^{\mathrm{T}} \,. \tag{12.54}$$

which can be solved as

$$\hat{\boldsymbol{G}}_n = \boldsymbol{X}_{n+1} \tilde{\boldsymbol{X}}_n^{\mathrm{T}} \left(\tilde{\boldsymbol{X}}_n \tilde{\boldsymbol{X}}_n^{\mathrm{T}} \right)^{-1} \,. \tag{12.55}$$

Pitfall: neighbors strung along one filament defeat the local fit

The inverse in the expression above exists only if the matrix $\tilde{\boldsymbol{X}}_n \tilde{\boldsymbol{X}}_n^{\mathrm{T}} / N_\epsilon$ has full rank, and squareness alone is not sufficient. One needs at least $N_\epsilon \geq d+1$ neighbors, and they must not be degenerately distributed within $\mathcal{B}_\epsilon(\boldsymbol{x}_n)$. Neighbors strung along a single filament of the attractor make the problem rank-deficient no matter how many of them there are. That is the usual failure mode in practice.

Together, $\hat{\boldsymbol{A}}_n$ and $\hat{\boldsymbol{b}}_n$ define a *local linear map* that approximates the true nonlinear map $\boldsymbol{F}$ at the point $\boldsymbol{x}_n$ of the reconstructed phase space; the collection of these maps over all n is globally nonlinear. The fit also predicts. A *new* data point $\boldsymbol{x} \in \mathcal{B}_\epsilon(\boldsymbol{x}_n)$ is carried to $\boldsymbol{y} = \hat{\boldsymbol{A}}_n \boldsymbol{x} + \hat{\boldsymbol{b}}_n$, which is the local model taken up in Chapter 15. For the purpose of estimating Lyapunov exponents it is often better to use a higher order fit and then keep only its linear piece, $\hat{\boldsymbol{A}}_n$.

Delay coordinates fix all but one row. If $\boldsymbol{x}_n$ comes from delay reconstruction with the forward convention of Eq. (8.8),

$$\begin{aligned} \boldsymbol{x}_n &= \left(x_n, x_{n+\tau}, \ldots, x_{n+(d-1)\tau}\right)^{\mathrm{T}} \,, \\ \boldsymbol{x}_{n+k} &= \left(x_{n+k}, x_{n+k+\tau}, \ldots, x_{n+k+(d-1)\tau}\right)^{\mathrm{T}} \,. \end{aligned} \tag{12.56}$$

where d is the embedding dimension, τ is integer delay. If $k = \tau$:

$$\boldsymbol{x}_{n+\tau} = \left(\underbrace{x_{n+\tau}}_{\boldsymbol{x}_n^{(2)}}, \underbrace{x_{n+2\tau}}_{\boldsymbol{x}_n^{(3)}}, \ldots, \underbrace{x_{n+(d-1)\tau}}_{\boldsymbol{x}_n^{(d)}}, x_{n+d\tau} \right)^{\mathrm{T}} \,, \tag{12.57}$$

so the first $d-1$ coordinates of $\boldsymbol{x}_{n+\tau}$ are the last $d-1$ coordinates of $\boldsymbol{x}_n$, shifted down by one place, and only the last coordinate is new:

$$\begin{aligned} \boldsymbol{x}_{n+\tau}^{(j)} &= \boldsymbol{x}_n^{(j+1)} = x_{n+j\tau} \,, \qquad j = 1, \ldots, d-1 \,, \\ \boldsymbol{x}_{n+\tau}^{(d)} &= x_{n+d\tau} = f(x_n, x_{n+\tau}, \ldots, x_{n+(d-1)\tau}) = f(\boldsymbol{x}_n) \,. \end{aligned} \tag{12.58}$$

Therefore, the local linear map has the companion form

$$\boldsymbol{x}_{n+\tau} \cong \begin{pmatrix} 0 & 1 & 0 & \cdots & 0 & 0 \\ 0 & 0 & 1 & \cdots & 0 & 0 \\ \vdots & & & \ddots & & \vdots \\ 0 & 0 & 0 & \cdots & 0 & 1 \\ a_{d1} & a_{d2} & a_{d3} & \cdots & a_{d(d-1)} & a_{dd} \end{pmatrix} \boldsymbol{x}_n + \boldsymbol{b}_n \,, \tag{12.59}$$

with the identity on the superdiagonal and the regression coefficients of f in the bottom row; only that row (and the last entry of $\boldsymbol{b}_n$) needs to be fitted, which reduces the number of free parameters from $d(d+1)$ to $d+1$ and makes the fit far better conditioned. (In the backward convention the shift runs the other way and the fitted row is the first; the two forms are related by reversing the coordinate order, which changes no eigenvalue.) For $k \neq \tau$ the structure is lost and the full matrix must be fitted.

Given an estimate $\hat{\boldsymbol{A}}^n = \hat{\boldsymbol{A}}_{n-1}\hat{\boldsymbol{A}}_{n-2}\cdots\hat{\boldsymbol{A}}_0$, we can look at eigenvalues of $\left(\hat{\boldsymbol{A}}^n\right)^{\mathrm{T}}\left(\hat{\boldsymbol{A}}^n\right)$. However, there is a problem: the resulting matrix is inherently *ill conditioned* since the eigenvalues are $e^{2\lambda_1 n}, e^{2\lambda_2 n}, \ldots, e^{2\lambda_d n}$, and, therefore $e^{2\lambda_1 n}/e^{2\lambda_2 n} = e^{2(\lambda_1-\lambda_2)n} \to \infty$ very rapidly for large n. The standard remedy is not to form the product at all, but to propagate an orthonormal frame and reorthonormalize at every step, accumulating the logarithms of the diagonal factors; this is the Gram–Schmidt (QR) procedure of Benettin *et al.* [9, 10] and Shimada and Nagashima [185], and it yields the whole spectrum rather than λ_1 alone. Even so, results of this estimation depend strongly on quality of data and your mileage may vary. However, given any perturbation $\boldsymbol{\delta}$ defined by

$$\boldsymbol{\delta} = \sum_{j=k}^{d} \alpha_j \hat{\boldsymbol{e}}_j \,, \tag{12.60}$$

where we assume that $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_d$, then

$$\lambda_k = \lim_{n\to\infty} \frac{1}{n} \ln \|D\boldsymbol{F}^n(\boldsymbol{x}_0)\boldsymbol{\delta}\| \tag{12.61}$$

with probability one. In particular,

$$\lambda_1 = \lim_{n\to\infty} \frac{1}{n} \ln \|D\boldsymbol{F}^n(\boldsymbol{x}_0)\boldsymbol{\delta}\| = \lim_{n\to\infty} \frac{1}{n} \ln \frac{\|\boldsymbol{\delta}_n\|}{\|\boldsymbol{\delta}_0\|} \,. \tag{12.62}$$

for *any* $\boldsymbol{\delta}_0$ with probability one.

12.3.1 The Largest Short-Time Lyapunov Exponent Estimate

Like the pointwise dimension estimation, Lyapunov exponents can be estimated using either a fixed distance or a fixed mass method. For a fixed distance methodology, let $\|\boldsymbol{\delta}_0\| \equiv \delta_0$ and $\|\boldsymbol{\delta}_n\| = \delta_n$, then Eq. (12.62) says $n\lambda_1 \sim \ln(\delta_n/\delta_0)$, or

$$\delta_n \sim \delta_0 e^{n\lambda_1} \tag{12.63}$$

for n *large*. At the same time, we do not want n *too* large since we expect δ_n to saturate at the scale of the attractor. We can also write:

$$\ln \frac{\delta_n}{\delta_0} \sim n\lambda_1 \,, \tag{12.64}$$

or if we average over many or all data points $\boldsymbol{x}_0 = \boldsymbol{x}_k$:

$$\left\langle \ln \frac{\delta_n}{\delta_0} \right\rangle \sim n\lambda_1 \,. \tag{12.65}$$

More specifically, following Kantz [107], we compute

$$\Delta_n = \left\langle \ln \left(\frac{1}{N_\epsilon} \sum_{\boldsymbol{x}_m \in \mathcal{B}_\epsilon(\boldsymbol{x}_k)} \|\boldsymbol{x}_{k+n} - \boldsymbol{x}_{m+n}\| \right) \right\rangle_k \,, \tag{12.66}$$

which is not quite $\langle \ln(\delta_n/\delta_0)\rangle$: the logarithm acts on a distance already averaged over the ϵ-ball, and δ_0 does not appear, its effect being absorbed into a constant offset that leaves the slope unchanged. The distinction from the single-neighbor estimator of Rosenstein *et al.* [167] is taken up in Section 12.5.6. Written out over reference points and neighbors,

$$\Delta_n = \frac{1}{N}\sum_{k=1}^{N} \ln\left[\frac{1}{N_\epsilon}\sum_{\boldsymbol{x}_m \in \mathcal{B}_\epsilon(\boldsymbol{x}_k)} \|\boldsymbol{x}_{k+n} - \boldsymbol{x}_{m+n}\|\right], \tag{12.67}$$

where N_ϵ is the number of points inside the ϵ-ball neighborhood of $\boldsymbol{x}_k$, $\mathcal{B}_\epsilon(\boldsymbol{x}_k)$, *subject to the exclusion* $|k-m| > w$ discussed in Section 12.5.4, and the indices are:

k**:** "initial condition" or reference point;

m**:** points inside $\mathcal{B}_\epsilon(\boldsymbol{x}_k)$;

n**:** number of steps ahead.

Then plotting Δ_n vs n should give us a slope λ_1 per sampling time.

Fixed mass method for estimating Lyapunov exponents fixes the number r of the nearest neighbor points of $\boldsymbol{x}_k$ and then calculates:

$$\Delta_n = \frac{1}{N}\sum_{k=1}^{N} \ln\left[\frac{1}{r}\sum_{m=1}^{r} \left\|\boldsymbol{x}_{k+n} - \boldsymbol{x}_{i(m)+n}\right\|\right], \tag{12.68}$$

or

$$\Delta_n = \frac{1}{N}\sum_{k=1}^{N} \ln\left[\sum_{m=1}^{r} \left\|\boldsymbol{x}_{k+n} - \boldsymbol{x}_{i(m)+n}\right\|\right] - \ln r\,, \tag{12.69}$$

where $\{i(m)\}_{m=1}^{r}$ are the indices of the r nearest neighbor points of $\boldsymbol{x}_k$, again excluding those with $|k-i(m)| \le w$ (Section 12.5.4). Without that exclusion the r nearest neighbors of a finely sampled record are simply its r temporal successors.

Pitfall: a neighborhood too small to populate, or too large to be local

For the fixed distance method one must be careful in choosing ϵ: if it is too small, many reference points have no neighbors at all inside the ϵ-ball and contribute nothing to the average. For the fixed mass method r should be chosen conservatively, since otherwise the corresponding neighborhood size is too large for accurate estimation. Both errors are silent: the plot of Δ_n against n still appears, and only its slope is wrong.

The important features of both methods can be combined in a weighted sum,

$$\Delta_n = \frac{1}{N}\sum_{k=1}^{N} \ln\left[\frac{\sum_{m=1}^{r} w_m \left\|\boldsymbol{x}_{k+n} - \boldsymbol{x}_{i(m)+n}\right\|}{\sum_{m=1}^{r} w_m}\right], \tag{12.70}$$

where $w_m = w\big(\|\boldsymbol{x}_k - \boldsymbol{x}_{i(m)}\|\big)$ is an appropriately chosen weighting function that weighs points closest to $\boldsymbol{x}_k$ more than the ones further from it. The weight must depend on each neighbor's distance: a weight depending only on the neighbor count r is constant across the sum and cancels identically. The results of the fixed-mass-based Lyapunov exponent estimation for our sample signals are shown in Fig. 12.4

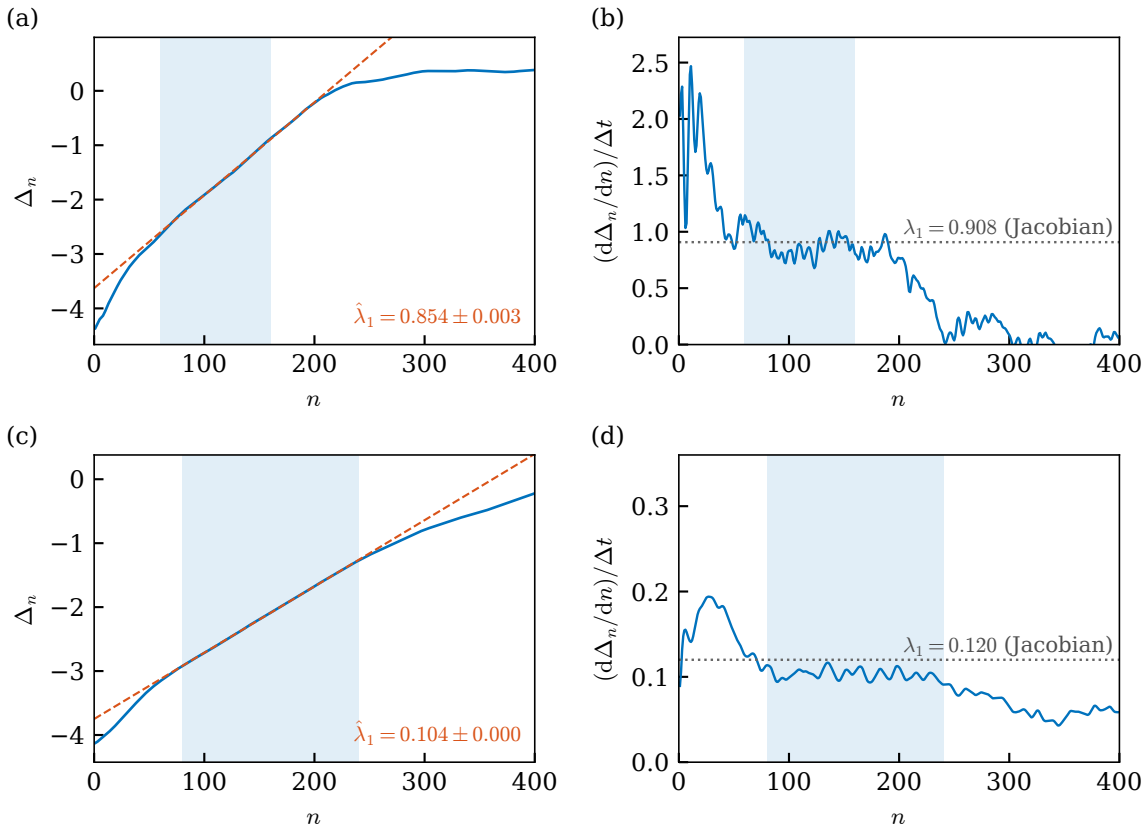


Figure 12.4: Largest Lyapunov exponent estimation by the fixed mass method (`lyapunov`, Appendix A.6, $r = 1$ neighbor, $N = 4000$ fiducial points, 400 steps) for the Lorenz (a–b) and two-well Duffing (c–d) signals. Lorenz x: $N = 20{,}000$ points at $\Delta t = 0.02$, $d = 3$, $\tau = 8$, $w = 3\tau = 24$; Duffing x: 60,000 points at $\Delta t = 0.1$, $d = 4$, $\tau = 14$, $w = 42$. (a),(c) the divergence curves Δ_n with the least-squares line through the shaded fit window; (b),(d) the local slope divided by Δt, against the exponent of the model from the Jacobian by the QR algorithm of Section 12.4.1 (0.9079 for the Lorenz system, eq. (12.73); 0.120 for this Duffing oscillator, whose two exponents sum to $-0.25 = -\gamma$). The fits give $\hat{\lambda}_1 = 0.854$ over $n = 60$–160 and 0.104 over $n = 80$–240; shifting each window by a fifth of its length moves them to 0.851–0.884 and 0.101–0.106, which is the uncertainty to quote, not the standard error of the fit (0.003 and 0.0001). Both are five to fourteen per cent low, the ceiling of a direct estimator at these record lengths (Section 12.5.2). Script: `python/mkfig_LyapunovExp.py`.

12.4 The Full Spectrum and the Problem of Spurious Exponents

The estimators of Section 12.3.1 recover λ_1 and nothing more. Obtaining the whole spectrum requires the tangent-space machinery of Section 12.2, and once we attempt it on reconstructed data a difficulty appears that has no counterpart in the model-based calculation: the reconstruction returns more exponents than the system possesses. This section states the algorithm properly, then confronts the extra exponents, which turn out to be neither noise nor numerical artifact.

12.4.1 The QR Algorithm

The obstruction identified at the end of Section 12.3 is that $\hat{\boldsymbol{A}}^n = \hat{\boldsymbol{A}}_{n-1} \cdots \hat{\boldsymbol{A}}_0$ becomes numerically unusable: its singular values separate as $\mathrm{e}^{\lambda_i n}$ (the eigenvalues of $\boldsymbol{B}_n$ as $\mathrm{e}^{2\lambda_i n}$), so all directional information collapses into the leading one within a few tens of iterations. The resolution, due independently to Benettin *et al.* [9, 10] and Shimada and Nagashima [185], is never to form the product. One instead propagates an orthonormal frame and reorthonormalizes at every step, accumulating the logarithms of the stretches as they occur. The frame is repeatedly pulled back to unit scale, so nothing is allowed to overflow, and the exponents emerge as time averages of quantities that remain $\mathcal{O}(1)$ throughout.

Concretely, given the sequence of Jacobians $\{\boldsymbol{A}_n\}$ along a trajectory and an initial $\boldsymbol{Q}_0 = \boldsymbol{I}_d$:

1. Form $\boldsymbol{Z}_n = \boldsymbol{A}_n \boldsymbol{Q}_n$, the image of the current frame under one step of the tangent dynamics.

2. Factor $\boldsymbol{Z}_n = \boldsymbol{Q}_{n+1} \boldsymbol{R}_{n+1}$ with $\boldsymbol{Q}_{n+1}$ orthogonal and $\boldsymbol{R}_{n+1}$ upper triangular with positive diagonal. Fixing the signs of the diagonal makes the factorization unique.

3. Accumulate $\sigma_i \leftarrow \sigma_i + \ln [\boldsymbol{R}_{n+1}]_{ii}$ for $i = 1, \dots, d$.

4. After N steps, $\lambda_i = \sigma_i / N$ for a map, or $\sigma_i / (N \Delta t)$ for a flow sampled at interval Δt.

The diagonal entries of $\boldsymbol{R}$ are the stretching factors along the successively orthogonalized directions, which is why the exponents appear already ordered, $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_d$. Two implementation points matter in practice. The sign fixing in step 2 is not cosmetic if the logarithms are taken of the signed diagonal entries: without it they pick up the arbitrary sign convention of the LAPACK routine and the averages fail to converge. Accumulating $\ln |R_{ii}|$ instead makes the sign irrelevant, and either convention is acceptable provided it is applied consistently. And for a flow, the state and the tangent frame must be integrated with the same order of accuracy; propagating $\boldsymbol{x}$ with a fourth-order scheme and $\boldsymbol{Q}$ with a first-order update leaves a bias that is plainly visible in the exponent that should vanish along the orbit.

12.4.2 Estimating the Tangent Map from Data

For a measured trajectory the $\boldsymbol{A}_n$ are not available and must be estimated, which is the local regression of Section 12.3. Sano and Sawada [172] and Eckmann *et al.* [50] introduced this route independently; Brown, Bryant, and Abarbanel [15, 17] developed it into a usable procedure, and Darbyshire and Broomhead [45] analyzed its statistical behavior. Parlitz [154] reviews the whole line of work.

One structural lesson from that literature deserves emphasis. Brown *et al.* [15] distinguish two dimensions that are usually conflated: a *global* embedding dimension, large enough to unfold the attractor and remove self-intersections, and a *local* dimension, used for the regression that estimates $\boldsymbol{A}_n$ in a neighborhood. There is no reason for these to coincide, and taking the local dimension equal to the global one is precisely what manufactures the difficulty of the next subsection. They further recommend fitting a local polynomial of order higher than one and retaining only its linear part, the same practice recommended for the local fits of Section 12.3.

12.4.3 Spurious Exponents

Suppose the underlying system has dimension n and we reconstruct in dimension $d > n$. A Jacobian-based method returns d exponents. Only n of them belong to the system; the remaining $d - n$ are *spurious* [15, 174]. Gençay and Dechert [70] state the counting cleanly: at $d = n$ the exponents of the reconstructed dynamics are exactly those of the original, while for $d > n$ the method yields d exponents of which only n are genuine.

It is tempting to suppose that the extra exponents are strongly negative artifacts, easily recognized and discarded. This is false, and the reason it is false is the central result of the subject. Sauer, Tempkin, and Yorke [174] traced the spurious exponents to the higher-order terms of the reconstruction map and obtained formulas for them in the low-noise limit; Tempkin and Yorke [201] made these explicit. The spurious exponents are *sums of the true ones*. For a two-dimensional system with exponents λ and μ, reconstructed in five dimensions by a generic embedding, the reconstructed spectrum is

$$\{\lambda,\ \mu,\ 2\lambda,\ \lambda+\mu,\ 2\mu\}\ , \tag{12.71}$$

and when $\mu < 0 < \lambda$ these order as

$$2\mu\ <\ \mu\ <\ \lambda+\mu\ <\ \lambda\ <\ 2\lambda\,. \tag{12.72}$$

Read Eq. (12.72) carefully. The largest exponent of the reconstructed system is spurious, and so is the smallest. The two true exponents sit in the interior of the list, adjacent to spurious neighbors on both sides. Neither "keep the largest" nor "discard the most negative" nor "keep the middle" survives contact with this example. Gençay and Dechert [70] report the same phenomenon from the opposite direction, exhibiting cases in which a spurious exponent exceeds the largest true exponent of the original system. Since one rarely knows n in advance, one does not even know how many entries of the list to distrust.

This also explains why spurious exponents cannot be dismissed as poorly-converged noise. In the low-noise limit in which the Tempkin–Yorke result holds, quantities such as 2λ and $\lambda + \mu$ are as reproducible as λ itself: they converge cleanly, survive refinement of the data, and remain stable when the record is lengthened. A spurious exponent that behaves impeccably is still spurious. Measurement noise breaks the pattern, and does so selectively. In a local-linear QR computation on the Hénon x record embedded in $d = 3$, the true exponent stayed at $\lambda \approx 0.39$ when 1% noise was added while the spurious one moved from $+0.77$ to -0.65; the spurious exponents live in the directions that carry no signal, and noise fills exactly those directions. That sensitivity is a weak but useful discriminator, and it is what Brown *et al.* exploit below.

12.4.4 Separating True from Spurious Exponents

Time reversal. The sharpest available test is due to Parlitz [153] and rests on a simple asymmetry. Reversing time inverts the dynamics, so every true exponent changes sign, $\lambda_i \mapsto -\lambda_i$, and the ordered spectrum reverses. The spurious exponents obey no such rule, because they are artifacts of the reconstruction map rather than properties of the flow. Computing the spectrum forward and backward and retaining only those exponents that reverse sign therefore identifies the true ones. Parlitz observed further that the degree of agreement between the two spectra is itself a useful objective function: the free parameters of the estimator, above all the number of nearest neighbors, can be tuned by requiring forward and reversed estimates to be consistent. This second use of the test is frequently overlooked and is arguably the more practical of the two.

Direction vectors and noise response. Brown *et al.* [15] identify spurious exponents from the Lyapunov direction vectors associated with them, and report that spurious and true exponents respond differently to added measurement noise. Both diagnostics require more bookkeeping than the time-reversal test but do not require running the dynamics backward, which matters when the record is short.

Regularization. A different tactic is to make the spurious exponents harmless rather than to identify them. Regularizing the local Jacobian fits, in the Tikhonov–Phillips sense, damps the directions that the data do not constrain; Parlitz [154] notes that the spurious exponents are thereby "shifted towards $-\infty$." Stoop and Parisi [194] construct their algorithm so as to avoid generating the spurious elements in the first place. It is worth being clear about what this accomplishes: regularization *makes* the folklore true by construction. The spurious exponents become very negative because they have been pushed there, not because they were there to begin with.

A caution on varying the dimension. Repeating the estimate at several embedding dimensions and keeping the exponents that persist is common practice, and it is a legitimate *consistency check*: a genuine exponent ought to survive. It is not a *classifier*. In the Tempkin–Yorke picture the spurious exponents are deterministic combinations of the true ones and are themselves stable under refinement, so persistence is necessary but not sufficient. Tempkin and Yorke [201] close with exactly this warning, that the interleaving of true and spurious values should caution anyone hoping to separate them by inspection, particularly as the number of spurious values is not known beforehand.

12.4.5 Current Practice

Two developments since the classical Jacobian methods bear on the full spectrum, and both should be understood with their limitations stated.

Reservoir computing. Pathak *et al.* [155] fit a recurrent reservoir to the measured series and then compute the Lyapunov spectrum of the fitted reservoir in closed form, its equations and output weights being known. The approach recovers many exponents of the Kuramoto–Sivashinsky equation and is genuinely model-free with respect to the observed system. Note, however, that it reproduces the structure of the present section in a new setting: the reservoir has far more exponents than the target, so the question of which are physical returns. Hart [89] supplies a sharp condition for faithful reconstruction, namely that the maximal conditional Lyapunov exponent of the driven reservoir be significantly more negative than the most negative exponent of the target system. This is a tuning requirement, not a guarantee, and the method is not turnkey. Vlachas *et al.* [218] benchmark reservoirs against trained recurrent architectures on the same class of problems.

Covariant Lyapunov vectors. The exponents alone do not locate the directions to which they belong. Covariant Lyapunov vectors, computed by the algorithm of Ginelli *et al.* [73] and analyzed by Kuptsov and Parlitz [124], supply a covariant, norm-independent tangent-space decomposition. In spatially extended dissipative systems this decomposition splits the tangent space into a finite set of entangled "physical" modes and a residual set of strongly decaying "spurious" ones, the split being characterized by the absence of tangencies between the two subspaces [199, 228]. The terminology invites a connection to the present section that should be resisted: there the spurious modes arise from excess degrees of freedom in a spatial discretization and lie off the inertial manifold, whereas here they arise from higher-order terms of the reconstruction map. The mechanisms are distinct.

Whether the tangency criterion can be transferred to the over-embedding problem, and used to separate true from spurious exponents in a delay reconstruction, appears to be open. Martin *et al.* [133] have taken the first step by estimating covariant vectors from data records without model equations, but the question has not been posed in the embedding setting.

A closing caveat applies to this entire subsection. Every demonstration cited above validates its estimated spectrum against a ground truth obtained from known equations, typically the Lorenz or Kuramoto–Sivashinsky systems. The authors are aware of no study establishing a full estimated spectrum against independent ground truth on genuinely experimental data. For measured signals the leading exponent remains far more trustworthy than the spectrum, and the practitioner is advised to report λ_1 with a scaling region, as in Section 12.3.1, and to treat the remainder as provisional.

12.5 Systematic Errors in Lyapunov Estimation

Section 12.4.3 exhibited one failure mode in detail. It is not an isolated defect of one algorithm but an instance of a general situation: every estimator in this chapter rests on assumptions that measured data may violate silently, returning a plausible number rather than an error. This section collects the failure modes by estimator family. Some are established in the literature and cited as such; two we have examined numerically for this book, and those are labelled explicitly, because the literature we could locate does not settle them.

The two families behave differently and it is worth stating the axis plainly. *Direct* methods, which track the divergence of neighboring trajectories, estimate λ_1 alone but produce a curve the user can inspect. Parlitz [154] counts this as their decisive advantage: they "give graphical feedback to the user to confirm exponential divergence," and that feedback "provides valuable information concerning the validity and accuracy of the estimation results." *Jacobian* methods return the full spectrum but offer no such check. The contrast matters more than any individual pitfall below.

12.5.1 The Scaling Region Dominates Everything

The single largest source of error in a direct estimate is not the algorithm but the choice of fit window. Hegger, Kantz, and Schreiber [91] are explicit that a linear region must actually be observed: if $S(\epsilon, d, t)$ "exhibits a linear increase with identical slope for all d larger than some d_0 and for a reasonable range of ϵ, then this slope can be taken as an estimate of the maximal exponent," and otherwise "one cannot draw any reasonable conclusion."

The magnitude of the effect deserves to be seen rather than asserted. For the Lorenz system sampled at $\Delta t = 0.01$ and embedded with $d = 5$, $\tau = 10$, a single fixed-distance computation with $\epsilon = 0.5$ yields, from one and the same $S(t)$ curve:

fit window in t	λ_1
[0.20, 0.90]	1.76
[0.50, 1.20]	1.14
[0.80, 1.60]	0.95
[1.10, 2.40]	0.99
[1.50, 2.50]	1.00
[0.20, 2.50]	1.08

against a reference value $\lambda_1 = 0.906$. The spread is nearly a factor of two, and it is systematic rather than statistical: the early window sits in a transient during which the perturbation reorients itself

into the unstable direction, and that transient has a steeper slope than the asymptotic one. An estimate reported without its fit window is therefore not reproducible. Zhou and Wang [231] treat the automatic identification of the linear region as a research problem in its own right, which is a fair measure of how unsatisfactory the manual choice is.

12.5.2 A Worked Example

The effect is worth carrying out once in full, because every step of it is reproducible and the conclusion is not the one a reader expects.

Take the Lorenz system of Eq. (14.17) at $\sigma = 10$, $\rho = 28$, $\beta = 8/3$, integrate it with a fourth-order Runge–Kutta scheme at $\Delta t = 0.01$, discard the first 2000 samples as transient, and keep 60,000 points of the x channel. Embed with $d = 3$ and $\tau = 12$, and compute the divergence curve from $N = 4000$ fiducial points with a single neighbor each, out to $n = 300$ steps:

```
[ns, lnd] = lyapunov( x, 3, 12, 4000, 1, 300 );
```

using the routine of Appendix A.6 in either language. The resulting Δ_n and its local slope are shown in Fig. 12.5.

Against this we need a reference. Because the model is known, the spectrum can be had exactly, from the Jacobian by the QR algorithm of Section 12.4.1 applied to the augmented state-and-tangent system:

$$\lambda = (0.9079,\ 0.0029,\ -14.5775)\ . \tag{12.73}$$

Two checks confirm the integration rather than merely reporting it. The exponents sum to -13.66667, which is $\nabla \cdot \boldsymbol{f} = -(\sigma + 1 + \beta)$ exactly, and the second exponent is zero to 3×10^{-3}, as it must be for a flow. The Kaplan–Yorke dimension that follows is $D_L = 2.062$, against the 2.06 of the literature.

Now fit the same Δ_n curve over different windows. Each entry below is the mean over five independent draws of the fiducial points, with the standard deviation over those draws:

fit window in n	$\hat{\lambda}_1$	
1–20	2.078 ± 0.051	
5–35	2.246 ± 0.031	the reorientation transient
10–50	2.230 ± 0.024	
20–80	1.769 ± 0.018	
50–150	0.903 ± 0.022	lands on the reference
90–290	0.830 ± 0.005	
150–300	0.838 ± 0.011	saturating
1–300	0.968 ± 0.004	the whole range

Three things in that table are worth more than the spread itself.

First, the spread and the scatter are unrelated. The estimates range from 0.83 to 2.25, a factor of 2.7, while the standard deviation within any one window is between 0.005 and 0.05. The estimator is therefore fifty times more precise than it is accurate, and reporting an uncertainty computed from repeated draws — the obvious thing to do — measures the wrong quantity entirely and makes the answer look far better than it is.

Second, the early windows are not merely noisy but systematically steep: 2.25 against a true 0.91. Over the first eighty steps the perturbation has not yet aligned with the unstable direction, and while it reorients, the separation grows faster than the asymptotic rate. The shaded region of

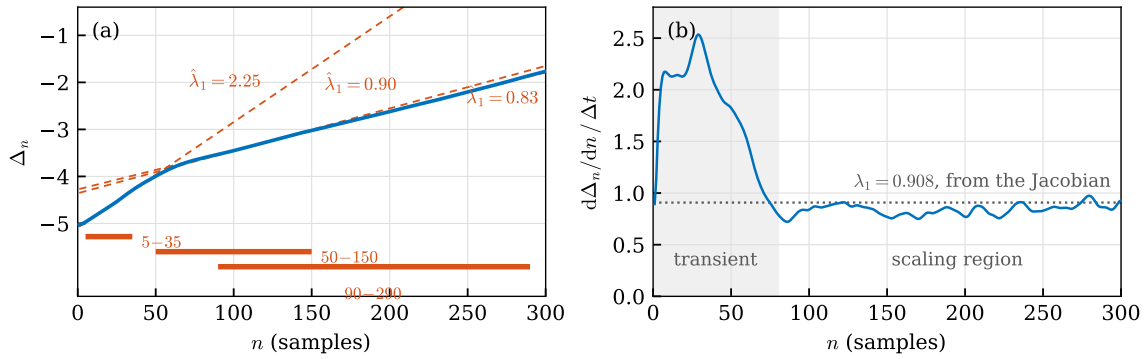


Figure 12.5: The fit window, for the Lorenz x channel at $\Delta t = 0.01$ embedded with $d = 3$, $\tau = 12$. (a) The divergence curve Δ_n, with three candidate fit windows marked below it and the corresponding least-squares lines extended across the full range. (b) The local slope $\mathrm{d}\Delta_n/\mathrm{d}n$ divided by Δt, against the reference $\lambda_1 = 0.908$ obtained from the Jacobian. The shaded region is the transient during which the perturbation reorients into the unstable direction; the plateau beyond it is the scaling region.

Fig. 12.5(b) is that transient. Any window inside it returns something close to two and a half times the exponent.

Third, and least expected, fitting the *whole* range returns 0.968 — within seven per cent of the reference. This is not a success. It is a too-steep transient averaging against a too-shallow tail, and the agreement is a coincidence of where the curve happens to saturate for this record length. A reader who fits everything gets a defensible-looking number and learns nothing about their data, and the same procedure on a shorter record, where the tail carries more weight, will not be so lucky.

What, then, is defensible? The curve does have a genuine scaling region, visible in Fig. 12.5(b) as the plateau beyond $n \approx 90$. Choosing the window in which the local slope varies least — here, the flattest forty-step span, at $n = 101$ to 141 — is an automatic rule, it requires no reference value, and it gives 0.877, which is 3.4 per cent low. That is about as well as a direct estimator does on clean data from a system whose exponent is known. The window that actually lands on the reference, 50–150, cannot be identified without the reference, which is precisely what an experiment does not have.

The practical rule follows from the table rather than from taste. Plot the local slope, not just Δ_n; identify the plateau before fitting anything; report the window alongside the exponent; and quote the spread over plausible windows, not the scatter over repeated draws, as the uncertainty.

12.5.3 A Measured Record

The same exercise on a measured record differs in one respect that changes everything: there is no reference value to land on. What follows is therefore not a comparison but the whole of the argument that the number deserves to be reported.

Take the two-well oscillator record of Section 1.4.1, 96,000 strain samples at 16 samples per forcing period. The delay is $\tau = 5$ samples, the first minimum of the average mutual information (Chapter 9); the embedding dimension is $d = 4$ (Chapter 10), and the estimate is repeated at $d = 5$,

6 and 7. The Theiler window is $w = 32$ samples, two forcing periods, comfortably beyond the first zero of the autocorrelation at 12 samples. Take $N = 5000$ fiducial points with one neighbor each and follow them for $n = 160$ steps, ten forcing periods:

```
[ns, lnd] = lyapunov( x, 4, 5, 5000, 1, 160, 32 );
```

The divergence curves and their local slopes are shown in Fig. 12.6.

The local slope climbs out of a short transient, is flat between steps 10 and 64, and falls away after that. Least squares over that window gives

$$\hat{\lambda}_1 = 0.61 \pm 0.02 \text{ nats per forcing period}\,, \tag{12.74}$$

the uncertainty being the spread over plausible windows. It is not the scatter over repeated draws, for the reason set out above. The value is quoted per forcing period because that is the unit in which it can be compared with anything else: the same exponent for the simulated two-well oscillator, or for a run of the same rig at another drive frequency. In seconds, with a forcing period of 0.1 s, $\hat{\lambda}_1 = 6.1\,\mathrm{s}^{-1}$. The reciprocal of Eq. (12.74) is an e-folding time of 1.6 forcing periods. That is also the mean residence time on one side of the mean reported in Section 1.4.1, so the record loses about one nat per well crossing.

The plateau beyond the fit window in Fig. 12.6(b) is not a second scaling region but saturation. Two nearby points cannot separate further than the attractor is wide. So Δ_n must bend over toward ln of the mean interpoint distance, which is 1.08 at $d = 5$ in units of the record's standard deviation. The curve is already within one unit of that value at $n = 160$. Fitting into the bend therefore mixes the exponential growth with a bounded quantity and can only bias the exponent downward: over steps 100–160 the slope is 0.21, and over the whole range 1–160 it is 0.42, both far below Eq. (12.74). This is the same failure as fitting the whole Lorenz range, and here there is no reference value to reveal it.

Four internal checks justify Eq. (12.74), and no external one exists.

1. *Stability over d.* The fits give 0.617, 0.610, 0.608 and 0.603 nats per forcing period at $d = 4, 5, 6, 7$. The plateau does not move as coordinates are added, which is the condition Hegger, Kantz and Schreiber require.

2. *Stability over the fit window.* Windows 6–64, 10–48, 16–64 and 20–72 give 0.609, 0.618, 0.609 and 0.595; only windows that reach into the saturation give less.

3. *The scatter is not the uncertainty.* Six independent draws of the fiducial points give a standard deviation of 0.006. That is three times smaller than the window spread, and quoting it would overstate the precision threefold.

4. *Stationarity and the Theiler window.* The two halves of the record give 0.611 and 0.629. Changing w from 16 to 128 samples moves the estimate by 0.001, so temporal correlation between the neighbors is not producing the growth.

The model offers a consistency check but not a reference. The two-well Duffing oscillator of Section 1.4 has $\lambda_1 = 0.120$ per unit time from its Jacobian, or 0.75 nats per forcing period. The same direct estimator returns 0.104 on a simulated record, which is 0.65 per forcing period (Fig. 12.4). The measured record gives 0.61. The rig is not that oscillator and the agreement proves nothing, but a measured value of 0.1 or 3 would have called for an explanation.

What would have made the number untrustworthy is worth stating, because none of it happened. A plateau that shifted with d would mean the reconstruction was still unfolding. A Δ_n curve that was straight on doubly logarithmic axes with slope near $1/2$ would indicate diffusion rather than exponential growth (Section 12.5.7). A slope that changed with w would mean the neighbors were temporal rather than dynamical. And a positive exponent alone is not evidence of chaos: the surrogate test of Chapter 16 on this record is what excludes the linear alternative.

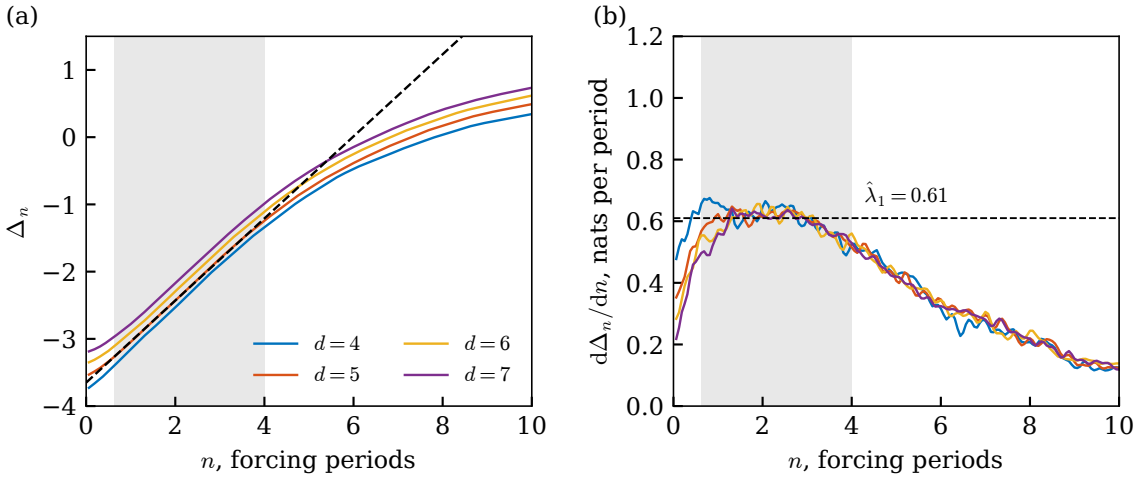


Figure 12.6: Largest Lyapunov exponent of the two-well oscillator record of Section 1.4.1 by the fixed-mass estimator (`lyapunov`, Appendix A.6): $\tau = 5$ samples, Theiler window $w = 32$ samples, $N = 5000$ fiducial points, $r = 1$ neighbor, $n = 160$ steps. The record is normalized to unit variance. Step number is converted to forcing periods with the 16 samples per period measured from the record. No value here depends on the sampling rate in hertz. (a) The divergence curves Δ_n for $d = 4, 5, 6, 7$, with the least-squares line for $d = 5$ over the shaded window extended across the range. (b) The local slope $d\Delta_n/dn$ in nats per forcing period. It is flat at 0.61 over the shaded window, steps 10 to 64, and falls afterwards as Δ_n saturates at the attractor diameter. Script: `python/mkfig_twowell_lyap.py`.

12.5.4 Temporally Correlated Neighbors

A point close to $\boldsymbol{x}_k$ in index need not be dynamically close to it: it may simply be the next sample of the same trajectory segment. Averaging over such neighbors measures the sampling rate rather than the dynamics. Theiler [202] identified the artifact for the correlation dimension, where it produces spuriously low estimates, and introduced the remedy of excluding pairs separated by fewer than w samples in time. Hegger *et al.* [91] carry the prescription over to Lyapunov estimation, where the exclusion window is now generally called the Theiler window: "Lyapunov estimates may be affected by serial correlations between reference points and neighbors. Therefore, a minimum time for $|n - n'|$ can and should be specified here as well." They recommend reading w off a space-time separation plot, which shows how far apart in time two points must lie before their separation is no longer dominated by temporal correlation along one trajectory segment. Exclusion removes that short-time dependence; it does not make the retained pairs independent (Section 11.6.2 of Chapter 11).

The exclusion is older than that prescription: Rosenstein *et al.* [167] already required the nearest neighbor of each reference point to be separated from it in time by more than the mean period of the record, estimated as the reciprocal of the mean frequency of the power spectrum, which is a Theiler window with w set to one characteristic period.

The estimators of Section 12.3.1 as stated impose no such exclusion, and should. In the sums defining Δ_n, restrict the neighbor set to

$$\boldsymbol{x}_m \in \mathcal{B}_\epsilon(\boldsymbol{x}_k) \quad \text{with} \quad |k-m| > w\,. \tag{12.75}$$

Two honest qualifications. First, we could locate no source stating the *direction* of the resulting bias for Lyapunov exponents, as opposed to for the correlation dimension. Second, in our own computations the correction proved less consequential than its prominence in the literature suggests: for the Lorenz configuration above, varying w from 0 to 200 samples moved λ_1 only between 0.996 and 1.038, which is within the scatter of the fit-window choice. The reason appears to be that a sufficiently small ϵ combined with an adequate delay already excludes temporally adjacent points geometrically. Theiler's own failure condition involves the record length relative to the autocorrelation time, so the window should be expected to matter most for short records — which is precisely the regime in which one is most tempted to skip it.

12.5.5 Oversampling and Degenerate Embeddings

Raising the sampling rate at fixed record length does not add information and can destroy an estimate. With $\tau = 1$ and $\Delta t = 0.01$, five consecutive samples of a smooth trajectory are very nearly collinear, so the delay vectors span a needle rather than a neighborhood. Repeating the Lorenz computation with $\tau = 1$ in place of $\tau = 10$, all else equal, gives $\lambda_1 = 0.51$ against the reference 0.906, a 44% underestimate. It is worth being clear that this is a failure of the embedding rather than of the Theiler window: the two are often conflated because both are cured by attending to time scales, but the mechanisms differ. The related pathology on the Jacobian side is recorded by Hegger *et al.* [91]: "when attractors are thin in the embedding space, some (or all) of the local Jacobians might be estimated very badly." This is the rank deficiency of the local Jacobian fit warned about in Section 12.3: neighbors lying along a single filament leave the regression matrix short of full rank. Here the same failure is seen from the geometry rather than the algebra.

12.5.6 Which Average? Kantz Versus Rosenstein

The two standard direct estimators are usually presented as interchangeable. They are not. Kantz's $S(\epsilon, d, t)$ [91, 107] averages *distances* over the whole ϵ-ball and then takes the logarithm,

$$S_{\mathrm{K}}(t) = \left\langle \ln\left(\frac{1}{|\mathcal{U}_k|} \sum_{\boldsymbol{x}_m \in \mathcal{U}_k} \|\boldsymbol{x}_{k+t} - \boldsymbol{x}_{m+t}\| \right) \right\rangle_k , \tag{12.76}$$

whereas the method of Rosenstein *et al.* [167] follows a single nearest neighbor per reference point, so that the inner average is absent and the logarithm acts on one distance,

$$S_{\mathrm{R}}(t) = \left\langle \ln \|\boldsymbol{x}_{k+t} - \boldsymbol{x}_{k^*+t}\| \right\rangle_k . \tag{12.77}$$

Both then average logarithms over reference points, so the distinction is not "mean of log versus log of mean" wholesale; it is confined to the inner average over the ball. At $t = 0$ the ball average exceeds

the distance to the nearest neighbor for every reference point (a mean is at least its minimum), so $S_{\mathrm{K}}(0) \geq S_{\mathrm{R}}(0)$. For $t > 0$ no inequality follows: the initially nearest neighbor need not stay below the ball average, and the sign of the gap is an empirical matter. A constant offset changes no slope in any case, so a bias in λ_1 requires the gap to grow with t.

It does, weakly. In our computations at $\epsilon = 1.0$ the gap widened from 1.86 at $t = 0$ to 2.29 at $t = 2.4$, contributing about 0.18 to the apparent slope, and the two estimators differed by 0.02 to 0.07 in λ_1 at small ϵ. At $\epsilon = 2.0$ they diverged by 0.25, with Kantz's estimate degrading as the ball grew large enough for its interior averaging to sample the nonlinear regime. The practical conclusion is that the choice matters little for small ϵ and matters considerably once ϵ is large — which is another reason to report ϵ alongside the exponent. We were unable to find this comparison analyzed in the literature, so the numbers above should be read as an illustration from one system rather than as a general result.

12.5.7 Stochastic Data Produce Positive Exponents

The most damaging failure is that noise does not announce itself. For a stochastic process the separation of nearby points grows diffusively, as $t^{1/2}$, and over a restricted range on a lin-log plot such growth is readily mistaken for a linear region and hence for a positive exponent. Hegger *et al.* [91] make the diagnostic concrete on human breath-rate data, noting that the curve shows "no linear increase, reflecting the lack of exponential divergence of nearby trajectories," and advising that the figure be replotted on doubly logarithmic axes, where "a power law behavior, which, with power 1/2, could be present for a diffusive growth of distances." Plotting $\log \Delta_n$ against $\log n$ and looking for slope 1/2 is thus a cheap and effective guard.

The corresponding statement for the Jacobian family is blunter, and is the strongest single caution we can offer about the full spectrum [91]:

> "In particular, when the algorithm is blindly applied to data from a random process, it cannot internally check for the consistency of the assumption of an underlying dynamical system. Therefore a Lyapunov spectrum is computed which now is completely meaningless."

There is a well-established precedent for this class of error in dimension estimation. Osborne and Provenzale [149] showed that stochastic processes with power-law spectra yield finite, low correlation dimensions, mimicking low-dimensional chaos; Theiler [207] examined the $1/f^{\alpha}$ case in particular, and Provenzale *et al.* [160] addressed the discrimination problem directly. The Lyapunov analogues were developed independently and by different groups — Dämmig and Mitschke [44] for the stochastic case, and Cencini *et al.* [24], who argue that at finite resolution the distinction is not sharp at all and should be replaced by resolution-dependent quantities such as the finite-size Lyapunov exponent. Gao *et al.* [69] pursue the same idea through a scale-dependent exponent.

The safeguards are therefore threefold, and no one of them suffices: a visibly linear scaling region over a range of ϵ and d; a scale-resolved exponent, which distinguishes a genuine exponent from diffusive growth by construction; and surrogate data testing in the sense of Theiler *et al.* [208], which is the standard test for nonlinearity and determinism and is the subject of Chapter 16. We note that surrogate testing is universally accepted as a nonlinearity test but that we found no source prescribing it specifically as the significance test for a positive exponent; it is reasonable and common practice rather than a codified procedure.

12.5.8 Limits That No Algorithm Can Evade

Two constraints bound the whole enterprise. Eckmann and Ruelle [53] established that the record length required to estimate dimensions and exponents grows with the dimension of the attractor, which places exponents of high-dimensional systems out of reach of any realistic experiment regardless of method. And Abarbanel, Brown, and Kennel [2] record a limitation that is rarely stated in practical accounts: local Lyapunov exponents "are ambiguous up to order $1/L$ in the number of steps due to the choice of coordinate system." Their earlier paper [1] quantifies the accompanying variability, finding that the spread of finite-time exponents about the asymptotic value narrows as a power law with exponent between 0.5 and 1.0 in the number of steps. Finite-time estimates are thus not merely noisy versions of the asymptotic ones; they are distributed quantities whose spread is itself a property of the attractor.

Finally, two preprocessing decisions deserve mention because they are made early and rarely revisited. Non-stationarity violates the single-invariant-measure assumption on which every estimator here depends, and recurrence and space-time separation plots are the standard tools for detecting it [91]. And low-pass filtering applied to "clean" a record alters the estimate: Mehdizadeh and Sanjari [140] document the effect of noise and filtering on λ_1 for gait data. A filter chosen for reasons unconnected to the dynamics can change the answer, and the choice should be reported with it.

Problems

Problem 12.1

Computational

Using the same time series of Problem 11.2, find the largest Lyapunov exponent of a candidate chaotic time series. Can you conclude in fact that the system is chaotic?

The remaining problems build a working Lyapunov-exponent code from scratch, use it to reproduce the pathology of Section 12.4.3, and then return to Problem 12.1 with the means to answer it properly. Problems 12.2–12.4 need only Sections 12.1–12.3; Problems 12.5–12.7 require Section 12.4, and Problems 12.8–12.9 require Section 12.5. As in Chapter 1, use MATLAB or Python throughout.

Problem 12.2

Core

Consider the Hénon map of Eq. (12.2) with $b = 0.3$.

1. Find both period-one fixed points analytically as functions of a and confirm that they exist only for $a \geq -(b-1)^2/4$.

2. Evaluate the Jacobian $D\boldsymbol{F}$ at each fixed point and compute the stability numbers $s_{1,2}$ as functions of a. Determine the range of a over which one fixed point is asymptotically stable.

3. Locate the value of a at which the stable fixed point loses stability, and identify the bifurcation from the behavior of $s_{1,2}$ there. Is it the same bifurcation as the one at $a = -(b-1)^2/4$?

4. Verify your answers to (2) and (3) numerically by iterating the map and plotting the long-time behavior of x_n against a.

Problem 12.3

Computational

Implement the QR algorithm of Section 12.4.1 for maps. Your function should accept a map, its Jacobian, an initial condition, and a number of iterations, and return the full spectrum.

1. Apply it to the Hénon map with $a = 1.4$, $b = 0.3$. You should obtain $\lambda_1 \approx 0.419$ and $\lambda_2 \approx -1.623$.

2. Verify Eq. (12.42), that $\lambda_1 + \lambda_2 = \ln|b|$, and report the residual. Explain why this identity is exact for the Hénon map and would not be for a map whose Jacobian determinant depends on the state.

3. Compute the Kaplan–Yorke dimension from your spectrum. Compare with the published value $D_{KY} \approx 1.258$.

4. Now recompute D_{KY} omitting the absolute value in the denominator, that is as $k + \lambda_1/\lambda_2$. What value do you obtain, and why is it impossible for a set that contains a curve?

5. Investigate what happens if you omit the sign fixing in step 2 of the algorithm, retaining whatever signs the `qr` routine returns. Plot the running estimates σ_i/n against n for both versions and explain the difference.

Problem 12.4

Computational

Extend your code to flows by integrating the state and the tangent frame together, as described at the end of Section 12.4.1.

1. Apply it to the Lorenz system with $\sigma = 10$, $\rho = 28$, $\beta = 8/3$. Confirm that one exponent vanishes and identify the direction it corresponds to.

2. Verify that $\sum_i \lambda_i$ agrees with $\langle \nabla \cdot \boldsymbol{f} \rangle$. Show analytically that $\nabla \cdot \boldsymbol{f}$ is constant for this system, so that the time average coincides with the pointwise value.

3. Repeat for the Rössler system

$$\dot{x} = -y - z\,, \qquad \dot{y} = x + ay\,, \qquad \dot{z} = b + z(x - c)\,,$$

with $a = 0.2$, $b = 0.2$, $c = 5.7$. Show that here $\nabla \cdot \boldsymbol{f} = a + x - c$ is *not* constant. Compare $\sum_i \lambda_i$ against both the pointwise divergence evaluated at a few points on the attractor and the time average $\langle \nabla \cdot \boldsymbol{f} \rangle$ accumulated along the trajectory. Which one does the sum of the exponents equal?

4. Now propagate the tangent frame with a first-order Euler update while keeping fourth-order integration of the state. Report the resulting value of the exponent that should vanish, and estimate how the error scales with the step size Δt.

Problem 12.5

Computational

This problem reproduces Eq. (12.71) numerically and is the point of Section 12.4.3.

Generate a long scalar time series from the Hénon map with $a = 1.4$, $b = 0.3$, taking $u_n = x_n$ as the measurement.

1. Embed the series in dimension $d = 2$ with unit delay and estimate the spectrum by local linear regression, following Section 12.3. You should recover approximately the two exponents of Problem 12.3.

2. Now repeat with $d = 3, 4, 5$. Tabulate all exponents obtained at each dimension.

3. For $d = 5$, compare your five exponents against the predicted set $\{\lambda,\ \mu,\ 2\lambda,\ \lambda + \mu,\ 2\mu\}$ using the values of λ and μ from Problem 12.3. How well does the prediction hold? Expect the most negative entry, $2\mu \approx -3.25$, to be resolved poorly (a value near -1.7 is typical): the most contracting direction of a set of dimension 1.26 is nearly empty of data and is underdetermined by the regression.

4. Identify which entries of your $d = 5$ list are the true exponents. Confirm that they are neither the largest nor the smallest. What would you have concluded had you assumed the spurious exponents were the most negative ones?

5. Lengthen the record by a factor of ten and repeat. Do the spurious exponents become less well converged? What does your answer imply about using convergence quality to detect them?

Problem 12.6

Computational

Implement Parlitz's time-reversal test [153].

1. Reverse the Hénon series of Problem 12.5 and estimate the spectrum of the reversed data at $d = 5$.

2. Compare the forward and reversed spectra. Which exponents satisfy $\lambda_i^{\mathrm{rev}} \approx -\lambda_i^{\mathrm{fwd}}$? Do they agree with the true exponents you identified in Problem 12.5(4)?

3. Vary the number of nearest neighbors used in the local regression over a range of values. For each, quantify the disagreement between forward and reversed spectra with a scalar measure of your choosing, and plot it. Use the plot to select a neighbor count, and comment on how this differs from choosing the neighbor count by eye.

4. Add Gaussian measurement noise at signal-to-noise ratios of 40, 20, and 10 dB and repeat. At what noise level does the test stop separating the two groups?

Problem 12.7

Computational

Return to the fixed-mass estimator of Section 12.3.1.

1. Implement it and apply it to the Lorenz $x(t)$ signal, sampled at a rate you justify. Plot Δ_n against n and identify the scaling region.

2. Estimate λ_1 from the slope and compare with the value from Problem 12.4. Convert between units of per-sample and per unit time and state which you are reporting.

3. Study the sensitivity of your estimate to the neighbor count r and to the length of the record. Over what range of r is the slope stable?

4. Implement the distance-weighted variant of the fixed-mass estimator given in Section 12.3.1, taking $w_m = w\big(\|\boldsymbol{x}_k - \boldsymbol{x}_{i(m)}\|\big)$ with a weight of your choosing that decays with distance. Compare against the unweighted estimate. Then show, analytically and numerically, that a weight depending only on r leaves the estimate unchanged.

5. Add measurement noise and determine the shortest n at which the scaling region remains identifiable. Explain the trade-off between the small-n contamination by noise and the large-n saturation at the attractor scale.

Problem 12.8

Computational

This problem quantifies the claim of Section 12.5.1, that the fit window matters more than the algorithm.

Using your Lorenz code and the fixed-distance estimator, compute a single Δ_n curve with $d = 5$, $\tau = 10$, $\epsilon = 0.5$ at a sampling interval $\Delta t = 0.01$.

1. Fit the slope over each of the windows $t \in [0.2, 0.9]$, $[0.5, 1.2]$, $[0.8, 1.6]$, $[1.1, 2.4]$ and $[0.2, 2.5]$. Tabulate λ_1 for each and compare against 0.906.

2. Plot Δ_n against n and mark your windows on it. Identify the transient region and explain, in terms of the orientation of $\boldsymbol{\delta}_0$ relative to the local unstable direction, why its slope exceeds the asymptotic one.

3. Repeat the whole computation at $\epsilon = 0.2$ and $\epsilon = 2.0$. Does the location of the scaling region move? Does its slope?

4. Now impose the Theiler window of Eq. (12.75), sweeping w from 0 to 200 samples. How large is the effect compared with that of part (1)?

5. Repeat part (1) with $\tau = 1$ instead of $\tau = 10$. Explain the result geometrically, and state whether a Theiler window would repair it.

Problem 12.9

needs Ch. 16 Extension

Return now to Problem 12.1, whose second question we were not yet equipped to answer.

1. Generate three records of equal length: the Lorenz $x(t)$ signal; Gaussian white noise; and a power-law correlated noise with spectrum $1/f^{\alpha}$, $\alpha \approx 1.5$, obtained by filtering white noise in the frequency domain.

2. Apply your direct estimator to all three. Report λ_1 for each, choosing a scaling region by eye and without looking at which record is which.

3. Now replot each Δ_n curve on doubly logarithmic axes. Which records show a slope near $1/2$? Explain what that slope indicates and why it can masquerade as a linear region on lin-log axes.

4. Generate a set of surrogates for each record by the method of Chapter 16 and compute the distribution of λ_1 over the surrogate ensemble. For which records does the original value fall outside the ensemble?

5. In light of parts (2)–(4), answer the question posed in Problem 12.1: what would it take to conclude that a measured signal is chaotic, and is a positive λ_1 alone sufficient?

Part III

Decomposition, Modeling, and Diagnostics

Part III — Decomposition, Modeling, and Diagnostics

This Part asks: *now that the record is reconstructed, what can be done with it?* Six answers, and they are chosen by the goal rather than by the data.

The Part divides in three. Chapters 13–14 recover a cleaner signal, by decomposing several channels or by projecting one reconstruction onto its own local geometry. Chapter 15 builds a model and asks it to predict. Chapters 16–18 diagnose: they test a hypothesis, track a change, and quantify structure that no single invariant captures.

One caution runs through all six. Each method has an assumption that its output does not advertise. A projective filter assumes a low-dimensional attractor and will manufacture one from noise. A surrogate test rejects a null hypothesis, which is not the same as establishing chaos. The chapters say where each assumption breaks.

You have several channels, or one channel and its derivatives.
Chapter 13. POD separates by energy, SOD by smoothness. Use SOD when the interesting motion is not the most energetic.

The record is noisy and the attractor is low-dimensional.
Chapter 14. Projective filtering, in its local and smooth forms. Check the residual for whiteness afterward; a filter that removes deterministic signal leaves a correlated residual.

You want to forecast, or to test determinism by forecasting.
Chapter 15. Local linear models, the error-versus-horizon curve, and the deterministic-versus-stochastic plot that reads the noise level off the neighborhood size.

You want to know whether nonlinearity is present at all.
Chapter 16. Choose the null before the statistic. A rejection against a phase-randomized surrogate says the record is not a filtered Gaussian process, and no more than that.

The system is drifting: fatigue, wear, a parameter creeping.
Chapter 17. Window the record, compute an invariant feature in each window, and track the feature. The drift must be slow compared with the dynamics for any of it to work.

The record is short, or nonstationary, or you want structure without an embedding.
Chapter 18. Recurrence quantification, permutation entropy, transfer entropy. These are descriptive measures; they need a null to become evidence.

The methods here are combined more often than they are used alone. A typical working sequence (Figure 1.8) is: reconstruct (Part II), filter (Chapter 14), estimate an invariant (Part II again), and test the estimate against surrogates (Chapter 16). The problems at the end of each chapter follow that sequence deliberately.

13 Multivariate Analysis

A *multivariate* record is one in which several quantities are measured at the same instants, so that each sample is a vector rather than a number. Every method in this chapter takes such a record and looks for a change of coordinates in which a few of the new variables carry most of what the record does. What distinguishes the methods from one another is the word "most": variance for the proper orthogonal decomposition, smoothness in time for the smooth orthogonal decomposition. The algebra is the same in both cases, an eigenvalue problem on a pair of matrices formed from the data, and the choice of the second matrix is the choice of what the decomposition will find.

It has many uses in data and model reduction, blind source signal separation, identifying the modal structure of dynamical systems, or identifying important dominant variables in data. Here, we will mainly focus on *proper orthogonal decomposition* (POD) and *smooth orthogonal decomposition* (SOD) as they are applied to the recorded measurements of a *scalar field*. The measurements can be taken from the scalar field using sensors or obtained by simulating a dynamical system. The multiple recorded measurements or their phase space reconstructions from the data matrices are used in the multivariate analysis.

The objective of POD is to obtain the best low-dimensional approximate description of high-dimensional data in the least-squares sense [27, 113, 162]. In contrast, SOD is aimed at obtaining the smoothest low-dimensional approximations of high-dimensional dynamical processes [26, 30, 31]. While POD only focuses on the spatial characteristics of the data, SOD considers both its temporal and spatial characteristics.

One convention before we begin, because the same quantities are indexed in several different ways in what follows. Spatial positions carry i and j and run to n; time samples carry k and run to m; modes carry p, and s where a second mode index is needed; and ν is the order of the time derivative. So u_j^k is the j-th sensor at the k-th instant, $\boldsymbol{\phi}_p$ is the p-th mode, and D^ν is the ν-th derivative operator.

13.1 Proper Orthogonal Decomposition

The POD (also referred to as *Karhunen-Loève decomposition* or *principal component analysis* [100, Ch. 7]) aims at finding a decomposition of the scalar field $u(\boldsymbol{x}, t)$, $\boldsymbol{x} \in \Omega \subset \mathbb{R}^n$ and $t \in T \subset \mathbb{R}$, into an orthonormal basis functions $\boldsymbol{\phi}_p(\boldsymbol{x})$ with time coefficients $q_p(t)$:

$$u(\boldsymbol{x}, t) = \sum_{p=1}^{\infty} q_p(t) \boldsymbol{\phi}_p(\boldsymbol{x}) \,, \tag{13.1}$$

such that, $\boldsymbol{\phi}_1(\boldsymbol{x})$ maximizes the projection of the field $u(\boldsymbol{x},t)$ onto itself:

$$\max_{\boldsymbol{\phi}_1}\left[\frac{1}{T}\int_0^T\left(\int_\Omega u(\boldsymbol{x},t)\boldsymbol{\phi}_1(\boldsymbol{x})d\boldsymbol{x}\right)^2 dt\right]; \tag{13.2}$$

$\boldsymbol{\phi}_2(\boldsymbol{x})$ maximizes the projection of the field onto the orthogonal complement of the first basis function, and so on. This is equivalent to the following constrained maximization problem:

$$\max_{\boldsymbol{\phi}_p}\left[\frac{1}{T}\int_0^T\left(\int_\Omega u(\boldsymbol{x},t)\boldsymbol{\phi}_p(\boldsymbol{x})\,\mathrm{d}\boldsymbol{x}\right)^2 \mathrm{d}t\right], \quad \text{with} \quad \int_\Omega \boldsymbol{\phi}_p(\boldsymbol{x})\boldsymbol{\phi}_s(\boldsymbol{x})\,\mathrm{d}\boldsymbol{x}=\delta_{ps} \tag{13.3}$$

This itself is reduced to the following integral eigenvalue problem:

$$\int_\Omega \boldsymbol{R}(\boldsymbol{x},\boldsymbol{y})\,\boldsymbol{\phi}_p(\boldsymbol{y})\,\mathrm{d}\boldsymbol{y}=\sigma_p^2\boldsymbol{\phi}_p(\boldsymbol{x}), \tag{13.4}$$

where

$$\boldsymbol{R}(\boldsymbol{x},\boldsymbol{y})=\frac{1}{T}\int_0^T u(\boldsymbol{x},t)u(\boldsymbol{y},t)dt, \tag{13.5}$$

is the auto-correlation function of our scalar field. It is convenient to read this as an operator acting on a field rather than as a symbol to be decorated: writing

$$\mathcal{R}[f](\boldsymbol{x},\boldsymbol{y})=\frac{1}{T}\int_0^T f(\boldsymbol{x},t)\,f(\boldsymbol{y},t)\,\mathrm{d}t, \tag{13.6}$$

the correlation of the field itself is $\mathcal{R}[u]$, and the correlation of any derived field is obtained by putting that field inside the brackets. We write $\boldsymbol{R}=\mathcal{R}[u]$ where no confusion can arise. The resultant eigenfunctions $\boldsymbol{\phi}_p(\boldsymbol{x})$ are called *proper orthogonal modes* (POMs) and the corresponding eigenvalues σ_p^2 are called *proper orthogonal values* (POVs). Now, the corresponding time dependent coefficients or *proper orthogonal coordinates* (POCs) are obtained as:

$$q_p(t)=\int_\Omega u(\boldsymbol{x},t)\boldsymbol{\phi}_p(\boldsymbol{x})\,\mathrm{d}\boldsymbol{x}. \tag{13.7}$$

Two things the energy ranking cannot do

The POD assures us that the first r POMs span the r-dimensional subspace that captures the most energy (mean square) of the scalar field. Energy is the only criterion, and two consequences follow. First, POD does not distinguish signal from noise. If the field is contaminated by white measurement noise of variance σ^2 that is independent across sensors and time, the expected correlation matrix becomes $\mathcal{R}[u]+\sigma^2\boldsymbol{I}$: the POMs are unchanged in expectation, every POV is raised by σ^2, and the modes whose POVs were below σ^2 are now indistinguishable from the $n-r$ noise directions, which all have POV $\approx\sigma^2$ (in a check with two harmonics mixed into five channels, POVs 1.52 and 0.056 became 1.76, 0.31, 0.25, 0.25, 0.25 under noise with $\sigma^2=0.25$, while the leading POM moved by less than 0.1°). The ordering of near-degenerate modes then becomes fragile, and the number of "significant" modes is set by the noise floor. Second, energy is not dynamical relevance: a mode with a small POV may be the one that carries the slow variable of interest. The smooth orthogonal decomposition below ranks by a different criterion, smoothness in time, under which white noise, which has huge roughness, is ranked *last* regardless of its energy; this is the point of Problem 13.2.

13.2 Smooth Orthogonal Decomposition

The basic idea of SOD is that given the same scalar field $u(\boldsymbol{x},t)$ we are looking for a linear projection of that field:

$$q(t) = \int_\Omega u(\boldsymbol{x},t)\boldsymbol{\psi}(\boldsymbol{x})d\boldsymbol{x}\,, \tag{13.8}$$

such that the resultant time coordinate $q(t)$ is *minimally rough* and has *maximal variance*. The minimal roughness or maximal smoothness can be characterized by the roughness metric that has been extensively used in smoothing splines [41, 163, 164, 177], where the γ-weighed roughness is minimized

$$RF(D^\nu f) = \int_a^b \gamma(t)(D^\nu f)(t)^2\,\mathrm{d}t\,, \tag{13.9}$$

over all functions f with ν-th derivatives defined on the time interval $[a,b]$, given the natural number ν and the non-negative integrable weight γ. For the uniform weight $\gamma = 1/T$ on $[0,T]$ the roughness is the time average of the squared derivative, $RF(D^\nu f) = \left\langle (D^\nu f)^2 \right\rangle$, where

$$\langle f \rangle = \frac{1}{T}\int_0^T f(t)\;\mathrm{d}t\,. \tag{13.10}$$

Most of the time, $\nu = 2$ is used in splines approximations. However, for our purposes, $\nu = 1$ shows good results and is less noisy when estimated from simulated or experimental data. As for the weighting, we use the uniform $\gamma = 1/T$ of Eq. (13.10), since our function f is empirical and would incorporate any further weighting naturally.

Therefore, the SOD idea translates into the following maximization problem

$$\max_{\boldsymbol{\psi}(\boldsymbol{x})} \frac{\left\langle q(t)^2 \right\rangle}{\left\langle (D^\nu q)(t)^2 \right\rangle}\,, \tag{13.11}$$

which can be written as constrained maximization problem

$$\max_{\boldsymbol{\psi}(\boldsymbol{x})} \left\langle q(t)^2 \right\rangle, \quad \text{under} \quad \left\langle (D^\nu q)(t)^2 \right\rangle = 1\,. \tag{13.12}$$

This in turn translates into the following Euler-Lagrange equation:

$$\delta \left\langle q(t)^2 \right\rangle = \lambda^2 \delta \left\langle (D^\nu q)(t)^2 \right\rangle\,, \tag{13.13}$$

where variations are in terms of $\boldsymbol{\psi}(\boldsymbol{x})$. We can evaluate the arguments of this Euler-Lagrange equation as

$$\begin{aligned}
\left\langle q(t)^2 \right\rangle &= \frac{1}{T}\int_0^T \int_\Omega u(\boldsymbol{x},t)\boldsymbol{\psi}(\boldsymbol{x})d\boldsymbol{x} \int_\Omega u(\boldsymbol{y},t)\boldsymbol{\psi}(\boldsymbol{y})d\boldsymbol{y}\;dt \\
&= \int_\Omega \int_\Omega \left[\frac{1}{T}\int_0^T u(\boldsymbol{x},t)u(\boldsymbol{y},t)\;dt\right]\boldsymbol{\psi}(\boldsymbol{x})d\boldsymbol{x}\;\boldsymbol{\psi}(\boldsymbol{y})d\boldsymbol{y} \\
&= \int_\Omega \int_\Omega \mathcal{R}[u]\,(\boldsymbol{x},\boldsymbol{y})\boldsymbol{\psi}(\boldsymbol{x})\,\mathrm{d}\boldsymbol{x}\;\boldsymbol{\psi}(\boldsymbol{y})\,\mathrm{d}\boldsymbol{y}\,,
\end{aligned} \tag{13.14}$$

and, similarly,

$$\begin{aligned}
\left\langle D^\nu q(t)^2 \right\rangle &= \frac{1}{T}\int_0^T \int_\Omega D^\nu u(\boldsymbol{x},t)\boldsymbol{\psi}(\boldsymbol{x})d\boldsymbol{x} \int_\Omega D^\nu u(\boldsymbol{y},t)\boldsymbol{\psi}(\boldsymbol{y})d\boldsymbol{y}\;dt \\
&= \int_\Omega \int_\Omega \left[\frac{1}{T}\int_0^T D^\nu u(\boldsymbol{x},t)D^\nu u(\boldsymbol{y},t)\;dt\right]\boldsymbol{\psi}(\boldsymbol{x})d\boldsymbol{x}\;\boldsymbol{\psi}(\boldsymbol{y})d\boldsymbol{y} \\
&= \int_\Omega \int_\Omega \mathcal{R}[D^\nu u]\,(\boldsymbol{x},\boldsymbol{y})\boldsymbol{\psi}(\boldsymbol{x})\,\mathrm{d}\boldsymbol{x}\;\boldsymbol{\psi}(\boldsymbol{y})\,\mathrm{d}\boldsymbol{y}\,,
\end{aligned} \tag{13.15}$$

where, in the notation of Eq. (13.6),

$$\mathcal{R}[D^\nu u]\,(\boldsymbol{x},\boldsymbol{y}) = \frac{1}{T}\int_0^T D^\nu u(\boldsymbol{x},t)\, D^\nu u(\boldsymbol{y},t)\;\mathrm{d}t \tag{13.16}$$

is the auto-correlation of the ν-th time derivative of the field. Nothing here is applied to the correlation, so there is no temptation to read it as a derivative of $\boldsymbol{R}$ — which would in any case be meaningless, since $\boldsymbol{R}(\boldsymbol{x},\boldsymbol{y})$ carries no time argument. Therefore, the Euler-Lagrange equation is transformed into the generalized integral eigenvalue problem:

$$\int_\Omega \mathcal{R}[u]\,(\boldsymbol{x},\boldsymbol{y})\boldsymbol{\psi}_p(\boldsymbol{y})\,\mathrm{d}\boldsymbol{y} = \lambda_p^2\int_\Omega \mathcal{R}[D^\nu u]\,(\boldsymbol{x},\boldsymbol{y})\boldsymbol{\psi}_p(\boldsymbol{y})\,\mathrm{d}\boldsymbol{y} \tag{13.17}$$

The solution to this eigenvalue problem will yield generalized eigenvalues λ_p^2 or *smooth orthogonal values* (SOVs) and corresponding generalized set of eigenfunctions $\boldsymbol{\psi}_p(\boldsymbol{x})$ or *smooth projective modes* (SPMs). Please note, that SPMs are not required to be orthogonal. Finally, we can reconstruct our field in terms of *smooth orthogonal modes* (SOMs) as

$$u(\boldsymbol{x},t) = \sum_{p=1}^{\infty} q_p(t)\boldsymbol{\phi}_p(\boldsymbol{x})\,, \tag{13.18}$$

where SOMs $\boldsymbol{\phi}_p(\boldsymbol{x})$ are bi-orthogonal set of SPMs $\boldsymbol{\psi}_p(\boldsymbol{x})$

$$\int_\Omega \boldsymbol{\phi}_p(\boldsymbol{x})\boldsymbol{\psi}_s(\boldsymbol{x})\,\mathrm{d}\boldsymbol{x} = \delta_{ps}\,, \tag{13.19}$$

and SOCs $q_p(t)$ also form an orthogonal set of functions:

$$\frac{1}{T}\int_0^T q_p(t)q_s(t)\,\mathrm{d}t = c_p\delta_{ps}\,. \tag{13.20}$$

13.3 Practical Calculation of POD and SOD

In the experimental context we usually sample the field $u(\boldsymbol{x},t)$ in discrete time intervals t_k $(k = 1,\dots,m)$ at discrete spatial points $\boldsymbol{x}_j$ $(j = 1,\dots,n)$. By introducing the following notation $u_j^k = u(\boldsymbol{x}_j,t_k)$, we can express the auto-correlation matrix components as

$$R_{ij} = \mathcal{R}[u]\,(\boldsymbol{x}_i,\boldsymbol{x}_j) = \frac{1}{m-1}\sum_{k=1}^{m} u_i^k u_j^k\,, \tag{13.21}$$

where the divisor $m-1$ replaces the $1/T$ of Eq. (13.6) (the two differ by the factor $m/(m-1)$, immaterial for m large, and the sample-covariance convention of Chapter 3 is kept). The columns of the data are assumed centered; if not, the first POM is the mean direction. Since every eigenvalue problem below involves the *ratio* or the eigenvectors of such matrices, the common factor cancels, and it may be dropped in favor of $\boldsymbol{Y}^{\mathrm{T}}\boldsymbol{Y}$. Now if we define a data matrix $\boldsymbol{Y}\in\mathbb{R}^{m\times n}$, with components $y_{kj} = u_j^k$, then the corresponding $n\times n$ auto-correlation matrices can be written as

$$\mathcal{R}[\boldsymbol{Y}] = \frac{1}{m-1}\boldsymbol{Y}^{\mathrm{T}}\boldsymbol{Y}, \quad \text{and} \quad \mathcal{R}[D^\nu\boldsymbol{Y}] = \frac{1}{m-1}(D^\nu\boldsymbol{Y})^{\mathrm{T}}D^\nu\boldsymbol{Y}\,. \tag{13.22}$$

If the measurements for $D^\nu\boldsymbol{Y}$ are not available, we can use finite-difference-based differential operators to estimate $D^\nu\boldsymbol{Y}$.

Given the estimate of the auto-correlation matrix, our *discrete eigenvalue problem* for POD is

$$\mathcal{R}[\boldsymbol{Y}]\ \boldsymbol{\phi}_p = \sigma_p^2 \boldsymbol{\phi}_p\,, \tag{13.23}$$

where the orthogonal set of $\{\boldsymbol{\phi}_p\}_{p=1}^n$ are POMs and σ_p^2 are the corresponding POVs. In practice, it is easier and more robust to just do the *singular value decomposition* [74, Ch. 2.4] of the data matrix $\boldsymbol{Y}$:

$$\boldsymbol{Y} = \boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{\Phi}^{\mathrm{T}}\,, \tag{13.24}$$

where $\boldsymbol{U} \in \mathbb{R}^{m\times n}$ has orthonormal columns, $\boldsymbol{\Sigma} = \mathrm{diag}(\sigma_p)$ is a diagonal matrix of *singular values*, and columns of $\boldsymbol{\Phi}$ are POMs. The POCs are the projections $\boldsymbol{q}_p = \boldsymbol{Y}\boldsymbol{\phi}_p = \sigma_p \boldsymbol{u}_p$, so they form the columns of

$$\boldsymbol{Q} = \boldsymbol{U}\boldsymbol{\Sigma} = \boldsymbol{Y}\boldsymbol{\Phi}\,, \qquad \boldsymbol{Q}^{\mathrm{T}}\boldsymbol{Q} = \boldsymbol{\Sigma}^2\,, \tag{13.25}$$

the same convention used for the SOCs below: the columns of $\boldsymbol{U}$ are the POCs normalized to unit length, not the POCs themselves. Since $\mathcal{R}[\boldsymbol{Y}] = \boldsymbol{Y}^{\mathrm{T}}\boldsymbol{Y}/(m-1)$, the POV is $\sigma_p^2/(m-1)$ rather than σ_p^2 itself; the two differ only by that fixed factor, so the ordering of the modes is unaffected.

For SOD we would need to estimate the corresponding time derivative of the data matrix, and then solve the following *generalized eigenvalue problem*:

$$\mathcal{R}[\boldsymbol{Y}]\ \boldsymbol{\psi}_p = \lambda_p^2\, \mathcal{R}[D^\nu \boldsymbol{Y}]\ \boldsymbol{\psi}_p\,. \tag{13.26}$$

In practice, similarly to POD, it is more expedient and robust to just calculate the *economy-size generalized singular value decomposition* [74, Ch. 8.7.4] of the matrix pair $\boldsymbol{A} = \boldsymbol{Y}$ and $\boldsymbol{B} = D^\nu \boldsymbol{Y}$ as

$$\boldsymbol{A} = \boldsymbol{U}_1\boldsymbol{\Sigma}_1\boldsymbol{\Phi}^{\mathrm{T}}\,, \qquad \text{and} \qquad \boldsymbol{B} = \boldsymbol{U}_2\boldsymbol{\Sigma}_2\boldsymbol{\Phi}^{\mathrm{T}}\,, \tag{13.27}$$

where $\boldsymbol{U}_1 \in \mathbb{R}^{m\times n}$ and $\boldsymbol{U}_2 \in \mathbb{R}^{(m-\nu)\times n}$ are matrices with orthonormal columns (the finite-difference derivative has ν fewer rows; trim $\boldsymbol{A}$ to the same rows), $\boldsymbol{\Sigma}_1 = \mathrm{diag}(\sigma_{1,p}) \in \mathbb{R}^{n\times n}$ and $\boldsymbol{\Sigma}_2 = \mathrm{diag}(\sigma_{2,p}) \in \mathbb{R}^{n\times n}$ are diagonal matrices, and columns of $\boldsymbol{\Phi} = [\boldsymbol{\phi}_1, \boldsymbol{\phi}_2, \ldots, \boldsymbol{\phi}_n] \in \mathbb{R}^{n\times n}$ contain the unit norm SOMs $\boldsymbol{\phi}_p$. (This is one of two normalizations in use: the GSVD routines of MATLAB and LAPACK return instead $\boldsymbol{\Sigma}_1^2 + \boldsymbol{\Sigma}_2^2 = \boldsymbol{I}$ with a non-normalized $\boldsymbol{\Phi}$, the convention adopted in Chapter 14. The SOVs and the directions of the SPMs and SOMs are the same under either; only the lengths of the columns differ.) SPMs are then given by the columns of

$$\boldsymbol{\Psi} = \boldsymbol{\Phi}^{-\mathrm{T}}\,, \tag{13.28}$$

and the corresponding SOVs or discrete generalized eigenvalues are the squares of the ratios of the two sets of singular values,

$$\lambda_p^2 = \left(\frac{\sigma_{1,p}}{\sigma_{2,p}}\right)^2\,. \tag{13.29}$$

The corresponding SOCs are orthogonal and form the columns of $\boldsymbol{Q} = \boldsymbol{U}_1\boldsymbol{\Sigma}_1 = \boldsymbol{Y}\boldsymbol{\Psi}$, or

$$\boldsymbol{q}_p = \boldsymbol{Y}\boldsymbol{\psi}_p\,, \tag{13.30}$$

and

$$\boldsymbol{Q}^{\mathrm{T}}\boldsymbol{Q} = \boldsymbol{\Sigma}_1\boldsymbol{U}_1^{\mathrm{T}}\boldsymbol{U}_1\boldsymbol{\Sigma}_1 = \boldsymbol{\Sigma}_1^2\,. \tag{13.31}$$

When the pencil is singular

The generalized eigenproblem is well posed only if $\mathcal{R}[D^\nu \boldsymbol{Y}]$ is positive definite, which requires $D^\nu \boldsymbol{Y}$ to have full column rank. If the field is exactly r-dimensional with $r < n$ (a noise-free simulation with r active modes, or more sensors than independent motions), both $\mathcal{R}[\boldsymbol{Y}]$ and $\mathcal{R}[D^\nu \boldsymbol{Y}]$ are rank r, the pencil is *singular*, and $n-r$ of the ratios $\sigma_{1,p}/\sigma_{2,p}$ are $0/0$: their numerical values are arbitrary and can appear anywhere in the ordered list. This is met immediately in the noise-free toy problems (Problem 13.1), where the remedy is to reduce the data first, by POD, to its r-dimensional range and solve the pencil there; with measured data the noise regularizes both matrices, and the spurious ratios are pushed to the bottom of the list, alongside the noise SOVs described in Section 13.1.

The same problem elsewhere

The maximization of $\langle q^2 \rangle / \langle \dot{q}^2 \rangle$ over linear projections is not unique to structural dynamics. In machine learning it is *slow feature analysis* [224], which extracts the slowest-varying linear (or, after a nonlinear expansion of the inputs, nonlinear) functions of a multivariate signal by minimizing $\langle \dot{q}^2 \rangle$ at unit variance, and which arrives at the same generalized eigenproblem of the pencil $(\mathcal{R}[\boldsymbol{Y}], \mathcal{R}[D\boldsymbol{Y}])$. In numerical linear algebra it is the symmetric-definite generalized eigenproblem, solved by the GSVD above. In linear vibration, POD of the response of a lightly damped structure to broadband excitation returns the linear normal modes when the mass matrix is proportional to the identity, a result due to Feeny and Kappagantu [63]; SOD returns them for a general mass matrix, since the pencil $(\mathcal{R}[\boldsymbol{Y}], \mathcal{R}[\dot{\boldsymbol{Y}}])$ inherits the structure of $(\boldsymbol{K}, \boldsymbol{M})$ [31]. This is the basis of SOD-based modal identification [31, 61].

13.4 Geometric Interpretation of POD

To illustrate the geometry of POD, we consider a matrix $\boldsymbol{Y} \in \mathbb{R}^{m \times n}$ containing a cloud of m points generated by our n-dimensional trajectory, such that each column of $\boldsymbol{Y}$ has zero mean. Then POD can be pictured as fitting the best ellipsoid to this data in the least-squares sense as shown in Fig. 13.1. The semi-axes of the covariance ellipsoid have lengths $\sigma_p/\sqrt{m-1}$, the standard deviations of the data along the POMs $\boldsymbol{\phi}_p$, which span the axis directions. The signal energy or variance in the p-th POM is the POV $\sigma_p^2/(m-1)$, and the total variance is $\sum_{p=1}^{n} \sigma_p^2/(m-1)$; the fraction captured by the first r modes is $\sum_{p \le r} \sigma_p^2 / \sum_p \sigma_p^2$, the number usually quoted.

13.5 Geometric Interpretation of SOD:

Now we consider a generic pair of consecutive points $\boldsymbol{y}_k$ and $\boldsymbol{y}_{k+1}$ from a scalar field matrix $\boldsymbol{Y}$ composed of trajectory points sampled with rate of $\Delta t = 1$ as shown in Fig. 13.2. Then the velocity vector for $\boldsymbol{y}_k$ can be approximated as $\boldsymbol{v}_k = \boldsymbol{y}_{k+1} - \boldsymbol{y}_k$. Now, if we follow the SOD maximization procedure, first we would like to identify a unit vector $\hat{\boldsymbol{\psi}}_1$, such that we maximize the ratio of the mean squared projections onto this vector, $q_{k1} = \boldsymbol{y}_k^{\mathrm{T}} \hat{\boldsymbol{\psi}}_1$ against $\dot{q}_{k1} = \boldsymbol{v}_k^{\mathrm{T}} \hat{\boldsymbol{\psi}}_1$. In other words $\hat{\boldsymbol{\psi}}_1$ corresponds to the stationary value of this fraction:

$$\lambda_1^2 = \max_{\hat{\boldsymbol{\psi}}_1} \left\{ \frac{\left\langle \left| \boldsymbol{y}_k^{\mathrm{T}} \hat{\boldsymbol{\psi}}_1 \right|^2 \right\rangle}{\left\langle \left| \boldsymbol{v}_k^{\mathrm{T}} \hat{\boldsymbol{\psi}}_1 \right|^2 \right\rangle} \right\} = \max_{\hat{\boldsymbol{\psi}}_1} \left\{ \frac{\langle q_{k1}^2 \rangle}{\langle \dot{q}_{k1}^2 \rangle} \right\}, \tag{13.32}$$

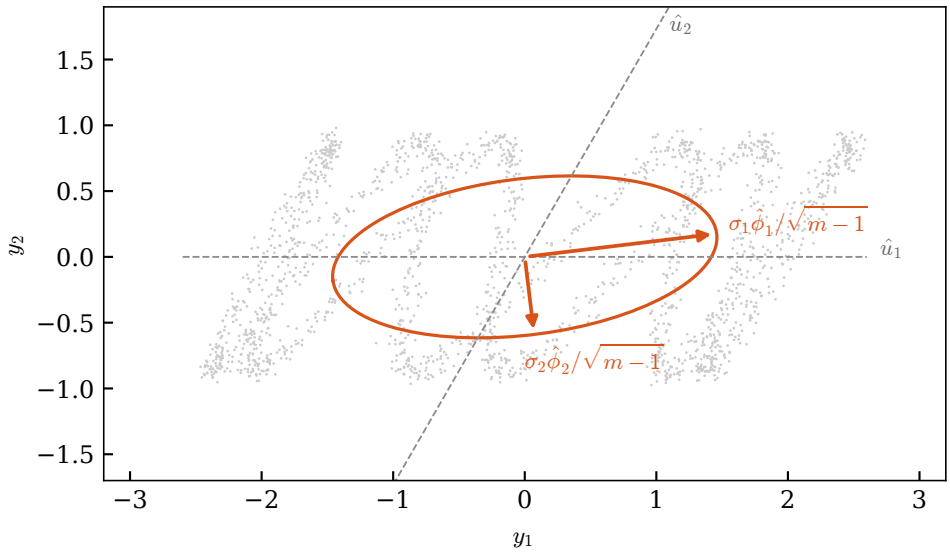


Figure 13.1: Geometric interpretation of POD, computed on a two-dimensional toy record: $m = 2000$ samples of $\boldsymbol{y}_k = 2\cos(0.15k)\,\hat{\boldsymbol{u}}_1 + \cos(0.9k + 1)\,\hat{\boldsymbol{u}}_2$ at $\Delta t = 1$ with Gaussian noise of standard deviation 0.05 in each channel (seed 0), where the component directions $\hat{\boldsymbol{u}}_1$ and $\hat{\boldsymbol{u}}_2$ (dashed) are at 0° and 60° and so are *not* orthogonal. The ellipse is the covariance ellipse, with semi-axes $\sigma_p/\sqrt{m-1} = 1.470$ and 0.594 along the POMs $\hat{\boldsymbol{\phi}}_p$ (arrows), which come out at 6.9° and 96.9°: POD returns the orthogonal axes of the ellipse, not the directions that generated the data. Script: `python/mkfig_PODSOD.py`.

where the resultant stationary value λ_1^2 is the smooth orthogonal value corresponding to the dominant SPM $\hat{\boldsymbol{\psi}}_1$, with $\boldsymbol{q}_1 = \boldsymbol{Y}\hat{\boldsymbol{\psi}}_1$ the associated SOC. Once we identify $\hat{\boldsymbol{\psi}}_1$, we look for the second direction $\hat{\boldsymbol{\psi}}_2$ among the vectors that are $\mathcal{R}[\boldsymbol{V}]$-orthogonal to $\hat{\boldsymbol{\psi}}_1$, where $\boldsymbol{V}$ is the matrix whose rows are the velocity vectors $\boldsymbol{v}_k^{\mathrm{T}}$, so that $\hat{\boldsymbol{\psi}}_2^{\mathrm{T}}\mathcal{R}[\boldsymbol{V}]\,\hat{\boldsymbol{\psi}}_1 = \langle \dot{q}_{k2}\,\dot{q}_{k1}\rangle = 0$. Linear independence alone is not a usable constraint: the supremum of the ratio over all $\hat{\boldsymbol{\psi}}_2$ merely not parallel to $\hat{\boldsymbol{\psi}}_1$ is λ_1^2 again, approached as $\hat{\boldsymbol{\psi}}_2 \to \hat{\boldsymbol{\psi}}_1$, and is not attained. Under the orthogonality constraint the maximum is the second stationary value of the ratio, the second SOV:

$$\lambda_2^2 = \max_{\hat{\boldsymbol{\psi}}_2^{\mathrm{T}}\mathcal{R}[\boldsymbol{V}]\hat{\boldsymbol{\psi}}_1=0} \left\{ \frac{\left\langle \left| \boldsymbol{y}_k^{\mathrm{T}}\hat{\boldsymbol{\psi}}_2 \right|^2 \right\rangle}{\left\langle \left| \boldsymbol{v}_k^{\mathrm{T}}\hat{\boldsymbol{\psi}}_2 \right|^2 \right\rangle} \right\} = \max_{\hat{\boldsymbol{\psi}}_2^{\mathrm{T}}\mathcal{R}[\boldsymbol{V}]\hat{\boldsymbol{\psi}}_1=0} \left\{ \frac{\langle q_{k2}^2 \rangle}{\langle \dot{q}_{k2}^2 \rangle} \right\}, \tag{13.33}$$

and so on, each new $\hat{\boldsymbol{\psi}}_p$ being $\mathcal{R}[\boldsymbol{V}]$-orthogonal to all the previous ones, until we obtain a full set of $\{\hat{\boldsymbol{\psi}}_p\}_{p=1}^n$. These are the generalized eigenvectors of the pencil $(\mathcal{R}[\boldsymbol{Y}], \mathcal{R}[\boldsymbol{V}])$, and they are automatically $\mathcal{R}[\boldsymbol{Y}]$-orthogonal as well, so the SOCs and their time derivatives are both mutually uncorrelated. Then the corresponding bi-orthogonal set of vectors are obtained by

$$\boldsymbol{\Phi} = \hat{\boldsymbol{\Psi}}^{-\mathrm{T}}, \tag{13.34}$$

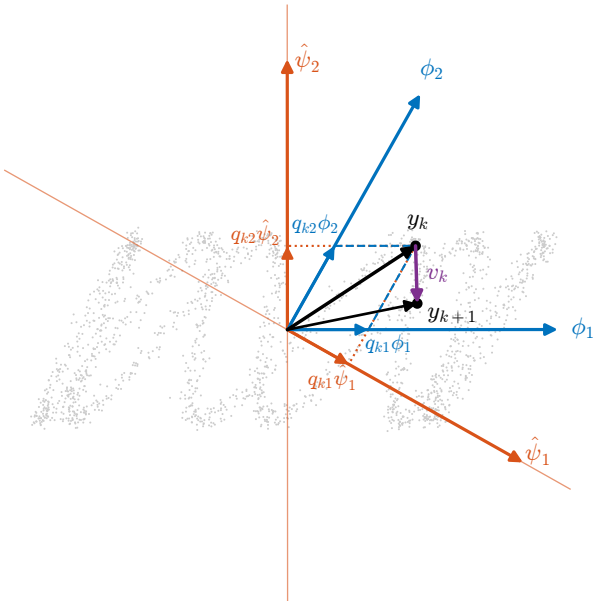


Figure 13.2: Geometric interpretation of smooth orthogonal decomposition, on the record of Fig. 13.1. For one pair of consecutive points the velocity is $\boldsymbol{v}_k = \boldsymbol{y}_{k+1} - \boldsymbol{y}_k$. Solving the pencil $(\mathcal{R}[\boldsymbol{Y}], \mathcal{R}[\boldsymbol{V}])$ gives SOVs $\lambda_1^2 = 38.7$ and $\lambda_2^2 = 1.31$ (the values for the noise-free record are $1/(2\sin(\omega_p/2))^2 = 44.5$ and 1.32, the slow component first), SPMs $\hat{\boldsymbol{\psi}}_p$ at $-29.4°$ and $90.0°$, and SOMs $\boldsymbol{\phi}_p$ at $0.0°$ and $60.6°$ — along the two component directions, which POD did not recover. The projections $q_{kp} = \boldsymbol{y}_k^{\mathrm{T}}\hat{\boldsymbol{\psi}}_p$ onto the SPMs are the coordinates of $\boldsymbol{y}_k$ in the SOM basis, $\boldsymbol{y}_k = \sum_p q_{kp}\boldsymbol{\phi}_p$; $\boldsymbol{\phi}_1$ is perpendicular to $\hat{\boldsymbol{\psi}}_2$ and $\boldsymbol{\phi}_2$ to $\hat{\boldsymbol{\psi}}_1$. Script: `python/mkfig_PODSOD.py`.

where $\boldsymbol{\Psi} = \left[\hat{\boldsymbol{\psi}}_1, \hat{\boldsymbol{\psi}}_2, \ldots, \hat{\boldsymbol{\psi}}_n\right] \in \mathbb{R}^{n\times n}$ is SPMs matrix and $\boldsymbol{\Phi} = [\boldsymbol{\phi}_1, \boldsymbol{\phi}_2, \ldots, \boldsymbol{\phi}_n] \in \mathbb{R}^{n\times n}$ are SOMs (no longer unit norm) that can be used to describe any point in our matrix $\boldsymbol{Y}$ as:

$$\boldsymbol{y}_k = \sum_{p=1}^{n} q_{kp}\boldsymbol{\phi}_p\,, \quad \text{and} \quad \dot{\boldsymbol{y}}_k = \sum_{p=1}^{n} \dot{q}_{kp}\boldsymbol{\phi}_p\,. \tag{13.35}$$

where $\boldsymbol{q}_p = \boldsymbol{Y}\hat{\boldsymbol{\psi}}_p$ or $q_{kp} = \boldsymbol{y}_k^{\mathrm{T}}\hat{\boldsymbol{\psi}}_p$. On the figure we only show two pairs of basis vectors. As shown, $\boldsymbol{\phi}_1$ is perpendicular to $\hat{\boldsymbol{\psi}}_2$, and $\boldsymbol{\phi}_2$ is perpendicular to $\hat{\boldsymbol{\psi}}_1$. While the $\hat{\boldsymbol{\psi}}_p$ are used during the maximization process, the $\boldsymbol{\phi}_p$ are used to describe our data.

Another alternative is to visualize SOD as a POD of $\boldsymbol{A}$ followed by a POD of $\boldsymbol{B}$, as shown in Fig. 13.3. We start from the SVD $\boldsymbol{A} = \boldsymbol{U}_1\boldsymbol{\Sigma}_1\boldsymbol{V}_1^{\mathrm{T}}$ and use its rotation $\boldsymbol{V}_1$ and scaling $\boldsymbol{\Sigma}_1$ to whiten $\boldsymbol{B}$ in the POD basis of $\boldsymbol{A}$, giving $\boldsymbol{B}_1 = \boldsymbol{B}\boldsymbol{V}_1\boldsymbol{\Sigma}_1^{-1}$. The SVD $\boldsymbol{B}_1 = \boldsymbol{U}_2\boldsymbol{\Sigma}_2\boldsymbol{V}_2^{\mathrm{T}}$ then identifies the principal directions of $\boldsymbol{B}$ in that basis, and undoing the whitening carries them back:

$$\boldsymbol{B} = \boldsymbol{U}_2\boldsymbol{\Sigma}_2\boldsymbol{\Phi}^{\mathrm{T}}\,, \qquad \boldsymbol{A} = \boldsymbol{A}_1\boldsymbol{\Phi}^{\mathrm{T}}\,, \qquad \boldsymbol{\Phi} \triangleq \boldsymbol{V}_1\boldsymbol{\Sigma}_1\boldsymbol{V}_2\,, \quad \boldsymbol{A}_1 \triangleq \boldsymbol{U}_1\boldsymbol{V}_2\,. \tag{13.36}$$

The two factorizations share $\boldsymbol{\Phi}$, which is what makes this a *generalized* singular value decomposition, and its columns are the smooth orthogonal modes. Two features of Eq. (13.36) are easy to get wrong. $\boldsymbol{A}_1$ is orthogonal, so the pair of diagonal factors is $(\boldsymbol{I}, \boldsymbol{\Sigma}_2)$ and the generalized singular values are

$1/\sigma_{2i}$. The smoothest mode is therefore the direction along which the whitened derivative matrix $\boldsymbol{B}_1$ has its *smallest* singular value, the direction of least derivative variance per unit displacement variance, and the order of the SVD of $\boldsymbol{B}_1$ must be reversed to list the modes from smoothest to roughest. And $\boldsymbol{A}$ is *not* $\boldsymbol{U}_1\boldsymbol{\Phi}^{\mathrm{T}}$: the rotation $\boldsymbol{V}_2$ found in the second SVD stays with the left factor. The check is `python/chk_sodfig.py`, which verifies both identities and confirms that $\boldsymbol{\Phi}$ and $1/\sigma_{2i}$ agree with `python/gsvd.py` on random pairs.

13.6 Where the Decompositions Go Next

POD and SOD are the two linear decompositions this book uses, and everything that follows applies one of them to a data matrix built in one of three ways. Nothing in the algebra above cares which.

Delay vectors as columns. A scalar record embedded as in Chapter 8 gives $\boldsymbol{Y} \in \mathbb{R}^{m\times d}$ with the d delay coordinates as columns. POD of this matrix is the singular-system analysis of Broomhead and King [14]: the POMs rotate the delay coordinates into directions ordered by variance, and the trailing POVs measure the noise floor. Applied globally this is a linear filter; applied to the neighborhood of each point it becomes the local projective noise reduction of Chapter 14, where the neighborhood cloud is decomposed, the trailing directions are discarded as noise, and the point is projected onto the leading ones. The SOD version of the same step decomposes a bundle of trajectory strands and keeps the smooth directions, which is where the roughness criterion pays: measurement noise is the roughest component present, so it is separated by its time behavior rather than by its amplitude, and the projection removes it at lower cost to the attractor's geometry. Chapter 14 quantifies the gain on the Lorenz and Duffing records.

Features as columns. Chapter 17 computes, from successive windows of a record, several scalar statistics (recurrence-based, entropy-based, distance-based) and stacks them as the columns of a feature matrix indexed by window. POD of that matrix asks which combination of features carries the most variance across windows; SOD asks which combination varies most slowly, and it is the slow combination that tracks a drifting parameter of the system under study. The argument for why a smooth slow variable emerges from features that are individually discontinuous in the parameter is made there.

Sensors as columns. In the original setting, a scalar field sampled at n locations, $\boldsymbol{Y}$ holds the responses of n sensors on a vibrating structure. The pencil $(\mathcal{R}[\boldsymbol{Y}], \mathcal{R}\left[\dot{\boldsymbol{Y}}\right])$ then inherits the structure of the stiffness–mass pencil $(\boldsymbol{K}, \boldsymbol{M})$, and SOD returns the linear normal modes and natural frequencies of the structure from output-only data [31, 61]; related generalized-eigenvalue formulations for time-domain modal identification are given in [232]. The present chapter supplies the variational chain and the GSVD computation on which these rest.

In each setting the same three questions decide whether the decomposition is informative, and they should be asked before the modes are interpreted: how many modes rise above the noise floor σ^2 of the trailing POVs; whether the pencil is well conditioned, i.e., whether $\mathcal{R}[D^{\nu}\boldsymbol{Y}]$ has full rank at the noise level present; and how the derivative was estimated, since every SOV is a ratio whose denominator is a differentiated record (Problem 13.3).

Pitfall: what POD and SOD can and cannot establish

Both decompositions are linear changes of coordinates fitted to second moments. POD returns the orthogonal directions of greatest variance. SOD returns coordinates ranked by smoothness, mutually uncorrelated and with uncorrelated derivatives. Neither recovers the directions that generated the data unless they happen to satisfy its criterion: in Fig. 13.1 the components lie at 0° and 60°, and POD returns 6.9° and 96.9°. Neither yields a nonlinear coordinate. Motion on a curved invariant manifold is spread over several linear modes, so the count of significant modes overstates the dimension of the motion. That count is set by the noise floor as much as by the system (the remark on the energy ranking in Section 13.1). An SOC is fixed only up to scale, and an SOV inherits the error of the derivative estimate in its denominator. The reading of SOMs as linear normal modes [31, 61] holds for a linear structure under the assumptions of those papers, not for a nonlinear response. And the smoothest coordinate is not automatically a slow variable of the system (Chapter 17).

Problems

Problem 13.1

Core

1. Generate a vector-valued time series of harmonic functions with different frequency and magnitude contents. Make sure you consider at least seven ($n = 7$) different frequencies and seven different magnitudes. Record the vector-valued time series to include at least several full periods of the highest frequency component. As a result you should have a matrix $Y \in \mathbb{R}^{m\times n}$, where $m > n$.
2. Apply POD and SOD to this time series and identify the corresponding time coordinates, modes, and eigenvalues.
3. Interpret the POMs and SOMs and the corresponding time coordinates.
4. Now take the same matrix Y and make sure each column is composed of harmonics with the same unit magnitude.
5. Redo the POD and SOD analysis. Please explain discrepancies, if any.

Problem 13.2

Computational

Problem 13.1 applied POD and SOD to synthetic harmonics. This problem asks what SOD buys that POD does not, using the property SOD is built on: minimal roughness at maximal variance.

1. Build a vector-valued record in which the *energy* order and the *smoothness* order of the components disagree: take one low-amplitude slow harmonic and one high-amplitude fast harmonic, mixed through a fixed random $n \times 2$ matrix, then add white noise to every channel.
2. Apply POD. Which mode does it rank first, and why?
3. Apply SOD with $\nu = 1$ in Eq. (13.9). Which mode does it rank first? Explain the difference in terms of the ratio $\langle q^2 \rangle / \langle (D^\nu q)^2 \rangle$ being maximized.
4. Vary the noise level and report the level at which POD can no longer recover the slow component while SOD still can, or vice versa.

5. Repeat with $\nu = 2$. The chapter says $\nu = 1$ "shows good results and is less noisy when estimated from simulated or experimental data." Test that claim: compare the mode estimates for $\nu = 1$ and $\nu = 2$ at each noise level and quantify which is more variable across independent realizations.

Problem 13.3

needs Ch. 14 Computational

SOD needs a data-based derivative, and that is where it is fragile.

1. Implement D^1 three ways: a two-point forward difference, a central difference, and a Savitzky–Golay differentiating filter of your choice of order and window.

2. For a clean record, verify all three give the same SOD modes.

3. Add white noise at 40, 30 and 20 dB SNR and repeat. Rank the three differentiators by how well the recovered modes match the noise-free answer.

4. Explain why differentiation amplifies high-frequency noise, and relate the Savitzky–Golay window length to the trade-off between that amplification and the bias it introduces.

5. Chapter 14 notes that SOD is inapplicable to map-generated data for exactly this reason. Demonstrate it: apply your SOD code to a Hénon record and describe what goes wrong.

Problem 13.4

Core

What a smooth orthogonal value measures. Take $\nu = 1$ and the uniform weight, so that $\lambda^2 = \langle q^2 \rangle / \langle \dot{q}^2 \rangle$.

1. For $q(t) = A \sin \omega t$ show that $\lambda^2 = 1/\omega^2$, independent of the amplitude A. When the derivative is replaced by the first difference $(q_{k+1} - q_k)/\Delta t$ of samples taken at Δt, show that $\lambda^2 = \Delta t^2 / [4 \sin^2(\omega \Delta t/2)]$, and that this reduces to $1/\omega^2$ for $\omega \Delta t \ll 1$.

2. For q_k white noise of variance σ^2 with the unit-step difference $q_{k+1} - q_k$ as the derivative, show that $\lambda^2 = 1/2$, independent of σ^2. Check both results numerically on 2×10^4 samples; you should reproduce the harmonic value to three digits and the white-noise value to two.

3. Mix two harmonics, $0.1 \sin t$ and $3 \sin 8t$ sampled at $\Delta t = 0.01$, into five channels through a fixed random 5×2 matrix, without noise. Apply POD and report the two nonzero POVs. Then solve the SOD pencil. It is singular (five channels, two independent motions), so first reduce the data to its two-dimensional POD range as the remark on singular pencils prescribes, then solve. You should find SOVs of about 10^4 and 156, the values $1/(\omega \Delta t)^2$ of part (1) for the two frequencies: SOD has ranked the weak slow harmonic first and the strong fast one second, the reverse of POD.

4. Add white noise of standard deviation 10^{-3} to every channel and solve the five-channel pencil directly. You should find the three trailing SOVs at about 0.5, the white-noise value, and the leading SOV reduced from 10^4 to a few thousand while the second is unchanged at 156. Explain, from the ratio being maximized, why noise of this size damages the slow mode's SOV and not the fast mode's, and estimate the noise level at which the two would swap order.

Problem 13.5

Computational

POD of a trajectory matrix, the singular-system analysis of Section 13.6, as a noise-floor estimator.

1. Generate the Lorenz x record at $\Delta t = 0.01$ (60,000 samples) and embed it with $\tau = 1$ and $d = 10$ so that the delay vectors are the rows of $\boldsymbol{Y} \in \mathbb{R}^{m \times 10}$. Centre the columns, compute the singular values, and report the POVs $\sigma_p^2/(m-1)$. You should find them falling geometrically, by a factor of forty to a hundred from each mode to the next, all the way down to the machine-precision floor near 10^{-15} at $p = 10$: a smooth flow sampled densely is very nearly low-rank, but not exactly.

2. Add white noise of standard deviation $0.05\sigma_x$ and then $0.2\sigma_x$ and repeat. You should find the trailing POVs flat at the noise variance σ^2, to within a few percent, while the leading POVs are raised by the same σ^2. Confirm the statement of Section 13.1 that the POMs are unchanged in expectation: compute the angle between the leading POM of the clean and noisy matrices.

3. Use the trailing POVs to estimate σ^2 without knowledge of the clean record, and hence the signal-to-noise ratio. How many modes stand above the floor at each noise level, and how does that number compare with the embedding dimension $d = 3$ found for this record in Chapter 10? Why are they not the same quantity?

4. Repeat part (1) with $\tau = 16$ instead of 1, the first minimum of the average mutual information for this record at this sampling interval (Chapter 9; the value scales with Δt, so it is twice the 8 samples quoted there for $\Delta t = 0.02$). How does the POV spectrum change, and why does a larger lag spread the variance over more modes? State what this implies for the "number of significant modes" as a dimension estimate.

Problem 13.6

Experimental

POD and SOD of the two-well oscillator record of Section 1.4.1.

1. Using the sampling rate and forcing frequency given with the record, embed the strain with $\tau = 1$ and a window $(d-1)\tau$ of about one forcing period, so that d is the number of samples per period. Form the trajectory matrix, centre its columns, and compute the POV spectrum. Plot it on a logarithmic axis.

2. Identify the flat floor of trailing POVs and read the noise variance σ^2 from it, as in Problem 13.5. Convert to a signal-to-noise ratio in decibels and compare with the spectral-floor estimate of Problem 4.6. Which of the two estimates is the more trustworthy for this record, and why might they differ?

3. Solve the SOD pencil for the same matrix with the first-difference derivative. How many SOVs stand clearly above the white-noise value of $1/2$ (Problem 13.4)? Compare that count with the number of POVs above the floor. Which decomposition separates the deterministic motion from the measurement noise more sharply, and does the answer depend on the noise level you found?

4. Plot the first three POCs and the first three SOCs against time over a few forcing periods. Identify the coordinate that follows the forcing, the one that follows the slow well-to-well switching, and the ones that carry noise. Which decomposition puts the slow switching coordinate first, and is that what Problem 13.2 predicts?

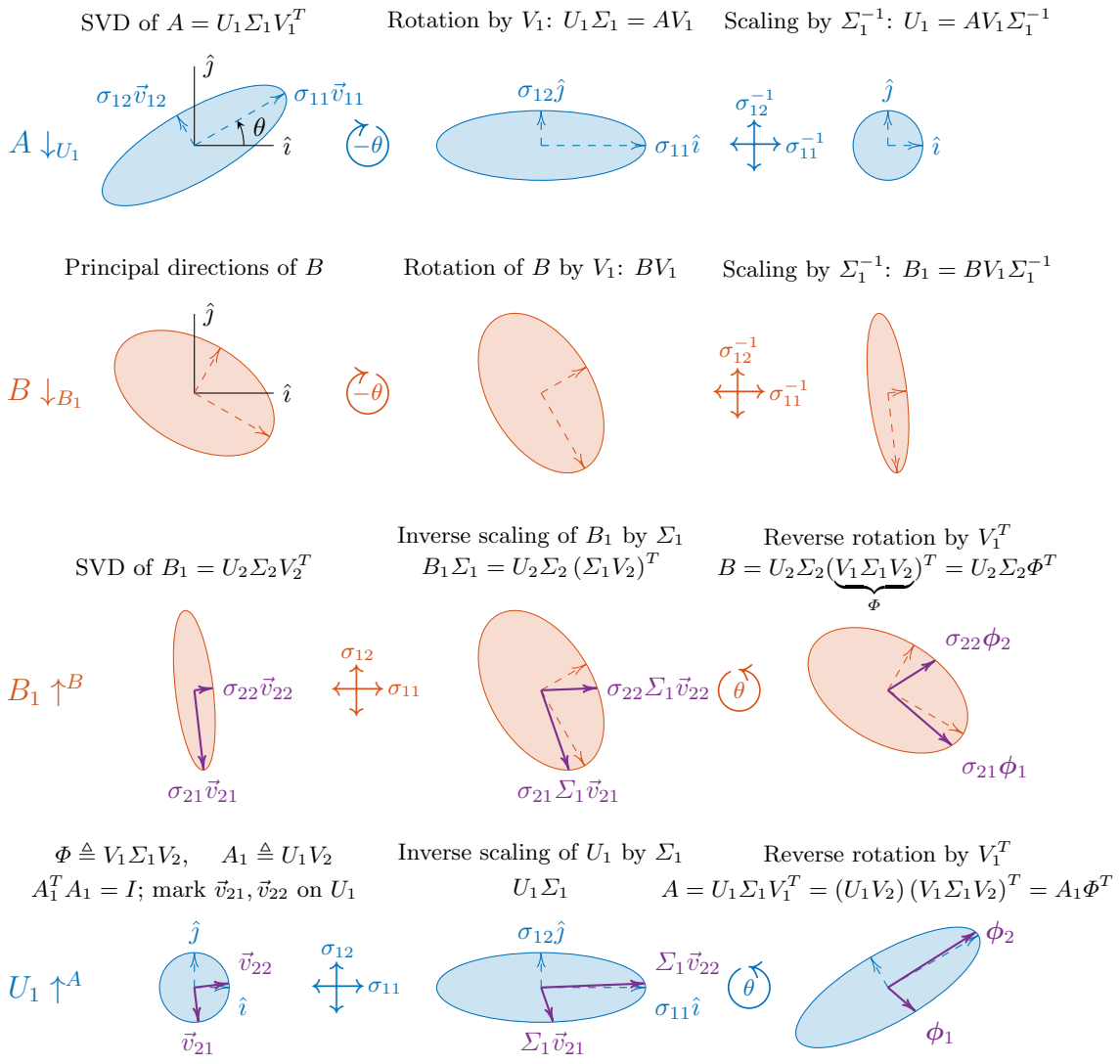


Figure 13.3: The generalized singular value decomposition of the pair $(\boldsymbol{A}, \boldsymbol{B})$ used by SOD, read as a sequence of transformations. Each shaded ellipse is $E(\boldsymbol{M}) = \boldsymbol{M}^{\mathrm{T}}(\text{unit ball})$, so its semi-axes are the singular values of $\boldsymbol{M}$ along the right singular vectors of $\boldsymbol{M}$, and right multiplication acts on the ellipse from the left: $E(\boldsymbol{MQ}) = \boldsymbol{Q}^{\mathrm{T}}E(\boldsymbol{M})$ for orthogonal $\boldsymbol{Q}$ and $E(\boldsymbol{MD}) = \boldsymbol{D}E(\boldsymbol{M})$ for diagonal $\boldsymbol{D}$. Rows 1 and 2 whiten $\boldsymbol{B}$ in the SVD basis of $\boldsymbol{A}$; rows 3 and 4 undo the whitening, and the two rows end with the same $\boldsymbol{\Phi}$, whose columns are the purple arrows of the last panel. Note that a left factor of $\boldsymbol{M}$ is invisible in $E(\boldsymbol{M})$, so the row-4 ellipses cannot distinguish $\boldsymbol{A}$ from $\boldsymbol{U}_1\boldsymbol{\Phi}^{\mathrm{T}}$; the algebra of Eq. (13.36) is what settles it. Angles and lengths are exact for $\theta = 30°$, $\Sigma_1 = \mathrm{diag}(3, 1)$ and $E(\boldsymbol{B})$ with semi-axes 5 and 3 at $-30°$; they are checked by `python/chk_sodfig.py`.

14

Nonlinear Noise Reduction

Every measured record of a chaotic system is contaminated. Transducer and amplifier noise enter at the measurement, quantization at the digitizer, and random forcing in the dynamics themselves. The estimators of Chapters 11 and 12 all suffer from this. Noise fills in the small scales of the correlation sum and floods the small-separation end of the divergence curve. It also adds false neighbors to the embedding statistics of Chapter 10. The spectral filters of Chapter 4 cannot help: a broadband chaotic signal and broadband noise occupy the same frequencies.

This chapter supplies the alternative. Noise reduction in the reconstructed phase space uses the one property that separates a deterministic signal from noise. Locally, the clean trajectory lies on a low-dimensional set; noise does not. Section 14.1 states the noise model and what "noise reduction" can mean for chaotic data. Section 14.2 gives the standard method, local projective noise reduction, as a procedure. Section 14.3 gives the smooth variant built on the smooth orthogonal decomposition of Chapter 13. It exploits the time ordering of the points and not only their positions. Sections 14.4 and 14.5 test both on the Lorenz and Duffing records, and Section 14.6 says how to choose the parameters and when to leave the data alone. Filtered records feed the predictors of Chapter 15 and the surrogate tests of Chapter 16. Chapter 18 asks that any filtering be reported with the recurrence statistics it precedes.

14.1 Noise in Measured Chaotic Data

Two different mechanisms are both called noise. They must be kept apart, because only one of them leaves a clean trajectory to recover. Throughout this chapter the true state of the system is $\boldsymbol{s}_i$ and the recorded scalar is x_i. Delay vectors are written $\boldsymbol{y}_i$, since the bold $\boldsymbol{x}_i$ of Chapter 8 would clash with the scalar record. A tilde marks the clean quantity that the record would show without noise.

Measurement noise and dynamical noise

Let the system evolve by $\boldsymbol{s}_{i+1} = \boldsymbol{F}(\boldsymbol{s}_i)$ and be observed through a smooth scalar function h. *Measurement noise* enters the observation only,

$$x_i = h(\boldsymbol{s}_i) + \gamma_i = \tilde{x}_i + \gamma_i \,, \tag{14.1}$$

where γ_i is a random sequence, usually independent of $\boldsymbol{s}_i$ and of its own past. *Dynamical noise* enters the evolution law,

$$\boldsymbol{s}_{i+1} = \boldsymbol{F}(\boldsymbol{s}_i) + \boldsymbol{\eta}_i \,, \qquad x_i = h(\boldsymbol{s}_i) \,, \tag{14.2}$$

and is carried forward by the dynamics. Under Eq. (14.1) a clean record $\{\tilde{x}_i\}$ exists and lies on the attractor of $\boldsymbol{F}$. Under Eq. (14.2) no such record exists: the perturbed orbit is the only orbit the system followed.

The distinction decides what noise reduction can mean. With measurement noise the target is $\tilde{x}_i$ itself. The quality of a filter is the reduction in $\langle (\bar{x}_i - \tilde{x}_i)^2 \rangle$, where $\bar{x}_i$ is the filtered value. With dynamical noise the best one can ask for is a deterministic trajectory of $\boldsymbol{F}$ that stays close to the measured one, a *shadowing trajectory*. Such a trajectory exists only when $\boldsymbol{\eta}_i$ is small compared with the scales on which $\boldsymbol{F}$ is nonlinear [75, 105]. The projective methods of this chapter are derived for Eq. (14.1). Applied to a record with small dynamical noise they return an approximate shadowing trajectory. Invariants estimated from it describe the noise-free system rather than the noisy one. Applied to a record with large dynamical noise, or with noise in the parameters of $\boldsymbol{F}$, they return a smoothed record whose attractor belongs to no deterministic system. The results of Chapters 11 and 12 computed from it are then not invariants of anything. Nothing in the algorithms warns of this; the record's provenance must.

Even a simulated record is not clean. Storing $\tilde{x}_i$ with resolution step Δ adds a rounding error bounded by $\Delta/2$. If the signal moves by many steps between samples the error is uniformly distributed, with root-mean-square value $\Delta/\sqrt{12} \approx 0.29\,\Delta$. A 12-bit converter spanning $\pm 5\sigma$ of the signal has $\Delta = 10\sigma/4096$. Its quantization noise is 0.07 % of the signal's standard deviation, a signal-to-noise ratio of 63 dB; the 5 % noise level of Section 14.5 is seventy times worse. Quantization matters for the correlation sum at scales below a few Δ, where it turns the attractor into a lattice. It also sets the floor beneath which no filter can claim to have gone.

The record used throughout is $\{x_i\}_{i=1}^{n}$, with n the record length. Signal-to-noise ratio is defined on standard deviations,

$$\mathrm{SNR} = 20 \log_{10} \frac{\sigma_{\tilde{x}}}{\sigma_{x-\tilde{x}}} \,, \tag{14.3}$$

so that noise standard deviations of 1/20, 1/10, 1/5, 2/5 and 4/5 of the signal's correspond to 26.0, 20.0, 14.0, 8.0 and 1.9 dB. These are the 5, 10, 20, 40 and 80 per cent noise levels in the usage of this chapter. After filtering the same ratio is computed with $\bar{x}_i$ in place of x_i. In an experiment $\tilde{x}_i$ is unknown and Eq. (14.3) cannot be evaluated; Section 14.6 says what to report instead.

14.1.1 Why Linear Filters Fail and Projection Does Not

A chaotic signal has a continuous, broad power spectrum (Chapter 7), and so does the noise. A low-pass filter therefore removes signal and noise in the same proportion above its cut-off and leaves both alone below it. It changes the shape of the attractor, and lowers the noise floor only where the signal has no power anyway. The distinction the spectrum cannot see is geometric. In a d-dimensional

delay reconstruction (Chapter 8) the clean points $\tilde{\boldsymbol{y}}_i$ lie on a set of dimension D_2 well below d. Within a small neighborhood that set is close to a q-dimensional linear subspace, $D_2 \le q < d$: the local tangent space, slightly thickened to accommodate curvature. Isotropic measurement noise of standard deviation σ_γ displaces each point by about σ_γ in every one of the d directions. Only the q components within the tangent space are indistinguishable from signal. The remaining $d - q$ components are pure noise and can be removed by projection, as Fig. 14.1 illustrates. The expected reduction in noise variance from a single projection is therefore $(d - q)/d$. From $d = 6$ to $q = 3$ one pass can remove at most half the noise power, or 3 dB. The rest is reached by iterating on the filtered record, whose neighborhoods are now better defined. This is the mechanism of every method in this chapter [75, 105, 117]; the methods differ only in how the q-dimensional subspace is identified.

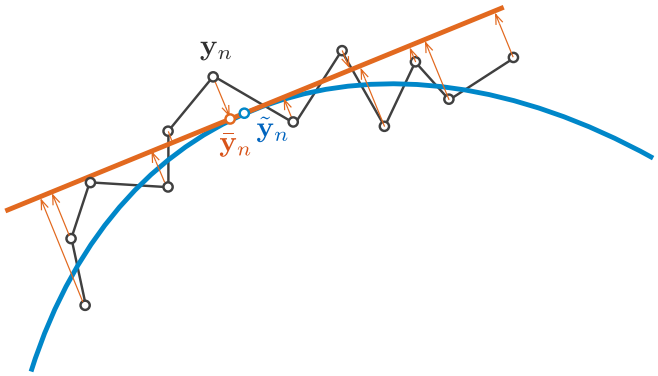


Figure 14.1: Projective noise reduction. The blue curve is the clean trajectory and the red line the tangent space at the clean point $\tilde{\boldsymbol{y}}_k$ (the figure's index n is the text's k). The black dots are the noisy delay vectors. Projecting the noisy $\boldsymbol{y}_k$ onto the tangent space gives the filtered $\bar{\boldsymbol{y}}_k \approx \tilde{\boldsymbol{y}}_k$; the components of the noise within the tangent space survive.

Two other families exist. Model-based filters [13] fit a global or local model and replace the record by a trajectory of the model. They are excellent when the equations are known and poor when they are not. Wavelet shrinkage and adaptive filters [68, 213] work when the noise is confined to scales the signal does not occupy, which requires prior knowledge of both. The projective methods below need only the reconstruction parameters of Chapters 9 and 10. They are the standard for experimental chaotic data [105, 106].

14.1.2 The Reconstruction Both Methods Share

The record is embedded with forward delays,

$$\boldsymbol{y}_i = \left[x_i,\, x_{i+\tau},\, \ldots,\, x_{i+(d-1)\tau}\right]^{\mathrm{T}}, \qquad i = 1, \ldots, n - (d-1)\tau\,, \tag{14.4}$$

with delay τ in samples. The points are the columns of $\boldsymbol{Y} \in \mathbb{R}^{d \times [n-(d-1)\tau]}$. Note the orientation: Chapter 13 stores time down the rows, so the cloud and strand matrices below are transposed before their decompositions. Each sample x_i appears in d columns of $\boldsymbol{Y}$, once in every coordinate; the first and last $(d-1)\tau$ samples appear in fewer. This redundancy turns a corrected trajectory back into a corrected scalar record: the d corrected copies of x_i are averaged.

The global dimension d is not the minimum embedding dimension D of Chapter 10. Noise reduction needs room for the noise to be seen, so d is taken well above D, typically $d \approx 2D$, and

the projection dimension at $q \approx D$. The false-nearest-neighbor statistic of Chapter 10 on the *noisy* record gives D only roughly, because noise itself creates false neighbors. The parameters are best confirmed by inspecting the filtered result for several q, as Section 14.6 describes. The delay τ is chosen as in Chapter 9. A filtered record should be re-embedded with the same τ and d before any invariant is estimated from it.

14.2 Local Projective Noise Reduction

The standard method estimates the local tangent space from the *geometry* of a cloud of neighboring points, not from the dynamics that carry the points forward [75, 105, 178]. The local linear maps of Chapter 15 are a different construction with a different purpose. The neighbors are the r points of $\boldsymbol{Y}$ nearest to $\boldsymbol{y}_k$ in Euclidean distance. They are found with the kd-tree searches already used for the false-nearest-neighbor and fixed-mass algorithms. For a densely sampled flow several of them are the temporal neighbors $\boldsymbol{y}_{k\pm1}, \boldsymbol{y}_{k\pm2}, \ldots$, and they are kept: they lie along the trajectory and inform the tangent direction. Neighborhoods may also be defined by a radius ϵ rather than a count [105]. The routine `podfilt` of Appendix A.4 uses a fixed count, which keeps every neighborhood populated in the sparse regions of the attractor at the cost of a variable radius.

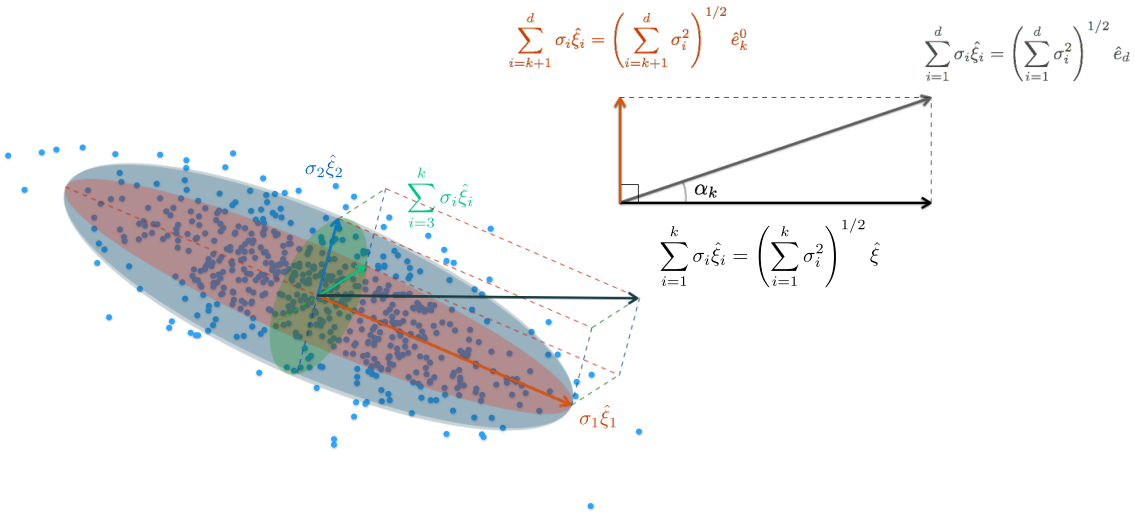


Figure 14.2: Geometry of the local projection (a schematic; the figure's k is the text's projection dimension q). Noise spreads the points of a q-dimensional set into a d-dimensional cloud of small but finite thickness. The cloud is approximated by its covariance ellipsoid, whose long axes span the local tangent space and whose short axes are the noise directions. Projecting onto the long axes removes the components along the short ones.

14.2.1 Centring, Local SVD, and Projection

Let $\{\boldsymbol{y}_k^{(j)}\}_{j=1}^{r}$ be the neighbors of $\boldsymbol{y}_k$, the point itself among them. Their centre of mass is

$$\boldsymbol{c}_k = \frac{1}{r} \sum_{j=1}^{r} \boldsymbol{y}_k^{(j)} . \tag{14.5}$$

The decomposition must be applied to the *centred* cloud $\boldsymbol{Z}_k = \left[\boldsymbol{y}_k^{(1)} - \boldsymbol{c}_k, \ldots, \boldsymbol{y}_k^{(r)} - \boldsymbol{c}_k\right] \in \mathbb{R}^{d\times r}$. Without centring, the leading singular direction of the cloud is the direction of $\boldsymbol{c}_k$ itself. That direction has nothing to do with the tangent space and would consume one of the q retained dimensions. The singular value decomposition, in the orientation of Chapter 13,

$$\boldsymbol{Z}_k^{\mathrm{T}} = \boldsymbol{P}_k \boldsymbol{\Sigma}_k \boldsymbol{\Xi}_k^{\mathrm{T}}\,, \tag{14.6}$$

gives the POMs of the cloud as the columns of $\boldsymbol{\Xi}_k \in \mathbb{R}^{d\times d}$ and the POCs as the columns of $\boldsymbol{P}_k \in \mathbb{R}^{r\times d}$. The POVs are the squared singular values, ordered $\sigma_1^2 \geq \cdots \geq \sigma_d^2$. The first q POMs, $\boldsymbol{\Xi}_{k,q} \in \mathbb{R}^{d\times q}$, span the estimate of the tangent space (Fig. 14.2). The filtered point is the centre of mass plus the projection of the point's own offset onto that span:

$$\bar{\boldsymbol{y}}_k = \boldsymbol{c}_k + \boldsymbol{\Xi}_{k,q}\boldsymbol{\Xi}_{k,q}^{\mathrm{T}}\big(\boldsymbol{y}_k - \boldsymbol{c}_k\big)\,. \tag{14.7}$$

The correction $\bar{\boldsymbol{y}}_k - \boldsymbol{y}_k = -\boldsymbol{\Xi}_{k,\perp}\boldsymbol{\Xi}_{k,\perp}^{\mathrm{T}}(\boldsymbol{y}_k - \boldsymbol{c}_k)$ removes exactly the component of the offset along the $d-q$ discarded directions. That is why one pass removes about a fraction $(d-q)/d$ of isotropic noise power and no more.

Weighting the End Coordinates

Kantz and Schreiber [105] do the decomposition in a weighted metric, $\boldsymbol{R} = \mathrm{diag}(\rho, 1, \ldots, 1, \rho)$ with $\rho \gg 1$. In Eq. (14.6) the cloud $\boldsymbol{Z}_k$ is replaced by $\boldsymbol{R}\boldsymbol{Z}_k$, and Eq. (14.7) becomes

$$\bar{\boldsymbol{y}}_k = \boldsymbol{c}_k + \boldsymbol{R}^{-1}\boldsymbol{\Xi}_{k,q}\boldsymbol{\Xi}_{k,q}^{\mathrm{T}}\boldsymbol{R}\big(\boldsymbol{y}_k - \boldsymbol{c}_k\big)\,. \tag{14.8}$$

The effect is that the first and last coordinates of $\boldsymbol{y}_k$ are left nearly unchanged, and the whole correction goes to the interior coordinates. The end coordinates of a delay vector are the ones least constrained by the rest of the vector. An error in $x_{k+(d-1)\tau}$ is propagated by the flow from the earlier coordinates, and correcting it from the geometry alone is unreliable. The routine `podfilt` uses $\rho = 1000$. In a test on a 6-dimensional cloud the end-coordinate corrections were then below 10^{-3} of the interior ones.

14.2.2 Adjusting for Bias due to Curvature

Equation Eq. (14.7) would be exact if the clean trajectory passed through the centre of mass of the cloud. On a curved arc it does not. The points of a neighborhood of radius ϵ on a curve of curvature κ lie on the convex side of their chord. Their mean is displaced from the curve toward the centre of curvature by about $\kappa\epsilon^2/6$, as Fig. 14.3 shows. The projection plane then intersects the attractor instead of touching it, and every filtered point is pulled inward. On a unit circle sampled densely and without noise, with $\epsilon = 0.3$, the centre of mass lies at radius 0.985, the predicted $1 - \epsilon^2/6$.

Following Grassberger *et al.* [75] and Kantz and Schreiber [105], the correction moves the centre of the projection to $2\boldsymbol{c}_k - \boldsymbol{c}_k'$, where

$$\boldsymbol{c}_k' = \frac{1}{r}\sum_{j=1}^{r} \boldsymbol{c}_k^{(j)} \tag{14.9}$$

is the mean of the centres of mass $\boldsymbol{c}_k^{(j)}$ of the neighborhoods of the r *spatial* neighbors $\boldsymbol{y}_k^{(j)}$. Each neighbor's centre of mass is displaced inward relative to its own point by about the same $\kappa\epsilon^2/6$. Their average is therefore displaced inward by about twice that amount relative to $\boldsymbol{y}_k$, and $2\boldsymbol{c}_k - \boldsymbol{c}_k'$



Figure 14.3: Curvature bias, computed on a unit circle sampled at 20,000 points. The figure's $\bar{\boldsymbol{y}}_n$ is the centre of mass $\boldsymbol{c}_k$ of the neighborhood of the clean point $\tilde{\boldsymbol{y}}_n$, and $\bar{\bar{\boldsymbol{y}}}_n$ is the mean $\boldsymbol{c}'_k$ of the centres of mass $\boldsymbol{c}_k^{(j)}$ of the spatial neighbors, drawn as the grey arc on the right. Left: the leading POM of a cloud on a curved arc is a chord, not a tangent, and $\boldsymbol{c}_k$ is displaced inward. Right: the neighbors' centres of mass are displaced inward by the same amount again, so $2\boldsymbol{c}_k - \boldsymbol{c}'_k$ lies back on the arc. Both panels are drawn at $\epsilon = 0.6$, where the displacement is 6 % of the radius and visible; the numbers quoted in the text are for $\epsilon = 0.3$. Generated by `python/mkfig_POD_Curv.py`.

cancels the bias to first order. On the unit circle above the corrected point lies at radius 0.9998. The neighbors must be the spatial ones. Building $\boldsymbol{c}'_k$ from the two temporally adjacent centres of mass $\boldsymbol{c}_{k\pm1}$ corrects nothing: for a densely sampled flow $(\boldsymbol{c}_{k-1} + \boldsymbol{c}_{k+1})/2 = \boldsymbol{c}_k + O(\Delta t^2)$. On the same circle it returns 0.985 again. The option `u = 0` of `podfilt` implements this temporal form and should be left at its default `u = 1`, no correction. The spatial form of Eq. (14.9) reuses the neighbor sets that the projection already finds and costs one extra average per point. Both forms are in `podfilt`, in the MATLAB and the Python versions alike (`u = 2` for Eq. (14.9), `u = 0` for a variant that takes $\boldsymbol{c}'_k$ from the two temporally adjacent points instead of the r spatial neighbors), and the comparison is worth reporting because it does not come out in favour of either.

The temporal variant diverges. On the Lorenz record at 5 % noise it loses 1.2 dB on the first pass and reaches −48 dB by the tenth. A second difference of neighborhood mass centres taken along the trajectory amplifies the jitter that comes from the neighbor set changing discontinuously from one point to the next, and iteration compounds it. It should not be used with the neighbor-cloud scheme.

The spatial form of Eq. (14.9) is stable, and still does not pay. Measured at each filter's own best iteration (`python/chk_curvcorr.py`), it costs 3.2 and 3.8 dB on the Lorenz record at 5 and 20 % noise, and 1.3 and 2.9 dB on the Duffing record. The reason is visible in the scaling. The bias the correction removes grows as $\kappa\epsilon^2/6$, so it should matter more as the neighborhood grows; the penalty duly shrinks, from −5.0 dB at $r = 32$ neighbors through −2.7 at $r = 128$ to −1.1 dB at $r = 1024$, where the neighborhood radius has reached $1.0\,\sigma_x$ and spans a good part of the attractor. It does not become positive anywhere in that range. Against the curvature bias it removes, the correction sets a variance: $\boldsymbol{c}_k - \boldsymbol{c}'_k$ is a difference of two noisy means over heavily overlapping neighbor sets, and on a noisy record that difference is dominated by the sampling of the sets rather than by the bend of the arc.

The practical reading is that the two effects trade against each other in the same parameter. Keeping the neighborhood small enough that the arc is nearly straight inside it is cheaper than enlarging it and correcting: $r = 64$ with no correction gives 21.0 dB where $r = 1024$ with the correction gives 18.4. The figures and tables of this chapter therefore use no curvature correction in the cloud scheme, and `podfilt` defaults to `u = 1`. The correction is kept in the routine because

the balance is a property of these records and not a theorem, and a record with a tighter attractor or a lower noise level may sit on the other side of it. The bias is second order in the neighborhood size. On a record long enough for small neighborhoods it is often below the residual noise, and the correction is not needed.

14.2.3 Iteration and the Procedure

After one pass the filtered points lie closer to the attractor and their neighborhoods are better defined, so the pass is repeated on the filtered record. The signal-to-noise ratio rises quickly for the first few iterations and then saturates or falls (Fig. 14.9). Once the residual is dominated by the components within the tangent space, further projections remove signal as readily as noise. The signal removed is the curvature of the attractor within each neighborhood. The recommended practice is a few iterations with neighborhoods that shrink as the noise is removed [75, 105]. Section 14.5 ran ten iterations with a fixed neighbor count and found the peak between the second and the tenth pass, later the noisier the record: on the Duffing record at 5 % noise the peak is at the second pass and the tenth pass is worse than no filtering at all.

Local projective noise reduction (`podfilt`, Appendix A.4)

Given the record $\{x_i\}_{i=1}^{n}$, delay τ, global dimension d, projection dimension $q < d$, and neighbor count $r > d$:

1. Embed by Eq. (14.4) into $\boldsymbol{Y} \in \mathbb{R}^{d\times[n-(d-1)\tau]}$ and build a kd-tree on its columns.
2. Extend $\boldsymbol{Y}$ at each end by $(d-1)\tau+1$ points of a shadow trajectory, as in Section 14.3.3: the predecessors of a nearest neighbor of the first point, the successors of a nearest neighbor of the last.
3. For every point $\boldsymbol{y}_k$ of the extended matrix: find its r nearest neighbors in $\boldsymbol{Y}$ and form the centre of mass Eq. (14.5). Centre and $\boldsymbol{R}$-weight the cloud, take the SVD Eq. (14.6), keep the q leading POMs, and set $\bar{\boldsymbol{y}}_k$ by Eq. (14.8). Optionally shift the centre to $2\boldsymbol{c}_k - \boldsymbol{c}'_k$ using the spatial neighbors' centres of mass, Eq. (14.9).
4. Discard the extension. Recover the filtered record by averaging the d copies of each sample, $\bar{x}_i = \frac{1}{d}\sum_{k=1}^{d} \bar{Y}(k,\, i+(d-k)\tau)$, which is Eq. (14.16) with $l = 0$. The first and last $(d-1)\tau$ samples average in values from the extension.
5. Repeat from step 1 on $\{\bar{x}_i\}$ for a fixed number of iterations, or until the root-mean-square correction $\left[\left\langle \|\bar{\boldsymbol{y}}_k - \boldsymbol{y}_k\|^2 \right\rangle\right]^{1/2}$ drops below the noise level expected from Section 14.6.

Report τ, d, q, r, the number of iterations, and the root-mean-square correction of the last pass.

The method applies to maps as well as flows, since it uses only the positions of the points. Schreiber's "extremely simple" variant [178] is the case $q = 0$ applied to the middle coordinate alone. Each x_k is replaced by the mean of the middle coordinates of its neighbors. It is a zeroth-order version of the same idea, useful as a first pass on very noisy data, and it exhibits the curvature bias undiluted.

14.3 Smooth Projective Noise Reduction

The local projection ranks directions by variance. Where the noise variance is comparable to that of a genuine but small signal direction, POD discards signal along with noise. The thin direction of a nearly two-dimensional attractor is such a direction; so is the small-amplitude coordinate of a strand near a turning point. The smooth orthogonal decomposition of Chapter 13 ranks directions by smoothness instead [31], and here the two rankings differ sharply. A coordinate that is white noise has smooth orthogonal value (SOV) $\lambda^2 = \langle q^2 \rangle / \langle \dot{q}^2 \rangle \approx 1/2$ when the derivative is a first difference, since differencing doubles the variance of white noise. A coordinate oscillating at angular frequency ω, sampled at Δt, has $\lambda^2 \approx 1/(\omega \Delta t)^2$; for the Lorenz record at $\omega = 8$, $\Delta t = 0.01$, this is 156. The gap is two orders of magnitude, independent of amplitude. That is the case for smooth projective noise reduction [32]. It replaces the neighborhood cloud by a short piece of the trajectory, and the SVD by the generalized SVD.

The price is that the method needs a data-based time derivative along the trajectory. It applies only to records from flows sampled densely enough that consecutive points are close. It cannot be applied to map-generated data, for which consecutive points are unrelated in position. The local projection of Section 14.2 remains the method for maps (Problem 14.3).

14.3.1 Strands and the Rank Condition

A *strand* is a set of $2l + 1$ consecutive reconstructed points centred on $\boldsymbol{y}_k$,

$$\boldsymbol{S}_k = [\boldsymbol{y}_{k-l}, \ldots, \boldsymbol{y}_k, \ldots, \boldsymbol{y}_{k+l}] \in \mathbb{R}^{d \times (2l+1)} , \tag{14.10}$$

with l the *half-length*. The definition is used consistently below: $l = 5$ means an eleven-point strand, and Fig. 14.5 shows weights for $l = 10$, a 21-point strand. The routine `sodfilt` of Appendix A.5 takes the full length $2l+1$ as its argument and increases an even value by one. The strand is centred like the cloud, with $\boldsymbol{c}_k = \frac{1}{2l+1} \sum_{j=-l}^{l} \boldsymbol{y}_{k+j}$ and $\boldsymbol{Z}_k = \boldsymbol{S}_k - \boldsymbol{c}_k \boldsymbol{1}^{\mathrm{T}}$. The reason is the same. The mean of a strand is a near-constant direction with almost zero derivative; without centring it would be ranked smoothest of all and consume one of the q dimensions.

The centred strand has rank at most $2l$, since its $2l + 1$ columns sum to zero. The generalized SVD of the next subsection requires the stacked pair $[\boldsymbol{Z}_k^{\mathrm{T}}; (D\boldsymbol{Z}_k)^{\mathrm{T}}]$ to have full column rank d. The rows of the difference matrix lie in the row space of $\boldsymbol{Z}_k^{\mathrm{T}}$, so the condition is

$$2l \geq d , \tag{14.11}$$

which is what `sodfilt` checks when it insists that the strand length exceed d. Numerically, with $d = 6$ a centred strand of six points failed the rank test and one of seven passed. In practice l is taken larger than this minimum, $l \approx d$ to $2d$, so that the difference quotients along the strand estimate the derivative with some averaging. But the strand must stay short compared with the radius of curvature of the trajectory. Otherwise the q smoothest directions are no longer a good local description of it. For the Lorenz record at $\Delta t = 0.01$ an eleven-point strand spans a tenth of a time unit, about an eighth of a circuit around one lobe. The three smoothest directions of such a strand include its curvature, which is why $q = 3$ rather than $q = 1$ is retained.

14.3.2 Smooth Subspace Identification and Projection

The time derivative along the strand is the first difference, $D\boldsymbol{Z}_k = [\boldsymbol{z}_{k-l+1} - \boldsymbol{z}_{k-l}, \ldots] \in \mathbb{R}^{d \times 2l}$. The SOD of the strand is the generalized SVD of the pair, in the notation of Chapter 13:

$$\boldsymbol{Z}_k^{\mathrm{T}} = \boldsymbol{U}_1 \boldsymbol{\Sigma}_1 \boldsymbol{\Phi}_k^{\mathrm{T}} , \qquad (D\boldsymbol{Z}_k)^{\mathrm{T}} = \boldsymbol{U}_2 \boldsymbol{\Sigma}_2 \boldsymbol{\Phi}_k^{\mathrm{T}} . \tag{14.12}$$

The columns of $\boldsymbol{\Phi}_k \in \mathbb{R}^{d\times d}$ are the SOMs and the columns of $\boldsymbol{\Psi}_k = \boldsymbol{\Phi}_k^{-\mathrm{T}}$ the SPMs. The SOCs are the columns of $\boldsymbol{U}_1\boldsymbol{\Sigma}_1 = \boldsymbol{Z}_k^{\mathrm{T}}\boldsymbol{\Psi}_k$. The SOVs are the squared ratios $\lambda_p^2 = (\sigma_{1,p}/\sigma_{2,p})^2$ of Eq. (13.29), ordered $\lambda_1^2 \geq \cdots \geq \lambda_d^2$ so that the first SOC is the smoothest. MATLAB's `gsvd` returns the pairs in ascending order and normalizes $\boldsymbol{\Sigma}_1^{\mathrm{T}}\boldsymbol{\Sigma}_1 + \boldsymbol{\Sigma}_2^{\mathrm{T}}\boldsymbol{\Sigma}_2 = \boldsymbol{I}$, whereas Chapter 13 normalizes the SOMs to unit length. The two normalizations scale $\boldsymbol{\Phi}_k$ and $\boldsymbol{\Psi}_k$ inversely and leave the SOVs and the projector below unchanged.

The q-dimensional smoothest approximation to the strand keeps the q leading SOMs and SPMs, $\boldsymbol{\Phi}_{k,q}, \boldsymbol{\Psi}_{k,q} \in \mathbb{R}^{d\times q}$, and adds the centre back:

$$\bar{\boldsymbol{S}}_k = \boldsymbol{c}_k\mathbf{1}^{\mathrm{T}} + \boldsymbol{\Phi}_{k,q}\boldsymbol{\Psi}_{k,q}^{\mathrm{T}}\big(\boldsymbol{S}_k - \boldsymbol{c}_k\mathbf{1}^{\mathrm{T}}\big)\,. \tag{14.13}$$

Because $\boldsymbol{\Psi}_{k,q}^{\mathrm{T}}\boldsymbol{\Phi}_{k,q} = \boldsymbol{I}_q$, the matrix $\boldsymbol{\Phi}_{k,q}\boldsymbol{\Psi}_{k,q}^{\mathrm{T}}$ is a projector, but an oblique one. The SOMs are not orthogonal, and the strand is projected along the rough directions rather than perpendicular to the smooth ones. Every point of the strand is corrected, not only the centre; the weighting along the strand in the next subsection exploits this. A check: a synthetic 23-point strand of a smooth three-dimensional curve was embedded in $\mathbb{R}^6$ with 5 % white noise. The SOVs came out as 50.6, 8.1, 1.2 for the three smooth directions and 0.58, 0.43, 0.37 for the three noise directions. A single projection with $q = 3$ removed a fifth of the noise power.

14.3.3 Data Padding to Mitigate the Edge Effects

The first and last l points of $\boldsymbol{Y}$ have no full strand. The first and last $(d-1)\tau$ samples of the record appear in fewer than d columns of $\boldsymbol{Y}$. Left alone, both effects leave the ends of the record under-filtered. The routines pad both ends of $\boldsymbol{Y}$ with trajectory segments of length $l + (d-1)\tau$ taken from the record itself. Let $\boldsymbol{y}_s$ be the nearest neighbor of the first point $\boldsymbol{y}_1$ that has at least $l + (d-1)\tau$ predecessors. Its predecessors $\boldsymbol{Y}_s = [\boldsymbol{y}_{s-l-(d-1)\tau}, \ldots, \boldsymbol{y}_{s-1}]$ are prepended. Likewise the successors $\boldsymbol{Y}_e = [\boldsymbol{y}_{e+1}, \ldots, \boldsymbol{y}_{e+l+(d-1)\tau}]$ of the nearest neighbor $\boldsymbol{y}_e$ of the last point are appended. The padded matrix is $\widehat{\boldsymbol{Y}} = [\boldsymbol{Y}_s, \boldsymbol{Y}, \boldsymbol{Y}_e] \in \mathbb{R}^{d\times[n+2l+(d-1)\tau]}$. The filtering runs over the base points $\hat{\boldsymbol{y}}_i$ with $i = l+1, \ldots, n+(d-1)\tau+l$, every one of which now has a full strand. This is *shadow completion*. The padding is a segment of a nearby trajectory. It follows the true continuation closely for a time of the order of the inverse Lyapunov exponent, and the padding lengths in use, a few tens of samples, are well within that. The filtered values at the first and last $(d-1)\tau$ samples nonetheless average in shadow values in some of their d coordinates, as Eq. (14.16) makes explicit. The ends of a filtered record should not be used to estimate anything sensitive. The local projective routine pads in the same way with l replaced by 1.

14.3.4 Weighting along the Strand and Averaging

Each base point $\hat{\boldsymbol{y}}_i$ produces a smoothed strand $\bar{\boldsymbol{S}}_i$, and each point of the padded trajectory lies in $2l+1$ strands. The corrections are combined by accumulation. Each is weighted by its position within the strand through $\boldsymbol{W} = \mathrm{diag}(w_{-l}, \ldots, w_l)$, peaked at the centre:

$$\bar{\boldsymbol{Y}}_i \leftarrow \bar{\boldsymbol{Y}}_i + \bar{\boldsymbol{S}}_i\boldsymbol{W}\,, \qquad \bar{\boldsymbol{w}}_i \leftarrow \bar{\boldsymbol{w}}_i + \mathbf{1}^{\mathrm{T}}\boldsymbol{W}\,, \tag{14.14}$$

Here $\bar{\boldsymbol{Y}}_i = [\bar{\boldsymbol{y}}_{i-l}, \ldots, \bar{\boldsymbol{y}}_{i+l}]$ denotes columns $i-l$ to $i+l$ of an accumulator $\bar{\boldsymbol{Y}} \in \mathbb{R}^{d\times[n+2l+(d-1)\tau]}$ initialized to zero. The row $\bar{\boldsymbol{w}}$ of the same length accumulates the weight each column receives. Three

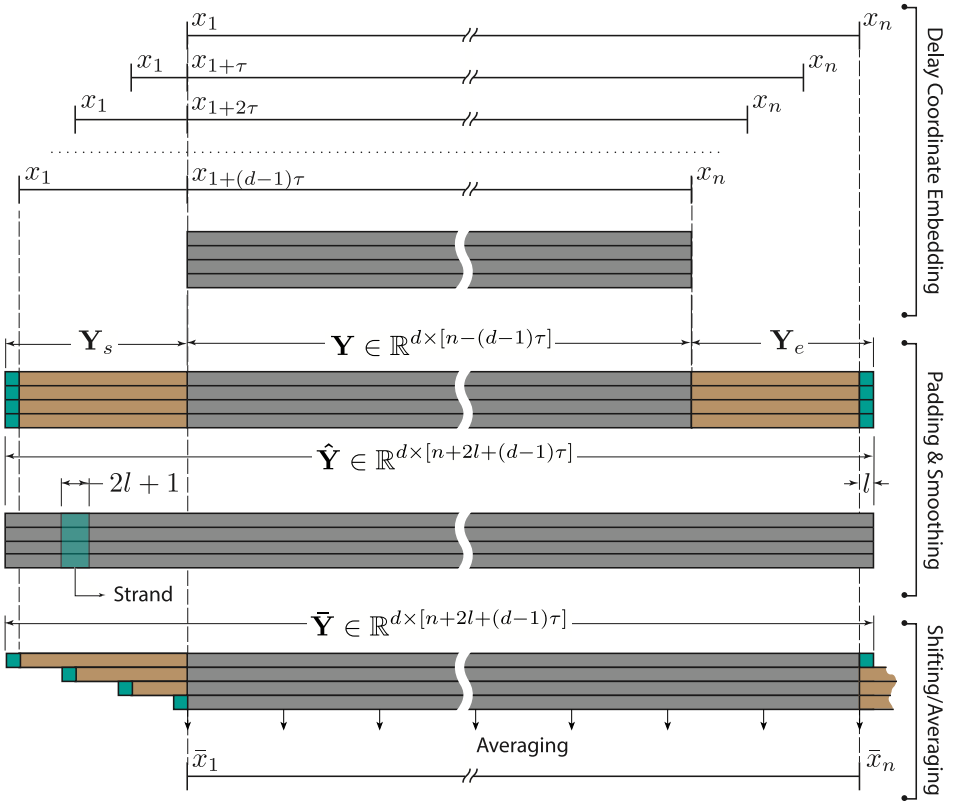


Figure 14.4: Smooth projective noise reduction, with the arrays drawn in row form for legibility. Top: delay embedding of the n-point record into $\boldsymbol{Y}$. Middle: padding by shadow segments $\boldsymbol{Y}_s$, $\boldsymbol{Y}_e$ and the sweep of $(2l+1)$-point strands over the padded $\widehat{\boldsymbol{Y}}$. Bottom: the accumulated corrections in $\bar{\boldsymbol{Y}}$ are aligned coordinate by coordinate and averaged down the columns to give the filtered record $\bar{x}_i$.

choices are used (Fig. 14.5): the *comb*, $w_0 = 1$ and every other weight zero; the *tent*, $w_j \propto l - |j|$; and the *exponential*, $w_j \propto e^{-|j|}$. The weights need not be normalized, because the next step divides each column by the weight it received. Under the comb each column receives a single contribution, the centre of its own strand. The method then reduces to replacing each point by the centre of its smoothed strand. Under the tent or exponential each point is a weighted mean of the $2l+1$ smoothed estimates that cover it. This is an additional averaging along the trajectory over $\pm l$ samples. It is what the results below use, and its cost is a slight smoothing of the sharpest turns of the attractor.

After the sweep, each column is divided by its accumulated weight,

$$\bar{\boldsymbol{y}}_j \leftarrow \bar{\boldsymbol{y}}_j / \bar{w}_j \,, \tag{14.15}$$

so that every point is the weighted average of the strand estimates that cover it. Interior points are covered by all $2l+1$ strands and the first and last l by fewer; Eq. (14.15) treats both alike. Finally, each sample x_i is recovered from the d coordinates in which it appears,

$$\bar{x}_i = \frac{1}{d} \sum_{k=1}^{d} \bar{Y}\big(k,\, i + l + (d-k)\tau\big) \,, \qquad i = 1, \ldots, n \,. \tag{14.16}$$

The column offset follows from the embedding and the padding. With $\widehat{\boldsymbol{Y}}$ padded in front by $l+(d-1)\tau$ points, $\widehat{Y}(k,j) = x_{j-l-(d-1)\tau+(k-1)\tau}$, so coordinate k holds x_i in column $j = i+l+(d-k)\tau$.

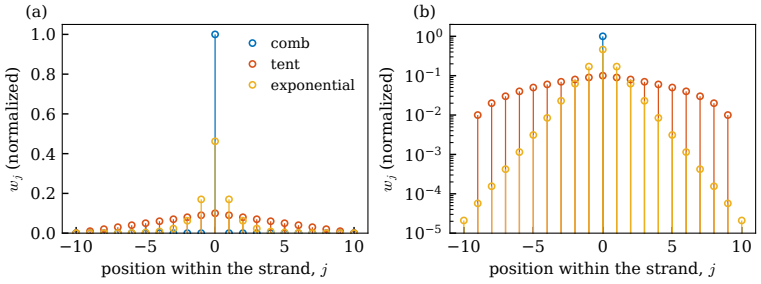


Figure 14.5: The three weightings along a strand of half-length $l = 10$ (21 points), as `nlutil.strand_weights` returns them, normalized to unit sum: (a) linear and (b) logarithmic scale. The comb is the single unit weight at $j = 0$. The tent $w_j \propto l - |j|$ vanishes at $j = \pm l$, so its two end points are absent from (b). Generated by `python/mkfig_weightingW.py`.

As k runs from d down to 1 the column index walks forward through the delay window. That is why each padding block must be at least $(d-1)\tau$ points long. On a test record with the identity in place of the projection, Eq. (14.16) returned every sample exactly except the first and last $(d-1)\tau$. Those take some of their d values from the padding.

Smooth projective noise reduction (`sodfilt`, Appendix A.5)

Given the record $\{x_i\}_{i=1}^{n}$ from a flow, delay τ, global dimension d, projection dimension $q < d$, strand half-length l with $2l \geq d$, a weighting along the strand, and a bundle size $r \geq 1$:

1. Embed by Eq. (14.4) into $\boldsymbol{Y}$ and build a kd-tree on its columns.
2. Pad both ends by shadow segments of $l + (d-1)\tau$ points (Section 14.3.3) to obtain $\widehat{\boldsymbol{Y}}$; zero the accumulators $\bar{\boldsymbol{Y}}$ and $\bar{\boldsymbol{w}}$.
3. For each base point $\hat{\boldsymbol{y}}_i$, $i = l+1, \ldots, n+(d-1)\tau+l$: form the strand Eq. (14.10) and centre it. Take the GSVD Eq. (14.12) of the strand and its first difference, and form the smoothed strand Eq. (14.13). If $r > 1$, do the same for the $r-1$ nearest temporally uncorrelated strands and replace $\bar{\boldsymbol{S}}_i$ by their weighted average (Section 14.4). Accumulate by Eq. (14.14).
4. Normalize by Eq. (14.15), discard the padding, and recover the filtered record by Eq. (14.16).
5. Repeat from step 1 on $\{\bar{x}_i\}$. In Section 14.5, with $r = 9$, most of the gain came in the first pass and the signal-to-noise ratio peaked at the first to the third pass on both records and then declined. Do not iterate to a fixed count.

Report τ, d, q, l, the weighting, r, and the number of iterations.

The same strand construction can be run with the SVD in place of the GSVD, ranking the strand's directions by variance rather than smoothness. The routine `podfilt` offers this as its option `f = 0`. It is a useful control. Whatever the strand-based SVD gains over the cloud-based one is due to the strand; whatever the GSVD gains beyond that is due to smoothness.

14.4 What the Strand Bundle Buys

Step 3 of the procedure allows a *bundle*: the base strand together with the strands centred on the $r-1$ nearest neighbors of the base point. Each neighbor must be at least one strand span, $(d-1)\tau + 2l + 1$ samples, away in time, so that no two strands share a sample. Each strand of the bundle is decomposed and projected on its own by Eq. (14.13). The smoothed strands are then averaged with weights $w(\delta_j)$ that fall off with the distance δ_j from the base point to the centre of the j-th strand. The weight is that of Eq. (15.10), $w(\delta) = a^4/(\delta^4 + a^4)$, and in `sodfilt` the scale a is the distance from the base point to the farthest strand of its own bundle, the $(r-1)$-th neighbor, recomputed for every base point. The base strand therefore carries weight one, the farthest strand weight one half, and the scale follows the local density of the reconstruction rather than a formula in d and n. It is the smoothed *points* that are averaged. The bundle therefore performs a spatial average across neighboring trajectory segments as well as a projection. The construction more usual in projective methods is different: stack the centred strands, take one decomposition of the stack, and project only the base strand, so that the bundle refines the subspace without moving the base point. Tested on the same records, it recovered considerably less.

Take the Lorenz x channel, 20,000 points at $\Delta t = 0.01$, with 20 and 40 per cent noise added. Embed with $\tau = 12$, $d = 6$, use eleven-point strands ($l = 5$), project onto $q = 3$, with tent weighting and three iterations. Vary only the bundle size r. The results are in Fig. 14.6 and Table 14.1.

Three quantities are reported, not one. Signal-to-noise ratio alone gives the wrong answer here, so the correlation dimension of Chapter 11 and the largest exponent of Chapter 12 are reported beside it. Both are measured through a *single* window taken from the clean record's scaling region: $\ln \epsilon \in [-3.05, -1.58]$ for D_2, steps 236 to 275 for $\hat{\lambda}_1$. An automatically chosen window is defensible for one curve, as Section 12.5.2 argues, but windows chosen per record are not comparable across records. The clean record itself reaches $D_2 = 1.97$ and $\hat{\lambda}_1 = 0.83$, against 2.05 and 0.906 from the model. Those four and nine per cent shortfalls are the estimators' own ceiling at this record length and these windows. The filtered records are therefore scored against the clean values rather than against the model.

Table 14.1: Bundle size r against signal-to-noise ratio, correlation dimension and largest exponent, for the Lorenz record described in the text. The clean record gives $D_2 = 1.974$ and $\hat{\lambda}_1 = 0.828$; the model gives 2.05 and 0.906. Bold marks the best entry of each column at each noise level.

	SNR (dB)	D_2	$\hat{\lambda}_1$
20 % noise, unfiltered	14.05	2.962	−0.006
$r = 1$	19.90	2.836	0.104
$r = 2$	24.94	2.474	0.322
$r = 4$	**27.67**	2.030	0.740
$r = 8$	27.58	**1.926**	**0.800**
40 % noise, unfiltered	7.97	2.974	0.038
$r = 1$	13.72	2.950	0.086
$r = 2$	18.72	2.834	0.106
$r = 4$	**21.36**	2.389	0.268
$r = 8$	21.23	**2.017**	**0.388**

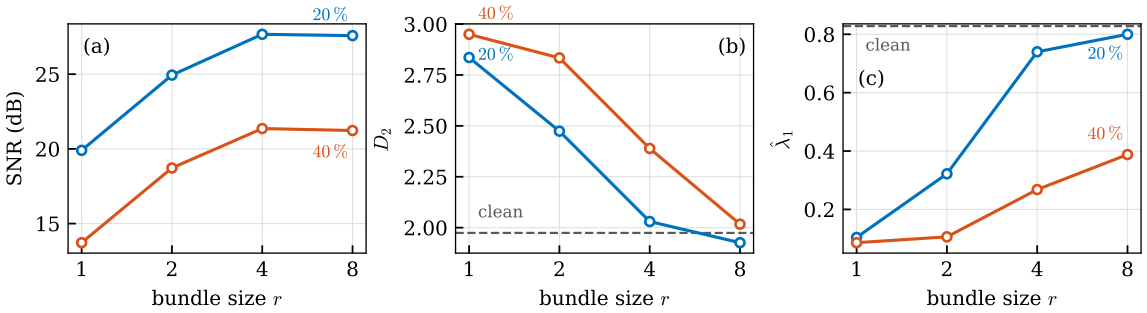


Figure 14.6: What the strand bundle buys, against bundle size r, for the Lorenz record of Table 14.1 at two noise levels. (a) Signal-to-noise ratio, which peaks at $r = 4$. (b) Correlation dimension, which falls from 44 per cent above the clean value at $r = 1$ to within three per cent of it by $r = 4$ at 20 per cent noise and by $r = 8$ at 40 per cent, and below it at $r = 8$ and 20 per cent noise. (c) Largest exponent, which improves monotonically; at 20 per cent noise it comes within four per cent of the clean value by $r = 8$, at 40 per cent it remains less than half of it.

Four things follow.

First, the bundle is not a marginal refinement. Going from $r = 1$ to $r = 4$ adds nearly 8 dB at both noise levels, more than the entire difference between the POD- and SOD-based schemes in Section 14.5.

Second, and more important than the decibels, the bundle is what recovers the *geometry*. At $r = 1$ the correlation dimension of the filtered record is still 44 per cent above the clean value. The filter has raised the signal-to-noise ratio by 6 dB while leaving the attractor as noise-thickened as it found it. By $r = 4$ the dimension is within three per cent of the clean value at 20 per cent noise; at 40 per cent noise it takes $r = 8$.

Third, the bundle has a bias of its own, and it must be named. Averaging the smoothed *points* of neighboring strands into the estimate of the base point is a kernel smoother of Nadaraya–Watson type. Each filtered point is pulled toward the local mean of the attractor over the bundle's radius. That shrinks the attractor toward its local means, thins its fine structure, and lowers the estimated D_2. This is why the dimension does not stop at the clean value but continues to 1.926, two per cent *below* it, at $r = 8$ and 20 per cent noise. The same pull toward the mean reduces the separation of nearby trajectories, and so depresses $\hat{\lambda}_1$. The signal-to-noise ratio peaks at $r = 4$ and falls slightly by $r = 8$ for the same reason. The stack-and-project construction mentioned above is free of this bias, since the base point is never moved toward its neighbors. Its lower signal-to-noise ratio is the price of the smaller bias; the point average buys a smaller variance with a larger bias. Choosing r by signal-to-noise ratio alone would stop at $r = 4$, which at 40 per cent noise is too early, before the geometry is recovered; at 20 per cent it happens to be right, but only the dimension shows that it is. Report both.

Fourth, the largest exponent is the last quantity to recover. At 20 per cent noise it is still 11 per cent low at $r = 4$, where the signal-to-noise ratio and the dimension have both arrived, and comes within 3 per cent of the clean record's only at $r = 8$. At 40 per cent noise it is 53 per cent low at $r = 8$, less than half the clean value, after the dimension has come within two per cent. Projective

noise reduction restores the shape of the attractor considerably better than it restores the rate at which nearby trajectories separate. Every averaging step, along the strand and across the bundle, moves neighboring points toward each other. The divergence estimator of Chapter 12 reads that as slower separation. At 40 per cent noise no bundle size tried closes the gap.

The cost is close to linear in r: on the record above, $r = 1$ took 6 seconds per three iterations, $r = 4$ took 42 and $r = 8$ took 69. For a clean or lightly contaminated record, $r = 1$ is the right default. For a noisy one, $r = 4$ is a better one. The next section uses $r = 9$ and ten iterations, a deliberately aggressive setting. That bundle size sits at the far end of the flat part of Fig. 14.6(a), and ten iterations of it are well past the peak: the bias described here compounds from pass to pass, and Section 14.5 shows what it costs.

14.5 Evaluation on Simulated and Measured Records

The two methods were compared on the book's running examples with the noise-free record known, so that Eq. (14.3) could be evaluated. The first is the Lorenz system [127] of Chapter 7,

$$\dot{x} = 10\,(y - x)\,, \quad \dot{y} = x\,(28 - z) - y\,, \quad \dot{z} = xy - \frac{8}{3}z\,, \tag{14.17}$$

was integrated from $(x_0, y_0, z_0) = (5, -5, 20)$ and 20,000 points of x were recorded at $\Delta t = 0.01$ after a 2000-sample transient. The two-well Duffing oscillator

$$\ddot{x} + 0.25\,\dot{x} - x + x^3 = 0.3\cos t \tag{14.18}$$

was sampled thirty times per forcing period ($\Delta t = 2\pi/30$) in its steady-state chaotic response, again for 20,000 points. Both records are standardized to zero mean and unit variance, so every quantity below is in units of σ_x. Gaussian white noise was added at 5, 10, 20, 40 and 80 per cent of the signal's standard deviation. The realized input signal-to-noise ratios are 26.0, 20.0, 14.0, 8.0 and 1.9 dB, the nominal values of Eq. (14.3) to two decimals. Both filters were applied for ten iterations. Performance was judged by the signal-to-noise ratio after each iteration and by the power spectral density. It was also judged by the correlation sum of Chapter 11, the short-time divergence curve of Chapter 12, and the reconstructed phase portraits.

The whole evaluation — both records, all five noise levels, both filters, ten iterations each, the spectra, correlation sums, divergence curves and both signal-to-noise tables — is generated by `python/_ch14data.py` together with four figure scripts distributed with the book's code. The records are 20,000 points long, which keeps the run to about forty minutes on two cores.

14.5.1 Lorenz Record

The delay is $\tau = 12$ samples, the value the bundle study of Section 14.4 also uses. It is not the first minimum of the average mutual information, which on this record lies at 17 samples; $\tau = 12$ is inside the shallow basin around it and was kept so that every result of this chapter refers to one reconstruction. The false-nearest-neighbor fraction (Fig. 14.7) falls to zero at $D = 3$ for the clean record. On the noisy records the knee is at $d = 4$, and beyond it the fraction flattens onto a noise floor instead of vanishing: 10 to 16 per cent at 80 % noise and about 2 per cent at 40 %, against below 0.2 per cent at 20 % and less at 5 and 10 %. The floor is Kennel's second criterion, which counts a neighbor false when the pair separates by a fixed fraction of the attractor's size; the ratio criterion alone has no floor and falls to zero by $d = 7$ at every noise level. Following Section 14.1,

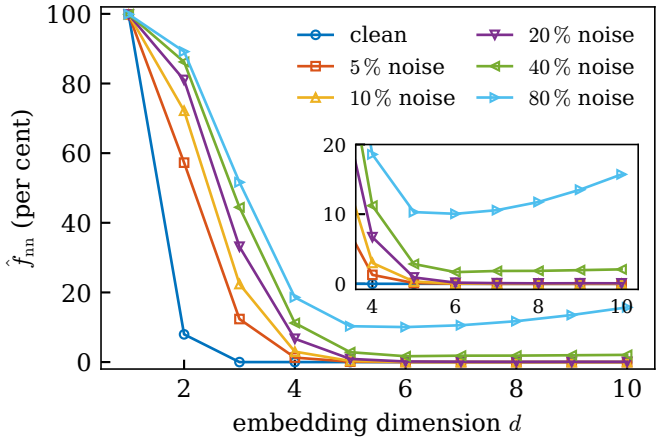


Figure 14.7: Kennel's composite false-nearest-neighbor fraction $\hat{f}_{\mathrm{nn}}$ (ratio threshold $R_{\mathrm{tol}} = 10$ and attractor-size threshold $A_{\mathrm{tol}} = 2$, plain nearest neighbor, Theiler window 3τ) against embedding dimension for the clean Lorenz record and the five noisy ones; $\tau = 12$, $N = 20{,}000$. The inset repeats $d = 4$ to 10 on a magnified scale, where the noise floor lies. Generated by `python/mkfig_fnn_nr.py`.

the global dimension was set at $d = 6$ and the projection dimension at $q = 3$. The local projective filter used $r = 64$ neighbors. The smooth filter used bundles of nine eleven-point strands ($l = 5$, $r = 9$) with tent weighting. Fig. 14.8 shows the clean and noise phase portraits and spectra, and the two filtered results at 5 % noise. In units of σ_x^2 per hertz, the spectral floor of the unfiltered 5 % record sits at -43 dB. After ten iterations it falls to -52 dB under the local projective filter and to -80 dB under the smooth one. At 20 % noise the unfiltered floor is -31 dB and the two filtered floors are -40 and -83 dB. The smooth filter's floor is not ten decibels below the other but thirty to forty, and it is essentially the same at both noise levels: after ten iterations that floor is set by the filter's own smoothing, not by the residual noise. The clean record's own floor, from an integration accurate to 10^{-10}, is far below either.

Fig. 14.9 and Table 14.2 give the signal-to-noise ratio against iteration. Neither filter improves monotonically over ten iterations, and the two fail differently. The smooth filter gains almost everything in its first pass — 10.8 dB at 5 % noise and 10.7 dB at 20 % — peaks at the second or third pass, and then loses about half a decibel per pass thereafter. The local projective filter climbs more slowly to a broad, flat maximum around the sixth pass and stays there. Measured at each filter's best iteration, the smooth filter gains 11.0 dB at 5 % noise and 12.9 dB at 20 %, against 6.8 and 7.1 dB for the local projective filter; across all five noise levels the gains are 11 to 13 dB against 6.4 to 7.1 dB. Measured after ten iterations the comparison reverses at low noise: at 5 % the over-smoothed smooth filter has fallen to 30.7 dB, below the local projective filter's 32.6. Ten iterations with a bundle of nine is past the useful stopping point at every noise level, and it is well past it at 5 %. Both readings are in Table 14.2, because the difference between them is the point.

Fig. 14.10 shows the correlation sum and the divergence curve after ten iterations, with the estimators of Chapters 11 and 12 at $d = 3$, $\tau = 12$ and the fit windows of Table 14.1. Both filters restore a scaling region the noisy record had lost, but they do not restore the same one. The local projective filter follows the clean correlation sum closely and returns $D_2 = 1.98$ at 5 % noise and 1.94 at 20 %, against 1.97 for the clean record. The smooth filter overshoots: its correlation sum

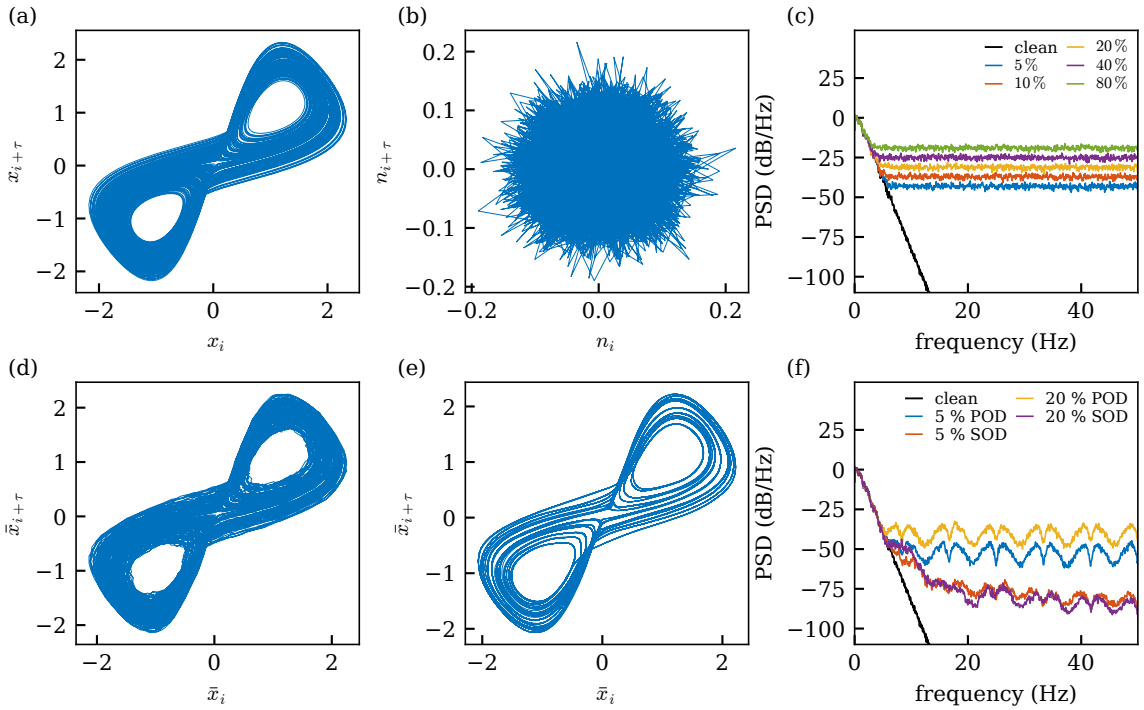


Figure 14.8: Lorenz record, $N = 20{,}000$, $\Delta t = 0.01$, standardized, $\tau = 12$. (a) Clean delay portrait; (b) portrait of the noise added at the 5 % level; (c) power spectra of the clean record and of the five noisy ones; (d) POD-filtered ($d = 6$, $q = 3$, 64 neighbors) and (e) SOD-filtered ($d = 6$, $q = 3$, $l = 5$, $r = 9$, tent) portraits of the 5 %-noise record after ten iterations; (f) the spectra of the clean record and of the filtered records at 5 and 20 % noise. Welch estimate, 2048-point Hann segments, half overlap; the clean record's spectrum leaves the frame below −110 dB/Hz. Generated by `python/mkfig_nr_inout.py`.

Table 14.2: Signal-to-noise ratio (dB) of the noisy Lorenz record ($N = 20{,}000$, $\Delta t = 0.01$, $\tau = 12$, $d = 6$, $q = 3$; POD with 64 neighbors, SOD with $l = 5$ and bundles of $r = 9$ strands, tent weighting) before filtering, after ten iterations, and at each filter's best iteration, whose number is in parentheses. Computed by `python/mkfig_nr_snr.py`.

Signal	Lorenz + 5% Noise			Lorenz + 20% Noise		
Algorithm	None	POD	SOD	None	POD	SOD
SNR after 10 iterations	26.03	32.59	30.67	14.01	21.06	25.80
SNR at the best iteration	—	32.80 (6)	**37.02** (2)	—	21.06 (10)	**26.90** (3)

flattens onto an excess of very close pairs two decades below the clean record's smallest resolved scale, and its slope through the same window is 1.84 and 1.79, six and nine per cent *below* the clean value. That is the kernel-smoother bias of Section 14.4, ten iterations of it rather than three. On the divergence curve the local projective filter tracks the clean curve almost exactly at 5 % noise ($\hat{\lambda}_1 = 0.83$ against the clean 0.83) but leaves large fluctuations at the smallest separations, where

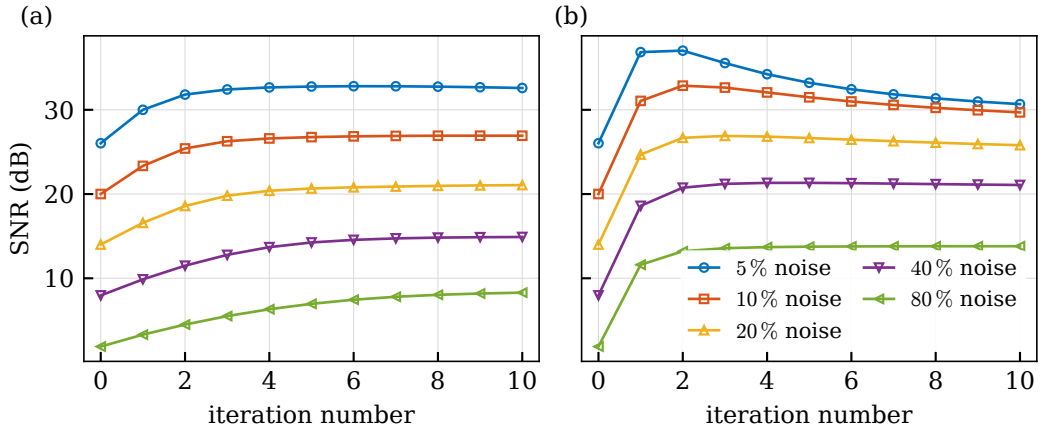


Figure 14.9: Signal-to-noise ratio of the noisy Lorenz record against the number of iterations of (a) the local projective filter ($d = 6$, $q = 3$, 64 neighbors) and (b) the smooth projective filter ($d = 6$, $q = 3$, $l = 5$, bundles of $r = 9$ strands, tent weighting), at the five noise levels. Iteration 0 is the unfiltered record. $N = 20{,}000$, $\tau = 12$. Generated by `python/mkfig_nr_snr.py`.

its neighborhoods are least well determined; the smooth filter is smooth there, and wrong: its curve starts two units of $\ln \delta_n$ below the clean one, because the filtered points have been drawn together, and then climbs through it, giving $\hat{\lambda}_1 = 1.13$ and 0.91, above the clean value rather than below it. A divergence curve that begins below the clean record's is a warning that the record has been smoothed, not cleaned. The phase portraits of Fig. 14.11 make the trade visible. Up to 40 % noise both filters return the two lobes, the local projective result the coarser of the two. At 80 % the local projective filter has lost the lobes altogether. The smooth filter still returns them, with the fine banding lost and the whole attractor shrunk.

14.5.2 Duffing Record

The delay is $\tau = 7$ samples, a little under a quarter of the forcing period. The mutual information of this record has two minima and the choice is between them. At $\Delta t = 2\pi/30$ the first true minimum lies at 23 samples, three quarters of the forcing period; the shallow flattening at 7 samples here is the same feature that appears as a local minimum at 1.4 time units when the oscillator is sampled at $\Delta t = 0.1$, which is where the τ of Chapter 10 comes from. Chapter 17 takes the other minimum, $\tau = 53$ at 72 samples per period, which is 4.6 time units against the 4.8 of the minimum here. Both are defensible; the two chapters are consistent once it is said which minimum each takes. This chapter uses the earlier one, so that the strand of $2l + 1 = 11$ points spans well under a period. The false-nearest-neighbor fraction vanishes at $D = 4$ for the clean record (Fig. 14.12), and the noisy records again flatten onto a floor of 10 to 15 per cent at 80 % noise and 2 to 3 per cent at 40 %. The same $d = 6$, $q = 3$, $r = 64$ neighbors and bundles of nine eleven-point strands were used. Fig. 14.13 repeats the layout of Fig. 14.8. The unfiltered 5 % floor is at -30 dB relative to σ_x^2/Hz; after ten iterations it is -37 dB under the local projective filter and -58 dB under the smooth one, and at 20 % noise -26 against -60 dB. As on the Lorenz record, the smooth filter's floor barely moves with the noise level, which shows it to be a smoothing floor rather than a residual-noise floor.

Here the signal-to-noise ratio peaks and then declines for both filters (Fig. 14.14, Table 14.3),

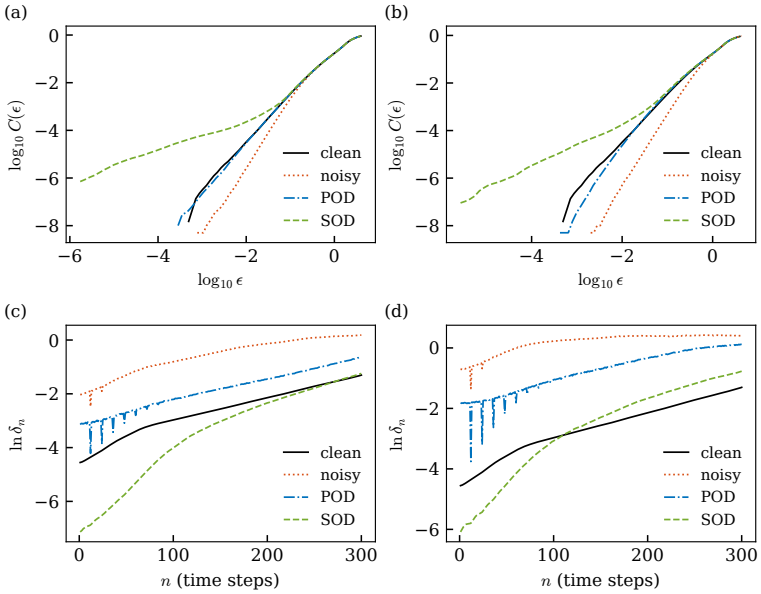


Figure 14.10: Correlation sum (a, b) and short-time divergence (c, d) for the Lorenz record at 5 % (a, c) and 20 % (b, d) noise: clean, noisy, and after ten iterations of each filter. Estimators at $d = 3$, $\tau = 12$ on the whole 20,000-point record: `c2r` with a Theiler window $d\tau$ and 60 length scales, `lyapunov` with 4000 fiducial points, one neighbor and 300 steps. Generated by `python/mkfig_nr_c2ly.py`.

and it peaks earlier the cleaner the record. At the best iteration the smooth filter gains 8.8 to 10.8 dB across the five noise levels and the local projective filter 3.9 to 5.9 dB. Ten iterations are too many at every level and destructive at the cleanest: at 5 % noise both filters end *below* the unfiltered record, the local projective one by 1.9 dB and the smooth one by 0.7 dB. A filter run past its peak on a record that was already good makes it worse, and nothing in the filtered record says so. The correlation sums (Fig. 14.15) are restored by the local projective filter, which gives $D_2 = 2.04$ at 5 % and 2.19 at 20 % against 1.96 for the clean record, and over-corrected by the smooth one, which gives 1.43 and 1.45, a quarter below the clean value. The divergence curves repeat the Lorenz pattern: the local projective filter fluctuates at small separations and tracks the clean curve, the smooth filter is drawn far below it at small n and crosses it. In the phase portraits of Fig. 14.16 the local projective filter recovers no deterministic structure at 40 or 80 % noise. The smooth filter returns a portrait topologically like the clean one at every level, with the fine structure lost and the attractor visibly shrunk at 80 %.

14.5.3 What the Comparison Shows

Four findings carry over to other records.

First, on flow data the smooth filter removes more noise than the local projective filter at the same d and q, and removes it in fewer passes. Measured at each filter's best iteration, the gains are 11 to 13 dB against 6 to 7 dB on the Lorenz record and 9 to 11 dB against 4 to 6 dB on the Duffing. Most of the smooth filter's gain arrives in its first pass. The advantage is largest at high noise, where variance no longer separates signal directions from noise directions but smoothness still does.

Second, the advantage is a mean-square advantage, and it does not carry over to the geometry.

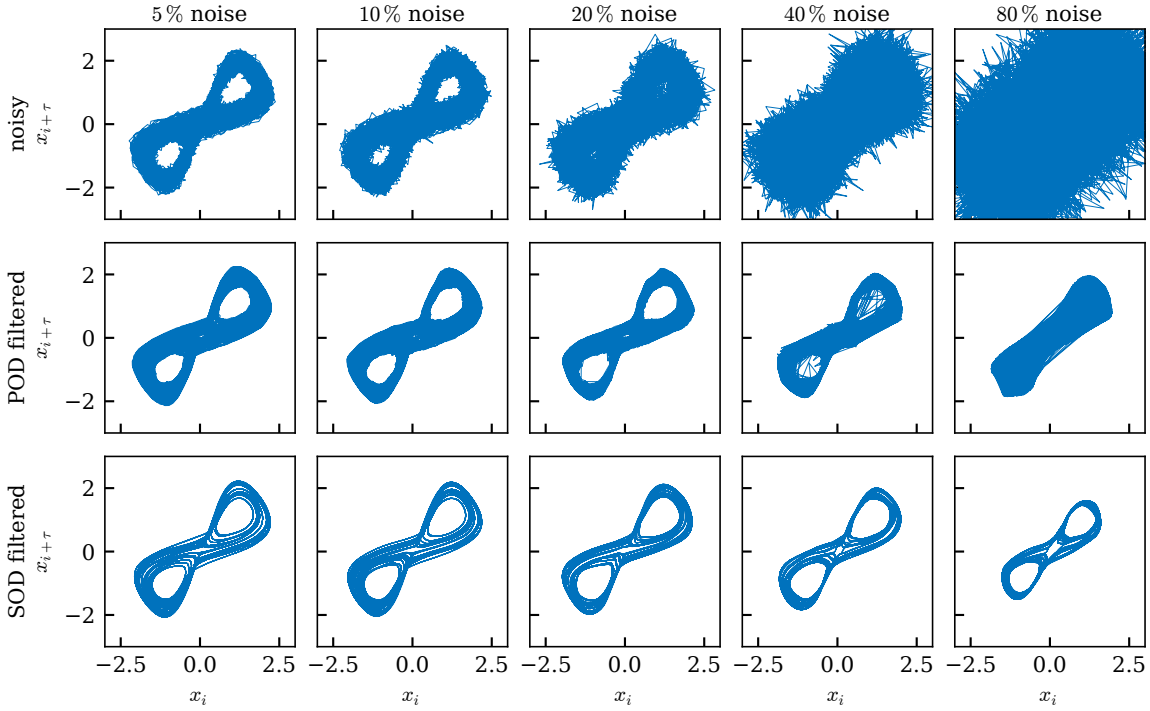


Figure 14.11: Reconstructed phase portraits x_i against $x_{i+\tau}$, $\tau = 12$, of the standardized Lorenz record at the five noise levels (top row) and after ten iterations of the POD-based (middle) and SOD-based (bottom) filters, at the parameters of Table 14.2. All fifteen panels share one set of axes, $\pm 3\sigma_x$, chosen to frame the clean attractor rather than the noise. The frame clips 1.5 % of the points of the 80 % noisy record and 0.08 % of the 40 % one; it clips nothing in the two filtered rows. Generated by `python/mkfig_nr_psgrid.py`.

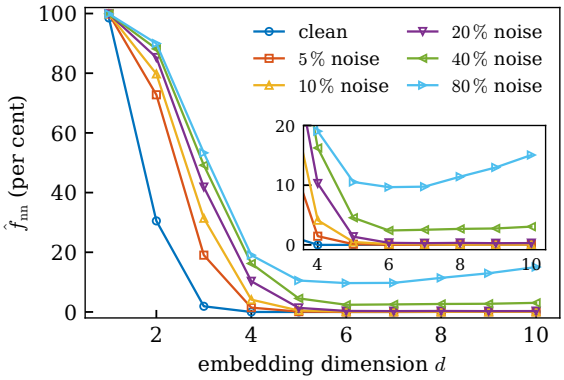


Figure 14.12: Kennel's composite false-nearest-neighbor fraction, thresholds and Theiler window as in Fig. 14.7, against embedding dimension for the clean Duffing record and the five noisy ones; $\tau = 7$, $N = 20{,}000$. Generated by `python/mkfig_fnn_nr.py`.

The same averaging along the strand and across the bundle that buys the decibels shrinks the attractor toward its local means. After ten iterations with a bundle of nine the smooth filter drives the correlation dimension six to nine per cent below the clean value on the Lorenz record and a quarter below it on the Duffing, and it drives the short-time divergence slope *above* the clean value

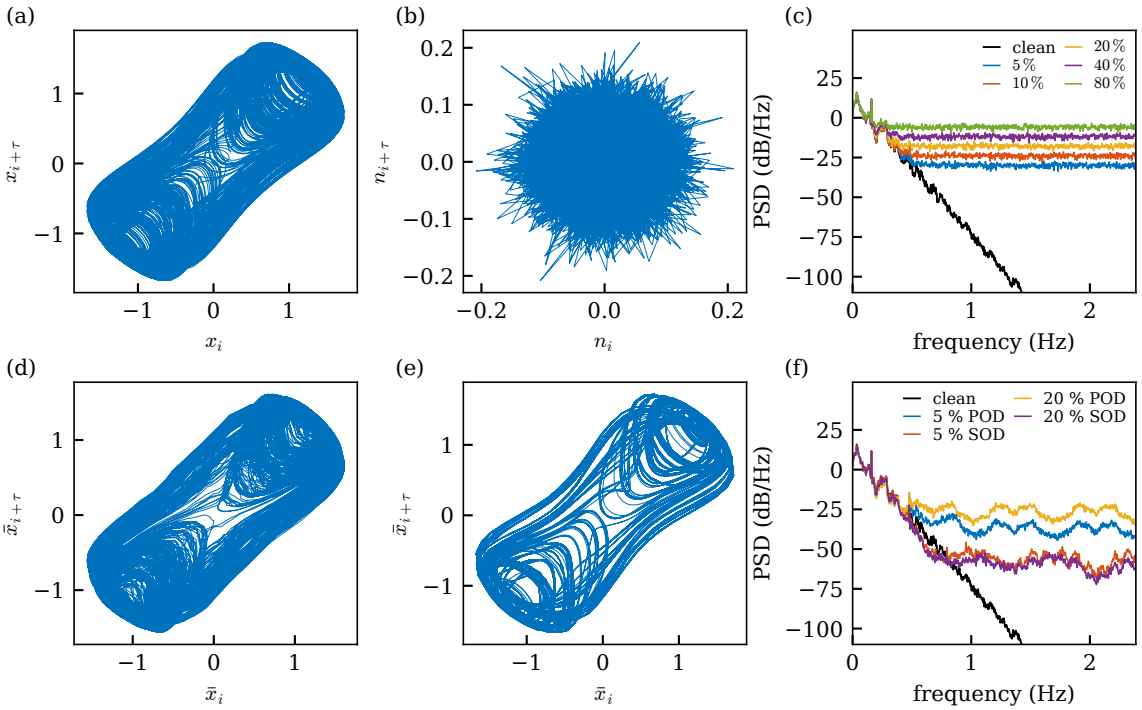


Figure 14.13: Duffing record, $N = 20{,}000$ at $\Delta t = 2\pi/30$, standardized, $\tau = 7$; panels as in Fig. 14.8. (a) Clean delay portrait; (b) the noise added at the 5 % level; (c) spectra of the clean and the five noisy records; (d) POD- and (e) SOD-filtered portraits of the 5 % record after ten iterations; (f) the corresponding spectra. Generated by `python/mkfig_nr_inout.py`.

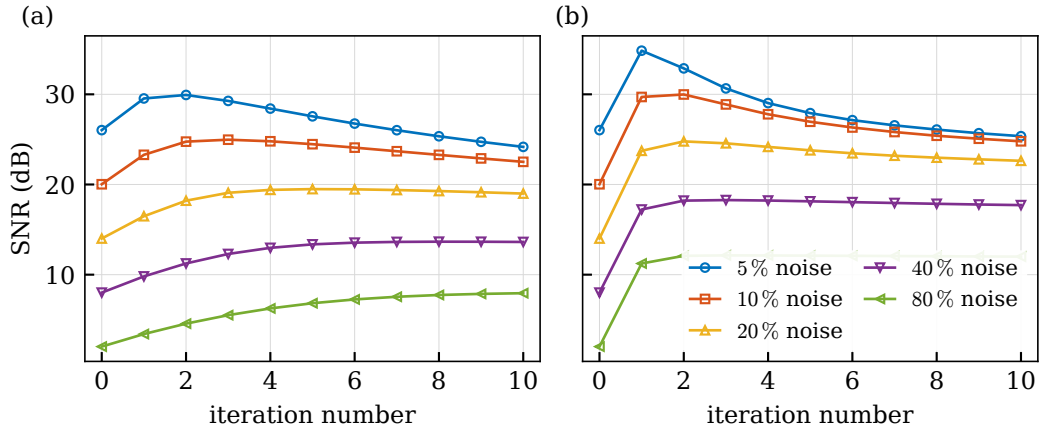


Figure 14.14: Signal-to-noise ratio of the noisy Duffing record against the number of iterations of (a) the local projective and (b) the smooth projective filter, at the parameters of Fig. 14.9 with $\tau = 7$. Iteration 0 is the unfiltered record; at 5 % noise both curves fall below it. Generated by `python/mkfig_nr_snr.py`.

on both, because the filtered points start closer together than the clean ones ever were. The local projective filter, which moves each point only onto its own neighborhood's tangent space, returns D_2 within two per cent and $\hat{\lambda}_1$ within one per cent of the clean Lorenz values at 5 % noise. The filter with the better signal-to-noise ratio is not the filter with the better attractor.

Table 14.3: Signal-to-noise ratio (dB) of the noisy Duffing record ($N = 20{,}000$, $\Delta t = 2\pi/30$, $\tau = 7$; filter parameters as in Table 14.2) before filtering, after ten iterations, and at each filter's best iteration, whose number is in parentheses. At 5 % noise ten iterations of either filter leave the record worse than unfiltered. Computed by `python/mkfig_nr_snr.py`.

Signal	Duffing + 5% Noise			Duffing + 20% Noise		
Filtering	None	POD	SOD	None	POD	SOD
SNR after 10 iterations	26.03	24.17	25.36	14.01	18.99	22.63
SNR at the best iteration	—	29.93 (2)	**34.84** (1)	—	19.49 (5)	**24.78** (2)

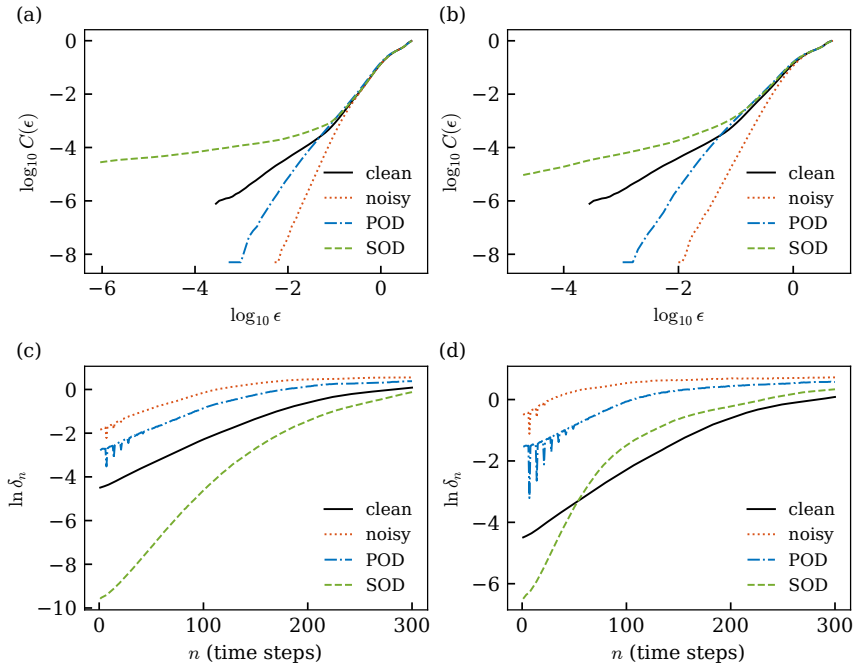


Figure 14.15: Correlation sum (a, b) and short-time divergence (c, d) for the Duffing record at 5 % (a, c) and 20 % (b, d) noise: clean, noisy, and after ten iterations of each filter. Estimators at $d = 4$, $\tau = 7$, otherwise as in Fig. 14.10; the divergence slope is read over steps 60 to 140, where the clean curve is straight. Generated by `python/mkfig_nr_c2ly.py`.

Third, iterating to a fixed count is the wrong stopping rule. Ten iterations are past the peak for the smooth filter at every noise level tried, on both records; on the Duffing record at 5 % noise ten iterations of either filter leave the record worse than unfiltered. The number of passes must be chosen from the record, by the rule of Section 14.6, and not fixed in advance.

Fourth, neither filter recovers the small scales at 80 % noise, and the largest exponent is the quantity that behaves worst throughout: the local projective filter leaves it low at high noise, the smooth filter overshoots it, and Section 14.4 shows it low at every bundle size at 40 per cent noise. The signal-to-noise ratios of Tables 14.2 and 14.3 are what a user with the clean record can measure, and even they do not identify the better filter without the dimension beside them. A user without the clean record must fall back on the diagnostics of the next section, which is what the following

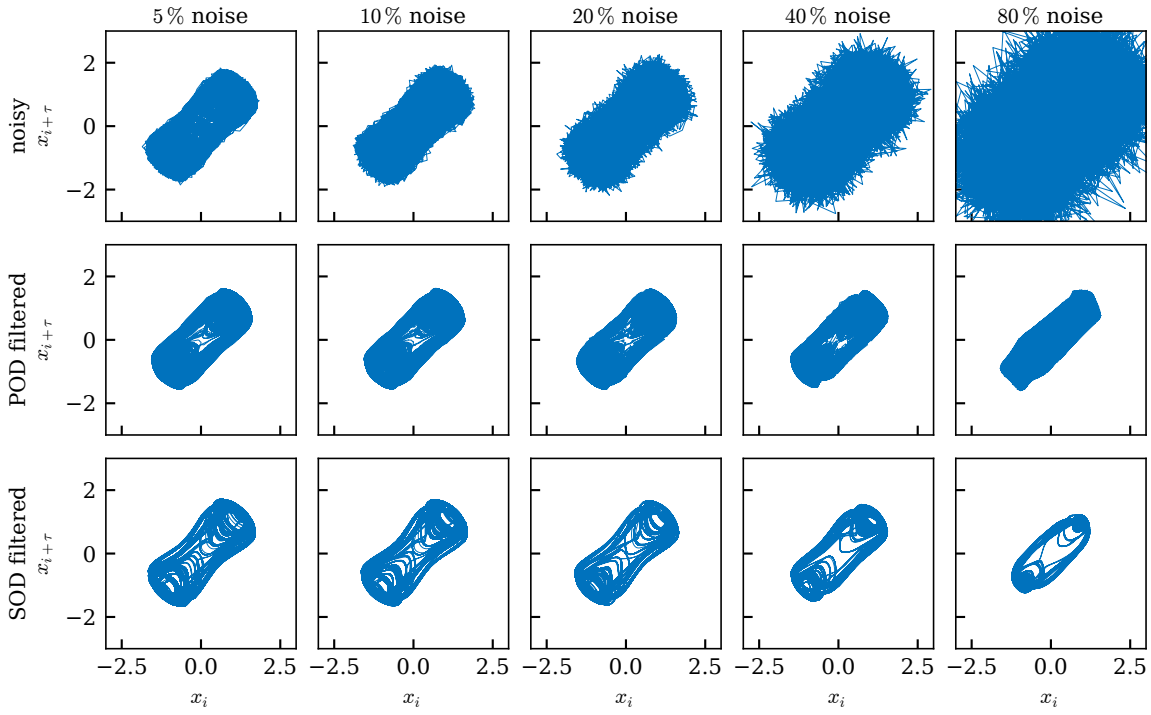


Figure 14.16: Reconstructed phase portraits x_i against $x_{i+\tau}$, $\tau = 7$, of the standardized Duffing record at the five noise levels (top row) and after ten iterations of the POD-based (middle) and SOD-based (bottom) filters. All fifteen panels share one set of axes as in Fig. 14.11, again $\pm 3\sigma_x$; the frame clips 0.94 % of the 80 % noisy record and nothing else. Generated by `python/mkfig_nr_psgrid.py`.

subsection does.

14.5.4 The Two-Well Oscillator Record

The measured record of Section 1.4.1 has no clean version, so Eq. (14.3) cannot be evaluated and neither Table 14.2 nor Table 14.3 has a counterpart here. That is the ordinary situation, and this subsection is what is left of the evaluation when the reference is removed.

The parameters come from the record by the procedure of Section 14.6. The delay is the first minimum of the average mutual information, $\tau = 5$ samples. The correlation dimension of Chapter 11 is $D_2 = 2.68$, so $q = \lceil D_2 \rceil = 3$, and $d \approx 2q = 6$: the same d and q as the two simulated records above. The local projective filter uses $r = 64$ neighbors. The smooth filter uses eleven-point strands with tent weighting and no bundle. Two of those choices are already forced rather than chosen. The bundle rule asks for the input signal-to-noise ratio, which is exactly what an unreferenced record cannot supply, so $r = 1$ is taken by default. And an eleven-point strand spans 0.69 of a forcing period at 16 samples per period, which is not the small fraction the procedure box asks for; the rank condition $2l \geq d$ admits nothing shorter than seven points, or 0.44 of a period. The smooth filter is being run at the edge of its assumptions on this record, and the reader should expect that to show.

Fig. 14.17 collects the four diagnostics that survive the loss of the reference. Panel (a) gives the root-mean-square correction per pass. Both filters move the record by about one per cent of σ_s on the first pass, $0.0096\,\sigma_s$ for the local projective filter and $0.0078\,\sigma_s$ for the smooth one, and the per-pass correction then falls to a floor near $0.005\,\sigma_s$ rather than to zero. The stopping rule of

Section 14.6 is therefore ambiguous here: the correction has flattened by the fourth pass, but every later pass still moves the record by the same amount, and the cumulative departure from the raw record grows almost linearly to $0.052\,\sigma_s$ (local projective) and $0.048\,\sigma_s$ (smooth) after ten passes. A flattening per-pass correction does not mean a converging record.

Panel (b) is the local slope of the correlation sum, computed at $d_E = 4$ and $\tau = 5$ on 30,000 points with a Theiler window of 20. The three curves are indistinguishable over the fitting band $\ln r \in [-1.45, -0.35]$ of Chapter 11 and above it. Read over that band, D_2 is 2.699 for the raw record, 2.652 after ten passes of the local projective filter and 2.742 after ten of the smooth one. Both changes are inside the ± 0.03 spread the raw estimate already shows over $d = 4$ to 8. Below $\ln r \approx -3$ the curves separate, and neither filter removes the trough in the local slope that Section 11.6.3 reports there: the local projective filter deepens it and the smooth filter raises it slightly. *Neither filter opens a scaling region that the raw record did not have.* On the evidence of panel (b) alone there is nothing to gain.

Panel (c) is the short-time divergence curve of Chapter 12, 5000 fiducial points, one neighbor, 80 steps, Theiler window 32, with the slope read over steps 10 to 64. The raw record gives $\hat{\lambda}_1 = 0.611$ nats per forcing period with a root-mean-square fit residual of 0.015. Ten passes of the smooth filter give 0.598 and a residual of 0.013; ten of the local projective filter give 0.661, eight per cent higher, with the residual more than doubled to 0.033. The local projective filter also drops the curve at $n = 0$ from $\ln \delta_0 = -3.73$ to -4.00. That is the warning sign named in the discussion of Fig. 14.10, and it is readable without a clean record, because here the raw record is the reference: filtered points that start closer together than the unfiltered ones have been drawn together, not cleaned.

Panel (d) settles it. The residual $s_i - \bar{s}_i$ is not white for either filter. Its lag-one autocorrelation is 0.83 for the local projective filter and 0.87 for the smooth one, against a white-noise band of $\pm 2/\sqrt{N} = \pm 0.0065$, and its spectrum carries the drive line at 1.4×10^3 and 2.6×10^4 times its own median level. By step 7 of the procedure box, and on the assumption that the measurement noise of this record is white, both filters are removing signal. Colored sensor noise, or electrical pickup at the drive frequency, would also leave a correlated residual carrying the drive line. The size of that line makes signal removal the likely reading here, but the residual alone does not prove it. At 16 samples per forcing period this is not surprising: the curvature the reconstruction has to resolve over a $d\tau = 30$-sample window is large, and both filters treat curvature as noise. The record does still pass the consistency checks of Chapter 10 after filtering — the first minimum of the average mutual information stays at 5 samples and the false-nearest-neighbor fraction still falls to below 0.3 per cent at $d = 4$ — so passing them is not evidence that filtering helped. It only shows that the filters have not destroyed the reconstruction.

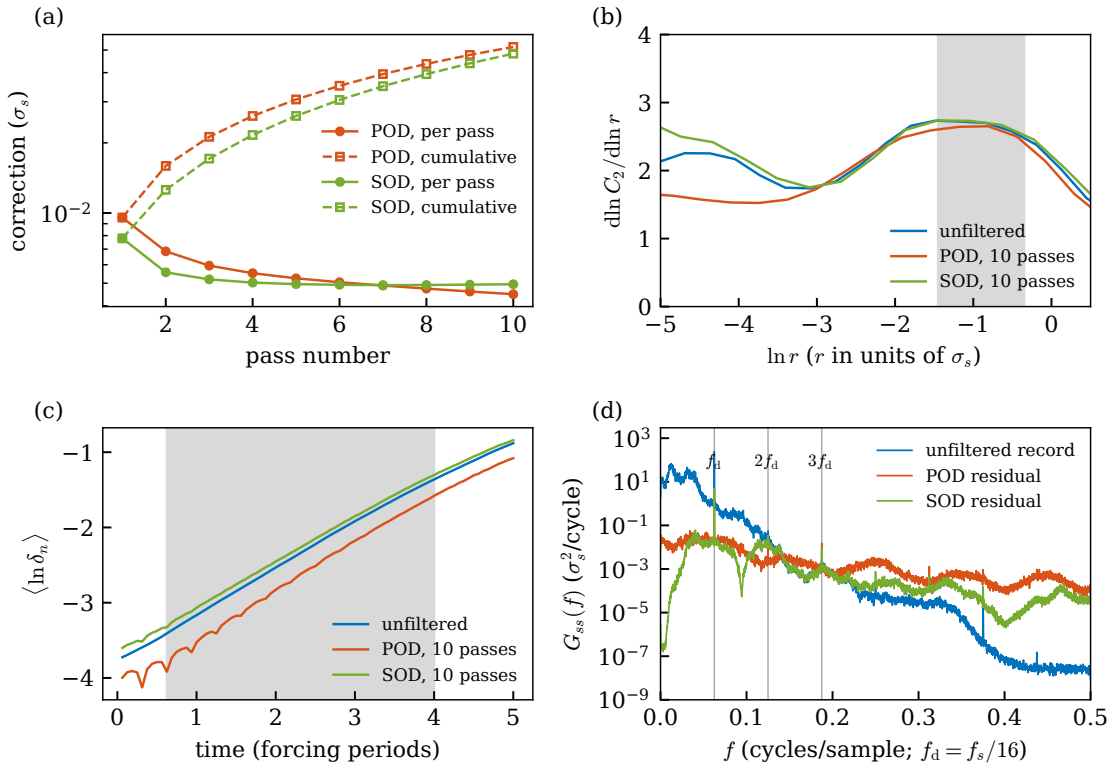


Figure 14.17: Noise reduction on the two-well oscillator record of Section 1.4.1, $N = 96{,}000$ samples, standardized; $\tau = 5$ samples, $d = 6$, $q = 3$; `podfilt` with 64 neighbors and `sodfilt` with eleven-point tent-weighted strands and no bundle, ten passes each. (a) Root-mean-square correction per pass (solid) and cumulative departure from the raw record (dashed), in units of σ_s. (b) Local slope of the correlation sum at $d_E = 4$, $\tau = 5$, 30,000 points, Theiler window 20; the shaded band is the fitting range $\ln r \in [-1.45, -0.35]$ of Chapter 11. (c) Short-time divergence at $d_E = 4$, $\tau = 5$, 5000 fiducial points, one neighbor, Theiler window 32; the shaded band is the fitting window, steps 10 to 64. (d) Welch spectral density (8192-point segments) of the raw record and of the two residuals, with the drive line $f_d = f_s/16$ and its first two harmonics marked. Generated by `python/mkfig_nr_twowell.py`.

Pitfall: filtering a record with no reference

Without a clean record the following can be established, and nothing more. *That a filter changed the record*, from the per-pass and cumulative corrections of Fig. 14.17(a). *That it removed signal*, from a residual that fails the whiteness test of step 7, provided the measurement noise is known to be white; the verdict is one-sided, since a white residual is not evidence that only noise was removed. *That an invariant is or is not sensitive to the filter*, by recomputing it after every pass and comparing the drift with the estimator's own uncertainty. *That the record has been smoothed rather than cleaned*, from a divergence curve that starts below the unfiltered record's. What cannot be established is the one thing Tables 14.2 and 14.3 give: whether the filtered record is closer to the truth. No diagnostic computed from the filtered record alone answers that, because every one of them is a property of a record the filter made.

On the two-well oscillator record all four diagnostics point the same way. The correlation sum already has a scaling region, and filtering does not extend it. The divergence curve is already straight over more than three forcing periods. The invariants move by less than their own uncertainty under the smooth filter and by more than it under the local projective one. Both residuals are strongly correlated. By the rule at the end of Section 14.6 this record is not noise-limited at the scales the estimators use, so the values reported in Chapters 11 and 12 are the unfiltered ones, and this subsection is the statement of why. A record that is noise-limited at these scales would show the opposite pattern: a scaling region that filtering extends, invariants that settle as passes accumulate, and a residual that passes the whiteness test. The diagnostics are the same; only the verdict changes.

14.6 Choosing the Parameters, and When Not to Filter

Noise reduction is an estimation problem with a bias–variance trade-off, and its parameters set the balance. Too small a q, too large a neighborhood or strand, or too many iterations remove signal (bias). Too large a q, too small a neighborhood, or too few iterations leave noise (variance). The following procedure fixes the parameters from quantities that can be computed on a record whose clean version is unknown.

Choosing τ, d, q, l or r, and the number of iterations

1. *Delay* τ: from the average mutual information of the noisy record (Chapter 9); noise raises the whole curve but rarely moves its first minimum.
2. *Projection dimension* q: the minimum embedding dimension D from the false-nearest-neighbor fraction of Chapter 10. Read it where the noisy curve changes slope rather than where it vanishes. When the correlation dimension of the noisy record is available, $q \geq \lceil D_2 \rceil$.
3. *Global dimension* d: about $2q$, and at least $q + 2$. Larger d removes more noise per pass by the $(d - q)/d$ rule but needs longer records to populate the neighborhoods.
4. *Neighborhood*: for the local projective filter, r between $2d$ and $10d$ neighbors (64 at $d = 6$ here). For the smooth filter, a half-length l from $d/2$ to $2d$ with $2l \geq d$; an eleven-point strand at $d = 6$. Check that the strand spans a small fraction of a characteristic period.
5. *Bundle* r: 1 unless the input signal-to-noise ratio is below about 15 dB; then 4 (Section 14.4).
6. *Iterations*: run to ten and plot the root-mean-square correction per pass. Stop where it flattens. Never take a fixed count on faith: in Section 14.5 the smooth filter with a bundle of nine peaked at the first to the third pass and lost ground after it, and on the cleanest Duffing record ten passes of either filter left the record worse than unfiltered. When the correlation dimension or the divergence curve is the goal, compute it after each pass and stop when it stops changing. A value that keeps drifting is being set by the filter, not the data.
7. *Check the residual* $x_i - \bar{x}_i$: its spectrum should be flat and its autocorrelation should vanish beyond lag zero. A residual with the spectral peaks of the signal, or with the attractor visible in its own delay plot, means signal was removed (Problem 14.2). This reading assumes white measurement noise; colored sensor noise or pickup at a drive frequency produces the same symptoms.

Report all of the above with any invariant computed from a filtered record.

Pitfall: filtering when it is not warranted

Noise reduction is not a routine preprocessing step. Applied to a record whose noise is dynamical (Section 14.1), it manufactures a deterministic-looking attractor that the system does not possess. Every invariant computed from it is then a property of the filter. Applied to a record that is already clean, it removes curvature and small-scale structure. On the 20 per cent Lorenz record the correlation dimension fell below the clean value at $r = 8$ (Table 14.1), and the largest exponent came out low at every setting. Applied to a stochastic record, such as the AR(2) process of Chapter 3 or any linearly filtered noise, it still lowers the "noise floor" and produces a smoother delay plot. A projection removes variance whatever its origin. The surrogate tests of Chapter 16 must therefore be run on the *unfiltered* record, or with the filter applied identically to every surrogate. Applied before a recurrence analysis (Chapter 18), it changes every recurrence statistic and must be declared. The safe rule: filter only when the estimator of interest is demonstrably noise-limited, by a correlation sum with no scaling region or a divergence curve flat at small separations. Report the estimate with and without filtering. If the two disagree by more than the estimator's own uncertainty, the filtered value is not to be preferred; the disagreement is the finding.

14.7 Discussion

Projective noise reduction rests on one geometric fact: measurement noise is isotropic in the reconstruction and the attractor is not. Everything else is estimation. The local projective method estimates the tangent space from the covariance of a neighborhood cloud. It is limited by the neighborhood size, which must be small enough for the tangent approximation and large enough to average the noise. The curvature bias is set by the same size. The smooth method estimates the tangent space from the smoothness of a short trajectory strand. It is limited by the derivative estimate, which restricts it to densely sampled flows. Both are biased toward the interior of the attractor, and both remove variance whatever its origin. Both are iterated on their own output, so their errors compound in ways that only a comparison with the clean record fully reveals.

The chapter has treated the result as an estimation problem. The target is the clean record under the measurement-noise model, the criterion is the mean-square error, and the bias is toward an attractor shrunk to its local means. Table 14.1 shows what that costs. The correlation dimension can be driven through the clean value, and the largest exponent, which measures the separation the averaging suppresses, is the slowest to recover and at 40 per cent noise is not recovered at any setting. Section 14.5 finds the same bias in the number of passes rather than the bundle size: ten iterations of the smooth filter with a bundle of nine drive the correlation dimension below the clean value and the divergence slope above it on both records, while raising the signal-to-noise ratio. Bias and mean-square error point in different directions here, and the mean-square error is the weaker guide. The lesson for the chapters that follow is procedural. A filtered record is a different record. Any invariant of Chapters 11 and 12, any predictor of Chapter 15, and any recurrence statistic of Chapter 18 computed from it must be reported with the filter's parameters. Wherever the clean record is unavailable, it must be reported beside the same quantity computed from the unfiltered data.

Two extensions are natural. The strand construction with POD in place of SOD isolates the contribution of time ordering from that of smoothness; Problem 14.3 asks for the comparison. The global counterpart of the local projection is singular spectrum analysis, which decomposes the whole

trajectory matrix of Chapter 8 at once [14]. It is a linear filter, since one subspace serves the whole attractor, and it therefore shares the limitation that Section 14.1 describes. The local methods succeed because the subspace is allowed to turn with the attractor.

Problems

Problem 14.1

Computational

Create an artificial time series by integrating the Chua's circuit by Matsumoto *et al.* (1985):

$$\dot{x} = \alpha \left(y - x + bx + \frac{1}{2}(a-b)\left(|x+1| - |x-1|\right) \right) \tag{14.19}$$

$$\dot{y} = x - y + z \tag{14.20}$$

$$\dot{z} = -\beta y \tag{14.21}$$

using these parameters: $\alpha = 9$, $\beta = 100/7$, $a = 8/7$, and $b = 5/7$. Start the simulation using the following initial conditions: $x(0) = 0$, $y(0) = 0$, and $z(0) = 0.6$. In particular, using the z time series containing at least 60,000 points:

1. Make sure your time series is appropriately sampled; make it zero mean and unit standard deviation.
2. Determine the needed embedding parameters.
3. Make noisy time series with one %, five %, and 20 % additive Gaussian noise (percents are in terms of relative standard deviations of the signal and the noise).
4. Apply both projective and smooth noise reduction schemes to the noisy time series.
5. Compare the results using phase portraits, power spectrum, correlation sum, and Lyapunov exponents.
6. Interpret your results.

Problem 14.2

needs Ch. 15 Computational

Problem 14.1 exercised the algorithms on Chua data. This problem asks how to know whether noise reduction has helped, which is harder than running it.

1. Generate a clean Lorenz record, then add white noise at a known SNR. You now have the ground truth that experimental work never has.
2. Apply POD-based local projective noise reduction with global embedding dimension $d = 8$ and tangent dimension $q = 1, \ldots, 7$. For each q, report the RMS error against the clean record.
3. Plot that error against q. Identify the optimal q and compare it with the dimension you would have chosen from the false-nearest-neighbor analysis of Chapter 10 and from the correlation dimension of Chapter 11.
4. Now discard the ground truth and consider what you could have used instead: the residual $\|\boldsymbol{y}_i - \bar{\boldsymbol{y}}_i\|$, the change in the estimated D_2, and the change in the one-step prediction error of Chapter 15. Which of these three would have selected the same q?
5. Over-project deliberately, taking q well below the optimum, and examine the residual. Noise reduction that removes signal leaves a residual that is *not* white — test yours for whiteness and use that as a diagnostic.

Problem 14.3

needs Ch. 16 Extension

The chapter states that SOD-based noise reduction applies to flows but not to maps. This problem contrasts the two methods where both apply.

1. Implement both the POD-based and the SOD-based projective schemes.

2. Apply both to the noisy Lorenz record of Problem 14.2 at several SNRs, using the optimal q found there. Compare RMS errors against the clean record.

3. Repeat for a record whose noise is *not* white — use the $1/f^{\alpha}$ noise of Problem 16.5. Does either method's advantage survive?

4. Both schemes assume the noise leaks into directions transverse to the local tangent space. Construct a case that violates the assumption: add noise *along* the trajectory (a small random timing jitter rather than an amplitude error) and report what each method does with it.

5. In light of part (4), state what class of measurement error projective noise reduction cannot address.

Problem 14.4

Core

The arithmetic behind two numbers quoted in Section 14.1.

1. A signal is rounded to a grid of step Δ. Assuming the rounding error is uniform on $[-\Delta/2, \Delta/2]$, show that its variance is $\Delta^2/12$. For a 12-bit converter spanning $\pm 5\sigma_x$, compute Δ, the root-mean-square quantization noise as a fraction of σ_x, and the signal-to-noise ratio Eq. (14.3). You should find 63 dB. Repeat for 16 bits and for a range set to $\pm 20\sigma_x$ (a gain set too low), and state which of the two choices costs more.

2. Isotropic noise of variance σ_γ^2 per coordinate is added to points on a q-dimensional linear subspace of $\mathbb{R}^d$. Show that orthogonal projection onto the subspace leaves noise power $q\sigma_\gamma^2$ out of $d\sigma_\gamma^2$, so that one pass gains $10\log_{10}(d/q)$ dB: 3.0 dB for $d = 6$, $q = 3$ and 0.8 dB for $q = 5$.

3. Test the rule with `podfilt` (Appendix A.4) on 20,000 points of the Lorenz x record at $\Delta t = 0.01$ with 10% noise (20 dB): $\tau = 12$, $d = 6$, 64 neighbors, one pass, for $q = 3$ and $q = 5$. You should find gains of about 3.3 and 1.3 dB, a little above the rule in both cases. Why can a single pass do slightly *better* than $(d - q)/d$ on a curved attractor whose local dimension is below q?

4. Iterate the $q = 3$ filter for three passes and tabulate the signal-to-noise ratio after each. If each pass removed the fraction $(d - q)/d$ of the remaining noise, the gain would be 3 dB per pass. Show that the measured gains fall well short of that from the second pass on, and explain which component of the noise a projection can never remove.

Problem 14.5

needs Ch. 16 Computational

The pitfall box warns that a projection removes variance whatever its origin. Show it on a record with no attractor.

1. Generate 20,000 samples of the AR(2) process of Chapter 3 with $\rho = 0.05$, normalized to unit variance, and 20,000 samples of Gaussian white noise. Filter each with `podfilt` ($\tau = 5$, $d = 6$, $q = 3$, 64 neighbors) for three passes. Report the root-mean-square correction per pass, as a fraction of the standard deviation. Both records are "denoised" by an amount comparable to the noise levels of Section 14.5, although neither contains a clean trajectory.

2. Compute the autocorrelation of the residual $x_i - \bar{x}_i$ for the AR(2) record and plot it with the white-noise band $\pm 2/\sqrt{N}$ of Chapter 6. You should find the residual strongly correlated (a lag-one coefficient near 0.9): the diagnostic of Section 14.6 correctly reports that signal, not noise, was removed. Does the same diagnostic fire for the white-noise record, where everything removed is noise by construction? What does that say about the diagnostic's power?

3. Plot the two-dimensional delay portraits of the AR(2) record before and after filtering, and estimate the correlation dimension of both with `c2r` at $d = 4$ with a Theiler window. Does the filtered record look, or measure, more deterministic than the raw one? If a colleague showed you only the filtered portrait, what could you conclude?

4. State the rule from the pitfall box that would have prevented the misreading, and the test from Chapter 16 that settles the question of determinism for a record like this.

Problem 14.6

needs Ch. 16 Experimental

Noise reduction of the two-well oscillator record of Section 1.4.1, and how to know whether it helped without the clean record.

1. Take τ from Problem 9.6 and D from Problem 10.6; set $q = \lceil D_2 \rceil$ from Problem 11.6 and $d = 2q$ as Section 14.6 recommends. Express the strand length in forcing periods and check the requirement that it span a small fraction of one. The record carries 16 samples per forcing period, so decimation is not available as a remedy; state instead what the rank condition $2l \geq d$ allows and what that costs the smooth filter. Run both `podfilt` and `sodfilt` for up to ten passes and plot the root-mean-square correction per pass. Does it fall to zero, or to a floor?

2. Compute the residual $s_i - \bar{s}_i$ after the pass at which the correction flattens. Test it for whiteness with the autocorrelation band, and compute its spectrum with Welch's method: is it flat, or does it carry the forcing line and its harmonics? Compare its variance with the measurement-noise variance estimated from the spectral floor (Problem 4.6) and from the trailing POVs (Problem 13.6). A residual variance well above both means signal is being removed.

3. Estimate the correlation dimension (Problem 11.6) and the spectrum of the record before and after each filter. Does the scaling region extend to smaller scales after filtering, and does the plateau value change by more than the error E reported in Problem 11.6? Apply the rule of the pitfall box of Section 14.5.4 to decide whether the filtered value may be preferred. Compare your answer with Fig. 14.17, and account for any difference by the parameters you chose.

4. Report, for whichever filter you would use, every quantity listed in its algorithm box, the residual whiteness statistic, and the before-and-after dimension. State which quantity you would compute from the filtered record and which from the raw one, and why the surrogate test of Chapter 16 belongs to the second group.

15

Modeling and Predicting Chaotic Time Series

To understand the behavior of a dynamical system in terms of some meaningful parameters, we seek the appropriate mathematical model that captures the important physical characteristics of the system that are observable experimentally. Ideally, we would like to have a simplified mathematical description of natural behavior consistent with the empirical observations that could also explore wider implications under different parameters or environmental conditions. Such models are called *phenomenological*, as opposed to *first principles* based models that are usually derived from fundamental physical principles (e.g., classical mechanics models). The first principles models also need to be consistent with empirical evidence; otherwise, we need to reexamine our fundamental understanding of the underlying physics.

In this chapter, we will discuss the development of data-based models. The subject was opened by Farmer and Sidorowich [60] and Casdagli [21], whose papers remain the natural entry points; Abarbanel *et al.* [3] review the state of the art as it stood after the first decade. In many cases, we would also have some tangential and incomplete information about the systems or the processes generating the data we use for modeling. However, we may need to proceed with just the available scalar or multivariate measurements and not much else. In any case, we cannot expect a unique solution to this modeling problem, and we have to often use other criteria in selecting a specific model (e.g., the simplest possible model consistent with observations is usually selected). Making that principle quantitative is the business of an information criterion such as Akaike's [5], which penalizes parameter count against likelihood. We can also use other metrics to evaluate the usefulness of a model, such as stability, convergence, or computability.

The main advantage of the first-principles models is that they inherently have physical meaning and parameters. In contrast, for models based on time series, we cannot expect to have parameters that have any physical meaning. However, a successful data-based model can faithfully reproduce empirically observed dynamics and the associated properties or statistics and, in this regard, is usually superior to the phenomenological model.

The process of data-based model building has two main components: (1) selection of particular model type or equation and (2) identification of model parameters by fitting or updating the model predictions to empirically available data. If the identified model parameters have any phenomenological meaning, the model is called *parametric*. Otherwise, the model is called *nonparametric*. The selection of model equation is usually guided by the purpose of the model, which could have conflicting objectives. The usual objectives for model building are predicting future evolution, noise reduction, control, system identification, or generation of some synthetic data sharing the same

statistical or dynamical properties for the empirically available data.

The basic sources of predictability in the data are linear and nonlinear correlations and determinism. For linear systems, determinism is equivalent to having linear correlations. Therefore, usual choices for linear models are *moving average* (MA), *autoregressive* (AR), or a combination of AR and MA (ARMA). There is no such connection for nonlinear systems, and to model chaotic time series, we cannot just rely on autocorrelations. The natural choice for nonlinear systems is to extend linear models by introducing local or global nonlinearity. For example, *threshold autoregressive* (TAR) models use a piecewise linear modeling paradigm, which employes a set of regular AR models that are valid only in small local neighborhoods of the vector space. These will be the members of local linear models. In contrast, we can extend the AR model by including nonlinear terms to create a globally nonlinear model. Thus, these would be classified as global nonlinear models.

In this chapter, we will focus on models for deterministic or mainly deterministic data. Readers who are interested in AR, MA, ARMA, or their extensions can refer to any time series analysis books such as Refs. [12, 87, 130]. In what follows, we will discuss nonlinear models built in the reconstructed phase space of dynamical systems.

15.1 Model Development in Reconstructed Phase Space

We assume that the dynamics on the attractor can be reconstructed through the delay coordinate embedding of scalar time series using the delay τ and embedding dimension d. The process should also work and may benefit by having vector-valued time series too. Thus, we assume that we are working in the reconstructed phase space and we have total N points $\{\boldsymbol{x}_k\}_{k=1}^{N}$, which are governed by some nonlinear map:

$$\boldsymbol{x}_{n+1} = \boldsymbol{F}(\boldsymbol{x}_n)\,, \tag{15.1}$$

where $\boldsymbol{F} : \mathbb{R}^d \to \mathbb{R}^d$. Then our objective is to approximate this map with a model:

$$\hat{\boldsymbol{x}}_{n+1} = \boldsymbol{P}(\boldsymbol{x}_n)\,. \tag{15.2}$$

To determine the appropriate form of $\boldsymbol{P}$ and its parameters, we need a criterion to evaluate the quality of the model. This criterion could be the minimization of some cost function that would depend on the purpose of the model. If, for example, we are looking for best possible predictions in the least squares sense, we may want to minimize mean squared *prediction error*:

$$e_k = \sqrt{\frac{1}{N-k}\sum_{n=1}^{N-k} \left\|\boldsymbol{x}_{n+k} - \boldsymbol{P}^k(\boldsymbol{x}_n)\right\|^2}\,, \tag{15.3}$$

where we are minimizing k-step prediction error. In many cases, we would minimize for $k = 1$. However, a particular choice of $k = \tau$ (which is usually the case for data generated by maps when $\tau = 1$) simplifies the modeling considerably. In particular, with the forward delay vectors $\boldsymbol{x}_n = [x_n, x_{n+\tau}, \ldots, x_{n+(d-1)\tau}]^{\mathrm{T}}$ of Eq. (8.8), only the last coordinate of $\boldsymbol{x}_{n+\tau}$ is new; the others are the last $d-1$ coordinates of $\boldsymbol{x}_n$, exactly as in Eq. (12.59) of the Lyapunov exponent chapter. Therefore, Eq. (15.2) can be written as

$$\hat{\boldsymbol{x}}_{n+\tau} = \left[x_{n+\tau}, x_{n+2\tau}, \ldots, x_{n+(d-1)\tau}, \hat{x}_{n+d\tau}\right]^{\mathrm{T}} = \left[x_{n+\tau}, x_{n+2\tau}, \ldots, x_{n+(d-1)\tau}, P(\boldsymbol{x}_n)\right]^{\mathrm{T}}, \tag{15.4}$$

or just as the scalar model

$$\hat{x}_{n+d\tau} = P(\boldsymbol{x}_n)\,, \tag{15.5}$$

which predicts the sample one delay beyond the end of the embedding window.

In many cases, even when data come from a continuous flow and not a map, one does not bother to choose an optimal delay τ and uses the unit delay ($\tau = 1$). Successive measurements are then strongly correlated. The latest sample in the forward vector $\boldsymbol{x}_n$ is x_{n+d-1}, and it is better to model the increment beyond it:

$$\hat{x}_{n+d} = x_{n+d-1} + \Delta t\, P(\boldsymbol{x}_n)\,, \tag{15.6}$$

where Δt is the sampling time.[1] The predicted sample lies one step beyond the embedding window, as in the scalar model above.

15.2 Method of Analogues and Simple Averaging

If the data is clean and plentiful, the local models have some advantages over the global models. Local models usually require more computation at each prediction step but can be more accurate if data is dense in the phase space and has little noise. The simplest of local models or algorithms is Lorenz's *method of analogues* [128], devised to ask how far atmospheric states with near-identical histories subsequently diverge. This method does not involve any modeling to predict the future evolution of a point in the phase space; all we need to do is lookup its nearest neighbor in our data and use its future image as a prediction. However, if we use this for long-time predictions, we would reproduce our base trajectory, which would yield a periodic time series. The *simple averaging* method looks up several nearest neighbors of the initial point and uses the average of their future images as the prediction as illustrated in Fig. 15.1 to mitigate this periodicity problem. This method could be effective when dealing with high dimensional state space or data containing a large stochastic component.

Two ways of predicting k steps ahead must be distinguished. A *direct* predictor fits a map from $\boldsymbol{x}_n$ to $\boldsymbol{x}_{n+k}$ and applies it once; an *iterated* predictor fits the one-step map and applies it k times. Farmer and Sidorowich [60] showed that the iterated predictor is the more accurate for chaotic data: the one-step map is smoother than the k-step map, whose complexity grows as $e^{\lambda_1 k \Delta t}$, so it is approximated better by the same local model from the same data, and the error of the iterated forecast grows as $e^{\lambda_1 k \Delta t}$ from a smaller base. The prediction error e_k above is written for the iterated form, $\boldsymbol{P}^k$; Eq. (15.7) below fits the direct form for a general k, and the two coincide for $k = 1$.

15.3 Local Linear Models

We can do better than the simple averaging method by building *local linear models* which show how neighborhoods about each data point are mapped forward in time [21, 60]. The same local regression underlies the Jacobian estimates of Section 12.3, and the rank condition noted there applies equally here. Therefore, we can write a simple local linear map relating the state at time n, $\boldsymbol{x}_n$ to the state $\boldsymbol{x}_{n+k}$ at time $n + k$ as:

$$\hat{\boldsymbol{x}}_{n+k} = \boldsymbol{A}_n^{(k)} \boldsymbol{x}_n + \boldsymbol{b}_n^{(k)}\,, \tag{15.7}$$

where the model parameter matrix $\boldsymbol{A}_n^{(k)}$ and parameter vector $\boldsymbol{b}_n^{(k)}$ (the superscript is the horizon, not a power) are determined by regression at each point in the data set. For the regression, we use either a fixed distance method or fixed mass method to define local neighborhoods $\mathcal{B}_\epsilon(\boldsymbol{x}_n)$.

[1] This is similar to the Euler integration of differential equations. For a general delay the same model reads $\hat{x}_{n+(d-1)\tau+1} = x_{n+(d-1)\tau} + \Delta t\, P(\boldsymbol{x}_n)$.

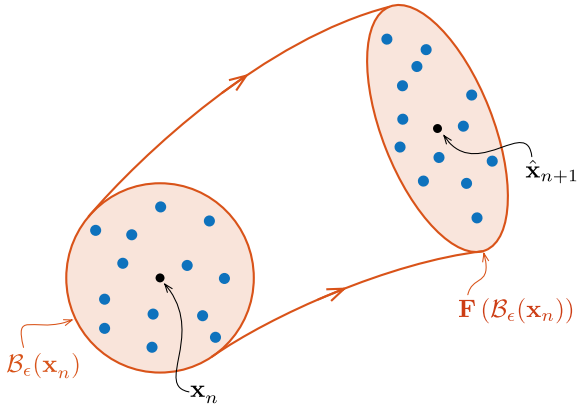


Figure 15.1: Simple averaging method looks up several nearest neighbors of $\boldsymbol{x}_n$ in its neighborhood and uses the average of their future images as the prediction $\hat{\boldsymbol{x}}_{n+1}$

The basic procedure for modeling is explained schematically in Fig. 15.2. Given a point $\boldsymbol{x}_n$ in the reconstructed phase space, we select a fixed number of points (for the fixed mass method), say r, in the neighborhood of that point $\mathcal{B}_\epsilon(\boldsymbol{x}_n)$. These same points, k time steps later, lie in some other small region of phase space $\boldsymbol{F}^k\mathcal{B}_\epsilon(\boldsymbol{x}_n)$ (k cannot be too large, since we expect to have at least one positive Lyapunov exponent). Then, an affine map Eq. (15.7) fitted in the least-squares sense between these two clusters of points provides a locally linear approximation to the dynamics of the system. Please note, the smaller the $\mathcal{B}_\epsilon(\boldsymbol{x}_n)$, the more accurate the fitted affine map is expected to be. Note, also, that for any fixed number r of neighboring points, the size of the $\mathcal{B}_\epsilon(\boldsymbol{x}_n)$ will be smaller if the data set is larger or the density of points in the phase space higher.

The collection of these local linear models gives us the global nonlinear model of the system. In addition, individual local linear models can be viewed as a linearization of this global model at the points of interest in the phase space. Therefore, we implicitly require that the global nonlinear model be continuously differentiable at our points of linearization. Thus, local linear models in effect are approximating a tangent space to the global map $\boldsymbol{F}(\boldsymbol{x}_n)$ at each $\boldsymbol{x}_n$. However, if the size of $\mathcal{B}_\epsilon(\boldsymbol{x}_n)$ is large compared to the inverse of the curvature of the true attractor at $\boldsymbol{x}_n$, then the approximation will be poor.

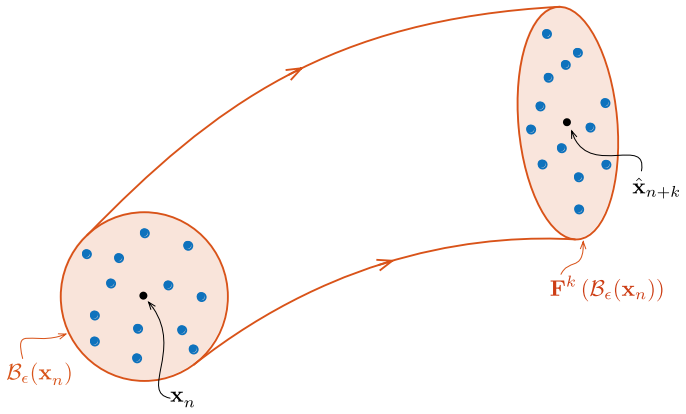


Figure 15.2: Schematic illustration of the basic idea about local linear models

Bias, variance, and record length

The neighborhood size ϵ (or the neighbor count r) is the one free parameter of a local linear model, and it controls a bias–variance trade-off of the standard nonparametric-regression kind. The neglected quadratic terms of $\boldsymbol{F}$ bias the fitted map by an amount of order $\epsilon^2 \left\| D^2 \boldsymbol{F} \right\|$ (curvature bias, growing with ϵ); the measurement noise of variance σ^2 on the r neighbors gives the fitted coefficients a variance of order σ^2/r (shrinking with ϵ, since $r \sim N\epsilon^D$ on an attractor of dimension D). Balancing the two for noise-free data gives the scaling of Farmer and Sidorowich [60]: with N points on a D-dimensional attractor and a local polynomial of order m, the neighborhood size is $\epsilon \sim N^{-1/D}$ and the one-step prediction error scales as

$$e \sim \epsilon^{m+1} \sim N^{-(m+1)/D}\,, \tag{15.8}$$

so that the local linear model ($m = 1$) improves as $N^{-2/D}$ while simple averaging ($m = 0$) improves only as $N^{-1/D}$. On the Hénon map ($D \approx 1.26$) with $r = 8$ neighbors and N from 10^3 to 3.2×10^4, the measured exponents are -1.60 for the local linear model and -0.85 for averaging, against -1.59 and -0.79 from Eq. (15.8) (`python/chk_fs_scaling.py`). Two further points. With noise, the neighbors are found in the noisy data as well as fitted to it, so the regressors themselves carry errors; this *errors-in-variables* effect shrinks the fitted slopes toward zero (attenuation) and is the true origin of the "slope bias" of local linear fits, which no weighting removes. And in the limit $\epsilon \to$ attractor size the local linear model becomes the global linear (AR) model of the record. Casdagli [22] turned this into a diagnostic, the *deterministic-versus-stochastic* (DVS) plot: out-of-sample error against neighborhood size, from the smallest usable r to $r = N$. A minimum at small r indicates low-dimensional deterministic dynamics, a minimum at large r indicates that a linear stochastic model does as well as any nonlinear one, and a minimum in between indicates nonlinear dynamics with noise. It is the natural first plot to make with any record, and Chapter 16 supplies the significance test for what it shows.

Small neighborhoods leave too few neighbors to estimate the $d(d+1)$ parameters of the affine map; large ones incur the curvature bias. Weighting the neighbors by distance softens the choice of ϵ. Basing our model on r nearest neighbors ($\boldsymbol{x}_{j(i)}$, $i = 1, \ldots, r$) of the point of interest $\boldsymbol{x}_n$, we can

find the optimal model parameters by minimizing

$$e_k^2 = \sum_{i=1}^{r} w_i \left\| \boldsymbol{x}_{j(i)+k} - \boldsymbol{A}_n^k \boldsymbol{x}_{j(i)} - \boldsymbol{b}_n^k \right\|^2 , \tag{15.9}$$

where w_i is a suitable weighting function that assigns greater weight to the points $\boldsymbol{x}_{j(i)}$ that are closest to the $\boldsymbol{x}_n$ within the needed neighborhood size. Throughout this book we use

$$w(\delta) = \frac{a^4}{\delta^4 + a^4} , \tag{15.10}$$

where δ is the distance from $\boldsymbol{x}_n$ and a is a length scale taken from the data itself, not from a formula: either the distance from $\boldsymbol{x}_n$ to its r-th nearest neighbor, which adapts the scale to the local density as in locally weighted regression, or, when one scale is to serve the whole record, the median nearest-neighbor distance of the data set. Both δ and a are lengths, so w is dimensionless and unchanged by a rescaling of the data; $w(0) = 1$, $w(a) = 1/2$, and $w \approx (a/\delta)^4$ beyond a. With the first choice the r-th neighbor receives half the weight of a coincident point, neighbors well inside the scale are weighted nearly alike, and those further away are suppressed as δ^{-4}. The routines of Appendix A (`loclin`, `lyapunov`, and the strand bundle of `sodfilt`) use the first choice, recomputing a for every neighborhood. The weight is written as a function of the distance δ; a weight given as a decreasing function of the neighbor's rank (the tricube kernel on $\delta/\delta_{(r)}$ of locally weighted regression is an example) is equally admissible and behaves similarly. Only a weight that is the same for every neighbor has no effect, since it then cancels from the normal equations below. If we define $\boldsymbol{B}_n^{(k)} = \left[\boldsymbol{A}_n^{(k)}, \boldsymbol{b}_n^{(k)}\right]$ and the following matrices

$$\boldsymbol{X}_n = \left(\boldsymbol{x}_{j(1)}, \boldsymbol{x}_{j(2)}, \ldots, \boldsymbol{x}_{j(r)}\right) , \qquad \boldsymbol{X}_{n+k} = \left(\boldsymbol{x}_{j(1)+k}, \ldots, \boldsymbol{x}_{j(r)+k}\right) , \tag{15.11}$$

$$\boldsymbol{W}_n = \mathrm{diag}(w_1, w_2, \ldots, w_r) , \tag{15.12}$$

$$\tilde{\boldsymbol{X}}_n = \begin{pmatrix} \boldsymbol{x}_{j(1)} & \boldsymbol{x}_{j(2)} & \cdots & \boldsymbol{x}_{j(r)} \\ 1 & 1 & \ldots & 1 \end{pmatrix} . \tag{15.13}$$

Then we can rewrite Eq. (15.7) as:

$$\boldsymbol{X}_{n+k} \approx \boldsymbol{B}_n^{(k)} \tilde{\boldsymbol{X}}_n . \tag{15.14}$$

If we use weighting and multiply the above from the right-hand side with $\boldsymbol{W}_n \tilde{\boldsymbol{X}}_n^{\mathrm{T}}$ we get:

$$\boldsymbol{X}_{n+k} \boldsymbol{W}_n \tilde{\boldsymbol{X}}_n^{\mathrm{T}} \approx \boldsymbol{B}_n^{(k)} \tilde{\boldsymbol{X}}_n \boldsymbol{W}_n \tilde{\boldsymbol{X}}_n^{\mathrm{T}} , \tag{15.15}$$

Now, it is easy to show that the optimal model parameters are given by

$$\boldsymbol{B}_n^{(k)} = \boldsymbol{X}_{n+k} \boldsymbol{W}_n \tilde{\boldsymbol{X}}_n^{\mathrm{T}} \left(\tilde{\boldsymbol{X}}_n \boldsymbol{W}_n \tilde{\boldsymbol{X}}_n^{\mathrm{T}} \right)^{-1} . \tag{15.16}$$

This closed form serves derivation only. Forming $\tilde{\boldsymbol{X}}_n \boldsymbol{W}_n \tilde{\boldsymbol{X}}_n^{\mathrm{T}}$ squares the condition number of the weighted design, and delay coordinates at unit delay are nearly collinear. In computation, $\boldsymbol{B}_n^{(k)\mathrm{T}}$ is the least-squares solution of $\boldsymbol{W}_n^{1/2} \tilde{\boldsymbol{X}}_n^{\mathrm{T}} \boldsymbol{B}_n^{(k)\mathrm{T}} \approx \boldsymbol{W}_n^{1/2} \boldsymbol{X}_{n+k}^{\mathrm{T}}$, obtained directly by QR or SVD; `loclin` does so.

A forecast is honest only if nothing it is scored against entered the model. The record is therefore split in time: the model is built on the first part and tested on the rest, with a gap of more than $(d-1)\tau$ samples between the last delay vector of the model and the first test vector, so that no sample of a test vector or of its forecast belongs to a delay vector of the model. When a test

point is drawn from the modeling set, as for an in-sample error, neighbors closer in time than a Theiler window (Section 12.5.4) are excluded, since consecutive points on one trajectory segment are close for that reason alone; `loclin` takes the window and the column index of the query as optional arguments. Two baselines calibrate the result: persistence, $\hat{x}_{n+(d-1)\tau+h} = x_{n+(d-1)\tau}$, and the global affine model, the limit $r \to N$ of the local one. The lead h is counted from the last sample of the starting vector, $x_{n+(d-1)\tau}$, not from its first.

An example of iterated local linear prediction is shown in Fig. 15.3 for a Chua's circuit signal, using delay six, embedding dimension three, and 16 nearest neighbors, with and without the weighting Eq. (15.10). The model uses the first 20,000 delay vectors, and every starting vector begins after the last sample they contain. The prediction is faithful for about ten time units and then departs from the record, as it must: averaged over 200 starting points, the root-mean-square error grows from 8.7×10^{-4} of the standard deviation of the record at a lead of one step to half of it at $t = 10.6$, about three and a half Lyapunov times, and then saturates near one, the error of predicting the mean. Over the range in which that growth is exponential it goes as $e^{\hat{\lambda} t}$ with $\hat{\lambda} = 0.42$, against the largest Lyapunov exponent $\lambda_1 = 0.333$ of the model; the excess is the model error of the local linear fit compounding on top of the dynamical divergence. The weighting changes the errors by at most 22 per cent up to $t = 10.6$. On the same starts, persistence has errors of 0.10 and 0.79 at leads of one and ten steps, and the global affine model 0.052 and 0.82, against 8.7×10^{-4} and 3.1×10^{-3} for the local linear model (`python/chk_predict_baselines.py`).

There could be two major problems when using local linear models for prediction: (1) we could have too few points to have sufficiently small neighborhoods for some points in the phase space, and (2) If the noise's magnitude exceeds the maximum allowable neighborhood size, we will fit just noise, not the deterministic component. In these cases, we may want to consider global nonlinear models using polynomial or radial basis functions. There is also a large body of literature utilizing neural networks for modeling chaotic behavior. The problem with the "black box" approaches, like neural networks, is that we lose any physical intuition about how and why they work. They may not be generalizable if we have small changes in the system's parameters or operating conditions.

Pitfall: what a local model can and cannot establish

A local linear model that forecasts well out of sample shows that the record is predictable over the horizon tested, at the noise level present. Forecast skill alone does not demonstrate determinism. The model must beat persistence and the global affine model on test data separated from the modeling data by the gap above, and the margin needs a significance test against surrogates (Chapter 16). The growth rate of the forecast error is not λ_1: model error compounds on the dynamical divergence, and in Fig. 15.3 the fitted rate is 0.42 against $\lambda_1 = 0.333$. More data lengthens the horizon only logarithmically, since it lowers the initial error and the error then grows exponentially; here the horizon is about three and a half Lyapunov times. The model is defined only where the record has been. A starting state off the sampled attractor, after a parameter change or on a transient, has no neighbors that describe it. And the coefficients of a local model carry no physical meaning, so the model neither identifies parameters nor extrapolates to other operating conditions.

15.4 Global, Operator, and Learned Models

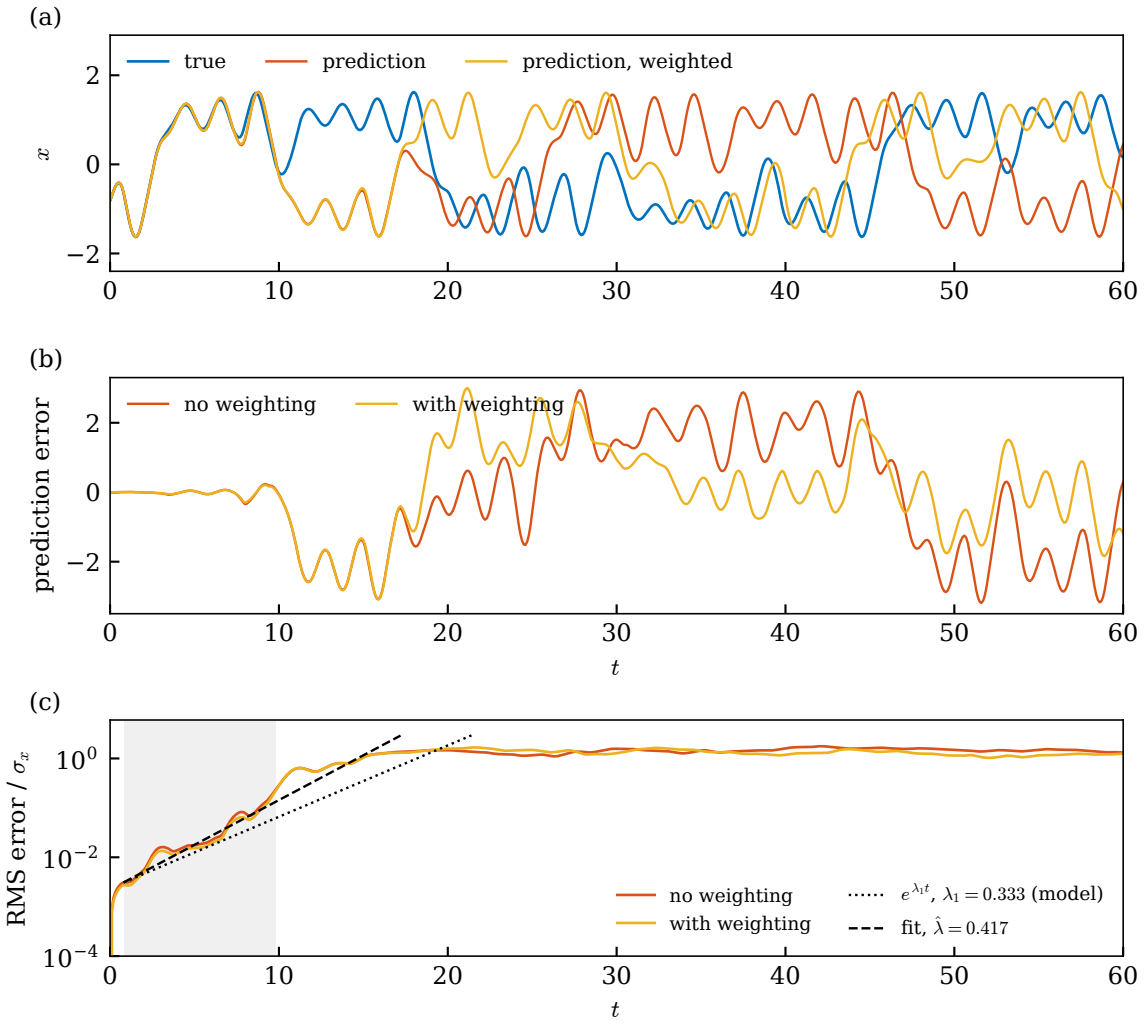


Figure 15.3: Iterated local linear prediction (`loclin`, Appendix A.7) of the Chua circuit x record of Eq. (1.6) with the parameters of Problem 10.1 ($\alpha = 9$, $\beta = 100/7$, $a = 8/7$, $b = 5/7$, from $(0, 0, 0.6)$, 200 time units of transient discarded), sampled at $\Delta t = 0.1$ and normalized to unit variance; 20,000 points are used for the model, $d = 3$, $\tau = 6$, $r = 16$ nearest neighbors, without and with the weighting Eq. (15.10); starting vectors begin after the last sample used by the model, and t is the lead beyond the last sample of the starting vector. (a) the true continuation and the two predictions over 600 steps from one starting vector; (b) their errors; (c) the root-mean-square error over 200 starting vectors against the lead, its exponential fit over the shaded range ($\hat{\lambda} = 0.417$) and the growth $e^{\lambda_1 t}$ at the exponent of the model. Script: `python/mkfig_Prediction.py`.

Local models are where this book stops. Several families of methods have grown beside them, and each answers a different question. They are placed here so the reader can locate them, not so they can be applied from this section.

Sparse regression. SINDy [16] fits $\dot{\boldsymbol{x}} = \boldsymbol{\Theta}(\boldsymbol{x})\boldsymbol{\xi}$ over a library $\boldsymbol{\Theta}$ of candidate functions and keeps the few terms a sparsity penalty allows. The result is a global equation with interpretable coefficients. It needs the right variables and their time derivatives. Delay coordinates of a scalar record are not the physical states, and a differentiated noisy record carries the difficulties of Chapter 14.

Operator methods. Dynamic mode decomposition [176] fits one linear map between successive columns of a data matrix. Its eigenvalues and modes approximate those of the Koopman operator, which acts linearly on observables of a nonlinear system [141, 169]; extended DMD enlarges the set of observables with a dictionary [223]. For a mixing chaotic attractor the Koopman spectrum has a continuous part, which a finite set of modes cannot represent. The discrete modes then describe the periodic and quasi-periodic content of the record, not its chaos.

Invariant-manifold reduction. Spectral submanifolds are invariant manifolds tangent to a slow spectral subspace of a fixed point [84]. Fitted to measured decays, they give low-order models, backbone curves and forced responses of mechanical structures [25]. They address the neighborhood of an equilibrium, not a chaotic attractor.

Input–output identification. Nonlinear system identification in structural dynamics is reviewed in [114, 147]. Most of its methods need a measured input and a model class; most of this book works from the response alone.

Learned models. Reservoir computers trained on a record reproduce its attractor and estimate its Lyapunov exponents [155]. Physics-informed neural networks [109, 161] put a known differential equation into the training loss.

Data assimilation. A model and noisy observations are combined sequentially, the model forecasting and each observation correcting the state. The ensemble Kalman filter [57] is the standard example; Law, Stuart and Zygalakis [125] give the mathematics, and Chatzi and Smyth [28] apply related filters to structural identification.

The division with this book is the model class. Every method above assumes one: a library, a dictionary, a manifold, a network, or a known equation. The methods of this book assume only a deterministic, finite-dimensional attractor behind the record. The two meet in the evaluation. A learned model needs a stationary, adequately sampled record (Part I), error bars that account for correlated samples, the out-of-sample protocol of the previous section, and a surrogate test of what it claims to have found (Chapter 16).

Problems

Prediction error is itself a useful discriminator: Sugihara and May [196] proposed using the way it grows with forecast horizon to separate chaos from measurement noise, a test whose statistical footing is the subject of Chapter 16.

Problem 15.1

Core

Use the same time series of Problem 11.2 and build a predictive local linear model from data. Split your data into two pieces, one to train the model and the other to test the prediction. What are the in-sample and out-of-sample RMS prediction errors? For the in-sample error, exclude neighbors within a Theiler window of each query (the optional arguments of `loclin`); how much does the in-sample error change without the exclusion?

Problem 15.2

Computational

Problem 15.1 built one local linear model. This problem compares the three predictors of this chapter on equal terms.

1. Implement all three: the method of analogues (nearest neighbor only), simple averaging over r neighbors, and the local linear model of Eq. (15.7).

2. Split a Lorenz record into training and test halves. For each method, plot the out-of-sample RMS error against prediction horizon h for $h = 1, \dots, 200$ samples.

3. The method of analogues reproduces a segment of the training trajectory, so for long horizons its prediction becomes periodic. Demonstrate this and identify the horizon at which it sets in.

4. For simple averaging, vary r. There is a bias–variance trade-off: small r is noisy, large r averages over a neighborhood too big to be linear. Locate the optimum and show it moves with the noise level.

5. Fit the error growth over the useful range to $e(h) \sim e_0 \mathrm{e}^{\lambda h}$ and compare the fitted λ with the largest Lyapunov exponent from Chapter 12. They should agree to within the scatter of the scaling-region choice — do they?

Problem 15.3

Computational

A model that fits the training data is not a model that predicts.

1. Using the local linear model, sweep the neighborhood size and the polynomial order (linear, quadratic, cubic, retaining only the linear part as recommended in Section 12.3). For each combination record both the in-sample and the out-of-sample RMS error, with a Theiler window around each in-sample query.

2. Plot the two against model complexity on the same axes. Locate the point at which in-sample error keeps falling while out-of-sample error begins to rise, and name what is happening.

3. For the *global* polynomial models (linear, quadratic, cubic in the d delay coordinates), compute Akaike's information criterion [5], $\mathrm{AIC} = N \ln(e_{\mathrm{in}}^2) + 2p$ with p the number of fitted coefficients and e_{in}^2 the in-sample mean squared error under a Gaussian residual model. Does it select the same model as the out-of-sample error does? Where they disagree, explain why. Explain also why the AIC cannot be applied as written to the local linear model: its effective number of parameters is not the $d + 1$ of one local fit but a quantity that grows as the neighborhood shrinks, and no likelihood was specified for it.

4. Repeat at 20 dB SNR. Does the optimal complexity increase or decrease with noise, and is that what you expected?

5. Finally, apply your best model to a record from a *different* initial condition on the same attractor. If the out-of-sample error is much worse, what does that say about what the model has actually learned?

Problem 15.4

Core

The weighted least-squares fit Eq. (15.15), and the one bias no weighting removes.

1. Show that $\boldsymbol{B}_n^{(k)} = \boldsymbol{X}_{n+k} \boldsymbol{W}_n \tilde{\boldsymbol{X}}_n^{\mathrm{T}} (\tilde{\boldsymbol{X}}_n \boldsymbol{W}_n \tilde{\boldsymbol{X}}_n^{\mathrm{T}})^{-1}$ minimizes $e_k^2 = \sum_i w_i \left\| \boldsymbol{x}_{j(i)+k} - \boldsymbol{B}_n^{(k)} \tilde{\boldsymbol{x}}_{j(i)} \right\|^2$ by setting the gradient with respect to $\boldsymbol{B}_n^{(k)}$ to zero. State the condition on the neighbors for $\tilde{\boldsymbol{X}}_n \boldsymbol{W}_n \tilde{\boldsymbol{X}}_n^{\mathrm{T}}$ to be invertible, and hence the smallest usable r for a d-dimensional reconstruction.

2. Show that for a globally linear map $\boldsymbol{x}_{n+1} = \boldsymbol{A}\boldsymbol{x}_n$ the fit returns $\boldsymbol{A}$ exactly from any $d+1$ neighbors in general position, whatever the weights. Confirm numerically with the AR(2) process of Chapter 3 written in companion form ($d = 2$, $\tau = 1$, no noise): three points suffice to recover a_1 and a_2 to machine precision, and $\boldsymbol{b}$ vanishes.

3. Now add measurement noise. Let $s_{n+1} = as_n + \eta_n$ with $a = 0.9$ be observed as $x_n = s_n + e_n$ with independent noise of variance σ_e^2. Show that the least-squares slope of x_{n+1} on x_n converges not to a but to $a\,\sigma_s^2/(\sigma_s^2 + \sigma_e^2)$, and verify with 10^5 samples for $\sigma_e = 0.5$, 1 and 2 (you should find slopes near 0.86, 0.75 and 0.50). This is the errors-in-variables attenuation of the remark on bias and variance.

4. The attenuation shrinks every fitted Jacobian toward zero. Which quantity of Chapter 12, estimated from local linear fits, is therefore biased, and in which direction? Explain why weighting the neighbors by distance, Eq. (15.10), cannot repair it, and name one thing that can (consider what the total-least-squares fit, or noise reduction before fitting, does to the regressors).

Problem 15.5

Computational

Casdagli's deterministic-versus-stochastic plot, described in the remark on bias and variance, on three records.

1. Implement the plot: split the reconstructed record into a modeling half and a test half, and for neighbor counts $r = 4, 8, 16, \ldots, 2048$ compute the one-step out-of-sample RMS error of the local linear model (`loclin` with $N = 1$) over a few hundred test points. If test points are drawn from the modeling half instead, pass `loclin` a Theiler window. Plot the error against r on logarithmic axes.

2. Apply it to the Lorenz x record ($\Delta t = 0.02$, $d = 3$, $\tau = 9$, 12,000 samples), to the same record with 10% additive noise, and to the AR(2) process with $\rho = 0.05$ ($d = 3$, $\tau = 1$), each normalized to unit variance. You should find the clean Lorenz error smallest at $r \approx 8$ and rising by two orders of magnitude toward $r = 2048$; the noisy Lorenz error flat and smallest at intermediate r of a hundred or so; and the AR(2) error falling monotonically to a floor at the largest r, where the local model has become the global AR model and the error has reached the innovation standard deviation, 0.1 in these units.

3. Explain each shape from the bias–variance argument: which term dominates at small r, which at large r, and why noise moves the minimum outward. Why does the AR(2) curve have no interior minimum at all?

4. Explain also why the error at $r = 4$ is very large for every record, from the rank condition of Problem 15.4, and what the DVS plot can and cannot say about a record whose minimum is at intermediate r.

Problem 15.6

Experimental

Predicting the two-well oscillator record of Section 1.4.1.

1. Embed the strain with the τ and d of Problems 9.6 and 10.6 and split the reconstruction into a modeling half and a test half. Build the DVS plot of Problem 15.5 for one-step prediction. Where is its minimum: at small r (deterministic), at large r (linear stochastic), or in between? What does the location say about the noise level of the record, and is it consistent with Problems 4.6 and 13.6?

2. At the best r, compute the out-of-sample RMS error of the iterated local linear predictor against horizon h, for h from one sample to two forcing periods, using the sampling rate given with the record to convert h to time. Normalize the error by the standard deviation of the record. At what horizon does it reach one half, and how many forcing periods is that?

3. Fit the growth $e(h) \sim e_0\, e^{\lambda h}$ over the range where it is exponential and convert λ to physical units. Compare with the largest Lyapunov exponent of the record from Chapter 12, and with the expectation of Section 1.4.1 that it is a fraction of the forcing frequency. Which of the two estimates of λ has the larger uncertainty, and why?

4. Prediction across a well crossing is the hard case. Separate the test points into those that stay in one well over the horizon and those that cross, and report the error for each group. Which group dominates the error at long horizons, and what does that say about where on the attractor the local linear approximation fails?

16 Surrogate Data Analysis

In practice, we usually want to classify or characterize our time series with few numbers or metrics (e.g., fractal dimension, Lyapunov exponent, model prediction error, or some entropy). These characteristic numbers can then be used to determine if the underlying dynamical system is operating as expected or needs attention. These metrics could also be used to track or follow some underlying hidden processes that may evolve on a slower time scale than the observable dynamics of machinery. Some examples include structural health monitoring, damage identification, monitoring and prediction; fault detection and localization; or medical diagnosis. In all these cases, we need to trust the estimated metrics: are they reflecting the underlying dynamics or just giving us some numbers as a result of algorithmic calculations that have no physical meaning?

The method developed in this chapter is due to Theiler *et al.* [208]; Schreiber and Schmitz [181] give the standard review, and their paper on constrained randomization [182] sets out the view that generalizes it: a surrogate is any record that satisfies the constraints defining the null hypothesis, and the Fourier-based recipes below are special cases.

We often do not have a strong indication or evidence that what we are estimating can be trusted or is a measure of some deterministic process. In these scenarios, we need to evaluate the probability that our measures reflect the physical reality by statistical hypothesis testing. However, due to the limited size of the experimental data, we usually are limited in the ability to estimate the probability distribution of our metrics using, for example, different subsets of our data. Furthermore, it is usually fairly difficult to develop a simple null hypothesis for testing the complicated metrics we are using here. Due to all of this, we usually resort to *surrogate data analysis*.

The basic idea of the surrogate data analysis is to compare our particular nonlinear metric to the distribution of the same metric generated from a large-synthetic ensemble of time series (i.e., surrogate data sets) that satisfy some null hypotheses. For example, suppose we test the presence of determinacy. In that case, this null hypothesis could be that the time series are generated by some linear stochastic process (i.e., autoregressive moving average or ARMA). Then, we can judge whether our metric could have arisen under the null hypothesis by observing if and where it falls within the distribution obtained from the surrogates. If the data's statistic falls in a tail of that distribution, so that the probability under the null hypothesis of a value at least as extreme is below a chosen level α, we reject the null hypothesis. If it does not, the test has found no evidence against the null hypothesis; that is not evidence for it.

The confidence level with which the null hypothesis can be rejected will depend on the number of surrogate signals used. We allow for a chance that the null hypothesis could be rejected even if it is true. For example, if we require a 90% confidence level, we accept a 10% chance that the null hypothesis could be falsely rejected. To quantify this, we assign a value of 0.10 (10 %) to the parameter α. The number of surrogate data sets required is then determined by α. Statistics used

to differentiate our metrics can be either single-sided or double-sided. Depending on which, the number of surrogates, β, will vary according to the following rules:

Single-sided Statistic: the direction of the departure is known in advance (the data statistic, if the null is false, can only be higher than, or only lower than, the surrogate values):

$$\beta = \frac{1}{\alpha} - 1$$

Double-sided Statistic: the direction is not known in advance, and a departure in either direction counts:

$$\beta = \frac{2}{\alpha} - 1$$

For this method to be effective, our surrogate data needs to have some properties that are identical or similar to our time series to establish the corresponding *null hypothesis*. Since we are mainly interested in testing for nonlinear metrics, we usually require that all or some of the linear properties of the test time series and surrogates be shared or identical. This requirement would support a powerful null hypothesis that the ARMA model generates the time series. Generally, we could consider these three null hypothesis or their combinations:

1. The data is composed of random numbers drawn from some fixed but unknown distribution.
2. The data is drawn from a stationary linear stochastic process with Gaussian inputs.
3. The data is a stationary linear Gaussian stochastic process observed through a static, invertible (monotone) measurement function [181, 208].

The third null is not "a linear process with unknown distribution": a linear process driven by non-Gaussian innovations has nonzero higher-order spectra that phase randomization destroys, and none of the surrogates below represent it. The phase-randomized surrogates test null hypothesis two, linear Gaussian; the amplitude-adjusted surrogates, iterated or not, test null hypothesis three, a static monotone transform of a linear Gaussian process. Each null hypothesis has a method of creating surrogate data sets associated with it. Each surrogate needs to have the same number of points as the original signal for the test to be valid. The three basic methods of surrogate data generation are:

Unconstrained randomization of time series: This is a very simple surrogate data set. We will simply reshuffle the time series in some random order.

Phase randomized surrogates: Here, we require that the surrogates have the same power spectrum as our data, which is equivalent of requiring to having identical linear correlations. Hence, if there are nonlinear correlations in our test data, we should be able to reject the hypothesis. This method will preserve the power spectrum amplitudes of the time series while randomizing its phase. In particular,

1. Taking the digital fast Fourier transform (DFFT) of our measured time series $\{x_n\}_{n=0}^{N-1}$:

$$X_n = \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} x_k e^{-\frac{i2\pi nk}{N}}, \tag{16.1}$$

(the sign convention and index range of Chapter 5; the symmetric $1/\sqrt{N}$ normalization is used throughout this chapter so that the forward and inverse transforms have the same prefactor and $\sum_n |X_n|^2 = \sum_n x_n^2$).

2. Replace the phases of X_n by random phases, $X'_n = |X_n|\, e^{i\phi_n}$, where ϕ_n are uniformly distributed in $[0, 2\pi)$ with the conjugate symmetry $\phi_{N-n} = -\phi_n$ enforced ($\phi_0 = 0$, and $\phi_{N/2} = 0$ for even N), so that the surrogate is real.

3. Compute an inverse digital fast Fourier transform (IDFFT) to get the surrogate:

$$x'_n = \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} X'_k e^{\frac{i2\pi nk}{N}} . \tag{16.2}$$

When the power of the record is spread over many frequencies, each surrogate value is a sum of many terms with independent random phases, and its amplitude distribution is approximately Gaussian whatever the data's was. The approximation is not exact, and it fails when a few spectral lines dominate: a phase-randomized single sinusoid is still a sinusoid, with the arcsine distribution and kurtosis 1.5 rather than the Gaussian 3. For the two-well record of Section 1.4.1 the approximation is good but not exact, kurtosis 3.04 for one surrogate against 1.73 for the data (`python/chk_surr_forcing.py`). The surrogate's circular autocorrelation equals that of the data at every lag; the DFT treats the record as one period of a periodic signal, so if x_0 and x_{N-1} differ appreciably (*end-point mismatch*) the surrogates inherit a spurious jump at the wrap-around, whose high-frequency power is spread through the record and is read by some statistics as nonlinearity. Trimming the record to a length over which the end points and end slopes nearly match [181] removes it.

Amplitude-adjusted surrogates (AAFT): To test null hypothesis three, Theiler *et al.* [208] first make the data Gaussian, then phase-randomize, then undo the Gaussianization by rank: (i) generate N Gaussian random numbers and sort them so that their ranks match the ranks of $\{x_n\}$, giving a Gaussian record $\{g_n\}$ with the time ordering of the data; (ii) phase-randomize $\{g_n\}$ as above to obtain $\{g'_n\}$; (iii) replace the values of $\{g'_n\}$ by the sorted data values in rank order, $x'_n = x_{(\text{rank } g'_n)}$. The surrogate has *exactly* the amplitude distribution of the data. Its power spectrum, however, only approximates that of the data, and the error is systematic: step (iii) is a nonlinear transformation of a Gaussian record, and a static nonlinearity moves power from the spectral peaks into the broadband floor, so AAFT surrogates have a *flatter* (whiter) spectrum than the data. The bias vanishes only as $N \to \infty$, and at finite N a statistic sensitive to the strength of the linear correlations, a prediction error for one, sees the surrogates as more random than the data and rejects a true null [180]. This is why the iterative scheme below exists.

Iteratively refined surrogates: This method uses an iterated amplitude adjusted Fourier transform [180] to obtain the needed surrogates. It alternates between two constraints, the periodogram of the data and its amplitude distribution, and at the end of the iteration one branch meets the first exactly and the other meets the second exactly; neither meets both. The surrogates remove the structure that a static monotone transform of a linear Gaussian process cannot carry, such as phase coupling between frequencies, and retain whatever the two constraints fix. They do not remove all nonlinear structure: whatever part of it is expressed in the periodogram or the histogram survives. The transforms below use the convention of the phase-randomized surrogates above.

1. Sort the original time series $\{x_n\}_{n=0}^{N-1}$ in ascending order into $\{y_n\}_{n=0}^{N-1}$, and store its

Fourier magnitudes

$$X_n = \frac{1}{\sqrt{N}} \left| \sum_{k=0}^{N-1} x_k e^{-\frac{i2\pi nk}{N}} \right| . \tag{16.3}$$

2. Initialize the iteration with a random-phase version of $\{x_n\}$. The routine `irsurr` of Appendix A draws an independent phase ϕ_k, uniform on $[0, 2\pi)$, for every bin, without imposing conjugate symmetry, and keeps the real part of the inverse transform:

$$r_n^{(0)} = \mathrm{Re} \left[\frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} X_k e^{i\phi_k} e^{\frac{i2\pi nk}{N}} \right] . \tag{16.4}$$

   Taking the real part loses about half the power and distorts the bin magnitudes, but this is immaterial: the first spectral step below keeps only the phases of $\{r_n^{(0)}\}$, and these are uniform and conjugate-symmetric. A random permutation of $\{x_n\}$, the start used by Schreiber and Schmitz [180], is equally acceptable.

3. For $i = 0, 1, 2, \ldots$, compute the DFT of $\{r_n^{(i)}\}_{n=0}^{N-1}$:

$$R_n^{(i)} = \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} r_k^{(i)} e^{-\frac{i2\pi nk}{N}} . \tag{16.5}$$

4. Replace the magnitudes $\left|R_n^{(i)}\right|$ by X_n, keep the phases $\psi_n^{(i)} = \arg R_n^{(i)}$, and invert (the spectral step):

$$x_n^{(i)} = \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} X_k e^{i\psi_k^{(i)}} e^{\frac{i2\pi nk}{N}} . \tag{16.6}$$

   The result is real because X_k and $\psi_k^{(i)}$ inherit the conjugate symmetry of the transform of a real series. Its periodogram equals the data's exactly.

5. Rank-order $\{x_n^{(i)}\}$ against the stored $\{y_n\}$ (the rank step):

$$r_n^{(i+1)} = y_{\mathrm{rank}\left(x_n^{(i)}\right)} . \tag{16.7}$$

   Its histogram equals the data's exactly. Return to step 3.

The iteration approaches a fixed point $r_n^{(i+1)} = r_n^{(i)}$, which is usually reached after a finite number of iterations. The two constraints cannot in general be met exactly at once. At any iteration the output of the rank step, $\{r_n^{(i+1)}\}$, has exactly the data's distribution and only approximately its periodogram; the output of the spectral step, $\{x_n^{(i)}\}$, has exactly the periodogram and only approximately the distribution. One stops on whichever branch is exact for the quantity the test statistic is most sensitive to, and reports the residual mismatch of the other. The routine of Appendix A returns both: `rb` is the histogram-exact branch and `sb` the spectrum-exact branch of the last iteration. At least one iteration is needed. Called with zero iterations, `irsurr` returns the starting series Eq. (16.4), which meets neither constraint; on the first 24,000 samples of the two-well record its histogram is not the data's and its largest bin-magnitude error is 0.63 of the largest data magnitude (`python/chk_surr_forcing.py`).

Pitfall: rejecting for the wrong reason

A surrogate test rejects the whole null hypothesis, including its unstated parts, and the commonest false rejection has nothing to do with nonlinearity. Every generator above assumes *stationarity*: the spectrum and distribution are treated as fixed over the record. A record with a drifting mean, variance, or spectrum has surrogates that average the drift away, and a statistic that sees the drift (a prediction error, a dimension) rejects. Timmer [214] showed that the standard tests reject linear but nonstationary records at rates far above α, and the remedy is not a better statistic but a check of stationarity (Chapter 2) before the test. Two further generators are useful when the Fourier-based ones fail. *Cyclic-shift* (time-shift) surrogates rotate the record by a random offset with wrap-around; they preserve everything about the record except its alignment with a second channel, and are the natural null for coupling and information-transfer statistics between two records (Chapter 17). *Twin surrogates* [210] are built from the recurrence structure of the reconstructed trajectory (Chapter 18): points with identical recurrence columns are "twins" and the surrogate trajectory jumps between twins at random, so it preserves the deterministic dynamics and tests a hypothesis about coupling or synchronization rather than about linearity.

16.1 Examples

To investigate the usefulness and abilities of this method, we will look at two separate time series. A simple linear stochastic process generates the first time series, and the second will come from a simulation of the Lorenz attractor.

16.1.1 Linear Stochastic System

A surrogate test rejects a *specific* null hypothesis, and choosing it badly is the commonest way to reach a wrong conclusion. Kugiumtzis [120] makes the point sharply: the surrogates themselves should be checked before any nonlinearity statistic is applied to them, since a measurement transform that the algorithm fails to reproduce will be read as nonlinearity in the underlying dynamics [121, 122]. The linear stochastic process can be described by the following map:

$$x_n = 0.99x_{n-1} + \eta_n \,, \tag{16.8}$$

where η_n are independent random numbers. To make this problem more interesting for our application, we will observe the process through a nonlinear function,

$$s_n = x_n^3 \,. \tag{16.9}$$

It should also be noted that the mean of the signal is removed prior to the nonlinear transformation. The process along with the nonlinear observation can be seen in Fig. 16.1(a,b). It is entirely possible to see a signal like this in real life. We often do not know the underlying governing equations and physical processes, especially in biological systems. When we take some measurement of that system, there could easily be a nonlinear rescaling in the process. In this case, it would be inappropriate to analyze the time series using nonlinear time series analysis techniques. For the sake of this problem, we are going to assume we know nothing about the underlying process and only see the nonlinear observation.

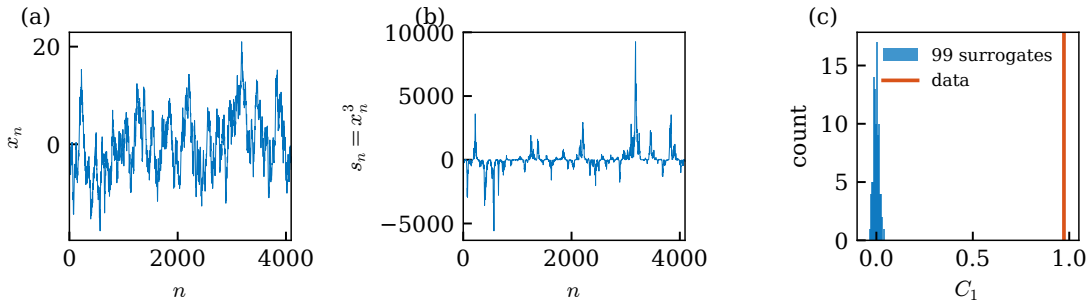


Figure 16.1: The linear stochastic process and the first surrogate test. (a) $N = 4096$ points of $x_n = 0.99x_{n-1} + \eta_n$ with η_n standard Gaussian, the first 2000 points discarded as transient and the mean removed; (b) the nonlinear observation $s_n = x_n^3$; (c) the lag-one sample autocorrelation C_1 of s_n (vertical line) against 99 shuffle surrogates (histogram). The data value 0.9726 lies far above the surrogate range $[-0.035, 0.044]$, so null hypothesis one is rejected. Generated by `python/mkfig_e15_1.py`.

For this task, it is appropriate to start with a test of the null hypothesis number one. Namely, we will test if a random process generates the data. The statistic chosen to evaluate the test is the autocorrelation at lag one. The surrogates generation method is unconstrained randomization of the time series.

Our objective is to pass every test in this chapter with a 99 % confidence level. This requires that $\alpha = 0.01$, and therefore, $\beta = \frac{1}{0.01} - 1 = 99$ surrogates for the single-sided tests and $\beta = \frac{2}{0.01} - 1 = 199$ surrogates for the double-sided tests. The autocorrelation is expected to be much lower for a random signal than a linearly-correlated stochastic signal. Therefore, this test will be single-sided, requiring 99 surrogates. Fig. 16.1(c) shows the result of this test: the horizontal axis is the value of C_1, the histogram is the surrogate ensemble, and the vertical line is the data. The autocorrelation of the data is $C_1 = 0.9726$, while the 99 shuffled surrogates span $[-0.035, 0.044]$, with a standard deviation of 0.015 against the $1/\sqrt{N} = 0.016$ expected for a shuffle of $N = 4096$ points. The data is the largest of the 100 values, rank 100 of 100. Therefore, the null hypothesis is rejected with a 99 % confidence level, and we can declare that the observation is not a random distribution of numbers.

This test does not tell us about the nonlinearity of the system. Therefore, the next null hypothesis will test if the time series is generated by a stationary linear stochastic process with Gaussian inputs (null hypothesis two). The corresponding surrogates are generated by phase randomization, which preserves the periodogram of the record exactly — bin by bin to 3×10^{-16} of the largest bin, as Fig. 16.2(a) shows. Suppose the autocorrelation at lag one is retained as the statistic, with 99 surrogates.

The autocorrelation at lag one is then computed for the original time series and 99 of its surrogates and plotted in Fig. 16.2(b). The data value, 0.972598, lies inside the surrogate range $[0.970907, 0.972623]$, at rank 89 of 100, and the null hypothesis is not rejected. This outcome carries no information, and the reason is structural rather than a matter of the statistic being "not powerful enough." By the Wiener–Khinchin theorem of Chapter 4 the autocorrelation is the inverse Fourier transform of the power spectrum, so a surrogate that preserves the periodogram preserves the circular autocorrelation at every lag by construction; here every surrogate returns the data's circular

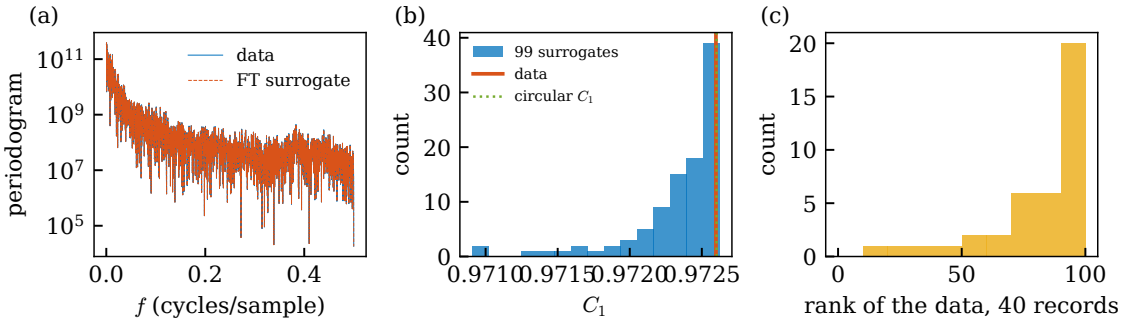


Figure 16.2: Phase-randomized (FT) surrogates of the record of Fig. 16.1. (a) Periodogram of the data and of one surrogate; they coincide to machine precision. (b) The ordinary sample autocorrelation C_1 of the data (solid line) against 99 FT surrogates, with the circular autocorrelation (dotted line), which every surrogate reproduces to 8×10^{-16}. (c) Rank of the data's ordinary C_1 among its 99 surrogates over 40 independent records of the same process: the wrap-around term pushes the data toward the top of the ensemble, but the statistic carries no information about nonlinearity. Generated by `python/mkfig_e15_3.py`.

$C_1 = 0.97260211$ to eight decimals, and the whole spread visible in Fig. 16.2(b) is the wrap-around term by which the ordinary sample autocorrelation differs from the circular one. That term is not centred: it is $x_1 x_N$ divided by the sample energy, and for a record that starts and ends at typical values it is more often positive for the data than for a surrogate whose end points have been randomized. Over 40 independent records the data's rank among its 99 surrogates averaged 80 of 100 and reached 100 in six cases (Fig. 16.2(c)), so this "statistic" would spuriously reject at the 99% level in about 15% of records — a false rejection produced entirely by end-point mismatch, not by dynamics. A statistic that is a function of the quantities the surrogates preserve cannot separate the data from the surrogates whatever the dynamics are, and a test built on it has no power: what little separation it shows is an artifact. Against phase-randomized surrogates one must use a statistic that is not determined by the power spectrum — the time-reversal asymmetry, a nonlinear prediction error, or the correlation dimension.

The test is repeated in Fig. 16.3 with two such statistics on the same record ($N = 4096$, $a = 0.99$, $s_n = x_n^3$; `python/mkfig_surr_trev.py`). The first is the normalized time-reversal asymmetry

$$T_{\mathrm{rev}} = \frac{\langle (s_{n+1} - s_n)^3 \rangle}{\langle (s_{n+1} - s_n)^2 \rangle^{3/2}} \,, \tag{16.10}$$

which vanishes in expectation for any time-reversible process, linear Gaussian processes and their static transforms included, and is nonzero for most nonlinear dynamics. The second is the excess kurtosis γ_2 of Chapter 2, which the phase-randomized surrogates do not preserve. With 99 phase-randomized surrogates the data give $T_{\mathrm{rev}} = -0.36$ against a surrogate range of $[-0.08, 0.11]$ (rank 1 of 100), and $\gamma_2 = 26.9$ against $[-0.6, 1.1]$ (rank 100 of 100): both reject null hypothesis two at the 99% level, Fig. 16.3(a,b). The rejection is correct, since s_n is not Gaussian, but the two statistics reject for different reasons. The kurtosis sees the heavy tails of the cubed record directly. The expectation of T_{rev} is zero for the data as well as for the surrogates; what differs is its sampling variance, which the heavy tails inflate by an order of magnitude (± 0.036 for the Gaussian surrogates), so a single realization of the data falls outside the surrogate range most of the time. Over 100 independent

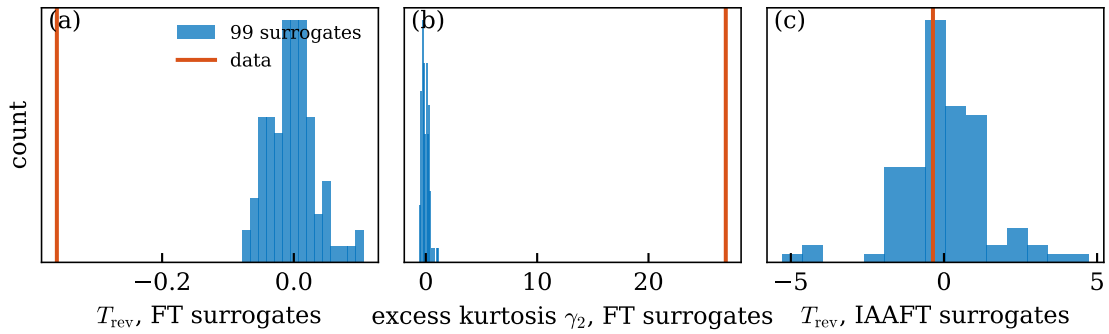


Figure 16.3: Surrogate tests on the record $s_n = x_n^3$ of Fig. 16.1 with statistics that the surrogates do not preserve; 99 surrogates in each panel, the data value as a vertical line. (a) Time-reversal asymmetry Eq. (16.10) against phase-randomized (FT) surrogates: the data lie outside the surrogate range and null hypothesis two is rejected. (b) Excess kurtosis against the same surrogates: rejected. (c) Time-reversal asymmetry against iteratively refined (IAAFT) surrogates, which reproduce the data's histogram exactly: the data fall at rank 39 of 100 and null hypothesis three is not rejected. The surrogate spread in (c) is forty times that in (a) because the heavy-tailed amplitudes of the record are now present in the surrogates too.

records of this process the two-sided test rejected in 78% of cases. That is the trap of Problem 16.4 in numbers: a static nonlinearity in the measurement is enough to reject the linear-Gaussian null with a statistic designed to detect nonlinear dynamics, and nothing in the rejection says which of the two is present.

Finally, we will test null hypothesis three, that the data is a linear Gaussian stochastic process observed through a static monotone measurement function. For this test, we will be using the error of one-time-step-ahead prediction as our statistic. This test is double-sided (as the prediction error could be higher or lower depending on the process) and requires 199 surrogates. The surrogate sets will be generated using the iteratively refined (or polished) randomization scheme. The histogram-exact branch is used: each surrogate has exactly the data's amplitude distribution, and its power spectrum matches the data's only approximately; the residual is reported below. Fig. 16.4(a) shows the Welch spectra of the signal and one surrogate. The two match fairly well together. The scheme cannot match both constraints exactly, and a residual spectral mismatch between data and surrogates is itself a difference that a prediction-error statistic can detect, so the surrogates must be iterated to convergence and the residual reported. For this record the ensemble-mean Welch spectrum of 199 histogram-exact surrogates and that of the data differ by a factor of at most 1.64 in any 256-point bin after 60 iterations, and by at most 1.61 after 200; the median bin factors are 1.13 and 1.12. The iteration has therefore converged and the remaining mismatch is the price of the exact histogram. The lag-one autocorrelation is 0.9726 for the data and 0.9666 ± 0.0033 over the surrogates. The cost is negligible (199 surrogates of $N = 4096$ points at 60 iterations take a few seconds), so there is no reason to stop early. With T_{rev} as the statistic, Fig. 16.3(c), the data fall at rank 39 of 100 and the null is not rejected. For predictive modeling, both the original time series and surrogates were embedded in three-dimensional space with unit delay; the statistic is the RMS one-step-ahead error of a local linear model on the eight nearest neighbors of each delay vector, neighbors within 20 samples in time excluded, divided by the standard deviation of the record. Fig. 16.4(b) shows

that the null hypothesis is not rejected at the 99% level here either: the data's prediction error is 0.4995 against a surrogate range $[0.293, 1.708]$ with mean 0.486, rank 139 of 200, whereas rejection would require rank 1 or 200. The test provides no evidence against the hypothesis that the data are a static monotone transform of a linear Gaussian process; it does not establish that hypothesis, and here we happen to know that it is true. A sequence of tests of this kind, from the weakest null to the strongest, is the correct way to read the evidence: the record is not white noise, is not linear Gaussian, and is consistent with a nonlinearly observed linear Gaussian process.

16.1.2 Lorenz Time Series Example

Two extensions are worth knowing before this example. For several simultaneously measured variables the phases must be randomized consistently across channels, as Prichard and Theiler [159] show, or the cross-spectrum is destroyed along with the nonlinearity. And for data with a strong cycle the linear null is a poor one; pseudoperiodic surrogates [187, 190] preserve the cycle while randomizing what rides on it. Small and Tse [188] survey the practical choices.

The record is the $x(t)$ coordinate of the Lorenz system ($\sigma = 10$, $\rho = 28$, $\beta = 8/3$), sampled at $\Delta t = 0.03$ after the transient is discarded, $N = 4096$ points, with Gaussian measurement noise of standard deviation 5 % of the record's RMS added: $s_n = x_n + \eta_n$, to make the problem more realistic. The surrogates are generated using the refined iterative randomization method, 60 iterations, taking the branch that reproduces the histogram exactly. Fig. 16.5 shows the results of the randomization: the surrogate has the same values as the record, in an order that destroys the alternation between the two lobes of the attractor. The preservation characteristics can be seen in Fig. 16.6, showing the histogram and the Welch spectrum of the data and one surrogate: the histograms are identical by construction and the spectra differ by at most a factor 1.7 in any 256-point bin, with a median factor of 1.2, while the lag-one autocorrelations are 0.9845 and 0.9844. All signals are embedded with $d = 3$ and $\tau = 6$ samples, the first minimum of the average mutual information of Chapter 9 for this record, and the prediction algorithm of Fig. 16.4 is run identically on the data and on every

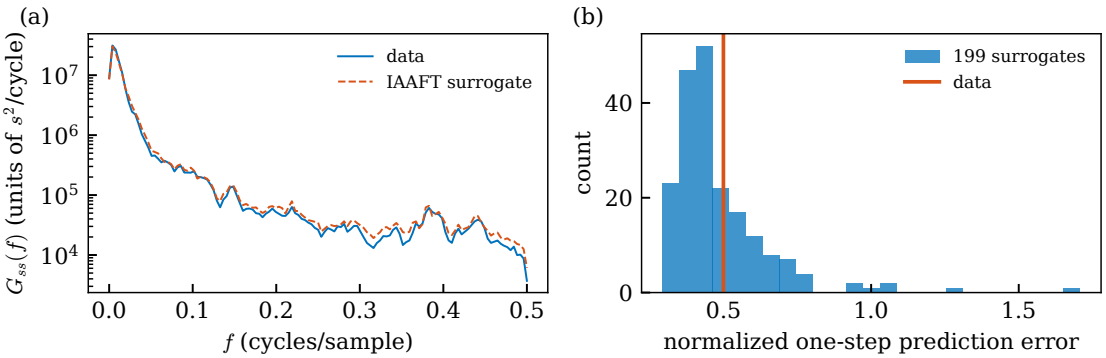


Figure 16.4: Iteratively refined (IAAFT, 60 iterations) surrogates of the record of Fig. 16.1. (a) Welch spectra (256-point Hann segments, 50% overlap) of the data and of one histogram-exact surrogate. (b) Normalized one-step-ahead local-linear prediction error ($d = 3$, $\tau = 1$, eight neighbors, Theiler window 20) of the data (vertical line) against 199 surrogates: the data fall at rank 139 of 200 and the double-sided test does not reject null hypothesis three. Generated by `python/mkfig_e15_5.py`.

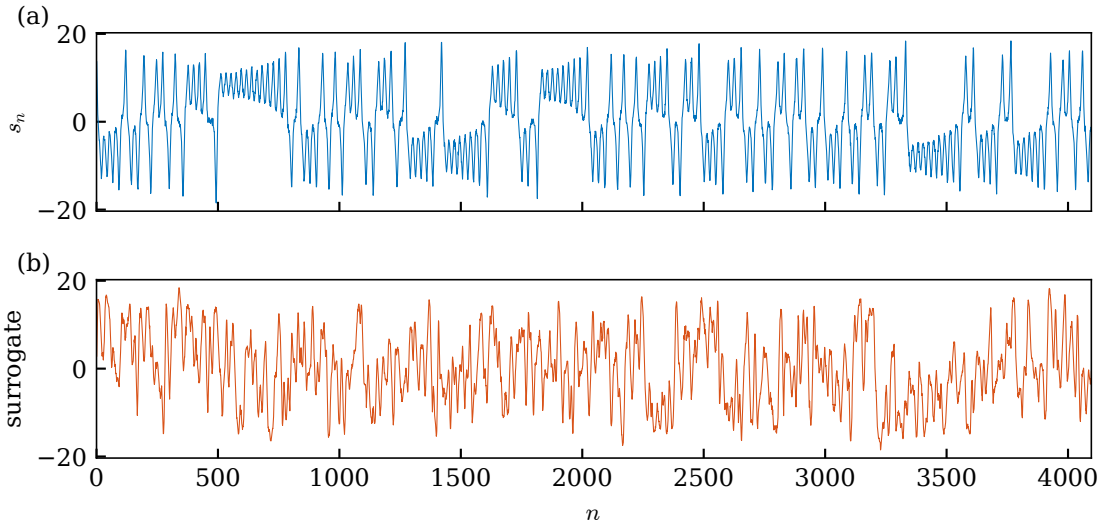


Figure 16.5: (a) The Lorenz observation $s_n = x_n + \eta_n$ ($\sigma = 10$, $\rho = 28$, $\beta = 8/3$, $\Delta t = 0.03$, $N = 4096$, Gaussian noise at 5 % of the record's RMS) and (b) one iteratively refined surrogate of it (60 iterations, histogram exact). Generated by `python/mkfig_e15_7.py`.

surrogate at a horizon of one step. Fig. 16.7 shows the results of the statistical test. The data's normalized prediction error is 0.091, against a surrogate range of $[0.173, 0.200]$ with mean 0.182: the data fall at rank 1 of 200, outside the distribution of the 199 surrogate values, so null hypothesis three is rejected at the 99% level and the record is not consistent with a static monotone transform of a linear Gaussian process. Note what has and has not been shown. Rejection identifies nonlinearity in the record beyond a static measurement nonlinearity; it does not by itself establish determinism or chaos, since a nonlinear stochastic process would also be rejected, and the stationarity caveat above applies.

16.1.3 The Two-Well Oscillator Record

The two simulated records above have known answers. The measured record of Section 1.4.1 does not, and the surrogate test is the only entry in Table 1.2 that is a decision rather than an estimate. It is worked here because the decision is closer than the table can show.

The null is hypothesis three: a static monotone measurement function applied to a stationary linear Gaussian process. The prerequisite is stationarity, and without a check of it the Timmer caution above would leave any rejection unreadable. Section 1.4.1 supplies part of that check. Over ten blocks of 600 forcing periods the block means lie between -0.222 and -0.143 V and the block standard deviations agree within 2.5%, so the record has no drift in level or in overall variance at the scale of a block. That is all the check shows. It tests two moments, and it cannot see structure tied to the phase of the drive. The record is driven, and at that scale it is cyclostationary rather than stationary. Its synchronous average over the 16 samples of a forcing period carries 4.4% of the variance, and the standard deviation of the residual about it varies from 1.44 to 1.67 V with forcing phase, a ratio of 1.15 (`python/chk_surr_forcing.py`). The null below is therefore a stationary null applied to a cyclostationary record, and the forcing-aware check that follows the main test is part

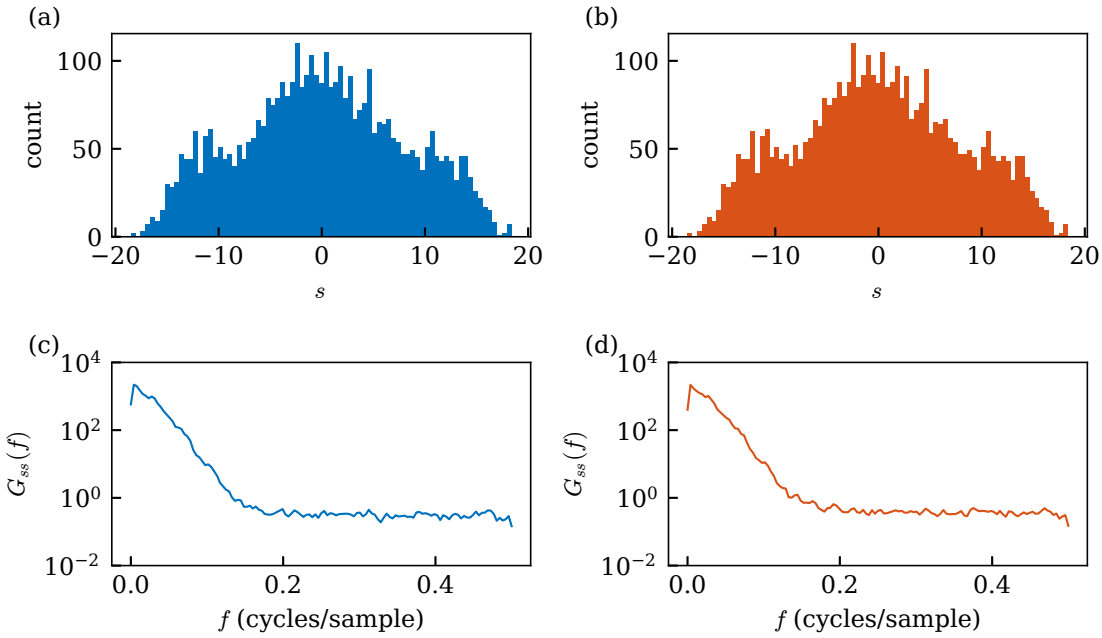


Figure 16.6: Histogram (a, b) and Welch spectrum (c, d) of the Lorenz record of Fig. 16.5 (left column) and of one iteratively refined surrogate (right column); 256-point Hann segments, 50 % overlap. Generated by `python/mkfig_e15_8.py`.

of the test.

The statistic and its tail are fixed before the data are examined. The statistic is $|T_{\mathrm{rev}}|$, the absolute value of the time-reversal asymmetry Eq. (16.10) with the one-step difference $s_{n+1} - s_n$ replaced by $s_{n+5} - s_n$, lag 5 being the delay the record's average mutual information selects. The absolute value is used for two reasons. Nonlinear dynamics predicts a departure from reversibility, not its sign. And the sign is not a property of the oscillator: $T_{\mathrm{rev}}(-s) = -T_{\mathrm{rev}}(s)$ exactly, and the polarity of a strain-gauge voltage is a sensor convention. A signed one-sided test would reject for one wiring of the bridge and not for the other. The statistic is not a function of the spectrum or of the amplitude distribution, so the surrogates do not preserve it. The surrogates are 39 iteratively refined records, 20 iterations, taking the histogram-exact branch, generated with `irsurr`. Large values of $|T_{\mathrm{rev}}|$ count against the null, so the rank test is one-sided in $|T_{\mathrm{rev}}|$: the null is rejected when the data value exceeds all 39 surrogate values, and 39 surrogates give $\alpha = 1/40 = 0.025$, the smallest level this ensemble can deliver.

The surrogates must be checked before the statistic is read off them. Every one reproduces the record's amplitude distribution exactly. The ensemble-mean Welch spectrum matches the data's to a median bin factor of 1.08 below $f_s/20$, where 92 per cent of the power sits, and of 1.17 between $f_s/20$ and $f_s/5$. Above $f_s/5$ the match fails badly, by three orders of magnitude in the worst bin, because the surrogates do not reproduce the roll-off of the anti-aliasing filter; that band carries a 10^{-9} fraction of the record's power, and a statistic sensitive to it would be testing the acquisition chain rather than the dynamics. Iterating further changes none of this: at 60 and 200 iterations the median bin factors and the test result are the same to the digits quoted.

Fig. 16.8(a) is the test on the first 24,000 samples, 1500 forcing periods. The data give $T_{\mathrm{rev}} =$

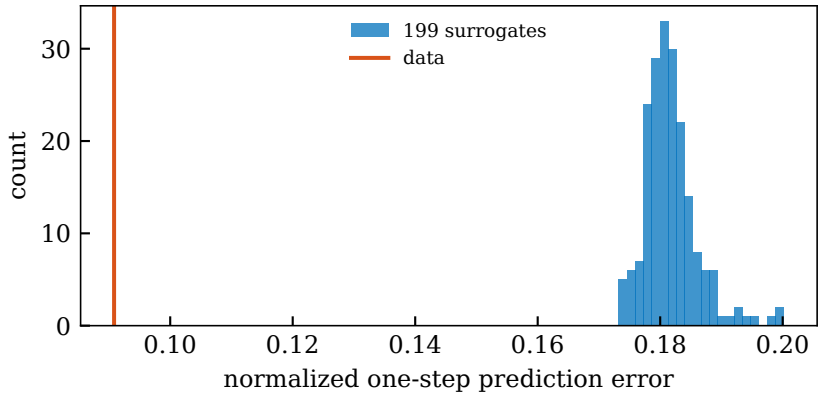


Figure 16.7: Normalized one-step-ahead local-linear prediction error ($d = 3$, $\tau = 6$, eight neighbors, Theiler window 20) of the Lorenz record of Fig. 16.5 (vertical line) against 199 iteratively refined surrogates. The data fall at rank 1 of 200 and null hypothesis three is rejected. Generated by `python/mkfig_e15_9.py`.

0.0697. The 39 surrogates have mean 0.004, standard deviation 0.025 and largest absolute value 0.062. The data value has rank 40 of 40 in $|T_{\mathrm{rev}}|$, so the null is rejected at $p = 0.025$, and T_{rev} lies $z = 2.7$ standard deviations above the surrogate mean. The signed statistic gives the same rank, 40 of 40, on this record and on the whole record below; the choice of $|T_{\mathrm{rev}}|$ changes the definition of the test, not its outcome here. Reversing the polarity of the record leaves the $|T_{\mathrm{rev}}|$ ranks at 40 of 40, whereas the signed upper-tail rank would fall to 1 of 40 (`python/chk_surr_forcing.py`).

That is a rejection, and it is thin. Three things weaken it. The margin is one rank: had a single surrogate landed 0.008 higher, the test would not have rejected. The statistic is noisy — the surrogate standard deviation, 0.025, is more than a third of the data value — and repeating the whole test with independent surrogate ensembles moves the summary numbers about. Over five ensembles, the one above and four more, the surrogate mean runs from -0.002 to 0.010, the standard deviation from 0.021 to 0.028 and z from 2.2 to 3.0. The data rank 40 of 40 in $|T_{\mathrm{rev}}|$ in four ensembles and 39 of 40 in the fifth, where one surrogate reaches $T_{\mathrm{rev}} = -0.078$; the signed rank is 40 of 40 in all five (`python/chk_surr_forcing.py`). At 24,000 samples the decision itself depends on the ensemble. And it is a single test with a single statistic, so nothing in it controls for the possibility that the one statistic chosen is the one that happened to separate.

Fig. 16.8(b) is the first of the three things that would strengthen it, and it is free. Run on the whole 96,000-sample record, the data value is unchanged at 0.0695, while the surrogate standard deviation falls to 0.014 — as $1/\sqrt{N}$ predicts for a third-moment statistic — and z rises to 5.0. The rank cannot improve past 40 of 40, but the separation is no longer marginal. The second is more surrogates: 199 instead of 39 would allow $p = 0.005$ and cost about a minute. The third is a second statistic of a different kind, a prediction error or a correlation dimension, computed against the same ensemble; agreement between two statistics that fail in different ways is worth more than a smaller p from one of them. The caution below is why the second statistic must be chosen carefully.

A forcing-aware null addresses the cyclostationarity found above. It keeps the phase-locked component of the record and randomizes only what rides on it: each surrogate is the synchronous average of the record plus an iteratively refined surrogate (20 iterations, histogram-exact branch) of

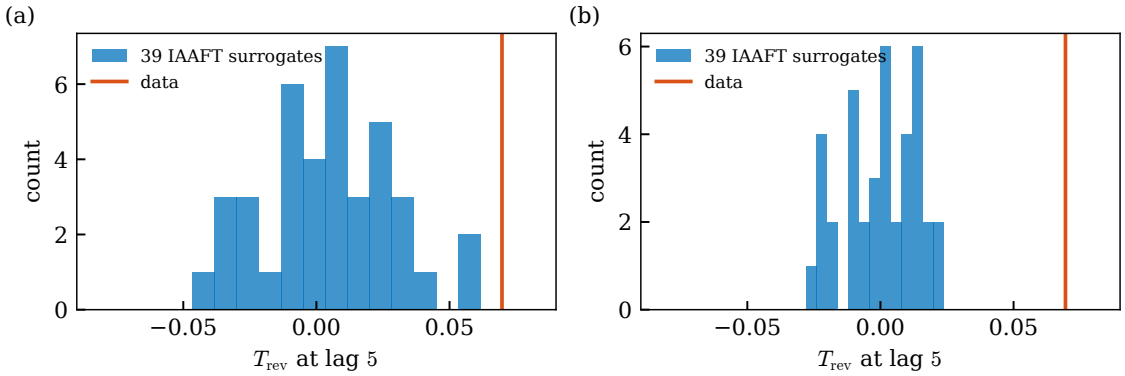


Figure 16.8: Surrogate test of null hypothesis three on the two-well oscillator record of Section 1.4.1. The time-reversal asymmetry Eq. (16.10) at lag 5 samples is plotted; the test statistic is its absolute value, and ranks are of $|T_{\mathrm{rev}}|$. The surrogates are 39 iteratively refined records (20 iterations, histogram-exact branch, `irsurr`), and the data value is the vertical line. (a) First 24,000 samples: data 0.0697, surrogates 0.004 ± 0.025, rank 40 of 40, $p = 0.025$, $z = 2.7$. (b) Whole 96,000-sample record: data 0.0695, surrogates 0.000 ± 0.014, rank 40 of 40, $z = 5.0$. Generated by `python/mkfig_surr_twowell.py`.

the residual about that average. This null preserves the forcing phase and the periodic waveform, but not the phase dependence of the residual variance, so it is a partial correction rather than an exact cyclostationary null. On the whole record the data again rank 40 of 40 in $|T_{\mathrm{rev}}|$ in each of two independent ensembles, with $z = 3.9$ and 4.7. On the first 24,000 samples the rank is 39 of 40 ($z = 2.1$) in one ensemble and 40 of 40 ($z = 2.3$) in the other (`python/chk_surr_forcing.py`). The phase-locked component does not produce the asymmetry: alone it gives $T_{\mathrm{rev}} = -0.023$, of opposite sign to the record's $+0.070$. The whole-record rejection therefore survives the forcing-aware null, and the short-record result is borderline under both nulls, which is a further reason to lead with Fig. 16.8(b). Rejection of either null is evidence against a linear stochastic description of the record, and it is not proof of chaos. A nonlinear stochastic process would also be rejected. The support for chaos comes from the estimates of Table 1.2, with this test as one piece of external evidence for them.

Finally, what a failure to reject would have meant. It would not have meant that the record is linear. With 39 surrogates and a statistic whose surrogate spread is a third of the data value, the test has little power, and the commonest reason for a non-rejection is that the statistic is insensitive to the nonlinearity present rather than that none is. A non-rejection would have left the correlation dimension and the Lyapunov exponent of Table 1.2 without their one piece of external support, and the correct report would have been that the linear null was not excluded at $p = 0.025$ by this statistic, not that the record is linear.

Pitfall: a robust statistic against IAAFT surrogates

Replacing the root-mean-square prediction error of Fig. 16.4 by the robust one, the median absolute error divided by the median absolute deviation of the record, makes the test of null hypothesis three reject when the null is true. On 20 independent records of the linear Gaussian process $x_{n+1} = 0.99x_n + \xi_n$, $N = 4096$, with 39 IAAFT surrogates each and the predictor of Fig. 16.4, the root-mean-square statistic rejected none at $\alpha = 0.05$ two-sided, mean $z = -0.63$. The robust statistic rejected 10 of 20, mean $z = +1.59$, with the data above the ensemble in almost every record. The same robust statistic against phase-randomized surrogates, which are exact for this null, rejected 2 of 20, mean $z = +0.70$. The statistic is not at fault by itself and the generator is not at fault by itself; the pair is.

The mechanism is a difference in the shape of the prediction-error distribution, not in its scale. The IAAFT surrogates of a smooth record carry a small number of samples that the local linear model cannot follow, and their prediction errors are far more heavy-tailed than the data's: sample kurtosis about 13 against 4 on the Gaussian record above, and 150 against 88 on the cubed record of Section 16.1.1. Phase-randomized surrogates of the same records give 5.9 against 4.0. The root-mean-square statistic is inflated by those tails, in its value and in its sampling variance together, and the two effects nearly cancel. The robust statistic discards the tails and reads the remaining difference in the bulk against a null spread from which the artifact's variance has been removed, which is why a moderate systematic offset becomes a large z. The two statistics put the data on *opposite* sides of the same ensemble, and that sign disagreement is itself the evidence that at least one of them is measuring the generator rather than the dynamics. Iterating the surrogates further does not help: on the cubed record the robust z is -4.3, -6.5 and -5.9 at 20, 60 and 200 iterations. The step that does it can be isolated by measuring the prediction error after each half of the iteration rather than at the end (`python/chk_iaaft_step.py`). On the cubed record the phase-randomized starting series has error kurtosis 5.1, and the first spectral step leaves it at 4.6: acting on a series that has never been rank-ordered, the spectral step does nothing. The first rank-ordering takes it to 32.7, and further iterations compound it to 47, 76 and 530. The earlier reading, that both branches carry the excess and so the rank step is innocent, mistook inheritance for cause: the spectrum-exact branch of iteration k is computed from the rank-ordered series of iteration $k-1$. Where the damage falls is what makes a robust statistic reject. Sorting the samples by $|s_n|$ into quintiles and averaging the squared prediction error within each, the data give 0.0011, 0.0047, 0.0137, 0.044, 1.19: it is predictable everywhere except at its own extremes, which is what cubing an AR(1) record produces. A twenty-iteration IAAFT surrogate gives 0.018, 0.014, 0.022, 0.046, 2.83. The two agree to within a factor of two in the top quintile and differ by a factor of three to sixteen in the bottom three. The rank step assigns the data's sorted amplitudes to positions ordered by a nearly Gaussian $\bar{s}$, and the mismatch between a heavy-tailed set of values and a light-tailed ordering scatters them: the surrogate is damaged in its bulk, and matched in its tail. A root-mean-square statistic is dominated by the tail and sees nothing. A statistic that discards the tail reads the bulk, where the surrogate is several times less predictable than the data, and rejects. The phase-randomized surrogate, which never rank-orders, has a flat profile across all five quintiles (0.073 to 0.094) and kurtosis 3.3.

The practical rule: a statistic that discards the tail of an error distribution must be checked against surrogates on a record where the null is known to hold, before it is used on one where it is not.

Problems

The problems below build the surrogate machinery from scratch, then use it twice: once on data where the null hypothesis is true and should not be rejected, and once to settle a question left open in Chapter 12. Problem 16.4 is the one worth doing even if the others are skipped — it exhibits the failure mode that makes surrogate testing hard to use correctly.

Problem 16.1

Core

Implement two surrogate generators for a scalar record $\{x_n\}_{n=1}^{N}$.

1. *Phase-randomized* (FT) surrogates: take the discrete Fourier transform, replace each phase by an independent uniform draw on $[0, 2\pi)$ while preserving the conjugate symmetry, and transform back. Verify that your surrogates have the same periodogram as the original to within numerical precision, and explain why preserving conjugate symmetry is necessary rather than cosmetic.

2. *Amplitude-adjusted* (AAFT) surrogates: rank-order the original data, generate a Gaussian record of the same length, reorder it to match the ranks of the original, phase-randomize it, and finally rank-reorder the original values onto the result. Verify that the amplitude distribution is now exactly that of the original.

3. For both, plot the periodogram and the histogram of the original against three surrogates. Which statistic does each method preserve *exactly*, and which only approximately?

Problem 16.2

Computational

A test that never fails to reject is useless. Check that yours can accept.

1. Generate $N = 4096$ points of the linear autoregressive process $x_{n+1} = ax_n + \xi_n$ with $a = 0.8$ and ξ_n Gaussian white noise.

2. Choose a nonlinearity statistic — the time-asymmetry statistic $\langle (x_{n+1} - x_n)^3 \rangle$ is a convenient one — and compute it for the original record and for 199 FT surrogates.

3. Report where the original falls in the surrogate distribution. At what rank would you reject at the 5% level for a two-sided test, and does the original exceed it?

4. The null hypothesis here is *true* by construction. What fraction of repeated trials should reject anyway, and does your implementation match that rate over 100 independent records?

Problem 16.3

Computational

Now a case where the null should be rejected, using the prediction-error statistic of Chapter 15.

1. Take the Lorenz $x(t)$ record of Section 16.1.2. Build the local linear predictor of Chapter 15 and use the out-of-sample RMS prediction error at a fixed horizon as your statistic.

2. Compute it for the original and for 199 AAFT surrogates. Report the rank of the original.

3. Explain why prediction error is a good statistic here while a purely spectral statistic would be a poor one.

4. Repeat with the correlation dimension D_2 of Chapter 11 as the statistic. Which of the two separates the original from the ensemble more cleanly, and at what computational cost?

Problem 16.4

Core

This problem exhibits the trap: a surrogate test can reject the null for a reason that has nothing to do with the dynamics.

1. Generate the linear Gaussian AR process of Problem 16.2, then pass it through the static monotone nonlinearity $s_n = x_n^3$. The *dynamics* are still linear; only the measurement function is not.

2. Apply your FT surrogate test with the time-asymmetry statistic to $\{s_n\}$. Does it reject? Should it?

3. Explain the mechanism: what does phase randomization do to the amplitude distribution of $\{s_n\}$, and why does the resulting mismatch register as nonlinearity?

4. Repeat with AAFT surrogates, and then with the iterative scheme of Schreiber and Schmitz [180]. Does the rejection survive?

5. State, in one sentence, what null hypothesis each of the three generators actually tests. This is the point Kugiumtzis [120] makes when he says the surrogates should be checked before the statistic is applied to them.

Problem 16.5

Extension

Return to Problem 12.1, which asked whether a positive largest Lyapunov exponent lets you conclude that a system is chaotic. You now have the means to answer it.

1. Generate three records of equal length: Lorenz $x(t)$; Gaussian white noise; and power-law correlated noise with spectrum $1/f^\alpha$, $\alpha \approx 1.5$.

2. For each, estimate λ_1 by the method of Section 12.3.1 and record the value.

3. For each, generate 199 AAFT surrogates and compute the distribution of λ_1 over the ensemble.

4. For which records does the original λ_1 fall outside the surrogate distribution? Compare your answer with what you would have concluded from the sign of λ_1 alone.

5. The $1/f^\alpha$ record has a positive λ_1 and is not chaotic. Explain how the surrogate test detects this, and what it would fail to detect.

Problem 16.6

Extension

Multivariate records need more care than applying a scalar method twice.

1. Take the Lorenz $x(t)$ and $y(t)$ channels together. Compute the cross-spectrum and the coherence between them.

2. Generate surrogates by phase-randomizing each channel *independently*. What happens to the coherence?

3. Now randomize both channels with the *same* set of phase increments, following Prichard and Theiler [159]. Verify that the cross-spectrum is preserved.

4. Explain which null hypothesis each construction corresponds to, and why the first is almost never the one you want.

Problem 16.7

Experimental

Does the tracking of Chapter 17 find slow structure, or manufacture it? SOD selects the smoothest coordinate a feature set spans, and a smooth-looking curve is what it returns whether or not anything is drifting. This problem builds the null that settles it, on the measured fatigue record: the tracking matrix $\boldsymbol{Y}$, 1728×405, of `data/psw_fatigue_2005.mat` (`python/chk_psw_problems.py`).

1. Compute the five largest smooth orthogonal values of the first 1200 rows, as in Problem 17.7(1). These are the numbers the null has to beat.

2. *Row-shuffle surrogate.* Permute the row order of $\boldsymbol{Y}$ at random and take the first 1200 rows of the permuted matrix. This preserves the marginal distribution of every one of the 405 features exactly, and destroys any ordering in slow time. Generate 99 such surrogates and build the distribution of the leading SOV. Report the largest value the ensemble reaches, and compare it with part (1). State which null hypothesis this construction tests.

3. *Column-wise Fourier surrogate.* Now phase-randomize each column of $\boldsymbol{Y}$ independently along the slow-time axis, by phase randomization, as in Fig. 16.2. Each feature keeps its own spectrum; what is destroyed is the phase relation *between* features. Repeat the test. Is the leading SOV still outside the ensemble, and by how much less than in part (2)?

4. The two nulls are not the same, and the second is the harder one. State what each rejects, which of the two an anonymous referee should ask for, and what neither of them establishes about the fatigue crack.

5. Repeat part (2) on the *last* 1200 rows, which contain the fracture. The original SOV rises, and so does the surrogate ensemble. Explain why a rejection here is weaker evidence of slow drift than the same rejection in part (2), and connect this to the healthy-stage question of Problem 17.7(2).

17
Invariant Features and the Tracking of Slow Drift

The invariants of Chapters 11 and 12 characterize an attractor by its long-time behavior. A fractal dimension bounds the number of active degrees of freedom. A Lyapunov exponent measures the average divergence rate of neighboring trajectories and separates chaotic from regular motion [4, 48, 105]. Both are estimation problems with a slow convergence rate. A correlation dimension needs a scaling region, a Lyapunov exponent needs a fit window, and both need records of thousands of orbits and low noise [47, 226]. In vibration-based monitoring they have been used to detect a change of state [56, 71, 98, 104, 146, 219, 220]. They are poor at the related task of *tracking*: following a parameter that drifts slowly and continuously through the record. Cheaper features have been proposed for that purpose, such as the Jacobian feature vector [39] and the local attractor variance ratio [215], and the phase-space-warping approach of [33, 34] recovers the drifting variable itself, at a considerable computational cost.

The tracking problem sets its own requirements. The feature must be a well-defined function of the state of the attractor, so that a change in the feature means a change in the system and not a change in the initial condition. It must converge on a short window, because the parameter must not move within the window. It must be cheap, because sensors that sample at high rates (a MEMS device, an atomic force microscope) cannot store raw records and must reduce the data locally. The standard local features are the mean, the variance, and a resonant frequency. They meet the last two requirements and are invariant, but they see only the second moments of the response.

This chapter supplies a class of features that meets all three requirements and shows what they can and cannot see. Section 17.1 states the theorem that makes any time average an invariant of the attractor, with the ergodicity assumption that the statement needs. Sections 17.1.1 and 17.1.2 define the generalized distance, the characteristic distance and the characteristic position, and Section 17.1.3 shows by expansion that, for a reference point far from the attractor, the characteristic distance is a function of the first two moments only. Section 17.2 sets up the fast–slow tracking problem, gives the argument for why the smooth orthogonal decomposition of Chapter 13 recovers the slow variable from features that are themselves discontinuous, tests it on a toy problem, and states the procedure. Sections 17.3–17.5 apply it to the Rössler and Duffing systems and to an experiment. The recurrence measures of Chapter 18 are further members of the same family and are used in the same way.

17.1 Time Averages as Invariant Quantities

Lyapunov exponents and fractal dimensions are rooted in ergodic theory [230], which classifies measures that are invariant under the time evolution. The theorem below is the reason a time average taken along one trajectory says anything about the attractor as a whole. Every metric in this chapter is such an average.

Consider the time evolution of a dynamical system in discrete time, given by a transformation $\mathcal{T} : X \to X$ on $X \subset \mathbb{R}^m$. If $\boldsymbol{x} \in X$ is the current state, then $\mathcal{T}(\boldsymbol{x})$ is the state after one time unit. At steady state the attractor is invariant, $\mathcal{T}(X) \subseteq X$. Invariance of the set alone does not make $\mathcal{T}$ measure preserving. What is needed is an invariant measure μ on X, that is, $\mu\left(\mathcal{T}^{-1}(B)\right) = \mu(B)$ for every measurable B. Such a measure exists for a continuous map on a compact invariant set by the Krylov–Bogolyubov theorem. The physically relevant one is the natural measure of Chapter 11, the one generated by orbits started from Lebesgue-almost every point of the basin. That chapter also states the assumption made throughout the book: the natural measure on a single attractor is *ergodic*, so that it cannot be split into invariant pieces of positive measure. The same assumption is made here. Without it the theorem still holds, but the limit it produces may depend on which piece the trajectory started in.

A system measurement (a velocity, an acceleration, a distance) is a function $f : X \to \mathbb{R}$. To measure a quantity of the system one takes n successive values $f(\boldsymbol{x}), f\left(\mathcal{T}(\boldsymbol{x})\right), \ldots, f\left(\mathcal{T}^{n-1}(\boldsymbol{x})\right)$ and averages them. The question is whether the average converges as $n \to \infty$ and whether the limit depends on $\boldsymbol{x}$.

Birkhoff Ergodic Theorem [230]

Let (X, B, μ) be a finite measure space and $\mathcal{T} : X \to X$ a measure-preserving transformation. For every $f \in L^1(X, B, \mu)$ the limit

$$f^*(\boldsymbol{x}) = \lim_{n\to\infty} \frac{1}{n} \sum_{k=0}^{n-1} f\left(\mathcal{T}^k(\boldsymbol{x})\right) \tag{17.1}$$

exists for μ-almost every $\boldsymbol{x}$, and the limit function is invariant: $f^*\left(\mathcal{T}(\boldsymbol{x})\right) = f^*(\boldsymbol{x})$. If μ is ergodic, f^* is constant μ-almost everywhere and equals the space average $\int_X f \, \mathrm{d}\mu$.

What the theorem does and does not give

The theorem gives existence of the time average and its invariance along the orbit. Independence of the initial condition, which is what a feature of *the attractor* needs, is the ergodic case. Applicability to a trajectory started from a measured, hence Lebesgue-typical, initial condition is the natural-measure case. Under both assumptions f^* is a single number attached to the attractor. It is an *invariant quantity*, not an invariant measure; the measure is μ. A record that visits two coexisting attractors has two such numbers, one per ergodic component, and its running average converges to neither (Problem 17.1).

Any integrable f therefore yields an invariant of the attractor. The dimension and the Lyapunov exponents are averages of particular, expensive f: a local scaling exponent and a local stretching rate. The metrics below use f that cost one distance evaluation per point. They converge at the rate of a sample mean of correlated samples (Section 6.4), need no scaling region and no fit, and tolerate noise, at the price of carrying less information about the dynamics. Which information they do carry is worked out in Section 17.1.3.

17.1.1 Generalized Distances

Let $g_q : X \times \mathbb{Z} \to \mathbb{R}$ be the *Minkowski distance* between a point $\boldsymbol{x} \in X \subset \mathbb{R}^m$ and its image after k time steps, $\mathcal{T}^k\boldsymbol{x} \in X$,

$$g_q(\boldsymbol{x}, k) = \left\|\boldsymbol{x} - \mathcal{T}^k\boldsymbol{x}\right\|_q = \left(\sum_{i=1}^{m} \left|x_i - \left(\mathcal{T}^k\boldsymbol{x}\right)_i\right|^q\right)^{\frac{1}{q}} . \tag{17.2}$$

The scalar *generalized distance* $d_q(k)$ is its time average,

$$d_q(k) = \lim_{n\to\infty} \frac{1}{n} \sum_{i=0}^{n-1} g_q(\mathcal{T}^i(\boldsymbol{x}), k) . \tag{17.3}$$

For $q = 2$ the sample estimate from a scalar record embedded with m forward delays reads

$$\begin{aligned} d_2(k) &= E\left[\|\boldsymbol{x}_i - \boldsymbol{x}_{i+k}\|_2\right]_i = E\left[\sqrt{(\boldsymbol{x}_i - \boldsymbol{x}_{i+k})^{\mathrm{T}}(\boldsymbol{x}_i - \boldsymbol{x}_{i+k})}\right]_i \\ &= \frac{1}{n-k} \sum_{i=0}^{n-k-1} \sqrt{\sum_{j=0}^{m-1} \left(x_{i+j\tau} - x_{i+k+j\tau}\right)^2} , \end{aligned} \tag{17.4}$$

and for general q

$$\begin{aligned} d_q(k) &= E\left[\|\boldsymbol{x}_i - \boldsymbol{x}_{i+k}\|_q\right]_i = E\left[\left(\sum_{j=0}^{m-1} |x_{i+j\tau} - x_{i+k+j\tau}|^q\right)^{\frac{1}{q}}\right]_i \\ &= \frac{1}{n-k} \sum_{i=0}^{n-k-1} \left(\sum_{j=0}^{m-1} |x_{i+j\tau} - x_{i+k+j\tau}|^q\right)^{\frac{1}{q}} . \end{aligned} \tag{17.5}$$

The connection to the autocovariance is exact only for the *squared* distance. Writing $\boldsymbol{C}_k = E\left[\boldsymbol{x}_i\boldsymbol{x}_{i+k}^{\mathrm{T}}\right]_i$ for the lag-k autocovariance matrix of the mean-removed embedded record,

$$E\left[\|\boldsymbol{x}_i - \boldsymbol{x}_{i+k}\|_2^2\right]_i = 2\left(\operatorname{tr}\boldsymbol{C}_0 - \operatorname{tr}\boldsymbol{C}_k\right) , \tag{17.6}$$

which is linear in the autocovariance and reduces, for a scalar record embedded with m delays, to $2m\left(R_{xx}(0) - R_{xx}(k)\right)$. The metric $d_q(k)$ takes the root *inside* the average, so it is not a linear statistic of the autocovariance for any q, including $q = 2$. By Jensen's inequality

$$d_2(k) \le \sqrt{2\left(\operatorname{tr}\boldsymbol{C}_0 - \operatorname{tr}\boldsymbol{C}_k\right)} , \tag{17.7}$$

with the gap set by the point-to-point spread of the distance distribution over the record. The two agree only when that spread vanishes. The gap is what $d_q(k)$ adds to the linear autocorrelation of Chapter 3: a measure of how unevenly the attractor's k-step displacements are distributed.

Different k at each q give a spectrum of characteristic lengths. Once $d_q(k)$ has grown to the size of the attractor the curve saturates and carries no further information, much as the divergence curve of Chapter 12 or the correlation sum of Chapter 11 saturate. Before saturation the first minimum of $d_q(k)$ is a candidate delay time, to be compared with the mutual-information choice of Chapter 9 (Problem 17.2).

17.1.2 Characteristic Distance and Position

Let $f : X \to \mathbb{R}$ be the Euclidean distance between a point $\boldsymbol{x} \in X \subset \mathbb{R}^m$ and a fixed reference point $\boldsymbol{y} \in \mathbb{R}^m$,

$$f(\boldsymbol{x}) = \|\boldsymbol{x} - \boldsymbol{y}\|_2 \,. \tag{17.8}$$

The scalar *characteristic distance* $D(\boldsymbol{y})$ is its time average,

$$D(\boldsymbol{y}) = \lim_{n\to\infty} \frac{1}{n} \sum_{i=0}^{n-1} f(\mathcal{T}^i(\boldsymbol{x})) \,. \tag{17.9}$$

By the Birkhoff theorem under the ergodicity assumption, $D(\boldsymbol{y})$ is an invariant quantity of the attractor: the mean distance from the reference point to the attractor, weighted by the natural measure. It is a sample mean of n nonnegative numbers, and it needs no scaling region. The Birkhoff theorem guarantees convergence but gives no rate. If the feature sequence $f(\mathcal{T}^i(\boldsymbol{x}))$ has a finite integrated autocorrelation time, which holds when the dynamics are mixing fast enough for the autocovariances of f to be summable, the error falls as $\sigma_f \left[(1 + 2\sum_{k\geq 1} \rho_f(k))/n\right]^{1/2}$. Here σ_f is the spread of $\|\boldsymbol{x} - \boldsymbol{y}\|$ over the attractor and ρ_f is the autocorrelation of this feature along the trajectory, not of the record (Section 6.4).

There is nothing special about the Euclidean distance beyond its cost. Other integrable functions of $\boldsymbol{x} - \boldsymbol{y}$ give other invariants. The vector-valued *characteristic position*

$$\boldsymbol{P}(\boldsymbol{y}) = \lim_{n\to\infty} \frac{1}{n} \sum_{i=0}^{n-1} \boldsymbol{f}_p(\mathcal{T}^i(\boldsymbol{x})) \,, \qquad \boldsymbol{f}_p(\boldsymbol{x}) = \frac{\boldsymbol{x} - \boldsymbol{y}}{\|\boldsymbol{x} - \boldsymbol{y}\|_2} \,, \tag{17.10}$$

averages the unit vector from $\boldsymbol{y}$ to the trajectory. It carries m numbers per reference point instead of one, and directional information that $D(\boldsymbol{y})$ discards: $\|\boldsymbol{P}(\boldsymbol{y})\|_2$ is near one when the attractor is seen from $\boldsymbol{y}$ within a narrow cone, and near zero when it surrounds $\boldsymbol{y}$ (Problem 17.3).

17.1.3 What the Characteristic Distance Measures

The characteristic distance is a function of the natural measure. How much of that measure it sees depends on where the reference point is. Let $\bar{\boldsymbol{x}} = \mathrm{E}[\boldsymbol{x}]$ and $\boldsymbol{C} = \mathrm{E}\left[(\boldsymbol{x} - \bar{\boldsymbol{x}})(\boldsymbol{x} - \bar{\boldsymbol{x}})^{\mathrm{T}}\right]$ be the mean and covariance of the attractor under μ, and write $\boldsymbol{r} = \boldsymbol{y} - \bar{\boldsymbol{x}}$, $\rho = \|\boldsymbol{r}\|_2$, $\hat{\boldsymbol{r}} = \boldsymbol{r}/\rho$.

Far-field expansion of the characteristic distance

Let the attractor have finite fourth moments and let ρ exceed the attractor's extent. Then

$$D(\boldsymbol{y}) = \rho + \frac{\operatorname{tr} \boldsymbol{C} - \hat{\boldsymbol{r}}^{\mathrm{T}} \boldsymbol{C} \hat{\boldsymbol{r}}}{2\rho} + O(\rho^{-2}) \,, \tag{17.11}$$

where the $O(\rho^{-2})$ term involves the third central moments and the $O(\rho^{-3})$ term the fourth.

The derivation is two lines. With $\boldsymbol{\xi} = \boldsymbol{x} - \bar{\boldsymbol{x}}$,

$$\|\boldsymbol{y} - \boldsymbol{x}\|_2 = \|\boldsymbol{r} - \boldsymbol{\xi}\|_2 = \rho \left(1 - \frac{2\hat{\boldsymbol{r}}^{\mathrm{T}}\boldsymbol{\xi}}{\rho} + \frac{\|\boldsymbol{\xi}\|^2}{\rho^2}\right)^{1/2} = \rho - \hat{\boldsymbol{r}}^{\mathrm{T}}\boldsymbol{\xi} + \frac{\|\boldsymbol{\xi}\|^2 - (\hat{\boldsymbol{r}}^{\mathrm{T}}\boldsymbol{\xi})^2}{2\rho} + O(\rho^{-2}) \,. \tag{17.12}$$

Averaging over μ kills the linear term, since $\mathrm{E}[\boldsymbol{\xi}] = \boldsymbol{0}$, and turns $\mathrm{E}\left[\|\boldsymbol{\xi}\|^2\right]$ into $\operatorname{tr} \boldsymbol{C}$ and $\mathrm{E}\left[(\hat{\boldsymbol{r}}^{\mathrm{T}}\boldsymbol{\xi})^2\right]$ into $\hat{\boldsymbol{r}}^{\mathrm{T}}\boldsymbol{C}\hat{\boldsymbol{r}}$. The correction term is the mean squared spread of the attractor *transverse* to the line of

sight, divided by 2ρ. Numerically, for delay vectors of the logistic map at $r = 3.9$ ($m = 2$), the error of Eq. (17.11) is 2.9×10^{-3} at $\rho = 1$ and 3.1×10^{-5} at $\rho = 10$, falling as ρ^{-2} (`python/chk_metrics.py`).

Three consequences follow.

Far points are moment statistics. A reference point well outside the attractor returns a smooth function of $\bar{\boldsymbol{x}}$ and $\boldsymbol{C}$, that is, of the same mean and variance that the standard local features report. However many far points are used, their characteristic distances carry, to leading order, no more than the $m + m(m+1)/2$ numbers in the first two moments. What they add is a large number of *different nonlinear functions* of those moments, and Section 17.2.1 shows why that is useful for tracking.

Near points see the shape. For a reference point inside or close to the attractor the expansion fails, and $D(\boldsymbol{y})$ depends on how the natural measure is distributed around $\boldsymbol{y}$: on the visited region, its holes and its density. This is the regime in which $D(\boldsymbol{y})$ is more than a moment statistic. The characteristic position $\boldsymbol{P}(\boldsymbol{y})$ of Eq. (17.10) is sensitive to the same structure at any distance, since its far-field limit is the unit vector $-\hat{\boldsymbol{r}}$ and all of its variation is in the correction terms.

Noise adds a constant. Isotropic measurement noise of variance σ^2 per coordinate, independent of the state, leaves $\bar{\boldsymbol{x}}$ unchanged and replaces $\boldsymbol{C}$ by $\boldsymbol{C} + \sigma^2\boldsymbol{I}$. By Eq. (17.11) it shifts $D(\boldsymbol{y})$ by

$$\Delta D \approx \frac{(m-1)\,\sigma^2}{2\rho}\,, \tag{17.13}$$

independent of the attractor. On the logistic-map cloud with $\sigma = 0.2$ (two thirds of the signal standard deviation) the measured shift is 0.0104 at $\rho = 2$ against the predicted 0.0100. A feature that shifts by a constant across the record is harmless for tracking, which is why the method survives the noise levels of Section 17.5. Near points are not covered by the argument, and there noise smears the density and the shift depends on $\boldsymbol{y}$ and on the state.

17.2 Reconstructing Slowly Drifting Variables

A slowly drifting parameter is a hidden variable. The fast measured response reflects it only through the changing shape of the attractor, and a tracking method has to read it back from that shape. The setting is a coupled system in which a slow state drives the parameters of a fast subsystem:

$$\dot{\boldsymbol{x}} = \boldsymbol{f}(\boldsymbol{x}, \boldsymbol{\mu}(\boldsymbol{\phi}), t), \quad \dot{\boldsymbol{\phi}} = \epsilon\boldsymbol{g}(\boldsymbol{\phi}, t), \tag{17.14}$$

where $\boldsymbol{x} \in X \subset \mathbb{R}^m$ is the fast state, $\boldsymbol{\phi} \in P \subset \mathbb{R}^s$ is the hidden slow state, which alters the parameter vector $\boldsymbol{\mu} \in \mathbb{R}^p$; $t \in \mathbb{T} \subset \mathbb{R}$ is time; overdots denote time differentiation; $0 < \epsilon \ll 1$ is a small rate constant setting the time-scale separation. Damage accumulation is the motivating case [35]. Over an intermediate time interval, long compared with the fast oscillation and short compared with $1/\epsilon$, $\boldsymbol{\phi}$ is approximately constant.

The fast subsystem of Eq. (17.14) has the general solution $\boldsymbol{x} = \boldsymbol{X}(t, \boldsymbol{x}_0, \boldsymbol{\phi})$, where $\boldsymbol{x}_0$ is the initial condition. The continuous-time characteristic distance for a reference point $\boldsymbol{y}$ is

$$D(\boldsymbol{y}) = \lim_{T\to\infty} \frac{1}{T}\int_0^T \|\boldsymbol{y} - \boldsymbol{X}(t, \boldsymbol{x}_0, \boldsymbol{\phi})\|_2 \; dt\,, \tag{17.15}$$

approximated over an intermediate interval T_n by

$$D(\boldsymbol{y}) \approx \frac{1}{T_n}\int_0^{T_n} \|\boldsymbol{y} - \boldsymbol{X}(t, \boldsymbol{x}_0, \boldsymbol{\phi})\|_2 \; dt\,. \tag{17.16}$$

The integrand depends on $\boldsymbol{\phi}$ through $\boldsymbol{\mu}(\boldsymbol{\phi})$, so D is a function of the slow state, written $D(\boldsymbol{\phi}, \boldsymbol{y})$ when both arguments matter.[1]

No smoothness is claimed. As $\boldsymbol{\phi}$ crosses a bifurcation the attractor can change position and topology abruptly, so $D(\boldsymbol{\phi}, \boldsymbol{y})$ is in general a piecewise-continuous function of $\boldsymbol{\phi}$ with jumps at the bifurcation values. It is nevertheless a well-defined function, by the theorem of Section 17.1 applied to each attractor separately, and that is what the construction below requires.

The tracking matrix. A scalar record from the fast subsystem is cut into N consecutive windows. The ℓth window ($\ell = 1, \ldots, N$) covers an intermediate interval over which the slow variable is effectively constant, $\boldsymbol{\phi} \approx \boldsymbol{\phi}_\ell = \langle \boldsymbol{\phi} \rangle_\ell$, the window average. Across windows, $\boldsymbol{\phi}_\ell$ changes gradually. Here ℓ indexes windows and j indexes reference points; neither is the phase-space index i of Eq. (17.9). In each window the record is embedded and the characteristic distances to L fixed reference points $\{\boldsymbol{y}_j\}_{j=1}^{L}$ are estimated and assembled into a feature vector,

$$\boldsymbol{Y}^\ell = [D(\boldsymbol{\phi}_\ell, \boldsymbol{y}_1);\, D(\boldsymbol{\phi}_\ell, \boldsymbol{y}_2);\, \ldots;\, D(\boldsymbol{\phi}_\ell, \boldsymbol{y}_L)] \,. \tag{17.17}$$

Stacked in time order the feature vectors form the *tracking matrix* $\boldsymbol{Y} \in \mathbb{R}^{N \times L}$, whose (ℓ, j) entry is $D(\boldsymbol{\phi}_\ell, \boldsymbol{y}_j)$. Each column is one feature followed through the record; each row is the state of the attractor in one window, as seen from L vantage points.

The reference points are drawn at random and then held fixed for the whole record. By Section 17.1.3 their placement decides what the features see. Points far outside the attractor give many nonlinear functions of the first two moments; points inside and near it see the shape of the natural measure. A workable default is to draw them uniformly in the bounding box of the pooled reconstructed trajectory, enlarged by a modest factor, so that the set contains both kinds. That is the rule used for every simulated example in this chapter: uniform in the bounding box of the pooled trajectory, enlarged by a factor 1.5 about its centre. It is the rule of the toy problem of Section 17.2.2 with a 50 % margin, and it is stated here rather than left to the individual examples because, by Section 17.1.3, the sampling region is the choice that decides what the features measure. It is not a neutral choice: on a system with one strongly skewed coordinate — the Rössler z, which sits near zero and spikes to more than 60 — the bounding box is set by the rare excursions and most of the reference points land far outside the attractor, where by Eq. (17.11) they report the first two moments and little else. Section 17.3 shows what that costs.

Smooth orthogonal decomposition of the tracking matrix. The slow variable is smooth in time; the individual features are not. The smooth orthogonal decomposition (SOD) of Chapter 13 is the tool that separates the two, and it has been used for this purpose before [34, 36]. The columns of $\boldsymbol{Y}$ are first centred, each column having its mean removed. Without this step the smoothest coordinate SOD returns is the constant one, the linear combination of the columns closest to a constant vector, whose derivative vanishes and whose smooth orthogonal value is arbitrarily large (Section 17.2.2 gives the numbers). SOD is then the generalized singular value decomposition of the centred matrix and its time derivative $\boldsymbol{DY}$, where $\boldsymbol{D}$ is a discrete differential operator, here the first difference across windows:

$$\boldsymbol{Y} = \boldsymbol{U}_1 \boldsymbol{\Sigma}_1 \boldsymbol{\Phi}^{\mathrm{T}} \quad \text{and} \quad \boldsymbol{DY} = \boldsymbol{U}_2 \boldsymbol{\Sigma}_2 \boldsymbol{\Phi}^{\mathrm{T}} \,, \tag{17.18}$$

[1] The same holds for the characteristic position, $\boldsymbol{P}(\boldsymbol{\phi}, \boldsymbol{y})$.

in the notation of Chapter 13. Here $\boldsymbol{U}_1 \in \mathbb{R}^{N\times L}$ and $\boldsymbol{U}_2 \in \mathbb{R}^{(N-1)\times L}$ have orthonormal columns, $\boldsymbol{\Sigma}_1$ and $\boldsymbol{\Sigma}_2$ are diagonal, and $\boldsymbol{\Phi} \in \mathbb{R}^{L\times L}$ is square and invertible, its columns holding the SOMs. The *smooth orthogonal coordinates* (SOCs) are the columns of $\boldsymbol{Q} = \boldsymbol{U}_1\boldsymbol{\Sigma}_1 = \boldsymbol{Y}\boldsymbol{\Psi}$, the *smooth projective modes* (SPMs) the columns of $\boldsymbol{\Psi} = \boldsymbol{\Phi}^{-\mathrm{T}}$, and the *smooth orthogonal values* (SOVs)

$$\lambda_p^2 = \left(\frac{\sigma_{1,p}}{\sigma_{2,p}}\right)^2 = \frac{\langle q_p^2\rangle}{\langle \dot{q}_p^2\rangle}, \qquad p = 1,\dots,L, \tag{17.19}$$

the ratio of the variance of the pth SOC to the variance of its time derivative. The larger the SOV, the smoother in time the coordinate. The claim to be justified is that the slow variable appears, up to scale and offset, in the SOC with the largest SOV. The decomposition needs $\boldsymbol{Y}$ of full column rank, so $N > L$ and no two features may be collinear. A tracking matrix built from far points only is nearly rank deficient by Section 17.1.3; it is reduced to its leading principal components before the GSVD.

17.2.1 Why the Smoothest Coordinate Tracks the Slow Variable

Two things need an argument. The individual features are discontinuous in $\boldsymbol{\phi}$, yet a linear combination of them is claimed to be continuous. And the leading coordinate is claimed to follow $\boldsymbol{\phi}$ closely, which is what the correlation coefficients reported below measure. Neither follows from the SOD variational principle alone, and the second holds only under conditions stated below. For a scalar slow variable ϕ the arguments run as follows.

Jump-subspace argument

Let the feature map $\phi \mapsto \boldsymbol{Y}(\phi) \in \mathbb{R}^L$ be piecewise continuous on the range of the drift, with J jumps at the bifurcation values $\phi_1,\dots,\phi_J$ and jump vectors $\boldsymbol{\Delta}_j = \boldsymbol{Y}(\phi_j^+) - \boldsymbol{Y}(\phi_j^-)$. Then

$$\boldsymbol{Y}(\phi) = \boldsymbol{S}(\phi) + \sum_{j=1}^{J} H(\phi-\phi_j)\,\boldsymbol{\Delta}_j\,, \tag{17.20}$$

with H the unit step and $\boldsymbol{S}$ continuous. Assume in addition that $\boldsymbol{S}$ has a square-integrable derivative on the range of the drift, as it does when $\boldsymbol{S}$ is piecewise C^1. The jump vectors span a subspace $\mathcal{J} \subseteq \mathbb{R}^L$ of dimension at most J. If $L > J$, every $\boldsymbol{\psi}$ in the orthogonal complement $\mathcal{J}^\perp$, of dimension at least $L-J$, gives a coordinate $q(\phi) = \boldsymbol{\psi}^{\mathrm{T}}\boldsymbol{Y}(\phi) = \boldsymbol{\psi}^{\mathrm{T}}\boldsymbol{S}(\phi)$ that is continuous in ϕ, hence in time. SOD selects among such $\boldsymbol{\psi}$ provided $\boldsymbol{S}$ has a component in $\mathcal{J}^\perp$ that varies with ϕ.

The decomposition Eq. (17.20) is a definition: subtracting the steps from $\boldsymbol{Y}$ leaves a function with no jumps, whatever the features do between bifurcations. The point is the dimension count. Each bifurcation contributes one direction to $\mathcal{J}$, so with L features there are at least $L-J$ directions in which no jump is visible. That SOD prefers them is a matter of roughness. A coordinate sampled at N windows spaced h apart, containing a step of size δ, has first difference δ/h at the step, which alone contributes $\delta^2/\big((N-1)h^2\big) \approx \delta^2 N/T^2$ to $\langle \dot{q}^2\rangle$ for a record of length $T = Nh$. This grows without bound as the windows are shortened, whereas $\langle \dot{q}^2\rangle$ of a continuous piecewise-C^1 coordinate converges to $\int_0^T \dot{q}^2\,\mathrm{d}t/T$. Any $\boldsymbol{\psi}$ with a component along $\mathcal{J}$ pays a roughness penalty growing like N, and for large N the maximizer of $\langle q^2\rangle / \langle \dot{q}^2\rangle$ is driven into $\mathcal{J}^\perp$. Continuity alone does not give this; the derivative hypothesis of the box does. A coordinate that is continuous but has a square-root

onset, as amplitudes commonly do at a local bifurcation, has a discrete $\langle \dot{q}^2 \rangle$ growing like $\log N$, and a Hölder-α cusp gives growth like $N^{1-2\alpha}$ (for $T = 1$ and $N = 10^2, 10^4, 10^6$: 1.64, 2.80, 3.95 for the square root, against 99, 9999, 10^6 for a unit jump; `python/chk_metrics.py`). Such coordinates still lose to a jump as N grows, but their derivative energy does not converge. At finite N the comparison is made against the estimation scatter of the features, which adds a roughness floor to every direction. Here J counts the bifurcations whose jumps stand above that scatter; a jump buried in the window-to-window scatter is not a jump for this purpose. The count that matters on the other side is the numerical rank of the centred tracking matrix, not L. Features that are smooth functions of a few common statistics, such as the far points of Section 17.1.3, add columns without adding rank. A rank well above J is necessary for the method. It is not sufficient, since $\boldsymbol{S}$ must also vary with ϕ in $\mathcal{J}^{\perp}$ by more than the scatter.

Slowest-coordinate statement

Let $q = \boldsymbol{\psi}^{\mathrm{T}} \boldsymbol{Y}_c(t)$ range over the span of the centred features. The first SOC maximizes $\langle q^2 \rangle / \langle \dot{q}^2 \rangle$ over this span. Over all mean-zero q with square-integrable derivative on $[0, T]$ the ratio is at most T^2/π^2, with equality only for $q \propto \cos(\pi t/T)$; for a record periodic in T the bound is $T^2/(4\pi^2)$, attained by the first harmonic. The first SOC is therefore the coordinate in the span that is slowest in time. If every feature is a function of a scalar slow variable, $\boldsymbol{Y}(t) = \boldsymbol{h}(\phi(t))$, the first SOC is a function of ϕ. It is affine in ϕ only when the affine coordinate is the slowest in time among those the span contains. That holds when $\phi(t) - \langle \phi \rangle$ is itself the slowest mode of the record, and it can fail otherwise.

The bound is the free-end (Neumann) eigenvalue problem on $[0, T]$: $\langle q^2 \rangle / \langle \dot{q}^2 \rangle$ is a Rayleigh quotient whose maximizer among mean-zero functions is the lowest nonconstant eigenfunction $\cos(\pi t/T)$. For single functions of the drift, with $\phi = t$ and with $\phi = \sin \omega t$ over whole periods, the ratios are

$$\frac{\langle q^2 \rangle}{\langle \dot{q}^2 \rangle} = \begin{cases} T^2/12 & q = t - T/2\,, \\ T^2/15 & q = t^2 - T^2/3\,, \end{cases} \qquad \frac{\langle q^2 \rangle}{\langle \dot{q}^2 \rangle} = \begin{cases} 1/\omega^2 & q = \sin \omega t\,, \\ 1/(4\omega^2) & q = \sin^2 \omega t - 1/2\,. \end{cases} \tag{17.21}$$

The integrals are elementary: $\langle (t - T/2)^2 \rangle = T^2/12$ against $\langle 1 \rangle = 1$; $\langle (t^2 - T^2/3)^2 \rangle = 4T^4/45$ against $\langle (2t)^2 \rangle = 4T^2/3$; and $\sin^2 \omega t - 1/2 = -\cos 2\omega t/2$ has variance $1/8$ and derivative variance $\omega^2/2$. Without mean removal the raw ratios are $T^2/3$ for $q = t$ and $3T^2/20$ for $q = t^2$, with the same ordering. All six values were checked numerically (`python/chk_metrics.py`). These pairwise orderings do not order spans. A span of several functions of ϕ can contain a combination slower in time than any of its members, and the box then selects that combination.

Two drift laws show both cases. For $\phi = \sin \omega t$ over whole periods, every $h(\phi(t))$ has period $2\pi/\omega$, so its ratio is at most $1/\omega^2$, with equality only for the first harmonic. The symmetry $q(\pi/\omega - t) = q(t)$ of any function of $\sin \omega t$ excludes the cosine, and the affine coordinate wins. For $\phi = t$, or for the quarter-period sinusoid $\phi = \sin(\pi t/2T)$ of the Rössler example in Section 17.3, the affine coordinate is not the slowest. With $\phi = t$ the affine coordinate has ratio $T^2/12$, while $\cos(\pi\phi/T)$, also a smooth function of ϕ, has T^2/π^2, and SOD of the pair returns the cosine. No trigonometric feature is needed: SOD of the span $\{\phi, \phi^2\}$ returns ϕ, but the span $\{\phi, \phi^2, \phi^3\}$ already yields a cubic with ratio $0.1013\,T^2$, whose affine fit on ϕ has $R^2 = 0.987$. With $\phi = \sin(\pi t/2T)$ the quadratic $\phi^2 = (1 - \cos \pi t/T)/2$ is exactly the slowest mode, with ratio T^2/π^2 against $0.0768\,T^2$ for ϕ itself, and SOD of the span $\{\phi, \phi^2\}$ returns ϕ^2; its affine fit on ϕ has $R^2 = 0.951$. These values are computed by `python/chk_metrics.py`.

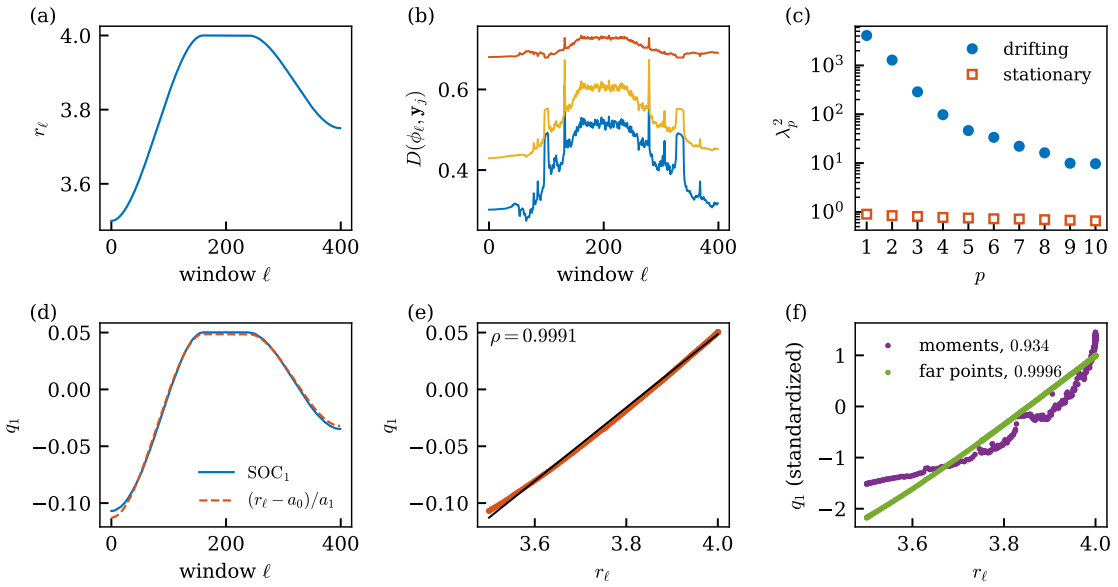


Figure 17.1: Tracking a drifting parameter of the logistic map. (a) Drift r_ℓ over $N = 400$ windows. (b) Three columns of the tracking matrix. (c) Ten largest SOVs, drifting and stationary records. (d) First SOC against window index, with r_ℓ affinely scaled onto it. (e) First SOC against r_ℓ with the affine fit. (f) First SOC from five moment features and from sixty far reference points, standardized, against r_ℓ.

The leading SOC is therefore a smooth coordinate that tracks the slow variable, not a calibrated estimate of it. Its relation to ϕ depends on the feature span, on how ϕ is distributed in time, and on the end conditions of the record. Recovering ϕ itself needs either a monotone calibration against known values or a drift law known to make the affine coordinate slowest. The SOCs after the first are smooth functions of the same ϕ and can also have large SOVs, so the number of large SOVs does not estimate the number of independent slow variables.

17.2.2 A Toy Problem

Logistic map with a drifting parameter

The map $x_{n+1} = r\,x_n(1 - x_n)$ is iterated with r following the drift of Fig. 17.1(a): $3.5 \to 4.0$ over the first 160 windows, a plateau of 80 windows, and a return to 3.75. The range crosses the period-doubling cascade and several periodic windows, so the attractor changes shape and support repeatedly. Each of the $N = 400$ windows holds 3000 iterates at a fixed r_ℓ, embedded with $m = 2$, $\tau = 1$. The reference set is $L = 60$ points drawn uniformly in the bounding box of the pooled trajectory, so most lie inside or near the attractor.

The first SOC follows r_ℓ with correlation coefficient 0.9991 (Fig. 17.1(d,e)); an affine fit maps it onto r_ℓ with residual 0.007 over a range of 0.5. The drift was chosen non-monotone and non-sinusoidal so that this could not be the lowest Fourier mode of time; the correlation of the SOC with time itself is 0.41. The leading SOV is 4.1×10^3, the second 1.3×10^3. The second SOC is 99 % explained by a cubic polynomial in r_ℓ: a higher-order function of the same variable, not a second slow variable. Numbers are from `python/mkfig_metrics_toy.py`.

The jump-subspace argument can be checked on this record. The individual features jump: the largest window-to-window increment of a column is 139 times its median increment. Marking a window as a jump when the increment vector exceeds ten times its median norm gives $J = 10$ jumps. The 10×60 matrix of jump vectors has numerical rank 3, since several jumps are the same periodic window entered and left. The complement $\mathcal{J}^\perp$ has dimension at least 57. The fraction of the total jump energy $\sum_j \|\boldsymbol{\Delta}_j\|^2$ falling along the leading SPM $\hat{\boldsymbol{\psi}}_1$ is 2.6×10^{-5}, against 1.8×10^{-2} for a random unit direction: SOD has found a direction orthogonal to the jumps, as the argument of Section 17.2.1 says it must.

A stationary null record is what separates this result from an artefact. On a record with $r = 3.9$ in every window, the same reference points and the same windowing, the leading SOVs are 0.90 and 0.84 (Fig. 17.1(c)), three and a half orders of magnitude below the drifting case. The standard deviation of the stationary first SOC is nevertheless 0.69 that of the drifting one. The SOV, not the amplitude of the coordinate, separates a drift from finite-window scatter. Without centring the columns the leading SOC of the drifting record is a constant: its standard deviation is 2×10^{-6} of its mean, its SOV exceeds the second by a factor 5×10^8, and its correlation with r_ℓ is 0.40.

Two baselines are shown in Fig. 17.1(f). The five moment features of each window, the two means and the three distinct covariance entries, tracked by the same SOD give correlation 0.934 with a visibly curved relation. Sixty reference points placed eight bounding-box half-widths away give 0.9996, better than the near points. By Section 17.1.3 the far features are functions of those same five moments; a regression of the far-point tracking matrix on the moments has $R^2 = 0.99998$, and of the near-point matrix $R^2 = 0.994$. The gain over the raw moments is not information the moments lack. It is the dimension count of Section 17.2.1. With $L = 5$ features and a jump subspace of rank 3, $\mathcal{J}^\perp$ has dimension two, and the smooth part of the features has little room in it. With $L = 60$ nonlinear re-encodings of the same moments the count is comfortable. In this toy problem the near points do carry information beyond the second moments (0.6 % of their variance) but do not need it to track r_ℓ. Which regime a measured record is in has to be tested, as Section 17.4 notes.

Noise behaves as Section 17.1.3 predicts. Gaussian measurement noise of standard deviation 0.1, one third of the signal standard deviation, is added to the iterates before embedding. The correlation of the first SOC with r_ℓ is then 0.996 for the near points and 0.977 for the far points. The far points

lose more, since noise of a fixed variance is a larger fraction of the transverse spread they see; the near points average it over the density.

17.2.3 Procedure

The steps below collect Sections 17.2–17.2.2 into a procedure. Every choice in it is an estimation choice. The window length trades bias from drift within the window against variance from the finite sample. The number of reference points trades the dimension count of Section 17.2.1 against the rank condition of the GSVD. The null comparison turns the smoothest coordinate into evidence of a drift.

Tracking a slow variable with characteristic distances

1. Choose the window length n: short enough that the slow variable moves less than the resolution sought within a window, long enough that the sample mean Eq. (17.9) converges to that resolution (error $\propto n^{-1/2}$ once n exceeds the feature's correlation time, under the summability assumption of Section 17.1.2; Problem 17.4).
2. Choose τ and m once, on a window from the middle of the record (Chapters 9 and 10), and keep them fixed for every window.
3. Draw L reference points at random in an enlarged bounding box of the pooled trajectory and hold them fixed; L large compared with the number of bifurcations crossed, and $L < N$.
4. Embed each window and compute $D(\boldsymbol{\phi}_\ell, \boldsymbol{y}_j)$ by Eq. (17.9), forming $\boldsymbol{Y} \in \mathbb{R}^{N\times L}$ (for the characteristic position, Eq. (17.10) and mL columns).
5. Centre the columns of $\boldsymbol{Y}$. If $\boldsymbol{Y}$ is numerically rank deficient, project onto its leading principal components.
6. Form $\boldsymbol{DY}$ by first differences and compute the GSVD Eq. (17.18); order by decreasing λ_p^2. The tracking estimate is $\hat{\phi}_\ell = (\boldsymbol{Y}\boldsymbol{\psi}_1)_\ell$, known up to scale and offset.
7. Repeat steps 4–6 on a stationary record with the same windowing, or on surrogates (Chapter 16), and compare the leading SOVs (Problem 17.6).
8. Report n, τ, m, L, N, the sampling region of the reference points, the leading SOVs with their null values, and any calibration applied to $\hat{\phi}_\ell$.

The cost is small. Each window needs nL distance evaluations, one pass over the data with no neighbor search, and the GSVD of an $N \times L$ matrix costs $O(NL^2)$ once for the whole record. A sensor that can form a sample mean can form the feature vector Eq. (17.17); the decomposition can be done later, on the $N \times L$ numbers that leave the sensor, instead of on the raw record.

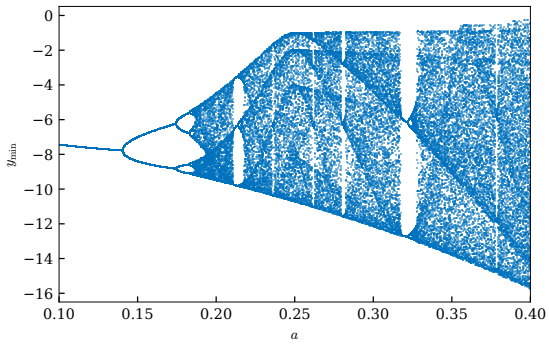


Figure 17.2: Bifurcation diagram of the Rössler system Eq. (17.22) against a, for $b = 0.6$ and $c = 6.0$. The plotted quantity is the local minimum of $y(t)$, collected over 4000 steps of length $t_s = 0.06$ at each of 1200 values of a after a transient of 20,000 steps. (y itself is positive over part of every orbit in this range, so the first-edition axis label $y_{\max}$ cannot have been the quantity shown.) Generated by `python/mkfig_rossler_bifu.py`.

Pitfall: the smoothest coordinate is not automatically the slow variable

SOD returns the smoothest linear combination of whatever columns it is given. Five things make that combination something other than the drift. Uncentred columns give a constant. Too few features relative to the number of bifurcations crossed give a coordinate that still jumps. Features that depend on ϕ only through a nonlinear function give that function. Too many features relative to the number of windows, L approaching N, let SOD assemble a smooth function of time out of scatter. That is why the stationary null is part of the procedure of Section 17.2.3 and not an optional check. And a record whose fast subsystem visits two coexisting attractors within one window has no single characteristic distance for that window, by Section 17.1. In every case the leading SOV, compared with its null value, is the diagnostic; the appearance of the coordinate is not.

17.3 Numerical Examples

The procedure of Section 17.2.3 is applied to the Rössler and Duffing systems, with a slow variable introduced by varying one parameter across the record. The first example works in the true three-dimensional phase space, the second in a delay reconstruction from a scalar record.

Rössler system. The equations are

$$\begin{aligned} \dot{x} &= -y - z\,, \\ \dot{y} &= x + a y\,, \\ \dot{z} &= b + z(x - c)\,. \end{aligned} \tag{17.22}$$

Here b is fixed at 0.6, c at 6.0, and a is varied sinusoidally between 0.1 and 0.4 as the slow variable, a quarter period over the record, $a_\ell = 0.1 + 0.3\sin(\pi\ell/2(N-1))$ across the $N = 786$ windows. These are not the standard Rössler parameters $(a, b, c) = (0.2, 0.2, 5.7)$; the bifurcation diagram of Fig. 17.2 belongs to this set. For each value of a, 5000 steady-state points are generated with time

step $t_s = 0.06$, about fifty orbits per window, after a transient of 20,000 steps. Filled regions of Fig. 17.2 are chaotic; the gaps are periodic windows, each of which is a bifurcation the drift crosses twice.

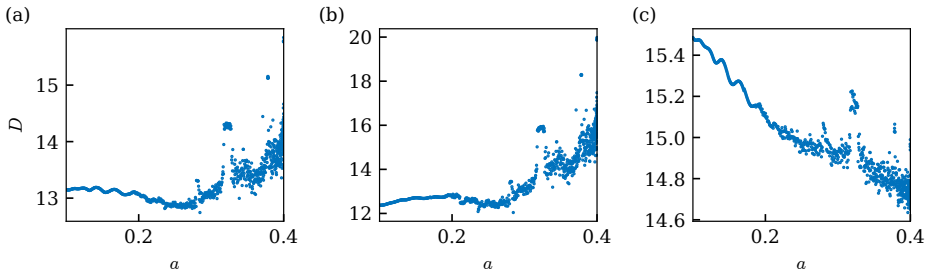


Figure 17.3: Characteristic distance D against a for three of the $L = 50$ reference points in the Rössler phase space, the three lying closest to the attractor. Reference points drawn uniformly in the bounding box of the pooled trajectory enlarged by 1.5; $N = 786$ windows of 5000 points at $t_s = 0.06$. Generated by `python/mkfig_cdistance.py`.

Characteristic distances to $L = 50$ reference points, drawn as Section 17.2 prescribes, are estimated in each window. Fig. 17.3 shows the three nearest the attractor against a. Each is a continuous function of a within a chaotic band and jumps at the periodic windows, most visibly near $a = 0.28$ and $a = 0.32$, as Section 17.2 anticipates. The 786×50 tracking matrix $\boldsymbol{Y}$ is decomposed by SOD; Fig. 17.4 shows the result. The first SOV, 1.05×10^3, is an order of magnitude above the second, 1.2×10^2 (Fig. 17.4(a)), and the corresponding SOC (Fig. 17.4(c)) follows the drift of a (Fig. 17.4(b)) monotonically, with correlation coefficient $r = 0.971$ (Fig. 17.4(d)).

The relation is not affine, and the statement of Section 17.2.1 predicts that it should not be. The drift is a quarter-period sinusoid, for which the slowest mode of the record, $\cos(\pi\ell/(N-1)) = 1 - 2\,((a_\ell - 0.1)/0.3)^2$, is a quadratic in a, not an affine function of it. Fig. 17.4(d) is monotone but visibly curved. The residual about the least-squares line has a standard deviation of 6.8 % of the range of SOC_1, and a quadratic in a explains 99.7 % of the variance of SOC_1, against 94.3 % for the affine fit. The curvature is systematic, not scatter. Lengthening the windows from 5000 to 20,000 points, with the reference points drawn by the same rule, gives $r = 0.958$ and a quadratic fit of 99.9 %. Adding features does not straighten the relation either: with $L = 200$ reference points $r = 0.948$, and 0.934 and 0.945 for two further draws of the reference set, with the quadratic fit between 99.0 and 99.5 % (`python/chk_track_rerun.py`). SOC_1 is nevertheless not the pure slowest mode: its correlation with $\cos(\pi\ell/(N-1))$ is 0.895 at $L = 50$ and 0.858 at $L = 200$, so the span shapes the result as well as the drift law. With $b = 0.6$ and $c = 6.0$ the attractor grows strongly nonlinearly with a (the z excursion rises from 2.4 at $a = 0.1$ to 67 at $a = 0.4$), and most reference points sit far outside it, where by Eq. (17.11) they report the same nonlinear growth of $\mathrm{tr}\,\boldsymbol{C}$. These checks do not separate the two causes. The Duffing example below, with a linear drift, gives $r = 0.9994$. The point the example makes is unaffected: one coordinate out of fifty features recovers the hidden parameter monotonically across every periodic window the drift crosses. Whether it does so affinely is a property of the system and of the reference set, exactly as Section 17.2.1 says.

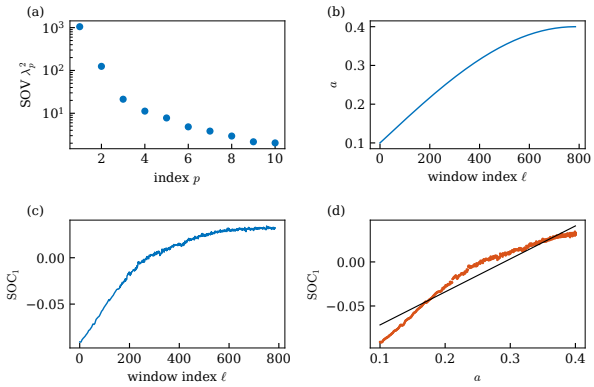


Figure 17.4: SOD of the 786 × 50 Rössler tracking matrix. First ten SOVs (a), parameter a against window index (b), dominant SOC$_1$, the coordinate of the largest SOV (c), and SOC$_1$ against a (dots) with the least-squares affine fit (line) (d). The correlation coefficient between a and SOC$_1$ is $r = 0.971$; the departure from the line in (d) is systematic, not scatter. Generated by `python/mkfig_track2.py`.

Duffing oscillator. Only the scalar record $x(t)$ of the two-well Duffing oscillator

$$\ddot{x} + \gamma\dot{x} + \alpha x + \beta x^3 = f\cos\omega t \tag{17.23}$$

is used, with $\gamma = 0.25$, $\alpha = -1$, $\beta = 1$, and $\omega = 1$ fixed and the forcing amplitude f as the slow variable. The bifurcation diagram is Fig. 17.5. The drift is linear from $f = 0.35$ to 0.4, a range in which the response is chaotic apart from narrow periodic windows (one is visible as the spike near $f \approx 0.375$ in Fig. 17.6).

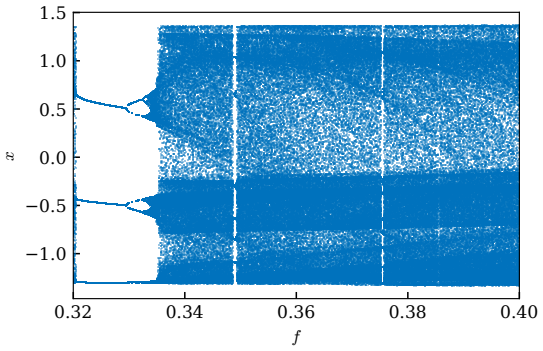


Figure 17.5: Bifurcation diagram of the two-well Duffing oscillator against the forcing amplitude f: x on the Poincaré section at forcing phase zero, 200 cycles kept after 400 discarded, at each of 1200 values of f. The chaotic band is entered by period doubling near $f = 0.334$; the periodic window near $f = 0.375$ is the one the drift of Fig. 17.6 crosses, the window near $f = 0.348$ lying just below the range used. Generated by `python/mkfig_duffing_bifu.py`.

The first 50 forcing cycles are discarded, and 12,000 steady-state points, about 166 forcing cycles, are recorded for each forcing amplitude with sampling time $\pi/36$, that is, 72 samples per

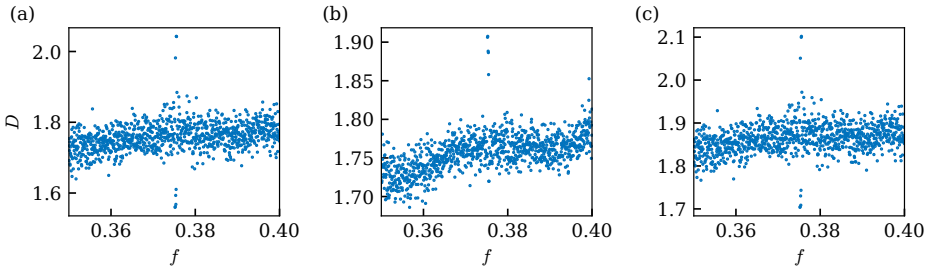


Figure 17.6: Characteristic distances from three of the $L = 400$ reference points in the reconstructed phase space of the Duffing oscillator ($d = 4$, $\tau = 53$), the three nearest the attractor, against f. The single outlier in each panel, at $f = 0.3755$, is the periodic window of Fig. 17.5. $N = 1001$ windows of 12,000 points. Generated by `python/mkfig_cdistance_duff.py`.

forcing period. (12,000 points leave the spike at the periodic window clearly above the window-to-window scatter, which is what Fig. 17.6 has to show.) The phase space is reconstructed with delay $\tau = 53$ samples and embedding dimension $d = 4$ [4, 105]. The delay is 0.74 of a forcing period. Average mutual information on this oscillator decays slowly and has shallow minima. Computed for $f \in [0.35, 0.4]$ at this sampling rate, its first minimum lies between 42 and 48 samples (0.58 to 0.67 of a period). Chapter 14 uses $\tau = 7$ at 30 samples per period (0.23 of a period) for the same oscillator at $f = 0.3$. The delay differs from the $\tau = 7$ that Chapter 14 uses on the same oscillator, and the two are not the same lag in different units. At this chapter's 72 samples per period, $\tau = 53$ is 0.74 of a forcing period, 4.63 time units; at Chapter 14's 30 samples per period, $\tau = 7$ is 0.23 of a period, 1.47 time units. They differ by a factor of three in time.

Neither is the first minimum of the average mutual information, and the reason is that this oscillator does not have one worth the name. Computed on 40,000-point records (`python/chk_metrics.py`), the first minimum sits at $\tau = 47$ at $f = 0.35$ and moves to 42 at $f = 0.40$, with a second at 59 and 57; the curve between them is flat, I varying from 0.33 to 0.28 across the whole span from 0.6 to 1.0 of a period. A single lag held fixed across the sweep cannot sit at the minimum for every f, and $\tau = 53$ sits inside the flat region for all of them.

Chapter 14's shorter lag is chosen against a different constraint. Its smooth filter follows strands of $2l+1 = 11$ points, and a strand has to span well under a forcing period for the trajectory through it to be nearly straight; at 0.23 of a period the eleven-point strand covers 2.3 periods at $\tau = 7$ against 8.1 at the equivalent of $\tau = 53$. The two chapters optimize different things, and both say which. Fig. 17.6 shows the characteristic distances to three randomly chosen reference points in the reconstructed space.

Four hundred reference points give a 1001×400 tracking matrix, and SOD of it gives Fig. 17.7. Here the first SOC *is* affine in the slow variable, with $r = 0.9994$ and an affine-fit residual of 1.0 % of the range (Fig. 17.7(d)), and the first SOV, 1.4×10^3, stands a factor 27 above the second. The contrast with the Rössler example needs care. The drift is linear in time, and for a linear drift the slowest mode of the record is $\cos(\pi\ell/(N-1))$, not f; that mode correlates with f at only 0.993 (Section 17.2.1). SOC_1 correlates with f at 0.9994 and with the cosine at 0.992 (`python/chk_track_rerun.py`), so it is the affine coordinate and not the slowest mode. A plausible reading, not tested here, is that the nonlinear directions of the span carry less smooth variation than

the window-to-window scatter, so that the affine direction is the slowest one SOD can resolve. It is consistent with this that the Duffing response does not change size appreciably over $f \in [0.35, 0.4]$, so the features are close to linear in f. The point-to-point scatter of each feature in Fig. 17.6 is the finite-window estimation error of Section 17.1.2; it is uncorrelated between windows and therefore rough, and SOD averages it away across the 400 columns. The one periodic window in the range is a jump in the sense of Section 17.2.1, and $L = 400$ leaves ample room orthogonal to it.

17.4 Experimental Data Example

The experimental test uses a modified version [42] of the two-well magneto-elastic oscillator [142], described in [36, 37] and sketched in Fig. 17.8. Two electromagnets, powered by a computer-controlled supply, perturb the magnetic potential at the free end of a vibrating cantilever beam. Two laser vibrometers measure near the clamp: CH2 reads the frame and CH1 the beam just above the notch, and their difference gives the beam position. A small notch is machined in the beam between the stiffeners and the clamped end, where it concentrates stress and initiates a fatigue crack. The beam is driven harmonically at 10 Hz with an amplitude set so that the response is nominally chaotic cross-well motion at full electromagnet load. The record is low-pass filtered at 50 Hz and sampled at 160 Hz — the same rig, and the same acquisition settings, as the record of Section 1.4.1.

Two things drift slowly in this experiment, and only one of them is interesting. The electromagnet supply voltages $v_1(t)$ and $v_2(t)$ are varied harmonically at a 2 : 1 frequency ratio, so their phase portrait is a figure-eight (Fig. 17.9(c)). They are *operating conditions*: known, imposed, and of no interest in themselves. A notch machined in the beam concentrates stress there, and a fatigue crack

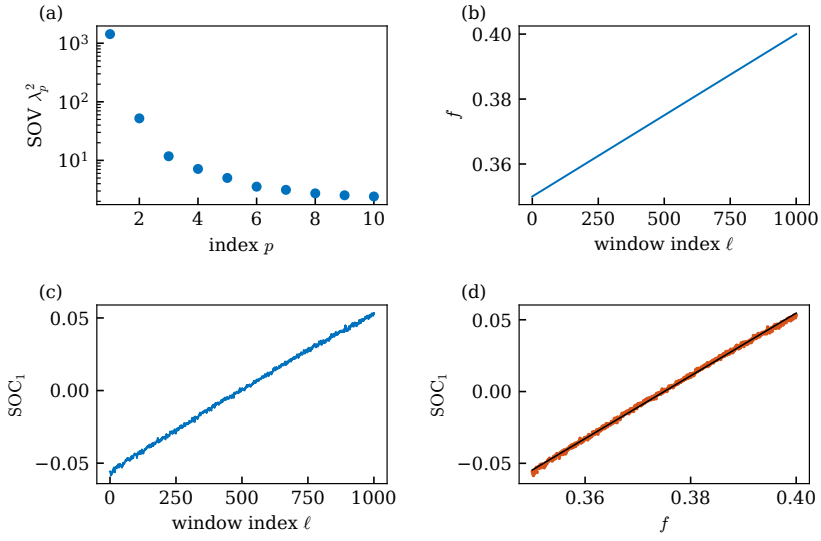


Figure 17.7: SOD of the 1001 × 400 Duffing tracking matrix. First ten SOVs (a), parameter f against window index (b), dominant SOC$_1$ (c), and SOC$_1$ against f (dots) with the least-squares affine fit (line) (d). The correlation coefficient between f and SOC$_1$ is $r = 0.9994$, with an affine-fit residual of 1.0 % of the range. Generated by `python/mkfig_track3.py`.

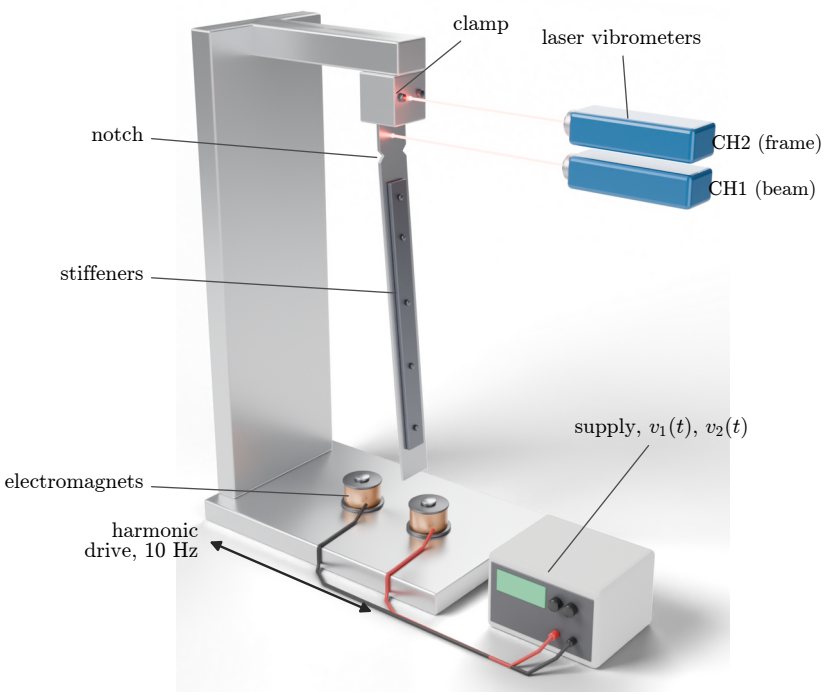


Figure 17.8: The fatigue rig (rendered model, not to scale). A notched cantilever beam hangs from a clamp on an inverted-L frame whose base is driven harmonically at 10 Hz; stiffeners make the beam rigid below the notch, so it bends between the clamp and the stiffeners. Two electromagnets under the free end, powered by a programmable supply, impose the operating conditions $v_1(t)$ and $v_2(t)$. Laser vibrometer CH2 reads the frame at the clamp and CH1 the beam just above the notch; their difference is the beam position. Scripts `python/render_rig_blender.py` and `python/mkfig_rig.py`.

grows through the run until the beam fractures at about 12.5 hours. That is the variable one wants, and it is not measured. Both drift on the same time scale, which is what makes the experiment a test: a method that merely finds *slow* structure will return the voltages and stop.

The run of [37] lasts 12.28 hours and produces about 7 million samples per channel. The fast-time phase space is reconstructed in five dimensions with a delay of six samples ($t_s = 1/160$ s) from a reference model built on the first 2^{15} points. The record is cut into $N = 1728$ windows of 2^{12} points, one every 25.6 s, the reference phase space is partitioned into 81 hypercuboids of roughly equal population, and five averaged PSW functions per region give a 1728×405 tracking matrix $\boldsymbol{Y}$. That matrix, not the raw vibration record, is the file `data/psw_fatigue_2005.mat` used here; the fast-time half of the procedure is taken as given and the example begins at step 4 of Section 17.2.3.

Stage one: the operating subspace. SOD is applied to the first 1200 rows, the healthy stage, over which the crack has not grown appreciably. Two SOVs stand clear of the rest, $\lambda_1 = 417$ and $\lambda_2 = 74$ against $\lambda_3 = 9.5$, a factor of eight (Fig. 17.9(a)). Their SOCs are the two smooth signals of Fig. 17.9(b).

The SOCs are known only up to a linear map and only as a *basis* of the smooth subspace, not as the individual voltages. Two comparisons are therefore proper and one is not. The improper one is

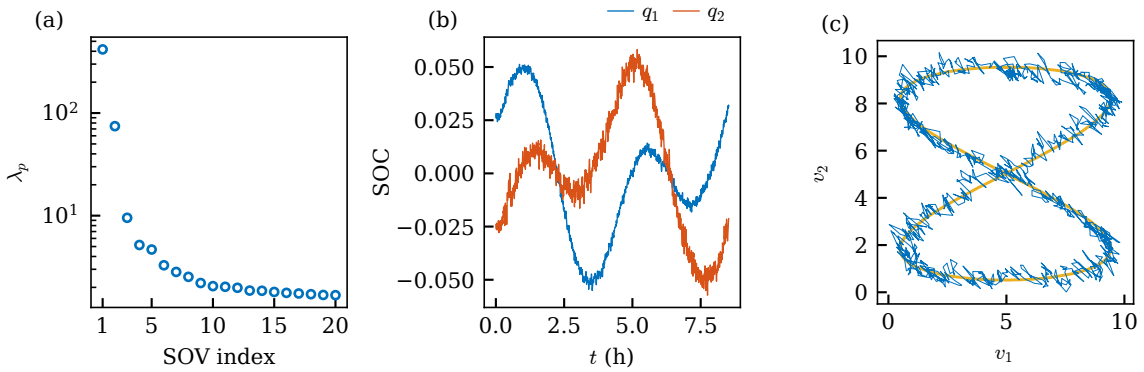


Figure 17.9: Stage one, the operating subspace, from the first 1200 rows of the 1728×405 tracking matrix. (a) The twenty largest SOVs: two stand clear, 417 and 74 against 9.5. (b) Their two SOCs against time. (c) The measured voltage portrait (thick, light) and the portrait reconstructed from the two SOCs through the least-squares affine map fitted on the same rows (thin, dark); $R^2 = 0.989$ for each voltage, and the principal angles between the two planes are 0.073 and 0.063 rad. Data of [37]. Generated by `python/mkfig_psw_track.py`.

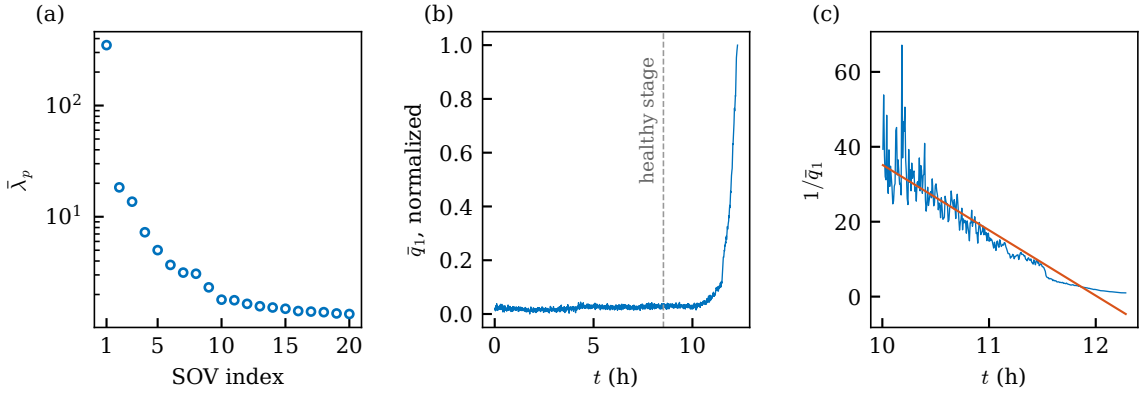


Figure 17.10: Stage two, the damage, after the two operating modes of Fig. 17.9 have been projected out of the whole tracking matrix. (a) The twenty largest SOVs of the truncated matrix: one dominates, 349 against 18. (b) Its SOC over the whole run, normalized to $[0, 1]$; the dashed line marks the end of the healthy stage used in Fig. 17.9. (c) The reciprocal over the last two hours, with the least-squares line the Paris law predicts: slope -17.5 h^{-1}, $r = -0.94$. Generated by `python/mkfig_psw_track.py`.

a correlation of SOC_1 with v_1: it depends on which basis of the subspace the algorithm happened to return. The proper ones are the angle between the two subspaces and the goodness of the affine map between them. The principal angles between the plane the voltages occupy in feature space and the plane spanned by the two SOCs are 0.073 and 0.063 rad, four degrees and less; and the least-squares affine map from $(\mathrm{SOC}_1, \mathrm{SOC}_2)$ to (v_1, v_2), fitted on the healthy stage, returns the two voltages with $R^2 = 0.989$ each. Fig. 17.9(c) shows the result: the reconstructed figure-eight, noisy, sitting on the measured one. The two-dimensional operating state is recovered as a linear image of itself. That two SOVs dominate is consistent with two independent slow variables and, by Section 17.2.1, it is also all the SOVs can say; the identification rests on the recovered plane matching the known one.

Stage two: the damage, in what is left. The first two smooth orthogonal modes span the operating conditions, so the damage, if it is anywhere, is in their complement. The tracking matrix is projected onto the remaining modes, $\bar{\boldsymbol{Y}} = \boldsymbol{Y}\left[\hat{\boldsymbol{\psi}}_3, \ldots, \hat{\boldsymbol{\psi}}_{405}\right]$, and SOD is applied again, now to the whole record. One SOV dominates the second decomposition, $\bar{\lambda}_1 = 349$ against $\bar{\lambda}_2 = 18$, a factor of 19 (Fig. 17.10(a)): one scalar slow process is left in the complement. Its SOC, Fig. 17.10(b), is flat for ten hours and then rises steeply — the shape of a fatigue crack that spends most of its life short.

The check is that this shape is the right one. The Paris law for macroscopic crack growth in this material makes the reciprocal of the damage variable decay linearly in time, and Fig. 17.10(c) shows $1/\bar{q}_1$ over the last two hours against a least-squares line: slope $-17.5\ \mathrm{h}^{-1}$, $r = -0.94$. The fit is good but not clean, and the departure is systematic rather than random, a bend near 11.5 h where the curve steepens. Two readings are open. Paris-law growth is a model for a crack already macroscopic, so a departure as the crack approaches the beam's width is expected rather than a defect of the tracking. And $\bar{q}_1$ is a monotone function of the crack length, not the crack length itself; nothing in the method fixes the scale, so a bend in $1/\bar{q}_1$ is a bend in that unknown function as much as in the physics. What the example establishes is the separation, not a calibrated crack length.

The order matters and is worth stating plainly. Run on the raw tracking matrix, SOD returns the voltages: they are the smoothest thing present and they are larger. The damage is recovered only after the operating subspace has been identified on a stretch of record where the damage is known to be small and then projected out. This is the general shape of the problem. When a nuisance drift and the drift of interest share a time scale, no ordering by smoothness separates them, and something outside the decomposition — here a healthy stage — has to supply the split.

The two-well oscillator record of Section 1.4.1 does not substitute for this experiment, and the reason is worth stating. That record is a single ten-minute block from the fatigue-damage tracking run of Chelidze and Liu [35], taken in the same laboratory on the same class of oscillator, and it is stationary over its own length by the block test given there. Tracking needs the opposite: a sequence of such blocks spanning the drift, so that one feature vector can be formed per block and the columns of a tracking matrix can be built. One block gives one column. What the record does supply is the fast-time half of the procedure — an embedding dimension, a delay, and the invariants of Chapters 11 and 12 measured on a block of the length this method would use — and it fixes the window length question of step 2 of Section 17.2.3 in one direction: 6000 forcing periods is long enough for those invariants to settle.

The features are not the only ones this record admits. The raw strain record behind the tracking matrix survives: 7,080,000 samples at 160 Hz, 12.29 hours, of which the first 1728×2^{12}

make the 1728 windows above and 13 seconds are left over at the end. The characteristic distances of Section 17.1.2 can therefore be computed on it directly and put through the same two stages, as a check on both the feature set and the procedure. With $L = 300$ reference points drawn by the rule of Section 17.2.3 in the five-dimensional reconstruction, the healthy stage gives SOVs 319 and 71 against 8.5; the two SOCs return the voltages with $R^2 = 0.989$ each and principal angles 0.075 and 0.068 rad; and the truncated matrix gives one SOV of 433 against 25, whose SOC has the same reciprocal decay over the last two hours, $r = -0.94$. Every one of those numbers matches the PSW result to the precision either is quoted at.

That is worth more than a consistency check. The PSW features of the source paper cost a reference model and a partition of the reconstructed phase space; the characteristic distances cost one distance evaluation per point and no model at all. On this record the two carry the same slow information.

Where the reference points go. With the raw record in hand the prediction of Section 17.1.3 can be put to a measured test. Table 17.1 runs the same two stages on five feature sets built from the same windows: the 300 mixed reference points above, their near and far halves split at the median distance from the attractor, a shell of 300 points at eight bounding-box half-widths — about seven attractor radii, squarely in the far field of Eq. (17.11) — and, as the baseline the expansion points at, the twenty moment features of each window, its five means and the fifteen distinct entries of its covariance. Two controls accompany it: how much of each tracking matrix is a function of those twenty moments, and what happens to the characteristic distances when only twenty of them are kept. SOD is invariant to scaling the columns individually, since a column scaling multiplies $\boldsymbol{Y}$ and $\boldsymbol{DY}$ alike, so no standardization is needed and none is done.

Table 17.1: The two stages of this section on five feature sets built from the same 1728 windows of the raw record. ρ is the distance from a reference point to the attractor in units of the attractor's RMS radius. The angles are the principal angles between the plane the voltages occupy and the plane of the two leading SOCs; the last column is the correlation of $1/\bar{q}_1$ with time over the final two hours. Computed by `python/chk_farfield_exp.py`.

features	L	healthy SOVs	R^2 on v_1, v_2	angles (rad)	damage SOVs	Paris r
PSW [37]	405	417, 74	0.989, 0.990	0.073, 0.063	349, 18	−0.942
all reference points	300	319, 71	0.989, 0.988	0.075, 0.068	433, 25	−0.940
near half, $\rho < 1.10$	150	103, 14	0.972, 0.970	0.102, 0.044	144, 11	−0.864
far half, $\rho > 1.10$	150	87, 14	0.974, 0.965	0.102, 0.053	103, 11	−0.920
far field, $\rho \approx 7$	300	271, 44	0.987, 0.989	0.063, 0.053	245, 23	−0.927
moment features	20	1.9, 1.6	0.374, 0.342	1.247, 0.127	1.2, 0.9	−0.638

Four readings, in order of what they settle.

First, the placement does not matter. The near and far halves return the same answer to within their own scatter, and the far-field shell — points from which the attractor subtends a few degrees — does as well as the full mixed set and better than either half. Fig. 17.11(a) shows the SOV spectra of the mixed and the far-field sets lying on top of each other; Fig. 17.11(b) shows the voltage portrait recovered from the shell alone. The expansion is not an approximation that costs something here. It is exact enough that a reference point seven attractor radii away is a perfectly good feature.

Second, the halves lose to the wholes, and the reason is count, not distance. Both halves have

$L = 150$ and both give $R^2 \approx 0.97$ against 0.99 for either 300-point set. Section 17.2.1 predicts exactly this: the smooth part of the features has to fit in the complement of the jump subspace, and halving L halves the room available.

Third, the characteristic distances on this record are moment statistics, and that goes for the near ones too. Regressing each tracking matrix on the twenty moment features of the same windows gives $R^2 = 0.998$ for the near half, 0.999 for the far half and 0.99994 for the shell. The shell figure is the expansion confirmed rather than anything discovered: Eq. (17.11) is linear in $\boldsymbol{C}$, so a shell feature has to be a function of the first two moments. The informative number is the near half's 0.998. The bounding-box rule with a 50 % margin puts the closest reference point at 0.24 attractor radii, so the near half here is near the *outside* of the attractor and not inside it, and even there the features carry two parts in a thousand beyond the second moments. This record does not test reference points embedded in the measure. The toy problem of Section 17.2.2, where near points carry 0.6 % of their variance beyond the second moments, remains the only case in this chapter where they carry anything at all.

Fourth, the twenty moment features nonetheless fail — SOVs 1.9 and 1.6, no dominant pair, the recovered plane 1.25 rad from the voltage plane, the damage coordinate noise until the last half hour (Fig. 17.11(c)) — and the reason is that there are twenty of them. Twenty characteristic distances drawn at random from the 300 fail in the same way and by the same margin: SOVs 3.3 and 1.5 against the moments' 1.9 and 1.6, $R^2 = 0.38$ and 0.46 against 0.37 and 0.34. The recovery climbs with L and with nothing else: $R^2 \approx 0.34$ at $L = 10$, 0.42 at 40, 0.84 at 80, 0.96 at 150, 0.99 at 300, and the leading SOV climbs 2, 8, 27, 100, 319 over the same range. Three hundred nonlinear re-encodings of twenty numbers succeed where the twenty do not, and it is not because the re-encodings know anything more. What they buy is room, in the sense of Section 17.2.1: the smooth part of the features has to fit in the complement of the jump subspace, and with twenty columns there is almost none.

Fifth, and this is the one place the features of [37] are not interchangeable with the chapter's. The PSW tracking matrix regressed on the same twenty moments gives $R^2 = 0.469$, with a median of 0.43 column by column. More than half of its variance is something else. That is the difference between a statistic of the window's point cloud and a statistic of the window's *dynamics*: the PSW function is a one-step prediction residual against a reference model, and a residual sees the local map, not only the measure. On this record the extra content is not needed, which is why the two feature sets agree to three digits on everything in Table 17.1. On a record whose slow variable moves the dynamics without moving the first two moments, they would not agree, and the cheaper features would be the ones to fail.

That question is answerable on the simulated examples, where the reference points are ours to place, and the answer is that the placement makes almost no difference. Splitting the reference points of Section 17.3 at their median distance from the attractor and tracking with each half separately (`python/chk_farfield.py`) gives, for the Rössler record, $r = 0.975$ from the 25 near points and 0.966 from the 25 far ones, against 0.971 from all 50; for the Duffing record, 0.9977 and 0.9975 from 200 points each, against 0.9994 from all 400. The near half carries no advantage. The toy problem of Section 17.2.2 pushes this further: sixty points at eight bounding-box half-widths, where Eq. (17.11) certainly applies, give 0.9996, better than the near points there.

Two cautions on reading that. The reference points of the Duffing example reach only 1.1 attractor radii, so its split compares an inner half with an outer half rather than near points with genuinely far ones; the Rössler set reaches 3.2 radii, and the toy problem is the only test squarely

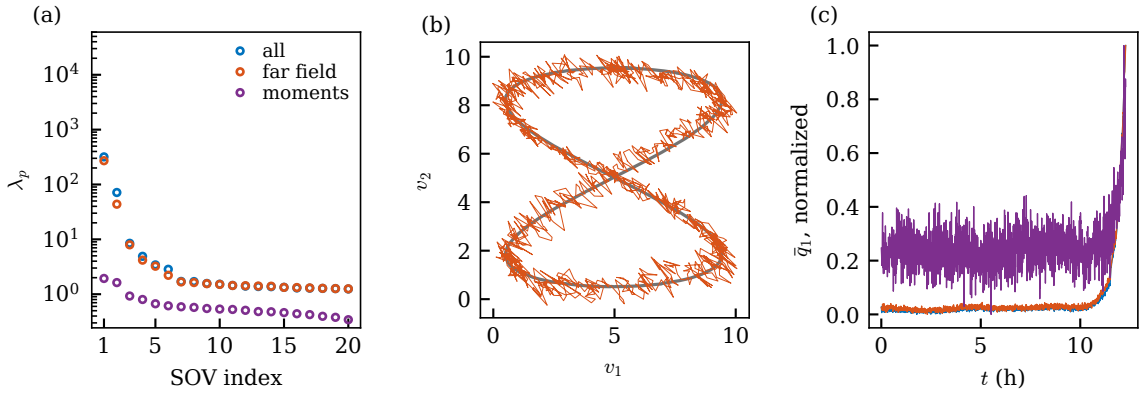


Figure 17.11: Where the reference points go, on the measured record. (a) Healthy-stage SOVs for the mixed reference points, the far-field shell at eight bounding-box half-widths, and the twenty moment features; the first two spectra coincide and the third has no dominant pair. (b) The measured voltage portrait (grey) and the one recovered from the far-field shell alone. (c) The normalized damage coordinate from each of the three feature sets; the mixed and far-field curves lie on top of each other. Generated by `python/mkfig_psw_farfield.py`.

in the far field. And the full set beats either half on the Duffing record, $\mathrm{SOV}_1 = 1.4 \times 10^3$ against 1.9×10^2, which is the feature count of Section 17.2.1 and not information the halves lack. The practical reading is that reference points need to be numerous and spread out, not close.

17.5 Noise Effects

Measurement noise enters the characteristic distance in a way that Eq. (17.11) makes explicit. Isotropic normally distributed noise is added to each coordinate of the Rössler trajectory of Section 17.3 at 1/5, 3/5 and 5/5 of the record's per-coordinate RMS (5.26), that is, at 20 %, 60 % and 100 % noise. Fig. 17.12 shows the characteristic distance to one reference point against a at the four noise levels. Each curve is shifted upward, by an amount that grows with the noise level and is nearly uniform in a, and the shape of the curve is preserved. That is the content of Eq. (17.13): isotropic noise of variance σ^2 adds $\sigma^2 \boldsymbol{I}$ to the covariance of the attractor and shifts D by about $(m-1)\sigma^2/(2\rho)$, a constant when σ does not change along the record. The prediction is quantitative. For the reference point of Fig. 17.12, at $\rho = 11.5$, the measured shifts at the three noise levels are 0.099, 0.888 and 2.366 against the predicted 0.096, 0.868 and 2.412; for a reference point at $\rho = 39.5$ they are 0.028, 0.250 and 0.692 against 0.028, 0.252 and 0.700. A uniform shift is removed by the centring step and does not affect the SOD. At 100 % noise the shift is no longer small compared with ρ and the higher-order terms of the expansion enter, which is why the near point overshoots the prediction at 60 % and undershoots it at 100 %. The spikes at the periodic windows are amplified, because there the attractor collapses to a few points and the noise supplies most of the transverse spread. The tracking results at the four noise levels are shown in Fig. 17.13. Even at 100 % noise the first SOC recovers the slow variable, for the reason just given: the noise moves the features almost uniformly and SOD is indifferent to a uniform move. The correlation of SOC_1 with a is 0.971, 0.979, 0.986 and 0.987 at the four levels — it does not fall, and rises slightly, plausibly because the noise rounds off the jumps at the periodic windows, which add to the departure from the affine fit

discussed in Section 17.3. What does fall is the leading SOV, from 1.05×10^3 to 3.4×10^2, 9.3×10^1 and 4.4×10^1, since the window-to-window scatter of every feature grows with σ. The SOV, not the correlation, is what noise degrades, and it is the SOV that the stationary null of Section 17.2.3 is compared against.

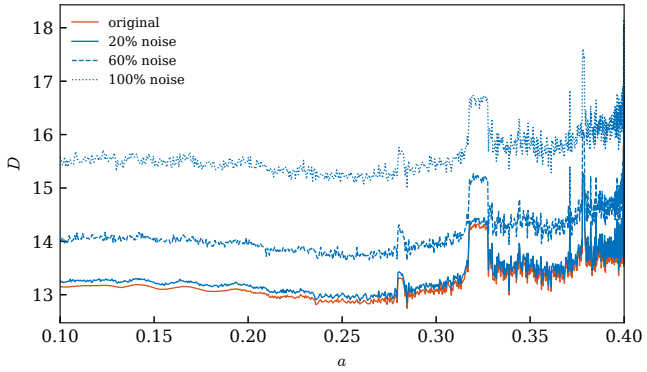


Figure 17.12: Characteristic distance to one reference point (the one nearest the attractor, $\rho = 11.5$) against a for the Rössler record without noise and with isotropic Gaussian noise at 20 %, 60 % and 100 % of the per-coordinate RMS. Generated by `python/mkfig_noise.py`.

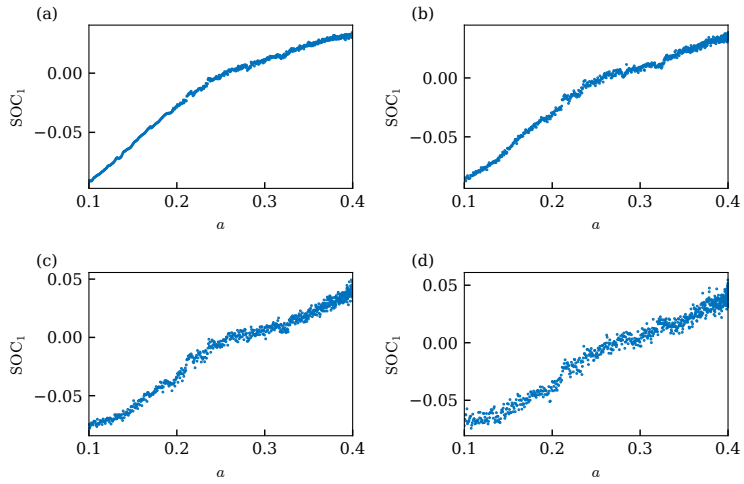


Figure 17.13: Tracking results for the Rössler record: SOC_1 against a at (a) no noise, $r = 0.971$; (b) 20 % noise, $r = 0.979$; (c) 60 % noise, $r = 0.986$; (d) 100 % noise, $r = 0.987$. The leading SOV falls from 1.05×10^3 to 4.4×10^1 across the four. Generated by `python/mkfig_track.py`.

The noise-free value is not recovered, only the shape. A characteristic distance reported as an absolute number is therefore biased upward by noise, by an amount that Eq. (17.13) estimates for far reference points if σ is known. Reported as a tracking coordinate it is not biased at all, since the coordinate is defined only up to offset. The noise floor that matters for tracking is the window-to-window scatter of the features, which grows with σ and, when the feature's autocovariances are

summable, shrinks as $n^{-1/2}$ once the window exceeds the feature's correlation time, and which SOD averages across the L columns (Problem 17.5).

17.6 Discussion

The characteristic distance is the cheapest invariant in this book. It is a sample mean, it converges as $n^{-1/2}$ once the window exceeds the feature's correlation time (given summable autocovariances), it needs no scaling region and no fit, and it costs one distance per point. What it sees depends on where it is measured from. From far away it sees the first two moments of the attractor, no more than the mean and variance that every monitoring system already reports. From inside it sees the shape of the natural measure. Its use for tracking does not rest on seeing more than the moments. It rests on supplying many features cheaply, so that the dimension count of Section 17.2.1 comes out in the analyst's favor.

SOD recovers the slow variable from features that are individually discontinuous because the discontinuities live in a subspace of dimension at most J, the number of bifurcations crossed, and the centred features have numerical rank well above J. In the complement of the jump subspace every coordinate is continuous, and among those with finite derivative energy the roughness functional selects the one slowest in time. Whether that coordinate is affine in the slow variable depends on the feature span and on the drift law. It is nearly affine in the Duffing simulation, with a linear drift, and the experiment's operating plane is recovered as an affine image of the voltages. It is a near-quadratic in the Rössler simulation, whose quarter-sine drift makes a quadratic in a the slowest mode of the record. The number of reference points is set by how many jumps have to be spanned and how much smooth signal has to be resolved against the scatter, not by the dimension of $\boldsymbol{\phi}$. Fifty points suffice in the Rössler simulation and the experiment needs 300. The characteristic position of Eq. (17.10) is cheaper in reference points because each contributes m columns: twenty points give $r = 0.964$ against a in the Rössler simulation (Fig. 17.14(a)), as good as fifty points with the characteristic distance. On the measured record sixty position points replace three hundred distance points and lose nothing (Fig. 17.14(b)): healthy-stage SOVs 314 and 72 against the distance features' 319 and 71, $R^2 = 0.990$ and 0.991 on the two voltages against 0.989 and 0.988, principal angles 0.065 and 0.052 rad against 0.075 and 0.068, and a Paris correlation of -0.957 against -0.940. Both matrices are 1728×300. That is the same arithmetic as the fourth reading of Section 17.4 seen from the other side: what the method needs is columns, and the characteristic position supplies five of them per reference point where the characteristic distance supplies one. It is a moment statistic too, regressing on the twenty moment features of Section 17.4 with $R^2 = 0.998$.

Three limits remain. SOD returns a coordinate, not a parameter: the SOC is known up to scale and offset, and only as a basis of the smooth subspace when the slow state has more than one component. Physical units need a calibration the method does not supply. The invariance that licenses the construction is per attractor, so a window that visits two coexisting attractors has no single characteristic distance (Problem 17.1). And the window length is squeezed from both sides: the record length within a window sets the scatter of each feature, while the drift rate sets a ceiling on the window; Problem 17.4 works the trade-off out. The stationary null of the procedure is the check that the smooth coordinate found is a drift and not the smoothest arrangement of that scatter.

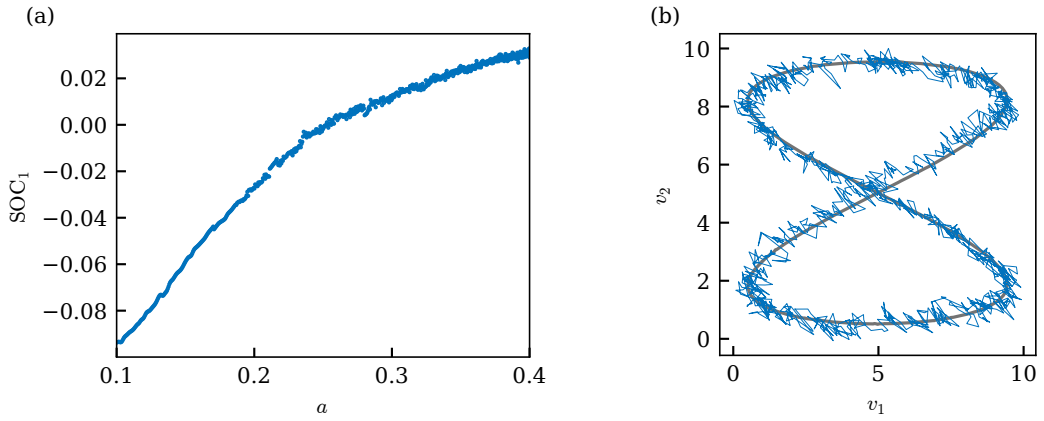


Figure 17.14: Tracking with the characteristic position, which gives m columns per reference point instead of one. (a) The Rössler parameter a recovered from twenty reference points, each contributing $m = 3$ columns, so a 786×60 tracking matrix; the correlation of SOC_1 with a is 0.964, against 0.971 for fifty reference points and the characteristic distance. (b) The measured record of Section 17.4: the voltage portrait (grey) and the one recovered from the two smoothest SOCs of a 1728×300 position tracking matrix built on sixty reference points, $R^2 = 0.990$ and 0.991. Generated by `python/mkfig_position.py` and `python/mkfig_position_exp.py`.

Problems

These problems build the metrics of this chapter, check the invariance that licenses them, and then put them to the use they were designed for: recovering a slow variable that is never measured directly. As throughout, use MATLAB or Mathematica.

Problem 17.1

Core

The characteristic distance $D(\boldsymbol{y})$ of Eq. (17.9) is asserted to be an invariant quantity of the attractor by the Birkhoff ergodic theorem under the ergodicity assumption of Section 17.1. Test that assertion rather than assuming it.

1. Generate a long Lorenz trajectory and embed it. Fix a reference point $\boldsymbol{y}$ somewhere off the attractor and compute the running average $\frac{1}{n}\sum_{i=0}^{n-1} f(\mathcal{T}^i(\boldsymbol{x}))$ as a function of n. Plot it and identify the n at which it has settled to three significant figures.

2. Repeat from ten different initial conditions $\boldsymbol{x}$ on the same attractor, keeping $\boldsymbol{y}$ fixed. Do the limits agree? Quantify the spread.

3. Now start from an initial condition in a different basin (use the two-well Duffing system, which has two coexisting attractors for suitable parameters). Does $D(\boldsymbol{y})$ take the same value? Relate your answer to the discussion of ergodic decomposition in Chapter 11: an invariant measure need not be ergodic, and D is invariant only per attractor.

4. How does the convergence rate in part (1) compare with that of the correlation dimension estimate for the same record? Which is the cheaper invariant to estimate, and why?

Problem 17.2

Computational

The generalized distance $d_q(k)$ is claimed to carry information about nonlinear correlations and to be useful for selecting a delay. Put both claims to the test.

1. Implement $d_q(k)$ for a scalar record. Compute $d_2(k)$ for the Lorenz $x(t)$ series over a range of k and plot it against k.

2. Plot $\sqrt{2m\,(R_{xx}(0) - R_{xx}(k))}$ for the same record on the same axes. By Eq. (17.6) and Jensen's inequality this is an upper bound on $d_2(k)$. Where is the bound tight and where does the gap open? Interpret the gap as a property of the distribution of k-step displacements.

3. Compute $d_q(k)$ for $q = 1, 2, 4$ and $q \to \infty$ (the last being the Chebyshev distance). Which q gives the sharpest first minimum?

4. Use the first minimum of $d_q(k)$ to select a delay τ and compare it with the delay you would choose from the first minimum of the average mutual information of Chapter 9. Do they agree? If not, embed with each and judge the two reconstructions by the false-nearest-neighbor fraction of Chapter 10.

5. The text warns that the information in this metric becomes redundant once the characteristic length reaches the attractor size. Identify that k in your plot and explain the resemblance to the saturation of Δ_n in Section 12.3.1.

Problem 17.3

Computational

The characteristic position $\boldsymbol{P}(\boldsymbol{y})$ of Eq. (17.10) averages a *unit* vector rather than a distance, so it is vector-valued and carries directional information that $D(\boldsymbol{y})$ discards.

1. Implement $\boldsymbol{P}(\boldsymbol{y})$ and compute it for the Lorenz attractor with $\boldsymbol{y}$ at several positions: inside one wing, between the wings, and far outside the attractor.

2. For each, report $\|\boldsymbol{P}(\boldsymbol{y})\|_2$. What does a value near zero mean geometrically, and what does a value near one mean?

3. Construct a case in which two distinct attractors give the same $D(\boldsymbol{y})$ but different $\boldsymbol{P}(\boldsymbol{y})$. This is the sense in which the vector metric is strictly more informative.

4. How sensitive is $\boldsymbol{P}(\boldsymbol{y})$ to the choice of $\boldsymbol{y}$? Sweep $\boldsymbol{y}$ along a line and plot each component.

Problem 17.4

Computational

This is the application the chapter is built toward: recovering a slow variable from fast-time measurements alone.

1. Build the coupled fast–slow system of Section 17.2: take a two-well Duffing oscillator as the fast subsystem and let one of its parameters drift slowly and monotonically over the record, on a time scale at least two orders of magnitude longer than the fast oscillation.

2. Partition the record into windows short enough that the parameter is effectively constant within each, yet long enough for the running average of Problem 17.1 to converge. State the two competing requirements explicitly and justify your choice of window length — this trade-off is the crux of the method.

3. Compute $D(\boldsymbol{y})$ in each window and plot it against window index. Compare with the true parameter drift. Report the correlation.

4. Repeat with $d_q(k)$ at a fixed k and with $\|\boldsymbol{P}(\boldsymbol{y})\|$. Which tracks the drift best?

5. Now make the drift non-monotonic (let the parameter go up and come back down). Can the metric distinguish the outbound leg from the return? What does your answer imply about using these metrics for damage tracking, where the underlying variable is usually assumed monotonic?

Problem 17.5

Computational

Every metric in this book degrades with noise; the question is how, and at what level.

1. Add Gaussian measurement noise to the record of Problem 17.4 at signal-to-noise ratios of 40, 30, 20 and 10 dB.

2. For each, recompute the tracking curve of Problem 17.4(3) and report the correlation with the true drift. At what SNR does tracking fail?

3. Does the noise bias $D(\boldsymbol{y})$, inflate its variance, or both? Distinguish the two by comparing the mean and the window-to-window scatter against the noise-free case, and compare the bias with the prediction of Eq. (17.13) for a reference point far from the attractor.

4. Compare the noise robustness of $D(\boldsymbol{y})$ with that of the correlation dimension of Chapter 11 and the largest Lyapunov exponent of Chapter 12 on the same records. Rank the three, and explain the ranking in terms of what each quantity has to estimate from the data.

5. Apply a low-pass filter to "clean" the noisiest record and repeat. Does filtering help or hurt? Relate your answer to the caution in Section 12.5.

Problem 17.6

Extension

Finally, a check that any new metric should be made to pass.

1. Apply your $D(\boldsymbol{y})$ tracking procedure to a record with *no* drift at all: a stationary Lorenz series of the same length, windowed identically.

2. Plot the resulting curve. Does it look flat, or does it wander? Quantify the wander and compare its size with the signal you detected in Problem 17.4.

3. Generate surrogates of the drifting record by the methods of Chapter 16 and compute the tracking curve for each. Does the drift you detected survive?

4. In light of parts (2) and (3), state what would constitute evidence that a tracking curve reflects a real slow variable rather than the finite-window fluctuation of a stationary invariant measure.

Problem 17.7

Experimental

The measured fatigue record of Section 17.4. The file `data/psw_fatigue_2005.mat` holds the finished tracking matrix $\boldsymbol{Y}$, 1728×405, one feature vector per 25.6 s window over 12.28 hours, together with the time vector and the two electromagnet supply voltages. The fast-time half of the procedure is taken as given; this problem is steps 4 onward of Section 17.2.3. Every number quoted below is produced by `python/chk_psw_problems.py`.

1. Reproduce stage one. Apply SOD to the first $H = 1200$ rows, the healthy stage, and report the five largest smooth orthogonal values. Fit the two voltages to the leading two smooth orthogonal coordinates by least squares with an intercept, and report R^2 for each. You should find SOVs near 417 and 74 against a third of 9.5, and $R^2 \approx 0.988$ for both voltages. Plot the reconstructed voltage portrait against the measured one; it is the figure-eight of Fig. 17.9(c).

2. The choice $H = 1200$ is not derived anywhere. Repeat part (1) for $H = 800$, 1000, 1400, 1600 and 1728 and tabulate the two R^2 values against H. The recovery is flat to three decimal places out to $H = 1400$ and then collapses: at $H = 1728$ you should find R^2 near 0.68 and 0.58, and the leading principal angle between the recovered plane and the measured voltage plane near 1.5 rad. Explain what enters the matrix between 1400 and 1728 that the operating conditions do not account for, and state the rule you would give someone who does not know in advance where the healthy stage ends.

3. Stage two. Project the two operating modes found in part (1) out of the whole 1728-row matrix and apply SOD to the remainder. One value should dominate. Normalize its coordinate to $[0, 1]$, plot it over the run, and fit a straight line to its reciprocal over the last two hours, as the Paris law predicts. Report the slope and the correlation coefficient.

4. How many features does this need? Draw L columns of $\boldsymbol{Y}$ at random, for $L = 5$, 10, 20, 60, 150, and repeat part (1) on each, twenty draws per L. Plot the mean R^2 and its spread against L. You should find the mean rising monotonically from about 0.5 at $L = 5$ to about 0.97 at $L = 150$, with the spread shrinking. The features are all of one kind, so this curve isolates the effect of *count*. Relate it to the dimension count of Section 17.2.1: with L features and a jump subspace of rank J, how much room is left for the smooth part?

18
Recurrence and Complexity Measures

The two most quoted numbers in this book so far are the correlation dimension of Chapter 11 and the largest Lyapunov exponent of Chapter 12. Both are invariants of the dynamics. Both are also expensive. A correlation dimension needs enough points to populate a scaling region over a decade of ϵ; a Lyapunov exponent needs a record long enough for a fit window to open between the transient and the folding. Both assume the record is stationary, because an invariant of a system whose parameters drift is not an invariant of anything.

Experimental records are frequently none of those things. A bearing run to failure is nonstationary by construction. A gait record from a fatiguing subject is a few hundred strides, not a few hundred thousand samples. A machine on a shop floor is measured through a sensor of unknown calibration, in the presence of a noise floor that nobody controls. Applied to such records, the estimators of Chapters 11 and 12 do not fail loudly. They return a number, and the number is wrong.

This chapter collects the methods that work anyway. They are built on one observation: a bounded trajectory returns arbitrarily close to where it has been, and the *pattern* of those returns carries dynamical information that survives shortness, noise and slow drift. Recurrence plots make the pattern visible; recurrence quantification analysis (RQA) reduces it to numbers; ordinal and entropy measures reduce the same trajectory to numbers by a different route. All of them are cheap, all of them tolerate a few thousand points, and none of them requires the record to be stationary over its whole length.

The price is stated here once and repeated throughout. *These are not estimates of an invariant of the dynamics.* They are statistics of the reconstructed trajectory, computed at a threshold, an embedding and a window that the analyst chose. Change the threshold and the numbers change. Two records analyzed with different parameters cannot be compared, and no absolute value of determinism or permutation entropy means anything on its own. What these measures support is *comparison* — between states of one machine, between a record and its surrogates, between a healthy baseline and a degraded one — and Section 18.6 is about doing that defensibly. Their use as condition indicators in machine diagnostics is reviewed in [129], which is the organizing source for Sections 18.4 and 18.5 and uses the same forward delay convention as this book.

18.1 The recurrence matrix

Take the embedded trajectory of Chapter 8. From a scalar record $\{x_n\}_{n=1}^{N}$, delay coordinates give

$$\boldsymbol{x}_n \;=\; [\,x_n,\; x_{n+\tau},\; \ldots,\; x_{n+(d-1)\tau}\,]^{\mathrm{T}}\,, \qquad n = 1, \ldots, M \equiv N - (d-1)\tau\,, \tag{18.1}$$

with τ chosen by average mutual information (Chapter 9) and d by false nearest neighbours (Chapter 10). Nothing in this chapter changes that part of the workflow.

Recurrence matrix

Fix a threshold $\epsilon > 0$. The *recurrence matrix* of the trajectory Eq. (18.1) is

$$R_{ij} \;=\; \Theta(\epsilon - \|\boldsymbol{x}_i - \boldsymbol{x}_j\|)\,, \qquad i,j = 1,\dots,M\,, \tag{18.2}$$

where $\Theta(\cdot)$ is the Heaviside step function and $\|\cdot\|$ is the Euclidean or the maximum norm. Thus $\boldsymbol{R} \in \{0,1\}^{M\times M}$, and $R_{ij} = 1$ records that the system state at time j lies within ϵ of the state at time i. The *recurrence plot* is the image of $\boldsymbol{R}$ with the 1-entries marked.

The construction is due to Eckmann, Kamphorst and Ruelle [54], and Eq. (18.2) is their definition; the modern account, including almost everything in Section 18.3, is the review of Marwan *et al.* [134].

Three properties follow immediately and are worth stating, because they are what makes the plot readable. $\boldsymbol{R}$ is symmetric, $R_{ij} = R_{ji}$, so only one triangle carries information. Its main diagonal is identically one, since $\|\boldsymbol{x}_i - \boldsymbol{x}_i\| = 0$; this is the *line of identity*, and it is an artefact, not a measurement. And $\boldsymbol{R}$ is invariant under any relabelling of state space that preserves distances, so it depends on the geometry of the reconstructed attractor and not on the coordinates used to draw it.

18.1.1 Reading the textures

The information in a recurrence plot is carried by its line structures, and the vocabulary is small enough to learn in one sitting.

- A *diagonal line* of length ℓ at offset k means $\|\boldsymbol{x}_{i+p} - \boldsymbol{x}_{i+k+p}\| < \epsilon$ for $p = 0,\dots,\ell-1$: the trajectory revisited a neighbourhood it occupied k samples earlier *and then evolved similarly for ℓ samples*. Diagonal lines are the signature of deterministic dynamics. Their lengths are limited by how fast nearby trajectories separate.

- A *vertical line* (equivalently, by symmetry, a horizontal one) means the state changed slowly or not at all while time advanced: the trajectory was trapped in a small region. Vertical structure indicates laminar states, intermittency, or a slow manifold.

- An *isolated point* means a single chance encounter with no continuation. A plot made mostly of isolated points is the signature of a stochastic process, or of a deterministic one buried under noise larger than ϵ.

From these three elements the standard textures are read off.

- *Periodic.* Long uninterrupted diagonals, equally spaced, the spacing equal to the period. A period-p orbit gives diagonals at every offset that is a multiple of p.

- *Quasiperiodic.* Diagonals again, but with spacings that are not commensurate; the pattern is regular without repeating, and the offsets at which long diagonals occur are the near-recurrence times of the torus, the denominators of the continued-fraction convergents of the frequency ratio.

- *Chaotic.* Short diagonals of many lengths, distributed over the whole plot. The lines are short precisely because neighbouring trajectories diverge; the distribution of their lengths is the raw material of Section 18.3.

- *Drifting.* The plot fades away from the main diagonal: states far apart in time are no longer within ϵ of one another, because the system is no longer the same system. A recurrence plot detects nonstationarity by inspection, which is the one thing the estimators of Chapters 11 and 12 cannot do.
- *White noise.* A uniform speckle of isolated points at density $\approx$ the recurrence rate, with no line structure at all beyond the line of identity.

Fig. 18.1 shows the textures on eight records of equal length, all at the same recurrence rate and Theiler window, so that they can be compared directly. Two features of the figure are not in the list above. The AR(2) record of Chapter 3, panel (f), is a *linear* stochastic process and yet shows short diagonals and a determinism of 0.86: linear correlation alone produces line structure, which is why a determinism value is meaningful only against the surrogates of Section 18.6. And the i.i.d. record, panel (e), embedded in $d = 3$ with $\tau = 1$, has a determinism of 0.44 rather than the $2\,\mathrm{RR} = 0.04$ that isolated points at density RR would give (and do give at $d = 1$): consecutive delay vectors share $d-1$ coordinates, so a chance recurrence of two vectors is likely to be followed by a recurrence of their successors, and the embedding itself manufactures diagonal lines. The effect grows with d (0.26, 0.44, 0.64 for $d = 2, 3, 5$; `python/mkfig_rp_gallery.py`), it is the reason Thiel *et al.* [211] estimate invariants from unembedded recurrence plots, and it is one more reason a determinism is comparable only between records analyzed with the same d and τ.

Why the plot works on short records

A correlation-dimension estimate uses the pair distances only through the scalar count $C(\epsilon)$, and needs many points before that count is smooth enough to differentiate. A recurrence plot uses the same pair distances but keeps the time labels, so each of the $M^2/2$ entries is informative about the temporal structure. That is why a few thousand points give a readable plot while, for attractor dimensions above about three or in the presence of noise, the same few thousand points give a $\log C(\epsilon)$ versus $\log \epsilon$ curve with no usable scaling region (for the Lorenz attractor itself, $D_2 \approx 2.05$, a few thousand points do suffice, Chapter 11). The information was always there; thresholding at one ϵ and keeping the indices extracts a different part of it.

18.2 Choosing the threshold

Everything in Sections 18.1 and 18.3 depends on ϵ, and ϵ is not given by the data. This is the central practical problem of the chapter and it deserves to be met head-on rather than buried in a parameter table.

There are two defensible strategies.

Fixed radius. Choose ϵ as an absolute distance, or as a fixed fraction of the attractor diameter or of the standard deviation of the record. The threshold then has a physical meaning: it is a length in state space, and states closer than that length are declared equivalent. The disadvantage is that the resulting recurrence rate depends on how densely the trajectory fills its attractor, so two records with different amplitudes or different dimensions produce recurrence plots of very different density, and the RQA measures computed from them are not comparable.

Fixed recurrence rate. Choose ϵ separately for each record so that a prescribed fraction of the matrix entries recur — RR fixed at a few per cent is the usual choice. Every plot then has the same

density and the line structures can be compared across records. This is the right default when the object of the exercise is comparison, which in practice it almost always is. The price is that ϵ is now a function of the data, so it absorbs amplitude changes: a record whose amplitude doubles gets a threshold that doubles, and any change in the dynamics that acts through amplitude alone becomes invisible.

Neither strategy is correct in the abstract. What is not defensible is choosing ϵ once, by eye, on one record, and then reporting RQA measures for a set of records to which it was never re-examined.

Two records at once

The recurrence matrix generalizes to a pair of records. The *cross-recurrence plot* $CR_{ij} = \Theta(\epsilon - \|\boldsymbol{x}_i - \boldsymbol{y}_j\|)$ marks the times at which the trajectory of one record visits the neighbourhood of the other's state; it is not symmetric, has no line of identity, and requires the two records to live in the same state space with comparable scales. Its diagonal structures show when the two systems evolve alike, and a bowed main line shows a time-varying lag between them. The *joint recurrence plot* $JR_{ij} = R^{(x)}_{ij} R^{(y)}_{ij}$ marks the times at which *both* systems recur simultaneously, needs no common state space, and is the recurrence-based test for generalized synchronization. Marwan *et al.* [134] treat both; they are the recurrence answer to the two-channel question of Section 18.5.

Pitfall: the threshold is not a nuisance parameter

1. *No RQA measure is invariant to ϵ.* Determinism rises monotonically toward one as ϵ grows, because a large enough threshold makes every state recurrent with every other and the plot fills solid. It falls toward zero as ϵ shrinks, because eventually nothing recurs. Between those limits it varies smoothly and without any distinguished value. There is no plateau to find and no analogue of the scaling region of Chapter 11. A determinism of 0.87 is a statement about the record *and* the threshold, and reporting it without the threshold is reporting nothing.

2. *ϵ must sit above the noise floor.* If ϵ is smaller than the typical noise-induced displacement, a genuine recurrence is broken into isolated points and the diagonal lines disappear: a clean deterministic record reads as noise. If ϵ is far above it, the noise is invisible but so is most of the dynamics. The workable band is bounded below by the noise level and above by the attractor size, and if the record's signal-to-noise ratio is poor that band can be empty. The quantitative rule is due to Thiel *et al.* [209], who computed the effect of additive observational noise of standard deviation σ_{noise} on the RQA measures and found that the line structures are recovered once

$$\epsilon \gtrsim 5\,\sigma_{\text{noise}}\,, \tag{18.3}$$

   while for smaller ϵ the determinism collapses toward its noise value; Fig. 18.1(g) is the case $\epsilon \approx 1.5\,\sigma_{\text{noise}}$. If Eq. (18.3) cannot be met below the attractor size, apply the noise reduction of Chapter 14 first, and report that you did.

3. *The noise floor moves between records.* A threshold set on a quiet baseline record may lie below the noise floor of a later, noisier one. The recurrence rate then collapses and the apparent loss of determinism is an instrumentation artefact. Check the noise level of every record in a series, not just the first.

4. *Comparisons across ϵ are meaningless.* If two records were analyzed at different thresholds — which is exactly what the fixed-recurrence-rate strategy does — then differences in their determinism cannot be attributed to the dynamics without further argument. State the strategy, and if you use fixed recurrence rate, report the ϵ each record required. That sequence of thresholds is itself a diagnostic quantity.

18.2.1 Temporal correlation and the Theiler window

The same difficulty that spoils a correlation-dimension estimate spoils a recurrence plot, and the same remedy fixes it.

Points adjacent in time are close in state space for a trivial reason: the trajectory is continuous and the sampling interval is short. Such pairs are not recurrences. They are the same visit counted twice. In Chapter 11 they inflate $C(\epsilon)$ at small ϵ and bias the correlation dimension downward, and the remedy is the *Theiler window*: exclude every pair with $|i - j| < w$ from the count, where w is at least the decorrelation time of the record.

In a recurrence plot the same pairs form a thick band along the line of identity. That band is a picture of the sampling rate, not of the dynamics. Left in place it inflates RR, and — worse — it contributes one very long diagonal to the line-length distribution, which corrupts determinism, the mean diagonal length, the maximum diagonal length and the line-length entropy simultaneously.

So:

$$R_{ij} \;=\; \Theta(\epsilon - \|\boldsymbol{x}_i - \boldsymbol{x}_j\|)\;\Theta(|i-j| - w)\;, \tag{18.4}$$

with w taken as in Chapter 11: the first zero of the autocorrelation, the first minimum of the average mutual information, or a few times the estimated decorrelation time, whichever the analyst is willing to defend. Every RQA measure in the next section is to be understood as computed from Eq. (18.4) and not from Eq. (18.2).

> Excluding the line of identity alone is not enough. It removes one diagonal; the temporal-correlation band is $2w$ wide. Software that offers only a "exclude main diagonal" option is doing the wrong thing, and the effect on the mean diagonal line length is large when the record is oversampled.

18.3 Recurrence quantification analysis

RQA replaces the picture with numbers computed from the line structures identified in Section 18.1.1. Write $P(\ell)$ for the histogram of diagonal line lengths in the matrix Eq. (18.4), and $P_v(v)$ for the histogram of vertical line lengths. A minimum length $\ell_{\min}$ (and $v_{\min}$) must be fixed, below which a run of ones is not counted as a line; $\ell_{\min} = 2$ is the usual choice and, like ϵ, it must be reported.

18.3.1 The measures

Recurrence rate. The density of the plot,

$$\mathrm{RR} \;=\; \frac{1}{M^2}\sum_{i,j=1}^{M} R_{ij}\,, \tag{18.5}$$

the density measure of Eckmann *et al.* [54]. With the Theiler window in place the normalization should exclude the excluded pairs; otherwise RR is biased low by a fixed amount that depends on w and M and not on the dynamics. Under the fixed-recurrence-rate strategy of Section 18.2, RR is not a measurement at all — it is the constraint that sets ϵ.

Determinism. The fraction of recurrent points that lie on diagonal lines,

$$\mathrm{DET} \;=\; \frac{\sum_{\ell=\ell_{\min}}^{M} \ell\, P(\ell)}{\sum_{\ell=1}^{M} \ell\, P(\ell)}\,. \tag{18.6}$$

This is the workhorse. A deterministic system produces long diagonals and DET near one; a stochastic one produces isolated points and DET near zero. It is the measure most often used as a condition indicator, and the measure whose threshold dependence is most often left unstated.

Average and maximum diagonal line length.

$$L \;=\; \frac{\sum_{\ell=\ell_{\min}}^{M} \ell\, P(\ell)}{\sum_{\ell=\ell_{\min}}^{M} P(\ell)}\,, \qquad L_{\max} \;=\; \max\{\ell_k\}_{k=1}^{N_\ell}\,, \tag{18.7}$$

where N_ℓ is the number of diagonal lines found. L is the mean time over which two segments of the trajectory stay within ϵ of one another: a predictability time. $L_{\max}$ is the longest such episode in the record.

Entropy of the line-length distribution. With $p(\ell) = P(\ell)/\sum_{\ell' \geq \ell_{\min}} P(\ell')$,

$$\mathrm{ENTR} \;=\; -\sum_{\ell=\ell_{\min}}^{M} p(\ell)\,\ln p(\ell)\,. \tag{18.8}$$

This is the Shannon entropy of the diagonal-line histogram, and it measures the *diversity* of the line lengths, not the complexity of the signal. A periodic orbit has all its diagonals the same length and ENTR small. A chaotic orbit has many different lengths and ENTR large. Uncorrelated noise, having almost no lines at all, also has ENTR small — so the measure is not monotone in anything one would call disorder, and it must not be read as if it were.

Laminarity and trapping time. The vertical-line analogues of DET and L,

$$\mathrm{LAM} \;=\; \frac{\sum_{v=v_{\min}}^{M} v\,P_v(v)}{\sum_{v=1}^{M} v\,P_v(v)}\,, \qquad \mathrm{TT} \;=\; \frac{\sum_{v=v_{\min}}^{M} v\,P_v(v)}{\sum_{v=v_{\min}}^{M} P_v(v)}\,. \tag{18.9}$$

LAM is the fraction of recurrent points in vertical structures and detects laminar or intermittent states; TT is the mean duration of such a state. The pair is the reason to compute RQA rather than only DET: a system that alternates between laminar phases and bursts has high LAM and unremarkable DET, and intermittency is common in machinery faults where a defect engages only on part of each revolution.

$$\mathrm{DET}(\epsilon, w, \ell_{\min}) \;=\; \frac{\sum_{\ell \geq \ell_{\min}} \ell\,P(\ell\,;\epsilon,w)}{\sum_{\ell \geq 1} \ell\,P(\ell\,;\epsilon,w)} \tag{18.10}$$

The arguments are written out because they are not optional. Two determinism values computed at different $(\epsilon, w, \ell_{\min})$ are two different statistics, not two measurements of one quantity.

18.3.2 $L_{\max}$ and the largest Lyapunov exponent

The claim is often made that $L_{\max}$, or its reciprocal $\mathrm{DIV} = 1/L_{\max}$, estimates the largest Lyapunov exponent λ_1 of Chapter 12. The heuristic is sound and the claim as usually stated is too strong, so it is worth separating the two.

The heuristic: a diagonal line persists as long as two trajectory segments remain within ϵ. If the segments separate exponentially at rate λ_1 from an initial separation δ_0, they exceed ϵ after a time of order $\lambda_1^{-1}\ln(\epsilon/\delta_0)$. Long diagonals are therefore short when λ_1 is large, and DIV should track λ_1 across a family of systems.

What that argument does not give is an estimator. The initial separation δ_0 is set by the sampling and the threshold, so the logarithm is not a constant across records. $L_{\max}$ is the maximum of a distribution, so it depends on the record length M: a longer record has more opportunities for a long line and $L_{\max}$ grows without the dynamics having changed. And the whole quantity scales with ϵ. The defensible statement is that DIV is *monotonically related* to λ_1 within one family of records analyzed identically, and is useful for ranking. It is not a Lyapunov exponent, it carries no units unless the sampling interval is divided out, and it should never be reported as $\lambda_1 \approx \mathrm{DIV}$. Marwan *et al.* [134] make the same point: DIV is related to the largest positive Lyapunov exponent only in the loose sense above, and they direct the reader to the estimator that follows.

18.3.3 Correlation entropy from the line-length distribution

The rigorous bridge from a recurrence plot to a dynamical invariant does not go through $L_{\max}$ but through the whole distribution $P(\ell)$. A diagonal line of length ℓ at offset k is a pair of trajectory segments that stay within ϵ for ℓ consecutive samples, which is exactly the event counted by the correlation sum of an ℓ-fold delay embedding (Chapter 11). The Grassberger–Procaccia definition of the correlation entropy, Eq. (11.61), says that this count decays as $e^{-K_2 \ell \Delta t}$ when one more coordinate is added; hence the *cumulative* distribution of line lengths,

$$P^{c}(\ell) \;=\; \sum_{\ell' \geq \ell} P(\ell') \;\propto\; e^{-K_2\, \ell\, \Delta t}, \tag{18.11}$$

for ℓ large enough that the exponential regime has set in and ϵ small enough that K_2 has converged. The slope of $\ln P^{c}$ against $\ell \Delta t$ estimates K_2 in units of inverse time. The construction is due to Faure and Korn [62]; Thiel *et al.* [211] showed that K_2 and D_2 can both be read from a recurrence plot, and that the plot of the *unembedded* scalar record suffices for the purpose. Since $K_2 \leq h_{\mathrm{KS}} \leq \sum_{\lambda_i > 0} \lambda_i$ (Ruelle's inequality, Chapter 11), the estimate is bounded above by λ_1 for a system with one positive exponent, and it is the quantity that DIV gestures at.

The estimate inherits the finite-data problems of Chapter 11. At short ℓ the distribution is dominated by lines that end because the pair started near the edge of the ϵ-ball, not because of the dynamics, and the local slope is far too steep; the exponential regime appears only for $\ell \Delta t$ beyond a few Lyapunov times, where P^{c} is small and the count is noisy. For the Lorenz x record of the worked example below ($M = 4000$, RR $= 2\%$), the slope of $\ln P^{c}$ is 2.7 for ℓ between 10 and 30 samples, 1.8 between 30 and 60, and 1.2–1.5 between 60 and 250 samples (1.2 to 5 time units), against $\lambda_1 = 0.906$; with 8000 points the last figure falls to 1.2. The estimate approaches the bound from above and slowly, and it should be reported with the range of ℓ used and the value of ϵ, exactly as a dimension is reported with its scaling region.

Pitfall: do not use RQA to launder an unconverged exponent

If a record is too short for the fit window of Chapter 12 to open, it is too short to support a Lyapunov exponent. Computing DIV from the same record and calling the result an exponent does not add information; it relabels the same insufficiency. The honest use of RQA on a short record is comparative: DIV for this record against DIV for its surrogates, or against DIV for a baseline record of the same length analyzed with the same parameters.

RQA of a scalar record

1. Estimate the noise level of $\{x_n\}$ and, if it is significant, apply the local projective noise reduction of Chapter 14. Record what was done.
2. Choose τ by average mutual information (Chapter 9) and d by false nearest neighbours (Chapter 10). Build $\boldsymbol{x}_n$ by Eq. (18.1), giving $M = N - (d-1)\tau$ vectors.
3. Choose the Theiler window w as in Chapter 11.
4. Fix the threshold strategy. For fixed recurrence rate, set ϵ to the corresponding quantile of the distribution of pairwise distances over the admissible pairs $|i-j| > w$ (no iteration is needed); for fixed radius, set ϵ as a stated fraction of the attractor diameter, and check it against Eq. (18.3).
5. Form $\boldsymbol{R}$ by Eq. (18.4). For large M, compute it in blocks; the matrix is $M \times M$ and is the memory bottleneck.
6. Scan each diagonal $j - i = k$ for $|k| \geq w$ and accumulate the run lengths of consecutive ones into $P(\ell)$. Scan each column for runs of consecutive ones into $P_v(v)$.
7. Fix $\ell_{\min}$ and $v_{\min}$ and evaluate Eqs. (18.5)–(18.9).
8. Report τ, d, w, ϵ, the threshold strategy, $\ell_{\min}$, $v_{\min}$ and M alongside every number. Without them the numbers cannot be reproduced or compared.
9. Repeat steps 5–7 on surrogates as in Section 18.6. A measure without a null distribution is not yet a result.

Two states of the Lorenz system

Two records of the Lorenz x-coordinate ($\sigma = 10$, $\beta = 8/3$, $\Delta t = 0.02$), one at $\rho = 28$ and one at $\rho = 35$, both chaotic and indistinguishable by eye in a phase portrait, are analyzed identically: $d = 3$, $\tau = 9$, $M = 4000$ vectors, $w = 50$, recurrence rate fixed at 2%, $\ell_{\min} = v_{\min} = 2$ (`python/mkfig_rqa_lorenz.py`; Fig. 18.2). The thresholds the two records required are $\epsilon = 2.49$ and 2.94, 0.31 and 0.33 of the respective standard deviations. The measures:

	DET	L	$L_{\max}$	ENTR	LAM	TT
$\rho = 28$	0.987	9.9	274	2.81	0.966	3.3
$\rho = 35$	0.979	8.9	339	2.61	0.962	3.5

The faster dynamics at $\rho = 35$ show as a lower determinism, a shorter mean line and a lower line-length entropy; $L_{\max}$ moves the other way, which illustrates that the maximum of a distribution is not a summary of it. Whether any of these differences is significant is decided against surrogates. For the $\rho = 28$ record, 99 iteratively refined surrogates (Chapter 16), each analyzed with the recurrence rate re-solved to 2%, give DET values of at most 0.978, with mean 0.976 and standard deviation 0.0008; the data value 0.987 is ranked 100 of 100, an effect size of 13 standard deviations. The determinism of a linear Gaussian process with the same spectrum and histogram is therefore itself high, and the difference of 0.011 is what the nonlinear dynamics contribute. The control runs the other way: the AR(2) record of Chapter 3 ($\rho = 0.05$), analyzed with the same d, τ, w and rate, has DET = 0.478 against a surrogate mean of 0.483 and standard deviation 0.009, rank 29 of 100: not rejected, as it should not be. (In Fig. 18.1(f) the same process embedded with $\tau = 5$ had DET = 0.86; the number depends on the embedding, as promised, and only the comparison with surrogates analyzed identically is portable.) Under a fixed *radius* the two states would have been separated by RR instead, since the $\rho = 35$ attractor is larger; under the fixed rate that information moved into the reported thresholds. This is the point of Section 18.2.

18.4 Ordinal and entropy measures

The recurrence matrix thresholds the pairwise distances between trajectory vectors. A second family of measures works on the same vectors but asks a different question of them. Following the classification of [129], these split into methods that compare vectors by *similarity* and methods that compare them by *ordinal structure*. The second is the more interesting experimentally, and is treated first.

18.4.1 Permutation entropy

Bandt and Pompe [6] proposed discarding the amplitudes of the embedded vector entirely and keeping only the order of its components. For each $\boldsymbol{x}_i$ of Eq. (18.1), sort the d components and record which permutation π_j of $\{1, \dots, d\}$ performs the sort. There are $d!$ such patterns. Counting their relative frequencies $p(\pi_j)$ over the record gives the *permutation entropy*

$$\mathrm{PE}(d) \;=\; -\sum_{j=1}^{d!} p(\pi_j)\, \log p(\pi_j)\,, \tag{18.12}$$

in the normalization of Bandt and Pompe [6]. It is bounded, $0 \le \mathrm{PE}(d) \le \log d!$, with the upper bound attained when all orderings are equally likely — the case for i.i.d. noise — and the lower

bound attained only by a monotone record, in which a single pattern occurs. A periodic record does not approach it: a finely sampled sinusoid with $d = 3$ produces the rising and the falling pattern in nearly equal measure, together with rarer peak and trough patterns, so PE(3) tends to $\log 2$ from above (numerically 0.75 at 200 samples per period), and a period-p orbit sampled p times per period can realize up to p distinct patterns. Dividing by $\log d!$ gives the normalized form, which is what should be reported when records of different d are compared.

Two properties make this useful at the bench.

- *It is nearly parameter-free.* Only d and τ enter, and d is bounded above in practice by the requirement $M \gg d!$, since otherwise most patterns are unobserved and the entropy is biased low by undersampling. There is no threshold, and therefore none of the difficulty of Section 18.2.

- *It is invariant to any monotonic transformation of the record.* If $y_n = g(x_n)$ with g strictly increasing, then every ordering is preserved and PE is unchanged exactly.

The second property is the reason to care experimentally. A transducer with a nonlinear but monotone characteristic — a strain gauge outside its calibrated range, a proximity probe with a nonlinear gap response, an uncalibrated microphone — distorts amplitudes while leaving the order of samples intact. Every amplitude-based statistic in this book is affected by such a sensor, though not all in the same way. The correlation dimension is invariant under a stronger condition than monotonicity. Suppose g is continuously differentiable with

$$0 < c \le |g'(x)| \le C \tag{18.13}$$

on the range of the data. Applied componentwise to the delay vectors, g is then bi-Lipschitz there: in the maximum norm, $c\,\|\boldsymbol{u} - \boldsymbol{v}\| \le \|g(\boldsymbol{u}) - g(\boldsymbol{v})\| \le C\,\|\boldsymbol{u} - \boldsymbol{v}\|$. The correlation sums of the two records satisfy $C(\epsilon/C) \le C_g(\epsilon) \le C(\epsilon/c)$, and their scaling exponents coincide (Chapter 11). Strict monotonicity alone does not supply the lower bound in Eq. (18.13). The cubic $g(x) = x^3$ is smooth and strictly increasing, but $g'(0) = 0$, and its inverse is not Lipschitz at the origin. A variable uniform on $[-1, 1]$ has $D_2 = 1$; its cube has $D_2 = 2/3$, because the density of the image diverges as $|y|^{-2/3}$ near zero. With 20,000 independent samples the fitted slopes are 1.000 and 0.654. On a delay-embedded Lorenz record the same cubic lowers the fitted slope from 2.04 to about 1.1 over two decades of ϵ. A saturating characteristic such as $g(x) = \tanh(\beta x)$ does satisfy Eq. (18.13) on any bounded range, with $C/c = \cosh^2(\beta x_{\max})$; for $\beta = 2$ on $[-1, 1]$, where $C/c \approx 14$, the transformed uniform variable gives a fitted slope of 0.99. The exponent is then preserved, but C/c can be large. What the distortion changes is the finite-sample estimate. The transformation is not an isometry and stretches the attractor non-uniformly, so the scaling region shrinks or shifts and the record length needed to resolve it grows.

Permutation entropy needs none of this. A strictly increasing g preserves every ordinal pattern. A strictly decreasing g reverses each pattern, which only relabels the patterns and leaves their probabilities, and hence PE, unchanged. In both cases the invariance is exact, at any finite N, with no scaling region to find, provided no ties occur before or after the transformation and its quantization. When the calibration is doubtful, this is not a convenience but the difference between a measurement and a guess.

Pitfall: what ordinal invariance costs

The same insensitivity that protects PE from a bad transducer makes it blind to everything the transducer would have distorted. Amplitude change alone — the response growing while its temporal structure stays the same — does not move PE. A fault that manifests as increased vibration level and nothing else is invisible to it. Ties in the record are a second trap: with a coarsely quantized signal, equal values are common and the sorting rule for ties becomes a hidden parameter that changes the answer. State the tie rule, or dither the record below the quantization step.

18.4.2 Trajectory-similarity entropies

The other family compares the reconstructed vectors by distance rather than by order. The organizing account is [129], Section 3.1, which groups them as trajectory-similarity entropies and gives the definitions quoted below. They share one structure: count how many pairs of embedded vectors lie within a tolerance r, at embedding dimension d and again at $d+1$, and take the logarithm of the ratio. The measures differ in how the counting is done.

Approximate entropy (ApEn), due to Pincus [157], is the original. With $M_d = N - (d-1)\tau$ vectors at dimension d and

$$C_i^d(r) \;=\; \frac{\#\{\, j \;:\; \mathrm{d}(\boldsymbol{x}_i, \boldsymbol{x}_j) \le r \,\}}{M_d}\,, \qquad \phi^d(r) \;=\; \frac{1}{M_d}\sum_{i=1}^{M_d} \ln C_i^d(r)\,, \tag{18.14}$$

where $\mathrm{d}(\cdot,\cdot)$ is the maximum norm between trajectory vectors and the count includes $j = i$ (so that $C_i^d \ge 1/M_d$ and the logarithm is finite), the approximate entropy is

$$\mathrm{ApEn}(d, r, N) \;=\; \phi^d(r) - \phi^{d+1}(r)\,. \tag{18.15}$$

Sample entropy (SampEn), due to Richman and Moorman [166], removes the self-matches that make Eq. (18.15) biased on short records and the per-point logarithm that makes it sensitive to empty neighbourhoods,

$$\mathrm{SampEn}(d, r, N) \;=\; -\ln\!\left(\frac{A}{B}\right)\,, \tag{18.16}$$

with B and A the numbers of vector pairs $i \neq j$ within r at embedding dimensions d and $d+1$ respectively, both counted over the same M_{d+1} vectors. Pincus's convention is $\tau = 1$ and $r = 0.2\,\sigma_x$; both must be stated when the numbers are quoted. *Fuzzy entropy* replaces the hard threshold of Eq. (18.14) with a graded membership function, which removes the discontinuity at r and improves reproducibility on short records.

Note that Eq. (18.14) is a correlation sum in the sense of Chapter 11, evaluated at a single r rather than swept over a range. The whole similarity-entropy family is what remains of the Grassberger–Procaccia construction when the record is too short to support a scaling region: one radius, one dimension increment, one number. Read that way, the tolerance r plays exactly the role of the threshold ϵ of Section 18.2, and every caution in that section applies unchanged.

The review [129] adds a third group — multiscale, dispersion and symbolic-dynamic entropies — which apply coarse-graining or symbolization to the same trajectories before computing an entropy, and are reported there to be the group used most often for degradation trending. We do not derive them. The family is large, the members differ in details of the symbolization rather than in principle,

and the choice among them is properly made by testing against a labelled data set rather than by argument.

18.5 Directed coupling between channels

Everything so far has been univariate. When two channels are measured — two accelerometers, a force and a response, a drive current and a casing vibration — a question arises that no single-channel measure can answer: does one drive the other, and in which direction?

Correlation cannot answer it, being symmetric. Coherence cannot answer it, for the same reason. What is wanted is a measure of the information that the past of one channel supplies about the future of the other, *over and above* what that channel's own past already supplies.

Transfer entropy

Let $\{x_n\}$ and $\{y_n\}$ be two measured channels, and write $x_n^{(k)} = (x_n, x_{n-1}, \dots, x_{n-k+1})$ for the k most recent values of x, and similarly $y_n^{(l)}$. (These are histories ending at n, written backward as in Schreiber's original definition; the book's forward delay vectors are not used here because the object being conditioned on is the past.) The *transfer entropy* from y to x is

$$T_{y\to x} \;=\; \sum p\Big(x_{n+1}, x_n^{(k)}, y_n^{(l)}\Big)\, \log \frac{p\Big(x_{n+1} \mid x_n^{(k)}, y_n^{(l)}\Big)}{p\Big(x_{n+1} \mid x_n^{(k)}\Big)}\,, \tag{18.17}$$

the sum running over all observed states. It is the Kullback–Leibler divergence between the true transition probability for x and the transition probability that would hold if x were independent of y's past.

The construction is due to Schreiber [179]. Three properties matter.

- *It is asymmetric.* $T_{y\to x} \neq T_{x\to y}$ in general, which is the entire point: the difference between the two is what identifies a direction.

- *It is model-free.* Nothing in Eq. (18.17) assumes a linear relation, a functional form, or a fixed lag. For jointly Gaussian processes it is equivalent to Granger causality (the two are proportional), but it does not become useless when the coupling is nonlinear.

- *It vanishes under conditional independence.* If the past of y adds nothing, the two conditional distributions in Eq. (18.17) coincide and $T_{y\to x} = 0$ exactly.

That last property holds for the true probabilities. It does not hold for the estimate.

Pitfall: transfer entropy is data-hungry and biased positive

Equation Eq. (18.17) requires the joint distribution of $k + l + 1$ variables. Estimating it from a finite record — by binning, by kernels, or by k-nearest-neighbour statistics — exhausts the data quickly as k and l grow. The plug-in (histogram) estimator is biased *upward*, by the same mechanism as the mutual information of Chapter 9: finite-sample fluctuations in the joint histogram look like dependence. The nearest-neighbour estimators (Chapter 9) have a smaller bias whose sign is not fixed. Two independent channels therefore return $\hat{T}_{y\to x} > 0$, often by a comfortable margin, and the asymmetry $\hat{T}_{y\to x} - \hat{T}_{x\to y}$ is likewise nonzero by chance alone. A raw transfer entropy value is uninterpretable. The only defensible use is against a null distribution built from surrogates that destroy the coupling while preserving everything else — which is the subject of Section 18.6, and is not optional here.

The reconstruction machinery of Chapter 8 enters directly: k and l are embedding dimensions for the two channels, and the histories may be taken with a delay $\tau > 1$ between their entries, as in Chapter 8, instead of consecutive samples. The review [129] places this within a broader class of model-free coupling measures built on reconstructed trajectories, including distance-based alternatives that estimate coupling without committing to embedding parameters, and network constructions that extract directed interactions from windowed trajectory graphs.

Common drive, and what transfer entropy does not prove

Two channels driven by a third, unmeasured source will show nonzero transfer entropy in both directions, and the larger of the two will be whichever channel the drive reaches first. This is a lag, not a coupling, and Eq. (18.17) cannot distinguish them from two channels alone. Conditioning on the third channel removes the effect — if the third channel was measured. When it was not, the correct report is that the channels are coupled to a common influence with a measurable delay, not that one causes the other.

18.6 Significance testing

None of the measures in this chapter has a useful sampling distribution in closed form. DET is a ratio of sums over a histogram of run lengths in a thresholded distance matrix whose entries are strongly dependent; nobody has written down its distribution under a plausible null, and it is not likely that anybody will. The same is true of LAM, of ENTR, of PE and of $\hat{T}_{y\to x}$.

This is not a defect peculiar to recurrence methods. It is the situation Chapter 16 was written to address, and the remedy is the one given there: generate an ensemble of surrogate records that share with the data everything the null hypothesis specifies and nothing else, compute the same statistic on each, and locate the data's value in the resulting distribution.

Surrogate test for an RQA or complexity measure

1. State the null hypothesis. For a single channel the usual ladder of Chapter 16 applies: i.i.d. noise with the measured distribution; a linear Gaussian process with the measured power spectrum; a monotonic nonlinear transform of such a process (the amplitude-adjusted class). Choose the weakest null the conclusion requires — rejecting i.i.d. noise establishes only that the record is correlated.
2. Generate B surrogates consistent with that null. For a one-sided test at level α, $B = 1/\alpha - 1$ is the minimum; $B = 99$ or $B = 999$ are the usual choices.
3. Compute the statistic on the data *and* on every surrogate, using identical τ, d, w, $\ell_{\min}$ and threshold *strategy*. Under the fixed-recurrence-rate strategy, re-solve for ϵ on each surrogate separately. Under fixed radius, hold ϵ fixed across the ensemble.
4. Report the rank of the data's value within the ensemble, and the effect size — how far the data sits from the surrogate mean in units of the surrogate spread. A rank alone conceals the difference between a large effect and a barely significant one.
5. Where the test is repeated over many records, windows or channels, correct for multiple comparisons. A sliding-window RQA over a long record performs hundreds of tests, and at $\alpha = 0.05$ a handful will reject by construction.

Two points about step 3 are worth isolating, because they are where surrogate tests of recurrence measures usually go wrong.

Pitfall: the surrogates must be analyzed exactly as the data were

1. *Re-solving the threshold matters.* If ϵ is fixed on the data and then applied to surrogates whose amplitude distribution differs, the test compares recurrence densities rather than recurrence structures, and it will reject for the wrong reason. Fixing the recurrence rate on every member of the ensemble forces the comparison onto the line structure, which is what the null is about.
2. *Re-estimating τ and d does not.* Delay and dimension should be fixed once from the data and held across the ensemble. Re-optimizing them per surrogate lets the surrogate ensemble adapt to the statistic and destroys the test's calibration.

The transfer entropy of Section 18.5 needs a different null, because the hypothesis concerns the relation between two channels rather than the structure of one. The appropriate surrogates leave each channel's own dynamics intact and destroy only the cross-channel relation: shifting one channel by a random offset larger than its autocorrelation time, or permuting whole segments of one channel while leaving the other untouched. Randomizing the phases of both channels independently is the wrong null here — it destroys each channel's own nonlinear structure as well, so a rejection establishes nothing about coupling.

A useful discipline: before running the test on the data, run it on a record you know satisfies the null. If a linear surrogate of your own record, treated as data, rejects the null more often than α of the time, the test is miscalibrated and the fault is in the pipeline, not in the physics. This costs one extra run of the whole analysis and it catches most of the errors that matter.

18.7 Discussion

Set beside Chapters 11 and 12, this chapter's methods give up a great deal.

They give up invariance. A correlation dimension is a property of the attractor; it does not depend on the embedding once the embedding is adequate, nor on the coordinates, nor on any threshold, and two laboratories analyzing the same system should obtain the same value. A determinism does not have this property in any degree. It depends on ϵ, on $\ell_{\min}$, on w, on d and τ, and on the record length. Two laboratories analyzing the same system with different settings will obtain different numbers, and neither will be wrong.

They give up dynamical meaning. A Lyapunov exponent has units of inverse time and a physical interpretation: the rate at which information about the initial condition is lost. DET has no units and no interpretation of that kind. It is a summary of a picture. Section 18.3.2 showed how far a plausible bridge back to λ_1 can be pushed through $L_{\max}$, which is not far; the defensible bridge is the correlation entropy of Section 18.3.3, and it carries the same finite-data caveats as the dimension.

What they buy is the ability to say something when the alternative is to say nothing.

- *Short records.* A few thousand points give a readable recurrence plot and a stable DET. The same points give a correlation dimension with no scaling region.
- *Nonstationarity.* The estimators of Part II require stationarity and are silent about its absence. A recurrence plot displays drift as a texture, and a sliding-window RQA turns the drift into a trend curve. This connects directly to the tracking metrics of Chapter 17: RQA measures are further members of the family of cheap scalar indicators computed on a reconstructed trajectory, and they are used in the same way — not for their absolute values but for their evolution.
- *Bad calibration.* Permutation entropy is exactly invariant, at finite record length, to a strictly monotone transducer nonlinearity, provided the record has no ties. The correlation dimension is invariant only as a scaling exponent, and only when the slope of the characteristic is bounded away from zero and from infinity on the range of the data, Eq. (18.13). The true mutual information of Chapter 9 is unchanged by any invertible transducer, but its equal-width-bin estimate is not. No other amplitude-based measure in this book is invariant.
- *Directionality.* Transfer entropy answers a question — which channel drives which — that has no answer in the univariate toolkit at all.

The correct place for these methods in an experimental program is therefore early and comparative. Compute them first, because they are cheap and because the recurrence plot will tell you whether the record is stationary enough for anything else. Use them comparatively, against surrogates and against baselines, never as absolute values. And when the record turns out to be long, clean and stationary after all, go back to Chapters 11 and 12, which measure something the system actually has.

Problems

Unless a problem says otherwise, generate your own data so the ground truth is known. Use the Lorenz system in the standard chaotic regime, sampled well enough to resolve the fastest time scale, and keep the integrator settings fixed across all parts of a problem so that results are comparable. Report τ, d, w, ϵ, the threshold strategy and $\ell_{\min}$ with every RQA number you quote — the chapter's central claim is that the number means nothing without them, and these problems are meant to make that concrete. As always, use MATLAB or Python, and the routines of Appendix A where they apply (`python/rqa.py` implements the recurrence matrix, the RQA measures and the K_2 estimator of this chapter).

Problem 18.1

Computational

The claim of Section 18.2 is that no RQA measure is invariant to the threshold. Test it on a single record, so that the dynamics is held exactly fixed.

1. Generate one Lorenz record of $M = 4000$ embedded points. Embed it once, with τ and d chosen by the methods of Chapters 9 and 10, and fix a Theiler window w.

2. Sweep ϵ over two decades and plot RR, DET, L, ENTR and LAM against ϵ on log axes. Mark the values of ϵ giving RR $= 1\%$, 5% and 10%.

3. Report the range of DET over the interval of ϵ that a reasonable analyst might choose. Is any part of the curve flat enough to be called a plateau, in the sense that the correlation sum of Chapter 11 has a scaling region?

4. Repeat parts (1)–(3) with $\ell_{\min} = 2, 3, 5$. Which matters more for the reported DET, the threshold or the minimum line length?

Problem 18.2

Computational

The Theiler window of Section 18.2.1 was introduced for the correlation dimension. Show that it is equally necessary here.

1. Using the record of Problem 18.1, compute RR, DET and $L_{\max}$ with $w = 0$ (main diagonal excluded only) and with w set from the first minimum of the average mutual information. Report both sets.

2. Resample the same trajectory at four times the rate and repeat. How does the $w = 0$ result move? How does the correctly windowed result move? Explain the difference in terms of what the temporal-correlation band contributes to $P(\ell)$.

3. Plot DET against w over a range from 0 to several decorrelation times. Is there a value beyond which the result is insensitive to w?

4. Compare that value with the Theiler window you would have used for a correlation-dimension estimate of the same record.

Problem 18.3

Core

Section 18.4.1 claims that permutation entropy is exactly invariant to a monotone distortion at any record length, and that the correlation dimension is invariant only as a scaling exponent, its finite-sample estimate being affected through the scaling region, provided the slope of the distortion satisfies Eq. (18.13) on the range of the record. Both halves are testable on one record.

1. Take a Lorenz record $\{x_n\}$ and form $y_n = g(x_n)$ for a strictly increasing, strongly nonlinear g — for example a saturating characteristic $g(x) = \tanh(\beta x)$ with β large enough to compress the outer excursions visibly.

2. Compute PE(d) for both records at several d and confirm that the values agree to machine precision. State your tie rule and confirm that it was never invoked.

3. Estimate the correlation dimension of both records by the method of Chapter 11. Report the two scaling regions and the two slopes. Do the slopes agree within their fitting uncertainty, as the bi-Lipschitz argument of Section 18.4.1 says they must for a characteristic whose slope is bounded away from zero on the record's range? Compute C/c for your β. How do the scaling regions differ in position and width, and how much longer a record would the distorted signal need to give a scaling region as wide as the undistorted one?

4. Now quantize both records to 8 bits. Recompute PE. Does the invariance survive? Explain the result in terms of the tie trap in the pitfall box of Section 18.4.1.

5. Repeat parts (2) and (3) with $g(x) = x^3$ applied to the zero-mean record. Which of the two invariance statements survives, and why? Which hypothesis of the bi-Lipschitz argument fails?

Problem 18.4

Computational

The chapter's reason for existing is that recurrence measures work on records too short for the estimators of Chapters 11 and 12. Establish that this is more than an assertion.

1. Generate Lorenz records of $M = 500$, 1000, 2000 and 10000 embedded points. For each, attempt a correlation-dimension estimate and report the extent of the scaling region. Identify the shortest record for which you would be willing to quote a dimension.

2. For every record, including the ones you rejected in part (1), compute DET at a fixed recurrence rate of 5%. Plot DET against M.

3. For each record generate 99 amplitude-adjusted surrogates as in Chapter 16, recompute DET on each with the threshold re-solved per surrogate, and report the rank of the data value and the effect size.

4. At what record length does the surrogate test stop rejecting? Compare with the length at which the correlation dimension became usable. Which method survives further down, and by how much?

Problem 18.5

Computational

A determinism value without a null distribution is decorative. Section 18.6 says so; this problem builds the counterexample that makes the point stick.

1. Construct a linear stochastic process — an autoregressive model of modest order driven by Gaussian noise — and tune its coefficients until its DET, computed at a fixed recurrence rate, matches the DET of your $M = 2000$ Lorenz record to two decimal places.

2. You now have two records with the same determinism and different origins. Confirm by inspection of their recurrence plots that the textures differ, and say which of the descriptions in Section 18.1.1 each matches.

3. Run the surrogate test of Section 18.6 on both, against the linear-Gaussian null. Confirm that it rejects for one record and not the other.

4. Repeat part (3) against the i.i.d. null instead. Both records should now reject. Explain why this outcome is worthless, in terms of step 1 of the algorithm box.

Problem 18.6

Computational

Transfer entropy is asymmetric by construction, biased positive in estimation, and useful only against surrogates. Confirm all three on a system whose coupling direction you set yourself.

1. Couple two Rössler oscillators unidirectionally, so that system A drives system B and B does not act on A. Record one scalar channel from each.

2. Estimate $\hat{T}_{A\to B}$ and $\hat{T}_{B\to A}$ from Eq. (18.17) at several choices of k and l. Confirm that the asymmetry points in the correct direction, and report how the two values behave as k and l grow at fixed record length.

3. Set the coupling strength to zero and repeat. Both estimates should be zero in the population and are not zero in your sample. Report the magnitude of the bias and its dependence on record length.

4. Build a null distribution by shifting one channel by random offsets larger than its autocorrelation time, and re-test both coupled and uncoupled cases. At what coupling strength does the test first detect the true direction? Compare that threshold with the coupling strength at which the asymmetry of the raw estimates first exceeds the bias measured in part (3), and comment on which number you would report.

Problem 18.7

Experimental

A recurrence plot of a record that is not stationary, and is not meant to be. Take the measured tracking matrix $\boldsymbol{Y}$, 1728×405, of `data/psw_fatigue_2005.mat`: one row per 25.6 s window over the 12.28-hour fatigue run of Section 17.4. The rows are already a trajectory in a 405-dimensional feature space, sampled in slow time, so no delay embedding is needed (`python/chk_psw_problems.py`).

1. Standardize each column to zero mean and unit variance, and build the recurrence matrix of the 1728 rows with the threshold set to the 10th percentile of all pairwise distances. Confirm

RR ≈ 0.10 by construction and report DET with $\ell_{\min} = 2$ and a Theiler window of one; you should find about 0.88.

2. Plot the recurrence matrix. Two structures should be visible: broad bands parallel to the main diagonal, and a non-stationary fading toward the top-right corner. Estimate the band spacing in hours and compare it with the periods of the two supply voltages, obtained from their own spectra. Which of the two operating frequencies sets the banding?

3. Build the recurrence plot of the two voltages alone, at the same recurrence rate, and put the two plots side by side. Which features of part (2) are explained by the operating conditions, and which are not?

4. Quantify the fading. For each block of 50 consecutive windows, compute the fraction of recurrences it shares with the first 300 windows, and plot that fraction against time. Interpret the result: is the trajectory leaving the region it started in, returning to it, or both?

5. Windowed RQA. Slide a window of 200 rows along the record, compute RR, DET and $L_{\max}$ within each window at a *fixed* threshold, and plot the three against time. Compare the result with the damage coordinate of Problem 17.7(3). State what a windowed RQA measure gives that the SOD coordinate does not, and what it costs: in particular, which of the three measures is confounded by the operating conditions and which is not.

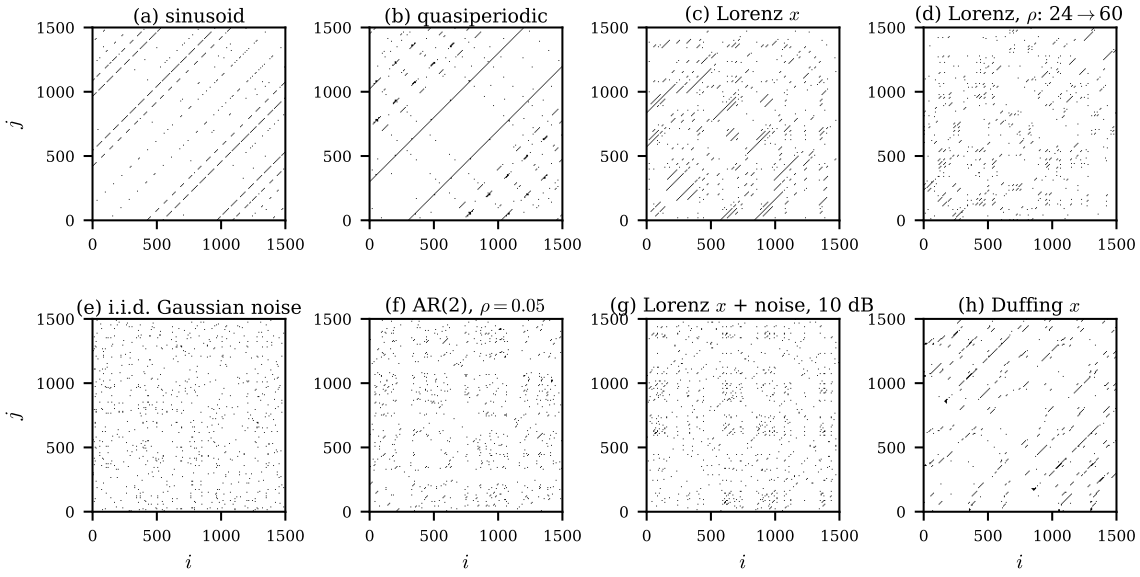


Figure 18.1: Recurrence plots of eight records of $M = 1500$ embedded points, all at recurrence rate RR = 2% and Theiler window $w = 30$ samples (`python/mkfig_rp_gallery.py`). (a) Sinusoid, period 60 samples ($d = 2$, $\tau = 15$): unbroken diagonals at multiples of the period. (b) Two-frequency torus, frequency ratio $\sqrt{2}$ ($d = 3$): diagonals at the near-recurrence times, never repeating exactly. (c) Lorenz x, $\Delta t = 0.02$ ($d = 3$, $\tau = 9$): short diagonals of many lengths, DET = 0.99, $L_{\max} = 217$. (d) The same with ρ ramped from 24 to 60 over the record: recurrences concentrate near the main diagonal and the far corners empty, the drift texture. (e) i.i.d. Gaussian noise ($d = 3$, $\tau = 1$): speckle, with the embedding-induced short diagonals discussed in the text. (f) The AR(2) process of Chapter 3 with $\rho = 0.05$ ($d = 3$, $\tau = 5$): linear correlation produces short diagonals, DET = 0.86. (g) The Lorenz record of (c) with white noise at 10 dB signal-to-noise ratio: the noise displacement exceeds ϵ and the lines break into points, DET = 0.27. (h) Two-well Duffing x, $\Delta t = 0.1$ ($d = 4$, $\tau = 8$): diagonals interrupted at the well-to-well transitions, DET = 1.00, $L_{\max} = 206$.

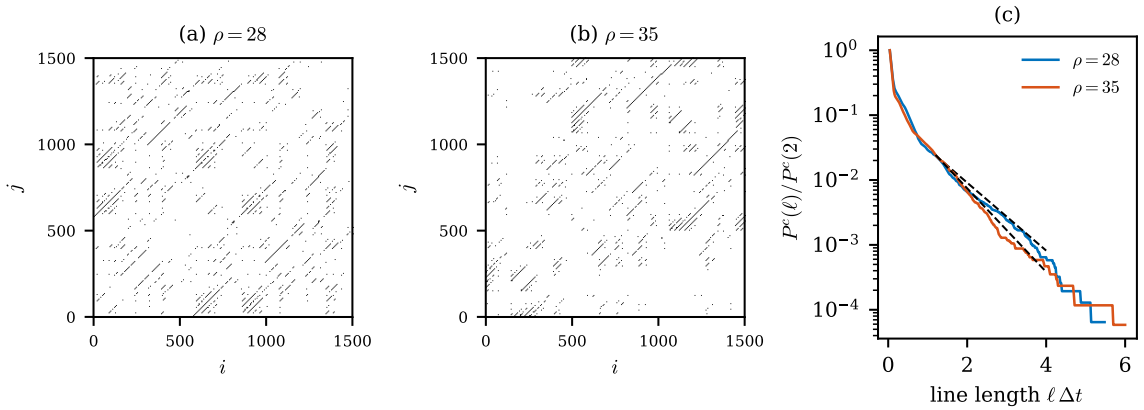


Figure 18.2: The worked example. (a,b) Recurrence plots of the first 1500 of $M = 4000$ delay vectors of the Lorenz x record at $\rho = 28$ and $\rho = 35$ ($d = 3$, $\tau = 9$, $w = 50$, RR = 2%). (c) Cumulative diagonal-line distributions $P^c(\ell)$, normalized at $\ell = 2$, against line length in time units; the dashed lines are the fits of Eq. (18.11) over $60 \le \ell \le 200$ samples, with slopes $K_2 = 1.2$ ($\rho = 28$) and 1.5 ($\rho = 35$) per unit time. The curves are steeper at short ℓ and flatten toward the bound λ_1 only at lengths where few lines remain.

Appendices

A
Algorithms in MATLAB and Python

Every algorithm below is given twice, once in MATLAB and once in Python, computing the same thing from the same arguments. The MATLAB versions are the originals; the Python versions were written against them and are checked by the same tests, so either may be used.

The listings are not typeset copies of the code but the files themselves, included from `matlab/` and `python/`. That is deliberate: the code in this appendix is the code that was run.

The MATLAB routines need the Statistics and Machine Learning Toolbox, for `createns` and `knnsearch`. The Python routines need NumPy and SciPy and nothing else; two of them, collected in Section A.11, supply what MATLAB provides as built-ins — in particular the generalized singular value decomposition, which SciPy does not have.

A.1 Average Mutual Information

A.1.1 Auto Average Mutual Information Algorithm

MATLAB ami.m

```
function [ Ixx ] = ami( x, k, nb )
% AMI calculates average mutual information for the input scalar time
%     series x. If no output is requested, it plots the AMI curve vs delay.
%
% CALL: Ixx = ami( x, k, nb );
%
% INPUT:
%   x  - first input scalar time series
%   k  - maximum delay to consider
%   nb - number of bins to use in calculation for x (default: nb = 32)
% OUTPUT:
%   Ixx - average mutual information in bits for 0 to k delay
%
% Copyright and Revised by David Chelidze 8/29/18, 9/5/18, 12/14/21
% chelidze@uri.edu

if nargin < 2 % not enough inputs
   error('Requires at least two input arguments!')
elseif nargin == 2 % set the number of bins to the default
   nb = 32;
end

% check the data and make a column vector of data
if min(size(x)) ~= 1
   error('First input should be a vector of scalar data points!')
else % make it into column vector and get its size
    x = x(:);
    n = size(x,1);
end
```

```
% get the domain boundaries and each bin size
minx = min(x); maxx = max(x); % domain
binx = (maxx - minx)/nb; % bin size
xx = minx + binx*(0:nb); % get the bin boundaries (edges)
xx(1) = -inf; xx(end) = inf; % extend the rage of the end points

% obtain the binned probability density for the data
Px = nonzeros( histcounts(x, xx, 'Normalization', 'probability') );

Ixx = zeros(k+1,1); % initialize Ixy
for ii = 0:k % calculate the average mutual information for each delay
    indx = 1:n-ii; % for the positive delays
    Pxx =  nonzeros( histcounts2(x(indx), x(indx + ii), xx, xx, ...
                                 'Normalization', 'probability') );
    Ixx(1+ii) = sum(Pxx.*log2(Pxx)) - 2*sum(Px.*log2(Px));
end

if nargout == 0 % plot the results
    plot(0:k, Ixx, 'o-', 'markerfacecolor', 'w')
    grid on, pbaspect([1.618 1 1])
    xlabel('Delay (samples)')
    ylabel('Average Mutual Information (bits)')
end
```

Python ami.py

```
"""Average mutual information.

Python port of ami.m (Appendix A.1.1). Same arguments, same return, same
binning. NumPy only. The three binning helpers are public because
ami_x.py uses them too, so the auto and cross routines cannot drift
apart.

One deliberate difference: MATLAB extends the outer bin edges to -Inf
and +Inf, which numpy.histogram will not accept. The outer edges are
widened by one bin instead, which includes the same points.

Copyright by David Chelidze. Python port.
"""
import numpy as np


def bin_edges(v, nb):
    """Uniform bin edges over the range of v, outer edges widened."""
    lo, hi = float(np.min(v)), float(np.max(v))
    b = (hi - lo)/nb
    e = lo + b*np.arange(nb + 1)
    e[0] -= b                   # MATLAB uses -Inf here
    e[-1] += b                  # and +Inf here
    return e


def binned_p(v, e):
    """Non-zero binned probabilities."""
    h, _ = np.histogram(v, bins=e)
    p = h/h.sum()
    return p[p > 0]


def joint_p(u, v, eu, ev):
    """Non-zero joint binned probabilities."""
    h, _, _ = np.histogram2d(u, v, bins=[eu, ev])
    p = h/h.sum()
    return p[p > 0]
```

```
def ami(x, k, nb=32):
    """Average mutual information of x with itself, delays 0 to k.

    x  : scalar time series
    k  : maximum delay to consider
    nb : number of bins (default 32)

    Returns Ixx, length k+1, in bits. Ixx[0] is the entropy of x, since
    I(X;X) = H(X) -- a useful check on any binning.
    """
    x = np.asarray(x, float).ravel()
    n = x.size
    xx = bin_edges(x, nb)
    Px = binned_p(x, xx)
    Ixx = np.zeros(k + 1)
    for i in range(k + 1):
        Pxx = joint_p(x[:n - i], x[i:], xx, xx)
        Ixx[i] = np.sum(Pxx*np.log2(Pxx)) - 2*np.sum(Px*np.log2(Px))
    return Ixx
```

A.1.2 Cross Average Mutual Information Algorithm

MATLAB ami_x.m

```
function [ Ixy ] = ami_x( x, y, k, nbx, nby )
% AMI_X calculates average-cross-mutual information for the input scalar
%       time series x and y. If no output requested, plots the AMI curve.
%
% CALL: Ixy = ami_x( x, y, m, nbx, nby );
%
% INPUT:
%   x  - first input scalar time series
%   y  - second inpit scalar time series
%   k  - maximum delay to consider
%   nbx - number of bins to use in calculation for x (default: nbx = 32)
%   nby - number of bins to use in calculation for y (default: nby = nbx)
% OUTPUT:
%   Ixy - average mutual information in bits for - k to k delay
%
% Copyright and Revised by David Chelidze 8/29/18, 9/5/18, 12/14/21
% chelidze@uri.edu

if nargin < 3 % not enough inputs provided
   error('Requires at least three input arguments!')
elseif nargin == 3 % set the number of bins to default
   nbx = 32;
   nby = 32;
elseif nargin == 4 % set bins for y the same as for x
    nby = nbx;
end

% check the input needed data's shape
if min(size(x)) ~= 1 || min(size(y)) ~= 1
   error('First two inputs should be vectors of scalar data points!')
else % make them into column vectors and get their sizes
    x = x(:); y = y(:);
    n = size(x,1);
    m = size(y,1);
end

% get the data domain boundaries and the uniform bin sizes
minx = min(x); maxx = max(x);
binx = (maxx - minx)/nbx;
miny = min(y); maxy = max(y);
biny = (maxy - miny)/nby;
```

```
% get the corresponding bin boundaries (edges)
xx = minx + binx*(0:nbx);
xx(1) = -inf; xx(end) = inf;
yy = miny + biny*(0:nby);
yy(1) = -inf; yy(end) = inf;

% obtain Px and Py, probability density functions
Px = nonzeros( histcounts(x, xx, 'Normalization', 'probability') );
Py = nonzeros( histcounts(y, yy, 'Normalization', 'probability') );

Ixy = zeros(2*k+1, 1); % initialize Ixy
% evaluate the AMI at zero delay by first finding the joint pdf
indx = 1:min(m, n);
Pxy =  nonzeros( histcounts2(x(indx), y(indx), xx, yy, ...
                                'Normalization', 'probability') );
Ixy(k+1) = sum(Pxy.*log2(Pxy)) - sum(Px.*log2(Px)) ...
                               - sum(Py.*log2(Py));
% now evaluate AMI for the other delays
for ii = 1:k % calculate the average mutual information
    indx = 1:min(m,n-ii); % for the negative delays
    Pxy =  nonzeros( histcounts2(x(indx + ii), y(indx), xx, yy, ...
                                'Normalization', 'probability') );
    Ixy(k+1-ii) = sum(Pxy.*log2(Pxy)) - sum(Px.*log2(Px)) ...
                                      - sum(Py.*log2(Py));
    indx = 1:min(n,m-ii); % for the positive delays
    Pxy =  nonzeros( histcounts2(x(indx), y(indx + ii), xx, yy, ...
                                'Normalization', 'probability') );
    Ixy(k+1+ii) = sum(Pxy.*log2(Pxy)) - sum(Px.*log2(Px)) ...
                                      - sum(Py.*log2(Py));
end

if nargout == 0 % plot the results
    plot(-k:k, Ixy, 'o-', 'markerfacecolor', 'w')
    grid on, pbaspect([1.618 1 1])
    xlabel('Delay (samples)')
    ylabel('Average Mutual Information (bits)')
end
```

Python ami_x.py

```
"""Cross average mutual information.

Python port of ami_x.m (Appendix A.1.2). The binning helpers come from
ami.py, so the auto and cross routines cannot drift apart.

Copyright by David Chelidze. Python port.
"""
import numpy as np
from ami import bin_edges, binned_p, joint_p


def ami_x(x, y, k, nbx=32, nby=None):
    """Cross average mutual information of x and y, delays -k to k.

    x, y : the two scalar time series
    k    : maximum delay to consider
    nbx  : number of bins for x (default 32)
    nby  : number of bins for y (default nbx)

    Returns Ixy, length 2k+1, in bits, with Ixy[k] the zero-delay value.
    Index k+i holds the positive delay i (y lagged behind x) and k-i the
    negative one, matching ami_x.m.
    """
    if nby is None:
        nby = nbx
    x = np.asarray(x, float).ravel()
    y = np.asarray(y, float).ravel()
```

```
    n, m = x.size, y.size
    xx, yy = bin_edges(x, nbx), bin_edges(y, nby)
    Px, Py = binned_p(x, xx), binned_p(y, yy)
    hx = np.sum(Px*np.log2(Px))
    hy = np.sum(Py*np.log2(Py))
    Ixy = np.zeros(2*k + 1)
    j = min(m, n)
    Pxy = joint_p(x[:j], y[:j], xx, yy)
    Ixy[k] = np.sum(Pxy*np.log2(Pxy)) - hx - hy
    for i in range(1, k + 1):
        j = min(m, n - i)                       # negative delays
        Pxy = joint_p(x[i:i + j], y[:j], xx, yy)
        Ixy[k - i] = np.sum(Pxy*np.log2(Pxy)) - hx - hy
        j = min(n, m - i)                       # positive delays
        Pxy = joint_p(x[:j], y[i:i + j], xx, yy)
        Ixy[k + i] = np.sum(Pxy*np.log2(Pxy)) - hx - hy
    return Ixy
```

A.2 Correlation Dimension Algorithm

MATLAB c2r.m

```
function [r, c] = c2r( x, d, m, n, r, p, q )
% C2R function calculates the correlation sum from a scalar time
%  series after reconstructing its phase space using delay coordinates
%  It uses Euclidean distances!
%
% CALL: [r, c] = c2r( x, d, m, n, r, p, q );
%       If no output is requested it will just plot the results
%
% INPUT:
%   x -- scalar time series
%   d -- embedding dimension
%   m -- delay time
%   n -- number of points to use in the reconstructed phase space
%   r -- number of bins (ball length-scales) to use if it's just a number,
%        or the array of ball sizes (bins) to consider if it's a vector
%   p -- (optional) supply 0 to NOT remove temporal correlations
%        or number of points to remove. The default is p = d*m.
%        NOTE: only need to remove these correlations for the continuous
%        flow data!
%   q -- (optional) only supply if instead of removing temporal
%        correlations you wish to randomly populate attractor with q sample
%        points (thus, this will ignore p input if provided, & use max
%        number of points possible in the phase space reconstruction)
%
% OUTPUT:
%   r -- ln of ball sizes
%   c -- ln of correlation sum
%
% copyright by David Chelidze 3/27/2002
% revised by DC 5/31/2002 & 6/12/2002 & 10/2/2002 & 10/30/2002
% & 11/04/2005 & 3/3/3009 & 11/4-21/2013 & 12/12/2013 & 10/20/17 & 12/12/17

if nargin < 5 % not enough input variables
    error('Error: Needs at least five input variables.')
end
if min(size(x)) ~= 1
   error('Error: The first input should be a vector of scalars.')
else % normalize the input time series
    x = (x(:)' - mean(x))/std(x); % make it zero mean and unit variance
end
nm = length(x) - (d-1)*m; % maximum number of reconstructed points possible
if isempty(n) % use max
    n = nm;
```

```
elseif n > nm % use max possible and notify user too
    fprintf(['WARNING: Max number of embedded vectors exceeded.\n'...
        'ACTION:  Continuing with max number of vectors possible.\n'])
    n = nm;
end
if nargin < 6 || isempty(p) % remove temporal correlations of this length
    p = d*m; % still open for debate, may want to use different length
end

%% reconstruct the phase space to analyze interpoint distance distributions
if nargin == 7 && isempty(q) < 1 % randomly populate attractor w/ points
    indx = randperm(n, q); % pick q random points from the reconstruction
    n = q; % new number of phase space points to use
    p = 0; % do not need to remove temporal correlations
else % use the whole phase space
    indx = 1:n;
end
y = zeros(d,n); % initialize
for i = 0:d-1 % reconstructed phase space
    y(i+1,:) = x(indx+i*m);
end

%% calculate the correlation sum
np = n - p;
if length(r) == 1 % determine the needed length scales or bins to use
    c = zeros(1,r);
else % use the provided length scale or bins
    r = r.^2;
    c = zeros(size(r));
end
for i = 1:np-1 % for every point in the reconstructed phase space
    indx = i+p+1:n; % index of points to calculate distance to
    dist = 0; % initialize distances
    for k = 1:d % calculate distances squared to all other points
        dist = dist + (y(k, indx) - y(k, i)).^2;
    end
    if i == 1 && length(r) == 1 % find the suitable bins or length scales
        r = exp( linspace( ...
            log(min(dist)/np),log(max(dist) + min(dist) ), r) );
    end
    c = c + cumsum( histcounts(dist, [0, r]) ); % correlation sum
end
c = log(2*c/np/(np-1)); % ln of correlation sum
r = log(r)/2; % ln of the corresponding ball sizes

if nargout == 0 % plot the results
    figure
    set(gcf,'PaperSize',[10.5 3.5],'PaperPosition', [-.75 -.25 12 4]);
    subplot(121), plot(r,c,'-','linewidth',1)
    set(gca,'fontsize',12)
    grid on, pbaspect([1.618, 1, 1])
    xlabel('$\ln(\epsilon)$','Interpreter','Latex','FontSize',14)
    ylabel('$\ln \hat C(\epsilon)$','Interpreter','Latex','FontSize',14)
    subplot(122), plot(r(1:end-1),diff(c)./diff(r),'.-','linewidth',1)
    set(gca,'fontsize',12)
    grid on, pbaspect([1.618, 1, 1])
    xlabel('$\ln(\epsilon)$','Interpreter','Latex','FontSize',14)
    ylabel(['$D_2 \approx \frac{\mathrm{d}\ln \hat C(\epsilon)}',...
        '{\mathrm{d}\ln(\epsilon)}$'],'Interpreter','Latex','FontSize',14)
end
```

Python `c2r.py`

```
"""Correlation sum from a scalar time series.

Python port of c2r.m (Appendix A.2). Euclidean distances, temporal
correlations removed by a Theiler window.
```

```
Copyright by David Chelidze. Python port.
"""
import numpy as np


def c2r(x, d, m, n=None, r=32, p=None, q=None, rng=None):
    """Return (lnr, lnc): ln of the ball sizes and of the correlation sum.

    x : scalar time series
    d : embedding dimension
    m : delay time
    n : number of reconstructed points to use (None for the maximum)
    r : number of length scales, or an array of ball sizes to use
    p : Theiler window; 0 to keep temporal correlations. Default d*m,
        and only needed for flow data.
    q : if given, populate the attractor with q random points instead of
        removing temporal correlations (p is then ignored)
    """
    rng = np.random.default_rng() if rng is None else rng
    x = np.asarray(x, float).ravel()
    x = (x - x.mean())/x.std()          # zero mean, unit variance
    nm = x.size - (d - 1)*m             # maximum number of points possible
    if n is None:
        n = nm
    elif n > nm:
        print("WARNING: Max number of embedded vectors exceeded.\n"
              "ACTION:  Continuing with max number of vectors possible.")
        n = nm
    if p is None:
        p = d*m                         # still open for debate

    # reconstruct the phase space
    if q is not None:
        indx = rng.permutation(n)[:q]   # random points from the attractor
        n, p = q, 0                     # no temporal correlations to remove
    else:
        indx = np.arange(n)
    y = np.array([x[indx + i*m] for i in range(d)])      # (d, n)

    # correlation sum
    npt = n - p
    r = np.atleast_1d(np.asarray(r, float))
    auto = r.size == 1
    if auto:
        nbins = int(r[0])
        c = np.zeros(nbins)
    else:
        r = r**2                        # work in squared distances
        c = np.zeros(r.size)
    for i in range(npt - 1):
        j = np.arange(i + p + 1, n)
        dist = np.sum((y[:, j] - y[:, i:i + 1])**2, axis=0)
        if i == 0 and auto:
            r = np.exp(np.linspace(np.log(dist.min()/npt),
                                   np.log(dist.max() + dist.min()), nbins))
        h, _ = np.histogram(dist, bins=np.concatenate([[0.0], r]))
        c = c + np.cumsum(h)
    with np.errstate(divide="ignore"):  # empty bins give -inf, as in MATLAB
        lnc = np.log(2*c/npt/(npt - 1)) # ln of the correlation sum
    lnr = np.log(r)/2                   # ln of the ball sizes
    return lnr, lnc
```

A.3 Pointwise Dimension Algorithm

MATLAB pwdim.m

```
 function [lnD, lnN] = pwdim( x, m, d, k, N, w )
% PWDIM: estimates average pointwise dimension using fixed distance and
%        fixed mass methods
% NOTE: York claims it is equivalent to information dimension,
%       Moon claims it is like correlation dimension
%
%   CALL [lnD, lnN] = pwdim( x, m, d, k, N, w );
%
% INPUT:
%   x - scalar time series
%   m - delay parameter (time delay for embedding)
%   d - embedding dimensions to use (it can be an array!)
%   k - number of distance partitions (for fixed distance) or points inside
%       a ball (for fixed mass)
%   N - maximum number of averages to do (optional -- 2^12 is default)
%   w - (optional) Theiler window to remove the correlations, by default
%       not temporal correlations are removed. This does not seem to alter
%       the results significantly, but considerably increase the
%       computational time.
%
% OUTPUT: The first two columns are fixed distance and the second are fixed
%         mass. If no output is requested, then it plots the results.
%   lnN(:,1)  - <ln(fraction of points inside the eps-ball)>
%   lnD(:,1)  - ln(eps)--i.e., log of the epsilon-ball
%   lnN(:,2)  - ln(n/N)--i.e., faction of points or mass
%   lnD(:,2)  - <ln(ball size containing n points)>
%
%     copyright by David Chelidze, 1998-2013
%     modified 10/26/17 by DC

%% Initialization and preprocessing
if nargin < 3
   error('Requires at least three input arguments!')
else
    x = x(:)'; % put 'em in correct form and get parameters
    n = length(x)- m*(d-1); % <- that's the maximum size possible!!!;
    if nargin < 6 % do not remove temporal correlations
        w = 0;
    end
    if nargin < 5 || N == 0 || isempty(N) % set the default
        N = min(2^12, n);
    else % use the provided or max possible points
        N = min(N, n);
    end
    if nargin < 4 || k == 0 || isempty(k) % use the default # of NNs
        k = 500;
    end
end
lnN = zeros(N,k); % initialize the log of measure array
y = zeros(d, n); % initialize array for embedded vectors
for i = 1:d % reconstruct the phase space
    y(i,:) = x((1:n) + (i-1)*m);
end % embedding

%% create the kd-partition search for nearest distances
ns = createns(y','NSMethod','kdtree','Distance','euclidean');
% search for the k points nearest to the test points (+ 2*w if removing
% temporal correlations)
np = k + 1 + 2*w; % total nearest neighbors to look up
[idx, dis] = knnsearch(ns,y(:,randperm(n,N))',...
    'Distance','euclidean','k',np);
if w == 0 % do not remove temporarily correlated nearest neighbor distances
    dis = dis(:,2:end); % only drop the first zero distance
else % remove temporal correlations
```

```
    dis = cell2mat( cellfun( @removecorrs,mat2cell(dis,ones(N,1),np), ...
          mat2cell(idx,ones(N,1),np),'UniformOutput',false ) );
end

%% the fixed mass based estimates:
lnDm = mean(log(dis)); % average log of the eps-ball
% lnNm = log(1:k) - log(N); % measure inside the eps-ball or use:
lnNm = psi(1:k) - log(N); % Grassberger (1985) measure inside the ball

%% the fixed distance based estimates
dm = max(dis(:)); % maximum distance
edgs = [0 exp(linspace(lnDm(1), mean([lnDm(end) log(dm)]), k))];
for ii = 1:N
    ln = log( cumsum(histcounts(dis(ii,:), edgs)) );
    ln( ln==-Inf ) = 0; % if count is zero assign zero
    lnN(ii,:) = ln;
end
lnN = [sum(lnN)./sum(lnN~=0) - log(N); lnNm]';
lnD = [log(edgs(2:end)); lnDm]';

%% plotting the results
if nargout == 0
    figure
    set(gcf,'PaperSize',[10.5 3.5],'PaperPosition', [-.75 -.25 12 4]);
    subplot(121)
    set(gca,'fontsize',12)
    plot(lnD,lnN,'linewidth',1), axis tight
    ylabel('$\langle \ln \hat \mu(\epsilon)\rangle,\; \ln \left(n/N\right)$',...
        'Interpreter','Latex','FontSize',14)
    xlabel('$\ln(\epsilon),\; \langle \ln(\epsilon) \rangle$','Interpreter',...
        'Latex','FontSize',14)
    legend('fixed distance', 'fixed mass','location','southeast')
    grid on, pbaspect([1.618, 1, 1])
    subplot(122)
    plot(lnD(1:end-1,:),diff(lnD).\diff(lnN),'linewidth',1)
    set(gca,'fontsize',12)
    grid on, pbaspect([1.618, 1, 1])
    xlabel('$\ln(\epsilon),\; \langle \ln(\epsilon) \rangle$','Interpreter',...
        'Latex','FontSize',14)
    ylabel('$\hat D_\mathrm{P}, \bar D_\mathrm{P}$','Interpreter','Latex','FontSize',14)
    legend('fixed distance', 'fixed mass','location','south')
    axis([min(lnD(1,2)) max(lnD(end,2)) 0 ...
        ceil(max(max(diff(lnD).\diff(lnN))))])
end

%% this removes distances to temporally correlated nearest neighbors
function d = removecorrs(d, id)
    d = d( abs(id - id(1)) > w );
    d = d(1:k);
end

end
```

Python pwdim.py

```
"""Average pointwise dimension, fixed distance and fixed mass.

Python port of pwdim.m (Appendix A.3).

Copyright by David Chelidze. Python port.
"""
import numpy as np
from scipy.spatial import cKDTree
from scipy.special import digamma


def pwdim(x, m, d, k=500, N=None, w=0, rng=None):
```

```
    """Return (lnD, lnN), each (k, 2): column 0 fixed distance, 1 fixed mass.

    x : scalar time series
    m : delay
    d : embedding dimension
    k : number of distance partitions, or points inside a ball
    N : maximum number of averages (default min(2**12, n))
    w : Theiler window; 0 keeps temporal correlations

    lnD[:, 0] = ln(eps),                    lnN[:, 0] = <ln mu(eps)>
    lnD[:, 1] = <ln(ball holding n points)>, lnN[:, 1] = ln(n/N)
    """
    rng = np.random.default_rng() if rng is None else rng
    x = np.asarray(x, float).ravel()
    n = x.size - m*(d - 1)              # maximum size possible
    N = min(2**12, n) if not N else min(N, n)
    y = np.array([x[np.arange(n) + i*m] for i in range(d)])     # (d, n)

    tree = cKDTree(y.T)
    npt = k + 1 + 2*w                   # neighbors to look up
    sel = rng.permutation(n)[:N]
    dis, idx = tree.query(y[:, sel].T, k=npt)
    if w == 0:
        dis = dis[:, 1:]                # drop the zero self-distance
    else:                               # remove temporal correlations
        keep = np.abs(idx - idx[:, :1]) > w
        dis = np.array([dis[i][keep[i]][:k] for i in range(N)])

    # fixed mass
    lnDm = np.log(dis).mean(axis=0)
    lnNm = digamma(np.arange(1, k + 1)) - np.log(N)   # Grassberger (1985)

    # fixed distance
    dm = dis.max()
    edgs = np.concatenate([[0.0], np.exp(np.linspace(
        lnDm[0], np.mean([lnDm[-1], np.log(dm)]), k))])
    acc = np.zeros((N, k))
    for i in range(N):
        h, _ = np.histogram(dis[i], bins=edgs)
        ln = np.log(np.cumsum(h), where=np.cumsum(h) > 0,
                    out=np.zeros(k))
        acc[i] = ln                     # zero count counts as zero
    nz = (acc != 0).sum(axis=0)
    lnNd = np.divide(acc.sum(axis=0), nz, out=np.zeros(k),
                     where=nz > 0) - np.log(N)
    lnN = np.column_stack([lnNd, lnNm])
    lnD = np.column_stack([np.log(edgs[1:]), lnDm])
    return lnD, lnN
```

A.4 Projective Noise Reduction Algorithm

This algorithm should work on both flow and map generated data.

MATLAB podfilt.m

```
function [ xf ] = podfilt( x, l, m, d, q, f, u )
% PODFILT: Projective nonlinear noise reduction using SVD.
% CALL:
%   [ xf ] = podfilt( x, l, m, d, q, f, u );
% REQUIRED INPUT:
%   x -- scalar time series (data) in a column or row form
%   l -- number of points in a cloud of nearest neighbors or strand length
%   m -- delay time for the phase space reconstruction (see ami.m)
%   d -- embedding dimension (see fnns.m) (d < l)
%   q -- local embedding dimension (q < d)
```

```
% OPTIONAL INPUT:
%   f -- f = 1 (default) for cloud based or f = 0 strand based POD
%   u -- curvature handling (cloud based scheme):
%        u = 1 (default) do not adjust for curvature;
%        u = 0 temporal correction, c'_k = (c_{k-1} + c_{k+1})/2;
%        u = 2 spatial correction, eq. (curvcorr): c'_k is the mean of the
%              mass centres of the l spatial neighbors of point k.
%        Both corrections apply 2 c_k - c'_k and differ only in which
%        points supply c'_k. The cloud scheme with u = 0 diverges under
%        repeated application; u = 2 does not. The strand scheme accepts
%        u = 0 and ignores u = 2.
% OUTPUT:
%   xf -- filtered time series in matrix form
%
% This function uses proper orthogonal decomposition to filter noisy
% nonlinear time series. The input time series is used to reconstruct phase
% space using delay coordinate embedding (generating a multivariate time
% series). To avoid edge effects, the reconstruction is continued on both
% ends by finding the nearest neighbor (nn) to the end points in the
% reconstruction and following their trajectories. Point cloud based
% projective noise reduction is based on standard published procedure (see
% Kantz/Schreiber). For strand based projection, an l-length segment of
% this vector-valued time series is called a strand. Each strand is
% decomposed using SVD and only the first q dominant modes are used for
% noise reduction. The filtering is done considering all the corrections to
% each individual sample.
%
% copyright and written by David Chelidze 03/16/2012-04/06/2012, 11/16/17
% contact email: chelidze@uri.edu

%% input check and initialization
if nargin < 5
   error('Requires at least five input arguments!\n')
elseif min(size(x)) ~= 1
   error('Error: data should be a vector of scalar data points.\n')
elseif d <= q
    error('make subspace dimension q smaller than embedding dimension d\n')
else % normalize the input time series
    x = x(:)';
    nx = m*(d-1); % shrinkage in reconstruction
    if nargin < 6 || isempty(f) % default: cloud based projection
        f = 1;
    end
    if nargin < 7 || isempty(u) % default: do not adjust for curvature
        u = 1; % NOTE: cloud based projection with u = 0 diverges under
    end        % repeated application; the strand branch is fine with it
end
%% Embedding and Partitioning for the Nearest Neighbor Search
n = length(x); % original length of time series
nrp = n - nx; % number of reconstructed points
%% embedding and partitioning for nearest neighbor (nn) search
y = zeros(d, nrp); % initialize array for embedded vectors
for i = 1:d % reconstruct the phase space
    y(i,:) = x( (1:nrp) + (i-1)*m );
end % embedding
tree = createns(y','NSMethod','kdtree','Distance','euclidean');
R = eye(d,d); R(1,1) = 1000; R(d,d) = 1000; % weighting function
%% Padding (extending the reconstruction to deal with edge effects)
if f == 1 % Cloud Based Projection
    np = 1 + nx; % padding needed for the cloud based scheme
elseif l <= d
        error('Error: make string length (l) larger (l > d!)')
else
    l = floor(l/2) + 1; % strand middle point
    sl = 2*l - 1; % strand length
    np = l + nx; % padding needed for the strand based scheme
end
```

```
[pqi, ~] = knnsearch(tree,y(:,1)','k',np); % find analogue to the first pnt
pqi = pqi( pqi >  np ); % choose the acceptable nns
ya = y(:, pqi(1) - np : pqi(1) - 1); % padding from the left
[pqi, ~] = knnsearch(tree,y(:,end)','k',np); % find analogue to the last p
pqi = pqi( pqi <  nrp - np ); % choose the acceptable nns
yb = y(:, pqi(1) + 1 : pqi(1) + np); % padding from the right
yp = [ ya y yb ]; % padded phase space
npr = nrp + 2*np; % new size of the phase space
%% Projecting the Noise Out
Yh = zeros(d, npr); % stores adjusted phase space points
if f == 1 % do nearest neighbor projection
    b = zeros(d, npr); % stores the bias
    [pqi, ~] = knnsearch(tree,yp','Distance','euclidean','k',l);
    for k = 1:npr % for each reconstructed point
        Y = y(:, pqi(k,:)); % the nearest neighbors
        b(:, k) = mean(Y, 2); % bias mean vector (crude mean filter)
        [U, S, V] = svd( ( Y - b(:,k)*ones(1,l) )'*R, 0 ); % POD
        C = V(:,1:q)*S(1:q,1:q)*U(:,1:q)'; % projection
        Yh(:,k) = R\C(:,1) + b(:,k); % adjustment
    end
    if u == 2 % spatial correction, eq. (curvcorr)
        % pqi indexes the unpadded y; b is indexed over the padded yp,
        % where y(:,j) sits at column j + np. Entry (i,k,j) of the
        % reshaped array is b(i, pqi(k,j) + np).
        cp = mean(reshape(b(:, pqi(:) + np), d, npr, l), 3); % c'_k
        Yh = Yh + b - cp; % move the projection centre to 2 c_k - c'_k
    end
    if u == 0
        Yh = Yh(:,2:npr-1) + b(:,2:end-1) - (b(:,1:end-2) + b(:,3:end))/2;
    else
        Yh = Yh(:,2:npr-1);
    end
else % do the strand projection
    b = zeros(d,npr-sl+1); % stores bias
    wd = [0:l-1, l-2:-1:0]; % use tent weighting for adjusting neighbors
    wd = wd/sum(wd); % normalize (not required: see the division below)
    W = diag(wd); % weighting applied along the strand
    wa = zeros(1, npr); % stores the weight accumulated at each point
    for k = 1:npr-sl+1 % for each reconstructed point
        Y = yp(:,k+(0:sl-1)); % get the corresponding strand
        b(:,k) = mean(Y,2); % bias mean vector (crude mean filter)
        [U,S,V] = svd((Y-b(:,k)*ones(1,sl))'*R,0); % POD
        C = V(:,1:q)*S(1:q,1:q)*U(:,1:q)'; % projection
        Yh(:, k+(0:sl-1) ) = Yh(:, k+(0:sl-1) )...
                + ( R\C + b(:,k)*ones(1,sl) )*W; % adjustment with weight
        wa(k+(0:sl-1)) = wa(k+(0:sl-1)) + wd; % weight these points got
    end % filtered phase space, now adjust for curvature
    % trim the padded edges and normalize each point by the weight it
    % got, so the first and last l points are weighted averages too
    Yh = Yh(:,l+1:npr-l)./( ones(d,1)*wa(l+1:npr-l) );
    if u == 0 % now adjust for curvature
        Yh = Yh + b(:,2:end-1) - (b(:,1:end-2)+b(:,3:end))/2;
    end
end
%% Shifting coordinates so they align in columns
xf = zeros(d,n);
for k = 1:d
    xf(k,:) = Yh(k,(m*(d-k)+1):end-m*(k-1));
end
xf = mean(xf, 1);
```

Python `podfilt.py`

```
"""POD-based projective noise reduction.

Python port of podfilt.m (Appendix A.4). Works on both flow and map
generated data.
```

```
Copyright by David Chelidze. Python port.
"""
import numpy as np
from nlutil import embed, pad, unshift


def podfilt(x, l, m, d, q, f=1, u=1):
    """Return the filtered time series xf.

    x : scalar time series
    l : points in the neighbor cloud, or the strand length
    m : delay time
    d : embedding dimension (d < l)
    q : projection dimension (q < d)
    f : 1 (default) for the neighbor-cloud scheme, 0 for strands
    u : curvature handling.
        1 (default) leave the curvature alone;
        0 the TEMPORAL correction, c'_k = (b_{k-1} + b_{k+1})/2;
        2 the SPATIAL correction of eq. (curvcorr), c'_k the mean of the
          mass centres of the l spatial neighbours of point k.
        Both apply 2 c_k - c'_k, and differ only in which points supply
        c'_k. The chapter specifies the spatial one. See
        chk_curvcorr.py for the measured comparison.
        NOTE: the cloud scheme with u = 0 diverges under repeated
        application -- a temporal second difference of the neighbourhood
        mass centres amplifies the jitter that comes from the neighbour
        set changing discontinuously from point to point. u = 2 does not
        have that failure mode, since it averages over the same
        neighbour set the projection already uses. The strand branch is
        fine with u = 0, and that is what the Chapter 14 figures used.
    """
    x = np.asarray(x, float).ravel()
    nx = m*(d - 1)
    if d <= q:
        raise ValueError("make subspace dimension q smaller than embedding "
                         "dimension d")
    n = x.size
    y, nrp = embed(x, d, m)
    R = np.eye(d)
    R[0, 0] = R[-1, -1] = 1000.0          # weighting toward the interior

    if f == 1:
        npd = 1 + nx
    else:
        if l <= d:
            raise ValueError("make strand length l larger than d")
        l = l//2 + 1                      # strand middle point
        sl = 2*l - 1
        npd = l + nx
    yp, tree = pad(y, nrp, npd)
    npr = nrp + 2*npd

    Yh = np.zeros((d, npr))
    if f == 1:                            # neighbor-cloud projection
        b = np.zeros((d, npr))
        _, pqi = tree.query(yp.T, k=l)
        for k in range(npr):
            Y = y[:, pqi[k]]
            b[:, k] = Y.mean(axis=1)      # crude mean filter
            U, S, Vt = np.linalg.svd(((Y - b[:, k:k + 1]).T @ R),
                                     full_matrices=False)
            C = (Vt.T[:, :q]*S[:q]) @ U[:, :q].T
            Yh[:, k] = np.linalg.solve(R, C[:, 0]) + b[:, k]
        if u == 2:                        # spatial correction, eq. (curvcorr)
            # pqi indexes the UNPADDED trajectory y; b is indexed over the
            # padded one, where y[:, j] sits at column j + npd.
```

```
            cp = b[:, pqi + npd].mean(axis=2)
            Yh = Yh + b - cp
        Yh = Yh[:, 1:npr - 1]
        if u == 0:                          # temporal correction
            Yh = Yh + b[:, 1:-1] - (b[:, :-2] + b[:, 2:])/2
    else:                                   # strand projection
        b = np.zeros((d, npr - sl + 1))
        wd = np.concatenate([np.arange(0, l), np.arange(l - 2, -1, -1)])
        wa = np.zeros(npr)
        for k in range(npr - sl + 1):
            Y = yp[:, k:k + sl]
            b[:, k] = Y.mean(axis=1)
            U, S, Vt = np.linalg.svd(((Y - b[:, k:k + 1]).T @ R),
                                     full_matrices=False)
            C = (Vt.T[:, :q]*S[:q]) @ U[:, :q].T
            Yh[:, k:k + sl] += (np.linalg.solve(R, C) + b[:, k:k + 1])*wd
            wa[k:k + sl] += wd
        # trim and normalize by the accumulated weight, then curvature
        Yh = Yh[:, l:npr - l]/wa[l:npr - l]
        if u == 0:
            Yh = Yh + b[:, 1:-1] - (b[:, :-2] + b[:, 2:])/2
    return unshift(Yh, d, m, n, nx)
```

A.5 Smooth Projective Noise Reduction Algorithm

This algorithm is only intended to work on flow generated time series and will fail if applied to map generated data.

MATLAB sodfilt.m

```
function [ xf ] = sodfilt( x, l, m, d, q, w, r, tw )
% SODFILTER: SOD based noise reduction in nonlinear time series.
% CALL:
%   [ xf ] = sodfilt( x, l, m, d, q, w );
% REQUIRED INPUT:
%   x -- scalar time series (data) in a column or row form
%   l -- number of points in each strand (use odd number, or it will add 1)
%   m -- delay time for the phase space reconstruction (see ami.m/cds.m)
%   d -- embedding dimension (see fnns.m) (d < l!)
%   q -- local embedding dimension (q < d!!)
%  OPTIONAL INPUT PARAMETERS:
%   w -- weighting used in filtering (0--comb def, 1--tent, 2--exponential)
%   r -- strands per bundle (default 1, i.e. no bundle). With r > 1 the
%        r-1 nearest temporally uncorrelated strands join the base one,
%        each is decomposed separately, and the smoothed strands are
%        averaged with weights w(delta) = a^4/(delta^4 + a^4), Eq.
%        (loclinw) of Chapter 15, delta the distance from the base point
%        to a strand centre and a the distance to the farthest (r-th)
%        strand centre of the bundle.
%        Costs about r times as much and pays only on noisy records:
%        r = 4 brought the correlation dimension within 3% of the clean
%        value at 20% noise (r = 8 at 40%) where r = 1 left it 44% high
%        (see the table in Sec. 13.2).
%   tw -- Theiler window excluding temporally correlated strands from the
%        bundle (default (d-1)*m + 2*l + 1, one strand span)
% OUTPUT:
%   xf -- filtered time series
%
% This function uses smooth orthogonal decomposition to filter noisy
% nonlinear time series. The input time series x is used to reconstruct
% phase space via delay coordinate embedding (generating a multivariate
% time series). To avoid edge effects, the reconstruction is continued on
% both ends by finding the nearest neighbor to the end points in the
% reconstruction and following their trajectories. An l-length segment of
% this vector-valued time series is called a strand. Each strand is
```

```
% decomposed using SOD and only the first q smooth modes are used for noise
% reduction. The filtering is done considering all the corrections to each
% individual sample.
%
% copyright and written by David Chelidze 03/16/2012 04/06/2012 11/16/17
% contact email: chelidze@uri.edu

%% input check and initialization
if nargin < 5
   error('Requires at least five input arguments!\n')
elseif min(size(x)) ~= 1
   error('Error: data should be a vector of scalar data points.\n')
else % normalize the input time series
    x = x(:)';
    nx = m*(d-1); % shrinkage in reconstruction
    n = length(x)- nx; % <- the maximum size possible!
    if nargin < 6 || isempty(w)
        w = 0;
    end
    if nargin < 7 || isempty(r)
        r = 1; % no bundle
    end
end
y = zeros(d, n); % initialize array for embedded vectors
%% check if input variables are feasible
if l <= d || isempty(l)
    error('make string length l larger than the embedding dimension d\n')
end
if d <= q
    error('make subspace dimension q smaller than embedding dimension d\n')
end
l = floor(l/2); % distance from first point to mid-point
sl = 2*l + 1; % strand length
if nargin < 8 || isempty(tw)
    tw = nx + sl; % one strand span, so no neighbor overlaps the base
end
%% embedding and partitioning for nearest neighbor (nn) search
for i = 1:d % reconstruct the phase space
    y(i,:) = x((1:n) + (i-1)*m);
end % embedding
tree = createns(y','NSMethod','kdtree','Distance','euclidean');
%% padding (extending edges for filtering and shrinkage mitigation)
np = l + nx; % padding required to deal with edge effects
[pqi, ~] = knnsearch(tree,y(:,1)','Distance','euclidean','k',np);
pqi = pqi( pqi >  np ); % find suitable nearest neighbor
ya = y(:,pqi(1)-np:pqi(1)-1); % pad the left side
[pqi, ~] = knnsearch(tree,y(:,end)','Distance','euclidean','k',np);
pqi = pqi( pqi <  n - np ); % find suitable neighbor
yb = y(:,pqi(1)+1:pqi(1)+np); % pad the right side
y = [ ya y yb ]; % padded phase space
npr = n+2*np; % new number of padded reconstructed points
%% bundle setup (only needed for r > 1)
if r > 1
    cen = (l+1):(npr-l); % strand centers available in the padded data
    btree = createns(y(:,cen)','NSMethod','kdtree',...
                     'Distance','euclidean');
    nq = min(numel(cen), r + 4*tw); % enough to survive the Theiler cut
end
%% weighting function for filtering
if w == 1 % use tent weighting function W (good results)
    wd = [0:l, l-1:-1:0];
elseif w == 2 % use exponential weighting function W (try if you want)
    wd = exp(-([l:-1:0, 1:l]));
else % use comb weighting function W (no averaging)
    wd = [zeros(1,l) 1 zeros(1,l)];
end
wd = wd/sum(wd); % normalize (not required: see the division below)
```

```
W = diag(wd); % weighting applied along the strand
%% SOD filtering part
Yh = zeros(d, npr); % stores adjusted trajectory points
wa = zeros(1, npr); % stores the weight accumulated at each point
for k = 1:npr-sl+1 % each reconstructed point
    if r == 1 % the base strand on its own
        cn = k + l; % its center
        Wn = 1;
    else % gather the bundle: base strand plus r-1 nearest neighbors
        [ci, dis] = knnsearch(btree, y(:,k+l)', 'k', nq);
        ci = cen(ci); % back to indices in the padded data
        ok = abs(ci - (k+l)) > tw; % temporally uncorrelated only
        ci = ci(ok); dis = dis(ok);
        nb = min(r-1, numel(ci));
        cn = [k+l, ci(1:nb)];
        dis = [0, dis(1:nb)];
        ok = (cn-l >= 1) & (cn+l <= npr); % full strands only
        cn = cn(ok); dis = dis(ok);
        a = dis(end); % scale: the farthest strand of the bundle
        if a > 0
            Wn = a^4 ./ ( dis.^4 + a^4 ); % falls off with distance
        else
            Wn = ones(size(dis));
        end
    end
    Ya = zeros(d, sl); % accumulates the smoothed strands of the bundle
    for j = 1:numel(cn)
        Y = y(:, cn(j)-l : cn(j)+l); % strand j of the bundle
        b = mean(Y, 2); % its bias vector
        Y = Y - b*ones(1,sl); % unbiased
        [U,~,X,C,~] = gsvd( Y', diff(Y'), 0 ); % SOD of this strand
        X = fliplr(X); % SOMs
        U = fliplr(U); % SOCs
        C = flipud(diag(C)); % SOVs
        % project onto the smooth subspace, add back the bias, weight it
        Ya = Ya + Wn(j)*( X(:,1:q)*diag(C(1:q))*U(:,1:q)' ...
                          + b*ones(1,sl) );
    end
    Ya = Ya/sum(Wn); % the weighted average over the bundle
    Yh(:, k:k+sl-1) = Yh(:, k:k+sl-1) + Ya*W;
    wa(k:k+sl-1) = wa(k:k+sl-1) + wd; % weight received by these points
end
%% final averaging to get filtered time series
% trim the padded edges and normalize each point by the weight it got:
% interior points are covered by all (2l+1) strands, the first and last l
% by fewer, so without this they would come out attenuated
Yh = Yh(:,l+1:npr-l)./( ones(d,1)*wa(l+1:npr-l) );
%% shifting coordinates so they align in columns
xf = zeros(d,n+nx);
for k = 1:d
    xf(k,:) = Yh(k,m*(d-k)+1:end-m*(k-1));
end
xf = mean(xf, 1); % average for filtered time series
```

Python `sodfilt.py`

```
"""SOD-based smooth projective noise reduction.

Python port of sodfilt.m (Appendix A.5). Flow generated data only: the
smooth decomposition needs a data-based derivative, which map generated
data does not have.

The optional nearest-neighbour strand bundle of Section 13.2, step 2, is
included here as the argument r. With r = 1 (the default) each point is
filtered from its own strand alone. With r > 1 the r-1 nearest
temporally uncorrelated strands join it, each is decomposed separately,
and the smoothed strands are averaged with weights that fall off with
```

```
distance from the base strand -- the reading the chapter's step 2(e)
states. The weight is w(delta) = a^4/(delta^4 + a^4), eq. (loclinw) of
Chapter 15, with a the distance from the base point to the farthest
(r-th) strand centre of its bundle, so the scale follows the local
density and the farthest strand gets half the weight of the base one.
It costs roughly r times as much and is worth it only on noisy records;
see the table in Section 13.2.

Copyright by David Chelidze. Python port.
"""
import numpy as np
from scipy.spatial import cKDTree
from nlutil import embed, pad, strand_weights, unshift
from gsvd import smooth_project


def sodfilt(x, l, m, d, q, w=0, r=1, tw=None):
    """Return the filtered time series xf.

    x  : scalar time series
    l  : points per strand (even values are increased by one)
    m  : delay time for the reconstruction
    d  : embedding dimension (d < l)
    q  : projection dimension (q < d)
    w  : weighting along the strand -- 0 comb (default), 1 tent,
         2 exponential
    r  : strands per bundle (default 1, no bundle). r = 4 is a good
         choice on noisy records.
    tw : Theiler window excluding temporally correlated strands from the
         bundle. Defaults to (d-1)*m + 2l + 1, one strand span, so no
         neighbour overlaps the base strand.
    """
    x = np.asarray(x, float).ravel()
    nx = m*(d - 1)                      # shrinkage in reconstruction
    if l <= d:
        raise ValueError("make strand length l larger than the embedding "
                         "dimension d")
    if d <= q:
        raise ValueError("make subspace dimension q smaller than embedding "
                         "dimension d")
    y, n = embed(x, d, m)
    l = l//2                            # distance to the mid-point
    sl = 2*l + 1                        # strand length
    npd = l + nx                        # padding for the edge effects
    y, _ = pad(y, n, npd)
    npr = n + 2*npd
    wd = strand_weights(l, w)

    if r > 1:
        if tw is None:
            tw = nx + sl                # one strand span
        cen = np.arange(l, npr - l)     # strand centres available
        tree = cKDTree(y[:, cen].T)

    Yh = np.zeros((d, npr))             # adjusted trajectory points
    wa = np.zeros(npr)                  # weight accumulated at each point
    for k in range(npr - sl + 1):
        if r == 1:
            Y = y[:, k:k + sl]
            b = Y.mean(axis=1, keepdims=True)      # strand bias vector
            Ya = smooth_project(Y - b, q) + b      # smooth projection
        else:
            base = k + l
            nq = min(cen.size, r + 4*tw)
            dis, idx = tree.query(y[:, base], k=nq)
            c = cen[np.atleast_1d(idx)]
            dis = np.atleast_1d(dis)
```

```
            keep = np.abs(c - base) > tw          # temporally uncorrelated
            c, dis = c[keep][:r - 1], dis[keep][:r - 1]
            c = np.concatenate([[base], c])
            dis = np.concatenate([[0.0], dis])
            ok = (c - l >= 0) & (c + l < npr)
            c, dis = c[ok], dis[ok]
            a = dis[-1]                           # farthest strand of the bundle
            Wn = a**4/(dis**4 + a**4) if a > 0 else np.ones(dis.size)
            acc = np.zeros((d, sl))
            for ci, wi in zip(c, Wn):
                S = y[:, ci - l:ci + l + 1]
                B = S.mean(axis=1, keepdims=True)
                acc += wi*(smooth_project(S - B, q) + B)
            Ya = acc/Wn.sum()
        Yh[:, k:k + sl] += Ya*wd
        wa[k:k + sl] += wd
    # trim the padding and normalize by the weight each point received,
    # so the first and last l points are weighted averages too
    Yh = Yh[:, l:npr - l]/wa[l:npr - l]
    return unshift(Yh, d, m, n, nx)
```

A.6 Largest Short-Time Lyapunov Exponent Algorithm

MATLAB lyapunov.m

```
function [ ns, lnd ] = lyapunov( x, d, m, N, r, n, w )
% This function calculates metrics for the largest short-time Lyapunov
% exponent from a scalar time series after reconstructing its phase space
% using delay coordinates
%
% CALL: [ ns, lnd ] = lyapunov( x, d, m, N, r, n, w );
%   If no output is requested it will just plot the results
%
% INPUT:
%   x -- scalar time series
%   d -- embedding dimension
%   m -- delay parameter
%   N -- number of initial (fiducial) points to use (should be large)
%   r -- number of nearest neighbors to use (r = 1 should work). For
%        r > 1 the neighbor trajectories are averaged with the weight
%        w(delta) = a^4/(delta^4 + a^4), Eq. (loclinw) of Chapter 15,
%        delta the distance from the fiducial point to the neighbor and
%        a the distance to its r-th (farthest) neighbor.
%   n -- maximum number of steps ahead to proceed
%   w -- (optional) supply 0 to not remove temporal correlations
%        or number of points to remove. The default is p=d*m.
%        NOTE: only need to remove these correlations for flow data!!
%
% OUTPUT:
%   ns -- the time steps used
%   lnd -- ln of averaged distances
%   If not output is requested it plots the results. rescale results using
%   appropriate sampling time interval to get actual short-time maximal LE.
%
% (c) by David Chelidze 11/04/2005
%     modified by DC on 11/16/2005, 4/11/2011, 11/26/2012, 11/11/2013,
%     11/2/17, 12/11/17, 1/5/18

if nargin < 6
    error('Error: Needs at least six input variables!')
end
if min(size(x)) ~= 1 % check data and make a raw vector of scalar data
   error('Error: First input should be a vector of scalar data points.')
else % make row vector with zero mean and unit variance
```

```
    x = (x(:)' - mean(x))/std(x);
end
nm = length(x) - (d-1)*m; % max number of points possible in phase space
if nargin < 7 || isempty(w)
    w = d*m; % remove temporal correlations of this length, open for debate
end
if N == 0 || isempty(N) % set the default
    N = min(2^12, nm);
else % use the provided or max possible points
    N = min(N, nm);
end
% reconstruct the phase space
y = zeros(d, nm);
for i = 0:d-1
    y(i+1,:) = x( (1:nm) + i*m );
end

% define N random initial conditions for `fiducial' points
indx = randperm(nm-n,N);
lnd = zeros(1,n); % initialize log of average distance array

% partition all suitable phase space points for fast searching
ns = createns(y(:,1:end-n)','NSMethod','kdtree','Distance','euclidean');
% search for the k points nearest to the test points (+ 2*w if removing
% temporal correlations)
np = r + 2*w + 1; % total nearest neighbors to look up
[idx, dis] = knnsearch(ns,y(:,indx)','Distance','euclidean','k',np);
if w == 0 % do not remove temporarily correlated nearest neighbor distances
    dis = dis(:,2:end); % only drop the first zero distance
    idx = idx(:,2:end);
else % remove temporal correlations
    [dis, idx] = cellfun(@removecorrs, mat2cell(dis,ones(N,1),np),...
                 mat2cell(idx,ones(N,1),np),'UniformOutput',false);
    dis = cell2mat(dis);
    idx = cell2mat(idx);
end

for k = 1:N % for each fiducial trajectory
    a = dis(k,end); % scale: the r-th neighbor distance
    if a > 0
        W = a^4 ./ ( dis(k,:).^4 + a^4 ); % weighting function
    else
        W = ones(1,r);
    end
    dist = zeros(1, n); % initialize local distances
    for inn = 1:r % all nearest neighbor trajectories
        % find the difference between nn and fiducial trajectories
        tmp = y(:, idx(k,inn)+(1:n)) - y(:, indx(k)+(1:n));
        % normalized distances between trajectories:
        dist = dist + W(inn)*sqrt(sum(tmp.*tmp));
    end
    lnd = lnd + log(dist/sum(W)); % log of average distance
end % k loop
lnd = lnd/N; % normalize by the number of reference/fiducial points
ns = 1:n; % just how far we went

if nargout == 0 % plot the results
    figure
    set(gcf,'PaperSize',[10.5 3.5],'PaperPosition', [-.75 -.25 12 4]);
    subplot(121)
    set(gca,'fontsize',12)
    plot(ns,lnd,'linewidth',1), axis tight
    grid on, pbaspect([1.618, 1, 1])
    xlabel('$n$','Interpreter','Latex','FontSize',14)
    ylabel('$\Delta_n = \langle \ln \delta_n - \ln \delta_0 \rangle$',...
        'Interpreter','Latex','FontSize',14)
    subplot(122)
```

```
    set(gca,'fontsize',12)
    plot(diff(ns).\diff(lnd),'linewidth',1), axis tight
    grid on, pbaspect([1.618, 1, 1])
    xlabel('$n$','Interpreter','Latex','FontSize',14)
    ylabel('$\Delta t \hat\lambda_1=\mathrm{d}\Delta_n/\mathrm{d}n$',...
        'Interpreter','Latex','FontSize',14)
end
%% this removes temporally correlated nearest neighbors
function [d, id] = removecorrs(d, id)
    cond =  abs(id - id(1)) > w;
    d = d(cond);
    id = id(cond);
    d = d(1:r);
    id = id(1:r);
end
end
```

Python `lyapunov.py`

```
"""Largest short-time Lyapunov exponent metrics.

Python port of lyapunov.m (Appendix A.6). Returns the mean logarithmic
divergence against step number; the exponent is the slope of the scaling
region, divided by the sampling interval.

With r > 1 the neighbor trajectories are averaged with the weight
w(delta) = a^4/(delta^4 + a^4), eq. (loclinw) of Chapter 15, where delta
is the distance from the fiducial point to the neighbor and a is the
distance to its r-th (farthest) neighbor. The weight is the same in
lyapunov.m. For r = 1 -- the value the header recommends -- the single
weight cancels and the result is independent of it.

Copyright by David Chelidze. Python port.
"""
import numpy as np
from scipy.spatial import cKDTree


def lyapunov(x, d, m, N, r, n, w=None, rng=None):
    """Return (ns, lnd): the step numbers and the mean log divergence.

    x : scalar time series
    d : embedding dimension
    m : delay
    N : number of fiducial points (should be large)
    r : number of nearest neighbors per fiducial point (1 works)
    n : maximum number of steps ahead
    w : Theiler window; 0 keeps temporal correlations. Default d*m, and
        needed for flow data.
    """
    rng = np.random.default_rng() if rng is None else rng
    x = np.asarray(x, float).ravel()
    x = (x - x.mean())/x.std()
    nm = x.size - (d - 1)*m
    if w is None:
        w = d*m                     # open for debate
    N = min(2**12, nm) if not N else min(N, nm)
    y = np.array([x[np.arange(nm) + i*m] for i in range(d)])

    indx = rng.permutation(nm - n)[:N]          # fiducial points
    tree = cKDTree(y[:, :-n].T)
    npt = r + 2*w + 1
    dis, idx = tree.query(y[:, indx].T, k=npt)
    if w == 0:
        dis, idx = dis[:, 1:], idx[:, 1:]
    else:                                       # remove temporal correlations
        keep = np.abs(idx - idx[:, :1]) > w
```

```
        dis = np.array([dis[i][keep[i]][:r] for i in range(N)])
        idx = np.array([idx[i][keep[i]][:r] for i in range(N)])

    lnd = np.zeros(n)
    steps = np.arange(1, n + 1)
    for k in range(N):
        a = dis[k, -1]                          # r-th neighbor distance
        W = a**4/(dis[k]**4 + a**4) if a > 0 else np.ones(r)
        dist = np.zeros(n)
        for j in range(r):
            tmp = y[:, idx[k, j] + steps] - y[:, indx[k] + steps]
            dist = dist + W[j]*np.sqrt(np.sum(tmp*tmp, axis=0))
        lnd = lnd + np.log(dist/W.sum())
    return steps, lnd/N
```

A.7 Sample MATLAB Function for Local Linear Model Prediction

MATLAB loclin.m

```
function yp = loclin( ym, yq, n, N, w, tw, iq )
% LOCLIN local linear modeling for prediction
%
% INPUT:
%   ym -- reconstructed phase space points (trajectory) used for modeling
%         should be d times M, where M is the number of points
%   yq -- initial point to start the prediction from
%         should be d times 1
%   n -- number of nearest neighbors to use for prediction
%   N -- how far ahead to do the prediction (number of time steps)
%   w -- optional, to use weighting w = 1, otherwise use no weighting.
%        The weight is w(delta) = a^4/(delta^4 + a^4), Eq. (loclinw) of
%        Chapter 15, with a the distance from the query point to its n-th
%        (last requested) neighbor, so the scale follows the local
%        density and the n-th neighbor gets half the weight of a
%        coincident point.
%   tw -- optional Theiler window (default 0: no exclusion); used with iq
%   iq -- optional column of ym that yq is (in-sample query); at step k
%         the columns j with |j - (iq+k-1)| <= tw are not used as neighbors
% OUTPUT:
%   yp -- predicted phase space trajectory starting from yq point
%
% EXAMPLE:
%   x = x(:)'; % experimental time series or generated from some model
%   np = size(x,2) - (d-1)*m; % maximum number of reconstructed points
%   y = zeros(d,np); % reconstructed phase space trajectory
%   for k = 1:d
%       y(k,:) = x((1:np) + (k-1)*m);
%   end
%   % use half points for modeling and other half for testing:
%   yp = loclin( y(:,1:np/2), y(:,np/2 + 10), 8, 1000 );
%   plot(yp(1,:)), hold on, plot(y(1,n/2+10:n/2+1000))
%
% written and copyrighted by David Chelidze on 12/4/17

[d, M] = size(ym);
ns = createns(ym(:,1:end-1)','NSMethod','kdtree','Distance','euclidean');
yp = [yq zeros(d,N)];
if nargin < 5 || w ~= 1 % use no weighting
    w = 0;
end
if nargin < 7 % no in-sample query index: no Theiler exclusion
    tw = 0; iq = 0;
```

```
end
excl = tw > 0;
if excl
    kq = min(n + 2*tw + 1, M - 1); % extra neighbors to discard
else
    kq = n;
end

for k = 1:N
    [indx, dis] = knnsearch(ns,yq','Distance','euclidean','k',kq);
    if excl % Theiler exclusion around the current trajectory point
        keep = abs(indx - (iq + k - 1)) > tw;
        indx = indx(keep); dis = dis(keep);
        indx = indx(1:n); dis = dis(1:n);
    end
    Yn = [ym(:,indx); ones(1,n)];   % (d+1) x n design X~
    Ynp1 = ym(:,indx+1);            % d x n images
    a = dis(end);                   % distance to the n-th neighbor
    if w == 1 && a > 0
        sw = sqrt(a^4 ./ ( dis.^4 + a^4 ))'; % square roots of weights, n x 1
    else
        sw = ones(n,1);
    end
    % weighted least squares on sqrt(W)*X~' directly (QR), no Gram matrix
    B = ( (sw.*Yn') \ (sw.*Ynp1') )';     % d x (d+1) = [A, b]
    yp(:,k+1) = B*[yq; 1];
    yq = yp(:,k+1);
end
```

Python `loclin.py`

```
"""Local linear model prediction.

Python port of loclin.m (Appendix A.7).

Copyright by David Chelidze. Python port.
"""
import numpy as np
from scipy.spatial import cKDTree


def loclin(ym, yq, n, N, w=0, tw=0, iq=None):
    """Predict N steps ahead from yq using local linear models.

    ym : (d, M) reconstructed trajectory used for modeling
    yq : (d,) point to start the prediction from
    n  : number of nearest neighbors per model
    N  : number of steps to predict
    w  : 1 to weight neighbors by distance, anything else for no weights.
         The weight is w(delta) = a^4/(delta^4 + a^4), eq. (loclinw) of
         Chapter 15, with a the distance from the query point to its n-th
         (last requested) neighbor, so the scale follows the local density
         and the n-th neighbor gets half the weight of a coincident point.
    tw : Theiler window (default 0: no exclusion). Used only with iq.
    iq : column of ym that yq is (in-sample query), or None. At step k the
         columns j with |j - (iq + k)| <= tw are not used as neighbors.

    Returns yp, shape (d, N+1), with yp[:, 0] = yq.
    """
    ym = np.atleast_2d(np.asarray(ym, float))
    d, M = ym.shape
    yq = np.asarray(yq, float).ravel().copy()
    tree = cKDTree(ym[:, :-1].T)
    excl = iq is not None and tw > 0
    kq = min(n + 2*tw + 1, M - 1) if excl else n   # extra neighbors to discard
    yp = np.zeros((d, N + 1))
    yp[:, 0] = yq
```

```
    for k in range(N):
        dis, idx = tree.query(yq, k=kq)
        dis, idx = np.atleast_1d(dis), np.atleast_1d(idx)
        if excl:                                    # Theiler exclusion
            keep = np.abs(idx - (iq + k)) > tw
            dis, idx = dis[keep][:n], idx[keep][:n]
        Yn = np.vstack([ym[:, idx], np.ones(n)])    # (d+1, n) design X~
        Ynp1 = ym[:, idx + 1]                       # (d, n) images
        a = dis[-1]                                 # n-th neighbor distance
        if w == 1 and a > 0:
            sw = np.sqrt(a**4/(dis**4 + a**4))      # square roots of weights
        else:
            sw = np.ones(n)
        # weighted least squares on sqrt(W) X~^T directly (SVD), no Gram matrix
        B = np.linalg.lstsq(sw[:, None]*Yn.T, sw[:, None]*Ynp1.T,
                            rcond=None)[0].T        # (d, d+1) = [A, b]
        yq = B @ np.append(yq, 1.0)
        yp[:, k + 1] = yq
    return yp
```

A.8 False Nearest Neighbors and Cao's Statistics

The statistics of Chapter 10 from one nearest-neighbor search per embedding dimension. `fnn` normalizes the record to $\sigma_x = 1$, builds the forward delay vectors, finds for every point its k nearest neighbors outside the Theiler window w, Eq. (10.12), selects among them by the strand criterion of Eq. (10.21) when $k > 1$ and $l > 0$, and returns the separations ϵ_i and δ_i of Eq. (10.1) for $d = 1, \ldots, d_{\max}$, together with the surrogate δ_i of Section 10.1.1. `fnnfrac` turns them into the original, Kennel, TISEAN, reliable and composite fractions, Eqs. (10.3), (10.5), (10.8), (10.13) and (10.20), for given thresholds, so a sweep over r or s costs nothing beyond the search. `cao` returns $E_1(d)$ and $E_2(d)$ of Eqs. (10.10)–(10.11) in the maximum norm with the same window. The MATLAB search `nnsearch` is shared by `fnn` and `cao`; in Python it is a function of the one module. Every figure of Chapter 10 was produced with the Python version (`python/mkfig_CFNNO.py` and its companions).

MATLAB `fnn.m`

```
function [eps, dlt, dsur] = fnn(x, tau, dmax, w, k, l)
% FNN nearest-neighbor separations for the false nearest neighbor test
%  and the composite fraction of Chapter "Selecting Embedding Dimension".
%
% CALL: [eps, dlt, dsur] = fnn(x, tau, dmax, w, k, l);
%
% INPUT:
%   x    -- scalar time series (normalized inside, so sigma_x = 1)
%   tau  -- delay
%   dmax -- largest embedding dimension
%   w    -- Theiler window in samples (0 keeps all pairs), Eq. (rnns)
%   k, l -- (optional) candidate neighbors and strand half-length of the
%           composite-fraction selection, Eq. (CFrac); default 1, 0
%
% OUTPUT (cell arrays, entry d holds the N - d*tau pairs of dimension d):
%   eps  -- d-dimensional separation eps_i to the selected neighbor
%   dlt  -- appended separation delta_i = |x(i+d*tau) - x(j+d*tau)|
%   dsur -- surrogate delta_i, neighbor's appended coordinate drawn at
%           random from the record
% Use FNNFRAC to turn these into the fractions, and CAO for E1, E2.
%
% copyright by David Chelidze 2026
if nargin < 6, l = 0; end
if nargin < 5, k = 1; end
```

```
x = x(:)'; x = (x - mean(x))/std(x); N = numel(x);
eps = cell(dmax, 1); dlt = eps; dsur = eps;
for d = 1:dmax
    n = N - d*tau;
    Y = zeros(n, d);
    for q = 1:d, Y(:, q) = x((1:n) + (q-1)*tau); end
    [J, D] = nnsearch(Y, w, k, 'euclidean');
    i = (1:n)';
    if k > 1 && l > 0                          % strand selection, Eq. (CFrac)
        S = zeros(n, k);
        for q = -l:l
            iq = min(max(i + q, 1), n); jq = min(max(J + q, 1), n);
            for m = 1:k
                S(:, m) = S(:, m) + sqrt(sum((Y(jq(:, m), :) - Y(iq, :)).^2, 2));
            end
        end
        [~, m] = min(S./max(D, realmin), [], 2);
        lin = sub2ind([n, k], i, m); j = J(lin); e = D(lin);
    else
        j = J(:, 1); e = D(:, 1);
    end
    eps{d} = e;
    dlt{d} = abs(x(i + d*tau) - x(j + d*tau))';
    dsur{d} = abs(x(i + d*tau) - x(randi(N, n, 1)))';
end
end
```

MATLAB nnsearch.m

```
function [Jk, Dk] = nnsearch(Y, w, k, metric)
% NNSEARCH k nearest neighbors of every row of Y outside a Theiler window.
%
% CALL: [J, D] = nnsearch(Y, w, k, metric);
%
% INPUT:
%   Y      -- n x d matrix of reconstructed points (rows)
%   w      -- Theiler window in samples: neighbors j of i need |i-j| > w
%   k      -- number of neighbors
%   metric -- 'euclidean' or 'chebychev' (maximum norm), as in knnsearch
%
% OUTPUT:
%   J, D   -- n x k indices and distances, nearest first
%
% copyright by David Chelidze 2026
n = size(Y, 1); K = min(n, k + 2*w + 1);
Jk = zeros(n, k); Dk = zeros(n, k); todo = (1:n)';
[J, D] = knnsearch(Y, Y, 'K', K, 'Distance', metric);
while ~isempty(todo)
    ok = abs(J - todo) > w; cnt = cumsum(ok, 2);
    done = cnt(:, end) >= k;
    for r = find(done)'
        sel = ok(r, :) & cnt(r, :) <= k;
        Jk(todo(r), :) = J(r, sel); Dk(todo(r), :) = D(r, sel);
    end
    todo = todo(~done);
    if ~isempty(todo)                          % too many close-in-time points
        K = min(n, 2*K);
        [J, D] = knnsearch(Y, Y(todo, :), 'K', K, 'Distance', metric);
    end
end
end
```

MATLAB fnnfrac.m

```
function f = fnnfrac(eps, dlt, r, s, a)
% FNNFRAC the false nearest neighbor fractions from the output of FNN.
%
```

```
% CALL: f = fnnfrac(eps, dlt, r, s, a);
%
% INPUT:
%   eps, dlt -- cell arrays from FNN (data dlt, or dsur for surrogates)
%   r, s, a  -- ratio threshold, size threshold in units of sigma_x, and
%               Kennel's A_tol (defaults 10, 0.5, 2)
%
% OUTPUT: structure of row vectors against d = 1:dmax
%   f.orig   -- original fraction f_nn(d;r), Eq. (fnnFrac)
%   f.kennel -- Kennel fraction with both criteria, Eq. (OrKennel)
%   f.tisean -- TISEAN fraction, Eq. (tisean) (zero if no admissible pair)
%   f.rel    -- reliable fraction, Eq. (K_Rel)
%   f.comp   -- composite fraction R_nn(d;r,s), Eq. (new)
%
% copyright by David Chelidze 2026
if nargin < 5, a = 2; end
if nargin < 4, s = 0.5; end
if nargin < 3, r = 10; end
dmax = numel(eps);
f = struct('orig', zeros(1, dmax), 'kennel', zeros(1, dmax), ...
           'tisean', zeros(1, dmax), 'rel', zeros(1, dmax), 'comp', zeros(1, dmax));
for d = 1:dmax
    e = eps{d}; dl = dlt{d};
    ratio = dl > r*e; adm = r*e < 1;
    f.orig(d) = mean(ratio);
    f.kennel(d) = mean(ratio | (dl.^2 + e.^2 > a^2));
    f.tisean(d) = sum(ratio & adm)/max(sum(adm), 1);
    f.rel(d) = mean(dl > s);
    f.comp(d) = mean(ratio | (dl > s));
end
end
```

MATLAB cao.m

```
function [E1, E2] = cao(x, tau, dmax, w)
% CAO Cao's statistics E1(d) and E2(d) for the minimum embedding dimension.
%
% CALL: [E1, E2] = cao(x, tau, dmax, w);
%
% INPUT:
%   x    -- scalar time series (normalized inside)
%   tau  -- delay
%   dmax -- largest embedding dimension
%   w    -- Theiler window in samples (0 for none)
%
% OUTPUT:
%   E1, E2 -- row vectors for d = 1:dmax, Eqs. (eq:ed_caoE1), (eq:ed_caoE2);
%             maximum norm, nearest neighbor outside the Theiler window
%
% copyright by David Chelidze 2026
x = x(:)'; x = (x - mean(x))/std(x); N = numel(x);
E = zeros(1, dmax + 1); Es = E;
for d = 1:dmax + 1
    n = N - d*tau;
    Y = zeros(n, d);
    for q = 1:d, Y(:, q) = x((1:n) + (q-1)*tau); end
    [J, D] = nnsearch(Y, w, 1, 'chebychev');
    i = (1:n)'; j = J(:, 1); e = D(:, 1);
    dl = abs(x(i + d*tau) - x(j + d*tau))';
    ok = e > 0;
    E(d) = mean(max(e(ok), dl(ok))./e(ok));
    Es(d) = mean(dl);
end
E1 = E(2:end)./E(1:end-1);
E2 = Es(2:end)./Es(1:end-1);
end
```

Python `fnn.py`

```python
"""False nearest neighbors, composite fraction, and Cao's statistics.

Python twin of fnn.m (Appendix A). Every statistic of Chapter `embdim`
from one nearest-neighbor search per embedding dimension:

  fnn(x, tau, dmax, w, k, l)  -> eps, dlt, dsur   (per-pair separations)
  fnnfrac(eps, dlt, r, s, a)  -> the five fractions f_nn, f_hat, f_tld,
                                 f_bar and R_nn against d
  cao(x, tau, dmax, w)        -> Cao's E1(d), E2(d)

The record is normalized to zero mean and unit variance, so sigma_x = 1
and every distance is in units of sigma_x. Delay vectors are forward,
x_i = [x_i, x_{i+tau}, ..., x_{i+(d-1)tau}], and only the N - d tau
points whose (d+1)-th coordinate x_{i+d tau} exists are used.

Copyright by David Chelidze. Python version.
"""
import numpy as np
from scipy.spatial import cKDTree


def embed(x, d, tau, n):
    """n forward delay vectors of dimension d as rows."""
    return np.array([x[np.arange(n) + q*tau] for q in range(d)]).T


def nnsearch(Y, w, k, p=2.0):
    """k nearest neighbors of every row of Y outside the Theiler window w,
    eq. (rnns). Returns index array J (n x k) and distance array D."""
    n = Y.shape[0]
    tree = cKDTree(Y)
    i = np.arange(n)
    K = min(n, k + 2*w + 1)
    D, J = tree.query(Y, k=K, p=p, workers=-1)
    Dk, Jk = np.empty((n, k)), np.empty((n, k), int)
    todo = i
    while todo.size:
        ok = np.abs(J - todo[:, None]) > w
        cnt = np.cumsum(ok, axis=1)
        done = cnt[:, -1] >= k
        sel = ok[done] & (cnt[done] <= k)      # exactly k per finished row
        Dk[todo[done]] = D[done][sel].reshape(-1, k)
        Jk[todo[done]] = J[done][sel].reshape(-1, k)
        todo = todo[~done]
        if todo.size:                          # too many close-in-time points
            K = min(n, 2*K)
            D, J = tree.query(Y[todo], k=K, p=p, workers=-1)
    return Jk, Dk


def fnn(x, tau, dmax, w=0, k=1, l=0, rng=None):
    """Nearest-neighbor separations for d = 1..dmax.

    x    : scalar record;  tau : delay;  dmax : largest dimension
    w    : Theiler window in samples, eq. (rnns); 0 keeps all pairs
    k, l : number of candidate neighbors and strand half-length of the
           composite-fraction selection, eq. (CFrac); k=1, l=0 is the
           plain nearest neighbor
    Returns three lists indexed by d-1: eps[d-1] the d-dimensional
    separations eps_i, dlt[d-1] the appended separations delta_i, and
    dsur[d-1] the surrogate delta_i with the neighbor's appended
    coordinate drawn at random from the record (Section sec:ed_records).
    """
    rng = np.random.default_rng(0) if rng is None else rng
    x = np.asarray(x, float).ravel()
    x = (x - x.mean())/x.std()
```

```
    N = x.size
    eps, dlt, dsur = [], [], []
    for d in range(1, dmax + 1):
        n = N - d*tau
        Y = embed(x, d, tau, n)
        J, D = nnsearch(Y, w, k)
        if k > 1 and l > 0:                    # strand selection, eq. (CFrac)
            i = np.arange(n)
            S = np.zeros((n, k))
            for q in range(-l, l + 1):
                iq = np.clip(i + q, 0, n - 1)
                jq = np.clip(J + q, 0, n - 1)
                S += np.linalg.norm(Y[jq] - Y[iq][:, None, :], axis=2)
            m = np.argmin(S/np.maximum(D, 1e-300), axis=1)
            j, e = J[i, m], D[i, m]
        else:
            j, e = J[:, 0], D[:, 0]
        i = np.arange(n)
        eps.append(e)
        dlt.append(np.abs(x[i + d*tau] - x[j + d*tau]))
        dsur.append(np.abs(x[i + d*tau] - x[rng.integers(0, N, n)]))
    return eps, dlt, dsur


def fnnfrac(eps, dlt, r=10.0, s=0.5, a=2.0):
    """The fractions of Chapter `embdim` for thresholds r, s, a (sigma_x = 1).

    Returns a dict of arrays over d: 'orig' f_nn (fnnFrac), 'kennel' f_hat
    (OrKennel), 'tisean' f_tld (tisean), 'rel' f_bar (K_Rel) and 'comp'
    R_nn (new). The TISEAN fraction is zero where it has no admissible pair.
    """
    out = {q: np.zeros(len(eps)) for q in ("orig", "kennel", "tisean", "rel", "comp")}
    for m, (e, dl) in enumerate(zip(eps, dlt)):
        ratio = dl > r*e
        out["orig"][m] = ratio.mean()
        out["kennel"][m] = (ratio | (dl**2 + e**2 > a**2)).mean()
        adm = r*e < 1.0
        out["tisean"][m] = (ratio & adm).sum()/max(adm.sum(), 1)
        out["rel"][m] = (dl > s).mean()
        out["comp"][m] = (ratio | (dl > s)).mean()
    return out


def cao(x, tau, dmax, w=0):
    """Cao's E1(d) and E2(d), eqs. (eq:ed_caoE1)-(eq:ed_caoE2), d = 1..dmax,
    maximum norm, nearest neighbor outside the Theiler window w."""
    x = np.asarray(x, float).ravel()
    x = (x - x.mean())/x.std()
    N = x.size
    E, Es = np.zeros(dmax + 1), np.zeros(dmax + 1)
    for d in range(1, dmax + 2):
        n = N - d*tau
        Y = embed(x, d, tau, n)
        J, D = nnsearch(Y, w, 1, p=np.inf)
        j, e = J[:, 0], D[:, 0]
        i = np.arange(n)
        dl = np.abs(x[i + d*tau] - x[j + d*tau])
        ok = e > 0
        E[d - 1] = np.mean(np.maximum(e[ok], dl[ok])/e[ok])
        Es[d - 1] = dl.mean()
    return E[1:]/E[:-1], Es[1:]/Es[:-1]
```

A.9 Iteratively Refined Surrogates

MATLAB irsurr.m

```
function [rb, sb] = irsurr(s, n)
% IRSURR generates the iteratively refined surrogates of the original time
% series.
%
% USE: >> [rb, sb] = irsurr(s, n);
%
% INPUT:
%   x -- original scalar time series
%   n -- number of iterations to do
%
% OUTPUT:
%   rb -- surrogates that exactly match in histogram (PDF)
%   sb -- surrogates that exactly match in FFT magnitudes (PSD)
%
% Written by David Chelidze on 7/12/18 according to the paper:
%
%   Schreiber, T., Schmitz, A., 2000. Surrogate time series.
%   Physica D 142 (3-4), 346-382.

s = s(:); % make a column vector
N = size(s,1); % get the length of the time series

c = sort(s); % sort and store the magnitudes in ascending order
S = abs( fft(s) ); % store the FFT magnitudes

% initialize the iterations by phase randomizing the original signal
rb = real(ifft(fft(s).*exp(1i*2*pi*rand(N,1))));

for k = 1:n
    sb = real(ifft( S.*exp(1i*angle(fft(rb))) )); % first match the PSD
    [~, indx] = sort(sb);  % now match the PDF
    rb(indx) = c;          % by substituting the original magnitudes
end
```

Python irsurr.py

```
"""Iteratively refined surrogates.

Python port of irsurr.m (Appendix A.8), after Schreiber and Schmitz,
Surrogate time series, Physica D 142 (2000) 346-382.

Copyright by David Chelidze. Python port.
"""
import numpy as np


def irsurr(s, n, rng=None):
    """Return (rb, sb): surrogates matching the histogram, and the PSD.

    s : original scalar time series
    n : number of refinement iterations

    rb matches the amplitude distribution of s exactly; sb matches its
    Fourier magnitudes exactly. Neither matches both, which is the point
    of the method -- report which one you used.
    """
    rng = np.random.default_rng() if rng is None else rng
    s = np.asarray(s, float).ravel()
    N = s.size
    c = np.sort(s)                              # the original magnitudes
    S = np.abs(np.fft.fft(s))                   # the original magnitudes of the FFT
    # start from a phase randomization of the original signal
    rb = np.real(np.fft.ifft(np.fft.fft(s)*np.exp(1j*2*np.pi*rng.random(N))))
    sb = rb.copy()
```

```
    for _ in range(n):
        sb = np.real(np.fft.ifft(S*np.exp(1j*np.angle(np.fft.fft(rb)))))
        rb[np.argsort(sb)] = c                  # match the distribution
    return rb, sb
```

A.10 Recurrence Plots and Recurrence Quantification

Given delay vectors $\boldsymbol{X} \in \mathbb{R}^{M \times d}$ (Chapter 18), `rqa` builds the recurrence matrix at threshold ϵ with a Theiler window w, accumulates the diagonal and vertical line-length histograms, and returns RR, DET, L, $L_{\max}$, ENTR, LAM and TT in the definitions of Section 18.3. The Python module also supplies `eps_for_rr` (threshold from a target recurrence rate) and `k2_from_lines` (the correlation entropy K_2 from the cumulative diagonal-line distribution). Memory is $O(M^2)$; for records longer than a few thousand points, subsample or window.

MATLAB `rqa.m`

```
function m = rqa(X, eps, w, lmin, vmin)
% RQA  Recurrence matrix and recurrence quantification measures (Chapter 18).
%
%   m = rqa(X, eps, w, lmin, vmin)
%
%   X     (M x d) delay vectors, one per row (Euclidean distances)
%   eps   recurrence threshold (same units as X)
%   w     Theiler window: pairs with |i-j| <= w are excluded (default 0)
%   lmin  minimum diagonal line length counted (default 2)
%   vmin  minimum vertical line length counted (default 2)
%
%   m     struct with fields R (logical M x M), RR, DET, L, Lmax, ENTR,
%         LAM, TT, P (diagonal line histogram, P(l+1) = count of length l),
%         Pv (vertical line histogram), eps
%
%   The definitions follow Marwan et al. (2007): RR is the fraction of
%   admissible pairs that recur, DET the fraction of recurrent points on
%   diagonal lines of length >= lmin, L and Lmax the mean and maximum
%   diagonal line length, ENTR the Shannon entropy of the diagonal
%   line-length distribution, LAM and TT the vertical-line analogues.
%   To set eps from a target recurrence rate rr, take the rr-quantile of
%   the admissible pairwise distances:
%       D = squareform(pdist(X)); A = triu(true(size(D)), w+1);
%       eps = quantile(D(A), rr);
%   Memory is O(M^2).
if nargin < 3 || isempty(w),    w = 0;    end
if nargin < 4 || isempty(lmin), lmin = 2; end
if nargin < 5 || isempty(vmin), vmin = 2; end
X = double(X); if size(X,1) < size(X,2), X = X.'; end
M = size(X,1);
D = sqrt(max(sum(X.^2,2) + sum(X.^2,2).' - 2*(X*X.'), 0));
R = D < eps;
[I, J] = ndgrid(1:M, 1:M);
R(abs(I - J) <= w) = false;
% diagonal line histogram (upper triangle, doubled by symmetry)
P = zeros(M+1, 1);
for k = w+1:M-1
    L = runs(diag(R, k));
    for q = 1:numel(L), P(L(q)+1) = P(L(q)+1) + 1; end
end
P = 2*P;
Pv = zeros(M+1, 1);
for j = 1:M
    L = runs(R(:, j));
    for q = 1:numel(L), Pv(L(q)+1) = Pv(L(q)+1) + 1; end
end
ell = (0:M).';
```

```
nadm = (M-w)*(M-w-1);
m.R = R; m.eps = eps;
m.RR = nnz(R)/nadm;
tot = sum(ell.*P);
sel = ell >= lmin;
m.DET = sum(ell(sel).*P(sel))/tot;
nl = sum(P(sel));
m.L = sum(ell(sel).*P(sel))/nl;
m.Lmax = max(ell(P > 0));
p = P(sel)/nl; p = p(p > 0);
m.ENTR = -sum(p.*log(p));
totv = sum(ell.*Pv); selv = ell >= vmin;
m.LAM = sum(ell(selv).*Pv(selv))/totv;
m.TT = sum(ell(selv).*Pv(selv))/sum(Pv(selv));
m.P = P; m.Pv = Pv;
end

function L = runs(v)
% lengths of runs of true in a logical vector
v = [false; v(:); false];
d = diff(double(v));
L = find(d == -1) - find(d == 1);
end
```

Python `rqa.py`

```
"""Recurrence plot and RQA measures (Chapter 18). NumPy only; O(M^2) memory.

    R = recmat(X, eps, w)                recurrence matrix with Theiler window w
    eps = eps_for_rr(X, rr, w)           threshold giving recurrence rate rr
    P, Pv = line_hist(R, w)              diagonal / vertical line-length histograms
    m = rqa(R, w, lmin=2, vmin=2)        dict RR, DET, L, Lmax, ENTR, LAM, TT
    K2, Pc, ells = k2_from_lines(P, dt, lmin, lmax)  correlation entropy from P(l)

X is (M, d): one delay vector per row. Distances are Euclidean.
"""
import numpy as np


def _dist(X):
    X = np.atleast_2d(np.asarray(X, float))
    if X.shape[0] < X.shape[1]:
        X = X.T
    sq = np.sum(X*X, axis=1)
    D2 = sq[:, None] + sq[None, :] - 2*X@X.T
    return np.sqrt(np.maximum(D2, 0.0))


def eps_for_rr(X, rr, w=0):
    """Threshold eps such that the fraction rr of admissible pairs (|i-j|>w) recur."""
    D = _dist(X); M = D.shape[0]
    i, j = np.triu_indices(M, k=w+1)
    return np.quantile(D[i, j], rr)


def recmat(X, eps, w=0):
    D = _dist(X); M = D.shape[0]
    R = D < eps
    idx = np.arange(M)
    R[np.abs(idx[:, None]-idx[None, :]) <= w] = False
    return R


def _runs(v):
    """Lengths of runs of True in a boolean vector."""
    v = np.concatenate(([False], v, [False])).astype(int)
    d = np.diff(v)
```

```
    return np.flatnonzero(d == -1) - np.flatnonzero(d == 1)


def line_hist(R, w=0):
    M = R.shape[0]
    P = np.zeros(M+1, int); Pv = np.zeros(M+1, int)
    for k in range(w+1, M):                       # upper triangle diagonals
        for L in _runs(np.diagonal(R, k)):
            P[L] += 1
    P *= 2                                         # symmetric: count both triangles
    for j in range(M):
        for L in _runs(R[:, j]):
            Pv[L] += 1
    return P, Pv


def rqa(R, w=0, lmin=2, vmin=2):
    M = R.shape[0]
    P, Pv = line_hist(R, w)
    ell = np.arange(M+1)
    nadm = (M-w)*(M-w-1)                           # admissible ordered pairs
    RR = R.sum()/nadm
    tot = np.sum(ell*P)
    DET = np.sum(ell[lmin:]*P[lmin:])/tot if tot else np.nan
    nl = P[lmin:].sum()
    L = np.sum(ell[lmin:]*P[lmin:])/nl if nl else np.nan
    Lmax = ell[P > 0].max() if P.any() else 0
    p = P[lmin:]/nl if nl else P[lmin:]
    ENTR = -np.sum(p[p > 0]*np.log(p[p > 0])) if nl else np.nan
    totv = np.sum(ell*Pv)
    LAM = np.sum(ell[vmin:]*Pv[vmin:])/totv if totv else np.nan
    nv = Pv[vmin:].sum()
    TT = np.sum(ell[vmin:]*Pv[vmin:])/nv if nv else np.nan
    return dict(RR=RR, DET=DET, L=L, Lmax=Lmax, ENTR=ENTR, LAM=LAM, TT=TT, P=P, Pv=Pv)


def k2_from_lines(P, dt, lmin, lmax):
    """Correlation entropy from the cumulative diagonal-line distribution
    P^c(l) = sum_{l'>=l} P(l') ~ exp(-K2 l dt) (Faure & Korn 1998). Least-squares
    slope of ln P^c over lmin <= l <= lmax; returns K2 (per unit time), P^c, l."""
    ell = np.arange(len(P))
    Pc = np.cumsum(P[::-1])[::-1].astype(float)
    sel = (ell >= lmin) & (ell <= lmax) & (Pc > 0)
    slope = np.polyfit(ell[sel]*dt, np.log(Pc[sel]), 1)[0]
    return -slope, Pc, ell
```

A.11 Python Support Modules

Two of the Python modules have no MATLAB counterpart, because they supply what MATLAB provides as built-ins.

Generalized singular value decomposition

SciPy has no `gsvd`, so the smooth orthogonal decomposition of Section A.5 needs one built from parts. This module forms it the standard way: a QR factorization of the stacked pair followed by a cosine–sine decomposition. The sines are taken as the column norms of $\boldsymbol{Q}_2\boldsymbol{Z}$ rather than as $\sqrt{1-c^2}$, so they stay accurate where c approaches one, which is where the smooth modes live. Note that the generalized singular values are returned in *descending* order, smoothest first, where MATLAB returns them ascending.

Python gsvd.py

```python
"""Generalized singular value decomposition, for the SOD-based routines.

SciPy has no gsvd, so it is built here the standard way -- a QR of the
stacked pair followed by a cosine-sine decomposition (Van Loan 1976,
Paige and Saunders 1981).

Given A (p x n) and B (q x n) with [A; B] of full column rank,

    M = [A; B] = Q R,        Q = [Q1; Q2] with orthonormal columns,
    Q1 = U C Z',  Q2 = V S Z',   C'C + S'S = I,

so with X' = Z' R,

    A = U C X',   B = V S X',

which is MATLAB's convention for [U,V,X,C,S] = gsvd(A,B,0).

The CS decomposition is taken from one SVD of Q1; the sines are then the
column norms of Q2 Z, computed directly rather than as sqrt(1 - c^2), so
they stay accurate where c is close to one -- which is exactly the
regime the smooth modes live in.

One deliberate difference from MATLAB: the pairs are returned with the
generalized singular values c/s in DESCENDING order, so column 0 is the
smoothest mode. MATLAB returns them ascending, which is why sodfilt.m
has to fliplr its factors and this module's callers do not.

Copyright by David Chelidze. Python port.
"""
import numpy as np


def gsvd(A, B):
    """Return U, V, X, c, s with A = U diag(c) X.T and B = V diag(s) X.T.

    A : (p, n) array
    B : (q, n) array
    Both must have n columns and [A; B] must have rank n.
    c/s comes back in descending order; c**2 + s**2 == 1.
    """
    A = np.atleast_2d(np.asarray(A, float))
    B = np.atleast_2d(np.asarray(B, float))
    p, n = A.shape
    q, nb = B.shape
    if nb != n:
        raise ValueError("A and B must have the same number of columns")
    M = np.vstack([A, B])
    if np.linalg.matrix_rank(M) < n:
        raise np.linalg.LinAlgError(
            "[A; B] is column rank deficient, so the generalized singular "
            "values are not defined. Lengthen the strand, lower the "
            "embedding dimension, or regularize the data.")
    Q, R = np.linalg.qr(M)                      # economy: Q is (p+q, n)
    Q1, Q2 = Q[:p, :], Q[p:, :]
    U, c, Zt = np.linalg.svd(Q1, full_matrices=False)   # c descending
    Z = Zt.T
    T = Q2 @ Z                                  # columns are orthogonal
    s = np.linalg.norm(T, axis=0)               # accurate for small s
    V = np.zeros_like(T)
    nz = s > 0
    V[:, nz] = T[:, nz]/s[nz]
    X = R.T @ Z                                 # A = U diag(c) X.T
    return U, V, X, c, s


def sod(S, Sdot=None):
```

```
    """Smooth orthogonal decomposition of a strand.

    S    : (d, N) strand, one column per point. The mean is NOT removed
           here -- remove it before calling, as the MATLAB code does.
    Sdot : (d, N-1) or (d, N) data-based derivative. Defaults to the
           first difference along the strand, which is what sodfilt uses.

    Returns Phi (SOMs, columns), Psi (SPMs, columns), lam (SOVs), and Q
    (SOCs, columns), all ordered smoothest first, so that

        S ~= Phi[:, :k] @ Psi[:, :k].T @ S

    is the k-dimensional smoothest approximation.
    """
    S = np.atleast_2d(np.asarray(S, float))
    if Sdot is None:
        Sdot = np.diff(S, axis=1)
    U, V, X, c, s = gsvd(S.T, np.asarray(Sdot, float).T)
    with np.errstate(divide="ignore", invalid="ignore"):
        lam = np.where(s > 0, (c/np.where(s > 0, s, 1.0))**2, np.inf)
    Phi = X                                     # SOMs
    Psi = np.linalg.inv(X.T)                    # SPMs
    Qc = U*c                                    # SOCs
    return Phi, Psi, lam, Qc


def smooth_project(S, k, Sdot=None):
    """The k-dimensional smoothest approximation to the strand S."""
    Phi, Psi, _, _ = sod(S, Sdot)
    return Phi[:, :k] @ (Psi[:, :k].T @ S)
```

Shared reconstruction utilities

The delay coordinate embedding, the edge padding, the strand weighting and the final shift-and-average are common to both noise reduction routines and are kept in one place so the two cannot drift apart.

Python `nlutil.py`

```
"""nlutil -- shared pieces of the projective noise reduction routines.

Delay coordinate embedding, the edge padding both routines use, the
strand weighting, and the final shift-and-average. Kept in one place so
podfilt and sodfilt cannot drift apart, and so the padding convention is
stated once.

Copyright by David Chelidze. Python port.
"""
import numpy as np
from scipy.spatial import cKDTree


def embed(x, d, m):
    """Delay coordinate embedding: y[i] = x[i : i+n] shifted by i*m."""
    x = np.asarray(x, float).ravel()
    n = x.size - m*(d - 1)
    return np.array([x[np.arange(n) + i*m] for i in range(d)]), n


def pad(y, n, npd):
    """Extend both ends by following the trajectory of a nearest neighbor
    of the first and last point, as podfilt.m and sodfilt.m do."""
    tree = cKDTree(y.T)
    _, i0 = tree.query(y[:, 0], k=npd)
    cand = np.atleast_1d(i0)
```

```
    cand = cand[cand > npd]
    if cand.size == 0:
        raise ValueError("no suitable left-hand neighbor for padding; "
                         "the record is too short for these parameters")
    ya = y[:, cand[0] - npd:cand[0]]
    _, i1 = tree.query(y[:, -1], k=npd)
    cand = np.atleast_1d(i1)
    cand = cand[cand < n - npd]
    if cand.size == 0:
        raise ValueError("no suitable right-hand neighbor for padding; "
                         "the record is too short for these parameters")
    yb = y[:, cand[0] + 1:cand[0] + 1 + npd]
    return np.hstack([ya, y, yb]), tree


def strand_weights(l, w):
    """The three weightings of sodfilt.m, as a row vector.
    0 comb (no averaging along the strand), 1 tent, 2 exponential.
    They need not be normalized: the caller divides by the weight each
    point accumulates."""
    if w == 1:
        wd = np.concatenate([np.arange(0, l + 1), np.arange(l - 1, -1, -1)])
    elif w == 2:
        wd = np.exp(-np.concatenate([np.arange(l, -1, -1),
                                     np.arange(1, l + 1)]))
    else:
        wd = np.concatenate([np.zeros(l), [1.0], np.zeros(l)])
    return np.asarray(wd, float)


def unshift(Yh, d, m, n, nx):
    """Average the d estimates of each sample. The offset is (d-k)*tau:
    each point sits one coordinate further back for each step down in k."""
    xf = np.array([Yh[k, m*(d - 1 - k):Yh.shape[1] - m*k]
                   for k in range(d)])
    return xf.mean(axis=0)
```

A.12 Welch Spectra, Coherence and Frequency Response

The estimators of Chapter 6 in one routine. Called with one record it returns the one-sided Welch auto-spectrum $\hat{G}_{xx}(f)$, Eq. (6.3), in units of x^2/Hz, so that $\sum_k \hat{G}_{xx}(f_k)\,\Delta f$ recovers the variance of the record. Called with two records it also returns the one-sided cross-spectrum $\hat{G}_{xy}$ with the conjugate on the first record (Eq. (6.32)), the coherence, and the H_1 and H_2 estimators of Eqs. (6.39)–(6.40) with x as the input, together with the number of segments K from Eq. (6.12) so that the random error can be quoted. Segments have their means removed individually; the window is normalized by $\sum_n w_n^2$ as in Eq. (6.6), so the estimate of a white spectrum is unbiased for any taper. The Python version was checked against `scipy.signal.welch`, `csd` and `coherence` for the rectangular, Hann and Hamming windows and agrees to 10^{-13} at every bin but the first, which both leave at round-off after detrending.

MATLAB `psdwelch.m`

```
function [f, S] = psdwelch(x, fs, L, D, win, y)
% PSDWELCH Welch spectral estimates from one or two records (Chapter 6).
%
% USE: >> [f, Gxx] = psdwelch(x, fs, L, D, win);      % one-sided auto-spectrum
%      >> [f, S]   = psdwelch(x, fs, L, D, win, y);   % struct: Gxx Gyy Gxy coh H1 H2
%
% INPUT:
%   x, y -- records (vectors); x is the input when an FRF is wanted
%   fs   -- sampling frequency (Hz)
```

```
%   L    -- segment length (samples);  D -- overlap (samples), default L/2
%   win  -- window: a vector of length L, or a name ('hann', 'rectwin', ...)
%
% OUTPUT: one-sided densities on f >= 0, units x^2/Hz, so sum(Gxx)*f(2) ~ var(x).
%   Gxy = one-sided cross-spectrum, coh = |Gxy|^2/(Gxx Gyy),
%   H1 = Gxy./Gxx,  H2 = Gyy./conj(Gxy).
%
% Written by David Chelidze.

if nargin < 4 || isempty(D), D = floor(L/2); end
if nargin < 5 || isempty(win), win = 'hann'; end
if ischar(win) || isstring(win), w = window(str2func(char(win)), L, 'periodic'); ...
else, w = win(:); end
U = sum(w.^2);                          % window power: E[G] = S for white noise
nf = floor(L/2) + 1;
f = (0:nf-1)'*fs/L;
scale = 2/(fs*U)*ones(nf, 1); scale(1) = scale(1)/2;
if mod(L, 2) == 0, scale(end) = scale(end)/2; end   % Nyquist bin not doubled
X = segfft(x, L, D, w, nf);
Gxx = scale.*mean(abs(X).^2, 2);
if nargin < 6, S = Gxx; return; end
Y = segfft(y, L, D, w, nf);
S.Gxx = Gxx;
S.Gyy = scale.*mean(abs(Y).^2, 2);
S.Gxy = scale.*mean(conj(X).*Y, 2);
S.coh = abs(S.Gxy).^2./(S.Gxx.*S.Gyy);
S.H1 = S.Gxy./S.Gxx;
S.H2 = S.Gyy./conj(S.Gxy);
S.K = size(X, 2);
end

function X = segfft(x, L, D, w, nf)
% overlapped, mean-removed, tapered segments in columns, transformed
x = x(:); K = floor((numel(x) - L)/(L - D)) + 1;
idx = (1:L)' + (L - D)*(0:K-1);
seg = x(idx); seg = seg - mean(seg, 1);
X = fft(seg.*w, [], 1); X = X(1:nf, :);
end
```

Python psdwelch.py

```python
"""Welch spectral estimates from one or two records (Chapter 6).

    f, G = psdwelch(x, fs, L, D, win)            # one-sided auto-spectrum
    f, S = psdwelch(x, fs, L, D, win, y)         # dict: Gxx, Gyy, Gxy, coh, H1, H2

Segments of L samples, overlapped by D samples, are tapered by win
(a string accepted by scipy.signal.get_window, or an array of length L),
transformed, and their scaled periodograms averaged.  Densities are
one-sided (f >= 0) in units of x^2/Hz, so that sum(G)*f[1] ~ var(x).
Gxy is the one-sided cross-spectrum, coh the ordinary coherence,
H1 = Gxy/Gxx and H2 = Gyy/Gyx the two FRF estimators (x = input).

Copyright by David Chelidze.
"""
import numpy as np
from scipy.signal import get_window


def _segments(x, L, D):
    """Return the K x L matrix of overlapped segments (mean removed)."""
    x = np.asarray(x, float).ravel()
    K = (x.size - L)//(L - D) + 1
    idx = np.arange(L)[None, :] + (L - D)*np.arange(K)[:, None]
    seg = x[idx]
    return seg - seg.mean(axis=1, keepdims=True)


def psdwelch(x, fs, L, D=None, win="hann", y=None):
    D = L//2 if D is None else D
    w = get_window(win, L, fftbins=True) if isinstance(win, str) else np.asarray(win, float)
    U = np.sum(w**2)                        # window power; makes E[G] = S for white noise
    nf = L//2 + 1
    f = np.arange(nf)*fs/L
    scale = np.ones(nf)*2/(fs*U)            # one-sided: double the interior bins
    scale[0] /= 2
    if L % 2 == 0:
        scale[-1] /= 2                      # Nyquist bin is not doubled either
    X = np.fft.rfft(_segments(x, L, D)*w, axis=1)
    Gxx = scale*np.mean(np.abs(X)**2, axis=0)
    if y is None:
        return f, Gxx
    Y = np.fft.rfft(_segments(y, L, D)*w, axis=1)
    Gyy = scale*np.mean(np.abs(Y)**2, axis=0)
    Gxy = scale*np.mean(np.conj(X)*Y, axis=0)
    out = dict(Gxx=Gxx, Gyy=Gyy, Gxy=Gxy,
               coh=np.abs(Gxy)**2/(Gxx*Gyy),
               H1=Gxy/Gxx, H2=Gyy/np.conj(Gxy), K=X.shape[0])
    return f, out
```

A.13 Multitaper Spectra and the Harmonic F-Test

The estimators of Section 6.3. `psdmt` returns the one-sided multitaper density, by Thomson's adaptive weights, Eq. (6.20), or by the plain average, Eq. (6.19); with a third output it also returns the harmonic F statistic and the complex line amplitude of Eq. (6.22) and the equivalent degrees of freedom of Eq. (6.21). `mtlines` finds the lines by the F-test on an eightfold zero-padded grid and subtracts them, so that `psdmt` of its output estimates the continuum alone. The MATLAB version needs `dpss` (Signal Processing Toolbox) and `finv` (Statistics and Machine Learning Toolbox). Run in MATLAB R2026a with its own `dpss` and `finv`, `chk_psdmt.m` reproduces the Python version to 10^{-10} or better in the density, the F statistic, the line amplitudes and the degrees of freedom, and

the taper concentrations to the printed digits. Both versions were checked on white noise (mean, $\nu = 2K$, Parseval, and the $F_{2,2K-2}$ null distribution), by `chk_psdmt.m` and `chk_multitaper.py`.

MATLAB `psdmt.m`

```
function [f, G, out] = psdmt(x, fs, NW, K, adaptive, nfft)
% PSDMT Multitaper spectral estimate and harmonic F-test (Chapter 6).
%
% USE: >> [f, Gxx] = psdmt(x, fs);                      % NW = 4, K = 7, adaptive
%      >> [f, Gxx, S] = psdmt(x, fs, NW, K, adaptive, nfft);
%
% INPUT:
%   x        -- record (vector), sampled at fs (Hz); its mean is removed
%   NW       -- time-bandwidth product, default 4; half-bandwidth W = NW*fs/N
%   K        -- number of Slepian tapers, default 2*NW - 1
%   adaptive -- true (default): Thomson's adaptive weights; false: plain average
%   nfft     -- transform length >= N (zero padding), default N
%
% OUTPUT: one-sided density on f >= 0 in x^2/Hz, so sum(Gxx)*f(2) ~ var(x).
%   S.F   -- harmonic F statistic, F(2, 2K-2) under the no-line hypothesis
%   S.mu  -- complex line amplitude: a line A*cos(2*pi*f0*t + phi) has
%            |mu(f0)| = A/2, with phase at t = 0
%   S.nu  -- equivalent degrees of freedom of the estimate at each frequency
%   S.Gavg, S.lam, S.K -- plain-average estimate, taper concentrations, K
%
% Needs dpss (Signal Processing Toolbox).
% Written by David Chelidze.

x = x(:) - mean(x); N = numel(x);
if nargin < 3 || isempty(NW), NW = 4; end
if nargin < 4 || isempty(K), K = floor(2*NW) - 1; end
if nargin < 5 || isempty(adaptive), adaptive = true; end
if nargin < 6 || isempty(nfft), nfft = N; end
nfft = max(nfft, N);
[v, lam] = dpss(N, NW, K);              % N x K tapers, unit energy; lam = concentrations
lam = lam(:).';                         % 1 x K
dt = 1/fs; nf = floor(nfft/2) + 1;
f = (0:nf-1).'*fs/nfft;
Y = fft(v.*x, nfft, 1); Y = Y(1:nf, :);  % eigencoefficients, nf x K
Sk = dt*abs(Y).^2;                      % eigenspectra (two-sided densities)
Savg = mean(Sk, 2);
if adaptive
    sig2 = var(x, 1)*dt;                % white level with the record's variance
    Sa = mean(Sk(:, 1:2), 2);           % start from the two best-concentrated tapers
    for it = 1:100
        b = Sa./(Sa*lam + sig2*(1 - lam));      % nf x K
        d2 = lam.*b.^2;                         % squared weights d_k^2
        Snew = sum(d2.*Sk, 2)./sum(d2, 2);
        done = max(abs(Snew - Sa)./Sa) < 1e-6;
        Sa = Snew;
        if done, break; end
    end
    nu = 2*sum(d2, 2).^2./sum(d2.^2, 2);
else
    Sa = Savg; nu = 2*K*ones(nf, 1);
end
one = 2*ones(nf, 1); one(1) = 1;
if mod(nfft, 2) == 0, one(end) = 1; end  % Nyquist bin is not doubled
G = one.*Sa;
if nargout < 3, return; end
V0 = sum(v, 1);                         % taper sums; zero for the odd tapers
mu = (Y*V0.')/sum(V0.^2);               % least-squares line amplitude
resid = sum(abs(Y - mu*V0).^2, 2);
out.F = (K - 1)*abs(mu).^2*sum(V0.^2)./resid;
out.mu = mu; out.nu = nu; out.Gavg = one.*Savg; out.lam = lam; out.K = K;
end
```

MATLAB mtlines.m

```
function [xc, L] = mtlines(x, fs, NW, K, p, pad)
% MTLINES Detect lines by the harmonic F-test and remove them (Chapter 6).
%
% USE: >> [xc, L] = mtlines(x, fs);                % NW = 4, K = 7, p = 1/N
%      >> [f, Gc] = psdmt(xc, fs);                 % continuous part
%
% A line is a local maximum of the F statistic above the 1 - p point of
% F(2, 2K-2), on a grid zero-padded by pad (default 8). Each detected
% sinusoid is subtracted from the record. With p = 1/N and pad = 8 the
% measured false-line rate for white noise (N = 1024) is 1.8 per record.
%
% OUTPUT: xc -- record with the lines removed (mean removed)
%         L  -- one row per line: [f0 (Hz), amplitude A, phase (rad), F]
%
% Needs finv (Statistics and Machine Learning Toolbox).
% Written by David Chelidze.

x = x(:) - mean(x); N = numel(x);
if nargin < 3 || isempty(NW), NW = 4; end
if nargin < 4 || isempty(K), K = floor(2*NW) - 1; end
if nargin < 5 || isempty(p), p = 1/N; end
if nargin < 6 || isempty(pad), pad = 8; end
[f, ~, S] = psdmt(x, fs, NW, K, false, pad*N);
F = S.F;
thr = finv(1 - p, 2, 2*K - 2);
pk = F > thr & F >= [0; F(1:end-1)] & F >= [F(2:end); 0];
pk([1 end]) = false;                    % no lines at 0 or Nyquist
idx = find(pk); t = (0:N-1).'/fs;
xc = x; L = zeros(numel(idx), 4);
for i = 1:numel(idx)
    j = idx(i); mu = S.mu(j);
    xc = xc - 2*real(mu*exp(2i*pi*f(j)*t));
    L(i, :) = [f(j), 2*abs(mu), angle(mu), F(j)];
end
end
```

Python psdmt.py

```
"""Multitaper spectral estimate and harmonic F-test (Chapter 6).

    f, G = psdmt(x, fs, NW)                     # one-sided auto-spectrum
    f, S = psdmt(x, fs, NW, K, adaptive, nfft)  # dict: G, Gavg, F, mu, nu, lam, K

The whole record is tapered by K discrete prolate spheroidal sequences
(Slepian tapers) of time-bandwidth product NW, half-bandwidth W = NW/(N dt).
Each taper gives an eigenspectrum; they are combined either by plain
averaging (adaptive=False) or by Thomson's adaptive weights (default).
Densities are one-sided (f >= 0) in x^2/Hz, so sum(G)*f[1] ~ var(x).

F is Thomson's harmonic F statistic with 2 and 2K-2 degrees of freedom,
mu the complex line amplitude estimate at each frequency (a line
A cos(2 pi f0 t + phi) has |mu(f0)| = A/2), and nu the equivalent degrees
of freedom of the adaptive estimate at each frequency.

Copyright by David Chelidze.
"""
import numpy as np
from scipy.signal.windows import dpss


def psdmt(x, fs, NW=4.0, K=None, adaptive=True, nfft=None, full=False):
    x = np.asarray(x, float).ravel()
    x = x - x.mean()
    N = x.size
    K = int(2*NW) - 1 if K is None else K
    nfft = N if nfft is None else max(int(nfft), N)
```

```
    v, lam = dpss(N, NW, Kmax=K, return_ratios=True)    # K x N, unit energy
    nf = nfft//2 + 1
    f = np.arange(nf)*fs/nfft
    dt = 1.0/fs
    y = np.fft.rfft(v*x[None, :], n=nfft, axis=1)        # eigencoefficients, K x nf
    Sk = dt*np.abs(y)**2                                 # eigenspectra (two-sided)
    Savg = Sk.mean(axis=0)
    if adaptive:
        sig2 = x.var()*dt                                # white level, two-sided
        S = Sk[:2].mean(axis=0)
        for _ in range(100):
            b = S[None, :]/(lam[:, None]*S[None, :] + (1 - lam[:, None])*sig2)
            d2 = lam[:, None]*b**2                       # d_k^2
            Snew = np.sum(d2*Sk, axis=0)/np.sum(d2, axis=0)
            if np.max(np.abs(Snew - S)/S) < 1e-6:
                S = Snew
                break
            S = Snew
        nu = 2*np.sum(d2, axis=0)**2/np.sum(d2**2, axis=0)
    else:
        S = Savg
        nu = 2*K*np.ones(nf)
    one = np.ones(nf)*2                                  # one-sided: double interior
    one[0] = 1
    if nfft % 2 == 0:
        one[-1] = 1
    G = one*S
    if not full:
        return f, G
    # harmonic F-test (Thomson 1982): only even tapers have V_k(0) != 0
    V0 = v.sum(axis=1)                                   # K
    mu = (V0[:, None]*y).sum(axis=0)/np.sum(V0**2)       # complex, units of x
    resid = np.sum(np.abs(y - mu[None, :]*V0[:, None])**2, axis=0)
    F = (K - 1)*np.abs(mu)**2*np.sum(V0**2)/resid
    return f, dict(G=G, Gavg=one*Savg, F=F, mu=mu, nu=nu, lam=lam, K=K)


def mtlines(x, fs, NW=4.0, K=None, p=None, pad=8):
    """Detect lines by the harmonic F-test and remove them (Thomson 1982).

    Returns xc, the record with every detected sinusoid subtracted, and an
    array of rows (f0, A, phase, F) for the lines found.  A line is a local
    maximum of F above the 1 - p point of F(2, 2K-2) on a grid zero-padded
    by `pad`; p defaults to 1/N.  With pad = 8 the measured false-line rate
    for white noise (N = 1024) is 1.8 per record, not N p/2 = 0.5, because the
    padded grid searches the maxima of F between the Fourier frequencies.
    The continuum is then estimated by psdmt(xc, ...).
    """
    from scipy.stats import f as fdist
    x = np.asarray(x, float).ravel()
    x = x - x.mean()
    N = x.size
    K = int(2*NW) - 1 if K is None else K
    p = 1.0/N if p is None else p
    f, S = psdmt(x, fs, NW, K, adaptive=False, nfft=pad*N, full=True)
    F = S["F"]
    thr = fdist.ppf(1 - p, 2, 2*K - 2)
    pk = (F > thr) & (F >= np.r_[0, F[:-1]]) & (F >= np.r_[F[1:], 0])
    pk[0] = pk[-1] = False                       # no lines at 0 or Nyquist
    t = np.arange(N)/fs
    xc = x.copy()
    rows = []
    for j in np.flatnonzero(pk):
        mu = S["mu"][j]
        xc -= 2*np.real(mu*np.exp(2j*np.pi*f[j]*t))
        rows.append((f[j], 2*np.abs(mu), np.angle(mu), F[j]))
    return xc, np.array(rows).reshape(-1, 4)
```

Bibliography

[1] H. D. I. Abarbanel, R. Brown, and M. B. Kennel. Variation of Lyapunov exponents on a strange attractor. *Journal of Nonlinear Science*, 1(2):175–199, 1991.

[2] H. D. I. Abarbanel, R. Brown, and M. B. Kennel. Local Lyapunov exponents computed from observed data. *Journal of Nonlinear Science*, 2(3):343–365, 1992.

[3] H. D. I. Abarbanel, R. Brown, J. J. Sidorowich, and L. Sh. Tsimring. The analysis of observed chaotic data in physical systems. *Reviews of Modern Physics*, 65(4):1331–1392, 1993.

[4] Henry Abarbanel. *Analysis of observed chaotic data*. Springer, New York, 1996.

[5] H. Akaike. A new look at the statistical model identification. *IEEE Transactions on Automatic Control*, 19(6):716–723, 1974.

[6] Christoph Bandt and Bernd Pompe. Permutation entropy: A natural complexity measure for time series. *Physical Review Letters*, 88(17):174102, 2002.

[7] M. S. Bartlett. Smoothing periodograms from time-series with continuous spectra. *Nature*, 161(4096):686–687, 1948.

[8] Julius S. Bendat and Allan G. Piersol. *Random Data: Analysis and Measurement Procedures*. John Wiley & Sons, Hoboken, NJ, 4 edition, 2010.

[9] G. Benettin, L. Galgani, A. Giorgilli, and J.-M. Strelcyn. Lyapunov characteristic exponents for smooth dynamical systems and for Hamiltonian systems; a method for computing all of them. part 1: Theory. *Meccanica*, 15(1):9–20, 1980.

[10] G. Benettin, L. Galgani, A. Giorgilli, and J.-M. Strelcyn. Lyapunov characteristic exponents for smooth dynamical systems and for Hamiltonian systems; a method for computing all of them. part 2: Numerical application. *Meccanica*, 15(1):21–30, 1980.

[11] George D. Birkhoff. Proof of the ergodic theorem. *Proceedings of the National Academy of Sciences*, 17(12):656–660, 1931.

[12] George EP Box, Gwilym M Jenkins, Gregory C Reinsel, and Greta M Ljung. *Time series analysis: forecasting and control*. John Wiley & Sons, 2015.

[13] J. Brocker, U. Parlitz, and M. Ogorzalek. Nonlinear noise reduction. *Proceedings of the IEEE*, 90(5):898–918, 2002.

[14] David S Broomhead and Gregory P King. Extracting qualitative dynamics from experimental data. *Physica D: Nonlinear Phenomena*, 20(2):217–236, 1986.

[15] Reggie Brown, Paul Bryant, and Henry D. I. Abarbanel. Computing the Lyapunov spectrum of a dynamical system from an observed time series. *Physical Review A*, 43(6):2787–2806, 1991.

[16] Steven L. Brunton, Joshua L. Proctor, and J. Nathan Kutz. Discovering governing equations from data by sparse identification of nonlinear dynamical systems. *Proceedings of the National Academy of Sciences*, 113(15):3932–3937, 2016.

[17] Paul Bryant, Reggie Brown, and Henry D. I. Abarbanel. Lyapunov exponents from observed time series. *Physical Review Letters*, 65(13):1523–1526, 1990.

[18] Liangyue Cao. Practical method for determining the minimum embedding dimension of a scalar time series. *Physica D: Nonlinear Phenomena*, 110(1):43–50, 1997.

[19] Liangyue Cao, Alistair Mees, and Kevin Judd. Dynamics from multivariate time series. *Physica D: Nonlinear Phenomena*, 121(1):75–88, 1998.

[20] James R Carey. Demography and population dynamics of the mediterranean fruit fly. *Ecological Modelling*, 16(2-4):125–150, 1982.

[21] Martin Casdagli. Nonlinear prediction of chaotic time series. *Physica D: Nonlinear Phenomena*, 35(3):335–356, 1989.

[22] Martin Casdagli. Chaos and deterministic versus stochastic non-linear modelling. *Journal of the Royal Statistical Society: Series B (Methodological)*, 54(2):303–328, 1992.

[23] Martin Casdagli, Stephen Eubank, J Doyne Farmer, and John Gibson. State space reconstruction in the presence of noise. *Physica D: Nonlinear Phenomena*, 51(1):52–98, 1991.

[24] M. Cencini, M. Falcioni, E. Olbrich, H. Kantz, and A. Vulpiani. Chaos or noise: Difficulties of a distinction. *Physical Review E*, 62(1):427–437, 2000.

[25] Mattia Cenedese, Joar Axås, Bastian Bäuerlein, Kerstin Avila, and George Haller. Data-driven modeling and prediction of non-linearizable dynamics via spectral submanifolds. *Nature Communications*, 13:872, 2022.

[26] A. Chatterjee, J. P. Cusumano, and D. Chelidze. Optimal tracking of parameter drift in a chaotic system: Experiment and theory. *Journal of Sound and Vibration*, 250(5):877–901, 2002.

[27] Anindya Chatterjee. An introduction to the proper orthogonal decomposition. *Current Science: Computational Science*, 78(7):808–817, 2000.

[28] Eleni N. Chatzi and Andrew W. Smyth. The unscented Kalman filter and particle filter methods for nonlinear structural system identification with non-collocated heterogeneous sensing. *Structural Control and Health Monitoring*, 16(1):99–123, 2009.

[29] C Chatzinakos and C Tsouros. Estimation of the dimension of chaotic dynamical systems using neural networks and robust location estimate. *Simulation Modelling Practice and Theory*, 51:149–156, 2015.

[30] D. Chelidze and M. Liu. Reconstructing slow-time dynamics from fast-time measurements. *Philosophical Transaction of the Royal Society A*, 366:729–3087, 2008.

[31] D. Chelidze and W. Zhou. Smooth orthogonal decomposition-based vibration mode identification. *Journal of Sound and Vibration*, 292(3):461–473, 2006.

[32] David Chelidze. Smooth local subspace projection for nonlinear noise reduction. *Chaos*, 24(1):013121, 2014.

[33] David Chelidze and Joseph P. Cusumano. Phase space warping: Nonlinear time series analysis for slowly drifting systems. *Philosophical Transactions of the Royal Society A*, 364, 2006.

[34] David Chelidze, Joseph P. Cusumano, and Anindya Chatterjee. Dynamical Systems Approach to Damage Evalution Tracking, Part I: Desccription and Experimental Application. *Journal of Vibration and Acoustics*, 124(2):250–257, 2002.

[35] David Chelidze and Ming Liu. Dynamical systems approach to fatigue damage identification. *Journal of Sound and Vibration*, 281:887–904, 2004.

[36] David Chelidze and Ming Liu. Multidimensional damage identification based on phase space warping: An experimental study. *Nonlinear Dynamics*, 46(1–2):61–72, 2006.

[37] David Chelidze and Ming Liu. Reconstructing slow-time dynamics from fast-time measurements. *Philosophical Transactions of the Royal Society A*, 366(1866):729–745, 2008.

[38] L. O. Chua, M. Komuro, and T. Matsumoto. The double scroll family. *IEEE Transactions on Circuits and Systems*, 33(11):1072–1118, 1986.

[39] Antoine Clement and Stephane Laurens. An alternative to the Lyapunov exponent as a damage sensitive feature. *Smart Materials and Structures*, 20(2), 2011.

[40] James W. Cooley and John W. Tukey. An algorithm for the machine calculation of complex Fourier series. *Mathematics of Computation*, 19(90):297–301, 1965.

[41] P Craven and G. Wahba. Smoothing noisy data with spline functions: estimating the correct degree of smoothing by the method of generalized cross-validation. *Numer. Math.*, 31:377–403, 1979.

[42] JP Cusumano and BW Kimble. A stochastic interrogation method for experimental measurements of global dynamics and basin evolution: Application to a two-well oscillator. *Nonlinear Dynamics*, 8(2):213–235, 1995.

[43] JP Cusumano and D-C Lin. Bifurcation and modal interaction in a simplified model of bending-torsion vibrations of the thin elastica. *Journal of vibration and acoustics*, 117(1):30–42, 1995.

[44] M. Dämmig and F. Mitschke. Estimation of Lyapunov exponents from time series: the stochastic case. *Physics Letters A*, 178(5–6):385–394, 1993.

[45] A. G. Darbyshire and D. S. Broomhead. Robust estimation of tangent maps and Liapunov spectra. *Physica D: Nonlinear Phenomena*, 89(3–4):287–305, 1996.

[46] Mingzhou Ding, Celso Grebogi, Edward Ott, Tim Sauer, and J. A. Yorke. Plateau onset for correlation dimension: When does it occur? *Physical Review Letters*, 70(25):3872–3875, 1993.

[47] Jonathan B Dingwell. Lyapunov exponents. *Wiley Encyclopedia of Biomedical Engineering*, 2006.

[48] Reik V. Donner and Susana M. Barbosa, editors. *Nonlinear Time Series Analysis in the Geosciences - Applications in Climatology, Geodynamics and Solar-Terrestrial Physics*. Berlin: Springer, 2008.

[49] Georg. Duffing. *Erzwungene schwingungen bei veränderlicher eigenfrequenz und ihre technische bedeutung*. F. Vieweg & Sohn, Braunschweig, 1918.

[50] J.-P. Eckmann, S. Oliffson Kamphorst, D. Ruelle, and S. Ciliberto. Liapunov exponents from time series. *Physical Review A*, 34(6):4971–4979, 1986.

[51] J.-P. Eckmann and D. Ruelle. Addendum: Ergodic theory of chaos and strange attractors. *Reviews of Modern Physics*, 57(4):1115, 1985.

[52] J.-P. Eckmann and D. Ruelle. Ergodic theory of chaos and strange attractors. *Reviews of Modern Physics*, 57(3):617–656, 1985.

[53] J.-P. Eckmann and D. Ruelle. Fundamental limitations for estimating dimensions and Lyapunov exponents in dynamical systems. *Physica D: Nonlinear Phenomena*, 56(2–3):185–187, 1992.

[54] Jean-Pierre Eckmann, S. Oliffson Kamphorst, and David Ruelle. Recurrence plots of dynamical systems. *Europhysics Letters*, 4(9):973–977, 1987.

[55] James Eells, Hassler Whitney, and Domingo Toledo. *Collected papers of Hassler Whitney*. Nelson Thornes, 1992.

[56] M. EtehadTavakol, E Y K Ng, C. Lucas, S. Sadri, and M. Ataei. Nonlinear analysis using Lyapunov exponents in breast thermograms to identify abnormal lesions. *Infrared Physics & Technology*, 55(4):345 – 352, 2012.

[57] Geir Evensen. Sequential data assimilation with a nonlinear quasi-geostrophic model using Monte Carlo methods to forecast error statistics. *Journal of Geophysical Research: Oceans*, 99(C5):10143–10162, 1994.

[58] D. J. Ewins. *Modal Testing: Theory, Practice and Application*. Research Studies Press, Baldock, Hertfordshire, 2nd edition, 2000.

[59] J Doyne Farmer, Edward Ott, and James A Yorke. The dimension of chaotic attractors. *Physica D: Nonlinear Phenomena*, 7(1-3):153–180, 1983.

[60] J. Doyne Farmer and John J. Sidorowich. Predicting chaotic time series. *Physical Review Letters*, 59(8):845–848, 1987.

[61] U. Farooq and BF Feeny. Smooth orthogonal decomposition for modal analysis of randomly excited systems. *Journal of Sound and Vibration*, 316(1):137–146, 2008.

[62] Philippe Faure and Henri Korn. A new method to estimate the Kolmogorov entropy from recurrence plots: its application to neuronal signals. *Physica D: Nonlinear Phenomena*, 122(1–4):265–279, 1998.

[63] Brian F. Feeny and Ravindra Kappagantu. On the physical interpretation of proper orthogonal modes in vibrations. *Journal of Sound and Vibration*, 211(4):607–616, 1998.

[64] Mitchell J. Feigenbaum. Quantitative universality for a class of nonlinear transformations. *Journal of Statistical Physics*, 19(1):25–52, 1978.

[65] Andrew M Fraser. Reconstructing attractors from scalar time series: a comparison of singular system and redundancy criteria. *Physica D: Nonlinear Phenomena*, 34(3):391–404, 1989.

[66] Andrew M Fraser and Harry L Swinney. Independent coordinates for strange attractors from mutual information. *Physical review A*, 33(2):1134, 1986.

[67] Paul Frederickson, James L. Kaplan, Ellen D. Yorke, and James A. Yorke. The Liapunov dimension of strange attractors. *Journal of Differential Equations*, 49(2):185–207, 1983.

[68] J. Gao, H. Sultan, J. Hu, and W.W. Tung. Denoising nonlinear time series by adaptive filtering and wavelet shrinkage: a comparison. *Signal Processing Letters, IEEE*, 17(3):237–240, 2010.

[69] J. B. Gao, J. Hu, W. W. Tung, and Y. H. Cao. Distinguishing chaos from noise by scale-dependent Lyapunov exponent. *Physical Review E*, 74(6):066204, 2006.

[70] Ramazan Gençay and W. Davis Dechert. The identification of spurious Lyapunov exponents in Jacobian algorithms. *Studies in Nonlinear Dynamics & Econometrics*, 1(3), 1996.

[71] S. Ghafari, F. Golnaraghi, and F. Ismail. Effect of localized faults on chaotic vibration of rolling element bearings. *Nonlinear Dynamics*, 53:287–301, 2008.

[72] John F Gibson, J Doyne Farmer, Martin Casdagli, and Stephen Eubank. An analytic approach to practical state space reconstruction. *Physica D: Nonlinear Phenomena*, 57(1):1–30, 1992.

[73] F. Ginelli, P. Poggi, A. Turchi, H. Chaté, R. Livi, and A. Politi. Characterizing dynamics with covariant Lyapunov vectors. *Physical Review Letters*, 99(13):130601, 2007.

[74] Gene H Golub and Charles F Van Loan. *Matrix computations*. Johns Hopkins University Press, Baltimore, MD, USA, 1996.

[75] P. Grassberger, R. Hegger, H. Kantz, C. Schaffrath, and T. Schreiber. On noise reduction methods for chaotic data. *Chaos: An Interdisciplinary Journal of Nonlinear Science*, 3(2):127–141, 1993.

[76] P. Grassberger and I. Procaccia. Measuring the strangeness of strange attractors. *Physica D: Nonlinear Phenomena*, 9(1):189–208, 1983.

[77] Peter Grassberger. Generalized dimensions of strange attractors. *Physics Letters A*, 97(6):227–230, 1983.

[78] Peter Grassberger. Generalizations of the Hausdorff dimension of fractal measures. *Physics Letters A*, 107(3):101–105, 1985.

[79] Peter Grassberger and Itamar Procaccia. Characterization of strange attractors. *Physical Review Letters*, 50(5):346–349, 1983.

[80] Peter Grassberger and Itamar Procaccia. Estimation of the Kolmogorov entropy from a chaotic signal. *Physical Review A*, 28(4):2591–2593, 1983.

[81] Peter Grassberger, Thomas Schreiber, and Carsten Schaffrath. Nonlinear time sequence analysis. *International Journal of Bifurcation and Chaos*, 1(03):521–547, 1991.

[82] John Guckenheimer and Philip Holmes. *Nonlinear Oscillations, Dynamical Systems, and Bifurcations of Vector Fields*. Number 42 in Applied Mathematical Sciences. Springer, New York, 1983.

[83] Allan Gut. *An intermediate course in probability*. Springer Texts in Statistics. Springer, New York, NY, 2009.

[84] George Haller and Sten Ponsioen. Nonlinear normal modes and spectral submanifolds: existence, uniqueness and use in model reduction. *Nonlinear Dynamics*, 86(3):1493–1534, 2016.

[85] Thomas C. Halsey, Mogens H. Jensen, Leo P. Kadanoff, Itamar Procaccia, and Boris I. Shraiman. Erratum: Fractal measures and their singularities: The characterization of strange sets. *Physical Review A*, 34(2):1601, 1986.

[86] Thomas C. Halsey, Mogens H. Jensen, Leo P. Kadanoff, Itamar Procaccia, and Boris I. Shraiman. Fractal measures and their singularities: The characterization of strange sets. *Physical Review A*, 33(2):1141–1151, 1986.

[87] James Douglas Hamilton. *Time series analysis*, volume 2. Princeton university press Princeton, 1994.

[88] F. J. Harris. On the use of windows for harmonic analysis with the discrete Fourier transform. *Proceedings of the IEEE*, 66(1):51–83, 1978.

[89] Joseph D. Hart. Attractor reconstruction with reservoir computers: The effect of the reservoir's conditional Lyapunov exponents on faithful attractor reconstruction. *Chaos: An Interdisciplinary Journal of Nonlinear Science*, 34(4):043123, 2024.

[90] Rainer Hegger and Holger Kantz. Improved false nearest neighbor method to detect determinism in time series data. *Physical Review E*, 60(4):4970, 1999.

[91] Rainer Hegger, Holger Kantz, and Thomas Schreiber. Practical implementation of nonlinear time series methods: The tisean package. *Chaos: An Interdisciplinary Journal of Nonlinear Science*, 9(2):413–435, 1999.

[92] M. Hénon. A two-dimensional mapping with a strange attractor. *Communications in Mathematical Physics*, 50(1):69–77, 1976.

[93] Michel Hénon. A two-dimensional mapping with a strange attractor. In *The Theory of Chaotic Attractors*, pages 94–102. Springer, 1976.

[94] H. G. E. Hentschel and Itamar Procaccia. The infinite number of generalized dimensions of fractals and strange attractors. *Physica D: Nonlinear Phenomena*, 8(3):435–444, 1983.

[95] P. J. Holmes and F. C. Moon. Addendum: A magnetoelastic strange attractor. *Journal of Sound and Vibration*, 69(2):339, 1980.

[96] Philip Holmes. A nonlinear oscillator with a strange attractor. *Philosophical Transactions of the Royal Society of London. Series A*, 292(1394):419–448, 1979.

[97] Eberhard Hopf. Abzweigung einer periodischen lösung von einer stationären lösung eines differentialsystems. *Berichte der Mathematisch-Physischen Klasse der Sächsischen Akademie der Wissenschaften zu Leipzig*, 94, 1942.

[98] Behshad Hosseinifard, Mohammad Hassan Moradi, and Reza Rostami. Classifying depression patients and normal subjects using machine learning techniques and nonlinear features from {EEG} signal. *Computer Methods and Programs in Biomedicine*, 109(3):339 – 345, 2013.

[99] Kensuke Ikeda. Multiple-valued stationary state and its instability of the transmitted light by a ring cavity system. *Optics Communications*, 30(2):257–261, 1979.

[100] Alan Julian Izenman. *Modern Multivariate Statistical Techniques: Regression, Classification, and Manifold Learning*. Springer Texts in Statistics. Springer, New York, 2008.

[101] Gwilym M. Jenkins and Donald G. Watts. *Spectral Analysis and Its Applications*. Holden-Day, San Francisco, 1968.

[102] Kevin Judd. An improved estimator of dimension and some comments on providing confidence intervals. *Physica D: Nonlinear Phenomena*, 56(2–3):216–228, 1992.

[103] Kevin Judd and Alistair Mees. Embedding as a modeling problem. *Physica D: Nonlinear Phenomena*, 120(3):273–286, 1998.

[104] S Kacimi and S Laurens. The correlation dimension: A robust chaotic feature for classifying acoustic emission signals generated in construction materials. *Journal of Applied Physics*, 106(2), 2009.

[105] H. Kantz and T. Schreiber. *Nonlinear time series analysis*, volume 7. Cambridge University Press, 2003.

[106] H. Kantz, T. Schreiber, I. Hoffmann, T. Buzug, G. Pfister, L.G. Flepp, J. Simonet, R. Badii, and E. Brun. Nonlinear noise reduction: A case study on experimental data. *Physical Review E*, 48(2):1529, 1993.

[107] Holger Kantz. A robust method to estimate the maximal Lyapunov exponent of a time series. *Physics Letters A*, 185(1):77–87, 1994.

[108] James L. Kaplan and James A. Yorke. Chaotic behavior of multidimensional difference equations. In Heinz-Otto Peitgen and Hans-Otto Walther, editors, *Functional Differential Equations and Approximation of Fixed Points*, number 730 in Lecture Notes in Mathematics, pages 204–227. Springer, Berlin, 1979.

[109] George Em Karniadakis, Ioannis G. Kevrekidis, Lu Lu, Paris Perdikaris, Sifan Wang, and Liu Yang. Physics-informed machine learning. *Nature Reviews Physics*, 3(6):422–440, 2021.

[110] G Kember and AC Fowler. A correlation function for choosing time delays in phase portrait reconstructions. *Physics Letters A*, 179(2):72–80, 1993.

[111] Matthew B Kennel and Henry DI Abarbanel. False neighbors and false strands: A reliable minimum embedding dimension algorithm. *Physical review E*, 66(2):026209, 2002.

[112] Matthew B Kennel, Reggie Brown, and Henry DI Abarbanel. Determining embedding dimension for phase-space reconstruction using a geometrical construction. *Physical review A*, 45(6):3403, 1992.

[113] Gaetan Kerschen, Jean-Claude Golinval, Alexander F. Vakakis, and Lawrence A. Bergman. The method of proper orthogonal decomposition for dynamical characterization and order reduction of mechanical systems: An overview. *Nonlinear Dynamics*, 41:147–169, 2005.

[114] Gaetan Kerschen, Keith Worden, Alexander F. Vakakis, and Jean-Claude Golinval. Past, present and future of nonlinear system identification in structural dynamics. *Mechanical Systems and Signal Processing*, 20(3):505–592, 2006.

[115] A. Khintchine. Korrelationstheorie der stationären stochastischen prozesse. *Mathematische Annalen*, 109(1):604–615, 1934.

[116] H_S Kim, R Eykholt, and JD Salas. Nonlinear dynamics, delay times, and embedding windows. *Physica D: Nonlinear Phenomena*, 127(1):48–60, 1999.

[117] E.J. Kostelich and T. Schreiber. Noise reduction in chaotic time-series data: a survey of common methods. *Physical Review E*, 48(3):1752, 1993.

[118] Ivana Kovacic and Michael J Brennan. *The duffing equation: Nonlinear oscillators and their behaviour*. Wiley, 2011.

[119] Alexander Kraskov, Harald Stögbauer, and Peter Grassberger. Estimating mutual information. *Physical Review E*, 69(6):066138, 2004.

[120] D. Kugiumtzis. Test your surrogate data before you test for nonlinearity. *Physical Review E*, 60(3):2808–2816, 1999.

[121] D. Kugiumtzis. Surrogate data test for nonlinearity including nonmonotonic transforms. *Physical Review E*, 62(1):R25–R28, 2000.

[122] D. Kugiumtzis. Statically transformed autoregressive process and surrogate data test for nonlinearity. *Physical Review E*, 66(2):025201, 2002.

[123] Dimitris Kugiumtzis. State space reconstruction parameters in the analysis of chaotic time series—the role of the time window length. *Physica D: Nonlinear Phenomena*, 95(1):13–28, 1996.

[124] Pavel V. Kuptsov and Ulrich Parlitz. Theory and computation of covariant Lyapunov vectors. *Journal of Nonlinear Science*, 22(5):727–762, 2012.

[125] Kody Law, Andrew Stuart, and Konstantinos Zygalakis. *Data Assimilation: A Mathematical Introduction*. Texts in Applied Mathematics. Springer, Cham, 2015.

[126] F. Ledrappier. Some relations between dimension and Lyapounov exponents. *Communications in Mathematical Physics*, 81(2):229–238, 1981.

[127] Edward N Lorenz. Deterministic nonperiodic flow. *Journal of the atmospheric sciences*, 20(2):130–141, 1963.

[128] Edward N. Lorenz. Atmospheric predictability as revealed by naturally occurring analogues. *Journal of the Atmospheric Sciences*, 26(4):636–646, 1969. DOI 10.1175/1520-0469(1969)26<636:APARBN>2.0.CO;2.

[129] Y. Lv, H. Liu, R. Yuan, X. Dong, Y. Wang, H. Wu, and David Chelidze. Advances and prospects of phase space reconstruction-based high-dimensional signal processing for fault diagnosis: A systematic review. *Mechanical Systems and Signal Processing*, 246:113919, 2026.

[130] Henrik Madsen. *Time series analysis*. CRC Press, 2007.

[131] Benoit B. Mandelbrot. *The Fractal Geometry of Nature*. W. H. Freeman, San Francisco, 1982.

[132] J. E. Marsden and M. McCracken. *The Hopf Bifurcation and Its Applications*. Number 19 in Applied Mathematical Sciences. Springer, New York, 1976.

[133] Christoph Martin, Nahal Sharafi, and Sarah Hallerberg. Estimating covariant Lyapunov vectors from data. *Chaos: An Interdisciplinary Journal of Nonlinear Science*, 32(3):033105, 2022.

[134] Norbert Marwan, M. Carmen Romano, Marco Thiel, and Jürgen Kurths. Recurrence plots for the analysis of complex systems. *Physics Reports*, 438(5-6):237–329, 2007.

[135] T Matsumoto, Leon O Chua, and S Tanaka. Simplest chaotic nonautonomous circuit. *Physical Review A*, 30(2):1155, 1984.

[136] Takashi Matsumoto. A chaotic attractor from chua's circuit. *IEEE Transactions on Circuits and Systems*, 31(12):1055–1058, 1984.

[137] Takashi Matsumoto, Leon Chua, and Motomasa Komuro. The double scroll. *IEEE Transactions on Circuits and Systems*, 32(8):797–818, 1985.

[138] A Maus and JC Sprott. Neural network method for determining embedding dimension of a time series. *Communications in Nonlinear Science and Numerical Simulation*, 16(8):3294–3302, 2011.

[139] Robert M. May. Simple mathematical models with very complicated dynamics. *Nature*, 261(5560):459–467, 1976.

[140] Sina Mehdizadeh and Mohammad Ali Sanjari. Effect of noise and filtering on largest Lyapunov exponent of time series associated with human walking. *Journal of Biomechanics*, 64:236–239, 2017.

[141] Igor Mezić. Spectral properties of dynamical systems, model reduction and decompositions. *Nonlinear Dynamics*, 41(1):309–325, 2005.

[142] FC Moon and Philip J Holmes. A magnetoelastic strange attractor. *Journal of Sound and Vibration*, 65(2):275–296, 1979.

[143] Francis C Moon and FC Moon. *Chaotic vibrations: an introduction for applied scientists and engineers*. Wiley Online Library, 1987.

[144] National Instruments. Understanding FFTs and windowing. Application note, https://www.ni.com/en/shop/data-acquisition/measurement-fundamentals/analog-fundamentals/understanding-ffts-and-windowing.html. Accessed 2026-09-05.

[145] Ju. I. Neimark. On some cases of periodic motions depending on parameters. *Doklady Akademii Nauk SSSR*, 129:736–739, 1959. In Russian; details from a secondary source and not independently confirmed.

[146] Jonathan Nichols and Michael Todd. *Nonlinear Features for SHM Applications*. John Wiley & Sons, Ltd, 2009.

[147] Jean-Philippe Noël and Gaetan Kerschen. Nonlinear system identification in structural dynamics: 10 more years of progress. *Mechanical Systems and Signal Processing*, 83:2–35, 2017.

[148] H. Nyquist. Certain topics in telegraph transmission theory. *Transactions of the American Institute of Electrical Engineers*, 47(2):617–644, 1928.

[149] A. R. Osborne and A. Provenzale. Finite correlation dimension for stochastic systems with power-law spectra. *Physica D: Nonlinear Phenomena*, 35(3):357–381, 1989.

[150] V. I. Oseledec. A multiplicative ergodic theorem. Lyapunov characteristic numbers for dynamical systems. *Transactions of the Moscow Mathematical Society*, 19:197–231, 1968.

[151] N. H. Packard, J. P. Crutchfield, J. D. Farmer, and R. S. Shaw. Geometry from a time series. *Physical Review Letters*, 45(9):712–716, 1980.

[152] Athanasios Papoulis and S. Unnikrishna Pillai. *Probability, Random Variables and Stochastic Processes*. McGraw-Hill, New York, 4 edition, 2002.

[153] U. Parlitz. Identification of true and spurious Lyapunov exponents from time series. *International Journal of Bifurcation and Chaos*, 2(1):155–165, 1992.

[154] Ulrich Parlitz. Estimating Lyapunov exponents from time series. In *Chaos Detection and Predictability*, number 915 in Lecture Notes in Physics, pages 1–34. Springer, Berlin, 2016.

[155] Jaideep Pathak, Zhixin Lu, Brian R. Hunt, Michelle Girvan, and Edward Ott. Using machine learning to replicate chaotic attractors and calculate Lyapunov exponents from data. *Chaos: An Interdisciplinary Journal of Nonlinear Science*, 27(12):121102, 2017.

[156] Donald B. Percival and Andrew T. Walden. *Spectral Analysis for Physical Applications: Multitaper and Conventional Univariate Techniques*. Cambridge University Press, Cambridge, 1993.

[157] Steven M. Pincus. Approximate entropy as a measure of system complexity. *Proceedings of the National Academy of Sciences*, 88(6):2297–2301, 1991.

[158] William H. Press, Saul A. Teukolsky, William T. Vetterling, and Brian P. Flannery. *Numerical Recipes: The Art of Scientific Computing*. Cambridge University Press, Cambridge, 3rd edition, 2007.

[159] Dean Prichard and James Theiler. Generating surrogate data for time series with several simultaneously measured variables. *Physical Review Letters*, 73(7):951–954, 1994.

[160] A. Provenzale, L. A. Smith, R. Vio, and G. Murante. Distinguishing between low-dimensional dynamics and randomness in measured time series. *Physica D: Nonlinear Phenomena*, 58(1–4):31–49, 1992.

[161] M. Raissi, P. Perdikaris, and G. E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019.

[162] Muruhan Rathinam and Linda R. Petzold. A new look at proper orthogonal decomposition. *SIAM J. Numer. Anal.*, 41(5):1893–1925, 2003.

[163] C.H. Reinsch. Smoothing by spline fiunctions. *Numer. Math.*, 10:177–183, 1967.

[164] C.H. Reinsch. Smoothing by spline fiunctions ii. *Numer. Math.*, 16:451–454, 1971.

[165] Alfréd Rényi. On measures of entropy and information. In *Proceedings of the Fourth Berkeley Symposium on Mathematical Statistics and Probability, Volume 1*, pages 547–561. University of California Press, Berkeley, 1961.

[166] Joshua S. Richman and J. Randall Moorman. Physiological time-series analysis using approximate entropy and sample entropy. *American Journal of Physiology-Heart and Circulatory Physiology*, 278(6):H2039–H2049, 2000.

[167] Michael T. Rosenstein, James J. Collins, and Carlo J. De Luca. A practical method for calculating largest Lyapunov exponents from small data sets. *Physica D: Nonlinear Phenomena*, 65(1–2):117–134, 1993.

[168] Michael T Rosenstein, James J Collins, and Carlo J De Luca. Reconstruction expansion as a geometry-based framework for choosing proper delay times. *Physica D: Nonlinear Phenomena*, 73(1):82–98, 1994.

[169] Clarence W. Rowley, Igor Mezić, Shervin Bagheri, Philipp Schlatter, and Dan S. Henningson. Spectral analysis of nonlinear flows. *Journal of Fluid Mechanics*, 641:115–127, 2009.

[170] David Ruelle. An inequality for the entropy of differentiable maps. *Boletim da Sociedade Brasileira de Matemática*, 9(1):83–87, 1978.

[171] Robert J. Sacker. On invariant surfaces and bifurcation of periodic solutions of ordinary differential equations. *Journal of Difference Equations and Applications*, 15(8–9):759–774, 2009. Republication of Report IMM-NYU 333, New York University, 1964.

[172] M. Sano and Y. Sawada. Measurement of the Lyapunov spectrum from a chaotic time series. *Physical Review Letters*, 55(10):1082–1085, 1985.

[173] Tim Sauer, James A Yorke, and Martin Casdagli. Embedology. *Journal of statistical Physics*, 65(3-4):579–616, 1991.

[174] Timothy D. Sauer, Joshua A. Tempkin, and James A. Yorke. Spurious Lyapunov exponents in attractor reconstruction. *Physical Review Letters*, 81(20):4341–4344, 1998.

[175] Jeffrey M Schiffman, David Chelidze, Albert Adams, David B Segala, and Leif Hasselquist. Nonlinear analysis of gait kinematics to track changes in oxygen consumption in prolonged load carriage walking: A pilot study. *Journal of biomechanics*, 42(13):2196–2199, 2009.

[176] Peter J. Schmid. Dynamic mode decomposition of numerical and experimental data. *Journal of Fluid Mechanics*, 656:5–28, 2010.

[177] I.J. Schoenberg. Contributions to the problem of approximation of equidistant data by analytic functions. *Quart. Appl. Math.*, 4:45–99 and 112–141, 1946.

[178] T. Schreiber. Extremely simple nonlinear noise-reduction method. *Physical Review E*, 47:2401–2404, 1993.

[179] Thomas Schreiber. Measuring information transfer. *Physical Review Letters*, 85(2):461–464, 2000.

[180] Thomas Schreiber and Andreas Schmitz. Improved surrogate data for nonlinearity tests. *Physical Review Letters*, 77(4):635, 1996.

[181] Thomas Schreiber and Andreas Schmitz. Surrogate time series. *Physica D: Nonlinear Phenomena*, 142(3–4):346–382, 2000.

[182] Thomas Schreiber and Andreas Schmitz. Constrained randomization of time series for nonlinearity tests. In *Nonlinear Dynamics and Statistics*, pages 219–232. Birkhäuser, Boston, 2001.

[183] Arthur Schuster. On the investigation of hidden periodicities with application to a supposed 26 day period of meteorological phenomena. *Terrestrial Magnetism*, 3(1):13–41, 1898.

[184] Claude Elwood Shannon. Communication in the presence of noise. *Proceedings of the IRE*, 37(1):10–21, 1949.

[185] Ippei Shimada and Tomomasa Nagashima. A numerical approach to ergodic problem of dissipative dynamical systems. *Progress of Theoretical Physics*, 61(6):1605–1616, 1979.

[186] D. Slepian. Prolate spheroidal wave functions, Fourier analysis, and uncertainty—V: The discrete case. *Bell System Technical Journal*, 57(5):1371–1430, 1978.

[187] Michael Small and C. K. Tse. Applying the method of surrogate data to cyclic time series. *Physica D: Nonlinear Phenomena*, 164(3–4):187–201, 2002.

[188] Michael Small and C. K. Tse. Detecting determinism in time series: the method of surrogate data. *IEEE Transactions on Circuits and Systems I: Fundamental Theory and Applications*, 50(5):663–672, 2003.

[189] Michael Small and Chi K Tse. Optimal embedding parameters: a modelling paradigm. *Physica D: Nonlinear Phenomena*, 194(3):283–296, 2004.

[190] Michael Small, Dejin Yu, and Robert G. Harrison. Surrogate test for pseudoperiodic time series data. *Physical Review Letters*, 87(18):188101, 2001.

[191] Leonard A. Smith. Intrinsic limits on dimension calculations. *Physics Letters A*, 133(6):283–288, 1988.

[192] I. S. Sokolnikoff and R. M. Redheffer. *Mathematics of Physics and Modern Engineering*. McGraw-Hill, New York, 2 edition, 1966.

[193] Miao Song, David B Segala, Jonathan B Dingwell, and David Chelidze. Slow-time changes in human emg muscle fatigue states are fully represented in movement kinematics. *Journal of biomechanical engineering*, 131(2):021004, 2009.

[194] R. Stoop and J. Parisi. Calculation of Lyapunov exponents avoiding spurious elements. *Physica D: Nonlinear Phenomena*, 50(1):89–94, 1991.

[195] Steven H. Strogatz. *Nonlinear Dynamics and Chaos: With Applications to Physics, Biology, Chemistry, and Engineering*. Chapman and Hall/CRC, Boca Raton, FL, 3 edition, 2024.

[196] George Sugihara and Robert M. May. Nonlinear forecasting as a way of distinguishing chaos from measurement error in time series. *Nature*, 344(6268):734–741, 1990.

[197] Floris Takens. Detecting strange attractors in turbulence. In D. A. Rand and L. S. Young, editors, *Dynamical systems and turbulence, Warwick 1980*, pages 366–381. Springer-Verlag, Berlin, 1981.

[198] Floris Takens. On the numerical determination of the dimension of an attractor. In *Dynamical Systems and Bifurcations*, volume 1125 of *Lecture Notes in Mathematics*, pages 99–106. Springer, Berlin, 1985.

[199] Kazumasa A. Takeuchi, Hong-Liu Yang, Francesco Ginelli, Günter Radons, and Hugues Chaté. Hyperbolic decoupling of tangent space and effective dimension of dissipative systems. *Physical Review E*, 84(4):046214, 2011.

[200] Jee Ian Tam and Philip Holmes. Revisiting a magneto-elastic strange attractor. *Journal of Sound and Vibration*, 333(6):1767–1780, 2014.

[201] Joshua A. Tempkin and James A. Yorke. Spurious Lyapunov exponents computed from data. *SIAM Journal on Applied Dynamical Systems*, 6(2):457–474, 2007.

[202] James Theiler. Spurious dimension from correlation algorithms applied to limited time-series data. *Physical Review A*, 34(3):2427–2432, 1986.

[203] James Theiler. Efficient algorithm for estimating the correlation dimension from a set of discrete points. *Physical Review A*, 36(9):4456–4462, 1987.

[204] James Theiler. Lacunarity in a best estimator of fractal dimension. *Physics Letters A*, 133(4–5):195–200, 1988.

[205] James Theiler. Estimating fractal dimension. *Journal of the Optical Society of America A*, 7(6):1055–1073, 1990.

[206] James Theiler. Statistical precision of dimension estimators. *Physical Review A*, 41(6):3038, 1990.

[207] James Theiler. Some comments on the correlation dimension of $1/f^{\alpha}$ noise. *Physics Letters A*, 155(8):480–493, 1991.

[208] James Theiler, Stephen Eubank, André Longtin, Bryan Galdrikian, and J. Doyne Farmer. Testing for nonlinearity in time series: the method of surrogate data. *Physica D: Nonlinear Phenomena*, 58(1–4):77–94, 1992.

[209] Marco Thiel, M. Carmen Romano, Jürgen Kurths, Riccardo Meucci, Enrico Allaria, and F. Tito Arecchi. Influence of observational noise on the recurrence quantification analysis. *Physica D: Nonlinear Phenomena*, 171(3):138–152, 2002.

[210] Marco Thiel, M. Carmen Romano, Jürgen Kurths, Martin Rolfs, and Reinhold Kliegl. Twin surrogates to test for complex synchronisation. *Europhysics Letters (EPL)*, 75(4):535–541, 2006.

[211] Marco Thiel, M. Carmen Romano, Peter L. Read, and Jürgen Kurths. Estimation of dynamical invariants without embedding by recurrence plots. *Chaos: An Interdisciplinary Journal of Nonlinear Science*, 14(2):234–243, 2004.

[212] D. J. Thomson. Spectrum estimation and harmonic analysis. *Proceedings of the IEEE*, 70(9):1055–1096, 1982.

[213] PE Tikkanen. Nonlinear wavelet and wavelet packet denoising of electrocardiogram signal. *Biological cybernetics*, 80(4):259–267, 1999.

[214] Jens Timmer. Power of surrogate data testing with respect to nonstationarity. *Physical Review E*, 58(4):5153–5156, 1998.

[215] M D Todd, J M Nichols, L M Pecora, and L N Virgin. Vibration-based damage assessment utilizing state space geometry changes: local attractor variance ratio. *Smart Materials and Structures*, 10(5):1000–1008, 2001.

[216] S. M. Ulam and John von Neumann. On combination of stochastic and deterministic processes. preliminary report. *Bulletin of the American Mathematical Society*, 53:1120, 1947. Abstract 403; printed within the report of the 1947 New Haven summer meeting, pp. 1100–1133.

[217] Roger I Vargas, John D Stark, and Toshiyuki Nishida. Population dynamics, habitat preference, and seasonal distribution patterns of oriental fruit fly and melon fly (diptera: Tephritidae) in an agricultural area. *Environmental Entomology*, 19(6):1820–1828, 1990.

[218] P. R. Vlachas, J. Pathak, B. R. Hunt, T. P. Sapsis, M. Girvan, E. Ott, and P. Koumoutsakos. Backpropagation algorithms and reservoir computing in recurrent neural networks for the forecasting of complex spatiotemporal dynamics. *Neural Networks*, 126:191–217, 2020.

[219] W. J. Wang, Z. T. Wu, and J. Chen. Fault identification in rotating machinery using the correlation dimension and bispectra. *Nonlinear Dynamics*, 25(4):383–393, 2001.

[220] W.J. Wang, J. Chen, X.K. Wu, and Z.T. Wu. The application of some non-linear methods in rotating machinery fault diagnosis. *Mechanical Systems and Signal Processing*, 15(4):697 – 705, 2001.

[221] P. Welch. The use of fast Fourier transform for the estimation of power spectra: A method based on time averaging over short, modified periodograms. *IEEE Transactions on Audio and Electroacoustics*, 15(2):70–73, 1967.

[222] Norbert Wiener. Generalized harmonic analysis. *Acta Mathematica*, 55:117–258, 1930.

[223] Matthew O. Williams, Ioannis G. Kevrekidis, and Clarence W. Rowley. A data-driven approximation of the Koopman operator: extending dynamic mode decomposition. *Journal of Nonlinear Science*, 25(6):1307–1346, 2015.

[224] Laurenz Wiskott and Terrence J. Sejnowski. Slow feature analysis: unsupervised learning of invariances. *Neural Computation*, 14(4):715–770, 2002.

[225] A. Wolf, J.B. Swift, H.L. Swinney, and J.A. Vastano. Determining lyapunov exponents from a time series. *Physica D: Nonlinear Phenomena*, 16(3):285–317, 1985.

[226] Alan Wolf. *Quantifying chaos with Lyapuonov exponents*, pages 273–290. Princeton University Press, 1986.

[227] K. Worden and G. R. Tomlinson. *Nonlinearity in Structural Dynamics: Detection, Identification and Modelling*. Institute of Physics Publishing, Bristol and Philadelphia, 2001.

[228] Hong-Liu Yang, Kazumasa A. Takeuchi, Francesco Ginelli, Hugues Chaté, and Günter Radons. Hyperbolicity and the effective dimension of spatially extended dissipative systems. *Physical Review Letters*, 102(7):074102, 2009.

[229] Lai-Sang Young. Dimension, entropy and Lyapunov exponents. *Ergodic Theory and Dynamical Systems*, 2(1):109–124, 1982.

[230] M. Pollicott Yuri. *Dynamical Systems and Ergodic Theory*. Cambridge University Press, 1998.

[231] Shuang Zhou and Xingyuan Wang. Identifying the linear region based on machine learning to calculate the largest Lyapunov exponent from chaotic time series. *Chaos: An Interdisciplinary Journal of Nonlinear Science*, 28(12):123118, 2018.

[232] Wenliang Zhou and David Chelidze. Generalized eigenvalue decomposition in time domain modal parameter identification. *Journal of Vibration and Acoustics*, 130(1):011001, 2008.